

NeetCode 150

**Priority-Grouped · Category-Ordered · 1×1 Portrait
Landscape**

**Python Only · Priority-Grouped ·
Difficulty-First · 2×1 Landscape**

**150 questions · 9 rounds · Inline Quick Review
· Chapter 2 Summary**

**High Easy → High Med → High Hard → Mid Easy
→ Mid Med → Mid Hard → Low Easy → Low Med
→ Low Hard**

Contents

★ = LeetCode Premium (Premium 98 list)

Round 1 | High | E Easy (13) ↗

Arrays & Hashing (3) ↗

☐ ● #1 Two Sum ↗

☐ ● #217 Contains Duplicate ↗

☐ ● #242 Valid Anagram ↗

Two Pointers (1) ↗

☐ ● #125 Valid Palindrome ↗

Sliding Window (1) ↗

☐ ● #121 Best Time to Buy and Sell Stock ↗

Binary Search (1) ↗

☐ ● #704 Binary Search ↗

Intervals (1) ↗

☐ ● ★ #252 Meeting Rooms ↗

Trees (6) ↗

☐ ● #100 Same Tree ↗

☐ ● #104 Maximum Depth of Binary Tree ↗

- ☐ ● #110 Balanced Binary Tree ↗
- ☐ ● #226 Invert Binary Tree ↗
- ☐ ● #543 Diameter of Binary Tree ↗
- ☐ ● #572 Subtree of Another Tree ↗

Round 2 | High | M Medium (43) ↗

Arrays & Hashing (6) ↗

- ☐ ● #36 Valid Sudoku ↗
- ☐ ● #49 Group Anagrams ↗
- ☐ ● #128 Longest Consecutive Sequence ↗
- ☐ ● #238 Product of Array Except Self ↗
- ☐ ● ★ #271 Encode and Decode Strings ↗
- ☐ ● #347 Top K Frequent Elements ↗

Two Pointers (3) ↗

- ☐ ● #11 Container With Most Water ↗
- ☐ ● #15 3Sum ↗
- ☐ ● #167 Two Sum II – Input Array Is Sorted ↗

Sliding Window (3) ↗

☐ ● #3 Longest Substring Without Repeating Characters ↗

☐ ● #424 Longest Repeating Character Replacement ↗

☐ ● #567 Permutation in String ↗

Binary Search (5) ↗

☐ ● #33 Search in Rotated Sorted Array ↗

☐ ● #74 Search a 2D Matrix ↗

☐ ● #153 Find Minimum in Rotated Sorted Array ↗

☐ ● #875 Koko Eating Bananas ↗

☐ ● #981 Time Based Key-Value Store ↗

Intervals (4) ↗

☐ ● #56 Merge Intervals ↗

☐ ● #57 Insert Interval ↗

☐ ● ★ #253 Meeting Rooms II ↗

☐ ● #435 Non-overlapping Intervals ↗

Trees (7) ↗

☐ ● #98 Validate Binary Search Tree ↗

- ☐ ● #102 Binary Tree Level Order Traversal ↗
- ☐ ● #105 Construct Binary Tree from Preorder and Inorder Traversal ↗
- ☐ ● #199 Binary Tree Right Side View ↗
- ☐ ● #230 Kth Smallest Element in a BST ↗
- ☐ ● #235 Lowest Common Ancestor of a Binary Search Tree ↗
- ☐ ● #1448 Count Good Nodes in Binary Tree ↗
- Graphs (12)** ↗
- ☐ ● #130 Surrounded Regions ↗
- ☐ ● #133 Clone Graph ↗
- ☐ ● #200 Number of Islands ↗
- ☐ ● #207 Course Schedule ↗
- ☐ ● #210 Course Schedule II ↗
- ☐ ● ★ #261 Graph Valid Tree ↗
- ☐ ● ★ #286 Walls and Gates ↗
- ☐ ● ★ #323 Number of Connected Components in an Undirected Graph ↗
- ☐ ● #417 Pacific Atlantic Water Flow ↗

☐ **● #684 Redundant Connection** ↗

☐ **● #695 Max Area of Island** ↗

☐ **● #994 Rotting Oranges** ↗

Advanced Graphs (3) ↗

☐ **● #743 Network Delay Time** ↗

☐ **● #787 Cheapest Flights Within K Stops** ↗

☐ **● #1584 Min Cost to Connect All Points** ↗

Round 3 | High | H Hard (11) ↗

Two Pointers (1) ↗

☐ **● #42 Trapping Rain Water** ↗

Sliding Window (2) ↗

☐ **● #76 Minimum Window Substring** ↗

☐ **● #239 Sliding Window Maximum** ↗

Binary Search (1) ↗

☐ **● #4 Median of Two Sorted Arrays** ↗

Intervals (1) ↗

☐ **● #1851 Minimum Interval to Include Each Query** ↗

Trees (2) ↗

☐ ● #124 Binary Tree Maximum Path Sum ↗

☐ ● #297 Serialize and Deserialize Binary Tree ↗

Graphs (1) ↗

☐ ● #127 Word Ladder ↗

Advanced Graphs (3) ↗

☐ ● ★ #269 Alien Dictionary ↗

☐ ● #332 Reconstruct Itinerary ↗

☐ ● #778 Swim in Rising Water ↗

Round 4 | Mid | E Easy (6) ↗

Linked List (3) ↗

☐ ● #21 Merge Two Sorted Lists ↗

☐ ● #141 Linked List Cycle ↗

☐ ● #206 Reverse Linked List ↗

Stack (1) ↗

☐ ● #20 Valid Parentheses ↗

Heap / Priority Queue (2) ↗

☐ ● #703 Kth Largest Element in a Stream ↗

☐ ● #1046 Last Stone Weight ↗

Round 5 | Mid | M Medium (33)

Linked List (6)

- ☐ ● #2 Add Two Numbers ↗
- ☐ ● #19 Remove Nth Node From End of List ↗
- ☐ ● #138 Copy List with Random Pointer ↗
- ☐ ● #143 Reorder List ↗
- ☐ ● #146 LRU Cache ↗
- ☐ ● #287 Find the Duplicate Number ↗

Stack (4)

- ☐ ● #150 Evaluate Reverse Polish Notation ↗
- ☐ ● #155 Min Stack ↗
- ☐ ● #739 Daily Temperatures ↗
- ☐ ● #853 Car Fleet ↗

Heap / Priority Queue (4)

- ☐ ● #215 Kth Largest Element in an Array ↗
- ☐ ● #355 Design Twitter ↗
- ☐ ● #621 Task Scheduler ↗
- ☐ ● #973 K Closest Points to Origin ↗

Tries (2) ↗

- ☐ ● #208 Implement Trie (Prefix Tree) ↗
- ☐ ● #211 Design Add and Search Words Data Structure ↗

Backtracking (9) ↗

- ☐ ● #17 Letter Combinations of a Phone Number ↗
- ☐ ● #22 Generate Parentheses ↗
- ☐ ● #39 Combination Sum ↗
- ☐ ● #40 Combination Sum II ↗
- ☐ ● #46 Permutations ↗
- ☐ ● #78 Subsets ↗
- ☐ ● #79 Word Search ↗
- ☐ ● #90 Subsets II ↗
- ☐ ● #131 Palindrome Partitioning ↗

Greedy (8) ↗

- ☐ ● #45 Jump Game II ↗
- ☐ ● #53 Maximum Subarray ↗
- ☐ ● #55 Jump Game ↗

- ☐ ● #134 Gas Station ↗
- ☐ ● #678 Valid Parenthesis String ↗
- ☐ ● #763 Partition Labels ↗
- ☐ ● #846 Hand of Straights ↗
- ☐ ● #1899 Merge Triplets to Form Target Triplet ↗

Round 6 | Mid | H Hard (6) ↗

Linked List (2) ↗

- ☐ ● #23 Merge k Sorted Lists ↗
- ☐ ● #25 Reverse Nodes in k-Group ↗

Stack (1) ↗

- ☐ ● #84 Largest Rectangle in Histogram ↗

Heap / Priority Queue (1) ↗

- ☐ ● #295 Find Median from Data Stream ↗

Tries (1) ↗

- ☐ ● #212 Word Search II ↗

Backtracking (1) ↗

- ☐ ● #51 N-Queens ↗

Round 7 | Low | E Easy (9)



1-D Dynamic Programming (2)

☐ ● #70 Climbing Stairs ↗

☐ ● #746 Min Cost Climbing Stairs ↗

Bit Manipulation (5)

☐ ● #136 Single Number ↗

☐ ● #190 Reverse Bits ↗

☐ ● #191 Number of 1 Bits ↗

☐ ● #268 Missing Number ↗

☐ ● #338 Counting Bits ↗

Math & Geometry (2)

☐ ● #66 Plus One ↗

☐ ● #202 Happy Number ↗

Round 8 | Low | M Medium (25)



1-D Dynamic Programming (10)

☐ ● #5 Longest Palindromic Substring ↗

☐ ● #91 Decode Ways ↗

☐ ● #139 Word Break ↗

- ☐ ● #152 Maximum Product Subarray ↗
- ☐ ● #198 House Robber ↗
- ☐ ● #213 House Robber II ↗
- ☐ ● #300 Longest Increasing Subsequence ↗
- ☐ ● #322 Coin Change ↗
- ☐ ● #416 Partition Equal Subset Sum ↗
- ☐ ● #647 Palindromic Substrings ↗

2-D Dynamic Programming (7) ↗

- ☐ ● #62 Unique Paths ↗
- ☐ ● #72 Edit Distance ↗
- ☐ ● #97 Interleaving String ↗
- ☐ ● #309 Best Time to Buy and Sell Stock with Cooldown ↗
- ☐ ● #494 Target Sum ↗
- ☐ ● #518 Coin Change II ↗
- ☐ ● #1143 Longest Common Subsequence ↗

Bit Manipulation (2) ↗

- ☐ ● #7 Reverse Integer ↗

☐ ● #371 Sum of Two Integers ↗

Math & Geometry (6) ↗

☐ ● #43 Multiply Strings ↗

☐ ● #48 Rotate Image ↗

☐ ● #50 Pow(x, n) ↗

☐ ● #54 Spiral Matrix ↗

☐ ● #73 Set Matrix Zeroes ↗

☐ ● #2013 Detect Squares ↗

Round 9 | Low | H Hard (4) ↗

2-D Dynamic Programming (4) ↗

☐ ● #10 Regular Expression Matching ↗

☐ ● #115 Distinct Subsequences ↗

☐ ● #312 Burst Balloons ↗

☐ ● #329 Longest Increasing Path in a Matrix ↗

Round 1 · High · Easy

Round 1

High Priority · Easy

13 questions · 6 patterns

- ☐ ● Arrays & Hashing #1 #217 #242 ↗
- ☐ ● Two Pointers #125 ↗
- ☐ ● Sliding Window #121 ↗
- ☐ ● Binary Search #704 ↗
- ☐ ● Intervals #252 ↗
- ☐ ● Trees #100 #104 #110 #226 #543 #572 ↗

Arrays & Hashing

Round 1 · High · Easy · 3 questions

Arrays & Hashing

□ #1 Two Sum

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table**

Lists: Grind 169

Problem

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`
Output: `[0,1]`
Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input: `nums = [3,2,4]`, `target = 6`
Output: `[1,2]`

Example 3:

Input: `nums = [3,3]`, `target = 6`
Output: `[0,1]`

Constraints:

- $2 \leq \text{nums.length} \leq 104$
- $-109 \leq \text{nums}[i] \leq 109$
- $-109 \leq \text{target} \leq 109$
- Only one valid answer exists.

Follow-up: Can you come up with an algorithm that is less than $O(n^2)$ time complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def twoSum(self, nums: list[int], target: int) -> list[int]:
        numToIndex = {}

        for i, num in enumerate(nums):
            if target - num in numToIndex:
                return numToIndex[target - num], i
            numToIndex[num] = i
```

• LeetDoocs

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        d = {}
        for i, x in enumerate(nums):
            if (y := target - x) in d:
                return [d[y], i]
            d[x] = i
```

• SimplyLeet

```
def two_sum(nums, target):
    # Create a dictionary to store the number and its index
    num_to_index = {}
    # Iterate over the list with index
    for index, num in enumerate(nums):
        # Calculate the complement needed to reach the target
        complement = target - num
        # Check if the complement is in the dictionary
        if complement in num_to_index:
            # Return the index of the complement and the current index

            return [num_to_index[complement], index]
        # Store the number with its index
        num_to_index[num] = index
    # Since the problem guarantees one solution, we don't need a return
    # statement here.

# Example usage:
# print(two_sum([2, 7, 11, 15], 9)) # Output: [0, 1]
```

• LC.ca

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        m = {}
        for i, x in enumerate(nums):
            y = target - x
            if y in m:
                return [m[y], i]
            m[x] = i
```

◆ Quick Review

Key Insights

- Use a hash table to store the elements and their indices for fast lookup.

- For each element, calculate its complement: target minus the current element.
- Check if the complement exists in the hash table.
- If found, return the indices of the current element and its complement.
- This approach only requires a single pass through the array.

Space & Time

Time Complexity: $O(n)$ – We traverse the list only once.

Space Complexity: $O(n)$ – In the worst-case, we store all elements in the hash table.

Solution

We use a hash table (dictionary) to map each array element to its index as we iterate. For each number, we compute the complement (target – number) and check if it already exists in our hash table. If the complement is found, we return the indices. This one-pass solution ensures linear time complexity and avoids the need for nested loops.

Arrays & Hashing

□ #217 Contains Duplicate

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sorting
Lists: Grind 169

Problem

Given an integer array `nums`, return `true` if any value appears at least twice in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums = [1,2,3,1]`

Output: `true`

Explanation:

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: `nums = [1,2,3,4]`

Output: `false`

Explanation:

All elements are distinct.

Example 3:

Input: `nums = [1,1,1,3,3,4,3,2,4,2]`

Output: true

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def containsDuplicate(self, nums: list[int]) -> bool:
        return len(nums) != len(set(nums))
```

• LeetDoocs

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return any(a == b for a, b in pairwise(sorted(nums)))
```

• SimplyLeet

```
# Function to determine if there are any duplicates in the list.
def contains_duplicate(nums):
    # Create a set to keep track of numbers seen so far.
    seen = set()
    # Iterate over each number in the list.
    for num in nums:
        # If num is already in seen, duplicate found.
        if num in seen:
            return True
    # Add num to the set.
```

```

        seen.add(num)
    # No duplicates found.
    return False

# Example usage:
if __name__ == "__main__":
    nums = [1, 2, 3, 1]
    print(contains_duplicate(nums)) # Expected output: True

```

• LC.ca

```

class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return len(set(nums)) < len(nums)

#####

class Solution(object):
    def containsDuplicate(self, nums):
        """
        :type nums: List[int]

        :rtype: bool
        """
        nums.sort()
        for i in range(0, len(nums) - 1):
            if nums[i] == nums[i + 1]:
                return True
        return False

```

◆ Quick Review

Key Insights

- A hash set can be used to keep track of seen numbers.
- Iterating once through the array provides an efficient solution.

- If a number is already in the set, a duplicate is found.
- Alternative approaches like sorting have a higher time complexity ($O(n \log n)$).

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution uses a hash set to track the numbers seen while iterating through the array. For each number in the array: Check if it exists in the hash set. If it does, return true immediately as a duplicate is found.

Otherwise, add the number to the hash set. If the iteration completes without finding any duplicates, return false. This approach efficiently determines if any duplicates exist in a single pass.

Arrays & Hashing

□ #242 Valid Anagram

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sorting
Lists: Grind 169

Problem

Given two strings *s* and *t*, return true if *t* is an anagram of *s*, and false otherwise.

Example 1:

Input: *s* = "anagram", *t* = "nagaram"

Output: true

Example 2:

Input: *s* = "rat", *t* = "car"

Output: false

Constraints:

- $1 \leq s.length, t.length \leq 5 * 10^4$
- *s* and *t* consist of lowercase English letters.

Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False

        count = collections.Counter(s)
        count.subtract(collections.Counter(t))
        return all(freq == 0 for freq in count.values())
```

• LeetDoocs

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        cnt = Counter(s)
        for c in t:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True
```

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        return Counter(s) == Counter(t)
```

• SimplyLeet

```
# Python solution for Valid Anagram

def isAnagram(s: str, t: str) -> bool:
    # Check if lengths differ; if so, they can't be anagrams
    if len(s) != len(t):
        return False

    # Create a dictionary to count frequency of each character in s
    char_count = {}
    for char in s:
```

```

        # Increase the count of the character
        char_count[char] = char_count.get(char, 0) + 1

    # Decrease the count for each character in t
    for char in t:
        if char not in char_count or char_count[char] == 0:
            # Character not found or count mismatch, not an anagram
            return False
        char_count[char] -= 1

    # If all counts are zero, it's a valid anagram
    return True

# Example usage:
print(isAnagram("anagram", "nagaram")) # Output: True
print(isAnagram("rat", "car"))         # Output: False

```

• LC.ca

```

class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        cnt = Counter(s)
        for c in t:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True

```

◆ Quick Review

Key Insights

- If *s* and *t* have different lengths, they cannot be anagrams.

- An anagram requires exactly the same character frequency for both strings.
- Using a hash table (or frequency array when limited to 26 lowercase letters) provides an efficient solution.
- An alternative approach is to sort both strings and compare them.
- For Unicode inputs, a hash table is more adaptable than a fixed-size frequency array.

Space & Time

Time Complexity: $O(n)$, where n is the length of the strings (using a hash table for counting).

Space Complexity: $O(1)$ if we use a fixed-size array (for 26 letters), otherwise $O(n)$ for a hash table with a larger character set.

Solution

We use a hash table (dictionary) to count the frequency of each character in the first string s . Then, we iterate over string t and decrement the count for each character. If at any point a character count goes negative or a character is missing in the hash table, t cannot be an anagram of s . Finally, if all counts return to zero, the strings are anagrams. This approach

is efficient and handles the problem in linear time.

Two Pointers

Round 1 · High · Easy · 1 question

Two Pointers

□ #125 Valid Palindrome

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String
Lists: Grind 169

Problem

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward.

Alphanumeric characters include letters and numbers.

Given a string *s*, return true if it is a palindrome, or false otherwise.

Example 1:

```
Input: s = "A man, a plan, a canal: Panama"
Output: true
Explanation: "amanaplanacanalpanama" is a palindrome.
```

Example 2:

```
Input: s = "race a car"
Output: false
Explanation: "raceacar" is not a palindrome.
```

Example 3:

Input: s = " "

Output: true

Explanation: s is an empty string "" after removing non-alphanumeric characters.

Since an empty string reads the same forward and backward, it is a palindrome.

Constraints:

- $1 \leq s.length \leq 2 * 10^5$
- s consists only of printable ASCII characters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        l = 0
        r = len(s) - 1

        while l < r:
            while l < r and not s[l].isalnum():
                l += 1
            while l < r and not s[r].isalnum():
                r -= 1

            if s[l].lower() != s[r].lower():
                return False
            l += 1
            r -= 1

        return True
```

• LeetDoocs


```

class Solution:
    def isPalindrome(self, s: str) -> bool:
        i, j = 0, len(s) - 1
        while i < j:
            if not s[i].isalnum():
                i += 1
            elif not s[j].isalnum():
                j -= 1
            elif s[i].lower() != s[j].lower():
                return False

        else:
            i, j = i + 1, j - 1
        return True

```

• SimplyLeet

```

class Solution:
    def isPalindrome(self, s: str) -> bool:
        # Initialize two pointers at the beginning and end of the string
        left, right = 0, len(s) - 1
        while left < right:
            # Skip non-alphanumeric characters from the left
            while left < right and not s[left].isalnum():
                left += 1
            # Skip non-alphanumeric characters from the right
            while left < right and not s[right].isalnum():
                right -= 1
            # Compare the characters in lowercase
            if s[left].lower() != s[right].lower():
                return False
            left += 1
            right -= 1
        return True

```

• LC.ca

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        i, j = 0, len(s) - 1
        while i < j:
            if not s[i].isalnum():
                i += 1
            elif not s[j].isalnum():
                j -= 1
            elif s[i].lower() != s[j].lower():
                return False

        else:
            i, j = i + 1, j - 1
        return True
```

◆ Quick Review

Key Insights

- Use two pointers (one starting at the beginning and one at the end) to compare characters.
- Skip characters that are not alphanumeric.
- Compare characters in a case-insensitive manner.
- Continue until the pointers meet or cross.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, as each character is processed at most once.

Space Complexity: $O(1)$, using only a few extra variables for pointers and temporary comparisons.

Solution

The solution employs a two–pointer approach: Initialize two pointers: one at the start (left) and one at the end (right) of the string. Skip any non–alphanumeric characters using these pointers. Convert the remaining characters at each pointer to lowercase and compare them. If any mismatch is found, return false; otherwise, continue until the pointers cross. Return true if all corresponding characters match. This method efficiently checks for a palindrome without needing additional space for a filtered string.

Sliding Window

Round 1 · High · Easy · 1 question

Sliding Window

□ #121 Best Time to Buy and Sell Stock

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the i th day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = $6 - 1 = 5$.

Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

Input: prices = [7,6,4,3,1]

Output: 0

Explanation: In this case, no transactions are done and the max profit = 0.

Constraints:

- $1 \leq \text{prices.length} \leq 105$
- $0 \leq \text{prices}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        sellOne = 0
        holdOne = -math.inf

        for price in prices:
            sellOne = max(sellOne, holdOne + price)
            holdOne = max(holdOne, -price)

        return sellOne
```

• LeetDoocs

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        ans, mi = 0, inf
        for v in prices:
            ans = max(ans, v - mi)
            mi = min(mi, v)
        return ans
```

• SimplyLeet

```
def maxProfit(prices):
    # Initialize min_price as infinity and max_profit as 0
    min_price = float('inf')
    max_profit = 0
    # Iterate through each price in the list
    for price in prices:
        # Update the minimum price if current price is lower
        if price < min_price:
            min_price = price
        else:
            # Calculate current profit and update max_profit if it is higher
            max_profit = max(max_profit, price - min_price)
    return max_profit

# Example usage:
# result = maxProfit([7,1,5,3,6,4])
# print(result) # Output should be 5
```

• LC.ca

```
...
>>> import math
>>> a = math.inf
>>> a
inf

>>> x = float('-inf')
>>> y = float('inf')
>>> print(x < y) # Output: True
True

>>>
>>> z = -math.inf
>>> x==z
True

...
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        ans, mi = 0, inf
        for v in prices:
```

```
        mi = min(mi, v)
        ans = max(ans, v - mi)
    return ans

#####

class Solution(object):
    def maxProfit(self, prices):
        """
        :type prices: List[int]

        :rtype: int
        """
        if not prices:
            return 0
        ans = 0
        pre = prices[0]
        for i in range(1, len(prices)):
            pre = min(pre, prices[i])
            ans = max(prices[i] - pre, ans)
        return ans
```

◆ Quick Review

Key Insights

- Traverse the list of prices once while keeping track of the minimum price encountered so far.
- At each day, calculate the profit if you sold the stock on that day.
- Update the maximum profit when the current profit exceeds the previous maximum.
- The optimal solution makes a single pass through the array with constant space usage.

Space & Time

Time Complexity: $O(n)$ – requires one pass through the prices array.

Space Complexity: $O(1)$ – only a few extra variables are used.

Solution

We solve the problem using a simple linear scan. We maintain two key variables: one for the minimum price seen so far and another for the maximum profit calculated so far. As we iterate, we continuously update the minimum price if a lower price is found, and then compute the potential profit at the current price. If this profit exceeds the current maximum profit, we update the maximum profit. The final maximum profit is returned after iterating through all prices.

Binary Search

Round 1 · High · Easy · 1 question

Binary Search

□ #704 Binary Search

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Grind 169

Problem

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [-1,0,3,5,9,12]`, `target = 9`
Output: `4`
Explanation: 9 exists in `nums` and its index is 4

Example 2:

Input: `nums = [-1,0,3,5,9,12]`, `target = 2`
Output: `-1`
Explanation: 2 does not exist in `nums` so return `-1`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$

- $-104 < \text{nums}[i], \text{target} < 104$
- All the integers in `nums` are unique.
- `nums` is sorted in ascending order.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        i = bisect.bisect_left(nums, target)
        return -1 if i == len(nums) or nums[i] != target else i
```

• LeetDoocs

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        l, r = 0, len(nums) - 1
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] >= target:
                r = mid
            else:
                l = mid + 1
        return l if nums[l] == target else -1
```

• SimplyLeet

```
# Python code for binary search

def search(nums, target):
    # Set initial boundaries for the search
    left, right = 0, len(nums) - 1

    # Loop until the search space is empty
    while left <= right:
        # Calculate middle index to avoid overflow
        mid = left + (right - left) // 2
```

```
# Check if the mid element matches the target
if nums[mid] == target:
    return mid
# If target is smaller than mid element, narrow search to left half
elif target < nums[mid]:
    right = mid - 1
# Else, narrow search to right half
else:
    left = mid + 1
```

```
# Target not found, return -1
return -1
```

```
# Example usage:
```

```
print(search([-1,0,3,5,9,12], 9)) # Expected output: 4
```

• LC.ca

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[mid] >= target:
                right = mid
            else:
                left = mid + 1
        return left if nums[left] == target else -1
```

◆ Quick Review

Key Insights

- Utilize binary search to achieve $O(\log n)$ runtime on a sorted array.

- Use two pointers to maintain the current search region.
- Narrow down the search range by comparing the middle element with the target.
- Return the index when the target is found; otherwise, return -1 if not found.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

We use an iterative binary search algorithm. We initialize two pointers, left and right, to represent the current segment of the array that could contain the target. At each iteration, we compute the middle index and compare the element there with the target. If they match, we return the index. Otherwise, based on whether the target is smaller or larger than the middle element, we adjust our pointers to narrow the search to either the left or right half of the array. This process continues until the target is found or until the pointers cross, indicating that the target is not present. No extra data structures are used, making the

space complexity $O(1)$.

Intervals

Round 1 · High · Easy · 1 question

Intervals

□ ★ #252 Meeting Rooms ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Given an array of meeting time intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, determine if a person could attend all meetings.

Example 1:

```
Input: intervals = [[0,30],[5,10],[15,20]]  
Output: false
```

Example 2:

```
Input: intervals = [[7,10],[2,4]]  
Output: true
```

Constraints:

- $0 \leq \text{intervals.length} \leq 104$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i < \text{end}_i \leq 106$

Community Solutions (Python)

- • WalkCC

```
class Solution:
    def canAttendMeetings(self, intervals: list[list[int]]) -> bool:
        intervals.sort()

        for i in range(1, len(intervals)):
            if intervals[i - 1][1] > intervals[i][0]:
                return False

        return True
```

• LeetDoocs

```
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        return all(a[1] <= b[0] for a, b in pairwise(intervals))
```

• SimplyLeet

```
# Python solution for checking if one can attend all meetings

def canAttendMeetings(intervals):
    # Sort intervals based on start time
    intervals.sort(key=lambda x: x[0])
    # Iterate through intervals to find any overlap
    for i in range(1, len(intervals)):
        # Check if the previous meeting ends after the current meeting starts
        if intervals[i-1][1] > intervals[i][0]:
            return False

    return True

# Example usage:
print(canAttendMeetings([[0, 30], [5, 10], [15, 20]])) # Expected output:
False
print(canAttendMeetings([[7, 10], [2, 4]]))           # Expected output:
True
```

• LC.ca

```
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        return all(a[1] <= b[0] for a, b in pairwise(intervals))
```

```
#####
```

```
from itertools import pairwise
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
```

```
        intervals.sort(key=lambda x: x[0])
        return not any(a[1] > b[0] for a,b in pairwise(intervals))
        # any is True, then conflict found
```

```
#####
```

```
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort(key=lambda x: x[0])
        for i in range(len(intervals) - 1):
```

```
            if intervals[i][1] > intervals[i + 1][0]:
                return False
        return True
```

```
#####
```

```
# Definition for an interval.
# class Interval(object):
# def __init__(self, s=0, e=0):
# self.start = s
```

```
# self.end = e
```

```
class Solution(object):
    def canAttendMeetings(self, intervals):
        """
        :type intervals: List[Interval]
        :rtype: bool
        """
        intervals = sorted(intervals, key=lambda x: x.start)
        for i in range(1, len(intervals)):
```

```
if intervals[i].start < intervals[i - 1].end:  
    return False  
return True
```

◆ Quick Review

Key Insights

- Sort the intervals based on their starting times.
- Compare each interval's end time with the next interval's start time.
- Detect overlap if the end time of one meeting exceeds the start time of the following meeting.
- If no overlaps are detected, all meetings can be attended.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(1)$ if sorting is done in-place; otherwise $O(n)$.

Solution

The solution starts by sorting the intervals in ascending order by their start times. After sorting, iterate through the intervals and for

each consecutive pair, check if the previous meeting's end time exceeds the next meeting's start time. An overlap indicates that the person cannot attend all meetings. The primary data structure used is an array (or list) for holding the intervals. Sorting makes it easier to compare adjacent intervals for potential overlaps.

Trees

Round 1 · High · Easy · 6 questions

Trees

□ #100 Same Tree

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS

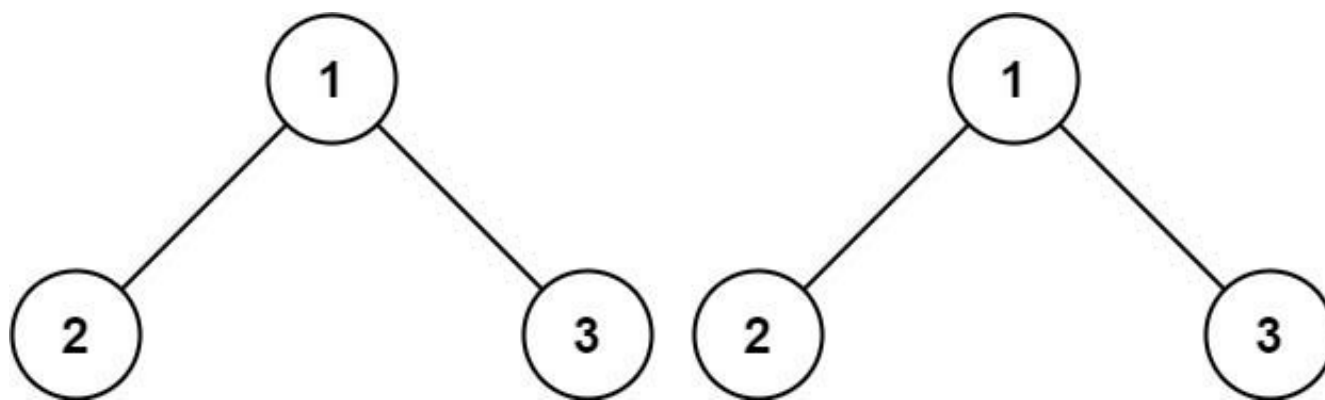
Lists: Grind 169

Problem

Given the roots of two binary trees p and q , write a function to check if they are the same or not.

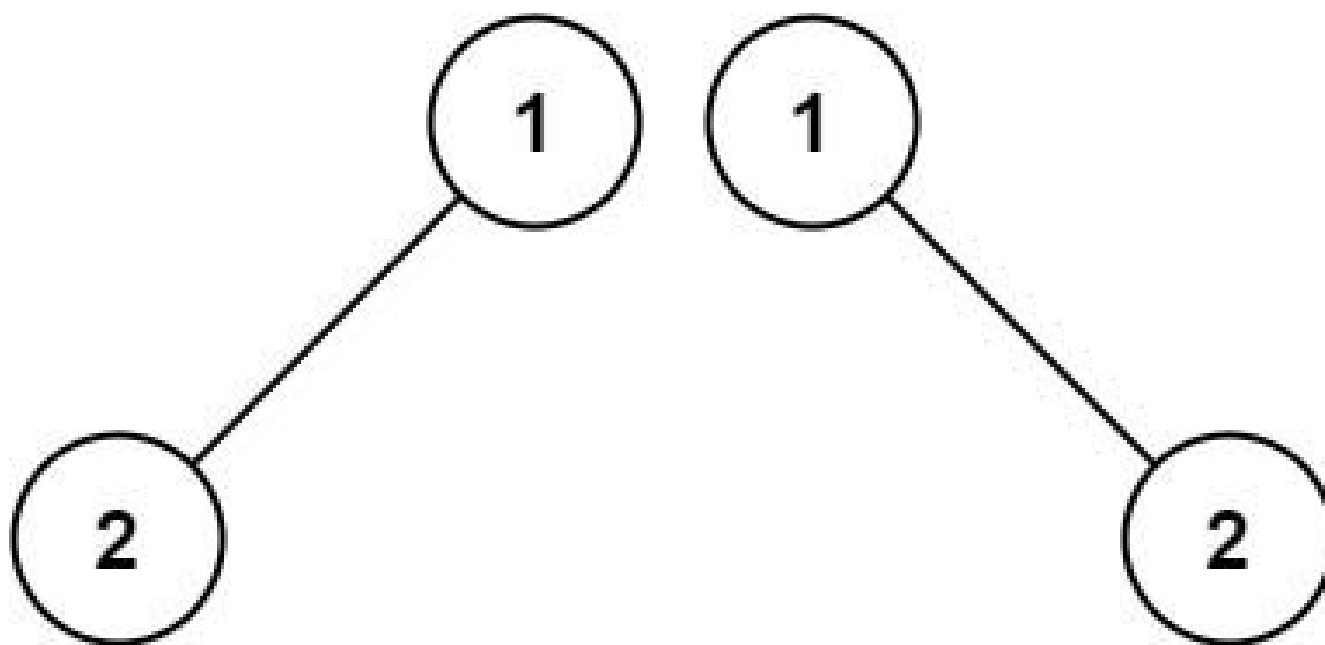
Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:



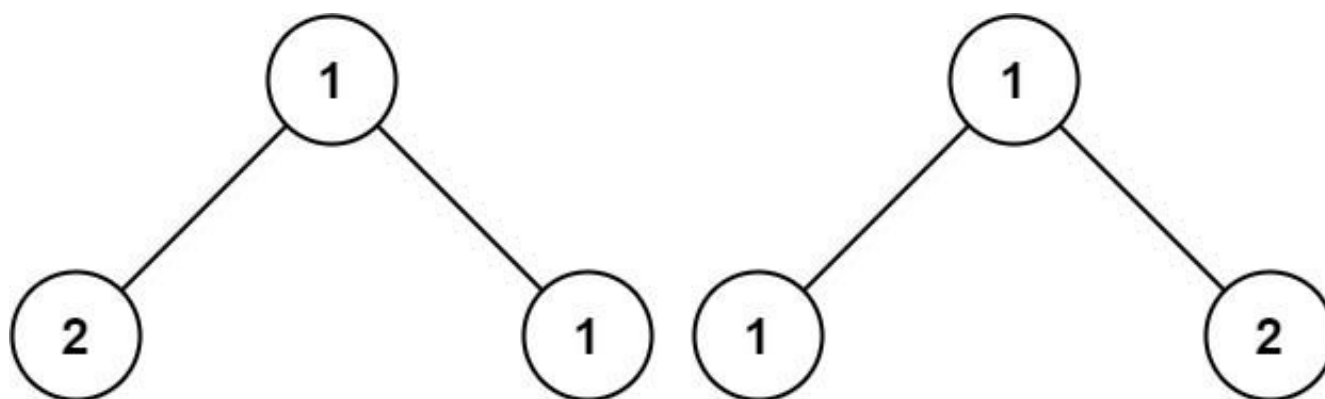
Input: $p = [1,2,3]$, $q = [1,2,3]$
Output: true

Example 2:



Input: $p = [1, 2]$, $q = [1, \text{null}, 2]$
Output: false

Example 3:



Input: $p = [1, 2, 1]$, $q = [1, 1, 2]$
Output: false

Constraints:

- The number of nodes in both trees is in the range $[0, 100]$.
- $-104 \leq \text{Node.val} \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isSameTree(self, p: TreeNode | None, q: TreeNode | None) -> bool:
        if not p or not q:
            return p == q
        return (p.val == q.val and
                self.isSameTree(p.left, q.left) and
                self.isSameTree(p.right, q.right))
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool
    :
        if p == q:
            return True

        if p is None or q is None or p.val != q.val:
            return False
        return self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.
right)
```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def isSameTree(p, q):
    # If both nodes are None, trees are identical at this branch.
    if not p and not q:
        return True
    # If one of the nodes is None, trees are not identical.
    if not p or not q:
        return False
    # Check if current nodes have the same value.
    if p.val != q.val:
        return False
    # Recursively compare the left subtrees and the right subtrees.
    return isSameTree(p.left, q.left) and isSameTree(p.right, q.right)

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x

```

```
# self.left = None
# self.right = None

class Solution(object):
    def isSameTree(self, p, q):
        """
        :type p: TreeNode
        :type q: TreeNode
        :rtype: bool
        """

        if not p or not q:
            return p == q # covers: p=None and q!=None, q=None and p!=None, both None
        return p.val == q.val and self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right)

# iteration
class Solution:
    def isSameTree(self, p: TreeNode, q: TreeNode) -> bool:
        if p is None:
            return q is None

        if q is None:
            return p is None

        stack1 = [p]
        stack2 = [q]

        while stack1 and stack2:
            current1 = stack1.pop()
            current2 = stack2.pop()
```

```
        if current1 is None and current2 is None:
            continue
        elif current1 is None or current2 is None:
            return False
        elif current1.val != current2.val:
            return False

        stack1.append(current1.left)
        stack2.append(current2.left)

        stack1.append(current1.right)
        stack2.append(current2.right)

    return not stack1 and not stack2
```

◆ Quick Review

Key Insights

- Use recursion to compare nodes in both trees.
- If both nodes being compared are null, they are the same.
- If one node is null while the other is not, the trees differ.
- Check if current nodes' values match, then recursively verify the left and right subtrees.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the trees, since every node is

visited once.

Space Complexity: $O(n)$ in the worst-case (unbalanced tree), due to recursion call stack usage. In a balanced tree, the space would be $O(\log n)$.

Solution

The problem can be solved using a recursive approach (Depth-First Search) where we compare the current nodes from both trees. If the current nodes are both null, they match; if one is null, then the trees are not identical. For non-null nodes, check if the values are equal. Then, recursively apply the same logic on the left and right children of the nodes. This approach ensures that both the structure and the node values are identical across the two trees.

Trees

□ #104 Maximum Depth of Binary Tree

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS

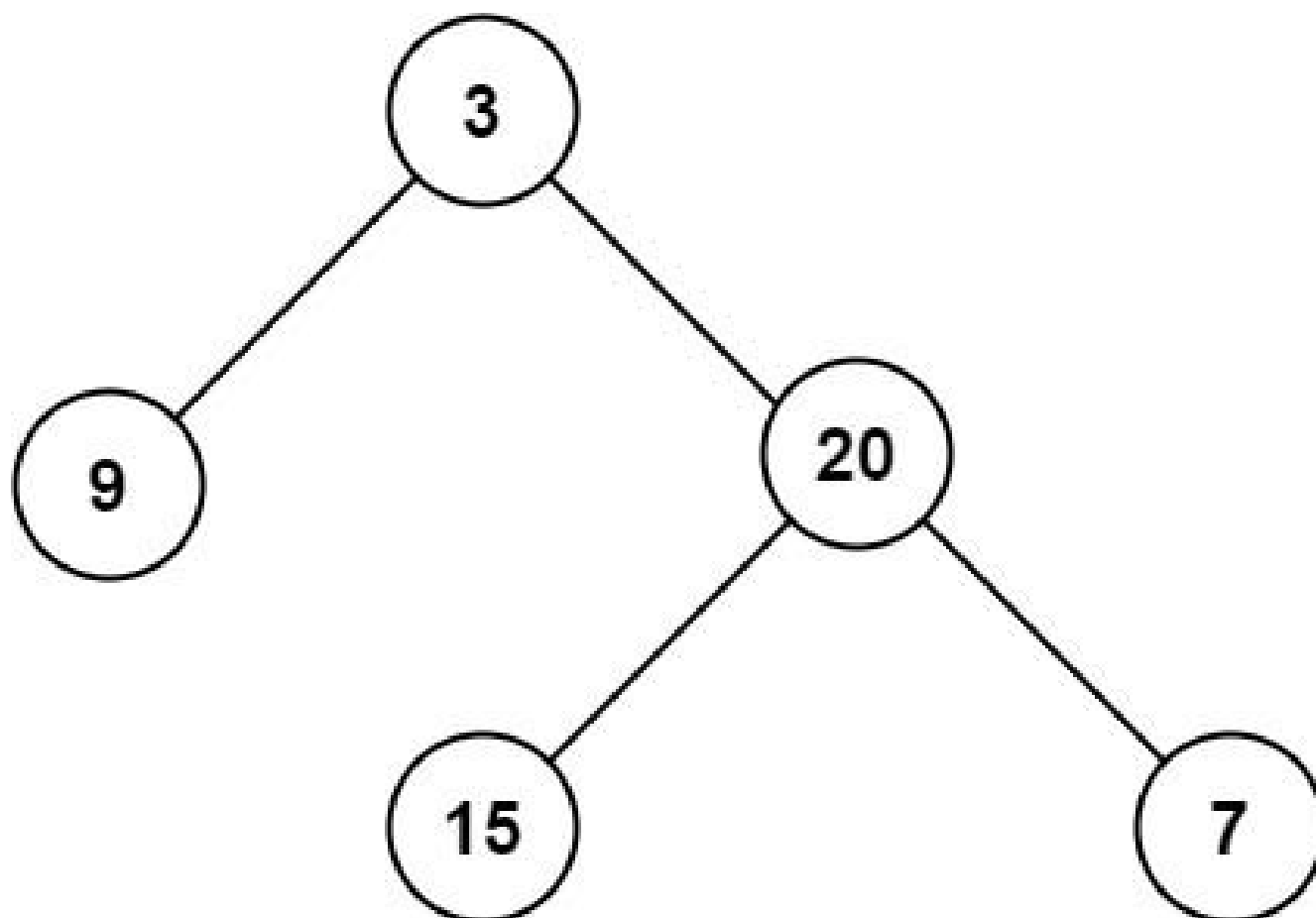
Lists: Grind 169

Problem

Given the root of a binary tree, return its maximum depth.

A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: 3

Example 2:

Input: root = [1,null,2]
Output: 2

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-100 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def maxDepth(self, root: TreeNode | None) -> int:
        if not root:
            return 0
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxDepth(self, root: TreeNode) -> int:
        if root is None:
            return 0

        l, r = self.maxDepth(root.left), self.maxDepth(root.right)
        return 1 + max(l, r)
```

• SimplyLeet

```
# Python implementation using DFS

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
```



```
def maxDepth(self, root: Optional[TreeNode]) -> int:
    # Base case: if the node is null, return 0.
    if not root:
        return 0

    # Recursively compute the maximum depth of the left subtree.
    left_depth = self.maxDepth(root.left)
    # Recursively compute the maximum depth of the right subtree.
    right_depth = self.maxDepth(root.right)

    # Return the greater depth plus one for the current node.
    return 1 + max(left_depth, right_depth)
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def maxDepth(self, root: TreeNode) -> int:
        if root is None:
            return 0

        l, r = self.maxDepth(root.left), self.maxDepth(root.right)
        return 1 + max(l, r)
```

◆ Quick Review

Key Insights

- Use a recursive strategy (DFS) to traverse the binary tree.
- At each node, compute the maximum depth of its left and right subtree.

- The maximum depth at the current node is 1 (current node) plus the maximum of the left and right subtree depths.
- Alternatively, an iterative approach using BFS (level-order traversal) can be used.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, since every node is visited once.

Space Complexity: $O(h)$ for recursion stack in DFS, where h is the height of the tree (worst-case $O(n)$ for a skewed tree); $O(n)$ for BFS in the worst-case scenario if the tree is balanced.

Solution

The problem is approached using Depth-First Search (DFS) with recursion. The idea is to visit every node and calculate the maximum depth of the left and right subtrees, then add 1 for the current node. This method naturally handles the base case when a null node (empty tree) is reached by returning 0.

Alternatively, a Breadth-First Search (BFS) approach can determine the tree level by level, with the total number of levels equaling the

maximum depth. Key data structures include recursion call stack for DFS or a queue for BFS.

Trees

□ #110 Balanced Binary Tree

EAS

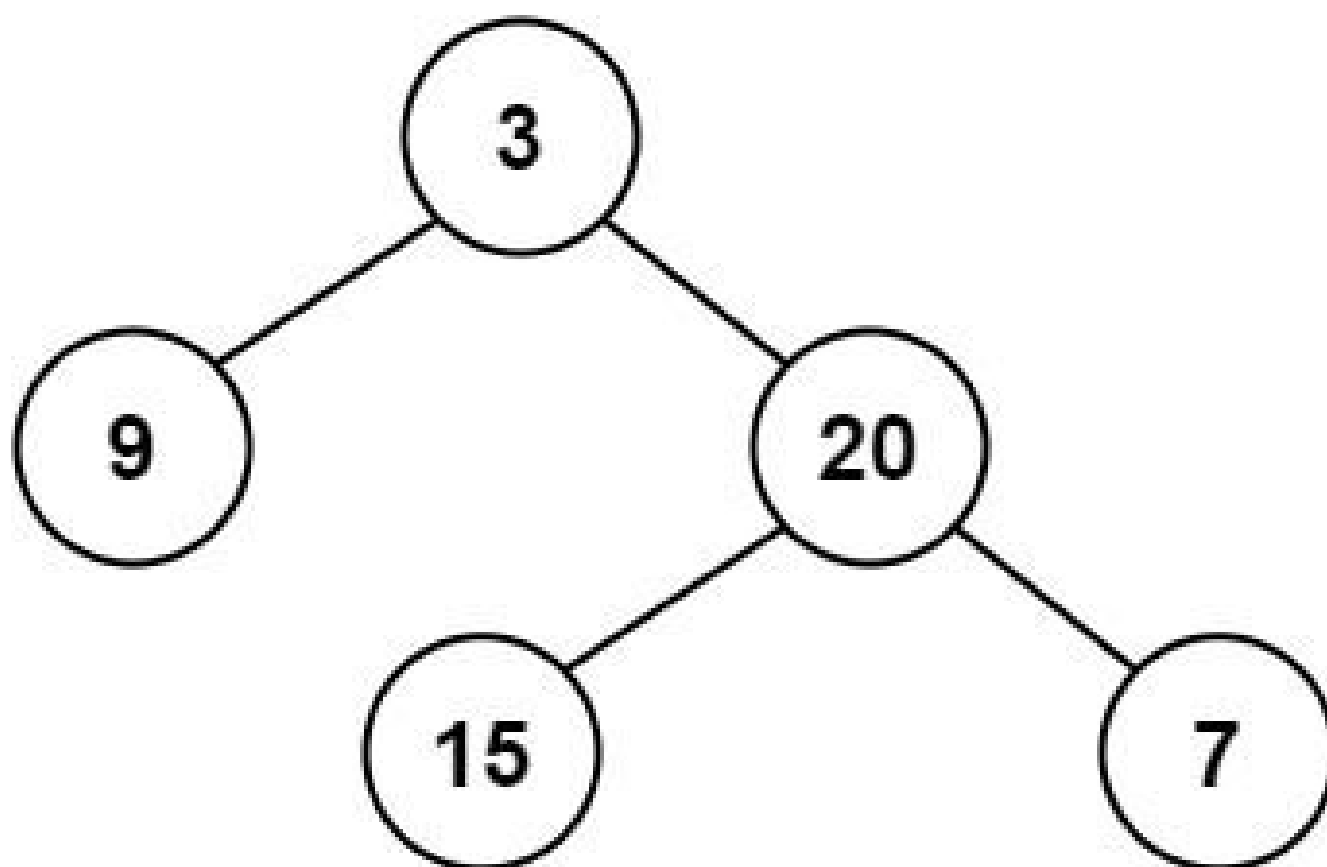
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: Grind 169

Problem

Given a binary tree, determine if it is height-balanced.

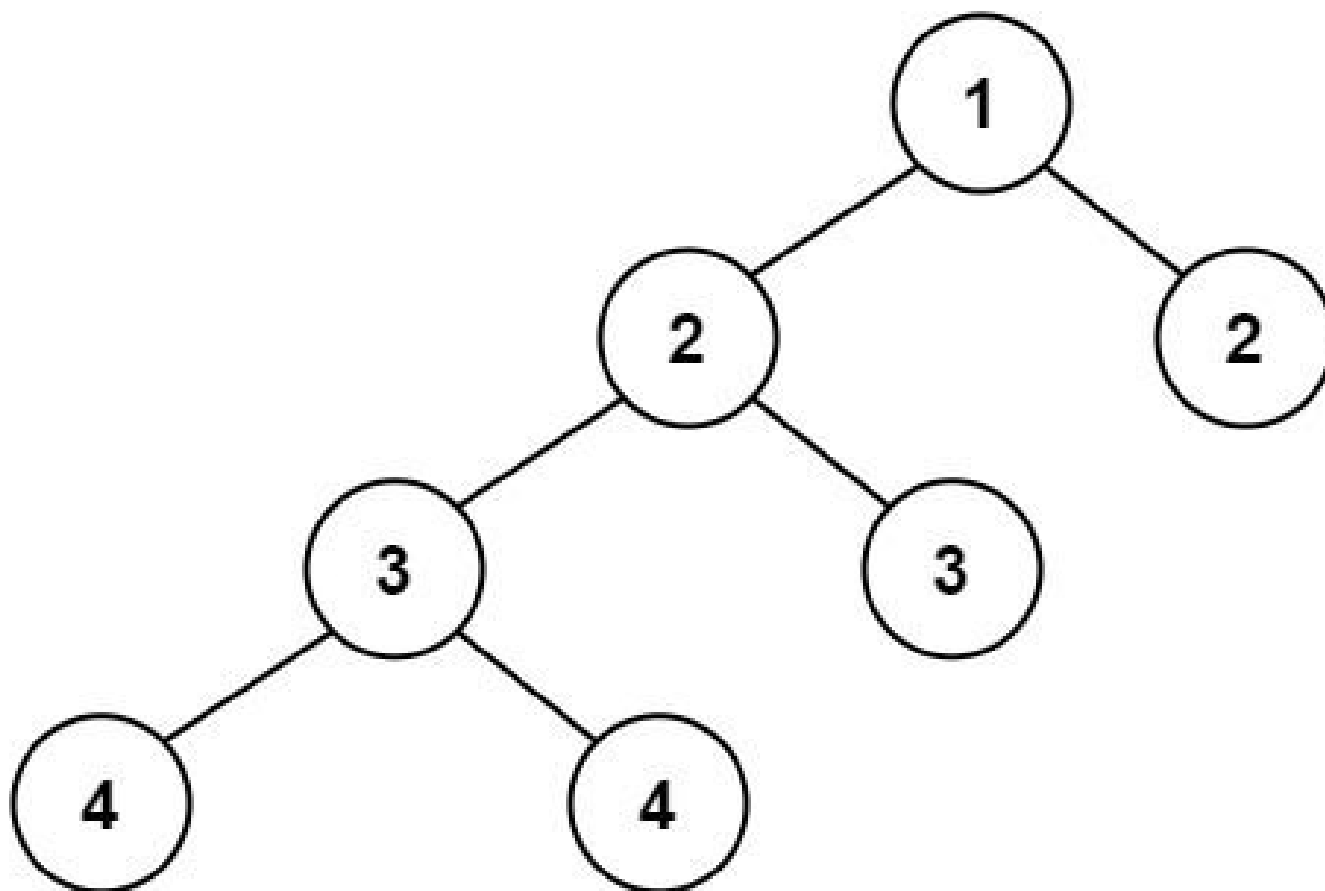
Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: true

Example 2:



Input: root = [1,2,2,3,3,null,null,4,4]

Output: false

Example 3:

Input: root = []

Output: true

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- $-104 \leq \text{Node.val} \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isBalanced(self, root: TreeNode | None) -> bool:
        if not root:
            return True

        def maxDepth(root: TreeNode | None) -> int:
            if not root:
                return 0
            return 1 + max(maxDepth(root.left), maxDepth(root.right))

        return (abs(maxDepth(root.left) - maxDepth(root.right)) <= 1 and
                self.isBalanced(root.left) and
                self.isBalanced(root.right))
```

```
class Solution:
    def isBalanced(self, root: TreeNode | None) -> bool:
        ans = True

        def maxDepth(root: TreeNode | None) -> int:
            """Returns the height of root and sets ans to false if root
            unbalanced."""
            nonlocal ans
            if not root or not ans:
                return 0
            left = maxDepth(root.left)

            right = maxDepth(root.right)
            if abs(left - right) > 1:
                ans = False
            return max(left, right) + 1

        maxDepth(root)
        return ans
```

```

class Solution:
    def isBalanced(self, root: TreeNode | None) -> bool:
        def maxDepth(root: TreeNode | None) -> int:
            """Returns the height of root if root is balanced; otherwise, returns
            -1."""
            if not root:
                return 0

            left = maxDepth(root.left)
            if left == -1:
                return -1

            right = maxDepth(root.right)
            if right == -1:
                return -1
            if abs(left - right) > 1:
                return -1

            return 1 + max(maxDepth(root.left), maxDepth(root.right))

        return maxDepth(root) != -1

```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def height(root):
            if root is None:
                return 0
            l, r = height(root.left), height(root.right)
            if l == -1 or r == -1 or abs(l - r) > 1:
                return -1
            return 1 + max(l, r)

        return height(root) >= 0

```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def isBalanced(self, root: TreeNode) -> bool:
        # Helper function to return the height of the subtree or -1 if
        # unbalanced.

        def checkHeight(node: TreeNode) -> int:
            # An empty tree has height 0 and is balanced.
            if not node:
                return 0

            # Recursively get the height of the left subtree.
            leftHeight = checkHeight(node.left)
            # If left subtree is unbalanced, return -1.
            if leftHeight == -1:
                return -1

            # Recursively get the height of the right subtree.
            rightHeight = checkHeight(node.right)
            # If right subtree is unbalanced, return -1.
            if rightHeight == -1:
                return -1

            # If the current node's subtrees differ in height by more than 1,
            # it's unbalanced.
            if abs(leftHeight - rightHeight) > 1:
                return -1

            # Return the height of the current node.
            return max(leftHeight, rightHeight) + 1

        # The tree is balanced if checkHeight does not return -1.
        return checkHeight(root) != -1
```


• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def height(root):
            if root is None:
                return 0
            l, r = height(root.left), height(root.right)
            if l == -1 or r == -1 or abs(l - r) > 1:
                return -1
            return 1 + max(l, r)

        return height(root) >= 0
```

◆ Quick Review

Key Insights

- Use a bottom-up DFS (Depth-First Search) traversal to compute the height of each subtree.
- If any subtree is found unbalanced, propagate an error indicator (e.g., -1) upward, allowing early termination.
- An empty tree is considered balanced.
- The key check is to ensure that for each node, the absolute difference between the

heights of its left and right subtrees is no more than 1.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes in the tree, since we visit each node once.

Space Complexity: $O(h)$ where h is the height of the tree, which corresponds to the recursion stack ($O(n)$ in the worst-case of a skewed tree).

Solution

The solution uses a recursive DFS approach. A helper function recursively computes the height of a subtree. If a subtree is found to be unbalanced (the height difference between its left and right subtrees exceeds 1), the helper returns a special value (e.g., -1) to indicate that the subtree is unbalanced. This value is then propagated up the recursion to terminate further unnecessary computations. The main function checks the result of the helper function and returns true if the tree is balanced (i.e., the helper did not return -1) and false otherwise.

Trees

□ #226 Invert Binary Tree

EAS

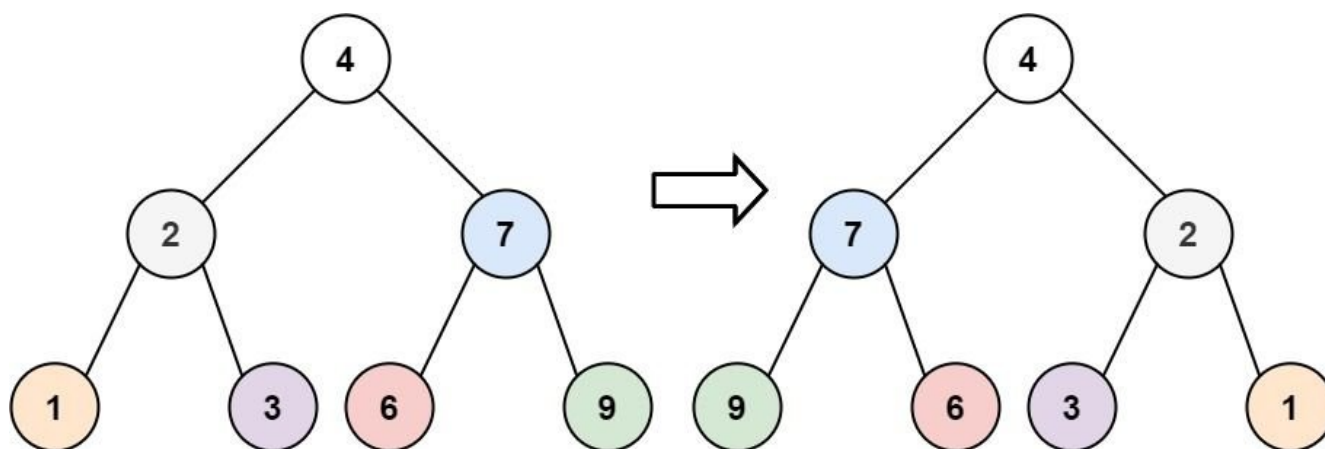
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, invert the tree, and return its root.

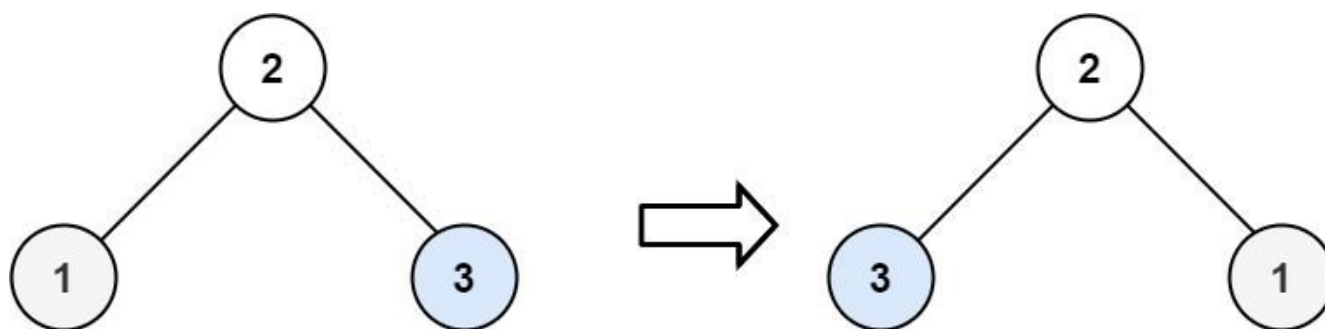
Example 1:



Input: root = [4,2,7,1,3,6,9]

Output: [4,7,2,9,6,3,1]

Example 2:



Input: root = [2,1,3]

Output: [2,3,1]

Example 3:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

• WalkCC

```

class Solution:
    def invertTree(self, root: TreeNode | None) -> TreeNode | None:
        if not root:
            return None

        left = root.left
        right = root.right
        root.left = self.invertTree(right)
        root.right = self.invertTree(left)
        return root
  
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if root is None:
            return None

        l, r = self.invertTree(root.left), self.invertTree(root.right)
        root.left, root.right = r, l
        return root
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val        # Initialize node value
        self.left = left       # Pointer to left child
        self.right = right      # Pointer to right child

class Solution:
    def invertTree(self, root: TreeNode) -> TreeNode:
        # Base case: if the node is None, return None.

        if not root:
            return None
        # Recursively invert the left and right subtrees.
        left_inverted = self.invertTree(root.left)
        right_inverted = self.invertTree(root.right)
        # Swap the left and right children.
        root.left, root.right = right_inverted, left_inverted
        return root
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        def dfs(root):
            if root is None:
                return
            root.left, root.right = root.right, root.left
            dfs(root.left)
            dfs(root.right)

        dfs(root)
        return root
```

◆ Quick Review

Key Insights

- The problem can be solved using recursion (depth-first search) by swapping the children of every node.
- Alternatively, an iterative approach (using a queue for breadth-first search) can be used.
- The recursion terminates when a null node is encountered.
- The operation is performed in-place, modifying the original tree structure.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes, since each node is visited once.

Space Complexity: $O(h)$ where h is the height of the tree, which is the recursion stack space in the recursive approach. In worst-case (skewed tree), this could be $O(n)$.

Solution

We solve the problem using recursion (depth-first search). The approach is to: Check the base case: if the node is null, simply return null. Recursively invert the left subtree and the right subtree. Swap the left and right children of the current node. Return the current node after the swap. The recursive approach elegantly handles all nodes and ensures the inversion process covers the complete tree.

Trees

□ #543 Diameter of Binary Tree

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS**

Lists: Grind 169

Problem

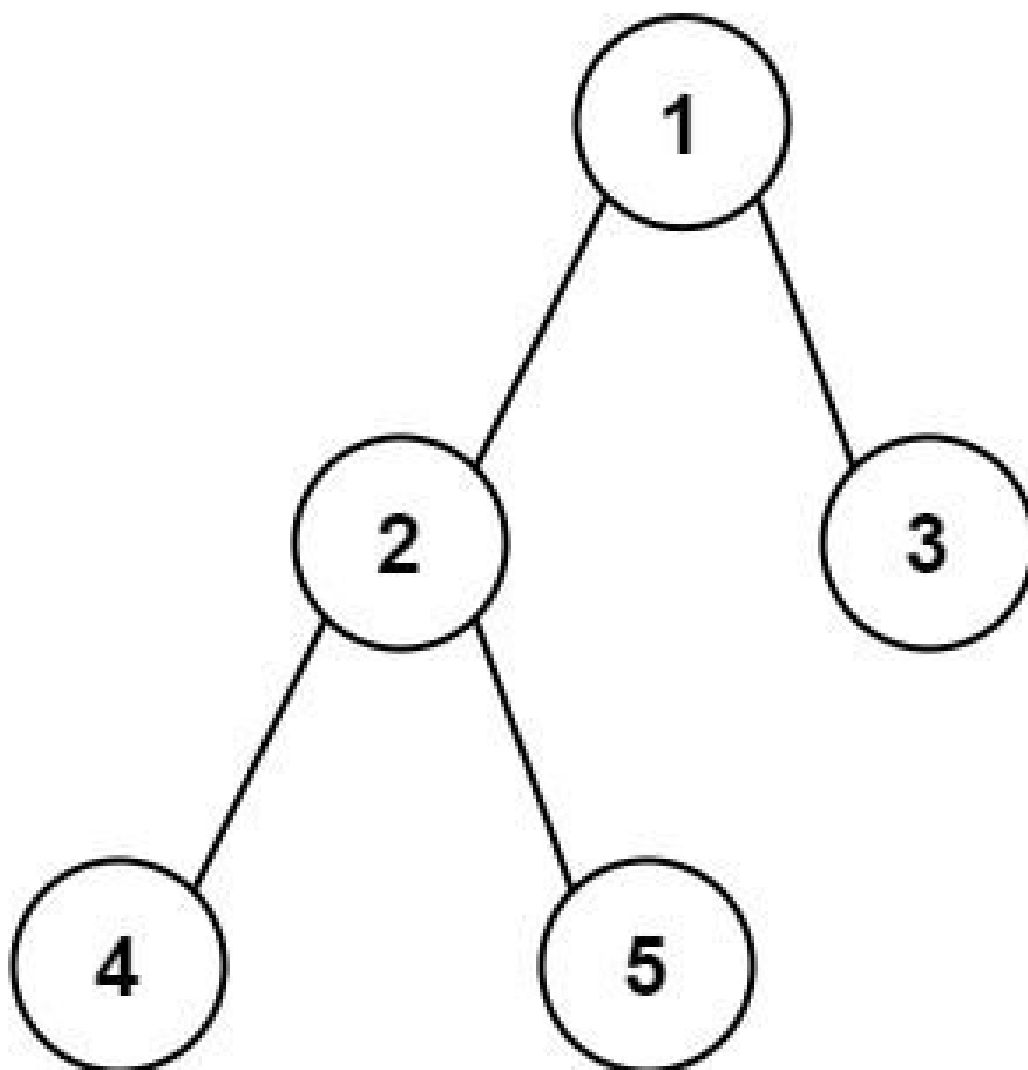
Given the root of a binary tree, return the length of the diameter of the tree.

The diameter of a binary tree is the length of the longest path between any two nodes in a tree.

This path may or may not pass through the root.

The length of a path between two nodes is represented by the number of edges between them.

Example 1:



Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].

Example 2:

Input: root = [1,2]

Output: 1

Constraints:

- The number of nodes in the tree is in the range [1, 10⁴].
- $-100 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def diameterOfBinaryTree(self, root: TreeNode | None) -> int:
        ans = 0

        def maxDepth(root: TreeNode | None) -> int:
            nonlocal ans
            if not root:
                return 0

            l = maxDepth(root.left)

            r = maxDepth(root.right)
            ans = max(ans, l + r)
            return 1 + max(l, r)

        maxDepth(root)
        return ans
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        def dfs(root):
            if root is None:
```

```

        return 0
    nonlocal ans
    left, right = dfs(root.left), dfs(root.right)
    ans = max(ans, left + right)
    return 1 + max(left, right)

ans = 0
dfs(root)
return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        # Global variable to track the maximum diameter found

```

```

        self.max_diameter = 0

```

```

    def dfs(self, node):
        # Base case: if node is None, depth is 0.
        if not node:
            return 0
        # Compute the depth of left and right subtrees.
        left_depth = self.dfs(node.left)
        right_depth = self.dfs(node.right)
        # Update the global diameter: the path through this node

```

```

        self.max_diameter = max(self.max_diameter, left_depth + right_depth)

    def dfs(self, node):
        # Return the depth of the tree rooted at this node.
        return max(left_depth, right_depth) + 1

    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        self.dfs(root)
        return self.max_diameter

```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        def dfs(root):
            if root is None:
                return 0
            nonlocal ans
            left, right = dfs(root.left), dfs(root.right)
            ans = max(ans, left + right)
            return 1 + max(left, right)

        ans = 0
        dfs(root)
        return ans
```

◆ Quick Review

Key Insights

- Use depth-first search (DFS) to explore the tree.
- The diameter at a node is the sum of the depths of its left and right subtrees.
- Track the maximum diameter found during the DFS traversal.
- Recursively calculating the depth of subtrees provides an efficient solution.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree.

Space Complexity: $O(n)$, in the worst-case scenario due to the recursion stack (skewed tree).

Solution

The solution employs a DFS traversal of the binary tree. At every node in the tree, compute the depth of the left and right subtrees. The sum of these depths gives the potential diameter (longest path through that node). Update a global maximum diameter variable if this sum exceeds the current maximum. Finally, return the maximum diameter after completing the DFS.

Trees

□ #572 Subtree of Another Tree

EAS

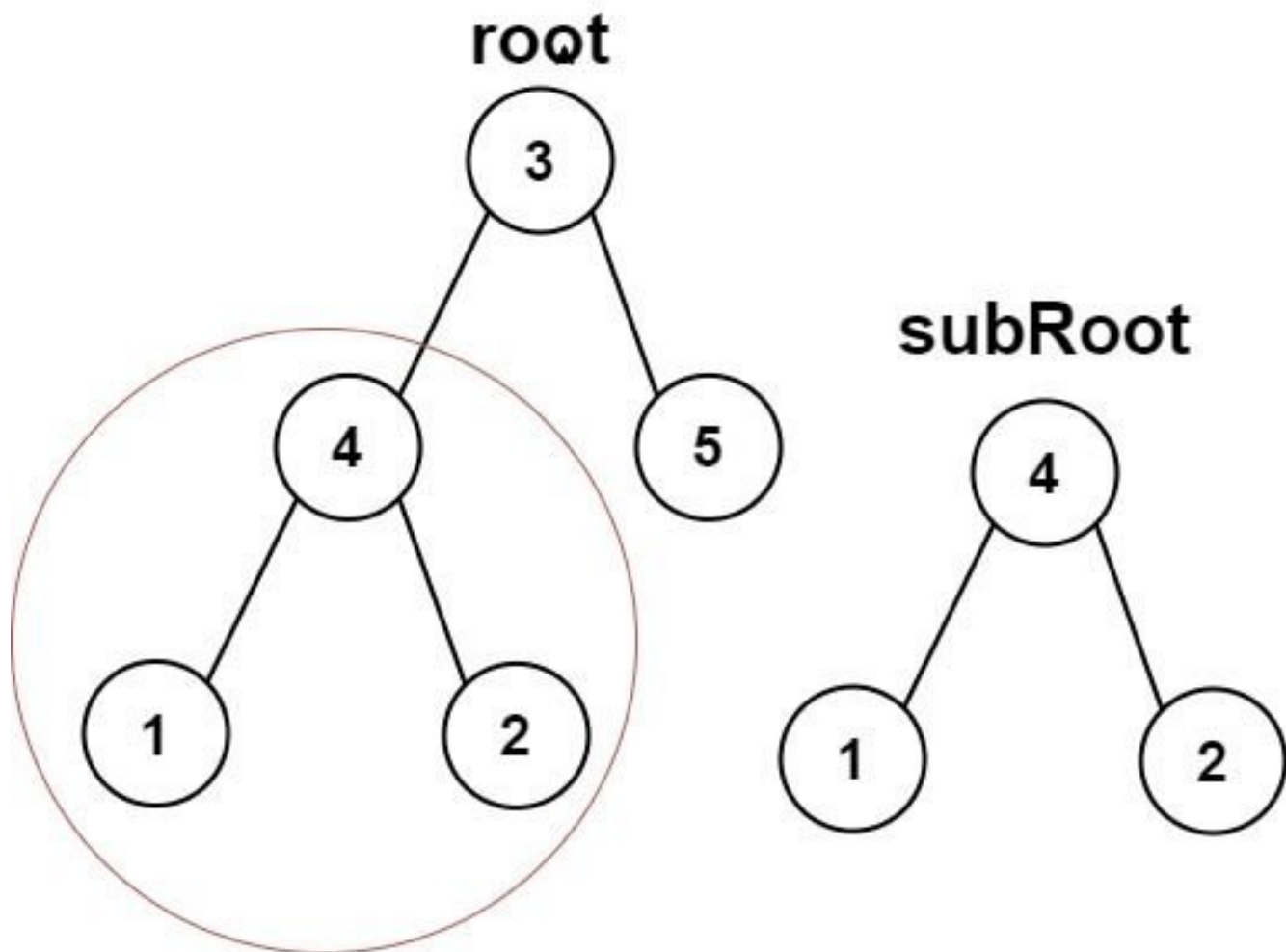
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · String Matching
Lists: Grind 169

Problem

Given the roots of two binary trees `root` and `subRoot`, return `true` if there is a subtree of `root` with the same structure and node values of `subRoot` and `false` otherwise.

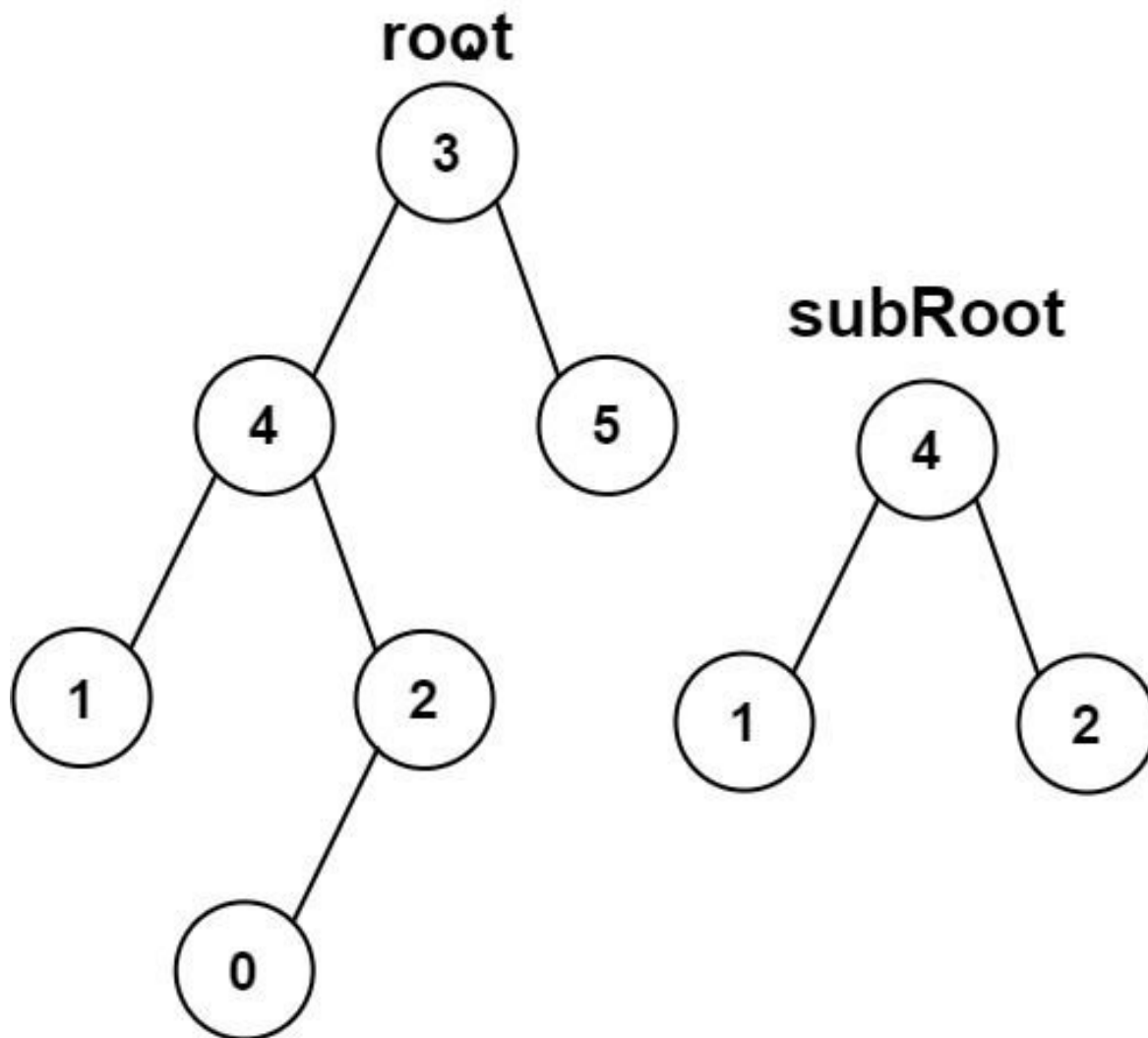
A subtree of a binary tree `tree` is a tree that consists of a node in `tree` and all of this node's descendants. The tree `tree` could also be considered as a subtree of itself.

Example 1:



Input: root = [3,4,5,1,2], subRoot = [4,1,2]
Output: true

Example 2:



Input: root = [3,4,5,1,2,null,null,null,null,0], subRoot = [4,1,2]
Output: false

Constraints:

- The number of nodes in the root tree is in the range [1, 2000].
- The number of nodes in the subRoot tree is in the range [1, 1000].
- $-104 \leq \text{root.val} \leq 104$
- $-104 \leq \text{subRoot.val} \leq 104$

Community Solutions (Python)

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def isSubtree(self, root: Optional[TreeNode], subRoot: Optional[TreeNode])
    -> bool:
        def same(p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
            if p is None or q is None:

                return p is q
            return p.val == q.val and same(p.left, q.left) and same(p.right, q
            .right)

        if root is None:
            return False
        return (
            same(root, subRoot)
            or self.isSubtree(root.left, subRoot)
            or self.isSubtree(root.right, subRoot)
        )
```

• SimplyLeet

```
# Python solution for checking if subRoot is a subtree of root.

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right

class Solution:
```

```

def isSubtree(self, root: TreeNode, subRoot: TreeNode) -> bool:
    # Helper function to check if two trees are identical.
    def isSameTree(s, t):
        # Both nodes are None, trees match.
        if not s and not t:
            return True
        # One node is None or values differ.
        if not s or not t or s.val != t.val:
            return False
        # Recurse on left and right subtrees.

        return isSameTree(s.left, t.left) and isSameTree(s.right, t.right)

    # Base case: if root is None, then no subtree exists.
    if not root:
        return False
    # Check if current subtree is identical to subRoot.
    if isSameTree(root, subRoot):
        return True
    # Recursively check left and right subtrees.
    return self.isSubtree(root.left, subRoot) or self.isSubtree(root.right
, subRoot)

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def isSubtree(self, root: TreeNode, subRoot: TreeNode) -> bool:
        def dfs(root1, root2):
            if root1 is None and root2 is None:

```

```
        return True
    if root1 is None or root2 is None:
        return False
    return (
        root1.val == root2.val
        and dfs(root1.left, root2.left)
        and dfs(root1.right, root2.right)
    )

    if root is None:

        return False
    return (
        dfs(root, subRoot)
        or self.isSubtree(root.left, subRoot)
        or self.isSubtree(root.right, subRoot)
    )
```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to visit each node in the main tree.
- At each node, check if the subtree starting from that node is identical to subRoot.
- A helper function is used to compare the structure and node values of two trees.
- Consider recursion as the primary approach to traverse and compare trees.
- Early exit when a mismatch is found can optimize average-case performance.

Space & Time

Time Complexity: $O(m * n)$ in the worst case, where m is the number of nodes in the main tree and n is the number of nodes in the subRoot tree.

Space Complexity: $O(\max(\text{depth of main tree}, \text{depth of subRoot}))$ due to recursive call stack.

Solution

We solve the problem by performing a DFS traversal on the main tree. For each visited node, a helper function checks whether the subtree starting at that node is identical to subRoot by comparing node values and recursively checking both left and right subtrees. This method leverages recursion effectively and provides early termination if mismatches occur, allowing for efficient traversal and comparison.

Round 2 · High · Medium

Round 2

High Priority · Medium

43 questions · 8 patterns

**Arrays & Hashing #36 #49 #128 #238 #271
#347**

Two Pointers #11 #15 #167

Sliding Window #3 #424 #567

Binary Search #33 #74 #153 #875 #981

Intervals #56 #57 #253 #435

Trees #98 #102 #105 #199 #230 #235 #1448

**Graphs #130 #133 #200 #207 #210 #261 #286
#323 #417 #684 #695 #994**

Advanced Graphs #743 #787 #1584

Arrays & Hashing

Round 2 · High · Medium · 6 questions

Arrays & Hashing

□ #36 Valid Sudoku

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Matrix
Lists: Grind 169

Problem

Determine if a 9 x 9 Sudoku board is valid. Only the filled cells need to be validated according to the following rules:

1. Each row must contain the digits 1–9 without repetition.
2. Each column must contain the digits 1–9 without repetition.
3. Each of the nine 3 x 3 sub-boxes of the grid must contain the digits 1–9 without repetition.

Note:

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.
- Only the filled cells need to be validated according to the mentioned rules.

Example 1:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Input: board =

```
[["5","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[","9","8",".",".",".","6","."],
["8",".",".","6",".",".","3"],
["4",".","8",".","3",".","1"],
["7",".",".","2",".",".","6"],
[","6",".",".","2","8","."],
["4","1","9",".","5"],
["8","7","9"]]
```

Example 2:


```

Input: board =
[["8","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[[".","9","8",".",".",".","6","."],
["8",".",".","6",".",".","3"],
["4",".","8",".","3",".","1"],
["7",".","2",".",".","6"],
[["6",".",".","2","8","."]]

```

Constraints:

- `board.length == 9`
- `board[i].length == 9`
- `board[i][j]` is a digit 1–9 or '.'.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def isValidSudoku(self, board: list[list[str]]) -> bool:
        seen = set()

        for i in range(9):
            for j in range(9):
                c = board[i][j]
                if c == '.':
                    continue
                if (c + '@row ' + str(i) in seen or
                    c + '@col ' + str(j) in seen or
                    c + '@box ' + str(i // 3) + str(j // 3) in seen):
                    return False
                seen.add(c + '@row ' + str(i))
                seen.add(c + '@col ' + str(j))
                seen.add(c + '@box ' + str(i // 3) + str(j // 3))

        return True

```

• LeetDoocs

```

class Solution:
    def isValidSudoku(self, board: List[List[str]]) -> bool:
        row = [[False] * 9 for _ in range(9)]
        col = [[False] * 9 for _ in range(9)]
        sub = [[False] * 9 for _ in range(9)]
        for i in range(9):
            for j in range(9):
                c = board[i][j]
                if c == '.':
                    continue

                num = int(c) - 1
                k = i // 3 * 3 + j // 3
                if row[i][num] or col[j][num] or sub[k][num]:
                    return False
                row[i][num] = True
                col[j][num] = True
                sub[k][num] = True

        return True

```

• SimplyLeet

```

# Python solution with inline comments
def isValidSudoku(board):
    # Initialize sets for rows, columns, and boxes
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]

    # Iterate over each cell in the board
    for r in range(9):
        for c in range(9):

            num = board[r][c]
            # Skip empty cells
            if num == '.':
                continue

            # Compute box index using integer division
            box_index = (r // 3) * 3 + (c // 3)
            # Check if the number is already present in the row, column, or box
            if num in rows[r] or num in cols[c] or num in boxes[box_index]:
                return False

            # Add number to row, column, and box sets

```

```

        rows[r].add(num)
        cols[c].add(num)
        boxes[box_index].add(num)

    return True

# Example usage:
if __name__ == "__main__":
    board = [
        ["5","3",".",".","7",".",".",".","."],

        ["6",".",".","1","9","5",".",".","."],
        [".","9","8",".",".",".","6","."],
        ["8",".",".","6",".",".","3"],
        ["4",".","8",".","3",".","."],
        ["7",".",".","2",".","."],
        [".","6",".","."],
        [".","4","1","9",".","5"],
        [".","8",".","7","9"]
    ]
    print(isValidSudoku(board)) # Expected output: True

```

• LC.ca

```

class Solution:
    def isValidSudoku(self, board: List[List[str]]) -> bool:
        row = [[False] * 9 for _ in range(9)]
        col = [[False] * 9 for _ in range(9)]
        sub = [[False] * 9 for _ in range(9)]
        for i in range(9):
            for j in range(9):
                c = board[i][j]
                if c == '.':
                    continue

                num = int(c) - 1
                k = i // 3 * 3 + j // 3
                if row[i][num] or col[j][num] or sub[k][num]:
                    return False
                row[i][num] = True
                col[j][num] = True
                sub[k][num] = True

        return True

```

◆ Quick Review

Key Insights

- Validate each row by checking for duplicate numbers while ignoring empty cells.
- Validate each column in the same way.
- Divide the board into nine 3x3 sub-boxes and check each for duplicate numbers.
- Use hash sets (or equivalent data structures) to keep track of seen digits for rows, columns, and boxes.
- The board size is fixed (9x9), so the overall time complexity is constant with respect to input size.

Space & Time

Time Complexity: $O(1)$ — Since the board size is fixed at 9x9.

Space Complexity: $O(1)$ — The extra space used by the sets is bounded by the fixed board size.

Solution

The solution uses three main data structures: an array of sets for rows, an array of sets for

columns, and an array of sets for 3x3 sub-boxes. For each cell, if it is not empty, check if the number is already present in the corresponding row, column, or box. If found, return false immediately. Otherwise, add the number to the respective sets. Compute the index for the 3x3 sub-box using the formula: $(\text{row_index} // 3) * 3 + (\text{col_index} // 3)$. If no duplicates are found after traversing the entire board, the board is considered valid.

Arrays & Hashing

□ #49 Group Anagrams

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Sorting
Lists: Grind 169

Problem

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form "bat".
- The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.
- The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other.

Example 2:

Input: strs = [""]

Output: [[""]]

Example 3:

Input: strs = ["a"]

Output: [["a"]]

Constraints:

- $1 \leq \text{strs.length} \leq 104$
- $0 \leq \text{strs}[i].\text{length} \leq 100$
- `strs[i]` consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def groupAnagrams(self, strs: list[str]) -> list[list[str]]:
        dict = collections.defaultdict(list)

        for str in strs:
            key = ''.join(sorted(str))
            dict[key].append(str)

        return dict.values()
```

• LeetDoocs

```
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        d = defaultdict(list)
        for s in strs:
            k = ''.join(sorted(s))
            d[k].append(s)
        return list(d.values())
```

• SimplyLeet

```
# Python solution for grouping anagrams
from collections import defaultdict

def groupAnagrams(strs):
    # Create a dictionary to hold lists of anagrams; default to an empty list
    # for new keys
    anagrams = defaultdict(list)
    # Iterate over each string in the input list
    for s in strs:
        # Sort the string to use as a key; sorted returns a list so join back
        # to string
        key = "".join(sorted(s))

        # Append the original string under the sorted key
        anagrams[key].append(s)
    # Return the grouped anagrams as a list of lists
    return list(anagrams.values())

# Example usage:
if __name__ == "__main__":
    strs = ["eat", "tea", "tan", "ate", "nat", "bat"]
    result = groupAnagrams(strs)
    print(result)
```

• LC.ca

```
'''
>>> from collections import defaultdict
>>> d = defaultdict(list)
>>> d["a"]=1
>>> d["b"]=2
>>> d
defaultdict(<class 'list'>, {'a': 1, 'b': 2})
>>> d.values()
dict_values([1, 2])
>>> list(d.values())
```



```
[1, 2]

### sorted(str) will return a list of chars
>>> a = "sdfddxyz"
>>> sorted(a)
['d', 'd', 'd', 'f', 's', 'x', 'y', 'z']
>>> "".join(sorted(a))
'dddfsxyz'
'''
```

```
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        d = defaultdict(list)
        for s in strs:
            k = "".join(sorted(s))
            d[k].append(s)
        return list(d.values())
```

◆ Quick Review

Key Insights

- Anagrams have the same characters when sorted alphabetically.
- Sorting each string provides a unique key for grouping.
- Use a hash table (dictionary) to map each sorted key to a list of its original strings.
- Finally, gather all the lists from the dictionary to form the result.

Space & Time

Time Complexity: $O(n * k \log k)$, where n is the number of strings and k is the maximum length of a string (due to sorting each string).
Space Complexity: $O(n * k)$ for storing the grouped strings in the hash table.

Solution

The primary idea is to iterate through each string, sort it to obtain a key that represents its character composition, and then group the original string under this key in a hash table. After processing all strings, the values of the hash table will be the groups of anagrams. This approach efficiently groups anagrams by leveraging sorting and a dictionary for constant-time lookups.

Arrays & Hashing

□ #128 Longest Consecutive Sequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Union Find
Lists: Grind 169 | CodeSignal

Problem

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

You must write an algorithm that runs in $O(n)$ time.

Example 1:

```
Input: nums = [100,4,200,1,3,2]
Output: 4
Explanation: The longest consecutive elements sequence is [1, 2, 3, 4].
Therefore its length is 4.
```

Example 2:

```
Input: nums = [0,3,7,2,5,8,4,6,0,1]
Output: 9
```

Example 3:

```
Input: nums = [1,0,1,2]
Output: 3
```

Constraints:

- $0 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestConsecutive(self, nums: list[int]) -> int:
        ans = 0
        seen = set(nums)

        for num in seen:
            # `num` is the start of a sequence.
            if num - 1 in seen:
                continue
            length = 0

            while num in seen:
                num += 1
                length += 1
            ans = max(ans, length)

        return ans
```

• LeetDoocs

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        s = set(nums)
        ans = 0
        d = defaultdict(int)
        for x in nums:
            y = x
            while y in s:
                s.remove(y)
                y += 1

            d[x] = d[y] + y - x
            ans = max(ans, d[x])
        return ans
```

• SimplyLeet

```
# Python solution for Longest Consecutive Sequence

def longestConsecutive(nums):
    # Create a set from the list for O(1) look-up times
    num_set = set(nums)
    longest_streak = 0

    # Iterate through each number in the set
    for num in num_set:
        # Only start counting if 'num-1' is not present in the set

        if num - 1 not in num_set:
            current_num = num
            current_streak = 1

            # Count consecutive numbers starting from current_num
            while current_num + 1 in num_set:
                current_num += 1
                current_streak += 1

            # Update the longest streak if current one is longer

            longest_streak = max(longest_streak, current_streak)

    return longest_streak

# Example usage:
print(longestConsecutive([100, 4, 200, 1, 3, 2])) # Output: 4
```

• LC.ca

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        s = set(nums)
        ans = 0
        for x in nums:
            if x - 1 not in s:
                y = x + 1
                while y in s:
                    y += 1
                ans = max(ans, y - x)
```

```
return ans
```

◆ Quick Review

Key Insights

- Utilize a set (or hash table) for constant time look-ups.
- Only start counting a sequence from a number that is the beginning (its previous number is not in the set).
- Each number is visited only once, ensuring an $O(n)$ time complexity.
- Update the maximum sequence length as you discover longer consecutive sequences.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

We first insert all the numbers into a hash set to enable $O(1)$ membership checks. For each number in the set, we check if it is the start of a sequence by ensuring that the number minus one is not in the set. If it is the beginning, we then count all consecutive numbers following

it, updating the maximum sequence length when necessary. This method guarantees that we only count each number once, satisfying the $O(n)$ time requirement.

Arrays & Hashing

□ #238 Product of Array Except Self

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Prefix Sum

Lists: Grind 169

Problem

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

The product of any prefix or suffix of `nums` is guaranteed to fit in a 32-bit integer.

You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example 1:

```
Input: nums = [1,2,3,4]
Output: [24,12,8,6]
```

Example 2:

```
Input: nums = [-1,1,0,-3,3]
Output: [0,0,9,0,0]
```

Constraints:

- $2 \leq \text{nums.length} \leq 105$
- $-30 \leq \text{nums}[i] \leq 30$
- The input is generated such that $\text{answer}[i]$ is guaranteed to fit in a 32-bit integer.

Follow up: Can you solve the problem in $O(1)$ extra space complexity? (The output array does not count as extra space for space complexity analysis.)

Community Solutions (Python)

• WalkCC

```
class Solution:
    def productExceptSelf(self, nums: list[int]) -> list[int]:
        n = len(nums)
        prefix = [1] * n # prefix product
        suffix = [1] * n # suffix product

        for i in range(1, n):
            prefix[i] = prefix[i - 1] * nums[i - 1]

        for i in reversed(range(n - 1)):
            suffix[i] = suffix[i + 1] * nums[i + 1]

        return [prefix[i] * suffix[i] for i in range(n)]
```

```

class Solution:
    def productExceptSelf(self, nums: list[int]) -> list[int]:
        n = len(nums)
        ans = [1] * n

        # Use ans as the prefix product array.
        for i in range(1, n):
            ans[i] = ans[i - 1] * nums[i - 1]

        suffix = 1 # suffix product

        for i, num in reversed(list(enumerate(nums))):
            ans[i] *= suffix
            suffix *= num

        return ans

```

• LeetCode

```

class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        ans = [0] * n
        left = right = 1
        for i, x in enumerate(nums):
            ans[i] = left
            left *= x
        for i in range(n - 1, -1, -1):
            ans[i] *= right

            right *= nums[i]
        return ans

```

• SimplyLeet

```

# Function to compute product of array except self
def productExceptSelf(nums):
    n = len(nums)
    # Initialize the output array with 1s for prefix multiplication
    answer = [1] * n

    # Compute prefix products: for each element answer[i] holds the product of
    all numbers left of i
    prefix_product = 1
    for i in range(n):
        answer[i] = prefix_product # store prefix product so far

        prefix_product *= nums[i] # update the prefix product

    # Compute suffix products and multiply with prefix products in answer array
    suffix_product = 1
    for i in range(n - 1, -1, -1):
        answer[i] *= suffix_product # multiply current answer with suffix
        product
        suffix_product *= nums[i] # update the suffix product

    return answer

# Example usage
nums = [1, 2, 3, 4]
print(productExceptSelf(nums)) # Output: [24, 12, 8, 6]

```

• LC.ca

```

class Solution: # 2 extra arrays
    def productExceptSelf(self, nums):
        if nums is None or len(nums) == 0:
            return nums

        # Product from index=0 to index=i-1
        from_left = [1] * len(nums)
        for i in range(1, len(nums)):
            from_left[i] = nums[i - 1] * from_left[i - 1]

```

```

# Product from index=n-1 to current position
from_right = [1] * len(nums)
for i in range(len(nums) - 2, -1, -1):
    from_right[i] = nums[i + 1] * from_right[i + 1]

# Calculate result
result = [0] * len(nums)
for i in range(len(nums)):
    result[i] = from_left[i] * from_right[i]

```

```

return result

```

```

#####

```

```

class Solution: # follow up, no extra space
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        ans = [0] * n
        left = right = 1
        for i, v in enumerate(nums):

```

```

            ans[i] = left
            left *= v
        for i in range(n - 1, -1, -1):
            ans[i] *= right
            right *= nums[i]
        return ans

```

```

#####

```

```

class Solution(object):
    def productExceptSelf(self, nums):
        """
        :type nums: List[int]
        :rtype: List[int]
        """
        dp = [1] * len(nums)
        for i in range(1, len(nums)):
            dp[i] = dp[i - 1] * nums[i - 1]

```

```
prod = 1
for i in reversed(range(0, len(nums))):
    dp[i] = dp[i] * prod
    prod *= nums[i]
return dp
```

◆ Quick Review

Key Insights

- The product for each index can be computed by multiplying the product of all numbers to the left and the product of all numbers to the right of that index.
- Division is not allowed, so we must compute the cumulative products explicitly.
- Create two passes: one forward pass for prefix products and one backward pass for suffix products.
- Use the output array to store the cumulative product values, achieving $O(1)$ extra space by not using additional arrays.

Space & Time

Time Complexity: $O(n)$ because we iterate through the array twice.

Space Complexity: $O(1)$ extra space (excluding the output array).

Solution

The approach uses two passes over the input array: Initialize the answer array and fill it with prefix products. For each index i , $\text{answer}[i]$ contains the product of all elements before i . Traverse the array from the end using a running suffix product. Update the answer array by multiplying the current suffix product with the prefix product already stored. This method leverages an output array to store intermediate results and uses a temporary variable for the suffix product, ensuring constant extra space.

Arrays & Hashing

★ #271 Encode and Decode Strings ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Design

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Design an algorithm to encode a list of strings to a string. The encoded string is then sent over the network and is decoded back to the original list of strings.

Machine 1 (sender) has the function:

```
string encode(vector<string> strs) {  
    // ... your code  
    return encoded_string;  
}
```

Machine 2 (receiver) has the function:

```
vector<string> decode(string s) {  
    //... your code  
    return strs;  
}
```

So Machine 1 does:

```
string encoded_string = encode(strs);
```

and Machine 2 does:

```
vector<string> strs2 = decode(encoded_string);
```

strs2 in Machine 2 should be the same as strs in Machine 1.

Implement the encode and decode methods.

You are not allowed to solve the problem using any serialize methods (such as eval).

Example 1:

```
Input: dummy_input = ["Hello","World"]
Output: ["Hello","World"]
Explanation:
Machine 1:
Codec encoder = new Codec();
String msg = encoder.encode(strs);
Machine 1 ----msg----> Machine 2
```

Example 2:

```
Input: dummy_input = [""]
Output: [""]
```

Constraints:

- $1 \leq \text{strs.length} \leq 200$
- $0 \leq \text{strs}[i].\text{length} \leq 200$
- **strs[i]** contains any possible characters out of 256 valid ASCII characters.

Follow up: Could you write a generalized algorithm to work on any possible set of characters?

Community Solutions (Python)

- WalkCC


```

class Codec:
    def encode(self, strs: list[str]) -> str:
        """Encodes a list of strings to a single string."""
        return ''.join(str(len(s)) + '/' + s for s in strs)

    def decode(self, s: str) -> list[str]:
        """Decodes a single string to a list of strings."""
        decoded = []

        i = 0

        while i < len(s):
            slash = s.find('/', i)
            length = int(s[i:slash])
            i = slash + length + 1
            decoded.append(s[slash + 1:i])

        return decoded

```

• LeetDoocs

```

class Codec:
    def encode(self, strs: List[str]) -> str:
        """Encodes a list of strings to a single string."""
        ans = []
        for s in strs:
            ans.append('{:4}'.format(len(s)) + s)
        return ''.join(ans)

    def decode(self, s: str) -> List[str]:
        """Decodes a single string to a list of strings."""

        ans = []
        i, n = 0, len(s)
        while i < n:
            size = int(s[i : i + 4])
            i += 4
            ans.append(s[i : i + size])
            i += size
        return ans

```

```
# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.decode(codec.encode(strs))
```

• SimplyLeet

```
class Codec:
    # Encodes a list of strings to a single string.
    def encode(self, strs):
        encoded = []
        # Iterate through each string in the list.
        for s in strs:
            # Append the length of the string, a '#' as delimiter, and the
            string.
            encoded.append(str(len(s)) + '#' + s)
        # Join all parts into one encoded string.
        return ''.join(encoded)
```

```
# Decodes a single string back to a list of strings.
def decode(self, s):
    decoded = []
    i = 0 # Pointer to traverse the encoded string.
    while i < len(s):
        j = i
        # Find the '#' delimiter to determine the length.
        while s[j] != '#':
            j += 1
```

```
        # Convert the substring from i to j into an integer length.
        length = int(s[i:j])
        # Extract the actual string using the obtained length.
        string = s[j+1:j+1+length]
        decoded.append(string)
        # Move the pointer to the start of the next encoded segment.
        i = j + 1 + length
    return decoded
```

```
# Example usage:
```

```
# codec = Codec()
# encoded_str = codec.encode(["Hello", "World"])
# decoded_list = codec.decode(encoded_str)
```

• LC.ca

```

'''
>>> s = 'abcde'
>>> '{:4}'.format(len(s)) + s
' 5abcde'
>>>
>>> '{:4}'.format(333)
' 333'
>>>
>>> '{:4}'.format(1234)
'1234'

>>> '{:4}'.format(12345)
'12345'

>>> '{0:4}'.format(12345)
'12345'
>>> '{1:4}'.format(12345)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: Replacement index 1 out of range for positional args tuple
>>> '{1:4}'.format(12345, 678910)

'678910'
>>> '{:4}'.format(12345, 678910)
'12345'
>>> '{0:4}'.format(12345, 678910)
'12345'
'''

class Codec: # no splitter
    def encode(self, strs: List[str]) -> str:
        """Encodes a list of strings to a single string."""
        ans = []

```

```

    for s in strs:
        ans.append('{:4}'.format(len(s)) + s)
    return ''.join(ans)

def decode(self, s: str) -> List[str]:
    """Decodes a single string to a list of strings."""
    ans = []
    i, n = 0, len(s)
    while i < n:
        size = int(s[i : i + 4]) # so here potential overflow, if larger
                                # than int-max, need to clarify assumption with interviewer

        i += 4
        ans.append(s[i : i + size]) # note: exclusive for right index
        i += size
    return ans

# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.decode(codec.encode(strs))

#####

class Codec:
    def encode(self, strs: List[str]) -> str:
        encoded = ""
        for s in strs:
            # cover for input: ["abcd", "####"]
            encoded += str(len(s)) + "#" + s.replace("#", "$$") + "#"

```

```

        return encoded

    def decode(self, s: str) -> List[str]:
        decoded = []
        i = 0
        while i < len(s):
            j = s.find("#", i)
            length = int(s[i:j])
            decoded.append(s[j+1:j+1+length].replace("$", "#"))
            i = j + 1 + length + 1

```

```

        return decoded

```

```

#####

```

```

class Codec: # wrong compared with above, for input: ["abcd", "####"]
    def encode(self, strs):
        encoded = ""
        for s in strs:
            encoded += str(len(s)) + "#" + s

```

```

        return encoded

```

```

    def decode(self, s):
        decoded = []
        i = 0
        while i < len(s):
            delimiter = s.index("#", i)
            length = int(s[i:delimiter])
            decoded.append(s[delimiter+1:delimiter+1+length])
            i = delimiter + 1 + length

```

```

        return decoded

```

◆ Quick Review

Key Insights

- Use length–prefix encoding for each string to avoid ambiguity.
- Append a delimiter (e.g., "#") after the length to separate it from the string content.
- This method ensures that even if the original strings contain special characters, including the delimiter, the decoding remains unambiguous.
- The approach handles edge cases, such as empty strings.

Space & Time

Time Complexity: $O(n)$, where n is the total number of characters across all strings.

Space Complexity: $O(n)$, required for the encoded string and the resulting decoded list.

Solution

We encode by iterating over each string and building a new string that contains the length of the string, a delimiter (e.g., "#"), and then the string itself. This way, during decoding, we can safely determine how many characters to extract by first reading until the delimiter to get the length, and then reading that number of characters. This length–prefix approach

avoids conflicts with potential characters in the strings.

Arrays & Hashing

□ #347 Top K Frequent Elements

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Divide and Conquer ·
Sorting · Heap · Bucket Sort · Counting ·
Quickselect

Lists: Denny Zhang

Problem

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order.

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`

Output: `[1,2]`

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$
- k is in the range $[1, \text{the number of unique elements in the array}]$.
- It is guaranteed that the answer is unique.

Follow up: Your algorithm's time complexity must be better than $O(n \log n)$, where n is the array's size.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        ans = []
        bucket = [[] for _ in range(len(nums) + 1)]

        for num, freq in collections.Counter(nums).items():
            bucket[freq].append(num)

        for b in reversed(bucket):
            ans += b

        if len(ans) == k:
            return ans
```

• LeetDoocs

```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        cnt = Counter(nums)
        return [x for x, _ in cnt.most_common(k)]
```

• SimplyLeet

```
# Python solution using bucket sort

def topKFrequent(nums, k):
    # Count the frequency of each element in nums
    frequency = {}
    for num in nums:
        frequency[num] = frequency.get(num, 0) + 1

    # Create buckets where index represents frequency
    n = len(nums)

    buckets = [[] for _ in range(n + 1)]
    for num, count in frequency.items():
        buckets[count].append(num)

    # Gather the k most frequent elements
    result = []
    # Iterate over buckets in reverse order (highest frequency first)
    for freq in range(n, 0, -1):
        for num in buckets[freq]:
            result.append(num)

    if len(result) == k:
        return result

# Example usage:
# print(topKFrequent([1,1,1,2,2,3], 2)) -> Output: [1, 2]
```

• LC.ca

```
'''
>>> nums = ["a", "a", "b", "c", "c"]

>>> cnt = Counter(nums)
>>> cnt
Counter({'a': 2, 'c': 2, 'b': 1})

# default to keys
>>> sorted_freqs = sorted(cnt, key=lambda x: (-cnt[x], x))
>>> sorted_freqs
```

```

['a', 'c', 'b']
'''
from collections import Counter

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        # Count the frequency of each element in the list
        cnt = Counter(nums)

        # Sort the elements by frequency in decreasing order

        # diff from below solution: sorted(cnt), not sorted(cnt.items())
        # more in https://leetcode.ca/2017-10-22-692-Top-K-Frequent-Words/
        sorted_freqs = sorted(cnt, key=lambda x: (-cnt[x], x))

        return sorted_freqs[:k]

#####

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:

        # Count the frequency of each element in the list
        freqs = Counter(nums)

        # Sort the elements by frequency in decreasing order
        sorted_freqs = sorted(freqs.items(), key=lambda x: x[1], reverse=True)
        # also passing 0J: sorted_freqs = sorted(freqs.items(), key=lambda x:
        -x[1])

        # Take the top k elements
        top_k = [num for num, freq in sorted_freqs[:k]]

```

```

        return top_k

#####

'''
>>> from heapq import heappush
>>> h = []
>>> heappush(h, (3,1))
>>> heappush(h, (1,1))
>>> heappush(h, (2,1))

>>> h
[(1, 1), (3, 1), (2, 1)]

>>> from heapq import heappop
>>> heappop(h)
(1, 1)
>>> heappop(h)
(2, 1)
>>> heappop(h)
(3, 1)

'''

from collections import Counter

class Solution: # heap
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        cnt = Counter(nums)
        hp = []
        for num, freq in cnt.items():
            heappush(hp, (freq, num)) # freq first, default sort by 1st element

            if len(hp) > k:
                heappop(hp)
        return [v[1] for v in hp]

#####

from typing import List
import random
# O(n log(n)) in the average case
# O(n^2) worst case

```

```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        freq_map = {}
        for num in nums:
            freq_map[num] = freq_map.get(num, 0) + 1

        freq_list = list(freq_map.items())
        self.quick_select(freq_list, 0, len(freq_list) - 1, k)

        return [num for num, freq in freq_list[:k]]

    def quick_select(self, freq_list, start, end, k):
        if start == end:
            return

        pivot_idx = random.randint(start, end)
        pivot_freq = freq_list[pivot_idx][1]

        left = start
        right = end

        while left <= right:
            while freq_list[left][1] > pivot_freq:
                left += 1
            while freq_list[right][1] < pivot_freq:
                right -= 1

            if left <= right:
                freq_list[left], freq_list[right] = freq_list[right], freq_list[
left]
                left += 1
```

```

        right -= 1

    if k <= right:
        self.quick_select(freq_list, start, right, k)
    elif k >= left:
        self.quick_select(freq_list, left, end, k)

```

```
#####
```

```
'''
```

```

>>> x = [1, 2, 3]
>>> x.append([4, 5])
>>> print(x)
[1, 2, 3, [4, 5]]

```

```

>>> x = [1, 2, 3]
>>> x.extend([4, 5])

```

```

>>> print(x)
[1, 2, 3, 4, 5]
'''

```

```

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        freq = Counter(nums)

        # not always filled, max possible frequency is len(nums) [1,1,1,1,...]
        buckets = [[] for _ in range(len(nums) + 1)]
        for num, freq in freq.items():

```

```

            buckets[freq].append(num)

        res = []
        for bucket in reversed(buckets):
            if bucket:
                res.extend(bucket)
                if len(res) >= k:
                    break

        return res[:k]

```

```
#####

class Solution(object):
    def topKFrequent(self, nums, k):
        """
        :type nums: List[int]
        :type k: int
        :rtype: List[int]
        """

        d = {}
        res = []
        ans = []
        buckets = [[] for _ in range(len(nums) + 1)]

        for num in nums:
            d[num] = d.get(num, 0) + 1

        for key in d:
            res.append((d[key], key))

        for t in res:
            freq, key = t
            buckets[freq].append(key)

        buckets.reverse()

        for item in buckets:
            if item and k > 0:
                while item and k > 0:

                    ans.append(item.pop())
                    k -= 1
            if k == 0:
                return ans

        return ans
```

◆ Quick Review

Key Insights

- Count the frequency of each element using a hash table.
- Use bucket sort to achieve linear time complexity by grouping numbers by their frequency.
- Traverse the frequency buckets from highest to lowest to collect the k most frequent elements.
- Alternative approaches include using heaps or Quickselect, but bucket sort meets the follow-up requirement of better than $O(n \log n)$ time complexity.

Space & Time

Time Complexity: $O(n)$ where n is the length of the array (bucket sort and frequency counting are linear).

Space Complexity: $O(n)$ for storing the counts and buckets.

Solution

The solution starts by counting the frequency of each element with a hash table (or dictionary). Then, we create an array of buckets where the index represents the

frequency. Each bucket holds the numbers that appear with that frequency. Finally, we iterate over the buckets in reverse order (from high frequency to low) and collect elements until we have k of them. This bucket sort approach avoids the higher complexity of sorting a list of frequencies and guarantees linear time performance.

Two Pointers

Round 2 · High · Medium · 3 questions

Two Pointers

□ #11 Container With Most Water

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Greedy
Lists: Grind 169

Problem

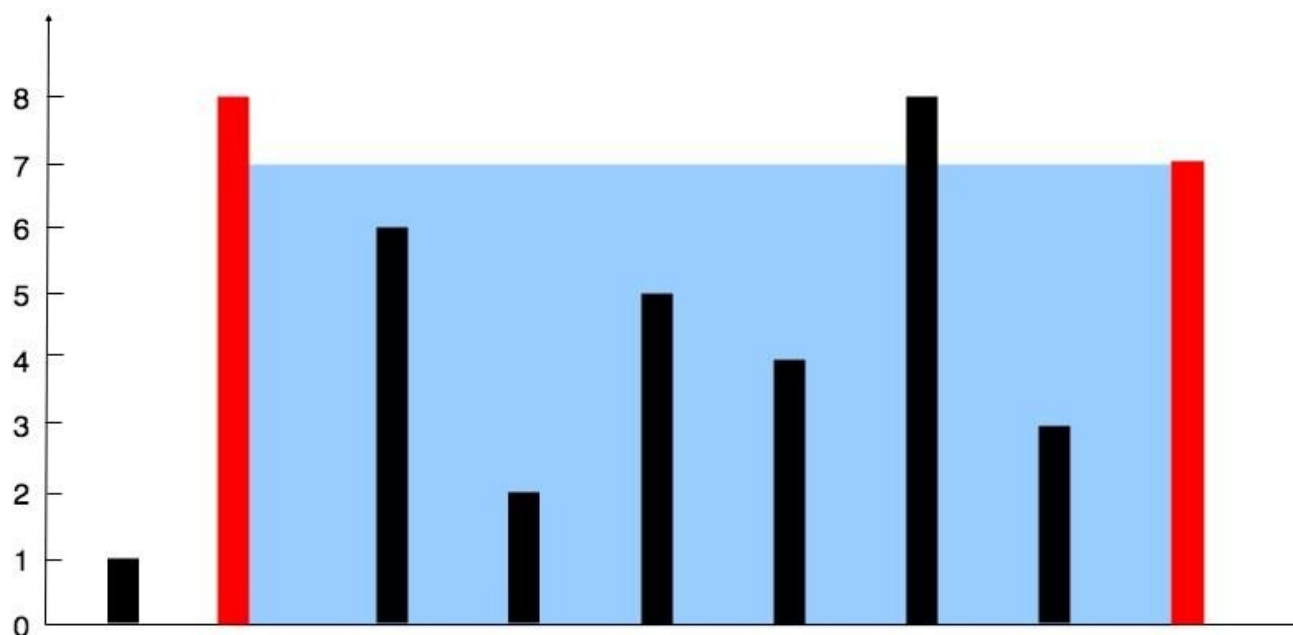
You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the `x`-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Example 1:



Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

Example 2:

Input: height = [1,1]

Output: 1

Constraints:

- $n == \text{height.length}$
- $2 \leq n \leq 105$
- $0 \leq \text{height}[i] \leq 104$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def maxArea(self, height: list[int]) -> int:
        ans = 0
        l = 0
        r = len(height) - 1

        while l < r:
            minHeight = min(height[l], height[r])
            ans = max(ans, minHeight * (r - l))
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1

        return ans
```

• LeetCode

```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        l, r = 0, len(height) - 1
        ans = 0
        while l < r:
            t = min(height[l], height[r]) * (r - l)
            ans = max(ans, t)
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1
        return ans
```

• SimplyLeet

```
# Function to calculate the maximum water container area
def maxArea(height):
    left, right = 0, len(height) - 1 # Initialize two pointers
    max_water = 0 # Variable to store the maximum area

    # Loop until pointers meet
    while left < right:
        # Calculate current area based on smaller height and distance between
        # pointers
        current_area = min(height[left], height[right]) * (right - left)
        max_water = max(max_water, current_area) # Update maximum area if
        # current is larger

        # Move the pointer corresponding to the smaller height
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return max_water

# Test the function with an example
if __name__ == "__main__":
    input_heights = [1,8,6,2,5,4,8,3,7]
    print(maxArea(input_heights)) # Expected output: 49
```

• LC.ca

```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        i, j = 0, len(height) - 1
        ans = 0
        while i < j:
            t = (j - i) * min(height[i], height[j])
            ans = max(ans, t)
            if height[i] < height[j]:
                i += 1
            else:
                j -= 1
        return ans
```

◆ Quick Review

Key Insights

- The container's area is determined by the distance between two lines multiplied by the shorter line's height.
- A brute-force approach checking all possible pairs would result in $O(n^2)$ time complexity, which is inefficient for large inputs.
- A two-pointer approach allows us to solve the problem in $O(n)$ time by initializing pointers at both ends and gradually moving them inward.
- By always moving the pointer corresponding to the shorter line, we are more likely to find a taller line that could potentially form a container with a larger capacity.

Space & Time

Time Complexity: $O(n)$ – The algorithm makes a single pass through the array with two pointers.

Space Complexity: $O(1)$ – Only a few extra variables are used, regardless of the input size.

Solution

The optimal approach uses a two–pointer strategy: Start with two pointers at each end of the array. Calculate the area formed between the two lines at these pointers. Update the maximum area if the calculated value is larger. Move the pointer pointing to the shorter line inward, as this is the only possible way to potentially increase the height of the shorter line and thus the area. Continue until the two pointers meet. This method effectively reduces the number of comparisons and ensures that every possible pair that can yield a larger area is considered.

Two Pointers

□ #15 3Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Grind 169

Problem

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that $i \neq j$, $i \neq k$, and $j \neq k$, and $nums[i] + nums[j] + nums[k] == 0$.

Notice that the solution set must not contain duplicate triplets.

Example 1:

Input: `nums = [-1,0,1,2,-1,-4]`

Output: `[[-1,-1,2],[-1,0,1]]`

Explanation:

`nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.`

`nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.`

`nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.`

The distinct triplets are `[-1,0,1]` and `[-1,-1,2]`.

Notice that the order of the output and the order of the triplets does not matter.

Example 2:

Input: `nums = [0,1,1]`

Output: `[]`

Explanation: The only possible triplet does not sum up to 0.

Example 3:

Input: `nums = [0,0,0]`

Output: `[[0,0,0]]`

Explanation: The only possible triplet sums up to 0.

Constraints:

- $3 \leq \text{nums.length} \leq 3000$
- $-105 \leq \text{nums}[i] \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def threeSum(self, nums: list[int]) -> list[list[int]]:
        if len(nums) < 3:
            return []

        ans = []

        nums.sort()

        for i in range(len(nums) - 2):

            if i > 0 and nums[i] == nums[i - 1]:
                continue
            # Choose nums[i] as the first number in the triplet, then search the
            # remaining numbers in [i + 1, n - 1].
            l = i + 1
            r = len(nums) - 1
            while l < r:
                summ = nums[i] + nums[l] + nums[r]
                if summ == 0:
                    ans.append((nums[i], nums[l], nums[r]))
```

```

    l += 1
    r -= 1
    while nums[l] == nums[l - 1] and l < r:
        l += 1
    while nums[r] == nums[r + 1] and l < r:
        r -= 1
    elif summ < 0:
        l += 1
    else:
        r -= 1

```

```

return ans

```

• LeetDoocs

```

class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        n = len(nums)
        ans = []
        for i in range(n - 2):
            if nums[i] > 0:
                break
            if i and nums[i] == nums[i - 1]:
                continue

```

```

            j, k = i + 1, n - 1
            while j < k:
                x = nums[i] + nums[j] + nums[k]
                if x < 0:
                    j += 1
                elif x > 0:
                    k -= 1
                else:
                    ans.append([nums[i], nums[j], nums[k]])
                    j, k = j + 1, k - 1

```

```

                    while j < k and nums[j] == nums[j - 1]:
                        j += 1
                    while j < k and nums[k] == nums[k + 1]:
                        k -= 1
            return ans

```

• SimplyLeet

```
# Function to find all unique triplets that sum to zero.
def threeSum(nums):
    # Sort the array to facilitate finding duplicates and using two pointers.
    nums.sort()
    result = []
    n = len(nums)

    # Iterate through the array, fixing one element at a time.
    for i in range(n - 2):
        # Skip duplicate elements for the first element.

        if i > 0 and nums[i] == nums[i - 1]:
            continue

        left, right = i + 1, n - 1
        while left < right:
            current_sum = nums[i] + nums[left] + nums[right]
            # Check if the triplet adds up to zero.
            if current_sum == 0:
                result.append([nums[i], nums[left], nums[right]])
                # Skip duplicates for the second element.

                while left < right and nums[left] == nums[left + 1]:
                    left += 1
                # Skip duplicates for the third element.
                while left < right and nums[right] == nums[right - 1]:
                    right -= 1
                left += 1
                right -= 1
            elif current_sum < 0:
                left += 1
            else:
                right -= 1

        right -= 1

    return result

# Example usage:
#print(threeSum([-1,0,1,2,-1,-4]))
```

• LC.ca

```
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        n = len(nums)
        ans = []
        for i in range(n - 2):
            if nums[i] > 0:
                break
            if i and nums[i] == nums[i - 1]:
                continue

            j, k = i + 1, n - 1
            while j < k:
                x = nums[i] + nums[j] + nums[k]
                if x < 0:
                    j += 1
                elif x > 0:
                    k -= 1
                else:
                    ans.append([nums[i], nums[j], nums[k]])
                    j, k = j + 1, k - 1

                    while j < k and nums[j] == nums[j - 1]:
                        j += 1
                    while j < k and nums[k] == nums[k + 1]:
                        k -= 1

        return ans
```

◆ Quick Review

Key Insights

- Sort the array to efficiently skip duplicates and facilitate the two pointers approach.
- Use a fixed pointer for the first element, and then use two additional pointers (left and right) to find pairs that sum to the negation of

the fixed element.

- Skip duplicate elements to ensure unique triplets.

Space & Time

Time Complexity: $O(n^2)$ where n is the number of elements in the array.

Space Complexity: $O(1)$ extra space (ignoring the space required for the output).

Solution

We first sort the array to bring duplicates together and allow the two-pointer technique to work efficiently. Then, iterate over the array and for each element, use two pointers (one at the beginning right after the current element and one at the end of the array) to find a pair that sums with the current element to zero. Skip duplicates for the current element and within the inner loop to avoid duplicate triplets. This ensures that each valid triplet is unique and the overall runtime remains efficient.

Two Pointers

□ #167 Two Sum II – Input Array Is Sorted

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search
Lists: Denny Zhang

Problem

Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be numbers[index1] and numbers[index2] where $1 \leq \text{index1} < \text{index2} \leq \text{numbers.length}$.

Return the indices of the two numbers index1 and index2, each incremented by one, as an integer array [index1, index2] of length 2.

The tests are generated such that there is exactly one solution. You may not use the same element twice.

Your solution must use only constant extra space.

Example 1:

Input: numbers = [2,7,11,15], target = 9

Output: [1,2]

Explanation: The sum of 2 and 7 is 9. Therefore, index1 = 1, index2 = 2. We return [1, 2].

Example 2:

Input: numbers = [2,3,4], target = 6

Output: [1,3]

Explanation: The sum of 2 and 4 is 6. Therefore index1 = 1, index2 = 3. We return [1, 3].

Example 3:

Input: numbers = [-1,0], target = -1

Output: [1,2]

Explanation: The sum of -1 and 0 is -1. Therefore index1 = 1, index2 = 2. We return [1, 2].

Constraints:

- $2 \leq \text{numbers.length} \leq 3 * 10^4$
- $-1000 \leq \text{numbers}[i] \leq 1000$
- numbers is sorted in non-decreasing order.
- $-1000 \leq \text{target} \leq 1000$
- The tests are generated such that there is exactly one solution.

Community Solutions (Python)

- WalkCC


```

class Solution:
    def twoSum(self, numbers: list[int], target: int) -> list[int]:
        l = 0
        r = len(numbers) - 1

        while l < r:
            summ = numbers[l] + numbers[r]
            if summ == target:
                return [l + 1, r + 1]
            if summ < target:
                l += 1
            else:
                r -= 1

```

• LeetCode

```

class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        n = len(numbers)
        for i in range(n - 1):
            x = target - numbers[i]
            j = bisect_left(numbers, x, lo=i + 1)
            if j < n and numbers[j] == x:
                return [i + 1, j + 1]

```

```

class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        i, j = 0, len(numbers) - 1
        while i < j:
            x = numbers[i] + numbers[j]
            if x == target:
                return [i + 1, j + 1]
            if x < target:
                i += 1
            else:
                j -= 1

```

• SimplyLeet

```
# Python solution for Two Sum II – Input Array Is Sorted
def twoSum(numbers, target):
    # Initialize two pointers at the beginning and end of the array
    left = 0
    right = len(numbers) - 1

    # Continue until the two pointers meet
    while left < right:
        # Calculate the sum of the two pointed values
        current_sum = numbers[left] + numbers[right]

        # If the sum matches the target, return the 1-indexed results
        if current_sum == target:
            return [left + 1, right + 1]
        # If the current sum is less than the target, move the left pointer to
        # the right
        elif current_sum < target:
            left += 1
        # Otherwise, move the right pointer to the left since the current sum
        # is too high
        else:
            right -= 1

# Example call
print(twoSum([2, 7, 11, 15], 9)) # Output: [1, 2]
```

• LC.ca

```
class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        i, j = 0, len(numbers) - 1
        while i < j:
            x = numbers[i] + numbers[j]
            if x == target:
                return [i + 1, j + 1]
            if x < target:
                i += 1
            else:
                j -= 1
```

◆ Quick Review

Key Insights

- The array is sorted in non-decreasing order, which makes the two pointers technique very effective.
- Using two pointers (one at the start and one at the end) allows checking pairs and adjusting the pointers based on the sum.
- Since the problem guarantees exactly one solution and requires constant space, using two pointers satisfies both constraints.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in the array, because in the worst-case scenario each element is visited at most once.

Space Complexity: $O(1)$, as no additional data structure is used that scales with the input size.

Solution

We use the two pointers approach: Initialize two pointers, left at the beginning (index 0) and right at the end (last index) of the array.

While left is less than right, compute the sum of the elements at the two pointers. If the sum equals the target, return the indices as [left+1, right+1] (to adjust for the 1-indexed array). If the sum is less than the target, move the left pointer to the right to increase the sum. If the sum is greater than the target, move the right pointer to the left to decrease the sum. This approach leverages the sorted property and uses constant extra space.

Sliding Window

Round 2 · High · Medium · 3 questions

Sliding Window

□ #3 Longest Substring Without Repeating Characters

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

Given a string *s*, find the length of the longest substring without duplicate characters.

Example 1:

Input: *s* = "abcabcbb"

Output: 3

Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: *s* = "bbbbbb"

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: *s* = "pwwkew"

Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints:

- $0 \leq s.length \leq 5 * 10^4$

- s consists of English letters, digits, symbols and spaces.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        ans = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            while count[c] > 1:
                count[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        ans = 0
        # The substring s[j + 1..i] has no repeating characters.
        j = -1
        # lastSeen[c] := the index of the last time c appeared
        lastSeen = {}

        for i, c in enumerate(s):
            # Update j to lastSeen[c], so the window must start from j + 1.

            j = max(j, lastSeen.get(c, -1))
            ans = max(ans, i - j)
            lastSeen[c] = i

        return ans
```

• LeetDoocs

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        cnt = Counter()
        ans = l = 0
        for r, c in enumerate(s):
            cnt[c] += 1
            while cnt[c] > 1:
                cnt[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

• SimplyLeet

```
# Function to find the length of the longest substring without repeating
characters
def length_of_longest_substring(s):
    # Dictionary to store the most recent index of each character
    char_index = {}
    # Initialize starting index of current window and max length
    left = 0
    max_length = 0

    # Iterate over the string with the right pointer
    for right, char in enumerate(s):
        # If char is in dictionary and its index is within the current window
        if char in char_index and char_index[char] >= left:
            # Move the left pointer to one position after the last occurrence
            # of char
            left = char_index[char] + 1
        # Update the latest index of the char
        char_index[char] = right
        # Update max_length with the current window size if larger
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
# print(length_of_longest_substring("abcabcbb")) # Expected output: 3
```


• LC.ca

```
class Solution:
    def lengthOfLongestSubstring(self, s): # map for index
        d = {} # value => its index
        i = 0
        ans = 0
        for j, c in enumerate(s):
            if c in d:
                # must max check i, example "abba"
                i = max(i, d[c] + 1)
            d[c] = j

        ans = max(ans, j - i + 1)
        return ans
```

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:

        if not s:
            return 0
```

```
l = 0 # left
r = 0 # right

result = 0
isFoundInWindow = [False] * 256

while r < len(s):

    while isFoundInWindow[ord(s[r])]:
        isFoundInWindow[ord(s[l])] = False
```

```

        l += 1

        isFoundInWindow[ord(s[r])] = True

        # check before shrink
        result = max(result, r - l + 1)

        r += 1

    return result

```

```

class Solution: # extra space for set()
    def lengthOfLongestSubstring(self, s: str) -> int:

        d = collections.defaultdict(int)
        start = 0
        ans = 0
        for i, c in enumerate(s):
            # shrink
            while d[c] > 0:

```

```

                d[s[start]] -= 1 # not s[start], not start
                start += 1
            ans = max(ans, i - start + 1)
            d[c] = 1
        return ans

```

```
#####
```

```

class Solution(object):
    def _lengthOfLongestSubstring(self, s): # no extra data structure

```

```

        """
        :type s: str
        :rtype: int
        """

        d = collections.defaultdict(int)
        l = ans = 0
        for i, c in enumerate(s):
            while l > 0 and d[c] > 0:
                d[s[i - l]] -= 1
                l -= 1

```

```
d[c] += 1
l += 1
ans = max(ans, l)
return ans

#####

class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        ss = set()

        i = ans = 0
        for j, c in enumerate(s):
            while c in ss:
                ss.remove(s[i])
                i += 1
            ss.add(c)
            ans = max(ans, j - i + 1)
        return ans
```

◆ Quick Review

Key Insights

- Use the sliding window technique to maintain a window of unique characters.
- Use a hash table (or dictionary) to quickly check for duplicate characters and update window boundaries.
- The window is expanded by moving a pointer (right) and contracted by moving another pointer (left) when duplicates are encountered.

- Time complexity can be maintained at $O(n)$ by ensuring each character is processed a limited number of times.

Space & Time

Time Complexity: $O(n)$, where n is the length of the input string.

Space Complexity: $O(\min(n, m))$, where m is the size of the character set (constant for fixed character sets).

Solution

The solution uses a sliding window technique with two pointers, left and right, to keep track of the current substring without repeating characters. A hash table (dictionary or map) is used to store the most recent index of each character encountered. As the right pointer moves forward, if a duplicate character is found within the current window, the left pointer is advanced to the position right after where the duplicate was last seen. This ensures that the window always contains unique characters. The maximum length is updated throughout the process.

Sliding Window

□ #424 Longest Repeating Character Replacement

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

You are given a string s and an integer k . You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most k times.

Return the length of the longest substring containing the same letter you can get after performing the above operations.

Example 1:

Input: $s = \text{"ABAB"}, k = 2$
Output: 4
Explanation: Replace the two 'A's with two 'B's or vice versa.

Example 2:

Input: $s = \text{"AABABBA"}, k = 1$
Output: 4
Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA". The substring "BBBB" has the longest repeating letters, which is 4. There may exists other ways to achieve this answer too.

Constraints:

- $1 \leq s.length \leq 105$
- s consists of only uppercase English letters.
- $0 \leq k \leq s.length$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        ans = 0
        maxCount = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            maxCount = max(maxCount, count[c])

            while maxCount + k < r - l + 1:
                count[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans

class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        maxCount = 0
        count = collections.Counter()

        # l and r track the maximum window instead of the valid window.
        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            maxCount = max(maxCount, count[c])
```

```

while maxCount + k < r - l + 1:
    count[s[l]] -= 1
    l += 1

return r - l + 1

```

• LeetCode

```

class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        cnt = Counter()
        l = mx = 0
        for r, c in enumerate(s):
            cnt[c] += 1
            mx = max(mx, cnt[c])
            if r - l + 1 - mx > k:
                cnt[s[l]] -= 1
                l += 1

        return len(s) - l

```

• SimplyLeet

```

# Python solution using the sliding window technique
def characterReplacement(s: str, k: int) -> int:
    # Dictionary to store frequency of characters in the current window
    char_counts = {}
    max_freq = 0 # Track the maximum frequency of a single character in the window
    max_length = 0
    left = 0 # Left pointer of the sliding window

    # Expand the sliding window with the right pointer
    for right in range(len(s)):

```

```

# Count the current character
char_counts[s[right]] = char_counts.get(s[right], 0) + 1
# Update the max frequency in the current window
max_freq = max(max_freq, char_counts[s[right]])

# Check if the number of replacements needed exceeds k
# (current window size) - (count of most frequent character) > k
if (right - left + 1) - max_freq > k:
    # If condition fails, shrink the window by moving left pointer
    char_counts[s[left]] -= 1

    left += 1

# Update the maximum length found so far
max_length = max(max_length, right - left + 1)

return max_length

# Example usage:
# print(characterReplacement("AABABBA", 1)) # Expected output: 4

```

• LC.ca

```

class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        counter = [0] * 26
        i = j = maxCnt = 0
        while i < len(s):
            counter[ord(s[i]) - ord('A')] += 1
            maxCnt = max(maxCnt, counter[ord(s[i]) - ord('A')])
            if i - j + 1 > maxCnt + k:
                counter[ord(s[j]) - ord('A')] -= 1
                j += 1

            i += 1
        return i - j

```

◆ Quick Review

Key Insights

- Use a Sliding Window algorithm to keep track of a contiguous substring.
- Maintain a frequency count of letters in the current window.
- Keep track of the count of the most frequent character in the window.
- If the current window size minus the maximum frequency is greater than k , shrink the window.
- The maximum window size seen during the process is the answer.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string, since the window traverses the string in linear time.

Space Complexity: $O(1)$ because the frequency count for letters (A–Z) has a constant size.

Solution

The problem is solved using the sliding window technique. We initialize two pointers representing the left and right bounds of the current window. As we expand the window by moving the right pointer, we update the frequency count of the letter at the right

pointer and track the letter that appears most frequently. The key observation is that the number of characters that need to be replaced in the current window is the window's size minus the count of the most frequently occurring letter. If this number exceeds k , we increment the left pointer to shrink the window until the condition is satisfied again. The maximum size of the window during the process gives the answer.

Sliding Window

□ #567 Permutation in String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Two Pointers · String · Sliding Window

Lists: NeetCode 150

Problem

Given two strings $s1$ and $s2$, return true if $s2$ contains a permutation of $s1$, or false otherwise. In other words, return true if one of $s1$'s permutations is the substring of $s2$.

Example 1:

```
Input: s1 = "ab", s2 = "eidbaooo"  
Output: true  
Explanation: s2 contains one permutation of s1 ("ba").
```

Example 2:

```
Input: s1 = "ab", s2 = "eidboaoo"  
Output: false
```

Constraints:

- $1 \leq s1.length, s2.length \leq 104$
- $s1$ and $s2$ consist of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        count = collections.Counter(s1)
        required = len(s1)

        for r, c in enumerate(s2):
            count[c] -= 1
            if count[c] >= 0:
                required -= 1
            if r >= len(s1): # The window is oversized.

                count[s2[r - len(s1)]] += 1
                if count[s2[r - len(s1)]] > 0:
                    required += 1
            if required == 0:
                return True

        return False
```

• LeetDoocs

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        cnt = Counter(s1)
        need = len(cnt)
        m = len(s1)
        for i, c in enumerate(s2):
            cnt[c] -= 1
            if cnt[c] == 0:
                need -= 1
            if i >= m:
                cnt[s2[i - m]] += 1
                if cnt[s2[i - m]] == 1:
                    need += 1
            if need == 0:
                return True
        return False
```

• SimplyLeet

```
# Python solution with line-by-line comments

def checkInclusion(s1: str, s2: str) -> bool:
    # Get lengths of the input strings
    len1, len2 = len(s1), len(s2)
    # Edge case: s1 cannot be a substring if longer than s2
    if len1 > len2:
        return False

    # Initialize frequency arrays for s1 and for the first window of s2

    s1_count = [0] * 26
    window_count = [0] * 26

    # Build frequency counts for s1 and the initial window in s2
    for i in range(len1):
        s1_count[ord(s1[i]) - ord('a')] += 1
        window_count[ord(s2[i]) - ord('a')] += 1

    # Check if initial window is a permutation of s1
    if s1_count == window_count:

        return True

    # Slide the window across s2, updating the counts on the fly
    for i in range(len1, len2):
        # Add the new character from the right end of the window
        window_count[ord(s2[i]) - ord('a')] += 1
        # Remove the character that is no longer in the window
        window_count[ord(s2[i - len1]) - ord('a')] -= 1

        # Check if current window matches s1's frequency distribution

        if window_count == s1_count:
            return True

    return False

# Example usage:
# print(checkInclusion("ab", "eidbaooo")) # Expected output: True
```

◆ Quick Review

Key Insights

- Use a sliding window approach to examine substrings of $s2$ of length equal to $s1$.
- Maintain frequency counts of characters for both $s1$ and the current window in $s2$.
- Compare the frequency counts to determine if a valid permutation exists.
- Increment the window by adding a new character and removing the leftmost one, keeping the operation efficient.

Space & Time

Time Complexity: $O(n)$ where n is the length of $s2$ (each character is processed a constant number of times).

Space Complexity: $O(1)$ since the frequency count arrays have a fixed size (26 for lowercase English letters).

Solution

The solution uses two fixed-length arrays (of size 26) to store character frequencies for $s1$ and the current window in $s2$. First, initialize the frequency arrays by processing $s1$ and the

first window of s2. Then, slide the window one character at a time, updating the counts by incrementing the count for the new character and decrementing the count for the character that is no longer in the window. After each move, compare both frequency arrays; if they match, s2 contains a permutation of s1. This technique avoids repeatedly computing the frequency distribution from scratch, making the algorithm efficient.

Binary Search

Round 2 · High · Medium · 5 questions

Binary Search

□ #33 Search in Rotated Sorted Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search

Lists: Grind 169

Problem

There is an integer array `nums` sorted in ascending order (with distinct values).

Prior to being passed to your function, `nums` is possibly left rotated at an unknown index `k` ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (0-indexed).

For example, `[0,1,2,4,5,6,7]` might be left rotated by 3 indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` after the possible rotation and an integer `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4

Example 2:

Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1

Example 3:

Input: nums = [1], target = 0
Output: -1

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $-104 \leq \text{nums}[i] \leq 104$
- All values of nums are unique.
- nums is an ascending array that is possibly rotated.
- $-104 \leq \text{target} \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def search(self, nums: list[int], target: int) -> int:
        l = 0
        r = len(nums) - 1

        while l <= r:
            m = (l + r) // 2
            if nums[m] == target:
                return m
            if nums[l] <= nums[m]: # nums[l..m] are sorted.
```

```

        if nums[l] <= target < nums[m]:
            r = m - 1
        else:
            l = m + 1
    else: # nums[m..n - 1] are sorted.
        if nums[m] < target <= nums[r]:
            l = m + 1
        else:
            r = m - 1

return -1

```

• LeetCode

```

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        n = len(nums)
        left, right = 0, n - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[mid] <= target:
                if nums[mid] <= target <= nums[right]:
                    right = mid
                else:
                    left = mid + 1
            else:
                if nums[mid] < target <= nums[n - 1]:
                    left = mid + 1
                else:
                    right = mid
        return left if nums[left] == target else -1

```

• SimplyLeet

```

# Define a function to search for the target in a rotated sorted array.
def search(nums, target):
    # Initialize pointers for binary search.
    left, right = 0, len(nums) - 1

    # Loop until the search space is exhausted.
    while left <= right:
        # Compute the middle index.
        mid = (left + right) // 2

        # If the mid element is the target, return its index.
        if nums[mid] == target:
            return mid

        # Check if the left part of the array is sorted.
        if nums[left] <= nums[mid]:
            # If the target is within the sorted left half, adjust the right
            # pointer.
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                # Otherwise, search the right half.
                left = mid + 1
        else:
            # The right half must be sorted.
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1

        # If the target is not found, return -1.

    return -1

# Example usage:
# result = search([4,5,6,7,0,1,2], 0)
# print(result) # Expected output: 4

```

 • LC.ca

```
# below 2 solutions, diff is while condition: left ('<' or '<=') right
```

```
# I like this better
```

```
class Solution:
```

```
    def search(self, nums: List[int], target: int) -> int:
```

```
        n = len(nums)
```

```
        left, right = 0, n - 1
```

```
        while left <= right:
```

```
            mid = (left + right) >> 1
```

```
            if nums[mid] == target:
```

```
                return mid
```

```
            elif nums[0] <= nums[mid]: # left half sorted
```

```
                if nums[0] <= target <= nums[mid]:
```

```
                    right = mid - 1
```

```
            else:
```

```
                left = mid + 1
```

```
            else: # right half sorted
```

```
                if nums[mid] < target <= nums[n - 1]:
```

```
                    left = mid + 1
```

```
            else:
```

```
                right = mid - 1
```

```
        return -1
```

```
#####
```

```
class Solution:
```

```
    def search(self, nums: List[int], target: int) -> int:
```

```
        n = len(nums)
```

```
        left, right = 0, n - 1
```

```
        while left < right:
```

```
            mid = (left + right) >> 1
```

```
            if nums[0] <= nums[mid]: # left half sorted
```

```
                if nums[0] <= target <= nums[mid]:
```

```
                    right = mid
```

```
            else:
```

```
                left = mid + 1
```

```
            else: # right half sorted
```

```
                if nums[mid] < target <= nums[n - 1]:
```

```
                    left = mid + 1
```

```
            else:
```

```
        right = mid
    return left if nums[left] == target else -1
```

◆ Quick Review

Key Insights

- The array is originally sorted and then rotated at an unknown pivot.
- Use a modified binary search to accommodate the rotation.
- At every step, determine which half of the array is properly sorted.
- Narrow down the search based on the sorted half and the target's value.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

We perform a binary search while accounting for the possibility that the array is rotated. At each iteration, we check if the left-to-mid portion is sorted. If it is, and the target falls within that range, we search the left half; otherwise, we search the right half. If the left

half is not sorted, then the right half must be sorted, and similarly, we check if the target is in that sorted half. This approach ensures logarithmic time complexity while handling the rotation correctly.

Binary Search

□ #74 Search a 2D Matrix

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Matrix
Lists: Grind 169

Problem

You are given an $m \times n$ integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` if `target` is in `matrix` or `false` otherwise.

You must write a solution in $O(\log(m * n))$ time complexity.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = `[[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, target = 3
Output: true

Example 2:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13

Output: false

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 100$
- $-104 \leq \text{matrix}[i][j], \text{target} \leq 104$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def searchMatrix(self, matrix: list[list[int]], target: int) -> bool:
        if not matrix:
            return False

        m = len(matrix)
        n = len(matrix[0])
        l = 0
        r = m * n

        while l < r:
            mid = (l + r) // 2
            i = mid // n
            j = mid % n
            if matrix[i][j] == target:
                return True
            if matrix[i][j] < target:
                l = mid + 1
            else:
                r = mid

        return False

```

• LeetDoocs

```

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m * n - 1
        while left < right:
            mid = (left + right) >> 1
            x, y = divmod(mid, n)
            if matrix[x][y] >= target:
                right = mid
            else:
                left = mid + 1
        return matrix[left // n][left % n] == target

```

• SimplyLeet

```
def searchMatrix(matrix, target):
    # Get number of rows and columns
    if not matrix or not matrix[0]:
        return False

    rows, cols = len(matrix), len(matrix[0])
    left, right = 0, rows * cols - 1

    # Perform binary search over the virtual flattened array
    while left <= right:

        # Find the middle index
        mid = (left + right) // 2
        # Map the middle index to row and column in the matrix
        row = mid // cols
        col = mid % cols
        mid_value = matrix[row][col]

        # Check if the mid_value equals the target
        if mid_value == target:
            return True

        elif mid_value < target:
            left = mid + 1 # Move to the right subarray
        else:
            right = mid - 1 # Move to the left subarray

    # Target not found
    return False

# Example usage:
# matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]

# target = 3
# print(searchMatrix(matrix, target))
```

• LC.ca

```

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m * n - 1
        while left <= right: # must be <=, not <, for matrix=[[1]],target=1
            mid = (left + right) >> 1
            x, y = divmod(mid, n) # note: divide column count
            if matrix[x][y] == target:
                return True
            elif matrix[x][y] > target:

                right = mid - 1
            else:
                left = mid + 1
        return False

#####

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])

        left, right = 0, m * n - 1
        while left < right:
            mid = (left + right) >> 1
            x, y = divmod(mid, n)
            if matrix[x][y] >= target:
                right = mid
            else:
                left = mid + 1
        return matrix[left // n][left % n] == target

```

◆ Quick Review

Key Insights

- The matrix is essentially a flattened sorted array due to its properties.

- Use binary search to efficiently determine if the target exists.
- Map the 1D binary search index to the corresponding 2D matrix coordinates using division and modulus operations.
- Achieve $O(\log(m*n))$ time complexity by navigating the matrix as if it were a single sorted list.

Space & Time

Time Complexity: $O(\log(m * n))$

Space Complexity: $O(1)$

Solution

We can treat the matrix as a flattened sorted list without physically copying the elements. Given the total number of elements = $m * n$, perform binary search on indices ranging from 0 to $(m*n - 1)$. For any index mid , calculate its corresponding row as $mid // n$ and column as $mid \% n$. Compare the element at $matrix[row][col]$ with the target. If it equals target, return true; if it is less than target, search the right half; otherwise, search the left half.

Binary Search

□ #153 Find Minimum in Rotated Sorted Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Grind 169

Problem

Suppose an array of length n sorted in ascending order is rotated between 1 and n times. For example, the array `nums = [0,1,2,4,5,6,7]` might become:

- `[4,5,6,7,0,1,2]` if it was rotated 4 times.
- `[0,1,2,4,5,6,7]` if it was rotated 7 times.

Notice that rotating an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`.

Given the sorted rotated array `nums` of unique elements, return the minimum element of this array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [3,4,5,1,2]`

Output: `1`

Explanation: The original array was `[1,2,3,4,5]` rotated 3 times.

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`

Output: `0`

Explanation: The original array was `[0,1,2,4,5,6,7]` and it was rotated 4 times.

Example 3:

Input: `nums = [11,13,15,17]`

Output: `11`

Explanation: The original array was `[11,13,15,17]` and it was rotated 4 times.

Constraints:

- `n == nums.length`
- `1 <= n <= 5000`
- `-5000 <= nums[i] <= 5000`
- All the integers of `nums` are unique.
- `nums` is sorted and rotated between 1 and `n` times.

Community Solutions (Python)

- WalkCC


```
class Solution:
    def findMin(self, nums: list[int]) -> int:
        l = 0
        r = len(nums) - 1

        while l < r:
            m = (l + r) // 2
            if nums[m] < nums[r]:
                r = m
            else:
                l = m + 1

        return nums[l]
```

• LeetCode

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        l, r = 0, len(nums) - 1
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] > nums[-1]:
                l = mid + 1
            else:
                r = mid
        return nums[l]
```

• SimplyLeet

```
def findMin(nums):
    # Initialize left and right pointers
    left, right = 0, len(nums) - 1
    # Continue while the search space is valid
    while left < right:
        mid = (left + right) // 2 # Compute middle index
        # If middle element is greater than the rightmost element,
        # the minimum must be in the right half
        if nums[mid] > nums[right]:
            left = mid + 1
```

```
        else:
            # Otherwise, the minimum is in the left half (including mid)
            right = mid
# When loop ends, left == right, pointing to the minimum element
return nums[left]
```

• LC.ca

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        if nums[0] <= nums[-1]:
            return nums[0]
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[0] <= nums[mid]:
                left = mid + 1
            else:
                right = mid
        return nums[left]
```

◆ Quick Review

Key Insights

- The rotated sorted array consists of two sorted subarrays.
- A modified binary search can be used to locate the pivot point where the rotation occurs.
- By comparing the middle element to the rightmost element, we can decide which half of the array contains the minimum.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

Use a binary search approach. Initialize two pointers, left and right, at the start and end of the array. While left is less than right, calculate the middle index. If the middle element is greater than the right element, then the minimum is in the right half, so move the left pointer to $\text{mid} + 1$; otherwise, the minimum must be in the left half including mid, so adjust the right pointer to mid. When the loop terminates, left equals right and points to the minimum element. This approach leverages the properties of the rotated sorted array to achieve logarithmic time complexity.

Binary Search

□ #875 Koko Eating Bananas

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: NeetCode 150

Problem

Koko loves to eat bananas. There are n piles of bananas, the i th pile has $piles[i]$ bananas. The guards have gone and will come back in h hours. Koko can decide her bananas-per-hour eating speed of k . Each hour, she chooses some pile of bananas and eats k bananas from that pile. If the pile has less than k bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return. Return the minimum integer k such that she can eat all the bananas within h hours.

Example 1:

Input: $piles = [3,6,7,11]$, $h = 8$
Output: 4

Example 2:

Input: piles = [30,11,23,4,20], h = 5
Output: 30

Example 3:

Input: piles = [30,11,23,4,20], h = 6
Output: 23

Constraints:

- $1 \leq \text{piles.length} \leq 104$
- $\text{piles.length} \leq h \leq 109$
- $1 \leq \text{piles}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minEatingSpeed(self, piles: list[int], h: int) -> int:
        def eatHours(m: int) -> bool:
            """Returns the hours to eat all the piles with speed m."""
            return sum((pile - 1) // m + 1 for pile in piles)
        l = 1
        r = max(piles)
        return bisect.bisect_left(range(l, r), True,
                                key=lambda m: eatHours(m) <= h) + l
```

• LeetDoocs

```
class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        def check(k: int) -> bool:
            return sum((x + k - 1) // k for x in piles) <= h
        return 1 + bisect_left(range(1, max(piles) + 1), True, key=check)
```

• SimplyLeet

```
def minEatingSpeed(piles, h):
    # Helper function to determine if Koko can finish with speed k
    def canFinish(k):
        hours = 0
        for pile in piles:
            # Calculate hours by using ceiling division for each pile
            hours += (pile + k - 1) // k
        return hours <= h

    left, right = 1, max(piles) # Speed is between 1 and the max pile element

    while left < right:
        mid = (left + right) // 2 # Candidate eating speed
        if canFinish(mid):
            right = mid # Try finding a smaller valid speed
        else:
            left = mid + 1 # Increase the speed because mid is too slow
    return left

# Example usage:
piles = [3, 6, 7, 11]

h = 8
print(minEatingSpeed(piles, h)) # Expected output: 4
```

◆ Quick Review

Key Insights

- The required eating speed k lies in the range $[1, \max(\text{piles})]$.
- For a given k , one can calculate the total hours needed by summing up $\text{ceil}(\text{pile} / k)$ for each pile.
- The problem can be solved efficiently using binary search to find the minimum k that

satisfies the time constraint.

- Early termination in the hour calculation can be used if the computed hours exceed h .

Space & Time

Time Complexity: $O(n \log(\max(\text{pile})))$ where n is the number of piles and $\max(\text{pile})$ is the maximum bananas in a pile.

Space Complexity: $O(1)$ extra space beyond the input.

Solution

We use binary search on the candidate eating rate k . The search space starts at 1 (minimum possible speed) and goes up to the maximum number of bananas in any pile. For each candidate k , we simulate the number of hours Koko would take by calculating $\text{hours} += (\text{pile} + k - 1) // k$ for each pile (this formula performs the ceiling division). If the total hours are less than or equal to h , then k might be a valid solution and we try to lower it; otherwise, we increase k . The binary search continues until we converge on the minimum valid eating speed.

Binary Search

□ #981 Time Based Key-Value Store

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Binary Search · Design
Lists: Grind 169

Problem

Design a time-based key-value data structure that can store multiple values for the same key at different time stamps and retrieve the key's value at a certain timestamp.

Implement the TimeMap class:

- **TimeMap()** Initializes the object of the data structure.
- **void set(String key, String value, int timestamp)** Stores the key key with the value value at the given time timestamp.
- **String get(String key, int timestamp)** Returns a value such that set was called previously, with $\text{timestamp_prev} \leq \text{timestamp}$. If there are multiple such values, it returns the value associated with the largest timestamp_prev. If

there are no values, it returns "".

Example 1:

Input

```
["TimeMap", "set", "get", "get", "set", "get", "get"]  
[[], ["foo", "bar", 1], ["foo", 1], ["foo", 3], ["foo", "bar2", 4], ["foo", 4],  
["foo", 5]]
```

Output

```
[null, null, "bar", "bar", null, "bar2", "bar2"]
```

Explanation

```
TimeMap timeMap = new TimeMap();
```

Constraints:

- $1 \leq \text{key.length}, \text{value.length} \leq 100$
- key and value consist of lowercase English letters and digits.
- $1 \leq \text{timestamp} \leq 107$
- All the timestamps timestamp of set are strictly increasing.
- At most $2 * 10^5$ calls will be made to set and get.

Community Solutions (Python)

- ☐
- WalkCC

```

class TimeMap:
    def __init__(self):
        self.values = collections.defaultdict(list)
        self.timestamps = collections.defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.values[key].append(value)
        self.timestamps[key].append(timestamp)

    def get(self, key: str, timestamp: int) -> str:

        if key not in self.timestamps:
            return ''
        i = bisect.bisect(self.timestamps[key], timestamp)
        return self.values[key][i - 1] if i > 0 else ''

```

• LeetDoocs

```

class TimeMap:
    def __init__(self):
        self.ktv = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.ktv[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        if key not in self.ktv:
            return ''

        tv = self.ktv[key]
        i = bisect_right(tv, (timestamp, chr(127)))
        return tv[i - 1][1] if i else ''

# Your TimeMap object will be instantiated and called as such:
# obj = TimeMap()
# obj.set(key,value,timestamp)
# param_2 = obj.get(key,timestamp)

```

• SimplyLeet

```
# TimeMap class in Python with line-by-line comments.
class TimeMap:
    def __init__(self):
        # Initialize the dictionary to hold key and list of (timestamp, value)
        # pairs.
        self.store = {}

    def set(self, key: str, value: str, timestamp: int) -> None:
        # Append the new (timestamp, value) pair for the given key.
        if key not in self.store:
            self.store[key] = []

        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        # Return the value for the largest timestamp <= the given timestamp.
        if key not in self.store:
            return ""

        values = self.store[key]
        left, right = 0, len(values) - 1

        # Binary search for the rightmost index with timestamp <= given
        # timestamp.
        while left <= right:
            mid = (left + right) // 2
            if values[mid][0] <= timestamp:
                left = mid + 1
            else:
                right = mid - 1

        # If no valid timestamp exists, right will be -1.
        if right == -1:
```

```

        return ""
    return values[right][1]

# Example usage:
# timeMap = TimeMap()
# timeMap.set("foo", "bar", 1)
# print(timeMap.get("foo", 1)) # Output: "bar"
# print(timeMap.get("foo", 3)) # Output: "bar"
# timeMap.set("foo", "bar2", 4)
# print(timeMap.get("foo", 4)) # Output: "bar2"

# print(timeMap.get("foo", 5)) # Output: "bar2"

```

• LC.ca

```

class TimeMap:
    def __init__(self):
        self.ktv = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.ktv[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        if key not in self.ktv:
            return ''

        tv = self.ktv[key]
        i = bisect_right(tv, (timestamp, chr(127)))
        return tv[i - 1][1] if i else ''

# Your TimeMap object will be instantiated and called as such:
# obj = TimeMap()
# obj.set(key,value,timestamp)
# param_2 = obj.get(key,timestamp)

```

◆ Quick Review

Key Insights

- Use a dictionary (hash table) to map each key to a list of (timestamp, value) pairs.
- Timestamps are strictly increasing when setting the values, which allows efficient appending.
- For getting the value, perform a binary search on the list of timestamps to find the largest timestamp that is $\leq$ the queried timestamp.
- If no valid timestamp exists, return an empty string.

Space & Time

Time Complexity:

set operation: $O(1)$ per call (amortized).

get operation: $O(\log N)$ per call, where N is the number of timestamps stored for the key.

Space Complexity: $O(\text{total number of set calls})$, to store all (timestamp, value) pairs.

Solution

We use a hash table to store keys mapping to a list of their timestamps and corresponding values. Since the timestamps are inserted in strictly increasing order, the list remains

sorted. For the get operation, a binary search is applied to quickly locate the appropriate value for a given timestamp. This approach minimizes the lookup time and efficiently handles the problem constraints.

Intervals

Round 2 · High · Medium · 4 questions

Intervals

□ #56 Merge Intervals

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting
Lists: Grind 169

Problem

Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Example 1:

Input: `intervals = [[1,3],[2,6],[8,10],[15,18]]`
Output: `[[1,6],[8,10],[15,18]]`
Explanation: Since intervals `[1,3]` and `[2,6]` overlap, merge them into `[1,6]`.

Example 2:

Input: `intervals = [[1,4],[4,5]]`
Output: `[[1,5]]`
Explanation: Intervals `[1,4]` and `[4,5]` are considered overlapping.

Example 3:

Input: `intervals = [[4,7],[1,4]]`
Output: `[[1,7]]`
Explanation: Intervals `[1,4]` and `[4,7]` are considered overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 104$

- `intervals[i].length == 2`
- `0 <= starti <= endi <= 104`

Community Solutions (Python)

• WalkCC

```
class Solution:
    def merge(self, intervals: list[list[int]]) -> list[list[int]]:
        ans = []

        for interval in sorted(intervals):
            if not ans or ans[-1][1] < interval[0]:
                ans.append(interval)
            else:
                ans[-1][1] = max(ans[-1][1], interval[1])

        return ans
```

• LeetDoocs

```
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort()
        ans = []
        st, ed = intervals[0]
        for s, e in intervals[1:]:
            if ed < s:
                ans.append([st, ed])
                st, ed = s, e
            else:
                ed = max(ed, e)
        ans.append([st, ed])
        return ans
```

• SimplyLeet

```
def merge(intervals):
    # Edge case: if the list is empty, return an empty list
    if not intervals:
        return []
    # Sort intervals based on the starting time
    intervals.sort(key=lambda x: x[0])
    merged = [] # List to store merged intervals

    # Iterate over each interval in the sorted list
    for interval in intervals:

        # If merged list is not empty and the current interval
        # overlaps with the last one in merged, merge them
        if merged and interval[0] <= merged[-1][1]:
            # Update the end of the last interval in merged if needed
            merged[-1][1] = max(merged[-1][1], interval[1])
        else:
            # No overlap, so add the current interval to merged
            merged.append(interval)
    return merged

# Example usage:
print(merge([[1,3],[2,6],[8,10],[15,18]]))
```

• LC.ca

```
# Definition for an interval.
# class Interval(object):
#     def __init__(self, s=0, e=0):
#         self.start = s
#         self.end = e

class Solution(object):
    def merge(self, intervals):
        """
        :type intervals: List[Interval]
        """
```

```

:rtype: List[Interval]
"""
ans = []
for intv in sorted(intervals, key=lambda x: x.start):
    if ans and ans[-1].end >= intv.start:
        ans[-1].end = max(ans[-1].end, intv.end)
    else:
        ans.append(intv)
return ans

```

```
#####
```

```
'''
```

```

>>> intervals = [[111,222],[1,3],[2,6],[8,10],[15,18]]
>>> intervals.sort()
>>> intervals
[[1, 3], [2, 6], [8, 10], [15, 18], [111, 222]]
'''

```

```

class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:

```

```

        intervals.sort() # default sort also ok
        ans = [intervals[0]]
        for s, e in intervals[1:]:
            if ans[-1][1] < s:
                ans.append([s, e])
            else:
                ans[-1][1] = max(ans[-1][1], e)
        return ans

```

◆ Quick Review

Key Insights

- Sort the intervals based on their start values.
- Use a pointer or iteration to compare the current interval with the last merged interval.

- If the current interval overlaps with the previous one, merge them by updating the end time.
- Otherwise, add the current interval to the results as a new merged interval.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(n)$ to store the merged intervals.

Solution

First, sort the array of intervals based on the starting time. Then, iterate through the sorted list and check if the current interval overlaps with the last interval in the merged list. If an overlap is detected (i.e., the start of the current interval is less than or equal to the end of the last merged interval), update the last merged interval's end to be the maximum of both ends. If there is no overlap, simply add the current interval to the merged list. This approach efficiently merges overlapping intervals and ensures that all intervals are processed in a single pass after sorting.

Intervals

□ #57 Insert Interval

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array

Lists: Grind 169

Problem

You are given an array of non-overlapping intervals `intervals` where `intervals[i] = [starti, endi]` represent the start and the end of the *i*th interval and `intervals` is sorted in ascending order by `starti`. You are also given an interval `newInterval = [start, end]` that represents the start and end of another interval.

Insert `newInterval` into `intervals` such that `intervals` is still sorted in ascending order by `starti` and `intervals` still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return `intervals` after the insertion.

Note that you don't need to modify `intervals` in-place. You can make a new array and return it.

Example 1:

Input: intervals = [[1,3],[6,9]], newInterval = [2,5]
 Output: [[1,5],[6,9]]

Example 2:

Input: intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval = [4,8]
 Output: [[1,2],[3,10],[12,16]]
 Explanation: Because the new interval [4,8] overlaps with [3,5],[6,7],[8,10].

Constraints:

- $0 \leq \text{intervals.length} \leq 104$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i \leq \text{end}_i \leq 105$
- intervals is sorted by start_i in ascending order.
- $\text{newInterval.length} == 2$
- $0 \leq \text{start} \leq \text{end} \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def insert(self, intervals: list[list[int]],
               newInterval: list[int]) -> list[list[int]]:
        n = len(intervals)
        ans = []
        i = 0

        while i < n and intervals[i][1] < newInterval[0]:
            ans.append(intervals[i])
            i += 1
```

```

# Merge overlapping intervals.
while i < n and intervals[i][0] <= newInterval[1]:
    newInterval[0] = min(newInterval[0], intervals[i][0])
    newInterval[1] = max(newInterval[1], intervals[i][1])
    i += 1

ans.append(newInterval)

while i < n:

    ans.append(intervals[i])
    i += 1

return ans

```

• LeetCode

```

class Solution:
    def insert(
        self, intervals: List[List[int]], newInterval: List[int]
    ) -> List[List[int]]:
        def merge(intervals: List[List[int]]) -> List[List[int]]:
            intervals.sort()
            ans = [intervals[0]]
            for s, e in intervals[1:]:
                if ans[-1][1] < s:
                    ans.append([s, e])

            else:
                ans[-1][1] = max(ans[-1][1], e)
            return ans

        intervals.append(newInterval)
        return merge(intervals)

```

• SimplyLeet

```
# Python solution for Insert Interval

class Solution:
    def insert(self, intervals, newInterval):
        # List to store the resulting intervals
        merged_intervals = []
        i = 0
        n = len(intervals)

        # Add all intervals that end before newInterval starts

        while i < n and intervals[i][1] < newInterval[0]:
            merged_intervals.append(intervals[i])
            i += 1

        # Merge overlapping intervals with newInterval
        while i < n and intervals[i][0] <= newInterval[1]:
            # Update the start and end of newInterval by merging with
            overlapping intervals
            newInterval[0] = min(newInterval[0], intervals[i][0])
            newInterval[1] = max(newInterval[1], intervals[i][1])
            i += 1

        # Add the merged newInterval
        merged_intervals.append(newInterval)

        # Add any remaining intervals that come after newInterval
        while i < n:
            merged_intervals.append(intervals[i])
            i += 1

        return merged_intervals
```

 • LC.ca


```

'''
>>> a = [1,2,3,4,5]
>>> a[1]
2
>>> a[~1]
4
'''

class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        start = newInterval[0]

        end = newInterval[1]

        left = list(filter(lambda x: x[1] < start, intervals)) # cast via list()
        right = list(filter(lambda x: x[0] > end, intervals))

        if left + right != intervals:
            start = min(start, intervals[len(left)][0]) # note, left not -1, because index starts at 0
            end = max(end, intervals[~len(right)][1]) # same, starting at right with index=0

        return left + [[start, end]] + right

#####

class Solution: # re-use Leetcode-56's merge solution
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        intervals.append(newInterval)
        return self.merge(intervals)

    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        ans = []

```

```

for intv in sorted(intervals, key=lambda x: x[0]):
    if ans and ans[-1][1] >= intv[0]:
        ans[-1][1] = max(ans[-1][1], intv[1])
    else:
        ans.append(intv)
return ans

```

```
#####
```

```
class Solution:
```

```

def insert(
    self, intervals: List[List[int]], newInterval: List[int]
) -> List[List[int]]:
    def merge(intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort()
        ans = [intervals[0]]
        for s, e in intervals[1:]:
            if ans[-1][1] < s:
                ans.append([s, e])
            else:
                ans[-1][1] = max(ans[-1][1], e)
        return ans

    intervals.append(newInterval)
    return merge(intervals)

```

◆ Quick Review

Key Insights

- The intervals array is sorted by start times.
- Three main steps are used:
- Add intervals that come completely before the new interval.

- Merge overlapping intervals with the new interval.
- Append intervals that start after the new (merged) interval.
- Iterating through the list once allows an $O(n)$ solution.

Space & Time

Time Complexity: $O(n)$ since we scan the list once.

Space Complexity: $O(n)$ to store the resulting list of intervals.

Solution

Traverse the intervals list and perform the following: Append all intervals that end before the new interval starts. For intervals that overlap with `newInterval`, update `newInterval` to merge them (using `min` for start and `max` for end). Append the new merged interval. Append all remaining intervals. This ensures that all intervals remain non-overlapping and sorted with a single pass.

Intervals

★ #253 Meeting Rooms II ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Greedy · Sorting · Heap
Lists: ★ Premium | Grind 169 | Premium 98

Problem

Given an array of meeting time intervals intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, return the minimum number of conference rooms required.

Example 1:

Input: intervals = [[0,30],[5,10],[15,20]]
Output: 2

Example 2:

Input: intervals = [[7,10],[2,4]]
Output: 1

Constraints:

- $1 \leq \text{intervals.length} \leq 10^4$
- $0 \leq \text{start}_i < \text{end}_i \leq 10^6$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def minMeetingRooms(self, intervals: list[list[int]]) -> int:
        minHeap = [] # Store the end times of each room.

        for start, end in sorted(intervals):
            # There's no overlap, so we can reuse the same room.
            if minHeap and start >= minHeap[0]:
                heapq.heappop(minHeap)
            heapq.heappush(minHeap, end)

        return len(minHeap)
```

```
class Solution:
    def minMeetingRooms(self, intervals: list[list[int]]) -> int:
        n = len(intervals)
        ans = 0
        starts = []
        ends = []

        for start, end in intervals:
            starts.append(start)
            ends.append(end)

        starts.sort()
        ends.sort()

        j = 0
        for i in range(n):
            if starts[i] < ends[j]:
                ans += 1
            else:
                j += 1

        return ans
```

• LeetDoocs

```

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        m = max(e[1] for e in intervals)
        d = [0] * (m + 1)
        for l, r in intervals:
            d[l] += 1
            d[r] -= 1
        ans = s = 0
        for v in d:
            s += v

            ans = max(ans, s)
        return ans

```

```

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        d = defaultdict(int)
        for l, r in intervals:
            d[l] += 1
            d[r] -= 1
        ans = s = 0
        for _, v in sorted(d.items()):
            s += v
            ans = max(ans, s)

        return ans

```

• SimplyLeet

```

import heapq

def minMeetingRooms(intervals):
    # Sort intervals based on start time
    intervals.sort(key=lambda x: x[0])

    # Initialize a min-heap to store end times of meetings that are currently
    # using rooms
    min_heap = []

    # Process each meeting interval

```

```

    for interval in intervals:
        start, end = interval

        # If the heap is not empty and the current meeting starts after the
        earliest ended meeting
        if min_heap and start >= min_heap[0]:
            heapq.heappop(min_heap) # Reuse the room

        # Allocate the current meeting end time into the heap (new or reused
        room)
        heapq.heappush(min_heap, end)

    # The number of rooms is the number of simultaneous meetings
    return len(min_heap)

# Example usage:
intervals = [[0,30],[5,10],[15,20]]
print(minMeetingRooms(intervals)) # Output: 2

```

• LC.ca

```

...
data = [3, 4, 6, 2, 1, 9, 0, 7, 5, 8]

list(accumulate(data, operator.mul)) # running product
[3, 12, 72, 144, 144, 1296, 0, 0, 0, 0]

list(accumulate(data, max)) # running maximum
[3, 4, 6, 6, 6, 9, 9, 9, 9, 9]

```

```

>>> from itertools import accumulate
>>> data = [3, 4, 6, 2, 1, 9, 0, 7, 5, 8]
>>> accumulate(data)
<itertools.accumulate object at 0x10fd78440>
>>> list(accumulate(data))
[3, 7, 13, 15, 16, 25, 25, 32, 37, 45]
>>> max(accumulate(data))
45

```

<https://docs.python.org/3/library/itertools.html#itertools.accumulate>

```
# Amortize a 5% loan of 1000 with 4 annual payments of 90
cashflows = [1000, -90, -90, -90, -90]
list(accumulate(cashflows, lambda bal, pmt: bal*1.05 + pmt))
[1000, 960.0, 918.0, 873.9000000000001, 827.5950000000001]
'''

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        delta = [0] * 1000010

        for start, end in intervals:
            delta[start] += 1
            delta[end] -= 1
        return max(accumulate(delta))
# why not delta.sort()?
# because accumulate() will go by order from index 0 to index final
# just like, from sortedcontainers import SortedDict

#####
```

```
class Solution(object):
    def minMeetingRooms(self, intervals):
        """
        :type intervals: List[Interval]
        :rtype: int
        """
        meetings = []
        for i in intervals:
            meetings.append((i.start, 1))
            meetings.append((i.end, 0))
```

```
meetings.sort()
ans = 0
count = 0
for meeting in meetings:
    if meeting[1] == 1:
        count += 1
    else:
        count -= 1
    ans = max(ans, count)
return ans
```


◆ Quick Review

Key Insights

- Sort the intervals based on start times to process meetings in order.
- Use a min-heap (priority queue) to keep track of meeting end times.
- When processing a new meeting, check if its start time is greater than or equal to the smallest end time from the heap.
- If so, it means a meeting room has been freed (the meeting with the smallest end time has ended) and can be reused.
- If not, allocate a new room by pushing the current meeting's end time into the heap.
- The size of the heap at any moment represents the number of rooms currently in use.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting and each heap operation (push/pop) taking $O(\log n)$.

Space Complexity: $O(n)$ in the worst-case scenario when all meetings overlap, requiring all end times in the heap.

Solution

The solution involves first sorting the meeting intervals by their start times. Then, we utilize a min-heap (priority queue) to keep track of the end times of ongoing meetings. For each meeting, we compare its start time with the smallest end time in the heap: If the meeting can start after the meeting at the top of the heap has ended (i.e., its start time is greater than or equal to the smallest end), we remove that end time from the heap (freeing up a room). Then, we add the current meeting's end time to the heap. By the end of processing all meetings, the size of the heap represents the minimum number of conference rooms required.

Intervals

□ #435 Non-overlapping Intervals

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy ·
Sorting
Lists: Grind 169

Problem

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.

Note that intervals which only touch at a point are non-overlapping. For example, `[1, 2]` and `[2, 3]` are non-overlapping.

Example 1:

```
Input: intervals = [[1,2],[2,3],[3,4],[1,3]]
Output: 1
Explanation: [1,3] can be removed and the rest of the intervals are
non-overlapping.
```

Example 2:

```
Input: intervals = [[1,2],[1,2],[1,2]]
Output: 2
Explanation: You need to remove two [1,2] to make the rest of the intervals
non-overlapping.
```

Example 3:

Input: intervals = [[1,2],[2,3]]

Output: 0

Explanation: You don't need to remove any of the intervals since they're already non-overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 105$
- $\text{intervals}[i].\text{length} == 2$
- $-5 * 10^4 \leq \text{start}_i < \text{end}_i \leq 5 * 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def eraseOverlapIntervals(self, intervals: list[list[int]]) -> int:
        ans = 0
        currentEnd = -math.inf

        for interval in sorted(intervals, key=lambda x: x[1]):
            if interval[0] >= currentEnd:
                currentEnd = interval[1]
            else:
                ans += 1

        return ans
```

• LeetDoocs

```
class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: x[1])
        ans = len(intervals)
        pre = -inf
        for l, r in intervals:
            if pre <= l:
                ans -= 1
                pre = r
        return ans
```

• SimplyLeet

Python implementation for Non-overlapping Intervals

```
def eraseOverlapIntervals(intervals):
    # Sort intervals based on the end time
    intervals.sort(key=lambda x: x[1])

    # Initialize the count of intervals in the non-overlapping set
    count = 0
    # Initialize the end of the last added interval to the minimum possible
    # value
    last_included_end = float('-inf')

    # Iterate over every interval in the sorted list
    for interval in intervals:
        # If the current interval does not overlap with the last added interval
        if interval[0] >= last_included_end:
            count += 1          # We include this interval
            last_included_end = interval[1] # Update end for subsequent
            comparisons
        # The minimum number of removals is total intervals minus non-overlapping
        # intervals count
    return len(intervals) - count
```

```
# Example usage:
print(eraseOverlapIntervals([[1,2],[2,3],[3,4],[1,3]])) # Expected output: 1
```

• LC.ca

```
class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: x[1])
        ans, t = 0, intervals[0][1]
        for s, e in intervals[1:]:
            if s >= t:
                t = e
            else:
                ans += 1
        return ans
```

◆ Quick Review

Key Insights

- **Sorting is the key:** By sorting intervals by their end times, we can greedily choose the optimal set of non-overlapping intervals.
- **Greedy approach:** Always select the interval that ends the earliest, as it leaves maximum room for subsequent intervals.
- **Counting removals:** The answer is the total number of intervals minus the count of intervals selected by this greedy approach.
- **Edge observation:** Intervals that touch at the boundary are not overlapping, so the check uses “>=” instead of “>”.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(n)$ if considering the storage for the sorted list (or $O(1)$ if sorting in-place).

Solution

The solution involves first sorting the intervals by their end times. Initialize a variable to track the end of the last chosen interval. Iterate over the sorted intervals: if the current interval's start is greater than or equal to the last chosen interval's end, update the last end and include this interval in the non-overlapping set. Otherwise, count the interval as one that must be removed. The final answer is the removal count, which is computed as the total number of intervals minus the count of non-overlapping intervals selected.

Trees

Round 2 · High · Medium · 7 questions

Trees

□ #98 Validate Binary Search Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST

Lists: Grind 169

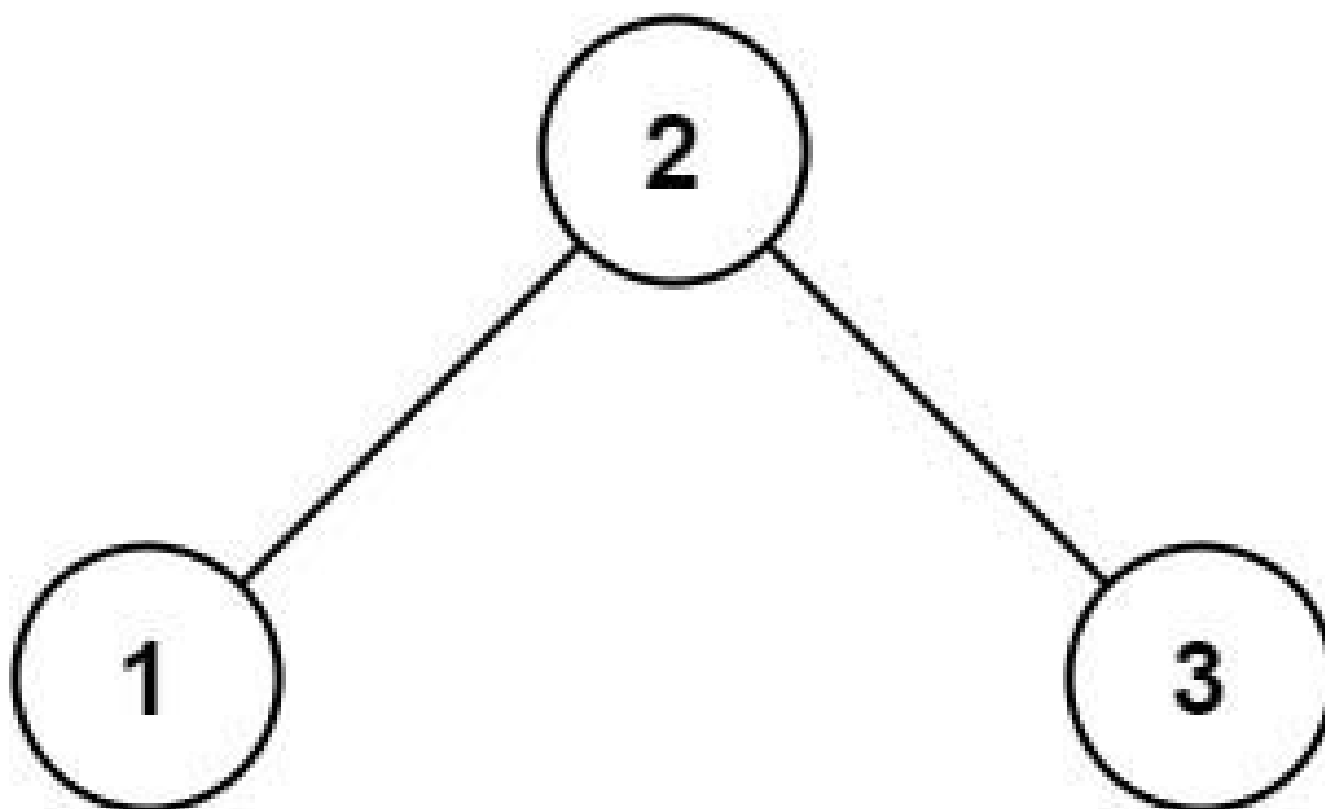
Problem

Given the root of a binary tree, determine if it is a valid binary search tree (BST).

A valid BST is defined as follows:

- The left subtree of a node contains only nodes with keys strictly less than the node's key.
- The right subtree of a node contains only nodes with keys strictly greater than the node's key.
- Both the left and right subtrees must also be binary search trees.

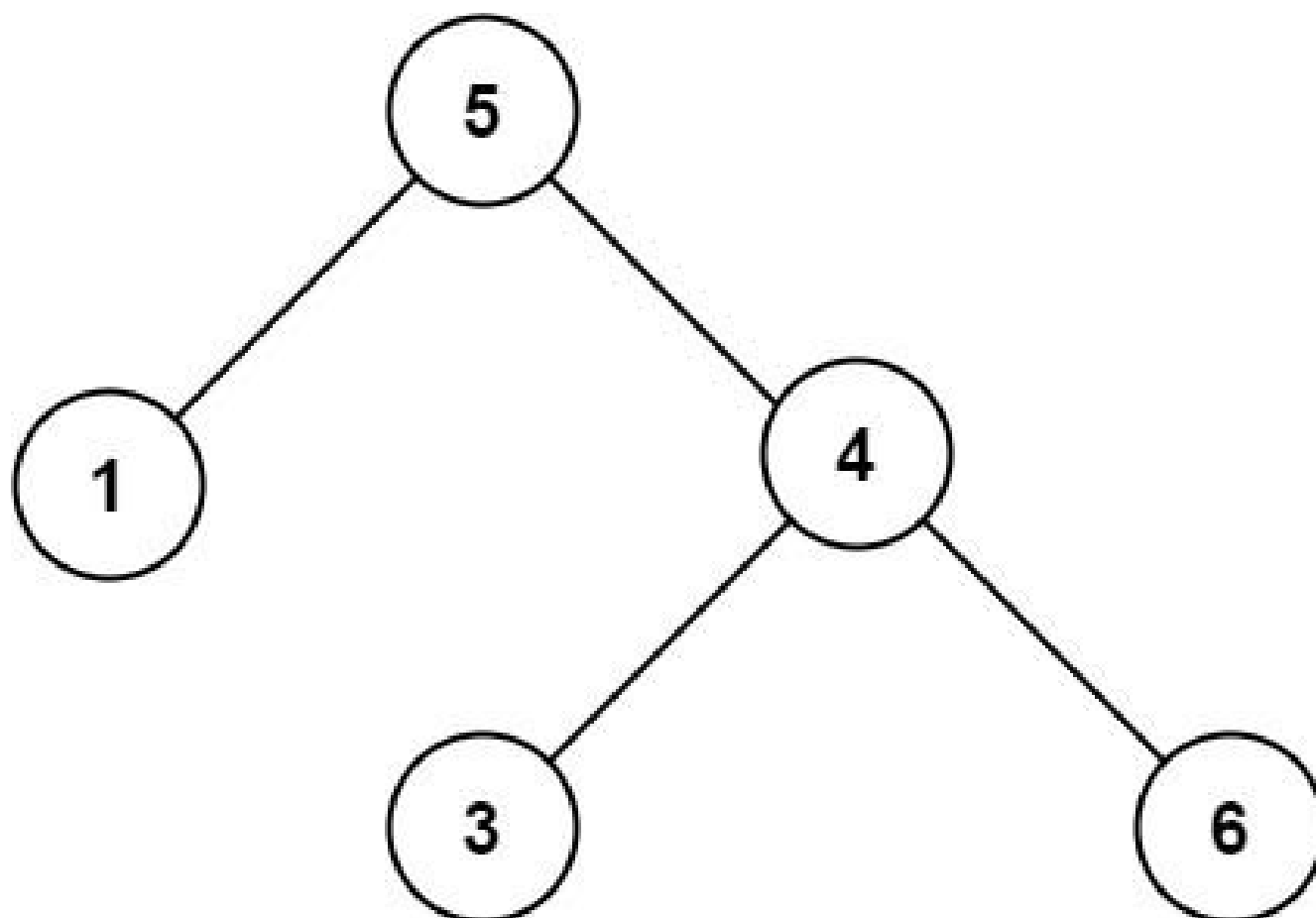
Example 1:



Input: root = [2,1,3]

Output: true

Example 2:



Input: root = [5,1,4,null,null,3,6]

Output: false

Explanation: The root node's value is 5 but its right child's value is 4.

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def isValidBST(self, root: TreeNode | None) -> bool:
        def isValidBST(root: TreeNode | None,
                        minNode: TreeNode | None, maxNode: TreeNode | None) -> bool
        :
            if not root:
                return True
            if minNode and root.val <= minNode.val:
                return False
            if maxNode and root.val >= maxNode.val:
                return False

            return (isValidBST(root.left, minNode, root) and
                    isValidBST(root.right, root, maxNode))

        return isValidBST(root, None, None)

```

```

class Solution:
    def isValidBST(self, root: TreeNode | None) -> bool:
        stack = []
        pred = None

        while root or stack:
            while root:
                stack.append(root)
                root = root.left
            root = stack.pop()

            if pred and pred.val >= root.val:
                return False
            pred = root
            root = root.right

        return True

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def dfs(root: Optional[TreeNode]) -> bool:
            if root is None:
                return True
            if not dfs(root.left):
                return False
            nonlocal prev
            if prev >= root.val:
                return False
            prev = root.val
            return dfs(root.right)

        prev = -inf

        return dfs(root)

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val      # Node's value
        self.left = left    # Left child
        self.right = right  # Right child

class Solution:
    def isValidBST(self, root: TreeNode) -> bool:
        # Helper function to recursively validate nodes

```

```
def validate(node, low, high):
    if not node:
        return True # An empty node/subtree is valid
    # If current node's value is out of the allowed range, return False
    if not (low < node.val < high):
        return False
    # Recursively check the left subtree and right subtree
    return validate(node.left, low, node.val) and validate(node.right,
node.val, high)

return validate(root, float("-inf"), float("inf"))
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val

# self.left = left
# self.right = right

class Solution: # recursive
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def dfs(root, mi, ma):
            if not root:
                return True
            return mi < root.val < ma and dfs(root.left, mi, root.val) and dfs
(root.right, root.val, ma)
```

```
        return dfs(root, -math.inf, math.inf)

class Solution: # iterative
    def isValidBST(self, root: TreeNode) -> bool:
        # store (node, lower, upper) tuples in a single queue
        queue = [(root, None, None)]

        # another option:
        # queue = [(root, -math.inf, math.inf)]

        while queue:
            node, lower, upper = queue.pop(0)

            if node is None:
                continue

            val = node.val
            if lower is not None and val <= lower:
                return False

            if upper is not None and val >= upper:
                return False

            queue.append((node.right, val, upper))
            queue.append((node.left, lower, val))

        return True
```

```
#####
```

```

class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def isValidBST(root: Optional[TreeNode],
                        minNode: Optional[TreeNode], maxNode: Optional[TreeNode]) -
> bool:
            if not root:
                return True
            if minNode and root.val <= minNode.val:
                return False
            if maxNode and root.val >= maxNode.val:
                return False

            return isValidBST(root.left, minNode, root) and isValidBST(root.right, r
oot, maxNode)

            return isValidBST(root, None, None)

#####

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):

# self.val = val
# self.left = left
# self.right = right
class Solution:
    def isValidBST(self, root: TreeNode) -> bool:
        def dfs(root):
            nonlocal prev
            if root is None:
                return True
            if not dfs(root.left):

```



```

        return False
    if prev >= root.val:
        return False
    prev = root.val
    if not dfs(root.right):
        return False
    return True

prev = -inf
return dfs(root)

```

```
#####
```

```

# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

```

```

class Solution(object):
    def isValidBST(self, root):
        """
        :type root: TreeNode
        :rtype: bool
        """
        prev = -float("inf")
        stack = [(1, root)]
        while stack:
            p = stack.pop()

            if not p[1]:
                continue
            if p[0] == 0:
                if p[1].val <= prev:
                    return False
                prev = p[1].val
            else:
                stack.append((1, p[1].right))
                stack.append((0, p[1]))
                stack.append((1, p[1].left))

```

```
return True
```

◆ Quick Review

Key Insights

- Every node must adhere to a range: all nodes in the left subtree should be less than the current node, and all nodes in the right subtree should be greater.
- A recursive depth-first search approach works well by passing down the valid range (lower and upper bounds) for each node.
- Using the limits (negative and positive infinity) simplifies handling edge cases at the tree root.
- An in-order traversal would alternatively yield a sorted order for node values, but updating ranges directly is more intuitive for recursive implementation.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes, as each node is visited once.

Space Complexity: $O(H)$, where H is the height of the tree, due to the recursion stack.

Solution

We use a recursive approach to validate the BST property. We maintain two variables, low and high, representing the valid range for the current node's value. For the left child, the valid range becomes (low, node.val) and for the right child, it becomes (node.val, high). By checking if each node's value falls within its respective range, we can ensure that the tree satisfies the BST properties.

Trees

□ #102 Binary Tree Level Order Traversal

MED

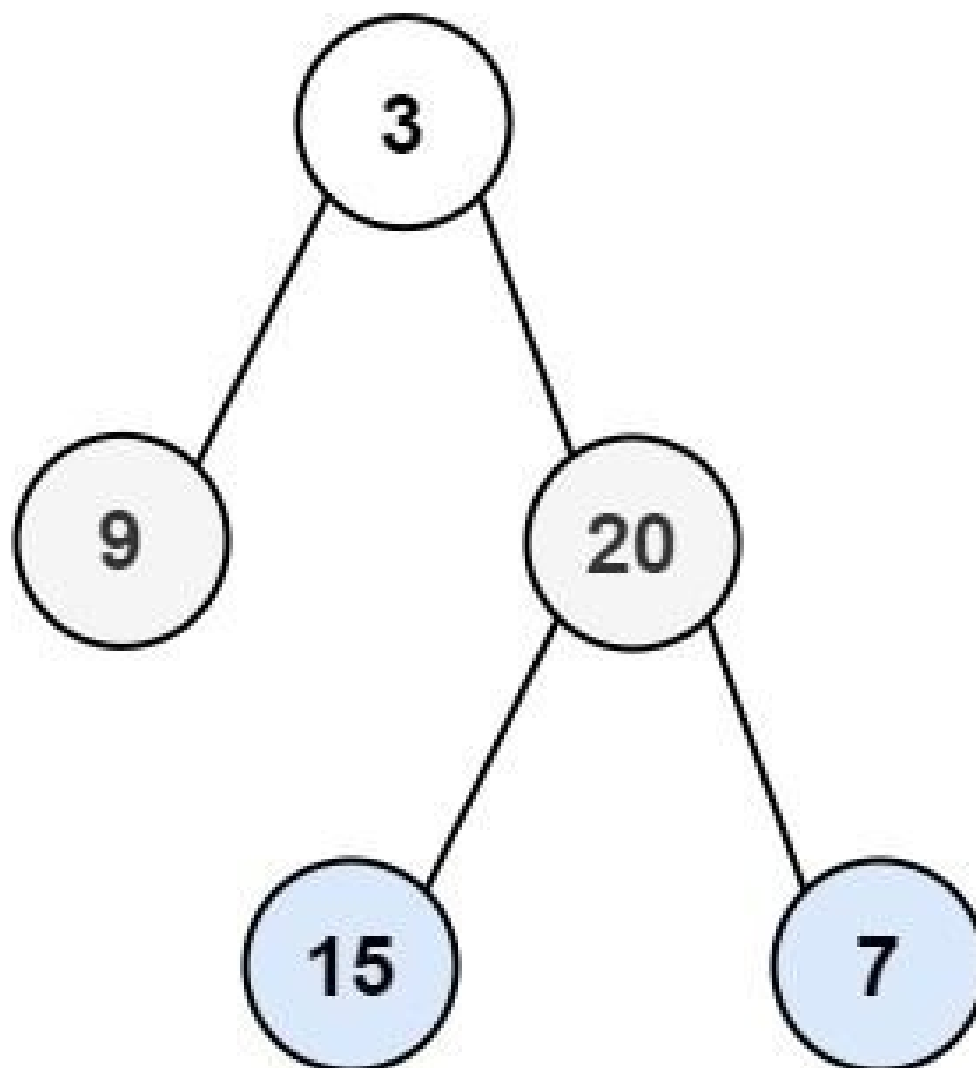
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 2000]$.
- $-1000 \leq \text{Node.val} \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def levelOrder(self, root: TreeNode | None) -> list[list[int]]:
        if not root:
            return []

        ans = []
        q = collections.deque([root])

        while q:
            currLevel = []

            for _ in range(len(q)):
                node = q.popleft()
                currLevel.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(currLevel)

        return ans
```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        while q:
            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(t)
        return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

from collections import deque

class Solution:

```

```

def levelOrder(self, root: TreeNode) -> List[List[int]]:
    # Return an empty list if the tree is empty.
    if not root:
        return []

    result = [] # Final result list containing values of each level.
    queue = deque([root]) # Initialize the queue with the root node.

    # Process nodes level by level.
    while queue:

        level_size = len(queue) # Number of nodes at the current level.
        level_nodes = [] # List to hold node values at the current level.

        for _ in range(level_size):
            node = queue.popleft() # Dequeue the next node.
            level_nodes.append(node.val)
            # Enqueue left child if it exists.
            if node.left:
                queue.append(node.left)
            # Enqueue right child if it exists.

            if node.right:
                queue.append(node.right)
        # Append the current level's values to the result.
        result.append(level_nodes)
    return result

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:

```



```
        return ans
    q = deque([root])
    while q:
        t = []
        for _ in range(len(q)):
            node = q.popleft()
            t.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        ans.append(t)
    return ans
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to traverse the tree.
- A queue data structure is ideal for maintaining the order of nodes at each level.
- For every level, process all nodes currently in the queue and add their children for the next level.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, because each node is processed once.

Space Complexity: $O(n)$, which accounts for the space used by the queue in the worst-case scenario.

Solution

This solution leverages a BFS approach using a queue. The algorithm starts by enqueueing the root node. Then, it enters a loop that continues until the queue is empty. For each iteration—representing one level of the tree—it computes the number of nodes at that level (`levelSize`), processes these nodes by dequeuing them, and collects their values. Their non-null children are then enqueued. The list of values for each level is added to the final result list. Key data structures include the queue (for managing nodes) and a list (to store level values).

Trees

□ #105 Construct Binary Tree from Preorder and Inorder Traversal

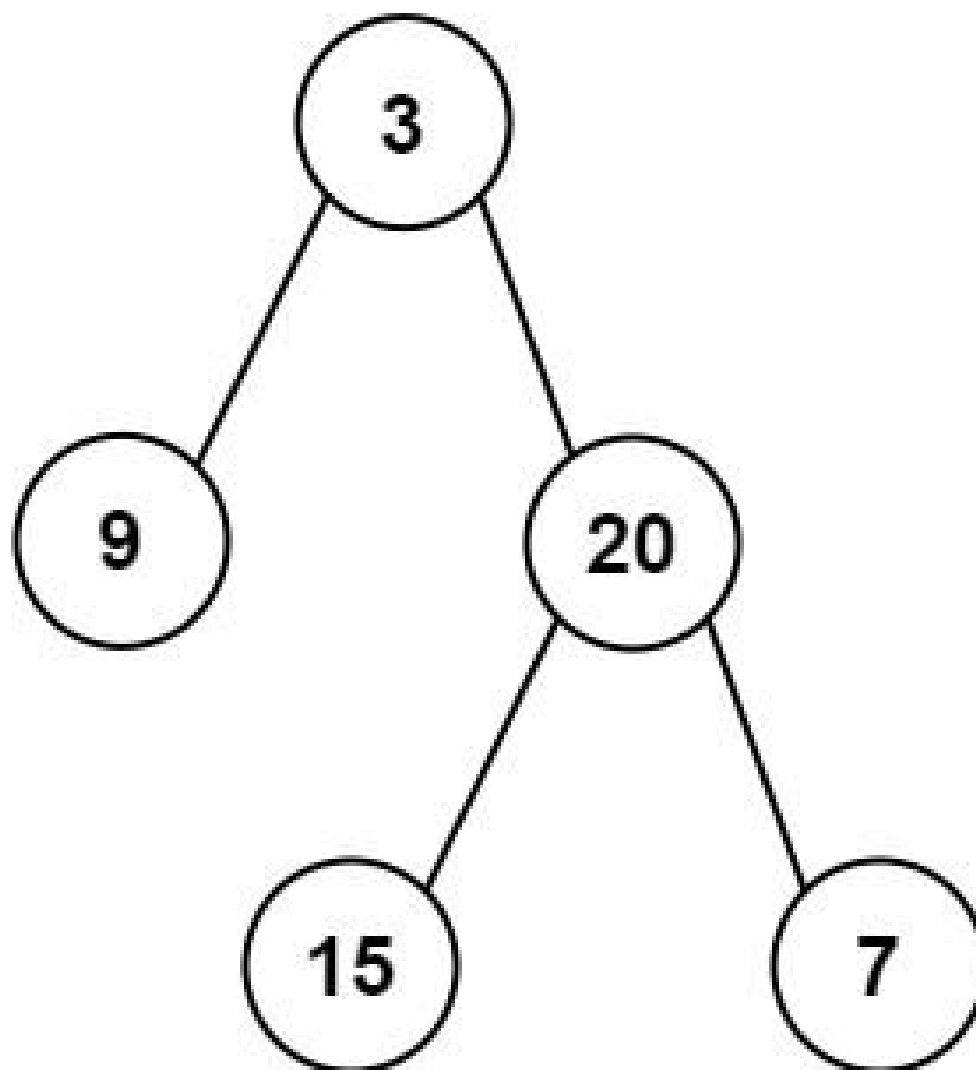
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Tree · DFS
Lists: Grind 169

Problem

Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return the binary tree.

Example 1:



Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
Output: [3,9,20,null,null,15,7]

Example 2:

Input: preorder = [-1], inorder = [-1]
Output: [-1]

Constraints:

- $1 \leq \text{preorder.length} \leq 3000$
- $\text{inorder.length} == \text{preorder.length}$
- $-3000 \leq \text{preorder}[i], \text{inorder}[i] \leq 3000$

- preorder and inorder consist of unique values.
- Each value of inorder also appears in preorder.
- preorder is guaranteed to be the preorder traversal of the tree.
- inorder is guaranteed to be the inorder traversal of the tree.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def buildTree(
        self,
        preorder: list[int],
        inorder: list[int],
    ) -> TreeNode | None:
        inToIndex = {num: i for i, num in enumerate(inorder)}

        def build(
            preStart: int,

            preEnd: int,
            inStart: int,
            inEnd: int,
        ) -> TreeNode | None:
            if preStart > preEnd:
                return None

            rootVal = preorder[preStart]
            rootInIndex = inToIndex[rootVal]
            leftSize = rootInIndex - inStart
```

```

root = TreeNode(rootVal)
root.left = build(preStart + 1, preStart + leftSize,
                  inStart, rootInIndex - 1)
root.right = build(preStart + leftSize + 1,
                  preEnd, rootInIndex + 1, inEnd)

return root

return build(0, len(preorder) - 1, 0, len(inorder) - 1)

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def dfs(i: int, j: int, n: int) -> Optional[TreeNode]:
            if n <= 0:
                return None
            v = preorder[i]
            k = d[v]
            l = dfs(i + 1, j, k - j)
            r = dfs(i + 1 + k - j, k + 1, n - k + j - 1)
            return TreeNode(v, l, r)

        d = {v: i for i, v in enumerate(inorder)}
        return dfs(0, 0, len(preorder))

```

```

class Solution:
    def getBinaryTrees(self, preOrder: List[int], inOrder: List[int]) -> List[
TreeNode]:
        def dfs(i: int, j: int, n: int) -> List[TreeNode]:
            if n <= 0:
                return [None]
            v = preOrder[i]
            ans = []
            for k in d[v]:
                if j <= k < j + n:
                    for l in dfs(i + 1, j, k - j):
                        for r in dfs(i + 1 + k - j, k + 1, n - 1 - (k - j)):
                            ans.append(TreeNode(v, l, r))
            return ans

        d = defaultdict(list)
        for i, x in enumerate(inOrder):
            d[x].append(i)
        return dfs(0, 0, len(preOrder))

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def buildTree(self, preorder: list[int], inorder: list[int]) -> TreeNode:
        # Create a hashmap to store value->index for inorder traversal

```

```
inorder_index = {value: idx for idx, value in enumerate(inorder)}

# Recursive helper function to build the tree
def helper(pre_start, in_left, in_right):
    # Base case: if there are no elements to construct subtree
    if in_left > in_right:
        return None

    # Pick up pre_start element as a root
    root_val = preorder[pre_start]

    root = TreeNode(root_val)

    # Get the inorder index of the root
    in_index = inorder_index[root_val]

    # Number of nodes in left subtree
    left_subtree_size = in_index - in_left

    # Recursively build the left subtree
    root.left = helper(pre_start + 1, in_left, in_index - 1)

    # Recursively build the right subtree
    root.right = helper(pre_start + left_subtree_size + 1, in_index +
1, in_right)

    return root

# Initiate recursion from the start of preorder and full inorder range
return helper(0, 0, len(inorder) - 1)
```

• LC.ca


```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

# best
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:

        if not preorder: # or, not inorder
            return None
        v = preorder[0]
        i = inorder.index(v)
        root = TreeNode(v)
        root.left = self.buildTree(preorder[1: i+1], inorder[:i])
        root.right = self.buildTree(preorder[i+1:], inorder[i+1:])
        return root

# inorder.index(preorder_val)
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def dfs(pleft, pright, ileft, iright):
            if pleft > pright or ileft > iright:
                return None
            k = inorder.index(preorder[pleft])
            root = TreeNode(preorder[pleft])
            root.left = dfs(pleft + 1, pleft + (k - ileft), ileft, k - 1)
            root.right = dfs(pleft + 1 + (k - ileft), pright, k + 1, iright)

```

```

        return root

    n = len(preorder)
    return dfs(0, n - 1, 0, n - 1)

# build dict for val => index
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def dfs(pleft, pright, ileft,  iright):

            if pleft > pright or ileft > iright:
                return None
            v = preorder[pleft]
            k = d[v]
            root = TreeNode(v)
            root.left = dfs(pleft + 1, pleft + (k - ileft), ileft, k - 1)
            root.right = dfs(pleft + 1 + (k - ileft), pright, k + 1, iright)
            return root

        d = {v: i for i, v in enumerate(inorder)}

        n = len(preorder)
        return dfs(0, n - 1, 0, n - 1)

#####

'''
search for index, also use .index(val)
>>> a = [1,2,3,4,5]
>>> a.index(3)
2

```

```

    """
class Solution(object):
    def buildTree(self, preorder, inorder):
        """
        :type preorder: List[int]
        :type inorder: List[int]
        :rtype: TreeNode
        """

        self.preindex = 0

        ind = {v: i for i, v in enumerate(inorder)}
        head = self.dc(0, len(preorder) - 1, preorder, inorder, ind)
        return head

    def dc(self, start, end, preorder, inorder, ind):
        if start <= end:
            mid = ind[preorder[self.preindex]]
            self.preindex += 1
            root = TreeNode(inorder[mid])
            root.left = self.dc(start, mid - 1, preorder, inorder, ind)

            root.right = self.dc(mid + 1, end, preorder, inorder, ind)
            return root

#####

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left

```

```
# self.right = right
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def dfs(i: int, j: int, n: int):
            if n <= 0:
                return None
            v = preorder[i]
            k = d[v]
            l = dfs(i + 1, j, k - j)
            r = dfs(i + 1 + k - j, k + 1, n - k + j - 1)

            return TreeNode(v, l, r)

        d = {v: i for i, v in enumerate(inorder)}
        return dfs(0, 0, len(preorder))
```

◆ Quick Review

Key Insights

- Preorder traversal always starts with the root node.
- Inorder traversal splits the tree into left subtree and right subtree based on the root.
- A hash table (map/dictionary) can be used to quickly locate the index of the root in the inorder array.
- Use recursion to construct left and right subtrees.

Space & Time

Time Complexity: $O(n)$ since we process every node exactly once.

Space Complexity: $O(n)$ due to the recursion stack and the hash map used for inorder indices.

Solution

We solve the problem using a recursive divide-and-conquer approach. First, create a hash map to store each value's index in the inorder array for quick lookups. The preorder array always provides the current root. Use the inorder array to divide the tree into left and right subtrees. Recursively construct the subtrees by adjusting the preorder index and the boundaries of the inorder array. The algorithm leverages recursion and a hash map to achieve $O(n)$ time complexity.

Trees

□ #199 Binary Tree Right Side View

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS
Lists: Grind 169

Problem

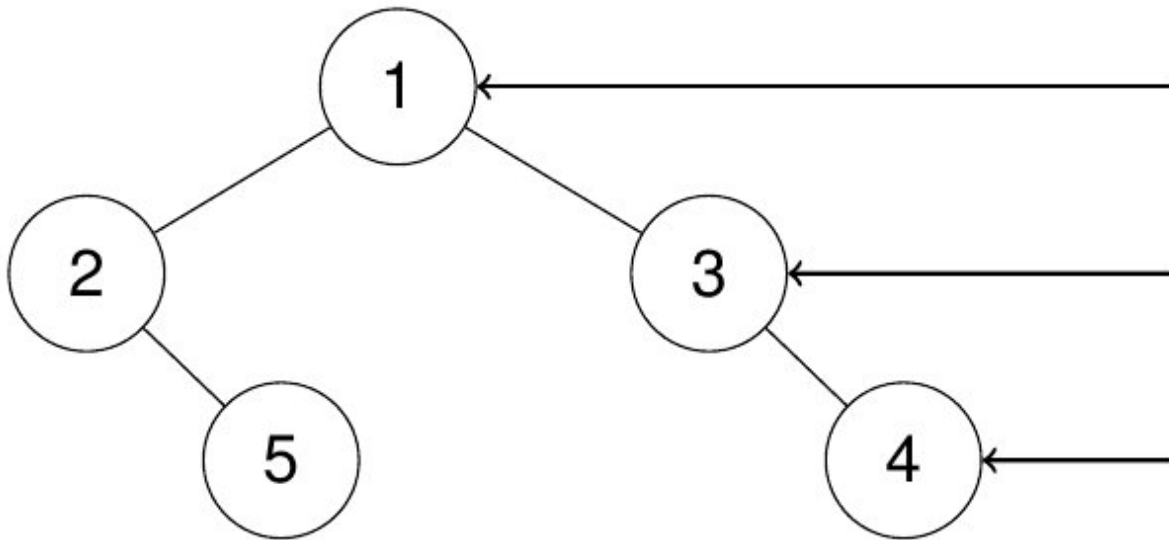
Given the root of a binary tree, imagine yourself standing on the right side of it, return the values of the nodes you can see ordered from top to bottom.

Example 1:

Input: root = [1,2,3,null,5,null,4]

Output: [1,3,4]

Explanation:

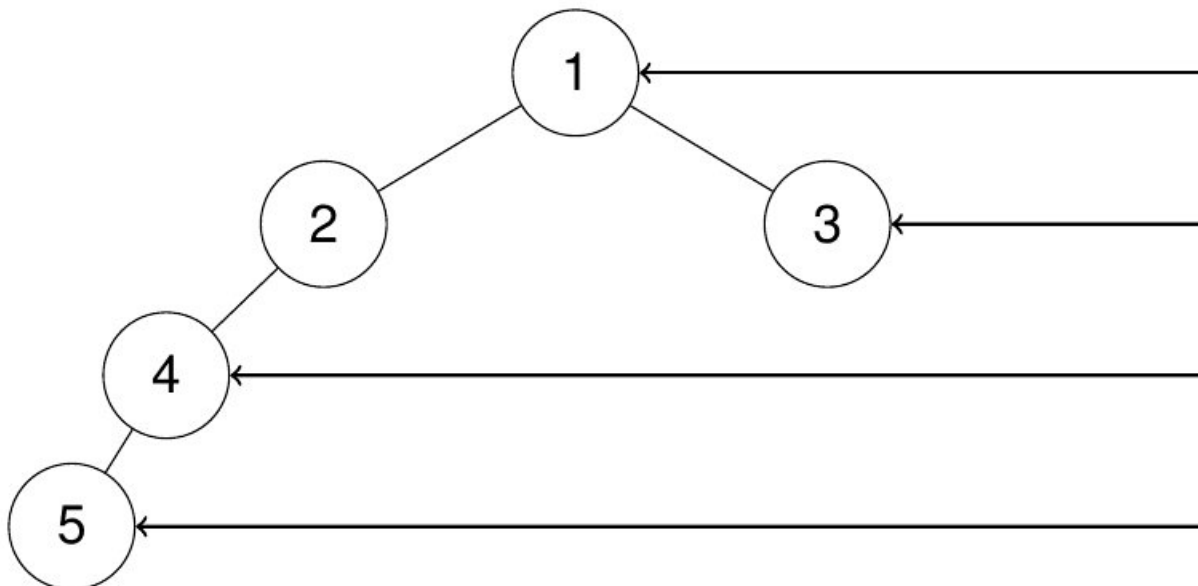


Example 2:

Input: root = [1,2,3,4,null,null,null,5]

Output: [1,3,4,5]

Explanation:



Example 3:

Input: root = [1,null,3]

Output: [1,3]

Example 4:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rightSideView(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = []
        q = collections.deque([root])

        while q:
            size = len(q)
```



```

    for i in range(size):
        root = q.popleft()
        if i == size - 1:
            ans.append(root.val)
        if root.left:
            q.append(root.left)
        if root.right:
            q.append(root.right)

    return ans

```

```

class Solution:
    def rightSideView(self, root: TreeNode | None) -> list[int]:
        ans = []

        def dfs(root: TreeNode | None, depth: int) -> None:
            if not root:
                return

            if depth == len(ans):
                ans.append(root.val)

            dfs(root.right, depth + 1)
            dfs(root.left, depth + 1)

        dfs(root, 0)
        return ans

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        if root is None:

```

```

        return ans
    q = deque([root])
    while q:
        ans.append(q[0].val)
        for _ in range(len(q)):
            node = q.popleft()
            if node.right:
                q.append(node.right)
            if node.left:
                q.append(node.left)

```

```

    return ans

```

• SimplyLeet

```

# Python solution using BFS
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # Node value
        self.left = left        # Left child
        self.right = right       # Right child

def rightSideView(root):

    # If the tree is empty, return an empty list
    if not root:
        return []

    right_view = []             # List to store right side view
    queue = deque([root])       # Initialize queue with the root node

    while queue:
        level_size = len(queue) # Number of nodes in current level
        for i in range(level_size):

```

```

node = queue.popleft() # Get the front node from the queue
# if it's the last node in the current level, add it.
if i == level_size - 1:
    right_view.append(node.val)
# add left child if exists
if node.left:
    queue.append(node.left)
# add right child if exists
if node.right:
    queue.append(node.right)

```

```

return right_view

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        while q:
            ans.append(q[-1].val) # add last node of previous level traversal
            results
            for _ in range(len(q)):
                node = q.popleft()
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)

```

```
        return ans

#####

# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

from collections import deque

class Solution(object):
    def rightSideView(self, root):
        """
        :type root: TreeNode
        :rtype: List[int]
        """

    def dfs(self, root, h):
        if root:
            if h == len(ans):
                ans.append(root.val)
                # pre-order, all the way to the right
                self.dfs(root.right, h + 1)
                self.dfs(root.left, h + 1)

    ans = []
    self.dfs(root, 0)

    return ans
```

◆ Quick Review

Key Insights

- The problem is essentially about capturing the last node at each level (or the rightmost node) in a binary tree.
- A Breadth-First Search (BFS) approach naturally works by processing nodes level by level.
- Alternatively, Depth-First Search (DFS) can be adapted by prioritizing the right subtree first, ensuring that the first node encountered at each depth is the visible one.
- The number of nodes is capped at 100, so both approaches are efficient.

Space & Time

Time Complexity: $O(n)$ – we visit every node in the tree.

Space Complexity: $O(n)$ – in the worst case, the queue (or call stack for DFS) may hold all nodes of the tree.

Solution

We can solve this problem using BFS (level-order traversal). The idea is to traverse the tree level by level using a queue. For each level, the last element added to the level represents the rightmost view from that level.

We then append that element's value to our result list. Alternatively, a DFS approach can be used where we visit the right subtree first. By keeping track of the current depth and checking if it's the first time we've encountered this depth, we can add the first node visited at that depth (which will be the rightmost one) to the result. In both approaches, appropriate data structures (a queue for BFS or recursion with a list for DFS) are used to store nodes and track the traversal state.

Trees

□ #230 Kth Smallest Element in a BST

MED

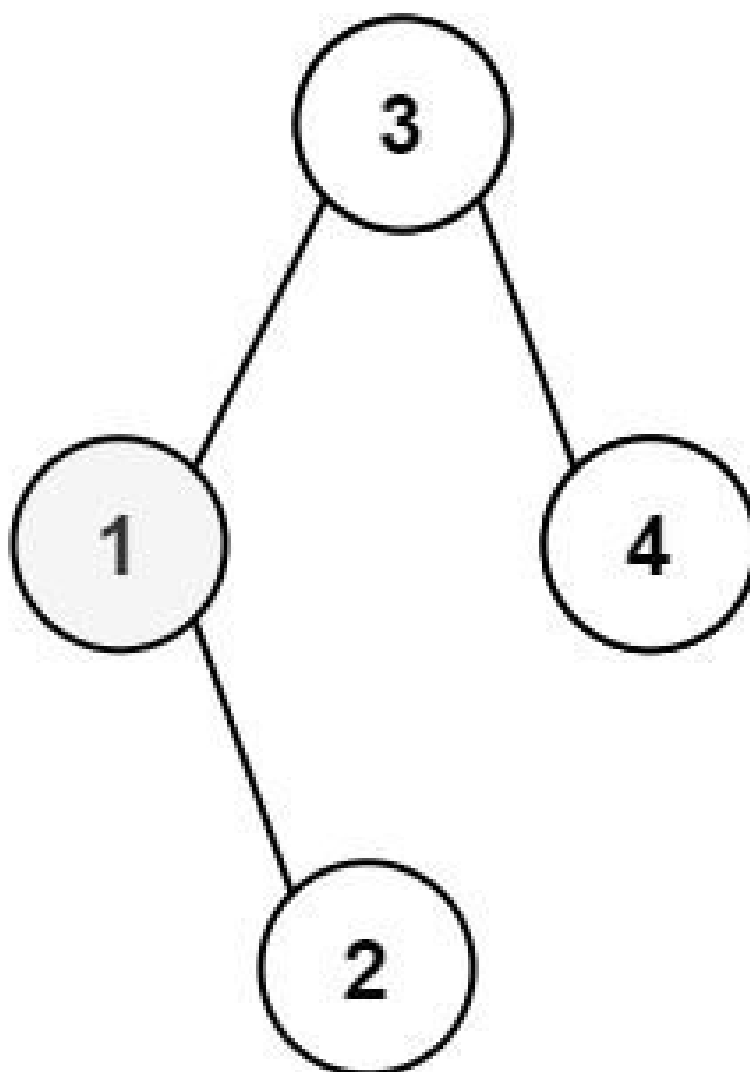
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST

Lists: Grind 169

Problem

Given the root of a binary search tree, and an integer k , return the k th smallest value (1-indexed) of all the values of the nodes in the tree.

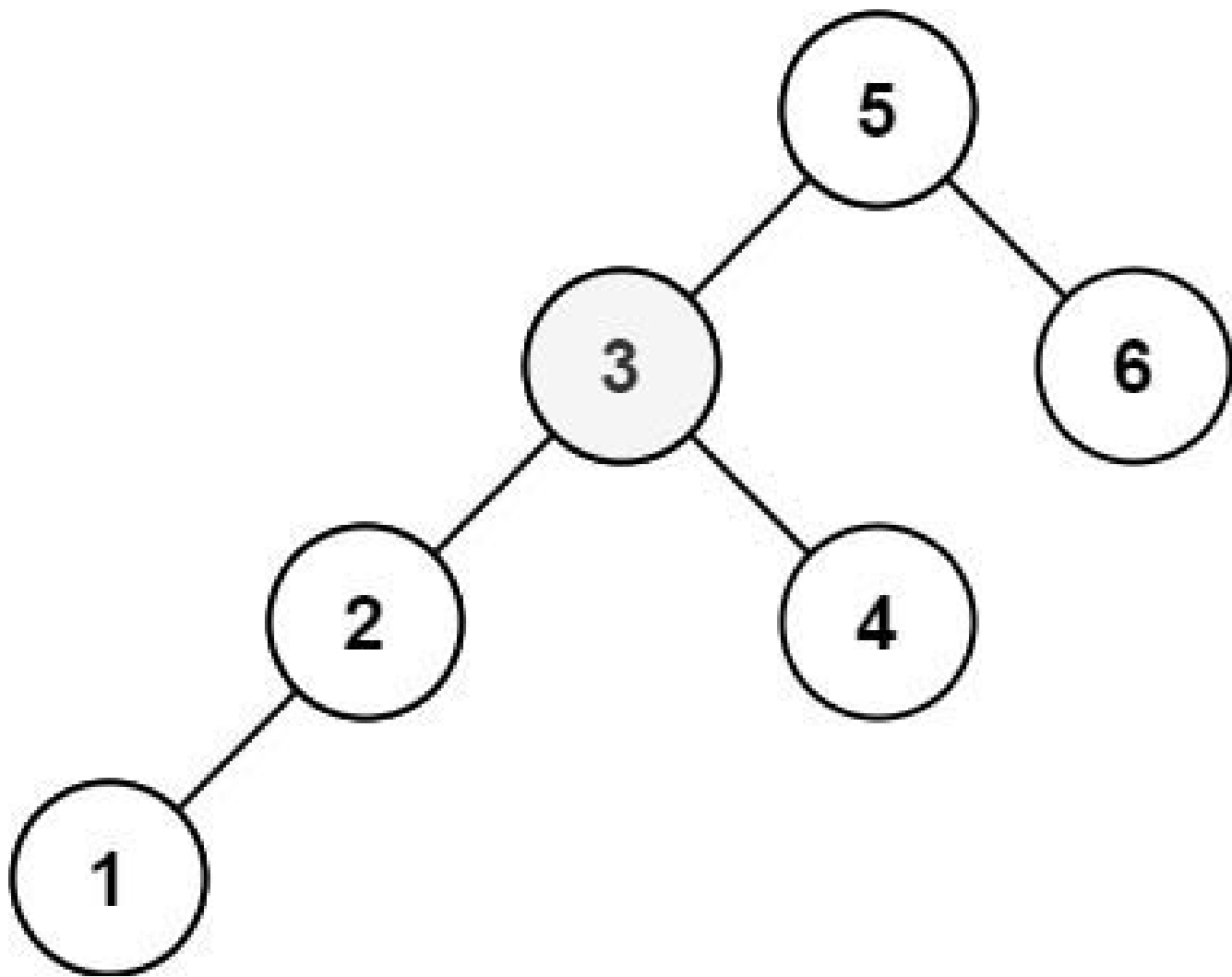
Example 1:



Input: root = [3,1,4,null,2], k = 1

Output: 1

Example 2:



Input: root = [5,3,6,2,4,null,null,1], k = 3
Output: 3

Constraints:

- The number of nodes in the tree is n.
- $1 \leq k \leq n \leq 10^4$
- $0 \leq \text{Node.val} \leq 10^4$

Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the kth smallest frequently, how would you optimize?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def kthSmallest(self, root: TreeNode | None, k: int) -> int:
        def countNodes(root: TreeNode | None) -> int:
            if not root:
                return 0
            return 1 + countNodes(root.left) + countNodes(root.right)

        leftCount = countNodes(root.left)

        if leftCount == k - 1:
            return root.val
        if leftCount >= k:
            return self.kthSmallest(root.left, k)
        return self.kthSmallest(root.right, k - 1 - leftCount) # leftCount < k

class Solution:
    def kthSmallest(self, root: TreeNode | None, k: int) -> int:
        rank = 0
        ans = 0

        def traverse(root: TreeNode | None) -> None:
            nonlocal rank
            nonlocal ans
            if not root:
                return

            traverse(root.left)
            rank += 1
            if rank == k:
                ans = root.val
                return
            traverse(root.right)

        traverse(root)
        return ans
```

```

class Solution:
    def kthSmallest(self, root: TreeNode | None, k: int) -> int:
        stack = []

        while root:
            stack.append(root)
            root = root.left

        for _ in range(k - 1):
            root = stack.pop()

            root = root.right
            while root:
                stack.append(root)
                root = root.left

        return stack[-1].val

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        stk = []
        while root or stk:

            if root:
                stk.append(root)
                root = root.left
            else:
                root = stk.pop()
                k -= 1
                if k == 0:
                    return root.val
                root = root.right

```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def kthSmallest(self, root: TreeNode, k: int) -> int:
        # Initialize counter and result

        self.k = k
        self.ans = None

        # In-order traversal: left -> node -> right
        def inorder(node):
            if not node:
                return
            # Traverse left subtree
            inorder(node.left)
            # Process current node

            self.k -= 1
            if self.k == 0:
                self.ans = node.val
                return
            # Traverse right subtree if kth element not found
            inorder(node.right)

        inorder(root)
        return self.ans
```

 • **LC.ca**

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        stk = []
        while root or stk:
```

```
            if root:
                stk.append(root)
                root = root.left
            else:
                root = stk.pop()
                k -= 1
                if k == 0:
                    return root.val
                root = root.right
```

```
#####
```

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution_followUp:
    def kthSmallest(self, root: TreeNode, k: int) -> int:
```

```
        node = self.build(root)
        return self.dfs(node, k)
```

```
class MyTreeNode:
    def __init__(self, x):
        self.val = x
        self.count = 1 # default 1
        self.left = None
        self.right = None
```

```

def build(self, root: TreeNode) -> MyTreeNode:
    if not root:
        return None
    node = self.MyTreeNode(root.val) # default count already is 1
    node.left = self.build(root.left)
    node.right = self.build(root.right)
    if node.left:
        node.count += node.left.count
    if node.right:
        node.count += node.right.count

    return node

def dfs(self, node: MyTreeNode, k: int) -> int:
    if node.left:
        cnt = node.left.count
        if k < cnt + 1:
            return self.dfs(node.left, k)
        elif k > cnt + 1:
            return self.dfs(node.right, k - 1 - cnt)
        else: # k == cnt+1
            return node.val
    else:
        if k == 1: # cannot move to beginning of dfs()
            return node.val
        return self.dfs(node.right, k - 1)

```

◆ Quick Review

Key Insights

- An in-order traversal of a BST visits nodes in increasing order.
- Use a counter to keep track of the number of nodes visited.

- As soon as the counter reaches k , the current node's value is the k th smallest.
- For frequent modifications (insertions/deletions) and queries, consider augmenting the BST with subtree counts.

Space & Time

Time Complexity: $O(n)$ in the worst-case scenario (when $k \approx n$), but on average the traversal may stop early.

Space Complexity: $O(h)$ where h is the height of the tree ($O(n)$ in the worst-case for a skewed tree).

Solution

We perform an in-order traversal of the BST which visits nodes in ascending order. During the traversal, we maintain a counter (or decrement k) to check when we have encountered the k th smallest element. When the counter reaches 0 (or k becomes 0), we capture the current node's value as our answer and return it. This approach is both simple and efficient for the given constraints.

Trees

□ #235 Lowest Common Ancestor of a Binary Search Tree

MED

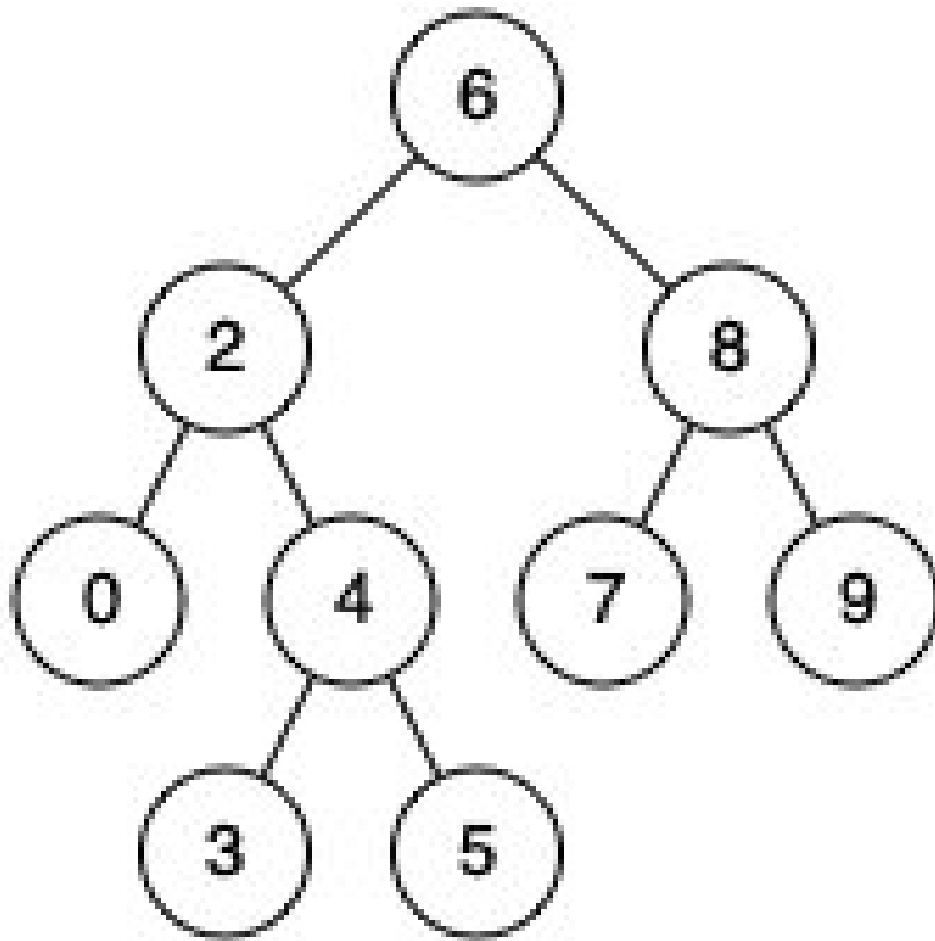
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST
Lists: Grind 169

Problem

Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Example 1:

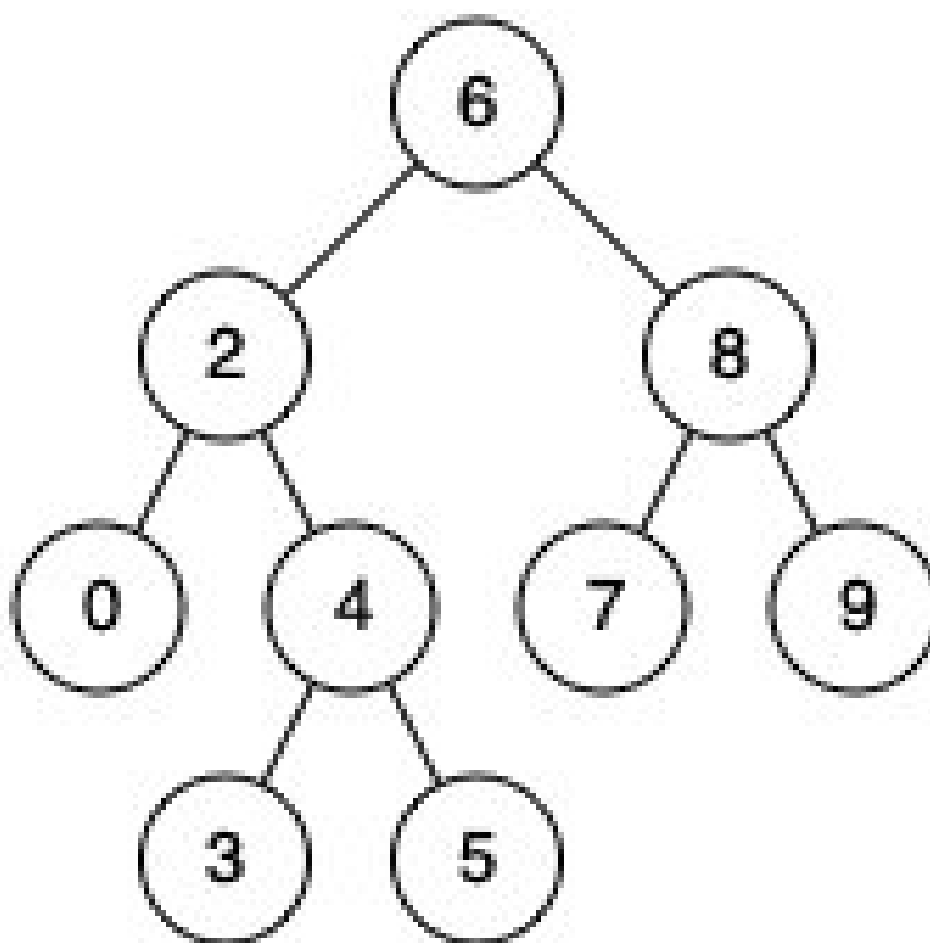


Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8

Output: 6

Explanation: The LCA of nodes 2 and 8 is 6.

Example 2:



Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4

Output: 2

Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [2,1], p = 2, q = 1

Output: 2

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- $-109 \leq \text{Node.val} \leq 109$

- All Node.val are unique.
- $p \neq q$
- p and q will exist in the BST.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lowestCommonAncestor(
        self,
        root: 'TreeNode',
        p: 'TreeNode',
        q: 'TreeNode',
    ) -> 'TreeNode':
        if root.val > max(p.val, q.val):
            return self.lowestCommonAncestor(root.left, p, q)
        if root.val < min(p.val, q.val):
            return self.lowestCommonAncestor(root.right, p, q)
        return root
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def lowestCommonAncestor(
```

```

        self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode'
    ) -> 'TreeNode':
        while 1:
            if root.val < min(p.val, q.val):
                root = root.right
            elif root.val > max(p.val, q.val):
                root = root.left
            else:
                return root

```

• SimplyLeet

```

def lowestCommonAncestor(root, p, q):
    # Iterate until we find the lowest common ancestor
    while root:
        # If both p and q are less than the current node, move to left subtree
        if p.val < root.val and q.val < root.val:
            root = root.left
        # If both are greater, move to right subtree
        elif p.val > root.val and q.val > root.val:
            root = root.right
        else:
            # Found the split point; return current node as LCA
            return root

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def lowestCommonAncestor(
        self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode'

```

```

    ) -> 'TreeNode':
        while 1:
            if root.val < min(p.val, q.val): # no =, so root is p or q is in
else block
                root = root.right
            elif root.val > max(p.val, q.val):
                root = root.left
            else:
                return root

#####

```

```

class Solution:
    def lowestCommonAncestor(self, root, p, q):
        if root is None:
            return root

        if root.val > max(p.val, q.val):
            return self.lowestCommonAncestor(root.left, p, q)
        elif root.val < min(p.val, q.val):
            return self.lowestCommonAncestor(root.right, p, q)

        else:
            return root

```

```

#####

class Solution(object):
    def lowestCommonAncestor(self, root, p, q):
        """
        :type root: TreeNode
        :type p: TreeNode

```

```
:type q: TreeNode
:rtype: TreeNode
"""
a, b = sorted([p.val, q.val])
while not a <= root.val <= b:
    if a > root.val:
        root = root.right
    else:
        root = root.left
return root
```

◆ Quick Review

Key Insights

- Utilize the BST property: for any node, values in the left subtree are less and values in the right subtree are greater.
- If both p and q have values less than the current node, the LCA must lie in the left subtree.
- If both p and q have values greater than the current node, the LCA must lie in the right subtree.
- When one node value is less than and the other is greater than (or equal to) the current node, the current node is the lowest common ancestor.
- An iterative approach is efficient and uses constant extra space.

Space & Time

Time Complexity: $O(h)$, where h is the height of the tree ($O(\log n)$ for balanced trees and $O(n)$ in the worst-case skewed tree).

Space Complexity: $O(1)$ with an iterative approach ($O(h)$ if using recursion due to call stack).

Solution

The solution leverages the BST properties.

Algorithm: Start at the root node. While the current node exists: If both p and q have values less than the current node's value, move to the left child. If both p and q have values greater than the current node's value, move to the right child. Otherwise, the current node is the split point where p and q diverge, so it is the lowest common ancestor. Return the current node. This approach efficiently finds the LCA using a single traversal from the root.

Trees

□ #1448 Count Good Nodes in Binary Tree

MED

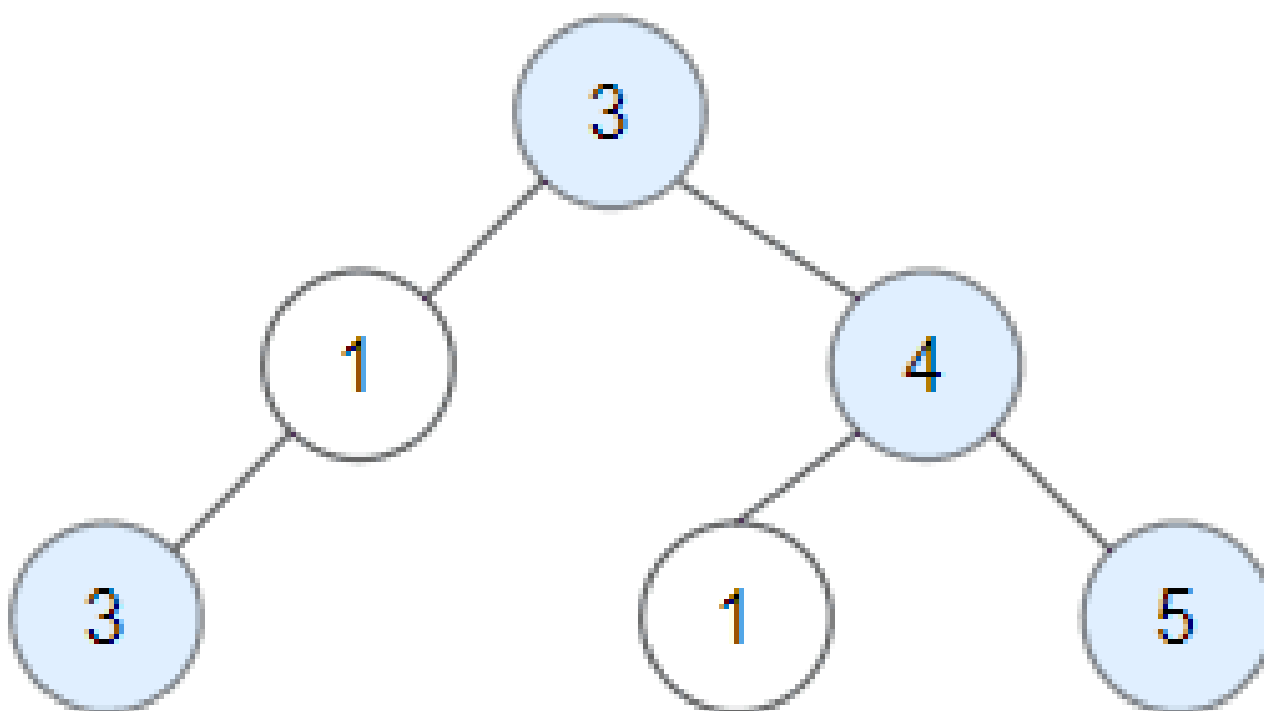
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Breadth-First
Search · Binary Tree
Lists: NeetCode 150

Problem

Given a binary tree root, a node X in the tree is named good if in the path from root to X there are no nodes with a value greater than X.

Return the number of good nodes in the binary tree.

Example 1:



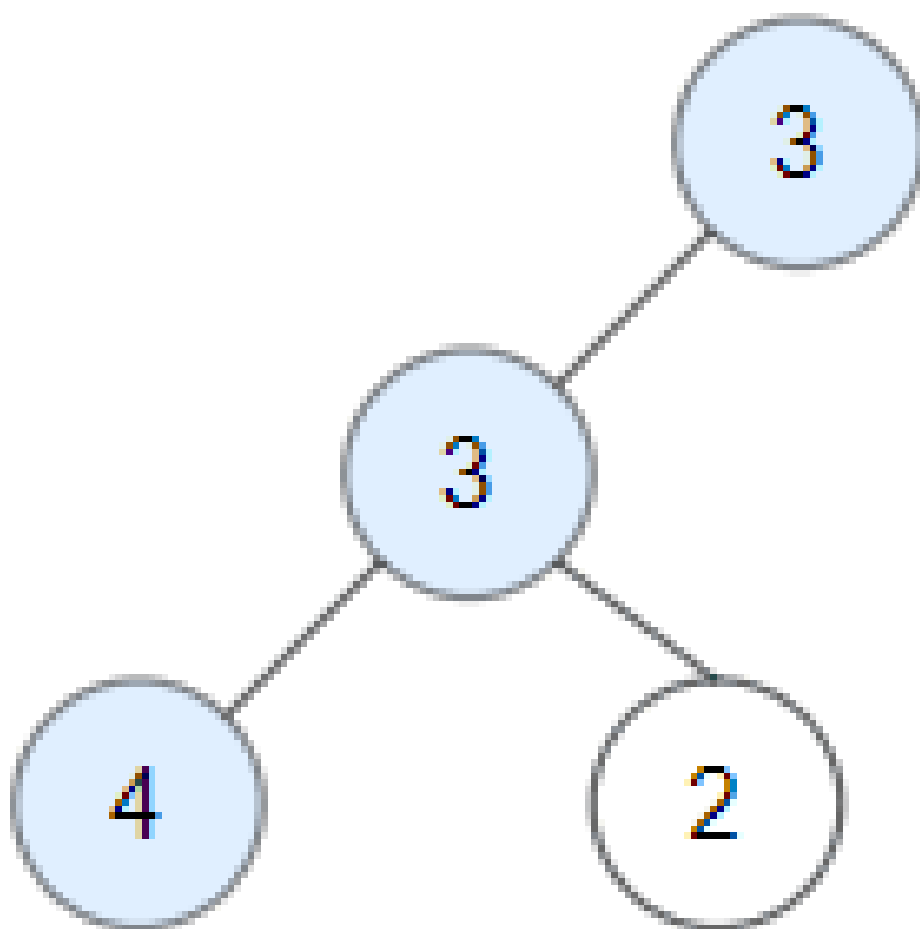
Input: root = [3,1,4,3,null,1,5]

Output: 4

Explanation: Nodes in blue are good.

Root Node (3) is always a good node.

Example 2:



Input: root = [3,3,null,4,2]

Output: 3

Explanation: Node 2 -> (3, 3, 2) is not good, because "3" is higher than it.

Example 3:

Input: root = [1]

Output: 1

Explanation: Root is considered as good.

Constraints:

- The number of nodes in the binary tree is in the range $[1, 10^5]$.
- Each node's value is between $[-10^4, 10^4]$.

Community Solutions (Python)

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def goodNodes(self, root: TreeNode) -> int:
        def dfs(root: TreeNode, mx: int):
            if root is None:
                return
            nonlocal ans
            if mx <= root.val:
                ans += 1
                mx = root.val
            dfs(root.left, mx)
            dfs(root.right, mx)

        ans = 0
        dfs(root, -1000000)

        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def goodNodes(self, root: TreeNode) -> int:
        # Helper function to perform DFS traversal.

        def dfs(node, currentMax):
            # If we reach a null node, there are no further nodes to count.
            if not node:
                return 0

            # Check if current node is good.
            count = 1 if node.val >= currentMax else 0
            # Update the current maximum value.
            newMax = max(currentMax, node.val)

            # Recursively count good nodes in the left and right subtrees.
            count += dfs(node.left, newMax)
            count += dfs(node.right, newMax)
            return count

        # Start DFS traversal from root with initial maximum value as root's
        # value.
        return dfs(root, root.val)
```

◆ Quick Review

Key Insights

- Perform a tree traversal (DFS or BFS) from the root.
- Keep track of the maximum value encountered along the current path.

- A node is "good" if its value is greater than or equal to the maximum value seen so far.
- Update the maximum value as you traverse deeper in the tree.
- The root is always a good node.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, since each node is visited exactly once.

Space Complexity: $O(h)$, where h is the height of the tree ($O(n)$ in the worst-case of a skewed tree).

Solution

We can solve this problem using Depth-First Search (DFS) recursion. Starting from the root, we pass along the current maximum value encountered on the path. For each node, we check if its value is greater than or equal to this maximum. If it is, we count it as a good node and update the maximum value if necessary. This process is continued recursively for both the left and right children of the node. Finally, we sum the counts from both subtrees.

Graphs

Round 2 · High · Medium · 12 questions

Graphs

□ #130 Surrounded Regions

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Union Find · Matrix
Lists: Denny Zhang

Problem

You are given an $m \times n$ matrix board containing letters 'X' and 'O', capture regions that are surrounded:

- **Connect:** A cell is connected to adjacent cells horizontally or vertically.
- **Region:** To form a region connect every 'O' cell.
- **Surround:** A region is surrounded if none of the 'O' cells in that region are on the edge of the board. Such regions are completely enclosed by 'X' cells.

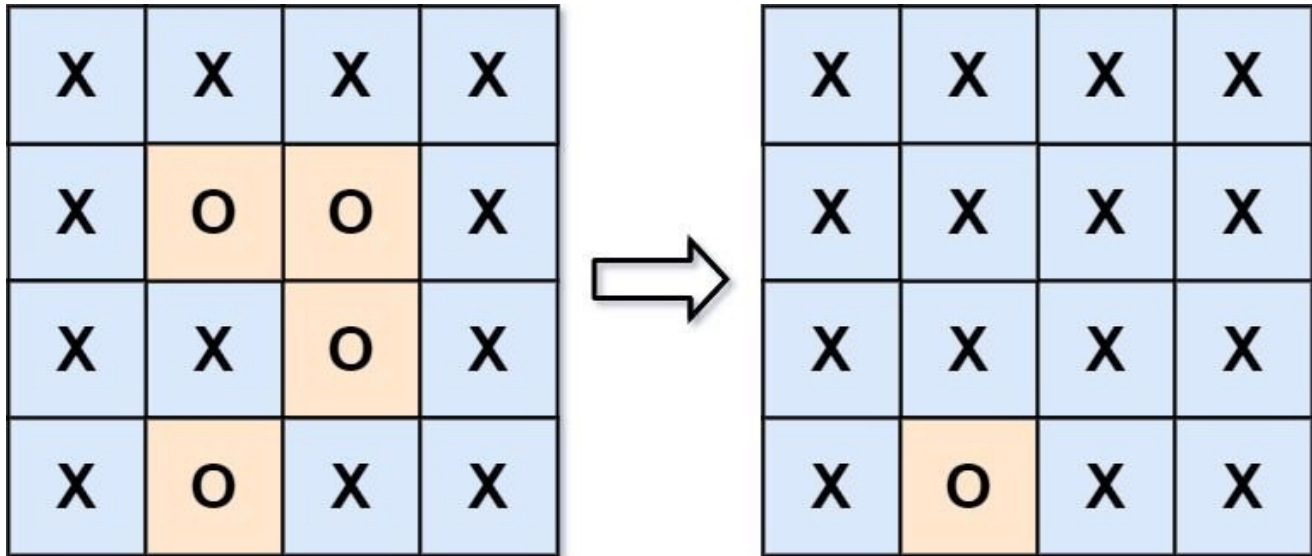
To capture a surrounded region, replace all 'O's with 'X's in-place within the original board. You do not need to return anything.

Example 1:

Input: board = `[["X","X","X","X"],["X","O","O","X"],["X","X","O","X"],["X","O","X","X"]]`

Output: `[["X","X","X","X"],["X","X","X","X"],["X","X","X","X"],["X","O","X","X"]]`

Explanation:



In the above diagram, the bottom region is not captured because it is on the edge of the board and cannot be surrounded.

Example 2:

Input: board = `[["X"]]`

Output: `[["X"]]`

Constraints:

- `m == board.length`
- `n == board[i].length`
- `1 <= m, n <= 200`

- `board[i][j]` is 'X' or 'O'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def solve(self, board: list[list[str]]) -> None:
        if not board:
            return

        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(board)
        n = len(board[0])
        q = collections.deque()

        for i in range(m):
            for j in range(n):
                if i * j == 0 or i == m - 1 or j == n - 1:
                    if board[i][j] == 'O':
                        q.append((i, j))
                        board[i][j] = '*'

        # Mark the grids that stretch from the four sides with '*'.
        while q:
            i, j = q.popleft()

            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if board[x][y] != 'O':
                    continue
                q.append((x, y))
                board[x][y] = '*'

        for row in board:
            for i, c in enumerate(row):
                row[i] = 'O' if c == '*' else 'X'
```

• LeetDoocs

```
class Solution:
    def solve(self, board: List[List[str]]) -> None:
        def dfs(i: int, j: int):
            if not (0 <= i < m and 0 <= j < n and board[i][j] == "0"):
                return
            board[i][j] = "."
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                dfs(i + a, j + b)

        m, n = len(board), len(board[0])

        for i in range(m):
            dfs(i, 0)
            dfs(i, n - 1)
        for j in range(n):
            dfs(0, j)
            dfs(m - 1, j)
        for i in range(m):
            for j in range(n):
                if board[i][j] == ".":
                    board[i][j] = "0"

                elif board[i][j] == "0":
                    board[i][j] = "X"

class Solution:
    def solve(self, board: List[List[str]]) -> None:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        m, n = len(board), len(board[0])
        p = list(range(m * n + 1))
        for i in range(m):
```

```

        for j in range(n):
            if board[i][j] == "0":
                if i in (0, m - 1) or j in (0, n - 1):
                    p[find(i * n + j)] = find(m * n)
                else:
                    for a, b in pairwise((-1, 0, 1, 0, -1)):
                        x, y = i + a, j + b
                        if board[x][y] == "0":
                            p[find(x * n + y)] = find(i * n + j)
        for i in range(m):

        for j in range(n):
            if board[i][j] == "0" and find(i * n + j) != find(m * n):
                board[i][j] = "X"

```

• SimplyLeet

```

def solve(board):
    if not board or not board[0]:
        return

    rows, cols = len(board), len(board[0])

    def dfs(r, c):
        # Return if out of bounds or current cell is not '0'
        if r < 0 or c < 0 or r >= rows or c >= cols or board[r][c] != '0':
            return

        # Mark the cell as safe
        board[r][c] = '*'
        # Explore four possible directions
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    # Traverse the board borders
    for i in range(rows):

```

```

        if board[i][0] == '0':
            dfs(i, 0)
        if board[i][cols - 1] == '0':
            dfs(i, cols - 1)
    for j in range(cols):
        if board[0][j] == '0':
            dfs(0, j)
        if board[rows - 1][j] == '0':
            dfs(rows - 1, j)

# Flip surrounded '0's and restore safe cells
for i in range(rows):
    for j in range(cols):
        if board[i][j] == '0':
            board[i][j] = 'X'
        elif board[i][j] == '*':
            board[i][j] = '0'

```

• LC.ca

```

class Solution:
    def solve(self, board: List[List[str]]) -> None:
        """
        Do not return anything, modify board in-place instead.
        """

        def dfs(i, j):
            board[i][j] = '.'
            for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
                x, y = i + a, j + b

                if 0 <= x < m and 0 <= y < n and board[x][y] == '0':
                    dfs(x, y)

        m, n = len(board), len(board[0])
        for i in range(m):
            for j in range(n):
                if board[i][j] == '0' and (
                    i == 0 or i == m - 1 or j == 0 or j == n - 1
                ):
                    dfs(i, j)

```

```
for i in range(m):
    for j in range(n):
        if board[i][j] == '0':
            board[i][j] = 'X'
        elif board[i][j] == '.':
            board[i][j] = '0'
```

◆ Quick Review

Key Insights

- Regions connected to the border are not captured.
- Use DFS/BFS from border 'O's to mark safe cells.
- After marking, flip all remaining 'O's to 'X' and then revert the marked cells back to 'O'.

Space & Time

Time Complexity: $O(m * n)$

Space Complexity: $O(m * n)$ in the worst case due to recursion stack or queue storage for DFS/BFS

Solution

The solution involves two main steps. First, traverse the board's borders and run a DFS or BFS from any encountered 'O', marking it (e.g., using ' ') to indicate that it should not be

flipped. Next, go through each cell in the board: flip any remaining 'O' (i.e., not marked safe) to 'X' and change the temporary marker ' ' back to 'O'. This approach ensures only the regions completely surrounded by 'X' are captured.

Graphs

□ #133 Clone Graph

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · DFS · BFS · Graph
Lists: Grind 169

Problem

Given a reference of a node in a connected undirected graph.

Return a deep copy (clone) of the graph.

Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.

```
class Node {  
    public int val;  
    public List<Node> neighbors;  
}
```

Test case format:

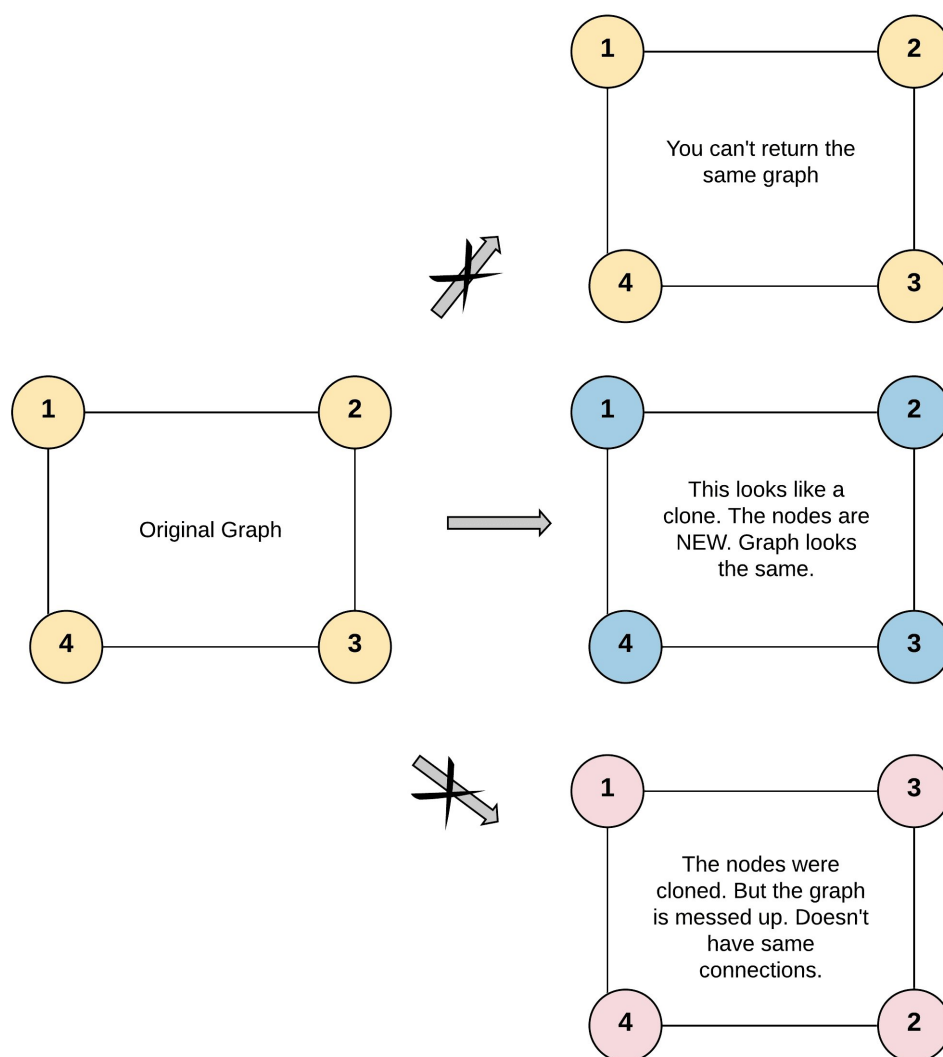
For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with `val == 1`, the second node with `val == 2`, and so on. The graph is represented in the test case using an adjacency list.

An adjacency list is a collection of unordered lists used to represent a finite graph. Each list

describes the set of neighbors of a node in the graph.

The given node will always be the first node with $val = 1$. You must return the copy of the given node as a reference to the cloned graph.

Example 1:



Input: `adjList = [[2,4],[1,3],[2,4],[1,3]]`

Output: `[[2,4],[1,3],[2,4],[1,3]]`

Explanation: There are 4 nodes in the graph.

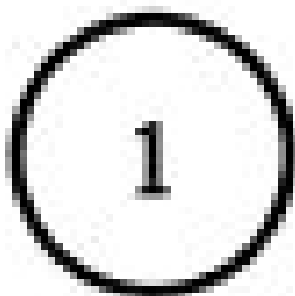
1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

Example 2:



Input: `adjList = [[]]`

Output: `[[]]`

Explanation: Note that the input contains one empty list. The graph consists of only one node with val = 1 and it does not have any neighbors.

Example 3:

Input: adjList = []

Output: []

Explanation: This an empty graph, it does not have any nodes.

Constraints:

- The number of nodes in the graph is in the range [0, 100].
- $1 \leq \text{Node.val} \leq 100$
- Node.val is unique for each node.
- There are no repeated edges and no self-loops in the graph.
- The Graph is connected and all nodes can be visited starting from the given node.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return None

        q = collections.deque([node])
        map = {node: Node(node.val)}

        while q:
            u = q.popleft()
```

```

    for v in u.neighbors:
        if v not in map:
            map[v] = Node(v.val)
            q.append(v)
            map[u].neighbors.append(map[v])

```

```

return map[node]

```

```

class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return None
        if node in self.map:
            return self.map[node]

        newNode = Node(node.val, [])
        self.map[node] = newNode

```

```

        for neighbor in node.neighbors:
            self.map[node].neighbors.append(self.cloneGraph(neighbor))

```

```

        return newNode

```

```

map = {}

```

• LeetDoocs

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val = 0, neighbors = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
"""
from typing import Optional

```

```

class Solution:
    def cloneGraph(self, node: Optional["Node"]) -> Optional["Node"]:
        def dfs(node):
            if node is None:
                return None
            if node in g:
                return g[node]
            cloned = Node(node.val)
            g[node] = cloned

            for nxt in node.neighbors:
                cloned.neighbors.append(dfs(nxt))
            return cloned

        g = defaultdict()
        return dfs(node)

```

• SimplyLeet

```

# Definition for a Node.
class Node:
    def __init__(self, val, neighbors=None):
        self.val = val
        # Initialize neighbors with empty list if None is provided
        self.neighbors = neighbors if neighbors is not None else []

def cloneGraph(node: 'Node') -> 'Node':
    # Hash map to track original nodes and their corresponding clones
    old_to_new = {}

    def dfs(current):
        # Base case: if the current node is None, return None
        if current is None:
            return None

        # If the node has already been cloned, return the clone to avoid
        # duplication and cycle issues
        if current in old_to_new:
            return old_to_new[current]

        # Create a clone for the current node
        copy = Node(current.val)

```

```

old_to_new[current] = copy
# Recursively clone all neighbors
for neighbor in current.neighbors:
    copy.neighbors.append(dfs(neighbor))
return copy

return dfs(node)

```

• LC.ca

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val = 0, neighbors = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
"""

```

```

class Solution:

```

```

    def cloneGraph(self, node: 'Node') -> 'Node':
        visited = defaultdict()

```

```

        def clone(node):
            if node is None:
                return None
            if node in visited:
                return visited[node]
            c = Node(node.val)
            visited[node] = c

```

```

            for e in node.neighbors:
                c.neighbors.append(clone(e))
            return c

```

```

        return clone(node)

```

◆ Quick Review

Key Insights

- The graph is undirected and connected.
- Each node must be cloned exactly once to prevent cycles and repetitions.
- Use a mapping/dictionary to keep track of cloned nodes.
- A depth-first search (DFS) or breadth-first search (BFS) can be used to traverse and clone the graph.
- Handle edge cases where the graph might be empty.

Space & Time

Time Complexity: $O(N + E)$, where N is the number of nodes and E is the number of edges, because each node and each edge are traversed once.

Space Complexity: $O(N)$, to store the mapping from original nodes to their clones, and for the recursion/queue used in traversal.

Solution

We employ a depth-first search (DFS) approach to clone the graph. The main idea is to create a mapping (hash table) that links each original node to its corresponding cloned node. During the DFS traversal, if a node is

encountered for the first time, a new node is created and stored in the mapping; then, the algorithm recursively clones all of its neighbors. For an already visited node, the cloned node is directly returned from the mapping. This ensures that every node is cloned only once and correctly handles cycles in the graph.

Graphs

□ #200 Number of Islands

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Union Find · Matrix
Lists: Grind 169

Problem

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return the number of islands.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
```

Example 2:


```

Input: grid = [
["1","1","0","0","0"],
["1","1","0","0","0"],
["0","0","1","0","0"],
["0","0","0","1","1"]
]
Output: 3

```

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 300$
- $\text{grid}[i][j]$ is '0' or '1'.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def numIslands(self, grid: list[list[str]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])

        def bfs(r, c):
            q = collections.deque([(r, c)])
            grid[r][c] = '2' # Mark '2' as visited.
            while q:
                i, j = q.popleft()
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if grid[x][y] != '1':
                        continue
                    q.append((x, y))
                    grid[x][y] = '2' # Mark '2' as visited.

```

```

ans = 0

for i in range(m):
    for j in range(n):
        if grid[i][j] == '1':
            bfs(i, j)
            ans += 1

return ans

```

```

class Solution:
    def numIslands(self, grid: list[list[str]]) -> int:
        m = len(grid)
        n = len(grid[0])

        def dfs(i: int, j: int) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return
            if grid[i][j] != '1':
                return

            grid[i][j] = '2' # Mark '2' as visited.
            dfs(i + 1, j)
            dfs(i - 1, j)
            dfs(i, j + 1)
            dfs(i, j - 1)

ans = 0

for i in range(m):

```

```

    for j in range(n):
        if grid[i][j] == '1':
            dfs(i, j)
            ans += 1

return ans

```

• LeetDoocs

```

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def dfs(i, j):
            grid[i][j] = '0'
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and grid[x][y] == '1':
                    dfs(x, y)

        ans = 0

```

```

        dirs = (-1, 0, 1, 0, -1)
        m, n = len(grid), len(grid[0])
        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    dfs(i, j)
                    ans += 1
        return ans

```

```

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def bfs(i, j):
            grid[i][j] = '0'
            q = deque([(i, j)])
            while q:
                i, j = q.popleft()
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == '1':

```

```

                        q.append((x, y))
                        grid[x][y] = '0'

```

```

        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        m, n = len(grid), len(grid[0])
        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    bfs(i, j)

```

```

        ans += 1
    return ans

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        dirs = (0, 1, 0)
        m, n = len(grid), len(grid[0])
        p = list(range(m * n))

        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    for a, b in pairwise(dirs):
                        x, y = i + a, j + b
                        if x < m and y < n and grid[x][y] == '1':
                            p[find(i * n + j)] = find(x * n + y)

        return sum(
            grid[i][j] == '1' and i * n + j == find(i * n + j)
            for i in range(m)
            for j in range(n)
        )

```

• SimplyLeet

```

# Function to count the number of islands in the grid
def numIslands(grid):
    # If the grid is empty, return 0
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    islands = 0

    # Depth-First Search helper to mark the entire island as visited
    def dfs(r, c):

```

```

# Check if the current cell is out of bounds or water
if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] == "0":
    return
# Mark the cell as visited by setting it to water
grid[r][c] = "0"
# Explore all four adjacent cells (up, down, left, right)
dfs(r + 1, c)
dfs(r - 1, c)
dfs(r, c + 1)
dfs(r, c - 1)

```

```

# Iterate over every cell in the grid
for r in range(rows):
    for c in range(cols):
        # If a land cell is found, conduct DFS to visit the entire island
        if grid[r][c] == "1":
            islands += 1
            dfs(r, c)

return islands

```

• LC.ca

```

# dfs
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def dfs(i, j):
            if not (0 <= i < m and 0 <= j < n and grid[i][j] == '1'):
                return
            grid[i][j] = '0'
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                dfs(x, y)

```

```

ans = 0
dirs = (-1, 0, 1, 0, -1)
m, n = len(grid), len(grid[0])
for i in range(m):
    for j in range(n):
        if grid[i][j] == '1':
            dfs(i, j)
            ans += 1
return ans

```

```
#####
```

```

# bfs
from collections import deque

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        if not grid:
            return 0

```

```

m, n = len(grid), len(grid[0])
directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]
islands = 0

for i in range(m):
    for j in range(n):
        if grid[i][j] == "1":
            islands += 1
            q = deque([(i, j)]) # no need to reset q, q already drained
from previous bfs

```

```

        while q:
            x, y = q.popleft()
            for dx, dy in directions:
                nx, ny = x + dx, y + dy
                if 0 <= nx < m and 0 <= ny < n and grid[nx][ny] ==
"1":
                    q.append((nx, ny))
                    grid[nx][ny] = "0" # mark as visited

        return islands

#####

```

```

# union find
# similar to UF in https://leetcode.ca/2016-09-30-305-Number-of-Islands-II/
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def check(i, j):
            return 0 <= i < m and 0 <= j < n and grid[i][j] == "1" # Check
for "1" instead of 1

```

```

    def find(x):
        if p[x] != x:

```

```

            p[x] = find(p[x])
        return p[x]

```

```

m, n = len(grid), len(grid[0])
p = list(range(m * n))
cur = 0 # Initialize cur as 0 instead of n
for i in range(m):
    for j in range(n):
        if grid[i][j] == "1":
            cur += 1 # Increment cur when encountering "1"

```

```

        for x, y in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            if check(i + x, j + y) and find(i * n + j) != find((i
+ x) * n + j + y):
                p[find(i * n + j)] = find((i + x) * n + j + y)
                cur -= 1

    return cur

```

```
#####
```

```

'''
>>> a = set()
>>>
>>> a |= {(1, 1)}
>>> a
{(1, 1)}
>>>
>>> a |= {(2, 2)}
>>> a
{(1, 1), (2, 2)}

```

```

>>>
>>> a.add((3, 3))
>>> a
{(1, 1), (3, 3), (2, 2)}
>>>
>>> (1,1) in a
True
>>> (10,10) in a
False
'''

```



```
class Solution(object):
    def numIslands(self, grid):
        """
        :type grid: List[List[str]]
        :rtype: int
        """
        visited = set()
        ans = 0

        def dfs(grid, i, j, visited):

            if i < 0 or i >= len(grid) or j < 0 or j >= len(grid[0]) or grid[i][j] =
            = "0" or (i, j) in visited:
                return False
            visited |= {(i, j)}
            for di, dj in [(-1, 0), (1, 0), (0, 1), (0, -1)]:
                newi, newj = i + di, j + dj
                dfs(grid, newi, newj, visited)
            return True

        for i in range(0, len(grid)):
            for j in range(0, len(grid[0])):

                if dfs(grid, i, j, visited):
                    ans += 1
        return ans
```

◆ Quick Review

Key Insights

- Use graph traversal (DFS/BFS) to explore connected components of land.
- Iterate over the matrix and start a traversal (e.g., DFS) when encountering an unvisited "1".
- During traversal, mark visited cells to avoid duplicate counting.

- Each DFS/BFS call completely explores one island.
- Alternative approach can use Union Find to group connected lands.

Space & Time

Time Complexity: $O(m * n)$, as each cell is visited at most once.

Space Complexity: $O(m * n)$ in the worst-case scenario due to recursion stack (DFS) or auxiliary data structures.

Solution

We solve the problem using a DFS-based approach. The algorithm iterates over each cell in the grid; when a "1" (land) is encountered, the DFS is initiated to mark the entire island (all connected "1"s) as visited (by setting them to "0"). This prevents recounting the same island. The island count is incremented each time a new island is discovered and fully explored.

Graphs

□ #207 Course Schedule

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · BFS · Graph · Topological Sort
Lists: Grind 169

Problem

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course 0 you have to first take course 1.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`

Output: `true`

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: numCourses = 2, prerequisites = [[1,0],[0,1]]

Output: false

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Constraints:

- $1 \leq \text{numCourses} \leq 2000$
- $0 \leq \text{prerequisites.length} \leq 5000$
- $\text{prerequisites}[i].\text{length} == 2$
- $0 \leq a_i, b_i < \text{numCourses}$
- All the pairs $\text{prerequisites}[i]$ are unique.

Community Solutions (Python)

• WalkCC

```
from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:
```

```

def canFinish(self, numCourses: int, prerequisites: list[list[int]]) -> bool
:
    graph = [[] for _ in range(numCourses)]
    states = [State.INIT] * numCourses

    for v, u in prerequisites:
        graph[u].append(v)

    def hasCycle(u: int) -> bool:
        if states[u] == State.VISITING:
            return True

        if states[u] == State.VISITED:
            return False
        states[u] = State.VISITING
        if any(hasCycle(v) for v in graph[u]):
            return True
        states[u] = State.VISITED
        return False

    return not any(hasCycle(i) for i in range(numCourses))

```

• LeetDoocs

```

class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        g = [[] for _ in range(numCourses)]
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1
        q = [i for i, x in enumerate(indeg) if x == 0]
        for i in q:
            numCourses -= 1

            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return numCourses == 0

```

• SimplyLeet

```

from collections import deque

def canFinish(numCourses, prerequisites):
    # Create a list to hold the number of prerequisites (indegree) for each
    # course.
    indegree = [0] * numCourses
    # Create an adjacency list for each course.
    graph = {i: [] for i in range(numCourses)}

    # Build the graph and update indegree for each course.
    for course, prereq in prerequisites:

        graph[prereq].append(course)
        indegree[course] += 1

    # Initialize a queue with courses that have no prerequisites.
    queue = deque([i for i in range(numCourses) if indegree[i] == 0])

    # Counter for courses that can be completed.
    visited = 0

    while queue:

        course = queue.popleft() # Remove course with no remaining
        # prerequisites.
        visited += 1 # Increment count of completed courses.
        # For each neighbor course, reduce its indegree.
        for neighbor in graph[course]:
            indegree[neighbor] -= 1 # Remove dependency.
            # If no more prerequisites, add to queue.
            if indegree[neighbor] == 0:
                queue.append(neighbor)

    # If we processed all courses, it's possible to finish.

    return visited == numCourses

# Example usage:
print(canFinish(2, [[1,0]])) # Expected output: True
print(canFinish(2, [[1,0],[0,1]])) # Expected output: False

```

• LC.ca

```

...
>>> from collections import defaultdict

>>> a = defaultdict(int)
>>> a
defaultdict(<type 'int'>, {})
>>>
>>> a['hehehe']
0
>>> a
defaultdict(<type 'int'>, {'hehehe': 0})

>>> a = defaultdict(list)
>>> a
defaultdict(<type 'list'>, {})
>>> a['hehehe']
[]
>>> a
defaultdict(<type 'list'>, {'hehehe': []})

```

```

...
class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        g = defaultdict(list)
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1
        cnt = 0
        q = deque([i for i, v in enumerate(indeg) if v == 0])

```

```
while q:
    i = q.popleft()
    cnt += 1
    for j in g[i]:
        indeg[j] -= 1
        if indeg[j] == 0:
            q.append(j)
return cnt == numCourses
```

◆ Quick Review

Key Insights

- This problem can be modeled as a directed graph where courses are nodes and prerequisites are edges.
- If the graph has a cycle, then it is not possible to complete all courses.
- We can detect cycles using either Topological Sort (BFS/Kahn's algorithm) or DFS-based cycle detection.
- The time complexity is $O(V + E)$, where V is the number of courses and E is the number of prerequisite pairs.

Space & Time

Time Complexity: $O(V + E)$

Space Complexity: $O(V + E)$

Solution

We solve the problem using the Topological Sort algorithm (BFS/Kahn's algorithm). First, we build a graph (using an adjacency list) and track the indegree (number of prerequisites) for each course. We then find all courses with an indegree of zero (courses with no prerequisites) and add them to a processing queue. While the queue is not empty, we remove a course from the queue, reduce the indegree for its neighboring courses, and if any neighbor's indegree becomes zero, we add it to the queue as well. If we successfully process all courses, then it means that the graph was acyclic and a valid schedule exists, returning true. If not, a cycle exists, and we return false.

Graphs

□ #210 Course Schedule II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · BFS · Graph · Topological Sort
Lists: Grind 169

Problem

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course 0 you have to first take course 1.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`

Output: `[0,1]`

Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is `[0,1]`.

Example 2:

Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]

Output: [0,2,1,3]

Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.

So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].

Example 3:

Input: numCourses = 1, prerequisites = []

Output: [0]

Constraints:

- $1 \leq \text{numCourses} \leq 2000$
- $0 \leq \text{prerequisites.length} \leq \text{numCourses} * (\text{numCourses} - 1)$
- $\text{prerequisites}[i].\text{length} == 2$
- $0 \leq a_i, b_i < \text{numCourses}$
- $a_i \neq b_i$
- All the pairs $[a_i, b_i]$ are distinct.

Community Solutions (Python)

• WalkCC

```
from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:
```

```
def findOrder(
    self,
    numCourses: int,
    prerequisites: list[list[int]],
) -> list[int]:
    ans = []
    graph = [[] for _ in range(numCourses)]
    states = [State.INIT] * numCourses

    for v, u in prerequisites:
```

```
    graph[u].append(v)
```

```
def hasCycle(u: int) -> bool:
    if states[u] == State.VISITING:
        return True
    if states[u] == State.VISITED:
        return False
    states[u] = State.VISITING
    if any(hasCycle(v) for v in graph[u]):
        return True
```

```
    states[u] = State.VISITED
    ans.append(u)
    return False
```

```
if any(hasCycle(i) for i in range(numCourses)):
    return []
```

```
return ans[::-1]
```

```
class Solution:
    def findOrder(
        self,
        numCourses: int,
        prerequisites: list[list[int]],
    ) -> list[int]:
        ans = []
        graph = [[] for _ in range(numCourses)]
        inDegrees = [0] * numCourses
        q = collections.deque()
```

```

# Build the graph.
for v, u in prerequisites:
    graph[u].append(v)
    inDegrees[v] += 1

# Perform topological sorting.
q = collections.deque([i for i, d in enumerate(inDegrees) if d == 0])

while q:

    u = q.popleft()
    ans.append(u)
    for v in graph[u]:
        inDegrees[v] -= 1
        if inDegrees[v] == 0:
            q.append(v)

return ans if len(ans) == numCourses else []

```

• LeetDoocs

```

class Solution:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
        g = defaultdict(list)
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1
        ans = []
        q = deque(i for i, x in enumerate(indeg) if x == 0)
        while q:

            i = q.popleft()
            ans.append(i)
            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return ans if len(ans) == numCourses else []

```

• SimplyLeet

```
# Python solution using BFS (Kahn's Algorithm)
from collections import deque

def findOrder(numCourses, prerequisites):
    # Build the graph and the in-degree array
    in_degree = [0] * numCourses
    graph = {i: [] for i in range(numCourses)}
    for course, pre in prerequisites:
        graph[pre].append(course)
        in_degree[course] += 1

    # Initialize the queue with all courses having zero in-degree
    queue = deque([i for i in range(numCourses) if in_degree[i] == 0])
    order = []

    # Process courses
    while queue:
        course = queue.popleft()
        order.append(course)
        for neighbor in graph[course]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    # If the ordering includes all courses, return it; otherwise, return an
    # empty list
    return order if len(order) == numCourses else []

# Example usage:
# print(findOrder(4, [[1,0],[2,0],[3,1],[3,2]]))
```

• LC.ca

```

class Solution:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
        g = defaultdict(list)
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1
        q = deque([i for i, v in enumerate(indeg) if v == 0])
        ans = [] # added from previous question
        while q:
            i = q.popleft()
            ans.append(i)
            # assumption is only one path
            # as in question 'You may assume that there are no duplicate edges
            # in the input prerequisites.'
            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return ans if len(ans) == numCourses else []

```

◆ Quick Review

Key Insights

- This is a topological sort problem on a directed graph.
- The courses are nodes, and prerequisites form directed edges.
- We can solve the problem by either:
- Using DFS with cycle detection and a stack for topological ordering.

- Using BFS (Kahn's Algorithm) to process nodes with zero in-degrees.
- If at any point no node with a zero in-degree is available, a cycle exists and no valid order can be produced.

Space & Time

Time Complexity: $O(V + E)$ where V is the number of courses and E is the number of prerequisite pairs.

Space Complexity: $O(V + E)$ for storing the graph and in-degree array.

Solution

We treat the problem as a topological sort on a directed acyclic graph (DAG). For a BFS approach using Kahn's algorithm: Build an adjacency list for the graph and an in-degree array to record the number of prerequisites (incoming edges) for each course. Initialize a queue with all courses that have an in-degree of zero (i.e., courses that can be taken without prerequisites). Dequeue nodes from the queue, add them to the ordering, and reduce the in-degree of their neighbors. If a neighbor's in-degree becomes zero, add it to the queue. If

the final ordering includes all courses, return the ordering; otherwise, return an empty array indicating a cycle. The chosen data structures are: A list (or array) for the in-degree counts. A dictionary (or array of lists) for the adjacency list. A queue to process nodes with zero in-degree.

Graphs

□ ★ #261 Graph Valid Tree ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · BFS · Union Find · Graph

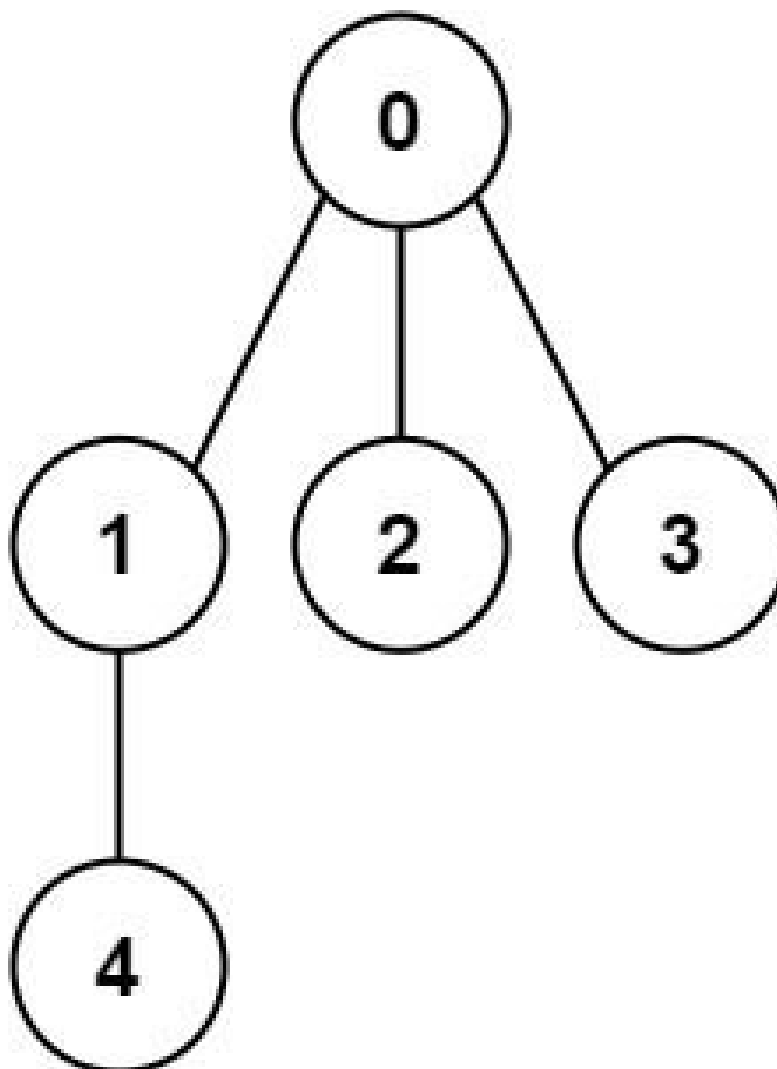
Lists: ★ Premium | Grind 169 | Premium 98

Problem

You have a graph of n nodes labeled from 0 to $n - 1$. You are given an integer n and a list of edges where $\text{edges}[i] = [a_i, b_i]$ indicates that there is an undirected edge between nodes a_i and b_i in the graph.

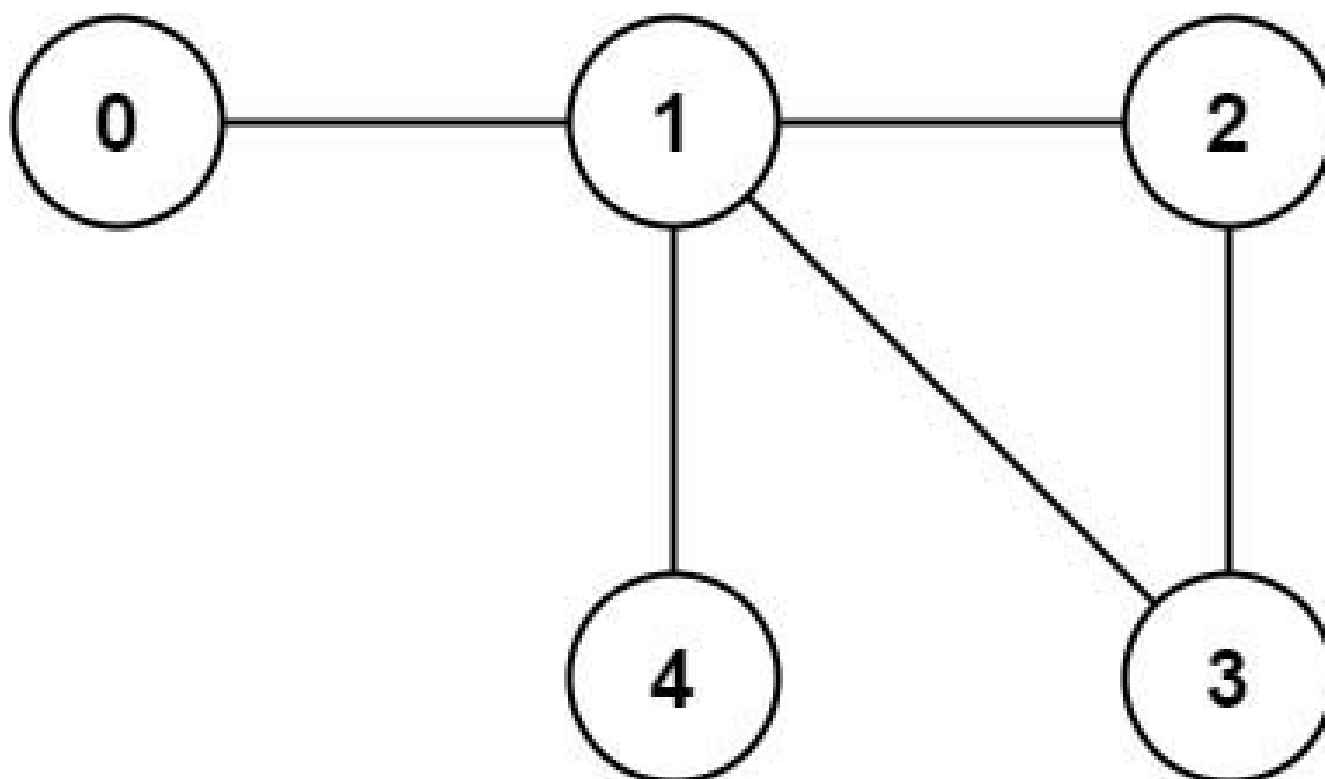
Return true if the edges of the given graph make up a valid tree, and false otherwise.

Example 1:



Input: $n = 5$, $edges = [[0,1],[0,2],[0,3],[1,4]]$
Output: true

Example 2:



Input: $n = 5$, $edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]$
Output: false

Constraints:

- $1 \leq n \leq 2000$
- $0 \leq edges.length \leq 5000$
- $edges[i].length == 2$
- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- There are no self-loops or repeated edges.

Community Solutions (Python)

- WalkCC

```
class Solution:
    def validTree(self, n: int, edges: list[list[int]]) -> bool:
        if n == 0 or len(edges) != n - 1:
            return False

        graph = [[] for _ in range(n)]
        q = collections.deque([0])
        seen = {0}

        for u, v in edges:

            graph[u].append(v)
            graph[v].append(u)

        while q:
            u = q.popleft()
            for v in graph[u]:
                if v not in seen:
                    q.append(v)
                    seen.add(v)

        return len(seen) == n

class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
```

```

        return
    if self.rank[i] < self.rank[j]:
        self.id[i] = j
    elif self.rank[i] > self.rank[j]:
        self.id[j] = i
    else:
        self.id[i] = j
        self.rank[j] += 1
    self.count -= 1

```

```

def _find(self, u: int) -> int:
    if self.id[u] != u:
        self.id[u] = self._find(self.id[u])
    return self.id[u]

```

```

class Solution:
    def validTree(self, n: int, edges: list[list[int]]) -> bool:
        if n == 0 or len(edges) != n - 1:
            return False

```

```

        uf = UnionFind(n)

        for u, v in edges:
            uf.unionByRank(u, v)

        return uf.count == 1

```

• LeetDoocs

```

class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(n))
        for a, b in edges:
            pa, pb = find(a), find(b)

```

```

        if pa == pb:
            return False
        p[pa] = pb
        n -= 1
    return n == 1

```

```

class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        def dfs(i: int):
            vis.add(i)
            for j in g[i]:
                if j not in vis:
                    dfs(j)

        if len(edges) != n - 1:
            return False

```

```

        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        vis = set()
        dfs(0)
        return len(vis) == n

```

• SimplyLeet

Python solution with DFS

```

def validTree(n, edges):
    # If the number of edges is not exactly n - 1, it cannot be a tree
    if len(edges) != n - 1:
        return False

    # Build the graph as an adjacency list
    graph = {i: [] for i in range(n)}
    for a, b in edges:

```

```

graph[a].append(b)
graph[b].append(a)

visited = set()

# DFS helper function that accepts current node and its parent node
def dfs(node, parent):
    # Add the current node to visited
    visited.add(node)
    # Traverse all the neighboring nodes

    for neighbor in graph[node]:
        # Skip the neighbor if it is the parent to avoid trivial cycle
        detection
        if neighbor == parent:
            continue
        # If neighbor is already visited, a cycle is detected
        if neighbor in visited:
            return False
        # Recurse deeper; if cycle is found in recursion, propagate False
        upward
        if not dfs(neighbor, node):
            return False

    return True

# Start DFS from node 0, with parent set to -1 (non-existent)
if not dfs(0, -1):
    return False

# Check if all nodes were visited (i.e., the graph is connected)
return len(visited) == n

# Example usage:

print(validTree(5, [[0, 1], [0, 2], [0, 3], [1, 4]])) # Expected output: True
print(validTree(5, [[0, 1], [1, 2], [2, 3], [1, 3], [1, 4]])) # Expected
output: False

```

• LC.ca


```
class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(n))
        for a, b in edges:
            if find(a) == find(b):
                return False
            p[find(a)] = find(b)
            n -= 1
        return n == 1
```

◆ Quick Review

Key Insights

- A tree must be acyclic and connected.
- For n nodes, a valid tree must have exactly $n - 1$ edges.
- A DFS/BFS traversal can check connectivity and simultaneously detect cycles.
- The Union-Find (disjoint set) approach provides an alternative for cycle detection and connectivity verification.

Space & Time

Time Complexity: $O(n + e)$, where e is the number of edges

Space Complexity: $O(n + e)$ for storing the graph and traversal/union–find structures

Solution

We can solve the problem using either a DFS/BFS approach or a Union–Find algorithm. For the DFS/BFS method, we first check if the number of edges is exactly $n - 1$. If not, it cannot be a tree. Then, we build an adjacency list for the graph and perform a DFS starting from node 0 while keeping track of visited nodes and parent information to avoid false–positive cycle detections. If during DFS we encounter a node that has been visited (and is not the parent), then a cycle exists. Finally, if all nodes are reached from node 0 and no cycle is found, the graph is a valid tree. Using Union–Find, we iterate over each edge and try to union the two nodes. If they already share the same parent (i.e., are in the same set), a cycle exists. After processing all edges, we confirm that exactly one connected component exists.

Graphs

□ ★ #286 Walls and Gates ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix

Lists: ★ Premium | Premium 98

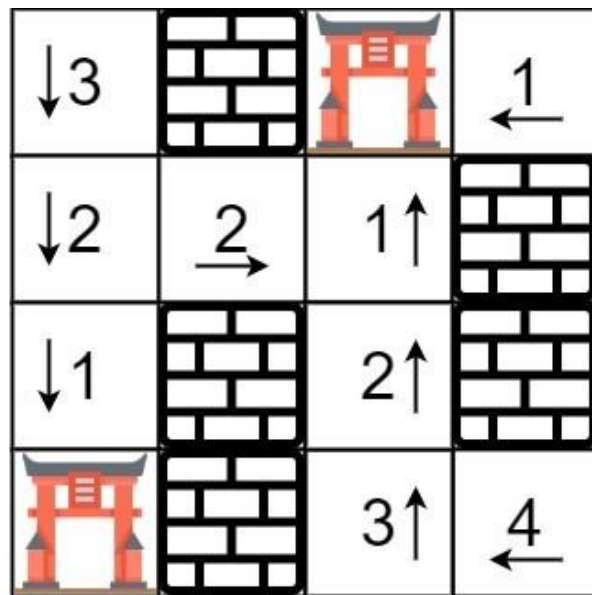
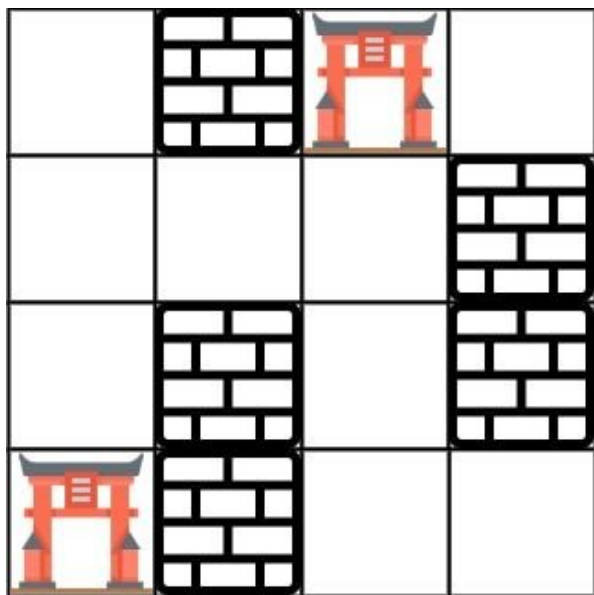
Problem

You are given an $m \times n$ grid rooms initialized with these three possible values.

- -1 A wall or an obstacle.
- 0 A gate.
- INF Infinity means an empty room. We use the value $2^{31} - 1 = 2147483647$ to represent INF as you may assume that the distance to a gate is less than 2147483647.

Fill each empty room with the distance to its nearest gate. If it is impossible to reach a gate, it should be filled with INF.

Example 1:



Input: rooms = `[[2147483647,-1,0,2147483647],[2147483647,2147483647,2147483647,-1],[2147483647,-1,2147483647,-1],[0,-1,2147483647,2147483647]]`
 Output: `[[3,-1,0,1],[2,2,1,-1],[1,-1,2,-1],[0,-1,3,4]]`

Example 2:

Input: rooms = `[[-1]]`
 Output: `[[-1]]`

Constraints:

- `m == rooms.length`
- `n == rooms[i].length`
- `1 <= m, n <= 250`
- `rooms[i][j]` is `-1`, `0`, or `231 - 1`.

Community Solutions (Python)

- WalkCC

```

class Solution:
    def wallsAndGates(self, rooms: list[list[int]]) -> None:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        INF = 2**31 - 1
        m = len(rooms)
        n = len(rooms[0])
        q = collections.deque((i, j)
                               for i in range(m)
                               for j in range(n)
                               if rooms[i][j] == 0)

        while q:
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if rooms[x][y] != INF:
                    continue

                rooms[x][y] = rooms[i][j] + 1
                q.append((x, y))

```

• LeetDoocs

```

class Solution:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        """
        Do not return anything, modify rooms in-place instead.
        """
        m, n = len(rooms), len(rooms[0])
        inf = 2**31 - 1
        q = deque([(i, j) for i in range(m) for j in range(n) if rooms[i][j] == 0])
        d = 0
        while q:

```

```

d += 1
for _ in range(len(q)):
    i, j = q.popleft()
    for a, b in [[0, 1], [0, -1], [1, 0], [-1, 0]]:
        x, y = i + a, j + b
        if 0 <= x < m and 0 <= y < n and rooms[x][y] == inf:
            rooms[x][y] = d
            q.append((x, y))

```

• SimplyLeet

```

from collections import deque

def wallsAndGates(rooms):
    if not rooms or not rooms[0]:
        return

    rows, cols = len(rooms), len(rooms[0])
    # Directions: up, down, left, right.
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    queue = deque()

    # Enqueue all gates (cells with 0).
    for r in range(rows):
        for c in range(cols):
            if rooms[r][c] == 0:
                queue.append((r, c))

    # Perform multi-source BFS.
    while queue:
        r, c = queue.popleft()

        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            # Process neighbor if it is within bounds and is an empty room.
            if 0 <= nr < rows and 0 <= nc < cols and rooms[nr][nc] == 21474836
47:
                rooms[nr][nc] = rooms[r][c] + 1 # Update distance.
                queue.append((nr, nc))

```

• LC.ca

```

class Solution:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        """
        Do not return anything, modify rooms in-place instead.
        """
        m, n = len(rooms), len(rooms[0])
        inf = 2**31 - 1
        q = deque([(i, j) for i in range(m) for j in range(n) if rooms[i][j] == 0])

        d = 0
        while q:

            d += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in [[0, 1], [0, -1], [1, 0], [-1, 0]]:
                    x, y = i + a, j + b
                    # bfs so first reaching inf is always the shortest
                    # set its value from inf to d, same as marking as visited
                    if 0 <= x < m and 0 <= y < n and rooms[x][y] == inf:
                        rooms[x][y] = d
                        q.append((x, y))

#####

class Solution: # dfs
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        if not rooms or not rooms[0]:
            return

        m, n = len(rooms), len(rooms[0])
        isVisited = [[False] * n for _ in range(m)] # isVisited map shared
        among all gates

```

```

def fill(i: int, j: int, distance: int) -> None:
    nonlocal m, n

    # rooms[i][j] <= 0 meaning == -1 or == 0, both stop
    if i < 0 or i >= m or j < 0 or j >= n or rooms[i][j] <= 0 or isVis
ited[i][j]:
        return

    rooms[i][j] = min(rooms[i][j], distance)
    isVisited[i][j] = True

    fill(i - 1, j, distance + 1)
    fill(i, j + 1, distance + 1)
    fill(i + 1, j, distance + 1)
    fill(i, j - 1, distance + 1)

    isVisited[i][j] = False

for i in range(m):
    for j in range(n):

        if rooms[i][j] == 0: # start from every gate
            fill(i, j, 0)

#####

from collections import deque

INF = 2147483647

```



```

class Solution(object):
    def wallsAndGates(self, rooms):
        """
        :type rooms: List[List[int]]
        :rtype: void Do not return anything, modify rooms in-place instead.
        """
        queue = deque([])
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        for i in range(0, len(rooms)):
            for j in range(0, len(rooms[0])):
                if rooms[i][j] == 0:
                    queue.append((i, j))

        while queue:
            i, j = queue.popleft()
            for di, dj in directions:
                p, q = i + di, j + dj
                if 0 <= p < len(rooms) and 0 <= q < len(rooms[0]) and rooms[p][q] == INF:
                    rooms[p][q] = rooms[i][j] + 1
                    queue.append((p, q))

```

◆ Quick Review

Key Insights

- Use a multi-source Breadth-First Search (BFS) starting from all the gates, as this allows spreading the shortest distance to each empty room.
- Instead of starting from every empty room separately, starting from all gates simultaneously helps update all distances in one pass.

- Only update a room if the new calculated distance is smaller than its current value to prevent unnecessary processing.
- The problem is essentially a multi-source shortest path problem on a grid.

Space & Time

Time Complexity: $O(m * n)$ where m and n are the dimensions of the grid, as each cell is processed at most once.

Space Complexity: $O(m * n)$ in the worst-case scenario used by the BFS queue.

Solution

The solution uses a multi-source BFS algorithm by initially enqueueing all the gate positions (cells with value 0). Then, for each cell obtained from the queue, check its four possible neighbors (up, down, left, right). If a neighbor is an empty room (INF), update its distance to be the current cell's distance plus one, and then enqueue the neighbor to process further. This ensures that each empty room gets the minimum distance from any gate.

Graphs

□ ★ #323 Number of Connected Components in an Undirected Graph ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

DFS · BFS · Union Find · Graph

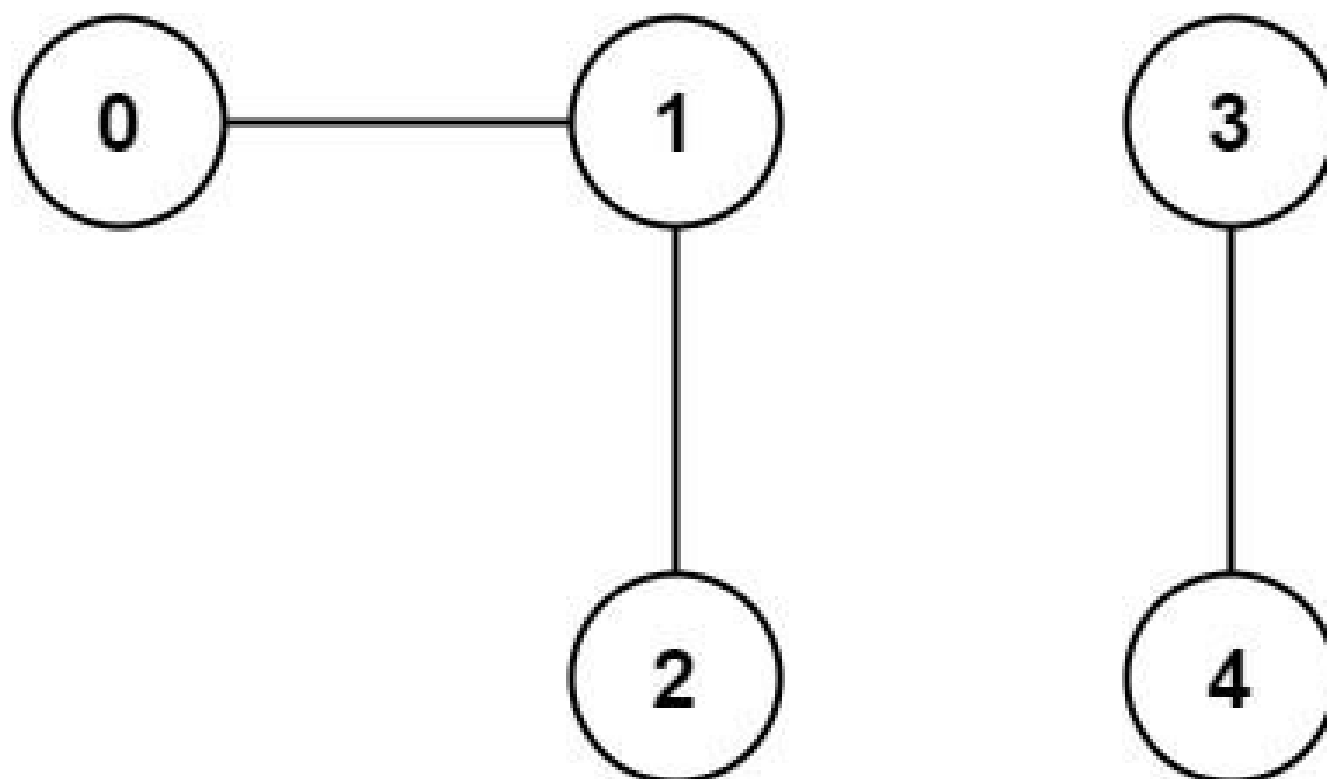
Lists: ★ Premium | Grind 169 | Premium 98

Problem

You have a graph of n nodes. You are given an integer n and an array `edges` where `edges[i] = [ai, bi]` indicates that there is an edge between a_i and b_i in the graph.

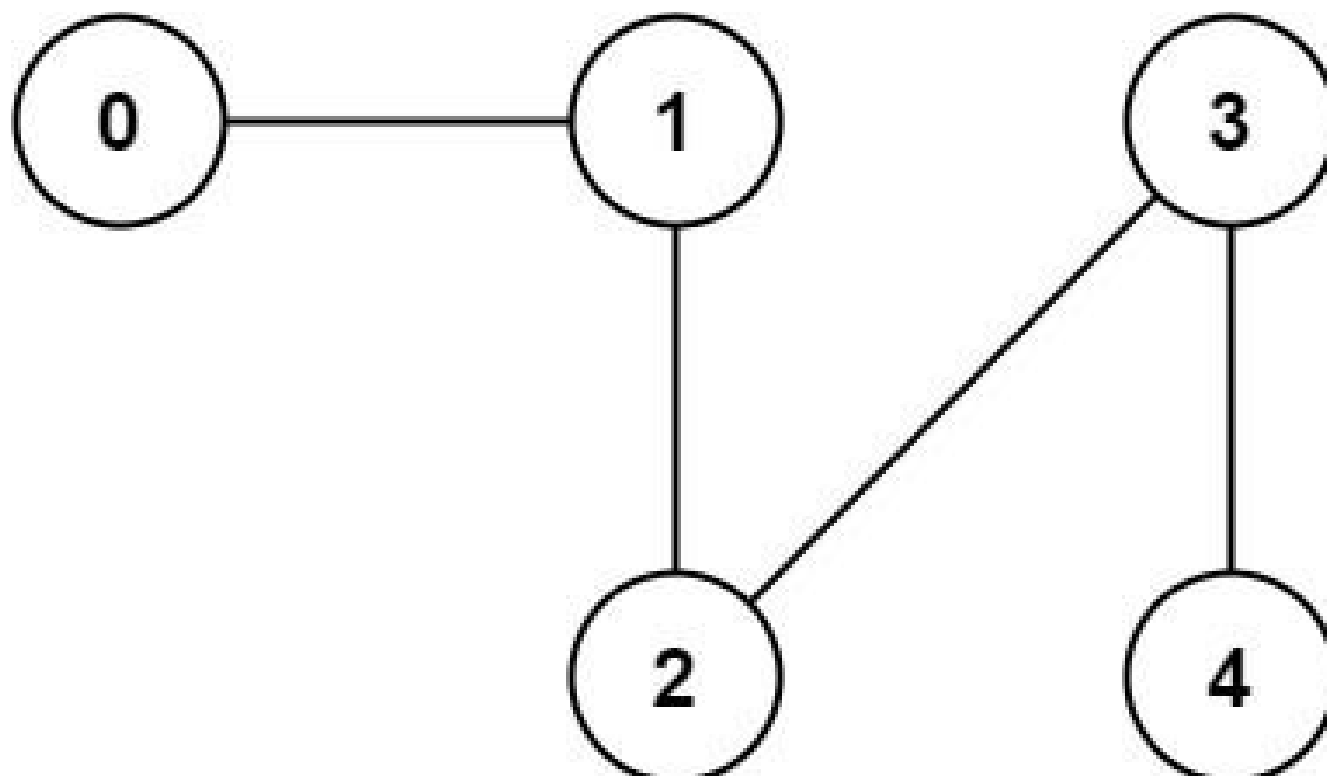
Return the number of connected components in the graph.

Example 1:



Input: $n = 5$, $edges = [[0,1],[1,2],[3,4]]$
Output: 2

Example 2:



Input: $n = 5$, $edges = [[0,1],[1,2],[2,3],[3,4]]$
 Output: 1

Constraints:

- $1 \leq n \leq 2000$
- $1 \leq edges.length \leq 5000$
- $edges[i].length == 2$
- $0 \leq ai \leq bi < n$
- $ai \neq bi$
- There are no repeated edges.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countComponents(self, n: int, edges: list[list[int]]) -> int:
        ans = 0
        graph = [[] for _ in range(n)]
        seen = set()

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        def bfs(node: int, seen: set[int]) -> None:
            q = collections.deque([node])
            seen.add(node)

            while q:
                u = q.pop()
                for v in graph[u]:
                    if v not in seen:
                        q.append(v)
                        seen.add(v)
```

```
for i in range(n):
    if i not in seen:
        bfs(i, seen)
        ans += 1
```

```
return ans
```

```
class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
```

```
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        self.count -= 1
```

```
    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]
```

```
class Solution:
    def countComponents(self, n: int, edges: list[list[int]]) -> int:
        uf = UnionFind(n)
```

```

for u, v in edges:
    uf.unionByRank(u, v)

return uf.count

```

• LeetDoocs

```

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        def dfs(i: int) -> int:
            if i in vis:
                return 0
            vis.add(i)
            for j in g[i]:
                dfs(j)
            return 1

```

```

g = [[] for _ in range(n)]
for a, b in edges:
    g[a].append(b)
    g[b].append(a)
vis = set()
return sum(dfs(i) for i in range(n))

```

```

class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

```

```

def union(self, a, b):
    pa, pb = self.find(a), self.find(b)
    if pa == pb:
        return False
    if self.size[pa] > self.size[pb]:
        self.p[pb] = pa
        self.size[pa] += self.size[pb]
    else:
        self.p[pa] = pb
        self.size[pb] += self.size[pa]

```

```

    return True

```

```

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        uf = UnionFind(n)
        for a, b in edges:
            n -= uf.union(a, b)
        return n

```

```

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        vis = set()
        ans = 0
        for i in range(n):
            if i in vis:

```

```

                continue
            vis.add(i)
            q = deque([i])
            while q:
                a = q.popleft()
                for b in g[a]:
                    if b not in vis:
                        vis.add(b)
                        q.append(b)
            ans += 1

```



```
return ans
```

• SimplyLeet

```
# Function to count the number of connected components in an undirected graph.
def count_components(n, edges):
    # Build the adjacency list for the graph.
    graph = {i: [] for i in range(n)}
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)

    # Set to keep track of visited nodes.
    visited = set()

    # Helper function to perform DFS traversal starting from a node.
    def dfs(node):
        stack = [node]
        while stack:
            current = stack.pop()
            if current not in visited:
                visited.add(current)
                for neighbor in graph[current]:
                    if neighbor not in visited:
                        stack.append(neighbor)

    # Count connected components by initiating DFS on unvisited nodes.
    count = 0
    for i in range(n):
        if i not in visited:
            dfs(i)
            count += 1
    return count

# Example usage:
print(count_components(5, [[0,1],[1,2],[3,4]])) # Expected output: 2
```

• LC.ca

```
'''
example-2: [[0, 1], [1, 2], [2, 3], [3, 4]]
```

```
for loop
```

```
[0, 1]
    0 parent set to 1
    1 parent set to 1
```

```
[1, 2]
```

```
    1 parent set to 2
    2 parent set to 2
```

```
[2, 3]
    2 parent set to 3
    3 parent set to 3
```

```
final:
    0 parent set to 1
    1 parent set to 2
```

```
    2 parent set to 3
    3 parent set to 3
```

```
==> so filter for 'i == find(i)'
'''
```

```
class Solution:
```

```
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
```

```
        def find(x):
```

```
            if p[x] != x:
```

```
                p[x] = find(p[x])
```

```
        return p[x]
```

```
    p = list(range(n))
```

```
    for a, b in edges:
```

```
        p[find(a)] = find(b)
```

```
    return sum(i == find(i) for i in range(n)) # or len(set(parents))
```

```
#####
```

```
class Solution: # more literal
```

```
def countComponents(self, n: int, edges: List[List[int]]) -> int:
    # Initialize parent array for union-find
    parent = [i for i in range(n)]

    # Perform union-find on each edge
    for u, v in edges:
        self.union(u, v, parent)

    # Count the number of distinct parents (connected components)
    count = len(set(self.find(x, parent) for x in range(n)))

    return count

def union(self, x: int, y: int, parent: List[int]) -> None:
    rootX = self.find(x, parent)
    rootY = self.find(y, parent)
    if rootX != rootY:
        parent[rootX] = rootY

def find(self, x: int, parent: List[int]) -> int:
    if parent[x] != x:
        parent[x] = self.find(parent[x], parent)
    return parent[x]
```

◆ Quick Review

Key Insights

- Use an adjacency list to represent the graph.
- A DFS (or BFS) traversal from an unvisited node will traverse an entire connected component.
- Each new DFS/BFS call from an unvisited node signals a separate connected component.

- Alternatively, the Union-Find data structure can be used to union connected nodes and count distinct sets.

Space & Time

Time Complexity: $O(n + e)$, where n is the number of nodes and e is the number of edges.

Space Complexity: $O(n + e)$ due to the storage of the graph and tracking of visited nodes.

Solution

We solve the problem by iterating over each node and performing a DFS for any node that hasn't been visited. The DFS uses an iterative approach with a stack to traverse all nodes in a connected component. Each DFS initiation corresponds to one connected component. This method efficiently explores the graph using an adjacency list and marks nodes as visited to prevent redundant traversals.

Graphs

□ #417 Pacific Atlantic Water Flow

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Matrix
Lists: Grind 169

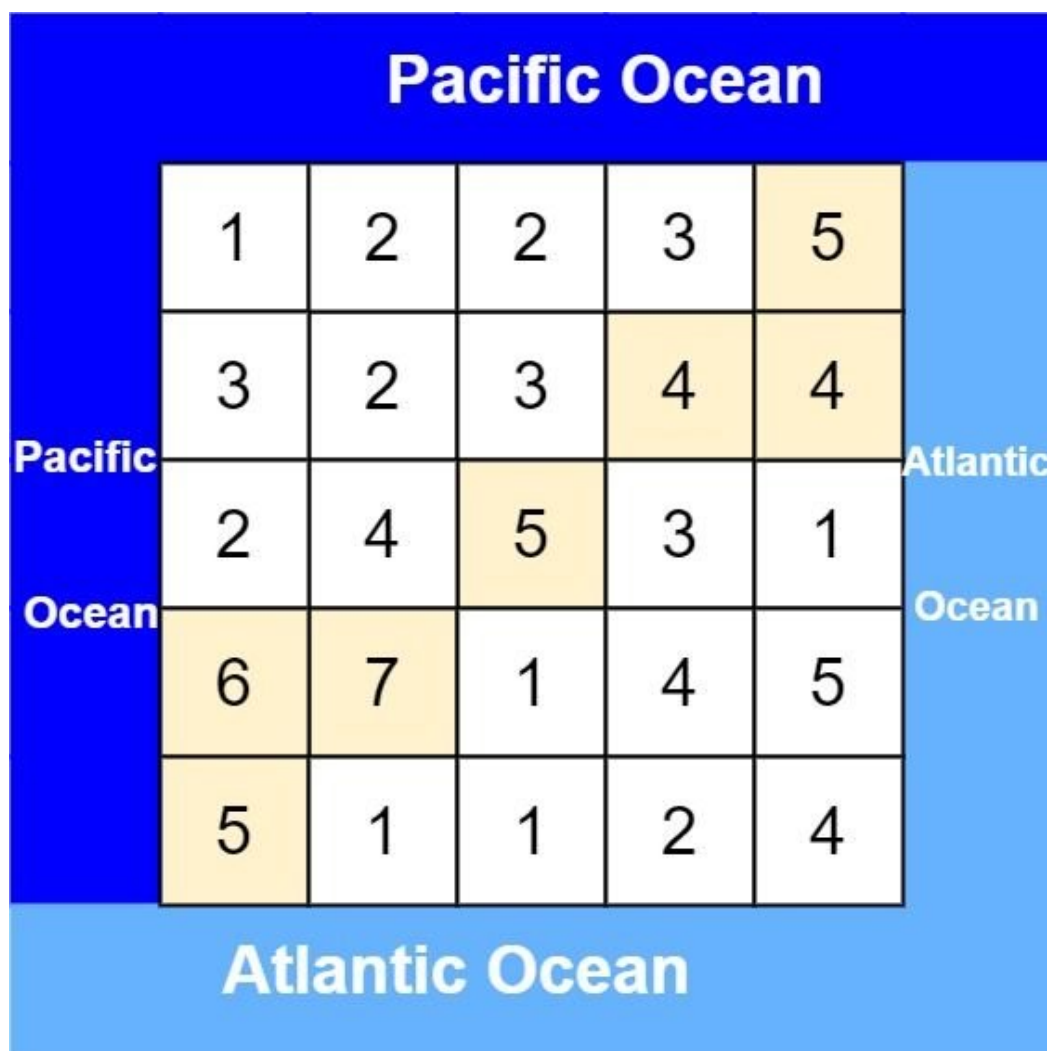
Problem

There is an $m \times n$ rectangular island that borders both the Pacific Ocean and Atlantic Ocean. The Pacific Ocean touches the island's left and top edges, and the Atlantic Ocean touches the island's right and bottom edges. The island is partitioned into a grid of square cells. You are given an $m \times n$ integer matrix `heights` where `heights[r][c]` represents the height above sea level of the cell at coordinate (r, c) .

The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is less than or equal to the current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.

Return a 2D list of grid coordinates result where $\text{result}[i] = [\text{ri}, \text{ci}]$ denotes that rain water can flow from cell (ri, ci) to both the Pacific and Atlantic oceans.

Example 1:



Input: heights = [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]
 Output: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]
 Explanation: The following cells can flow to the Pacific and Atlantic oceans, as shown below:
 [0,4]: [0,4] -> Pacific Ocean
 [0,4] -> Atlantic Ocean
 [1,3]: [1,3] -> [0,3] -> Pacific Ocean
 [1,3] -> [1,4] -> Atlantic Ocean
 [1,4]: [1,4] -> [1,3] -> [0,3] -> Pacific Ocean

Example 2:

Input: heights = [[1]]
 Output: [[0,0]]
 Explanation: The water can flow from the only cell to the Pacific and Atlantic oceans.

Constraints:

- $m == \text{heights.length}$
- $n == \text{heights[r].length}$
- $1 \leq m, n \leq 200$
- $0 \leq \text{heights[r][c]} \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def pacificAtlantic(self, heights: list[list[int]]) -> list[list[int]]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(heights)
        n = len(heights[0])
        qP = collections.deque()
        qA = collections.deque()
        seenP = [[False] * n for _ in range(m)]
        seenA = [[False] * n for _ in range(m)]
```

```

for i in range(m):
    qP.append((i, 0))
    qA.append((i, n - 1))
    seenP[i][0] = True
    seenA[i][n - 1] = True

for j in range(n):
    qP.append((0, j))
    qA.append((m - 1, j))
    seenP[0][j] = True

    seenA[m - 1][j] = True

def bfs(q: deque, seen: list[list[bool]]):
    while q:
        i, j = q.popleft()
        h = heights[i][j]
        for dx, dy in DIRS:
            x = i + dx
            y = j + dy
            if x < 0 or x == m or y < 0 or y == n:
                continue
            if seen[x][y] or heights[x][y] < h:
                continue
            q.append((x, y))
            seen[x][y] = True

bfs(qP, seenP)
bfs(qA, seenA)

return [[i, j] for i in range(m) for j in range(n) if seenP[i][j] and seen
A[i][j]]

```



```

class Solution:
    def pacificAtlantic(self, heights: list[list[int]]) -> list[list[int]]:
        m = len(heights)
        n = len(heights[0])
        seenP = [[False] * n for _ in range(m)]
        seenA = [[False] * n for _ in range(m)]

        def dfs(i: int, j: int, h: int, seen: list[list[bool]]) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return

            if seen[i][j] or heights[i][j] < h:
                return

            seen[i][j] = True
            dfs(i + 1, j, heights[i][j], seen)
            dfs(i - 1, j, heights[i][j], seen)
            dfs(i, j + 1, heights[i][j], seen)
            dfs(i, j - 1, heights[i][j], seen)

        for i in range(m):
            dfs(i, 0, 0, seenP)
            dfs(i, n - 1, 0, seenA)

        for j in range(n):
            dfs(0, j, 0, seenP)
            dfs(m - 1, j, 0, seenA)

        return [[i, j]
                for i in range(m)
                for j in range(n)

                if seenP[i][j] and seenA[i][j]]

```

• LeetDoocs

```

class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        def bfs(q: Deque[Tuple[int, int]], vis: List[List[bool]]) -> None:
            while q:
                x, y = q.popleft()
                for dx, dy in pairwise(dirs):
                    nx, ny = x + dx, y + dy
                    if (
<= nx < m
                                and 0 <= ny < n
                                and not vis[nx][ny]
                                and heights[nx][ny] >= heights[x][y]
                    ):
                        vis[nx][ny] = True
                        q.append((nx, ny))

        m, n = len(heights), len(heights[0])
        vis1 = [[False] * n for _ in range(m)]
        vis2 = [[False] * n for _ in range(m)]
        q1: Deque[Tuple[int, int]] = deque()

        q2: Deque[Tuple[int, int]] = deque()
        dirs = (-1, 0, 1, 0, -1)

        for i in range(m):
            q1.append((i, 0))
            vis1[i][0] = True
            q2.append((i, n - 1))
            vis2[i][n - 1] = True

        for j in range(n):
            q1.append((0, j))
            vis1[0][j] = True
            q2.append((m - 1, j))
            vis2[m - 1][j] = True

        bfs(q1, vis1)
        bfs(q2, vis2)

        return [(i, j) for i in range(m) for j in range(n) if vis1[i][j] and vis2[i][j]]

```

• SimplyLeet

```
# Python solution for Pacific Atlantic Water Flow

class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        if not heights or not heights[0]:
            return []

        rows, cols = len(heights), len(heights[0])
        pacific_reachable = [[False for _ in range(cols)] for _ in range(rows)]
        atlantic_reachable = [[False for _ in range(cols)] for _ in range(rows)]

        # Directions: up, down, left, right
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

        # DFS function to mark reachable cells
        def dfs(r, c, reachable):
            reachable[r][c] = True
            for dr, dc in directions:
                new_r, new_c = r + dr, c + dc
                # Check bounds and conditions to flow water
                if (0 <= new_r < rows and 0 <= new_c < cols and
                    not reachable[new_r][new_c] and
                    heights[new_r][new_c] >= heights[r][c]):
                    dfs(new_r, new_c, reachable)

        # DFS from Pacific ocean borders (top row and left column)
        for i in range(rows):
            dfs(i, 0, pacific_reachable)
        for j in range(cols):
            dfs(0, j, pacific_reachable)
```

```

# DFS from Atlantic ocean borders (bottom row and right column)
for i in range(rows):
    dfs(i, cols - 1, atlantic_reachable)
for j in range(cols):
    dfs(rows - 1, j, atlantic_reachable)

# Collect cells that can reach both oceans
result = []
for r in range(rows):
    for c in range(cols):
        if pacific_reachable[r][c] and atlantic_reachable[r][c]:
            result.append([r, c])
return result

```

• LC.ca

```

class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        def bfs(q, vis):
            while q:
                for _ in range(len(q)):
                    i, j = q.popleft()
                    for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
                        x, y = i + a, j + b
                        if (
                            0 <= x < m
                            and 0 <= y < n
                            and (x, y) not in vis
                            and heights[x][y] >= heights[i][j]
                        ):
                            vis.add((x, y))
                            q.append((x, y))

        m, n = len(heights), len(heights[0])
        vis1, vis2 = set(), set()
        q1 = deque()

```

```
q2 = deque()
for i in range(m):
    for j in range(n):
        if i == 0 or j == 0:
            vis1.add((i, j))
            q1.append((i, j))
        if i == m - 1 or j == n - 1:
            vis2.add((i, j))
            q2.append((i, j))
bfs(q1, vis1)

bfs(q2, vis2)
return [
    (i, j)
    for i in range(m)
    for j in range(n)
    if (i, j) in vis1 and (i, j) in vis2
]
```

◆ Quick Review

Key Insights

- Reverse the typical DFS/BFS perspective: start from the oceans and work inward.
- Maintain two matrices (or sets) to keep track of cells reachable from the Pacific and Atlantic edges.
- A cell can be added to the result if it is reached in both traversals.
- The height constraint ensures water flows only in non-increasing order.

Space & Time

Time Complexity: $O(m * n)$ where m is the number of rows and n is the number of columns. Each cell is processed up to twice (once from each ocean).

Space Complexity: $O(m * n)$ for the visited matrices and the recursion/queue used in DFS/BFS.

Solution

The solution uses depth-first search (DFS) from the ocean boundaries. We initialize two boolean matrices, one for each ocean. For the Pacific, we start DFS from the leftmost column and top row; for the Atlantic, we start from the rightmost column and bottom row. In each DFS, if the next cell has a height equal to or greater than the current cell and hasn't been visited, we continue the search. Finally, we scan through the grid to collect cells that are reachable from both oceans. This reverse search simplifies the path validations and leverages the allowed water flow conditions.

Graphs

□ #684 Redundant Connection

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Depth-First Search · Breadth-First Search ·
Union-Find · Graph Theory
Lists: NeetCode 150

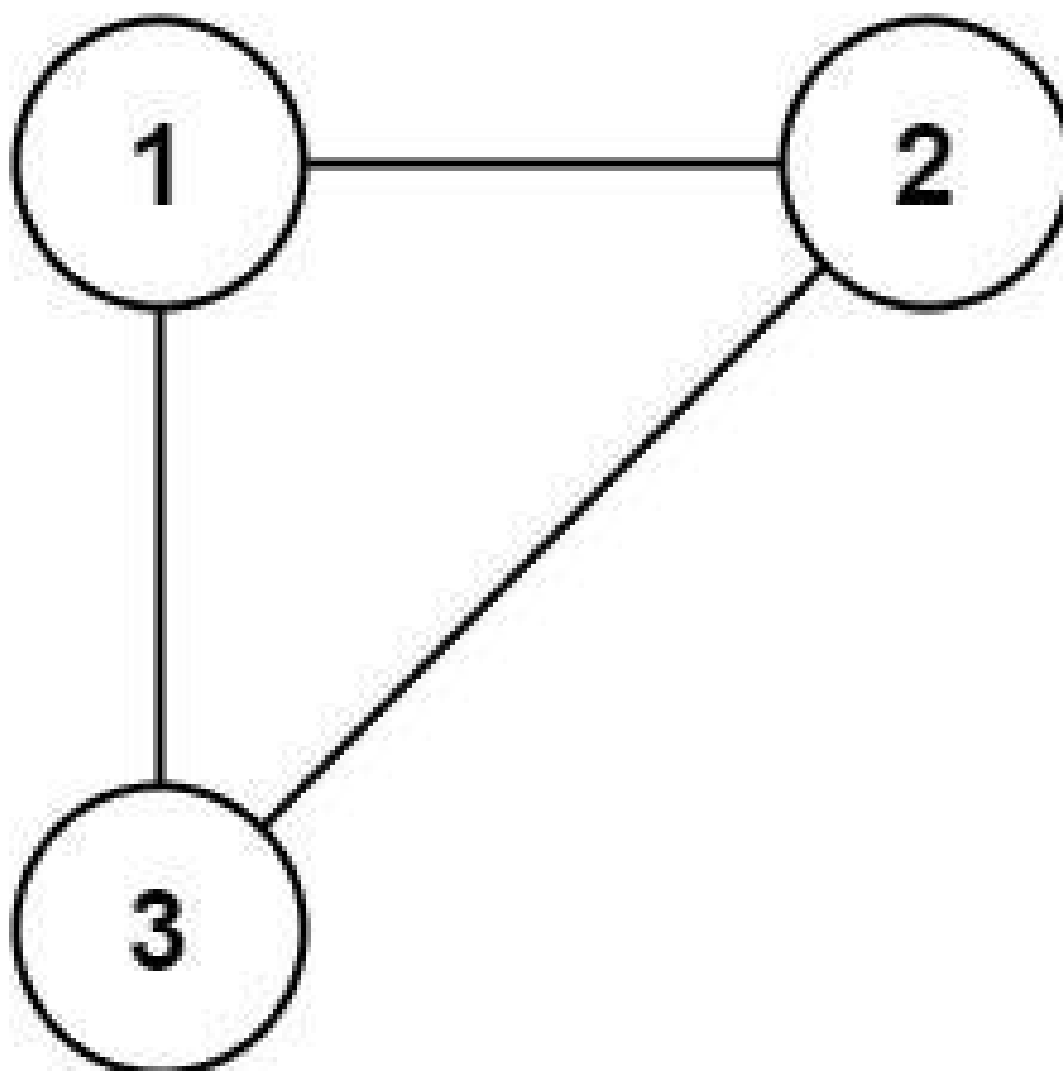
Problem

In this problem, a tree is an undirected graph that is connected and has no cycles.

You are given a graph that started as a tree with n nodes labeled from 1 to n , with one additional edge added. The added edge has two different vertices chosen from 1 to n , and was not an edge that already existed. The graph is represented as an array `edges` of length n where `edges[i] = [ai, bi]` indicates that there is an edge between nodes ai and bi in the graph.

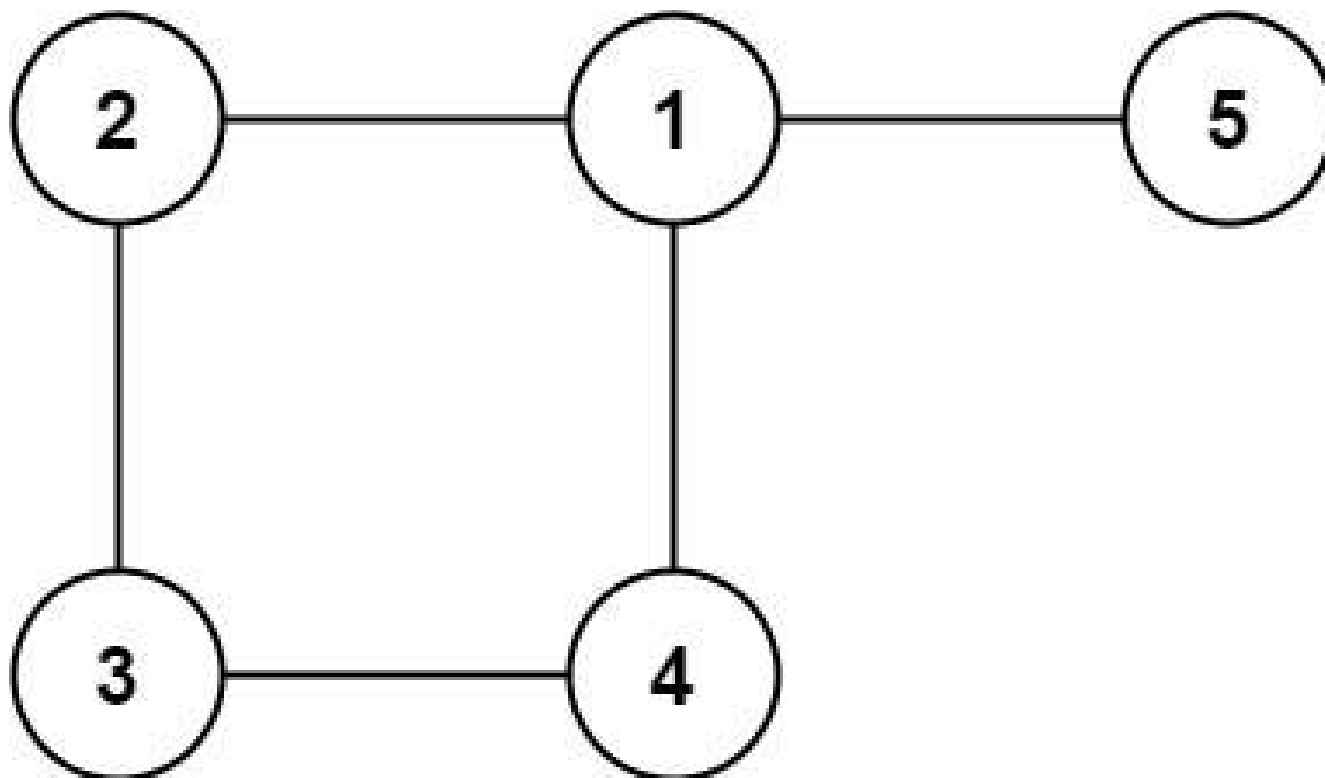
Return an edge that can be removed so that the resulting graph is a tree of n nodes. If there are multiple answers, return the answer that occurs last in the input.

Example 1:



Input: edges = [[1,2],[1,3],[2,3]]
Output: [2,3]

Example 2:



Input: edges = [[1,2],[2,3],[3,4],[1,4],[1,5]]

Output: [1,4]

Constraints:

- $n == \text{edges.length}$
- $3 \leq n \leq 1000$
- $\text{edges}[i].\text{length} == 2$
- $1 \leq a_i < b_i \leq \text{edges.length}$
- $a_i \neq b_i$
- There are no repeated edges.
- The given graph is connected.

Community Solutions (Python)

- WalkCC

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> bool:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return False

        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        return True

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def findRedundantConnection(self, edges: list[list[int]]) -> list[int]:
        uf = UnionFind(len(edges) + 1)

        for edge in edges:
            u, v = edge
            if not uf.unionByRank(u, v):
                return edge

```

• LeetDoocs

```

class Solution:
    def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(len(edges)))
        for a, b in edges:
            pa, pb = find(a - 1), find(b - 1)

            if pa == pb:
                return [a, b]
            p[pa] = pb

class UnionFind:
    __slots__ = "p", "size"

    def __init__(self, n: int):
        self.p: List[int] = list(range(n))
        self.size: List[int] = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])

        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:

```

```

        self.p[pa] = pb
        self.size[pb] += self.size[pa]
    return True

```

```

class Solution:
    def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
        uf = UnionFind(len(edges))
        for a, b in edges:
            if not uf.union(a - 1, b - 1):
                return [a, b]

```

• SimplyLeet

Python solution using Union Find with path compression.

```

class UnionFind:
    def __init__(self, size):
        # Initialize parent pointers for each node.
        self.parent = list(range(size))

    def find(self, x):
        # Find the root of x with path compression.
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        # Union the sets of x and y.
        rootX = self.find(x)
        rootY = self.find(y)
        if rootX == rootY:
            return False # Cycle detected.
        self.parent[rootY] = rootX

```

```
        return True

def findRedundantConnection(edges):
    # Number of nodes is equal to len(edges) because edges were from a tree.
    n = len(edges)
    uf = UnionFind(n + 1) # Account for node labels starting at 1.
    redundant_edge = None
    for edge in edges:
        a, b = edge
        # If union returns False, a cycle is detected.

        if not uf.union(a, b):
            redundant_edge = edge
    return redundant_edge

# Example usage:
edges = [[1,2],[1,3],[2,3]]
print(findRedundantConnection(edges)) # Output: [2, 3]
```

◆ Quick Review

Key Insights

- The problem can be solved using a Union Find (Disjoint Set Union) data structure.
- As we iterate over each edge, we check whether the two vertices are already connected.
- If connecting the two vertices would create a cycle (i.e., they already belong to the same set), then the current edge is redundant.
- Alternatives like DFS or BFS are possible, but Union Find offers a more straightforward and

efficient solution.

Space & Time

Time Complexity: $O(n * \alpha(n))$ where $\alpha(n)$ is the inverse Ackermann function (almost constant in practice).

Space Complexity: $O(n)$, for storing the parent array used in the Union Find structure.

Solution

We use the Union Find algorithm by maintaining an array where each node points to its parent. For each edge, we perform a find operation on both endpoints:

– If the endpoints belong to the same set, adding this edge would create a cycle and it is marked as redundant. – Otherwise, we union the two sets. This algorithm efficiently detects the extra edge that introduces a cycle in the graph.

Graphs

□ #695 Max Area of Island

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Union Find · Matrix
Lists: Denny Zhang

Problem

You are given an $m \times n$ binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

The area of an island is the number of cells with a value 1 in the island.

Return the maximum area of an island in grid. If there is no island, return 0.

Example 1:

0	0	1	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	1	1	0	1	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	0	1	0	1	0	0
0	1	0	0	1	1	0	0	1	1	1	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	1	1	0	0	0	0

Input: grid = [[0,0,1,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,1,1,0,1,0,0,0,0,0,0,0,0],[0,1,0,0,1,1,0,0,1,0,1,0,0],[0,1,0,0,1,1,0,0,1,1,1,0,0],[0,0,0,0,0,0,0,0,0,0,1,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,0,0,0,0,0,0,0,1,1,0,0,0]]

Output: 6

Explanation: The answer is not 11, because the island must be connected 4-directionally.

Example 2:

Input: grid = [[0,0,0,0,0,0,0,0]]

Output: 0

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $\text{grid}[i][j]$ is either 0 or 1.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxAreaOfIsland(self, grid: list[list[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            if i < 0 or i == len(grid) or j < 0 or j == len(grid[0]):
                return 0
            if grid[i][j] != 1:
                return 0

            grid[i][j] = 2

            return 1 + dfs(i + 1, j) + dfs(i - 1, j) + dfs(i, j + 1) + dfs(i, j - 1)

        return max(dfs(i, j) for i in range(len(grid)) for j in range(len(grid[0])))
```

• LeetDoocs

```
class Solution:
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            if grid[i][j] == 0:
                return 0
            ans = 1
            grid[i][j] = 0
            dirs = (-1, 0, 1, 0, -1)
            for a, b in pairwise(dirs):
                x, y = i + a, j + b

                if 0 <= x < m and 0 <= y < n:
                    ans += dfs(x, y)
            return ans

        m, n = len(grid), len(grid[0])
        return max(dfs(i, j) for i in range(m) for j in range(n))
```

• SimplyLeet

```

# Function to compute the maximum area of an island in a grid using DFS
def maxAreaOfIsland(grid):
    # Determine the dimensions of the grid
    rows = len(grid)
    cols = len(grid[0]) if rows else 0

    # Helper function for performing DFS on the grid
    def dfs(r, c):
        # Check if the current cell is out-of-bounds or is water
        if r < 0 or c < 0 or r >= rows or c >= cols or grid[r][c] == 0:
            return 0
        # Mark the current cell as visited by setting it to 0 (water)
        grid[r][c] = 0
        area = 1 # Count the current cell as part of the island
        # Recursively explore all four directions
        area += dfs(r + 1, c)
        area += dfs(r - 1, c)
        area += dfs(r, c + 1)
        area += dfs(r, c - 1)
        return area

    max_area = 0
    # Iterate through each cell in the grid
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                # Update max_area with the area obtained from the DFS
                max_area = max(max_area, dfs(r, c))
    return max_area

```

• LC.ca

```
class Solution:
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            if grid[i][j] == 0:
                return 0
            ans = 1
            grid[i][j] = 0
            dirs = (-1, 0, 1, 0, -1)
            for a, b in pairwise(dirs):
                x, y = i + a, j + b

                if 0 <= x < m and 0 <= y < n:
                    ans += dfs(x, y)
            return ans

        m, n = len(grid), len(grid[0])
        return max(dfs(i, j) for i in range(m) for j in range(n))
```

◆ Quick Review

Key Insights

- Traverse each cell in the grid.
- Use Depth-First Search (DFS) or Breadth-First Search (BFS) to explore connected 1's.
- Mark visited cells to prevent double counting (e.g., by setting them to 0).
- Maintain and update a maximum area count during traversal.
- The grid's boundaries are naturally water, so no extra edge processing is needed.

Space & Time

Time Complexity: $O(m * n)$ where m and n are the dimensions of the grid since every cell is visited at most once.

Space Complexity: $O(m * n)$ in the worst-case scenario for the recursion stack (or auxiliary data structures) when the grid is entirely land.

Solution

The solution leverages a DFS approach to iterate over each cell in the given grid. When a cell with a value of 1 is encountered, a DFS is initiated from that cell to calculate the area of the island connected to it. Cells are marked as visited (by changing them from 1 to 0) during the DFS to avoid revisiting. The DFS explores all four directions (up, down, left, right) and accumulates the count of connected cells. The maximum area found among all islands is returned at the end. This approach efficiently computes the largest island area with a simple traversal mechanism.

Graphs

□ #994 Rotting Oranges

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix
Lists: Grind 169

Problem

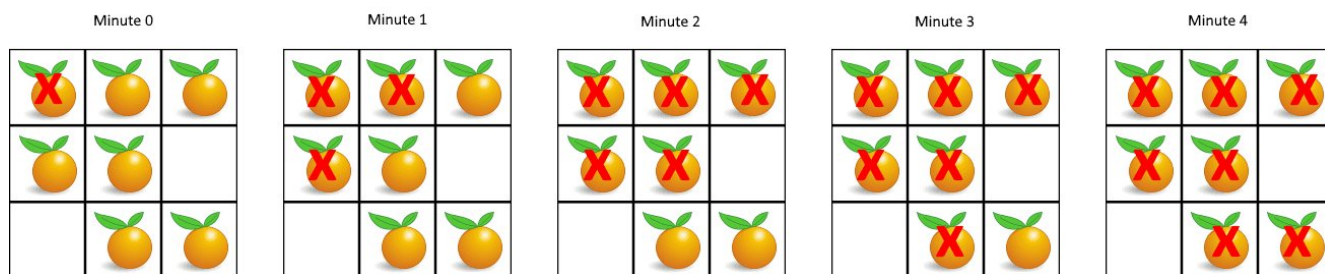
You are given an $m \times n$ grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1 .

Example 1:



Input: grid = [[2,1,1],[1,1,0],[0,1,1]]

Output: 4

Example 2:

Input: grid = [[2,1,1],[0,1,1],[1,0,1]]

Output: -1

Explanation: The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens 4-directionally.

Example 3:

Input: grid = [[0,2]]

Output: 0

Explanation: Since there are already no fresh oranges at minute 0, the answer is just 0.

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 10$
- $\text{grid}[i][j]$ is 0, 1, or 2.

Community Solutions (Python)

- LeetDoocs

```

class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        cnt = 0
        q = deque()
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                if x == 2:
                    q.append((i, j))
                elif x == 1:
                    cnt += 1

        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        while q and cnt:
            ans += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == 1:
                        grid[x][y] = 2
                        q.append((x, y))
                        cnt -= 1
            if cnt == 0:
                return ans
        return -1 if cnt else 0

```

• SimplyLeet

```

from collections import deque

def orangesRotting(grid):
    # Dimensions of the grid
    rows, cols = len(grid), len(grid[0])
    # Queue to perform BFS, containing tuples of (row, col)
    rotten_queue = deque()
    fresh_count = 0

    # Step 1: Initialize the queue with all initially rotten oranges and count
    # fresh oranges.

```

```

for r in range(rows):
    for c in range(cols):
        if grid[r][c] == 2:
            rotten_queue.append((r, c))
        elif grid[r][c] == 1:
            fresh_count += 1

# If there are no fresh oranges, return 0 immediately.
if fresh_count == 0:
    return 0

```

```

minutes_passed = 0
# Directions for adjacent cells: up, down, left, right.
directions = [(-1,0), (1,0), (0,-1), (0,1)]

# Step 2: BFS to rot adjacent fresh oranges.
while rotten_queue:
    # Process oranges that are rotten in the current minute.
    for _ in range(len(rotten_queue)):
        x, y = rotten_queue.popleft()

```

```

        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            # Check if the neighbor is inside the grid and is a fresh
orange.
            if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 1:
                # Mark the fresh orange as rotten.
                grid[nx][ny] = 2
                fresh_count -= 1
                # Add the new rotten orange into the queue.
                rotten_queue.append((nx, ny))
        # Increment minutes after processing one level.

```

```

minutes_passed += 1

```

```

# If there are still fresh oranges left, return -1.
return minutes_passed - 1 if fresh_count == 0 else -1

```

```

# Example usage:
# grid = [[2,1,1],[1,1,0],[0,1,1]]
# print( orangesRotting(grid) )

```


• LC.ca

```
class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        q = deque()
        cnt = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 2:
                    q.append((i, j))
                elif grid[i][j] == 1:
                    cnt += 1

        ans = 0
        while q and cnt:
            ans += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in [[0, 1], [0, -1], [1, 0], [-1, 0]]:
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == 1:
                        cnt -= 1
                        grid[x][y] = 2
                        q.append((x, y))
            return ans if cnt == 0 else -1
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) starting from all initially rotten oranges.
- Process the grid level by level, where each level represents one minute.

- Keep a count of fresh oranges and decrement it as they become rotten.
- If after the BFS there are still fresh oranges, then it is impossible to rot every orange.

Space & Time

Time Complexity: $O(m * n)$ as every cell is processed at most once.

Space Complexity: $O(m * n)$ in the worst case for the queue and auxiliary storage.

Solution

We begin by scanning the grid to locate all rotten oranges and count the fresh ones.

These rotten oranges are added to a queue for a BFS. For each minute (BFS level), we process all the current rotten oranges and try to infect their adjacent fresh oranges. If an adjacent cell contains a fresh orange, we mark it rotten, add it to the queue, and decrement our fresh count. We continue this process level by level (minute by minute) until there are no fresh oranges left. If, after processing, there are still fresh oranges remaining, we return -1 , indicating that not all oranges can become rotten.

Advanced Graphs

Round 2 · High · Medium · 3 questions

Advanced Graphs

□ #743 Network Delay Time

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Depth-First Search · Breadth-First Search ·
Graph Theory · Heap (Priority Queue) · Shortest
Path

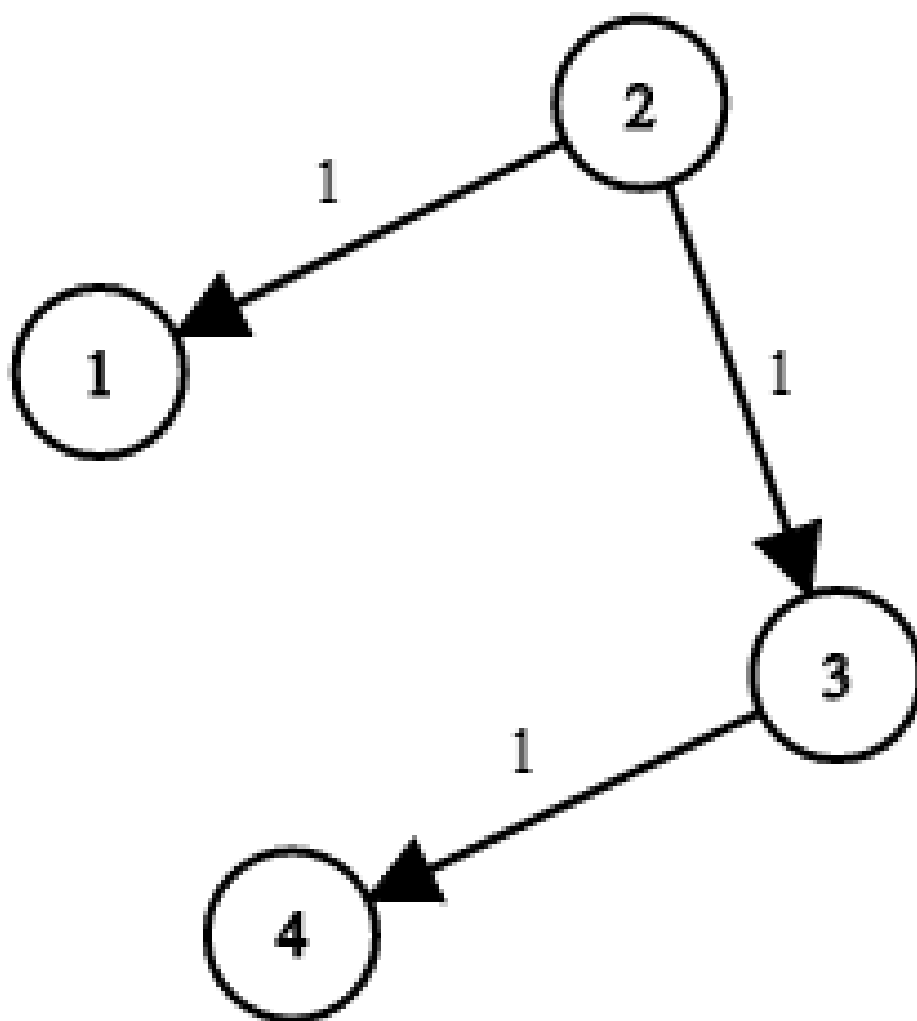
Lists: NeetCode 150

Problem

You are given a network of n nodes, labeled from 1 to n . You are also given times, a list of travel times as directed edges $\text{times}[i] = (u_i, v_i, w_i)$, where u_i is the source node, v_i is the target node, and w_i is the time it takes for a signal to travel from source to target.

We will send a signal from a given node k .
Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1 .

Example 1:



Input: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2
Output: 2

Example 2:

Input: times = [[1,2,1]], n = 2, k = 1
Output: 1

Example 3:

Input: times = [[1,2,1]], n = 2, k = 2
Output: -1

Constraints:

- $1 \leq k \leq n \leq 100$

- $1 \leq \text{times.length} \leq 6000$
- $\text{times}[i].\text{length} == 3$
- $1 \leq u_i, v_i \leq n$
- $u_i \neq v_i$
- $0 \leq w_i \leq 100$
- All the pairs (u_i, v_i) are unique. (i.e., no multiple edges.)

Community Solutions (Python)

• WalkCC

```
class Solution:
    def networkDelayTime(self, times: list[list[int]], n: int, k: int) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in times:
            graph[u - 1].append((v - 1, w))

        return self._dijkstra(graph, k - 1)

    def _dijkstra(self, graph: list[list[tuple[int, int]]], src: int) -> int:

        dist = [math.inf] * len(graph)

        dist[src] = 0
        minHeap = [(dist[src], src)] # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
```

```

        if d + w < dist[v]:
            dist[v] = d + w
            heapq.heappush(minHeap, (dist[v], v))

    maxDist = max(dist)
    return maxDist if maxDist != math.inf else -1

```

• LeetCode

```

class Solution:
    def networkDelayTime(self, times: List[List[int]], n: int, k: int) -> int:
        g = [[inf] * n for _ in range(n)]
        for u, v, w in times:
            g[u - 1][v - 1] = w
        dist = [inf] * n
        dist[k - 1] = 0
        vis = [False] * n
        for _ in range(n):
            t = -1
            for j in range(n):
                if not vis[j] and (t == -1 or dist[t] > dist[j]):
                    t = j
            vis[t] = True
            for j in range(n):
                dist[j] = min(dist[j], dist[t] + g[t][j])
        ans = max(dist)
        return -1 if ans == inf else ans

```

• SimplyLeet

```

import heapq

def networkDelayTime(times, n, k):
    # Build the graph as an adjacency list
    graph = {i: [] for i in range(1, n + 1)}
    for u, v, w in times:
        graph[u].append((v, w))

    # Initialize distances: infinity for all nodes except the starting node k
    # with distance 0
    dist = {i: float('inf') for i in range(1, n + 1)}

```

```
dist[k] = 0
# Use a heap as a priority queue storing (distance, node) pairs
heap = [(0, k)]

while heap:
    current_dist, node = heapq.heappop(heap)
    # If we have already found a better path, skip
    if current_dist > dist[node]:
        continue
    # Update the path for neighbors

    for neighbor, weight in graph[node]:
        new_dist = current_dist + weight
        if new_dist < dist[neighbor]:
            dist[neighbor] = new_dist
            heapq.heappush(heap, (new_dist, neighbor))

    # Maximum distance among all nodes represents the time taken for the signal
    # to reach every node
    max_time = max(dist.values())
    return max_time if max_time != float('inf') else -1

# Example usage:
print(networkDelayTime([[2,1,1],[2,3,1],[3,4,1]], 4, 2))
```

◆ Quick Review

Key Insights

- This is a single-source shortest path problem on a directed graph.
- Since all edge weights are non-negative, Dijkstra's algorithm is well-suited for this problem.
- Construct an adjacency list for the graph.

- Utilize a min-heap (priority queue) to efficiently select the next node with the smallest cumulative travel time.
- The answer is the maximum travel time among all reachable nodes; if any node is unreachable, return -1 .

Space & Time

Time Complexity: $O(E \log V)$ where E is the number of edges and V is the number of nodes.

Space Complexity: $O(V + E)$ due to the adjacency list and distance storage.

Solution

We solve the problem using Dijkstra's algorithm. First, an adjacency list is built using the input times array. A distance map (or array) records the shortest known distance from the starting node k to all other nodes, initialized to infinity except for k which is set to 0. A min-heap stores nodes prioritized by their current shortest distance.

The algorithm repeatedly extracts the node with the smallest distance from the heap. For each neighbor of the current node, it checks if

a shorter path is found via the current node; if so, the neighbor's distance is updated and it is added to the heap. Once all nodes are processed, the answer is the maximum distance from the k node. If any node remains unreachable (has distance equal to infinity), return -1.

Advanced Graphs

□ #787 Cheapest Flights Within K Stops

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · DFS · BFS · Graph ·
Heap

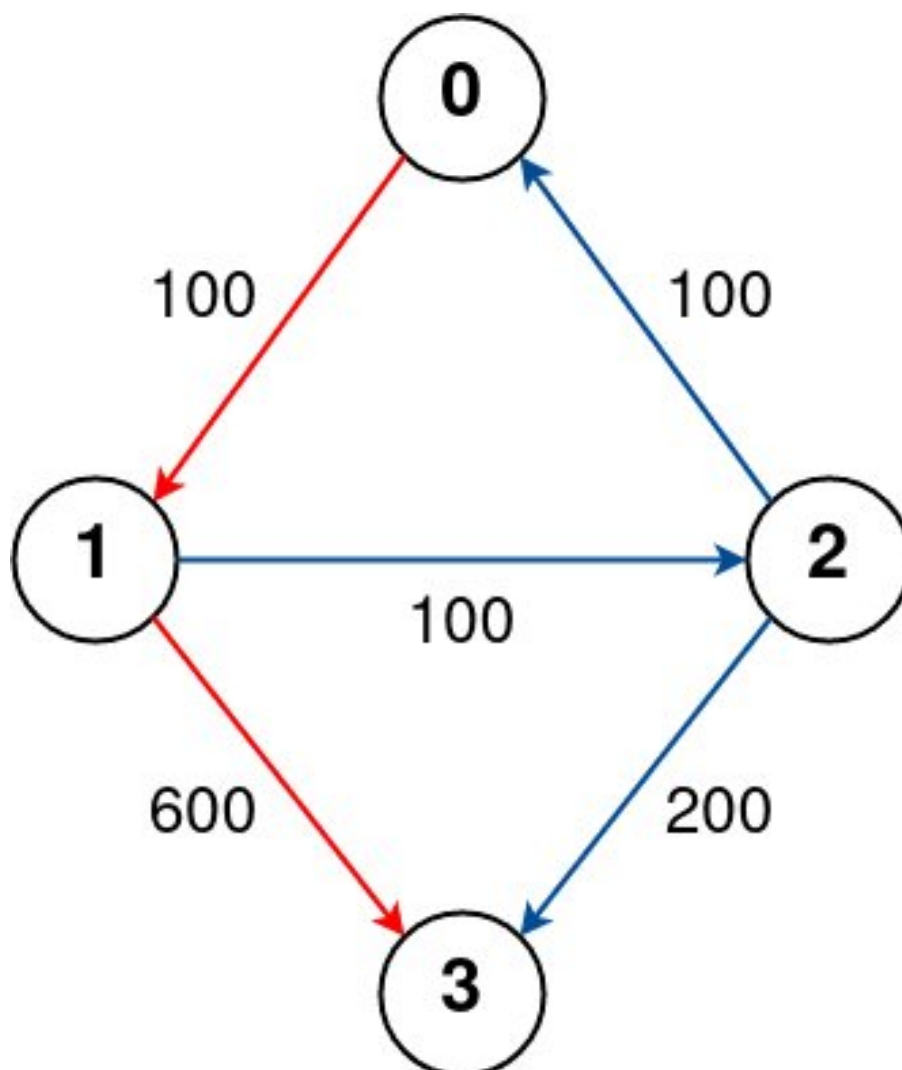
Lists: Grind 169

Problem

There are n cities connected by some number of flights. You are given an array `flights` where `flights[i] = [fromi, toi, pricei]` indicates that there is a flight from city `fromi` to city `toi` with cost `pricei`.

You are also given three integers `src`, `dst`, and `k`, return the cheapest price from `src` to `dst` with at most `k` stops. If there is no such route, return `-1`.

Example 1:



Input: $n = 4$, $flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]$,
 $src = 0$, $dst = 3$, $k = 1$

Output: 700

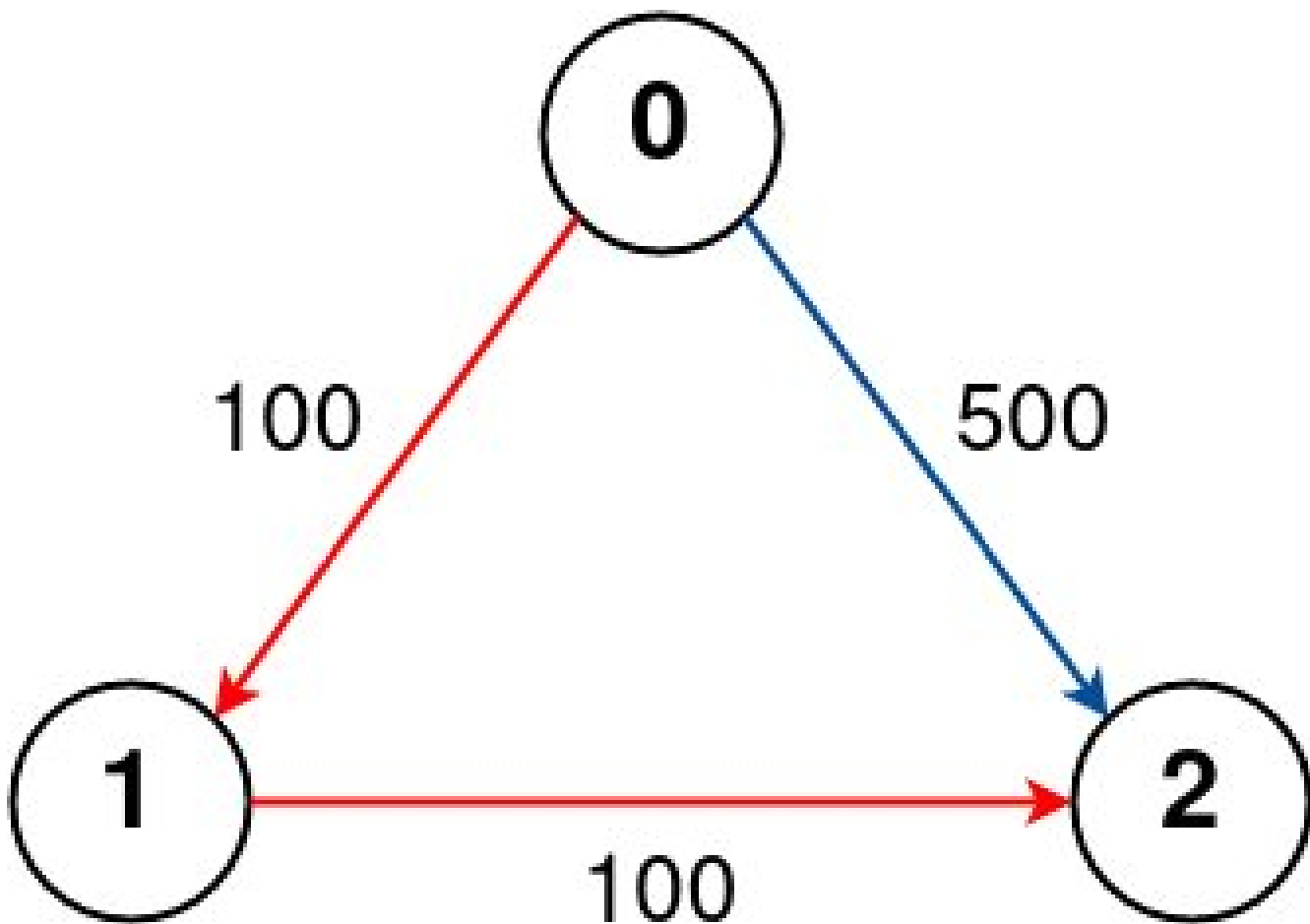
Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost $100 + 600 = 700$.

Note that the path through cities $[0,1,2,3]$ is cheaper but is invalid because it uses 2 stops.

Example 2:



Input: $n = 3$, $flights = [[0,1,100],[1,2,100],[0,2,500]]$, $src = 0$, $dst = 2$, $k = 1$

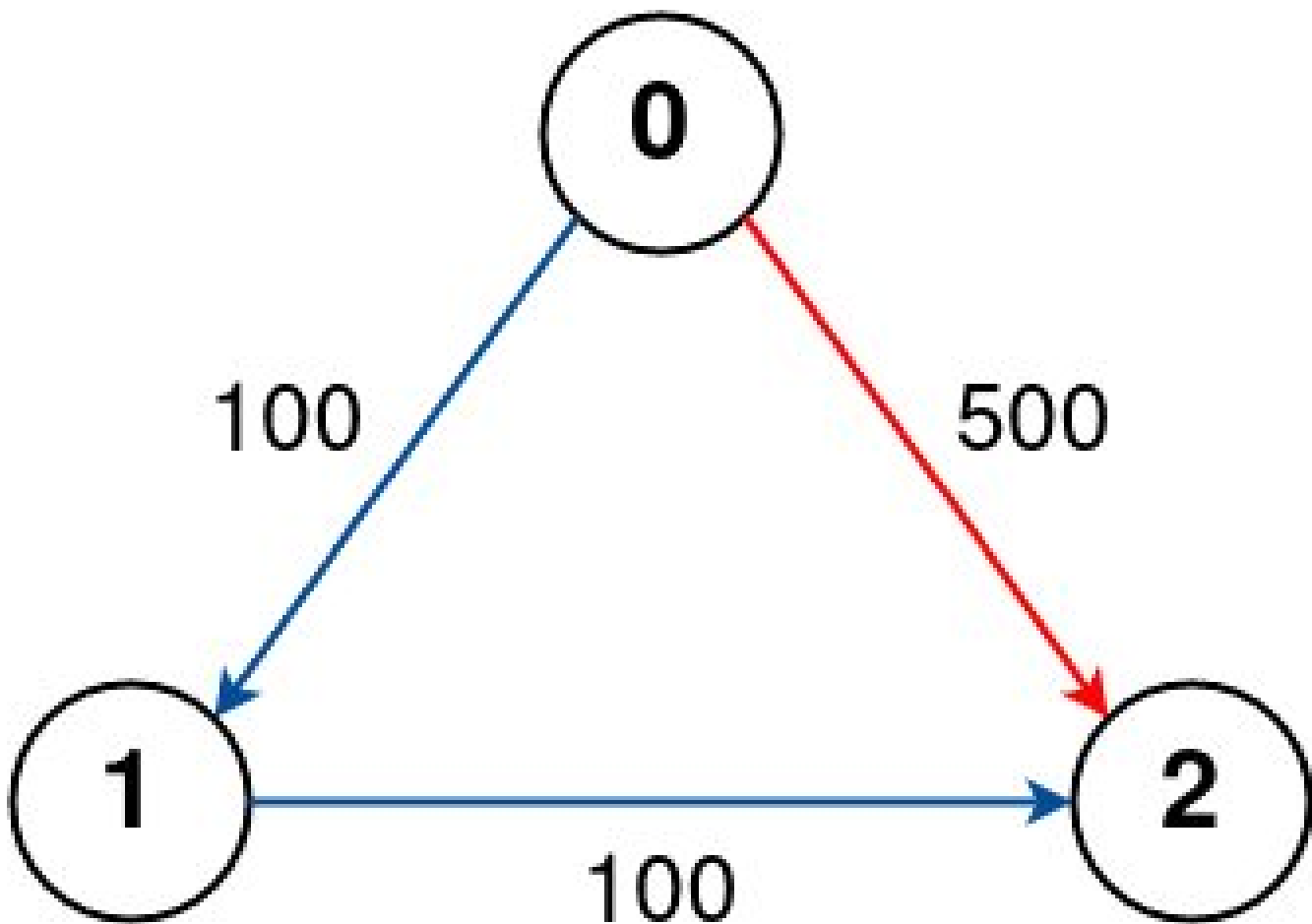
Output: 200

Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 2 is marked in red and has cost $100 + 100 = 200$.

Example 3:



Input: $n = 3$, $flights = [[0,1,100],[1,2,100],[0,2,500]]$, $src = 0$, $dst = 2$, $k = 0$

Output: 500

Explanation:

The graph is shown above.

The optimal path with no stops from city 0 to 2 is marked in red and has cost 500.

Constraints:

- $2 \leq n \leq 100$
- $0 \leq flights.length \leq (n * (n - 1) / 2)$
- $flights[i].length == 3$
- $0 \leq from_i, to_i < n$
- $from_i \neq to_i$

- $1 \leq \text{price}_i \leq 104$
- There will not be any multiple flights between two cities.
- $0 \leq \text{src}, \text{dst}, k < n$
- $\text{src} \neq \text{dst}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findCheapestPrice(
        self,
        n: int,
        flights: list[list[int]],
        src: int,
        dst: int,
        k: int,
    ) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in flights:
            graph[u].append((v, w))

        return self._dijkstra(graph, src, dst, k)

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
```

```

        dst: int,
        k: int,
    ) -> int:
        dist = [[math.inf] * (k + 2) for _ in range(len(graph))]

        dist[src][k + 1] = 0
        minHeap = [(dist[src][k + 1], src, k + 1)] # (d, u, stops)

        while minHeap:
            d, u, stops = heapq.heappop(minHeap)

            if u == dst:
                return d
            if stops == 0 or d > dist[u][stops]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v][stops - 1]:
                    dist[v][stops - 1] = d + w
                    heapq.heappush(minHeap, (dist[v][stops - 1], v, stops - 1))

        return -1

```

• LeetDoocs

```

class Solution:
    def findCheapestPrice(
        self, n: int, flights: List[List[int]], src: int, dst: int, k: int
    ) -> int:
        INF = 0x3F3F3F3F
        dist = [INF] * n
        dist[src] = 0
        for _ in range(k + 1):
            backup = dist.copy()
            for f, t, p in flights:
                dist[t] = min(dist[t], backup[f] + p)
        return -1 if dist[dst] == INF else dist[dst]

```

• SimplyLeet


```

import heapq

def findCheapestPrice(n, flights, src, dst, k):
    # Build graph: each city maps to a list of (neighboring city, flight price)
    graph = {i: [] for i in range(n)}
    for from_city, to_city, price in flights:
        graph[from_city].append((to_city, price))

    # Priority queue: (total_cost, current_city, stops so far)
    heap = [(0, src, 0)]

    # Dictionary to store the best cost to reach a city with a given stop count
    best = {}

    while heap:
        cost, city, stops = heapq.heappop(heap)
        # Return cost if destination is reached
        if city == dst:
            return cost

        # If the current number of stops exceeds k, skip further processing
        if stops > k:
            continue

        # Expand the neighboring flights
        for neighbor, price in graph[city]:
            new_cost = cost + price
            # Only proceed if this route is cheaper or not yet explored for
            # this many stops
            if (neighbor, stops + 1) not in best or new_cost < best[(neighbor,
            stops + 1)]:
                best[(neighbor, stops + 1)] = new_cost

            heapq.heappush(heap, (new_cost, neighbor, stops + 1))

    return -1

# Example usage:
print(findCheapestPrice(4, [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]],
, 0, 3, 1))

```

• LC.ca

```
class Solution:
    def findCheapestPrice(
        self, n: int, flights: List[List[int]], src: int, dst: int, k: int
    ) -> int:
        INF = 0x3F3F3F3F
        dist = [INF] * n
        dist[src] = 0
        for _ in range(k + 1):
            backup = dist.copy()
            for f, t, p in flights:
                dist[t] = min(dist[t], backup[f] + p)
        return -1 if dist[dst] == INF else dist[dst]
```

◆ Quick Review

Key Insights

- Use a variant of BFS combined with a min-heap (priority queue) to always expand the cheapest route.
- Each state in the queue includes the current cost, current city, and the number of stops used so far.
- Track the best-known cost to reach a city with a certain number of stops to prune non-optimal routes.
- Ensure that you do not exceed k stops during exploration.

Space & Time

Time Complexity: $O(E \log(E))$ in the worst-case scenario, where E is the number of flights, due to the heap operations.

Space Complexity: $O(n + E)$ for storing the graph and auxiliary data structures like the heap and best-cost map.

Solution

We solve the problem using a modified Dijkstra's algorithm. The approach employs a min-heap to ensure that we always process the state with the lowest cumulative cost next. Each state tracks the total cost so far, the current city, and the number of stops made. When a state reaches the destination city, we return its cost. To avoid unnecessary work, we record the lowest cost at which each city has been reached with a given number of stops. This state-tracking helps prune routes that are more expensive or exceed the allowed number of stops.

Advanced Graphs

□ #1584 Min Cost to Connect All Points

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Union-Find · Graph Theory · Minimum
Spanning Tree

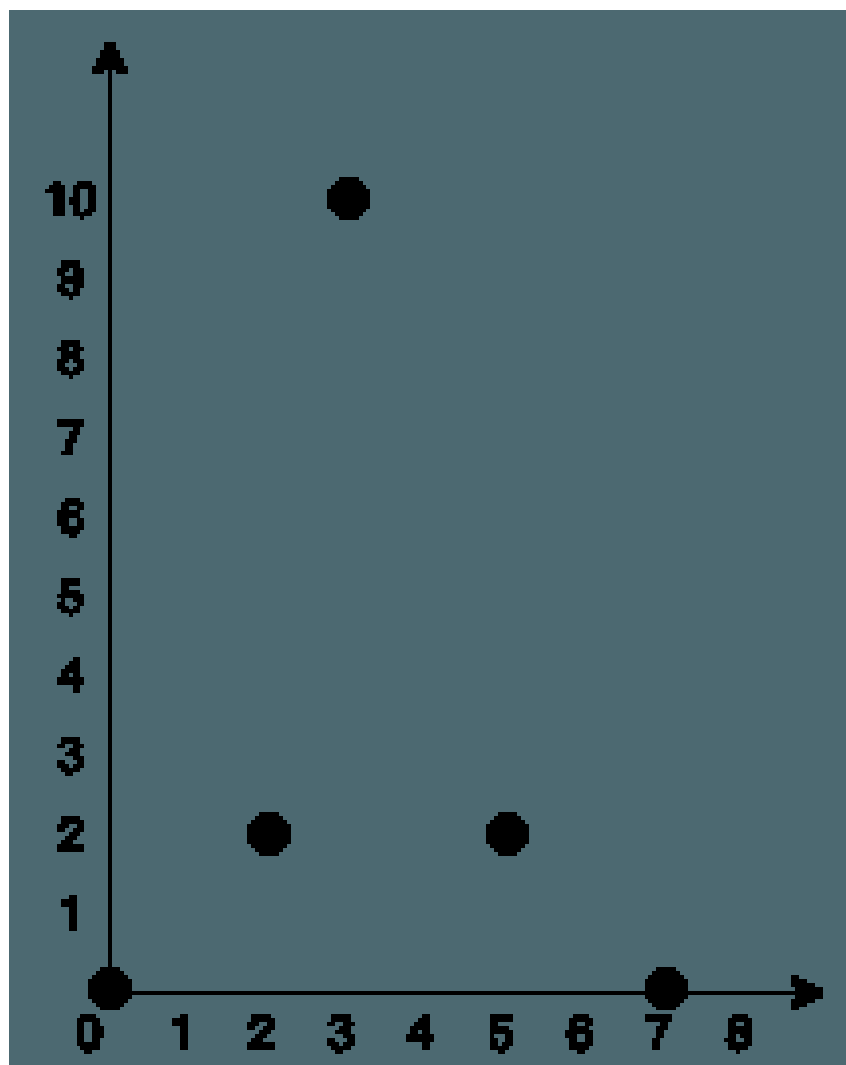
Lists: NeetCode 150

Problem

You are given an array `points` representing integer coordinates of some points on a 2D-plane, where `points[i] = [xi, yi]`.

The cost of connecting two points `[xi, yi]` and `[xj, yj]` is the manhattan distance between them: $|xi - xj| + |yi - yj|$, where $|val|$ denotes the absolute value of `val`.

Return the minimum cost to make all points connected. All points are connected if there is exactly one simple path between any two points.
Example 1:



Input: points = [[0,0],[2,2],[3,10],[5,2],[7,0]]

Output: 20

Explanation:

We can connect the points as shown above to get the minimum cost of 20.
Notice that there is a unique path between every pair of points.

Example 2:

Input: points = [[3,12],[-2,5],[-4,1]]

Output: 18

Constraints:

- $1 \leq \text{points.length} \leq 1000$
- $-106 \leq x_i, y_i \leq 106$

- All pairs (xi, yi) are distinct.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minCostConnectPoints(self, points: list[int]) -> int:
        # dist[i] := the minimum distance to connect the points[i]
        dist = [math.inf] * len(points)
        ans = 0

        for i in range(len(points) - 1):
            for j in range(i + 1, len(points)):
                # Try to connect the points[i] with the points[j].
                dist[j] = min(dist[j], abs(points[i][0] - points[j][0]) +
                               abs(points[i][1] - points[j][1]))
                # Swap the points[j] (the point with the minimum distance) with the
                # points[i + 1].
                if dist[j] < dist[i + 1]:
                    points[j], points[i + 1] = points[i + 1], points[j]
                    dist[j], dist[i + 1] = dist[i + 1], dist[j]
            ans += dist[i + 1]

        return ans
```

• LeetDoocs

```
class Solution:
    def minCostConnectPoints(self, points: List[List[int]]) -> int:
        n = len(points)
        g = [[0] * n for _ in range(n)]
        dist = [inf] * n
        vis = [False] * n
        for i, (x1, y1) in enumerate(points):
            for j in range(i + 1, n):
                x2, y2 = points[j]
                t = abs(x1 - x2) + abs(y1 - y2)
```

```

        g[i][j] = g[j][i] = t
    dist[0] = 0
    ans = 0
    for _ in range(n):
        i = -1
        for j in range(n):
            if not vis[j] and (i == -1 or dist[j] < dist[i]):
                i = j
        vis[i] = True
        ans += dist[i]

```

```

        for j in range(n):
            if not vis[j]:
                dist[j] = min(dist[j], g[i][j])
    return ans

```

```

class Solution:
    def minCostConnectPoints(self, points: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(points)
        g = []
        for i, (x1, y1) in enumerate(points):

```

```

            for j in range(i + 1, n):
                x2, y2 = points[j]
                t = abs(x1 - x2) + abs(y1 - y2)
                g.append((t, i, j))
        p = list(range(n))
        ans = 0
        for cost, i, j in sorted(g):
            pa, pb = find(i), find(j)
            if pa == pb:
                continue

```

```

        p[pa] = pb
        ans += cost
        n -= 1
        if n == 1:
            break
    return ans

```

• SimplyLeet

Python solution using Prim's algorithm

```
class Solution:
```

```
    def minCostConnectPoints(self, points):
```

```
        # number of points
```

```
        n = len(points)
```

```
        # initialize cost for each point as infinity, except the first point
```

```
        minCost = [float('inf')] * n
```

```
        minCost[0] = 0 # start from the first point
```

```
        # keep track of which points are included in the MST
```

```
        inMST = [False] * n
```

```
        # total cost of connecting all points
```

```
        total_cost = 0
```

```
        for _ in range(n):
```

```
            # select the next point with the smallest connection cost
```

```
            curr = -1
```

```
            curr_cost = float('inf')
```

```
            for i in range(n):
```

```
                if not inMST[i] and minCost[i] < curr_cost:
```

```
                    curr = i
```

```
                    curr_cost = minCost[i]
```

```
            # add the cost of the chosen point
```

```
            total_cost += curr_cost
```

```
            inMST[curr] = True
```

```
        # update the connection costs for the remaining points
```



```

        for next_point in range(n):
            if not inMST[next_point]:
                # compute the Manhattan distance between the current and
next point
                (x1, y1), (x2, y2) = points[curr], points[next_point]
                dist = abs(x1 - x2) + abs(y1 - y2)
                if dist < minCost[next_point]:
                    minCost[next_point] = dist

        return total_cost

```

```

# Example usage:
# sol = Solution()
# print(sol.minCostConnectPoints([[0,0],[2,2],[3,10],[5,2],[7,0]]))

```

◆ Quick Review

Key Insights

- The problem can be reinterpreted as finding a Minimum Spanning Tree (MST) on a complete graph.
- Manhattan distance is used to calculate the weight/cost between every pair of vertices.
- Prim's or Kruskal's algorithm can be applied to solve the MST problem.
- A direct construction of the complete graph is unnecessary; distances can be computed on the fly, which is efficient given the problem constraints.

Space & Time

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

Solution

We can solve the problem using Prim's algorithm. We start from an arbitrary point (for example, the first point) and iteratively add the point that has the smallest distance to the growing MST. Instead of explicitly constructing all the edges, we compute the Manhattan distance on the fly between the current MST and the remaining points. An array is used to track the minimum connection cost for each point that is not yet included in the tree. At each iteration, we select the point with the smallest cost to add into the MST and update the costs for the remaining points. This approach guarantees that we connect all points with the minimum total cost.

Round 3 · High · Hard

Round 3

High Priority · Hard

11 questions · 7 patterns

- ☐ ● Two Pointers #42 ↗
- ☐ ● Sliding Window #76 #239 ↗
- ☐ ● Binary Search #4 ↗
- ☐ ● Intervals #1851 ↗
- ☐ ● Trees #124 #297 ↗
- ☐ ● Graphs #127 ↗
- ☐ ● Advanced Graphs #269 #332 #778 ↗

Two Pointers

Round 3 · High · Hard · 1 question

Two Pointers

□ #42 Trapping Rain Water

HAR

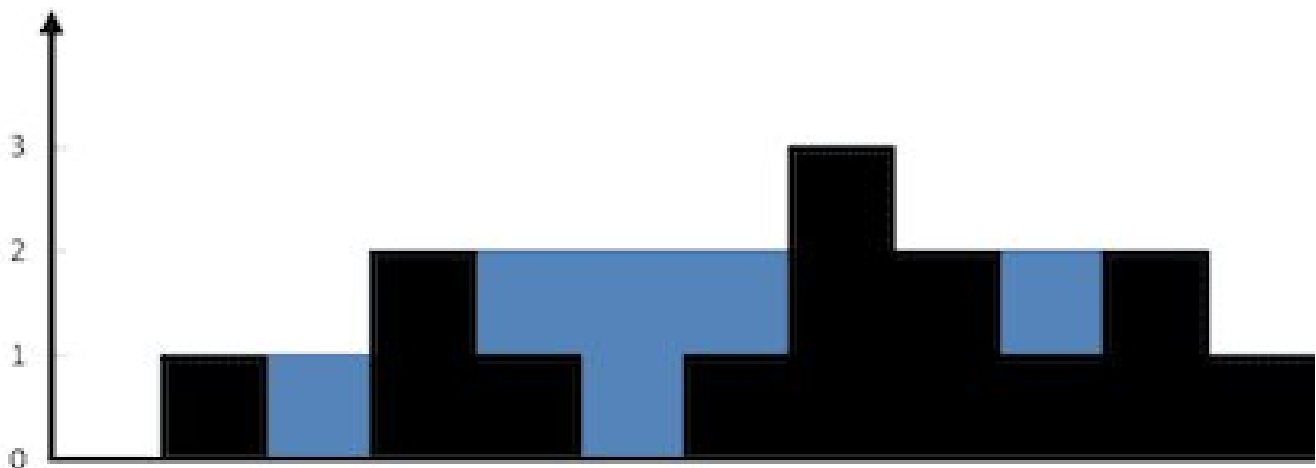
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Dynamic Programming ·
Stack · Monotonic Stack

Lists: Grind 169

Problem

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:



Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:

Input: height = [4,2,0,3,2,5]

Output: 9

Constraints:

- $n == \text{height.length}$
- $1 \leq n \leq 2 * 10^4$
- $0 \leq \text{height}[i] \leq 10^5$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def trap(self, height: list[int]) -> int:
        n = len(height)
        l = [0] * n # l[i] := max(height[0..i])
        r = [0] * n # r[i] := max(height[i..n))

        for i, h in enumerate(height):
            l[i] = h if i == 0 else max(h, l[i - 1])

        for i, h in reversed(list(enumerate(height))):
            r[i] = h if i == n - 1 else max(h, r[i + 1])

        return sum(min(l[i], r[i]) - h
                    for i, h in enumerate(height))
```

```
class Solution:
    def trap(self, height: list[int]) -> int:
        if not height:
            return 0

        ans = 0
        l = 0
        r = len(height) - 1
        maxL = height[l]
        maxR = height[r]
```

```
        while l < r:
            if maxL < maxR:
                ans += maxL - height[l]
                l += 1
                maxL = max(maxL, height[l])
            else:
                ans += maxR - height[r]
                r -= 1
                maxR = max(maxR, height[r])
```

```
        return ans
```

• LeetDoocs

```
class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        left = [height[0]] * n
        right = [height[-1]] * n
        for i in range(1, n):
            left[i] = max(left[i - 1], height[i])
            right[n - i - 1] = max(right[n - i], height[n - i - 1])
        return sum(min(l, r) - h for l, r, h in zip(left, right, height))
```

• SimplyLeet

```

# Python solution using the two pointers approach
def trap(height):
    # Initialize left and right pointers and water accumulator
    left, right = 0, len(height) - 1
    left_max, right_max = 0, 0
    water = 0

    # Traverse the elevation map from both ends
    while left < right:
        # Update left_max and right_max as we encounter new bars

        if height[left] < height[right]:
            # left side is the determining side
            if height[left] >= left_max:
                left_max = height[left] # update left max since current height
is higher
            else:
                water += left_max - height[left] # water trapped at left
position
            left += 1 # move left pointer to the right
        else:
            # right side is the determining side
            if height[right] >= right_max:
                right_max = height[right] # update right max since current
height is higher
            else:
                water += right_max - height[right] # water trapped at right
position
            right -= 1 # move right pointer to the left
    return water

# Example usage:
print(trap([0,1,0,2,1,0,1,3,2,1,2,1])) # Expected output: 6

```

• LC.ca


```

class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        if n < 3:
            return 0

        lmx = [height[0]] * n
        rmx = [height[n - 1]] * n
        for i in range(1, n):
            lmx[i] = max(lmx[i - 1], height[i]) # i self compare

            rmx[n - 1 - i] = max(rmx[n - i], height[n - 1 - i])

        # no negative, worst is, min(left,right) is itself
        return sum( min(lmx[i], rmx[i]) - height[i] for i in range(n) )

#####

class Solution: # zip
    def trap(self, height: List[int]) -> int:
        n = len(height)

        left = [height[0]] * n
        right = [height[-1]] * n
        for i in range(1, n):
            left[i] = max(left[i - 1], height[i])
            right[n - i - 1] = max(right[n - i], height[n - i - 1])
        return sum(min(l, r) - h for l, r, h in zip(left, right, height))

#####

class Solution: # find the highest bar first

    def trap(self, height: List[int]) -> int:
        if not height:
            return 0

        total_water = 0
        max_index = 0
        max_height = float('-inf')

        # Find the index of the maximum height
        for i in range(len(height)):

```

```
    if height[i] > max_height:
        max_height = height[i]
        max_index = i

    # Calculate water trapped on the left side of the maximum height
    left_max = height[0]
    for i in range(1, max_index):
        if left_max < height[i]:
            left_max = height[i]
        total_water += left_max - height[i]

    # Calculate water trapped on the right side of the maximum height
    right_max = height[-1]
    for i in range(len(height) - 2, max_index, -1):
        if right_max < height[i]:
            right_max = height[i]
        total_water += right_max - height[i]

    return total_water
```

◆ Quick Review

Key Insights

- The water trapped at any index is determined by the minimum of the maximum height on its left and right minus the height at that index.
- A two-pointers approach can efficiently solve the problem in one pass by maintaining left and right boundaries.
- Alternative approaches include dynamic programming with pre-computed left max and

right max arrays, or using a monotonic stack to determine boundaries.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$ when using the two-pointer approach

Solution

We use a two-pointers strategy by initializing pointers at the beginning and the end of the array. Keep track of the maximum height seen so far from the left (`left_max`) and the right (`right_max`). At each step, move the pointer with the lesser height because the water trapped is determined by the shorter boundary. Compute the water at that position as the difference between the boundary (`left_max` or `right_max`) and the current height, accumulating it into the total. This approach ensures linear time scanning with constant extra space.

Sliding Window

Round 3 · High · Hard · 2 questions

Sliding Window

□ #76 Minimum Window Substring

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

Given two strings s and t of lengths m and n respectively, return the minimum window substring of s such that every character in t (including duplicates) is included in the window. If there is no such substring, return the empty string `""`.

The testcases will be generated such that the answer is unique.

Example 1:

```
Input: s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"
Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string t.
```

Example 2:

```
Input: s = "a", t = "a"
Output: "a"
Explanation: The entire string s is the minimum window.
```

Example 3:

Input: s = "a", t = "aa"

Output: ""

Explanation: Both 'a's from t must be included in the window.
Since the largest window of s only has one 'a', return empty string.

Constraints:

- $m == s.length$
- $n == t.length$
- $1 \leq m, n \leq 105$
- s and t consist of uppercase and lowercase English letters.

Follow up: Could you find an algorithm that runs in $O(m + n)$ time?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        count = collections.Counter(t)
        required = len(t)
        bestLeft = -1
        minLength = len(s) + 1

        l = 0
        for r, c in enumerate(s):
            count[c] -= 1
```

```

if count[c] >= 0:
    required -= 1
while required == 0:
    if r - l + 1 < minLength:
        bestLeft = l
        minLength = r - l + 1
    count[s[l]] += 1
    if count[s[l]] > 0:
        required += 1
    l += 1

return '' if bestLeft == -1 else s[bestLeft: bestLeft + minLength]

```

• LeetCode

```

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        need = Counter(t)
        window = Counter()
        cnt = l = 0
        k, mi = -1, inf
        for r, c in enumerate(s):
            window[c] += 1
            if need[c] >= window[c]:
                cnt += 1

        while cnt == len(t):
            if r - l + 1 < mi:
                mi = r - l + 1
                k = l
            if need[s[l]] >= window[s[l]]:
                cnt -= 1
            window[s[l]] -= 1
            l += 1
        return '' if k < 0 else s[k : k + mi]

```

• SimplyLeet

```
# Python implementation of Minimum Window Substring solution
```

```
def minWindow(s: str, t: str) -> str:
```

```
    # If t is longer than s, no solution exists.
```

```
    if len(t) > len(s):
```

```
        return ""
```

```
    # Dictionary to store the frequency of each character in t.
```

```
    t_freq = {}
```

```
    for char in t:
```

```
        t_freq[char] = t_freq.get(char, 0) + 1
```

```
    # Dictionary to keep track of the characters in the current window.
```

```
    window_freq = {}
```

```
    required = len(t_freq) # Unique characters in t to be matched
```

```
    formed = 0 # Number of unique characters in the current window that meet
    required frequency
```

```
    # (window_length, left, right)
```

```
    answer = (float('inf'), None, None)
```

```
    l = 0 # Left pointer
```

```
    # Start sliding the window with the right pointer.
```

```
    for r, char in enumerate(s):
```

```
        # Add the current character to the window counter.
```

```
        window_freq[char] = window_freq.get(char, 0) + 1
```

```
        # Check if the current character's frequency in the window matches that
    in t.
```

```
        if char in t_freq and window_freq[char] == t_freq[char]:
```

```
            formed += 1
```



```

        # Try and contract the window if all characters are matched.
        while l <= r and formed == required:
            character = s[l]

            # Update answer if the current window is smaller than the previous
            best.

            if r - l + 1 < answer[0]:
                answer = (r - l + 1, l, r)

            # The character at the left is going to be removed from the window.

            window_freq[character] -= 1
            if character in t_freq and window_freq[character] < t_freq[character]:
                formed -= 1

            # Move the left pointer ahead.
            l += 1

        # Return the smallest window, if found.
        return "" if answer[0] == float('inf') else s[answer[1]:answer[2]+1]

# Example usage:
if __name__ == "__main__":
    s = "ADOBECODEBANC"
    t = "ABC"
    print(minWindow(s, t)) # Expected output: "BANC"

```

• LC.ca

```

'''
>>> deq = collections.deque([])
>>> deq.append(11)
>>> deq.append(22)
>>> deq.append(33)
>>>
>>> deq[0]
11
'''

```

```

import collections

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        cnt = 0
        need = collections.Counter(t)
        start, end = len(s), 3 * len(s) # Arbitrary 3, just make sure end-start
        # is larger than input s length, for later min-check
        d = {}
        # Using queue to store indexes, good for a large amount of API calls
        deq = collections.deque([])

        for i, c in enumerate(s):
            if c in need:
                deq.append(i)
                d[c] = d.get(c, 0) + 1
                # '=' also +1, because it's increased already one line above :)
                if d[c] <= need[c]:
                    cnt += 1
                while deq and d[s[deq[0]]] > need[s[deq[0]]]:
                    d[s[deq.popleft()]] -= 1
                if cnt == len(t) and deq[-1] - deq[0] < end - start:

                    start, end = deq[0], deq[-1]
        return s[start:end + 1]

#####

from collections import Counter

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        ans = ''

```

```

m, n = len(s), len(t)
if m < n:
    return ans
need = Counter(t)
window = Counter()
i, cnt, mi = 0, 0, inf
for j, c in enumerate(s):
    window[c] += 1
    if need[c] >= window[c]: # >= , because 1 line above already +=1
        cnt += 1

    while cnt == n:
        if j - i + 1 < mi: # in while
            mi = j - i + 1
            ans = s[i : j + 1]
            c = s[i]
            if need[c] >= window[c]: # char in window but not in need,
                need[c]=0, window[c]=1..2..
                cnt -= 1
            window[c] -= 1
            i += 1
    return ans

```

◆ Quick Review

Key Insights

- Use a sliding window approach to dynamically adjust the window boundaries.
- Utilize a hash table (or dictionary) to keep track of the frequency of characters required (from t) and the frequency within the current window.
- Expand the window until it satisfies the requirements, then contract to minimize the

window.

- Efficiently update counts and check valid window with two pointers (left and right).

Space & Time

Time Complexity: $O(m + n)$, where m is the length of s and n is the length of t .

Space Complexity: $O(m + n)$ in the worst case due to the storage for the hash maps.

Solution

We use a sliding window technique with two pointers representing the window's boundaries. First, create a frequency map of characters from t . Then, expand the right pointer to include characters in the window until the window contains all required characters (meeting or exceeding the counts in t). Once the requirement is met, try contracting the window by moving the left pointer to reduce its size while still maintaining a valid window. Throughout the process, if a valid window is found that is smaller than the previously recorded minimum window, update the minimum. The solution leans heavily on hash tables to track counts and compares

them against the required counts from t.

Sliding Window

□ #239 Sliding Window Maximum

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Queue · Sliding Window · Monotonic
 Queue

Lists: Grind 169

Problem

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window.

Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]
Explanation:
Window position Max
-----
[1 3 -1] -3 5 3 6 7 3
1 [3 -1 -3] 5 3 6 7 3
1 3 [-1 -3 5] 3 6 7 5
```

Example 2:

Input: nums = [1], k = 1

Output: [1]

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$
- $1 \leq k \leq \text{nums.length}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxSlidingWindow(self, nums: list[int], k: int) -> list[int]:
        ans = []
        maxQ = collections.deque()

        for i, num in enumerate(nums):
            while maxQ and maxQ[-1] < num:
                maxQ.pop()
            maxQ.append(num)
            if i >= k and nums[i - k] == maxQ[0]: # out-of-bounds

                maxQ.popleft()
            if i >= k - 1:
                ans.append(maxQ[0])

        return ans
```

• LeetDoocs

```

class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = [(-v, i) for i, v in enumerate(nums[: k - 1])]
        heapify(q)
        ans = []
        for i in range(k - 1, len(nums)):
            heappush(q, (-nums[i], i))
            while q[0][1] <= i - k:
                heappop(q)
            ans.append(-q[0][0])

        return ans

```

```

class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = deque()
        ans = []
        for i, x in enumerate(nums):
            if q and i - q[0] >= k:
                q.popleft()
            while q and nums[q[-1]] <= x:
                q.pop()
            q.append(i)

            if i >= k - 1:
                ans.append(nums[q[0]])
        return ans

```

• SimplyLeet

```

from collections import deque

def maxSlidingWindow(nums, k):
    # Create a deque to store indices of elements in the current window
    deq = deque()
    result = []

    for i, num in enumerate(nums):
        # Remove indices from the front if they are out of the current window
        if deq and deq[0] == i - k:

```



```

        deq.popleft()

    # Remove elements from the back while the current number is larger
    # than the number at the indices stored in the deque
    while deq and nums[deq[-1]] < num:
        deq.pop()

    # Append the current index to the deque
    deq.append(i)

    # If we've hit the window size, append the current max to result
    if i >= k - 1:
        result.append(nums[deq[0]])

    return result

# Example usage:
# print(maxSlidingWindow([1,3,-1,-3,5,3,6,7], 3))

```

• LC.ca

```

from collections import deque

class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = deque()
        ans = []
        for i, v in enumerate(nums):
            if q and i - k + 1 > q[0]:
                q.popleft() # remove index if out of window left
            while q and nums[q[-1]] <= v: # `<=`, not `<`, to ensure the bigger
                # index stored in deque
            q.append(i)
            if i >= k - 1:
                ans.append(nums[q[0]])
        return ans

```

```

        q.pop()
        q.append(i)
        if i >= k - 1:
            ans.append(nums[q[0]])
    return ans

#####

class Solution(object):
    def maxSlidingWindow(self, nums, k):

        """
        :type nums: List[int]
        :type k: int
        :rtype: List[int]
        """

        if k == 0:
            return []
        ans = [0 for _ in range(len(nums) - k + 1)]
        stack = collections.deque([])
        for i in range(0, k):

            while stack and nums[stack[-1]] < nums[i]:
                stack.pop()
            stack.append(i)
        ans[0] = nums[stack[0]]
        idx = 0
        for i in range(k, len(nums)):
            idx += 1
            if stack and stack[0] == i - k:
                stack.popleft()
            while stack and nums[stack[-1]] < nums[i]:
                stack.pop()
            stack.append(i)
            ans[idx] = nums[stack[0]]

        return ans

```

◆ Quick Review

Key Insights

- Use a deque (double-ended queue) to maintain the indices of potential maximum elements in a decreasing order.
- Ensure that the deque always has the current window's indices by removing elements that fall outside the current window.
- Only add elements that are greater than the ones at the back of the deque, thus preserving a monotonic decreasing order.
- The maximum for the current window is at the front of the deque.

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in `nums`, since each element is processed at most twice.

Space Complexity: $O(k)$, as the deque stores indices for at most k elements at any time.

Solution

The solution uses a monotonic deque to maintain indices of potential maximum elements in the current window. As the window slides, we: Remove indices from the front if they are out of the window. Remove

indices from the back if the current element is larger, because they cannot be the maximum if the current element is in the window. Append the current index. After the first k elements, the element at the front of the deque is the maximum for that window. This approach ensures that each element is pushed and popped from the deque only once, achieving an overall $O(n)$ time complexity.

Binary Search

Round 3 · High · Hard · 1 question

Binary Search

□ #4 Median of Two Sorted Arrays

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Divide and Conquer
Lists: Grind 169

Problem

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays.

The overall run time complexity should be $O(\log(m+n))$.

Example 1:

```
Input: nums1 = [1,3], nums2 = [2]
Output: 2.00000
Explanation: merged array = [1,2,3] and median is 2.
```

Example 2:

```
Input: nums1 = [1,2], nums2 = [3,4]
Output: 2.50000
Explanation: merged array = [1,2,3,4] and median is (2 + 3) / 2 = 2.5.
```

Constraints:

- `nums1.length == m`
- `nums2.length == n`
- $0 \leq m \leq 1000$

- $0 \leq n \leq 1000$
- $1 \leq m + n \leq 2000$
- $-106 \leq \text{nums1}[i], \text{nums2}[i] \leq 106$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMedianSortedArrays(self, nums1: list[int], nums2: list[int]) -> float:
        n1 = len(nums1)
        n2 = len(nums2)
        if n1 > n2:
            return self.findMedianSortedArrays(nums2, nums1)

        l = 0
        r = n1

        while l <= r:
            partition1 = (l + r) // 2
            partition2 = (n1 + n2 + 1) // 2 - partition1
            maxLeft1 = -2**31 if partition1 == 0 else nums1[partition1 - 1]
            maxLeft2 = -2**31 if partition2 == 0 else nums2[partition2 - 1]
            minRight1 = 2**31 - 1 if partition1 == n1 else nums1[partition1]
            minRight2 = 2**31 - 1 if partition2 == n2 else nums2[partition2]
            if maxLeft1 <= minRight2 and maxLeft2 <= minRight1:
                return (max(maxLeft1, maxLeft2) + min(minRight1, minRight2)) * 0.5 if
(n1 + n2) % 2 == 0 else max(maxLeft1, maxLeft2)
            elif maxLeft1 > minRight2:
                r = partition1 - 1
            else:
                l = partition1 + 1
```

• LeetDoocs

```

class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        def f(i: int, j: int, k: int) -> int:
            if i >= m:
                return nums2[j + k - 1]
            if j >= n:
                return nums1[i + k - 1]
            if k == 1:
                return min(nums1[i], nums2[j])
            p = k // 2
            x = nums1[i + p - 1] if i + p - 1 < m else inf
            y = nums2[j + p - 1] if j + p - 1 < n else inf
            return f(i + p, j, k - p) if x < y else f(i, j + p, k - p)

        m, n = len(nums1), len(nums2)
        a = f(0, 0, (m + n + 1) // 2)
        b = f(0, 0, (m + n + 2) // 2)
        return (a + b) / 2

```

• SimplyLeet

```

# Function to find the median of two sorted arrays
def findMedianSortedArrays(nums1, nums2):
    # Ensure nums1 is the smaller array for faster binary search
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1

    m, n = len(nums1), len(nums2)
    low, high = 0, m

    while low <= high:

```



```

# Partition indexes for nums1 and nums2
partition_nums1 = (low + high) // 2
partition_nums2 = (m + n + 1) // 2 - partition_nums1

# If partition is 0 then there are no elements on left side. Use
-infinity for comparison.
maxLeftNums1 = float('-inf') if partition_nums1 == 0 else nums1[partition_nums1 - 1]
# If partition is length then there are no elements on right side. Use
infinity for comparison.
minRightNums1 = float('inf') if partition_nums1 == m else nums1[partition_nums1]

# Similarly for nums2

maxLeftNums2 = float('-inf') if partition_nums2 == 0 else nums2[partition_nums2 - 1]
minRightNums2 = float('inf') if partition_nums2 == n else nums2[partition_nums2]

# Check if we have found the correct partition
if maxLeftNums1 <= minRightNums2 and maxLeftNums2 <= minRightNums1:
    # If total number of elements is even
    if (m + n) % 2 == 0:
        return (max(maxLeftNums1, maxLeftNums2) + min(minRightNums1, minRightNums2)) / 2.0
    else:
        return float(max(maxLeftNums1, maxLeftNums2))

# If not, adjust binary search range
elif maxLeftNums1 > minRightNums2:
    # Move partition in nums1 to left
    high = partition_nums1 - 1
else:
    # Move partition in nums1 to right
    low = partition_nums1 + 1

# Example usage:
print(findMedianSortedArrays([1,3], [2])) # Output: 2.0

print(findMedianSortedArrays([1,2], [3,4])) # Output: 2.5

```

• LC.ca

```

class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:

        def findKth(i: int, j: int, k: int) -> float:
            if i >= m:
                return nums2[j + k - 1]
            if j >= n:
                return nums1[i + k - 1]
            if k == 1:
                return min(nums1[i], nums2[j])

            # Division a // b : floordiv(a, b)
            midVal1 = nums1[i + k // 2 - 1] if i + k // 2 - 1 < m else math.in
f
            midVal2 = nums2[j + k // 2 - 1] if j + k // 2 - 1 < n else math.in
f

            if midVal1 < midVal2:
                return findKth(i + k // 2, j, k - k // 2) # '+' or '-' k//2
            else:
                return findKth(i, j + k // 2, k - k // 2)

        m = len(nums1)
        n = len(nums2)
        # Division a // b : floordiv(a, b)
        left = (m + n + 1) // 2
        right = (m + n + 2) // 2
        return (findKth(0, 0, left) + findKth(0, 0, right)) / 2.0

#####

# iteration version

```

```

class Solution(object):
    def findMedianSortedArrays(self, nums1, nums2):
        a, b = sorted((nums1, nums2), key=len)
        m, n = len(a), len(b)
        after = (m + n - 1) / 2
        lo, hi = 0, m
        while lo < hi:
            i = (lo + hi) / 2
            if after - i - 1 < 0 or a[i] >= b[after - i - 1]:
                hi = i
            else:
                lo = i + 1
        i = lo
        nextfew = sorted(a[i:i + 2] + b[after - i:after - i + 2])
        return (nextfew[0] + nextfew[1 - (m + n) % 2]) / 2.0

```

◆ Quick Review

Key Insights

- The median divides the sorted array into two equal halves.
- We can perform binary search on the smaller array to partition both arrays correctly.
- We need to ensure that every element in the left partition is less than or equal to every element in the right partition.
- Edge cases must be handled when partitions are at the boundaries of either array.

Space & Time

Time Complexity: $O(\log(\min(m, n)))$

Space Complexity: $O(1)$

Solution

The solution uses a binary search technique on the smaller array. We define a partition index i for `nums1` and compute the corresponding index j for `nums2` such that $i + j$ equals half of the total number of elements. The algorithm checks if the largest element on the left side of `nums1` (if any) is less than or equal to the smallest element on the right side of `nums2`, and vice versa for `nums2`. If the condition holds, we have found the correct partition and compute the median. If not, adjust the binary search range until the correct partition is found. This partitioning strategy allows us to find the median in logarithmic time.

Intervals

Round 3 · High · Hard · 1 question

Intervals

□ #1851 Minimum Interval to Include Each Query **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Sweep Line · Sorting ·
Heap (Priority Queue)

Lists: NeetCode 150

Problem

You are given a 2D integer array `intervals`, where `intervals[i] = [lefti, righti]` describes the *i*th interval starting at `lefti` and ending at `righti` (inclusive). The size of an interval is defined as the number of integers it contains, or more formally $righti - lefti + 1$.

You are also given an integer array `queries`. The answer to the *j*th query is the size of the smallest interval *i* such that $lefti \leq queries[j] \leq righti$. If no such interval exists, the answer is `-1`.

Return an array containing the answers to the queries.

Example 1:

Input: intervals = [[1,4],[2,4],[3,6],[4,4]], queries = [2,3,4,5]

Output: [3,3,1,4]

Explanation: The queries are processed as follows:

- Query = 2: The interval [2,4] is the smallest interval containing 2. The answer is $4 - 2 + 1 = 3$.
- Query = 3: The interval [2,4] is the smallest interval containing 3. The answer is $4 - 2 + 1 = 3$.
- Query = 4: The interval [4,4] is the smallest interval containing 4. The answer is $4 - 4 + 1 = 1$.
- Query = 5: The interval [3,6] is the smallest interval containing 5. The answer is $6 - 3 + 1 = 4$.

Example 2:

Input: intervals = [[2,3],[2,5],[1,8],[20,25]], queries = [2,19,5,22]

Output: [2,-1,4,6]

Explanation: The queries are processed as follows:

- Query = 2: The interval [2,3] is the smallest interval containing 2. The answer is $3 - 2 + 1 = 2$.
- Query = 19: None of the intervals contain 19. The answer is -1.
- Query = 5: The interval [2,5] is the smallest interval containing 5. The answer is $5 - 2 + 1 = 4$.
- Query = 22: The interval [20,25] is the smallest interval containing 22. The answer is $25 - 20 + 1 = 6$.

Constraints:

- $1 \leq \text{intervals.length} \leq 105$
- $1 \leq \text{queries.length} \leq 105$
- $\text{intervals}[i].\text{length} == 2$
- $1 \leq \text{lefti} \leq \text{righti} \leq 10^7$
- $1 \leq \text{queries}[j] \leq 10^7$

Community Solutions (Python)

- LeetDoocs

```

class Solution:
    def minInterval(self, intervals: List[List[int]], queries: List[int]) -> List[int]:
        n, m = len(intervals), len(queries)
        intervals.sort()
        queries = sorted((x, i) for i, x in enumerate(queries))
        ans = [-1] * m
        pq = []
        i = 0
        for x, j in queries:
            while i < n and intervals[i][0] <= x:
                a, b = intervals[i]
                heappush(pq, (b - a + 1, b))
                i += 1
            while pq and pq[0][1] < x:
                heappop(pq)
            if pq:
                ans[j] = pq[0][0]
        return ans

```

• SimplyLeet

```

import heapq

def minInterval(intervals, queries):
    # Sort intervals by their starting point.
    intervals.sort(key=lambda itv: itv[0])

    # Prepare queries sorted by their value, keeping track of original index.
    sorted_queries = sorted([(q, i) for i, q in enumerate(queries)])

    result = [-1] * len(queries)

```



```
min_heap = []
i = 0 # for intervals

# Process each query in ascending order.
for q, qi in sorted_queries:
    # Add all intervals that start <= current query.
    while i < len(intervals) and intervals[i][0] <= q:
        start, end = intervals[i]
        # Compute the size of the interval.
        size = end - start + 1

        # Push tuple: (size, end) into the min-heap.
        heapq.heappush(min_heap, (size, end))
        i += 1
    # Remove intervals that do not cover the current query.
    while min_heap and min_heap[0][1] < q:
        heapq.heappop(min_heap)
    # The top of the heap (if any) is the smallest interval covering the
    query.
    if min_heap:
        result[qi] = min_heap[0][0]
    return result

# Example usage
intervals = [[1,4],[2,4],[3,6],[4,4]]
queries = [2,3,4,5]
print(minInterval(intervals, queries)) # Output: [3, 3, 1, 4]
```

◆ Quick Review

Key Insights

- Sort the intervals by their start values to efficiently determine when an interval becomes relevant for a given query.
- Process queries in increasing order so that intervals can be added to a data structure

(min-heap) as they "start" to cover queries.

- Use a min-heap keyed by the size (length) of the interval to quickly retrieve the smallest interval currently containing the query.
- Remove any intervals from the heap that no longer cover the current query (i.e., their end is less than the query).
- Maintain the original order of queries by storing the indices while sorting.

Space & Time

Time Complexity: $O((N + Q) \log N)$ where N is the number of intervals and Q is the number of queries. Sorting and heap operations dominate the complexity.

Space Complexity: $O(N + Q)$ for storing intervals, queries, and the heap.

Solution

The solution uses a two-step approach:

- Sort the queries while keeping track of their original indices.
- Sort intervals by their starting values. For each query (in increasing order), add intervals that start before or at this query to a min-heap, where each entry is the size of the interval along with its end value.

Then, remove intervals from the heap that do not cover the query (their end value is less than the query). The heap's top element (if available) holds the smallest interval size that contains the query. – Place the answer in a result array at the position corresponding to the original query order.

Data structures used: Sorting arrays and a min-heap for efficient retrieval and removal.

Trees

Round 3 · High · Hard · 2 questions

Trees

□ #124 Binary Tree Maximum Path Sum

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Tree · DFS

Lists: Grind 169

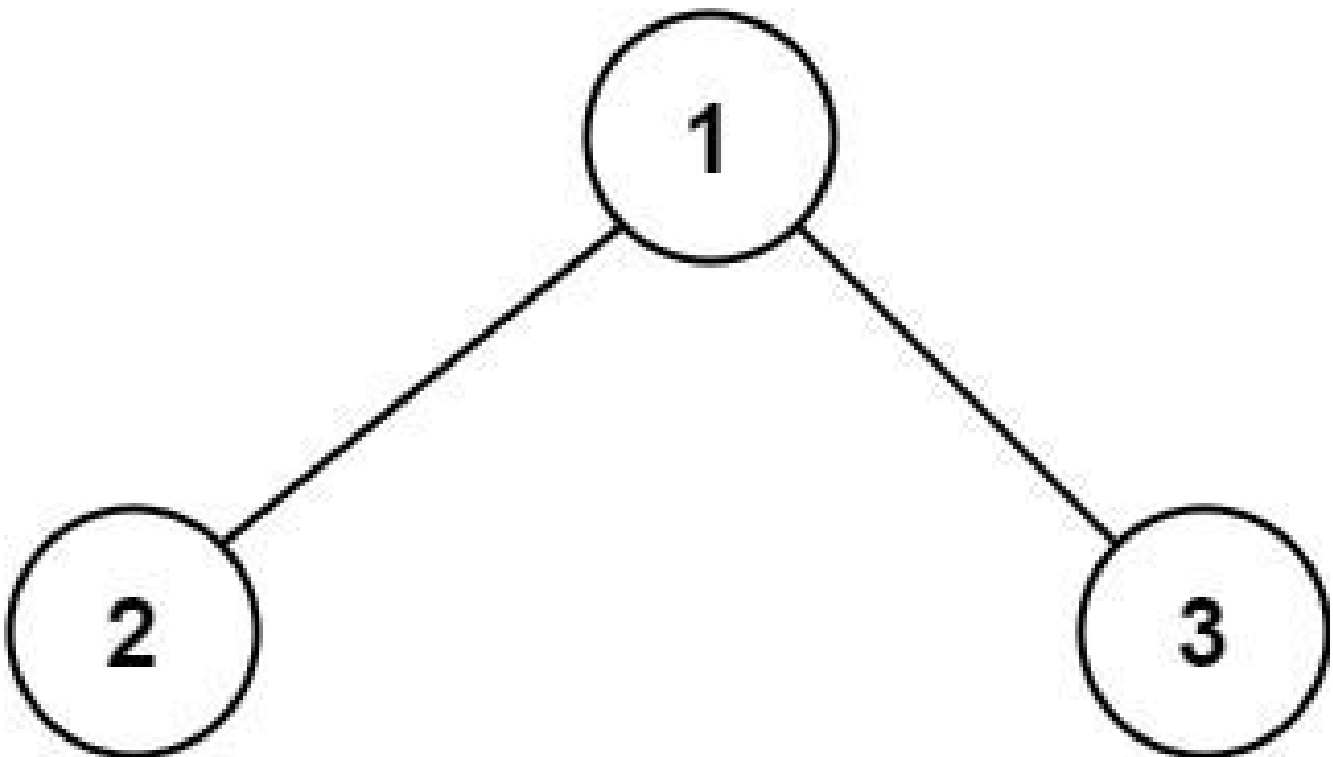
Problem

A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Note that the path does not need to pass through the root.

The path sum of a path is the sum of the node's values in the path.

Given the root of a binary tree, return the maximum path sum of any non-empty path.

Example 1:

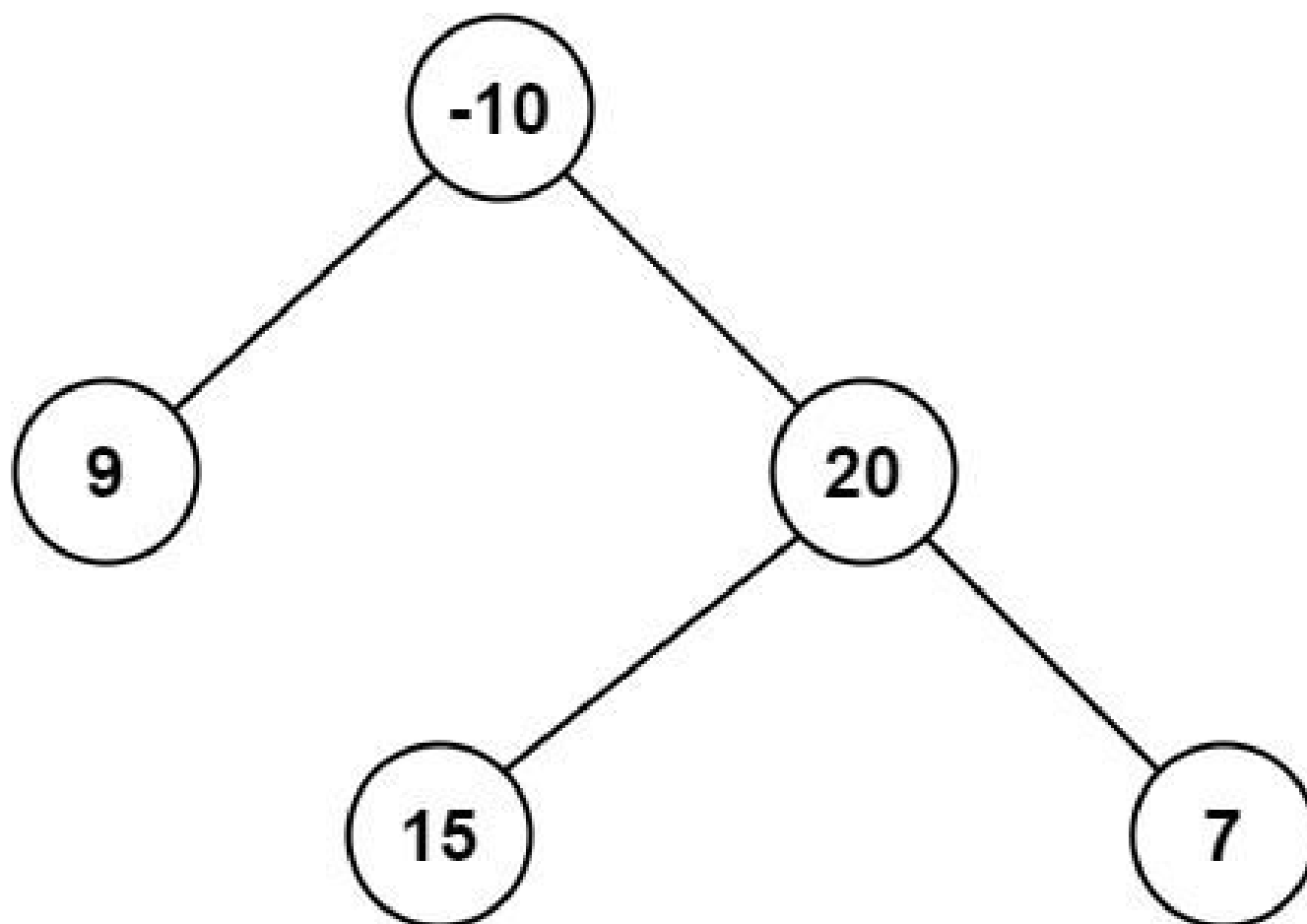


Input: root = [1,2,3]

Output: 6

Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of $2 + 1 + 3 = 6$.

Example 2:



Input: root = [-10,9,20,null,null,15,7]

Output: 42

Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of 15 + 20 + 7 = 42.

Constraints:

- The number of nodes in the tree is in the range [1, 3 * 10⁴].
- -1000 ≤ Node.val ≤ 1000

Community Solutions (Python)

- WalkCC

```

class Solution:
    def maxPathSum(self, root: TreeNode | None) -> int:
        ans = -math.inf

        def maxPathSumDownFrom(root: TreeNode | None) -> int:
            """
            Returns the maximum path sum starting from the current root, where
            root.val is always included.
            """
            nonlocal ans

            if not root:
                return 0

            l = max(0, maxPathSumDownFrom(root.left))
            r = max(0, maxPathSumDownFrom(root.right))
            ans = max(ans, root.val + l + r)
            return root.val + max(l, r)

        maxPathSumDownFrom(root)
        return ans

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:

```



```

        return 0
    left = max(0, dfs(root.left))
    right = max(0, dfs(root.right))
    nonlocal ans
    ans = max(ans, root.val + left + right)
    return root.val + max(left, right)

ans = -inf
dfs(root)
return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def maxPathSum(self, root: TreeNode) -> int:
        # Initialize global maximum path sum with a very small number

        self.global_max = float('-inf')

        def dfs(node: TreeNode) -> int:
            if not node:
                return 0

            # Recursively calculate max gain from left and right children,
            # discard negative gains
            left_gain = max(dfs(node.left), 0)
            right_gain = max(dfs(node.right), 0)

```


- Use Depth-First Search (DFS) to explore the tree in a post-order fashion.
- For each node, calculate the maximum gain including that node and optionally one of its subtrees.
- A node's best contribution to its parent can be either just its own value or its value plus the maximum gain from either the left or right child.
- Update a global maximum path sum that considers the possibility of combining both left and right gains with the current node.
- Handle negative values by comparing gains against 0 to avoid decreasing the path sum.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree, since each node is visited exactly once.

Space Complexity: $O(n)$ in the worst-case scenario due to the recursion stack ($O(H)$ where H is the height of the tree, which can be $O(n)$ for skewed trees).

Solution

We use a recursive DFS approach (post-order traversal) to compute the maximum gain from each subtree. For every node: Recursively compute the maximum gain from the left and right child, ensuring that negative gains are treated as 0. The maximum gain that can be provided to the node's parent is the node's value plus the larger gain from its left or right child. Simultaneously, the potential maximum path sum including the current node as the highest (or root) node is the node's value plus both left and right gains. Update a global variable that holds the maximum path sum found so far. Return the maximum gain (node value plus one best child) to be used by the node's parent. This strategy ensures that every possible path sum is considered while avoiding double-counting nodes.

Trees

□ #297 Serialize and Deserialize Binary Tree

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Tree · DFS · BFS · Design
Lists: Grind 169

Problem

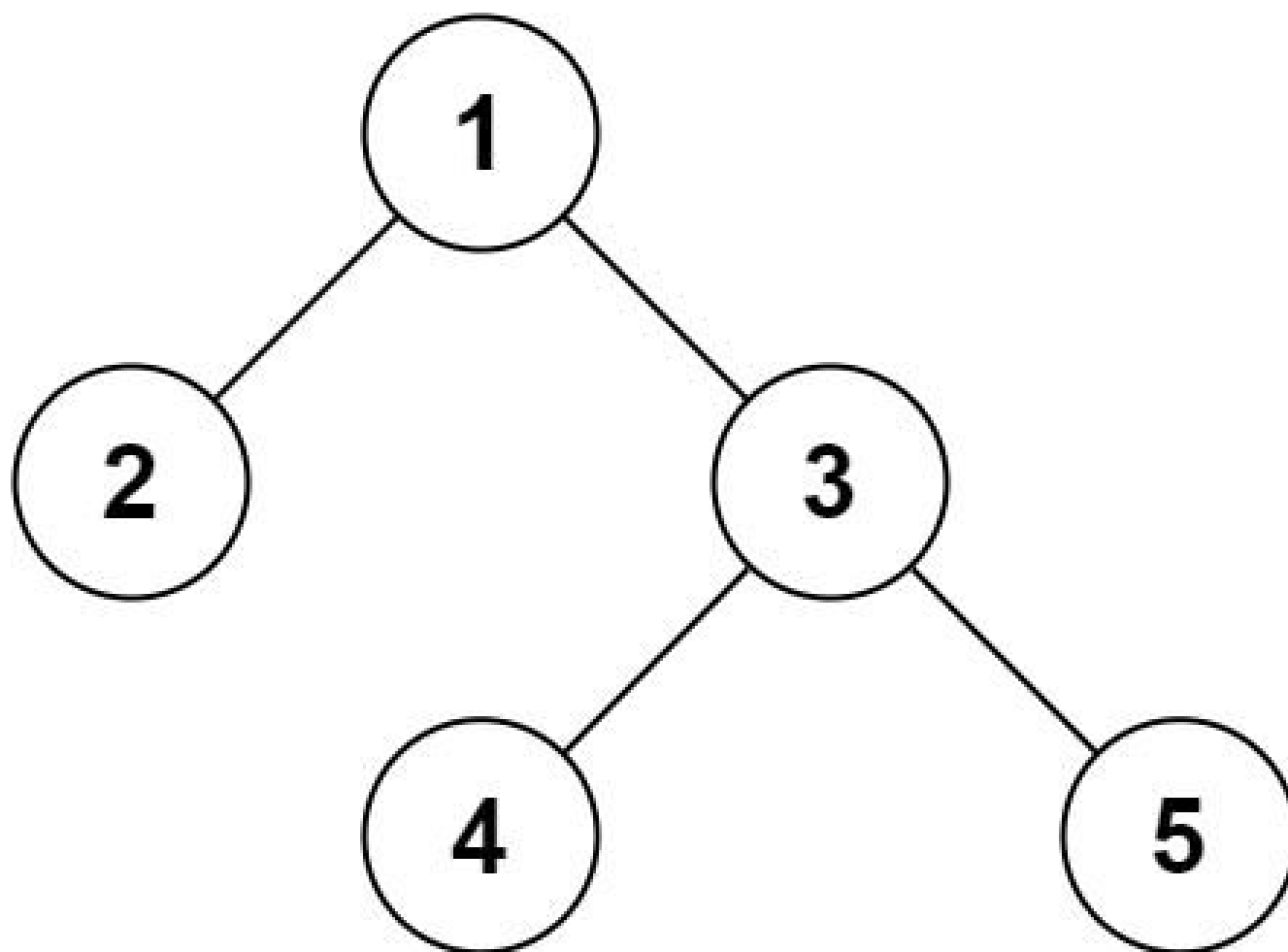
Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as how LeetCode serializes a binary tree.

You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:



Input: root = [1,2,3,null,null,4,5]

Output: [1,2,3,null,null,4,5]

Example 2:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-1000 \leq \text{Node.val} \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Codec:
    def serialize(self, root: 'TreeNode') -> str:
        """Encodes a tree to a single string."""
        if not root:
            return ''

        s = ''
        q = collections.deque([root])

        while q:

            node = q.popleft()
            if node:
                s += str(node.val) + ' '
                q.append(node.left)
                q.append(node.right)
            else:
                s += 'n '

        return s

    def deserialize(self, data: str) -> 'TreeNode':
        """Decodes your encoded data to tree."""
        if not data:
            return None

        vals = data.split()
        root = TreeNode(vals[0])
        q = collections.deque([root])

        for i in range(1, len(vals), 2):
```

```

node = q.popleft()
if vals[i] != 'n':
    node.left = TreeNode(vals[i])
    q.append(node.left)
if vals[i + 1] != 'n':
    node.right = TreeNode(vals[i + 1])
    q.append(node.right)

return root

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Codec:
    def serialize(self, root):

        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str
        """

        if root is None:
            return ""
        q = deque([root])
        ans = []
        while q:

```



```

        node = q.popleft()
        if node:
            ans.append(str(node.val))
            q.append(node.left)
            q.append(node.right)
        else:
            ans.append("#")
    return ",".join(ans)

```

```

def deserialize(self, data):

```

```

    """Decodes your encoded data to tree.

```

```

    :type data: str
    :rtype: TreeNode
    """

```

```

    if not data:
        return None
    vals = data.split(",")
    root = TreeNode(int(vals[0]))
    q = deque([root])

```

```

    i = 1
    while q:
        node = q.popleft()
        if vals[i] != "#":
            node.left = TreeNode(int(vals[i]))
            q.append(node.left)
        i += 1
        if vals[i] != "#":
            node.right = TreeNode(int(vals[i]))
            q.append(node.right)

```

```

        i += 1
    return root

```

```

# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.deserialize(codec.serialize(root))

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val      # value of the node
        self.left = left    # left child
        self.right = right   # right child

class Codec:
    # Encodes a tree to a single string using BFS.
    def serialize(self, root):

        if not root:
            return "" # Return empty string for an empty tree
        result = []
        queue = [root] # Initialize the queue with the root node
        while queue:
            node = queue.pop(0)
            if node:
                result.append(str(node.val)) # Append the value of the node
                queue.append(node.left)      # Enqueue left child
                queue.append(node.right)     # Enqueue right child
            else:
                result.append("null")        # Placeholder for null node
        return ",".join(result) # Join list with commas to form the string

    # Decodes the encoded data to reconstruct the tree using BFS.
    def deserialize(self, data):
        if not data:
            return None # Return None for empty string
        nodes = data.split(",")
        root = TreeNode(int(nodes[0])) # Create root node from first element

        queue = [root]
        i = 1 # Index for traversing nodes list
        while queue:
            current = queue.pop(0)
            if i < len(nodes) and nodes[i] != "null":
                current.left = TreeNode(int(nodes[i]))
                queue.append(current.left)
            i += 1
            if i < len(nodes) and nodes[i] != "null":
                current.right = TreeNode(int(nodes[i]))

```

```

        queue.append(current.right)
    i += 1
    return root

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

```

```

class Codec:
    def serialize(self, root):

```

```

        """Encodes a tree to a single string.

```

```

        :type root: TreeNode
        :rtype: str
        """

```

```

        if root is None:
            return ''
        res = []

```

```

        def preorder(root):
            if root is None:
                res.append("#,")
                return
            res.append(str(root.val) + ",")
            preorder(root.left)
            preorder(root.right)

```

```

        preorder(root)
        return ''.join(res)

```

```
def deserialize(self, data):
    """Decodes your encoded data to tree.

    :type data: str
    :rtype: TreeNode
    """
    if not data:
        return None
    vals = data.split(',')

```

```
def inner():
    first = vals.pop(0)
    if first == '#':
        return None
    return TreeNode(int(first), inner(), inner())
    # seems using a constructor __init__(val, left, right)

return inner()

```

```
# Your Codec object will be instantiated and called as such:
# ser = Codec()
# deser = Codec()
# ans = deser.deserialize(ser.serialize(root))

```

```
#####

```

```
from collections import deque

```

```
# bfs, each level based
class Codec:
    def serialize(self, root):
        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str
        """
        ret = []
        queue = deque([root])

```

```
while queue:
    top = queue.popleft()
    if not top:
        ret.append("None")
        continue
    else:
        ret.append(str(top.val))
        queue.append(top.left)
        queue.append(top.right)
return ",".join(ret)
```

```
def deserialize(self, data):
    """Decodes your encoded data to tree.

    :type data: str
    :rtype: TreeNode
    """
    data = data.split(",")
    if data[0] == "None":
        return None
```

```
    root = TreeNode(int(data[0]))
    queue = deque([root])
    i = 0
    while queue and i < len(data):
        top = queue.popleft()
        i += 1
        left = right = None
        if i < len(data) and data[i] != "None":
            left = TreeNode(int(data[i]))
            queue.append(left)
```

```
        i += 1
        if i < len(data) and data[i] != "None":
            right = TreeNode(int(data[i]))
            queue.append(right)

        top.left = left
        top.right = right

    return root
```

```
# Your Codec object will be instantiated and called as such:  
# codec = Codec()  
# codec.deserialize(codec.serialize(root))
```

◆ Quick Review

Key Insights

- The tree can be traversed using either depth-first search (DFS) or breadth-first search (BFS).
- Using BFS (level-order traversal) simplifies reconstruction since nodes are processed level-by-level.
- Placeholder markers (e.g., "null") are used to indicate missing children.
- Both serialization and deserialization processes should handle empty trees and edge cases gracefully.
- Ensuring a one-to-one mapping between the tree structure and the serialized string is critical.

Space & Time

Time Complexity: $O(n)$ for both serialization and deserialization, where n is the number of nodes in the tree.

Space Complexity: $O(n)$ due to the storage used for the serialized string and additional data structures (queue) during processing.

Solution

This solution uses level-order traversal (BFS) for both serialization and deserialization. During serialization, each node is processed sequentially with missing children recorded as "null". For deserialization, the string is split by commas, and a queue is used to rebuild the tree level by level. The root is created from the first token, then subsequent tokens are assigned as left and right children accordingly, ensuring an accurate reconstruction of the original tree.

Graphs

Round 3 · High · Hard · 1 question

Graphs

□ #127 Word Ladder

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · BFS
Lists: Grind 169

Problem

A transformation sequence from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord` → `s1` → `s2` → ... → `sk` such that:

- Every adjacent pair of words differs by a single letter.
- Every `si` for $1 \leq i \leq k$ is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- `sk == endWord`

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return the number of words in the shortest transformation sequence from `beginWord` to `endWord`, or 0 if no such sequence exists.

Example 1:

Input: beginWord = "hit", endWord = "cog", wordList = ["hot", "dot", "dog", "lot", "log", "cog"]

Output: 5

Explanation: One shortest transformation sequence is "hit" -> "hot" -> "dot" -> "dog" -> "cog", which is 5 words long.

Example 2:

Input: beginWord = "hit", endWord = "cog", wordList = ["hot", "dot", "dog", "lot", "log"]

Output: 0

Explanation: The endWord "cog" is not in wordList, therefore there is no valid transformation sequence.

Constraints:

- $1 \leq \text{beginWord.length} \leq 10$
- $\text{endWord.length} == \text{beginWord.length}$
- $1 \leq \text{wordList.length} \leq 5000$
- $\text{wordList}[i].\text{length} == \text{beginWord.length}$
- beginWord, endWord, and wordList[i] consist of lowercase English letters.
- $\text{beginWord} \neq \text{endWord}$
- All the words in wordList are unique.

Community Solutions (Python)

- ☐
- WalkCC

```
class Solution:
    def ladderLength(
        self,
        beginWord: str,
        endWord: str,
        wordList: list[str],
    ) -> int:
        wordSet = set(wordList)
        if endWord not in wordSet:
            return 0

        q = collections.deque([beginWord])

        step = 1
        while q:
            for _ in range(len(q)):
                wordList = list(q.popleft())
                for i, cache in enumerate(wordList):
                    for c in string.ascii_lowercase:
                        wordList[i] = c

                word = ''.join(wordList)
                if word == endWord:
                    return step + 1
                if word in wordSet:
                    q.append(word)
                    wordSet.remove(word)
                wordList[i] = cache
            step += 1

        return 0
```

• LeetDoocs

```

class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str])
-> int:
    words = set(wordList)
    q = deque([beginWord])
    ans = 1
    while q:
        ans += 1
        for _ in range(len(q)):
            s = q.popleft()
            s = list(s)

            for i in range(len(s)):
                ch = s[i]
                for j in range(26):
                    s[i] = chr(ord('a') + j)
                    t = ''.join(s)
                    if t not in words:
                        continue
                    if t == endWord:
                        return ans
                    q.append(t)

            words.remove(t)
            s[i] = ch

    return 0

```

• SimplyLeet

```

# Python solution using BFS

from collections import defaultdict, deque

def ladderLength(beginWord, endWord, wordList):
    # Create a set for fast lookup
    wordSet = set(wordList)
    if endWord not in wordSet:
        return 0

```

```

# Preprocessing: build a dictionary of intermediate states to words
L = len(beginWord)
all_combo_dict = defaultdict(list)
for word in wordSet:
    for i in range(L):
        # Key with a wildcard at the i-th position
        intermediate = word[:i] + '*' + word[i+1:]
        all_combo_dict[intermediate].append(word)

# Initialize BFS

queue = deque([(beginWord, 1)])
visited = set([beginWord])

while queue:
    current_word, level = queue.popleft()
    for i in range(L):
        intermediate = current_word[:i] + '*' + current_word[i+1:]
        # Check all words sharing the same intermediate state
        for word in all_combo_dict[intermediate]:
            if word == endWord:
                return level + 1
            if word not in visited:
                visited.add(word)
                queue.append((word, level + 1))
        # Once processed, clear the intermediate state list to save
        # processing time
        all_combo_dict[intermediate] = []
    return 0

# Example usage:
# print(ladderLength("hit", "cog", ["hot","dot","dog","lot","log","cog"]))

```

• LC.ca

```

# native BFS
class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str])
-> int:
    words = set(wordList)
    q = deque([beginWord])
    ans = 1
    while q:
        ans += 1
        for _ in range(len(q)):
            s = q.popleft()

            s = list(s) # must convert to list, cannot directly update
            s[i]='a'

            for i in range(len(s)):
                ch = s[i]
                for j in range(26):
                    # will re-generate s itself
                    # but s not in words-set, s removed after added to
                    words-set

                    s[i] = chr(ord('a') + j)
                    t = ''.join(s)
                    if t not in words:
                        continue

                    if t == endWord:
                        return ans
                    q.append(t)
                    words.remove(t) # equivalent to set t as visited
                s[i] = ch # restore

            return 0

#####

```

```

'''
BFS → BFS 問題を解くためのテンプレート

1. 問題の q1, q2 を初期化 → 問題の条件に従って初期化
2. 問題の m1, m2 を初期化 → 問題の条件に従って初期化
3. 問題の条件に従ってループを回す
4. 問題の条件に従ってループを回す
'''

class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str])
    -> int:

        def extend(m1, m2, q):
            for _ in range(len(q)):
                s = q.popleft() # so, every time, starting from start-word...
                # and later if t in m1 will skip repeated part...
                step = m1[s]
                s = list(s) # s = "abc", list(s) ==> ['a', 'b', 'c']
                for i in range(len(s)):
                    ch = s[i]
                    for j in range(26):
                        s[i] = chr(ord('a') + j)
                        t = ''.join(s)

                        if t in m1 or t not in words:
                            continue
                        if t in m2:
                            return step + 1 + m2[t]
                        m1[t] = step + 1
                        q.append(t)
                    s[i] = ch
            return -1

        words = set(wordList)

```

```

        if endWord not in words:
            return 0
        q1, q2 = deque([beginWord]), deque([endWord])
        m1, m2 = {beginWord: 0}, {endWord: 0}
        while q1 and q2:
            t = extend(m1, m2, q1) if len(q1) <= len(q2) else extend(m2, m1, q
2)

            if t != -1:
                return t + 1
        return 0

```

```
#####
```

```

import string
from collections import deque

```

```

class Solution(object):
    def ladderLength(self, beginWord, endWord, wordList):
        """

```

```

        :type beginWord: str

```

```

        :type endWord: str
        :type wordList: Set[str]
        :rtype: int
        """

```

```

    def getNbrs(src, dest, wordList):
        res = []
        for c in string.ascii_lowercase:
            for i in range(0, len(src)):
                newWord = src[:i] + c + src[i + 1:]

```



```

        if newWord == src:
            continue
        if newWord in wordList or newWord == dest:
            # about yield https://stackoverflow.com/questions/231767/what-does
            # the-yield-keyword-do-in-python
            # yield is a keyword that is used like return, except the function
            # will return a generator.
            yield newWord

queue = deque([beginWord])
length = 0
while queue:

    length += 1
    for k in range(0, len(queue)):
        top = queue.popleft()
        for nbr in getNbrs(top, endWord, wordList):
            wordList.remove(nbr)
            if nbr == endWord:
                return length + 1
            queue.append(nbr)
    return 0

#####

while q1 and q2:
    if len(q1) <= len(q2):
        # Prioritize the queue with fewer elements for expansion
        extend(m1, m2, q1)
    else:
        extend(m2, m1, q2)

```

```
def extend(m1, m2, q):  
    # New round of expansion  
    for _ in range(len(q)):  
        p = q.popleft()  
        step = m1[p]  
        for t in next(p):  
            if t in m1:  
                # Already visited before  
                continue  
            if t in m2:  
                # The other direction has been searched, indicating that a  
                # shortest path has been found  
                return step + 1 + m2[t]  
        q.append(t)  
        m1[t] = step + 1
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to explore transformations level by level to guarantee the shortest path.
- Preprocess words by generating intermediate representations (e.g., replacing each letter with a wildcard) to quickly find neighbors.
- Use a visited set to avoid cycles.
- Each transformation counts as a level in the BFS, so the answer is the number of levels traversed.

Space & Time

Time Complexity: $O(N * M^2)$ where N is the number of words and M is the length of each word.

Space Complexity: $O(N * M)$ for the storage of intermediate mappings and the BFS queue.

Solution

We solve the problem using a BFS approach. First, we preprocess the wordList by creating a mapping of all possible intermediate states. For example, for the word "dog" and wildcard position, we generate " og", "d g", and "do*". This helps us find all adjacent words (neighbors) that are only one letter different in constant time. We then initialize a queue with the beginWord and initiate the BFS. For each word dequeued, we explore all valid transformations using the precomputed intermediate mapping. If the endWord is reached during this process, we return the current level (transformation sequence length). Otherwise, we continue the BFS until all possibilities are exhausted. If no valid transformation sequence is found, we return 0.

Advanced Graphs

Round 3 · High · Hard · 3 questions

Advanced Graphs

□ ★ #269 Alien Dictionary ★

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · DFS · BFS · Graph · Topological
Sort

Lists: ★ Premium | Grind 169 | Premium 98

Problem

There is a new alien language that uses the English alphabet. However, the order of the letters is unknown to you.

You are given a list of strings words from the alien language's dictionary. Now it is claimed that the strings in words are sorted lexicographically by the rules of this new language.

If this claim is incorrect, and the given arrangement of string in words cannot correspond to any order of letters, return "".

Otherwise, return a string of the unique letters in the new alien language sorted in lexicographically increasing order by the new language's rules. If there are multiple solutions,

return any of them.

Example 1:

```
Input: words = ["wrt","wrf","er","ett","rftt"]  
Output: "wertf"
```

Example 2:

```
Input: words = ["z","x"]  
Output: "zx"
```

Example 3:

```
Input: words = ["z","x","z"]  
Output: ""  
Explanation: The order is invalid, so return "".
```

Constraints:

- $1 \leq \text{words.length} \leq 100$
- $1 \leq \text{words}[i].\text{length} \leq 100$
- $\text{words}[i]$ consists of only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:  
    def alienOrder(self, words: list[str]) -> str:  
        graph = {}  
        inDegrees = [0] * 26  
  
        self._buildGraph(graph, words, inDegrees)  
        return self._topology(graph, inDegrees)  
  
    def _buildGraph(  
        self,
```

```

graph: dict[str, set[str]],
words: list[str],
inDegrees: list[int],
) -> None:
    # Create a node for each character in each word.
    for word in words:
        for c in word:
            if c not in graph:
                graph[c] = set()

    for first, second in zip(words, words[1:]):
        length = min(len(first), len(second))
        for j in range(length):
            u = first[j]
            v = second[j]
            if u != v:
                if v not in graph[u]:
                    graph[u].add(v)
                    inDegrees[ord(v) - ord('a')] += 1
                break # The order of characters after this are meaningless.

    # First = 'ab', second = 'a' . invalid
    if j == length - 1 and len(first) > len(second):
        graph.clear()
        return

def _topology(self, graph: dict[str, set[str]], inDegrees: list[int]) -> str
:
    s = ''
    q = collections.deque()

    for c in graph:

```

```

    if inDegrees[ord(c) - ord('a')] == 0:
        q.append(c)

    while q:
        u = q.pop()
        s += u
        for v in graph[u]:
            inDegrees[ord(v) - ord('a')] -= 1
            if inDegrees[ord(v) - ord('a')] == 0:
                q.append(v)

    # Words = ['z', 'x', 'y', 'x']
    return s if len(s) == len(graph) else ''

```

• LeetDoocs

```

class Solution:
    def alienOrder(self, words: List[str]) -> str:
        g = [[False] * 26 for _ in range(26)]
        s = [False] * 26
        cnt = 0
        n = len(words)
        for i in range(n - 1):
            for c in words[i]:
                if cnt == 26:
                    break
                o = ord(c) - ord('a')
                if not s[o]:
                    cnt += 1
                    s[o] = True
            m = len(words[i])
            for j in range(m):
                if j >= len(words[i + 1]):
                    return ''
                c1, c2 = words[i][j], words[i + 1][j]
                if c1 == c2:

```



```

        continue
    o1, o2 = ord(c1) - ord('a'), ord(c2) - ord('a')
    if g[o2][o1]:
        return ''
    g[o1][o2] = True
    break
for c in words[n - 1]:
    if cnt == 26:
        break
    o = ord(c) - ord('a')

```

```

    if not s[o]:
        cnt += 1
        s[o] = True

indegree = [0] * 26
for i in range(26):
    for j in range(26):
        if i != j and s[i] and s[j] and g[i][j]:
            indegree[j] += 1
q = deque()

```

```

ans = []
for i in range(26):
    if s[i] and indegree[i] == 0:
        q.append(i)
while q:
    t = q.popleft()
    ans.append(chr(t + ord('a')))
    for i in range(26):
        if s[i] and i != t and g[t][i]:
            indegree[i] -= 1

```

```

        if indegree[i] == 0:
            q.append(i)
return '' if len(ans) < cnt else ''.join(ans)

```

• SimplyLeet

```
# Python solution for Alien Dictionary
from collections import defaultdict, deque

def alienOrder(words):
    # Create data structures: graph and in_degree for each unique character.
    graph = defaultdict(set)
    in_degree = {char: 0 for word in words for char in word}

    # Build the graph.
    for i in range(len(words) - 1):

        word1, word2 = words[i], words[i+1]
        # Check for invalid case where prefix of next word is the same but
        word1 is longer.
        if len(word1) > len(word2) and word1.startswith(word2):
            return ""

        # Compare the words character by character.
        for c1, c2 in zip(word1, word2):
            if c1 != c2:
                # If we haven't already recorded this edge, add it.
                if c2 not in graph[c1]:
                    graph[c1].add(c2)

                    in_degree[c2] += 1
                break

    # Prepare for topological sort by adding all nodes with in_degree == 0.
    queue = deque([char for char in in_degree if in_degree[char] == 0])
    order = []

    # Kahn's algorithm for topological sort.
    while queue:
        current = queue.popleft()
```

```

    order.append(current)
    for neighbor in graph[current]:
        in_degree[neighbor] -= 1
        if in_degree[neighbor] == 0:
            queue.append(neighbor)

    # Check if topological sort is possible.
    if len(order) == len(in_degree):
        return "".join(order)
    else:

        return ""

# Example usage:
print(alienOrder(["wrt","wrf","er","ett","rftt"]))

```

• LC.ca

```

'''
>>> a = ['a','b','c']
>>> zip(a, a[1:])
[('a', 'b'), ('b', 'c')]

# same as pairwise()
>>> from itertools import pairwise
>>> pairwise(a)
<itertools.pairwise object at 0x108458730>

>>> list(pairwise(a))
[('a', 'b'), ('b', 'c')]
'''

'''
>>> inDegree = {'a':11, 'b':22, 'c':33}

>>> [print(c) for c in inDegree.keys()]
a
b

```

```

c
[None, None, None]

# same as for c in inDegree.keys()
>>> [print(c) for c in inDegree]
a
b
c
[None, None, None]

>>> [print(c) for c in inDegree.values()]
11
22
33
[None, None, None]

>>> [print(c) for c in inDegree.items()]
('a', 11)
('b', 22)
('c', 33)

[None, None, None]
'''

from collections import defaultdict, deque
from typing import Dict, List, Set

class Solution:
    def alienOrder(self, words: List[str]) -> str:
        graph = defaultdict(set)
        inDegree = defaultdict(int)

```

```

self._buildGraph(graph, words, inDegree)
return self._topology(graph, inDegree)

def _buildGraph(self, graph: Dict[str, Set[str]], words: List[str], inDegree: Dict[str, int]) -> None:
    # Create a node for each character in each word
    for word in words:
        for c in word:
            inDegree[c] = 0 # necessary for final char counting

    for first, second in zip(words, words[1:]): # or pairwise(words)

        length = min(len(first), len(second))
        for j in range(length):
            u = first[j]
            v = second[j]
            if u != v:
                if v not in graph[u]:
                    graph[u].add(v)
                    inDegree[v] += 1
                break # Later characters' order is meaningless
        if j == length - 1 and len(first) > len(second):

            # If 'ab' comes before 'a', it's an invalid order
            graph.clear()
            return

    def _topology(self, graph: Dict[str, Set[str]], inDegree: Dict[str, int])
-> str:
    result = ''
    q = deque([c for c in inDegree if inDegree[c] == 0])

    while q:
        u = q.popleft()

```

```

        result += u
        for v in graph[u]:
            inDegree[v] -= 1
            if inDegree[v] == 0:
                q.append(v)

    # If there are remaining characters in inDegree, it means there's a
cycle
    if any(inDegree.values()):
        return ''

```

```

    # Words = ['z', 'x', 'y', 'x']
    return result if len(result) == len(indegree) else ''

```

```
#####
```

```
import collections
```

```
class Node(object):
    def __init__(self, val):

```

```

        self.val = val
        self.neighbors = []

```

```

    def connect(self, node):
        self.neighbors.append(node)

```

```

    def getNbrs(self):
        return self.neighbors

```

```

class Solution(object):
    def alienOrder(self, words):
        """
        :type words: List[str]
        :rtype: str
        """

    def dfs(root, graph, visited):
        visited[root] = 1
        for nbr in graph[root].getNbrs():

            if visited[nbr.val] == 0:
                if not dfs(nbr.val, graph, visited):
                    return False
            elif visited[nbr.val] == 1:
                return False

        visited[root] = 2
        self.ans += root
        return True

    self.ans = ""
    graph = {}
    visited = collections.defaultdict(int)
    self.topNum = 0
    for i in range(0, len(words) - 1):
        a = words[i]
        b = words[i + 1]
        i = 0
        while i < len(a) and i < len(b):
            if a[i] != b[i]:

                nodeA = nodeB = None
                if a[i] not in graph:
                    nodeA = Node(a[i])
                    graph[a[i]] = nodeA
                else:
                    nodeA = graph[a[i]]
                if b[i] not in graph:
                    nodeB = Node(b[i])
                    graph[b[i]] = nodeB
                else:

```

```

        nodeB = graph[b[i]]
        nodeA.connect(nodeB)
        break
    i += 1
    if i < len(a) and i >= len(b):
        return ""

    for c in graph:
        if visited[c] == 0:
            if not dfs(c, graph, visited):

                return ""

    unusedSet = set()
    for word in words:
        for c in word:
            unusedSet.add(c)

    for c in unusedSet:
        if c not in graph:
            self.ans += c

    return self.ans[::-1]

```

◆ Quick Review

Key Insights

- Build a graph where each unique character is a node.
- Compare adjacent words to determine the first different character, which gives the ordering (directed edge).
- Ensure all characters, even those without edges, are represented.

- Use a topological sort (using DFS or BFS) to determine a valid character ordering.
- Handle edge cases, such as words where a longer word appears before its prefix which is invalid.

Space & Time

Time Complexity: $O(N * L + V + E)$ where N is the number of words, L is the average length of each word, V is the number of unique characters, and E is the number of edges derived from adjacent word comparisons.

Space Complexity: $O(V + E)$ for storing the graph and additional data structures for topological sorting.

Solution

We use a graph-based approach to solve the problem. First, we initialize a graph (adjacency list) and in-degree dictionary for all unique characters. We then iterate through each pair of consecutive words, comparing them character by character to find the first pair that differs. This difference implies an order: the first character should come before the second character. We add a directed edge

accordingly and update the in-degree for the target character. To get the final ordering, we perform a topological sort using Kahn's algorithm: Enqueue characters with in-degree zero. Remove characters from the queue and append them to our result. Decrement the in-degree of neighboring nodes. If a neighbor's in-degree becomes zero, enqueue it. If the result's length equals the number of unique nodes, we have a valid ordering; otherwise, a cycle exists, and we return an empty string.

Advanced Graphs

□ #332 Reconstruct Itinerary

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Depth-First Search · Graph
Theory · Sorting · Heap (Priority Queue) ·
Eulerian Circuit

Lists: NeetCode 150

Problem

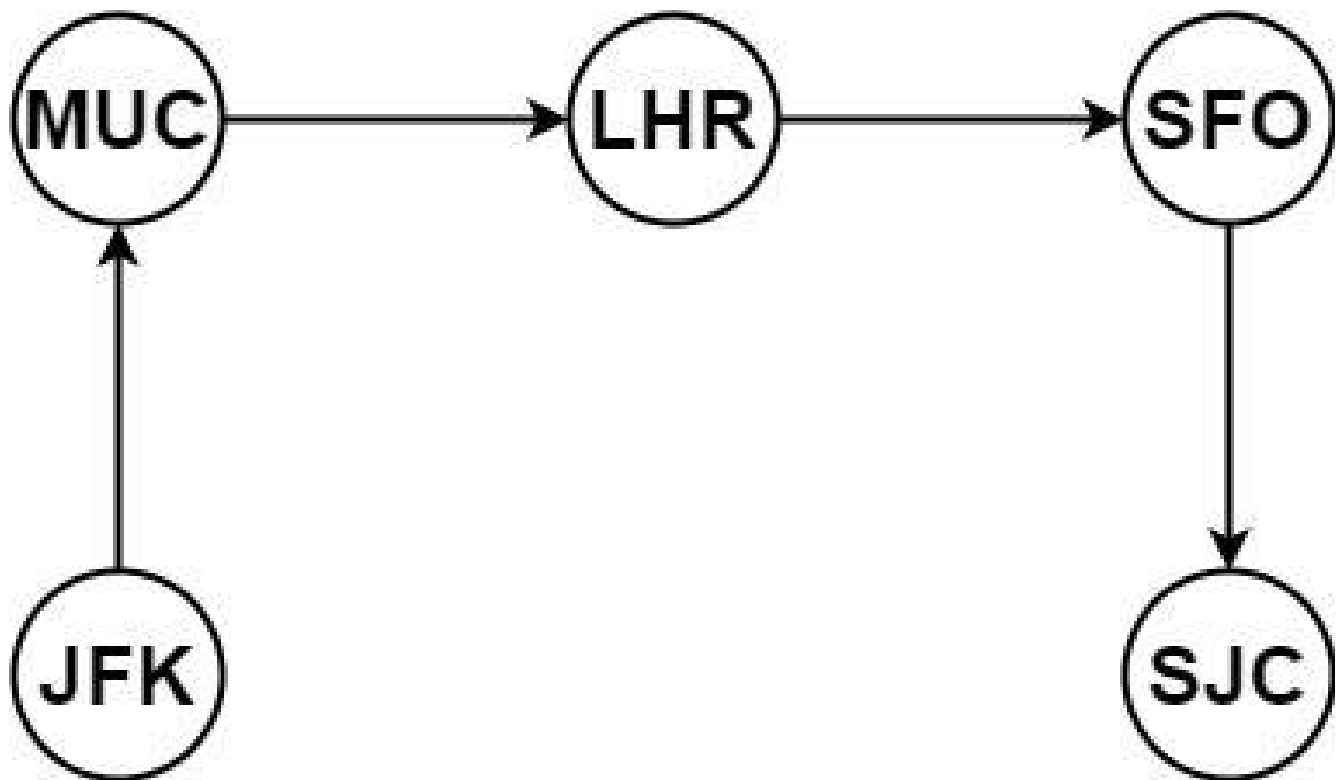
You are given a list of airline tickets where $\text{tickets}[i] = [\text{from}_i, \text{to}_i]$ represent the departure and the arrival airports of one flight.

Reconstruct the itinerary in order and return it. All of the tickets belong to a man who departs from "JFK", thus, the itinerary must begin with "JFK". If there are multiple valid itineraries, you should return the itinerary that has the smallest lexical order when read as a single string.

- For example, the itinerary ["JFK", "LGA"] has a smaller lexical order than ["JFK", "LGB"].

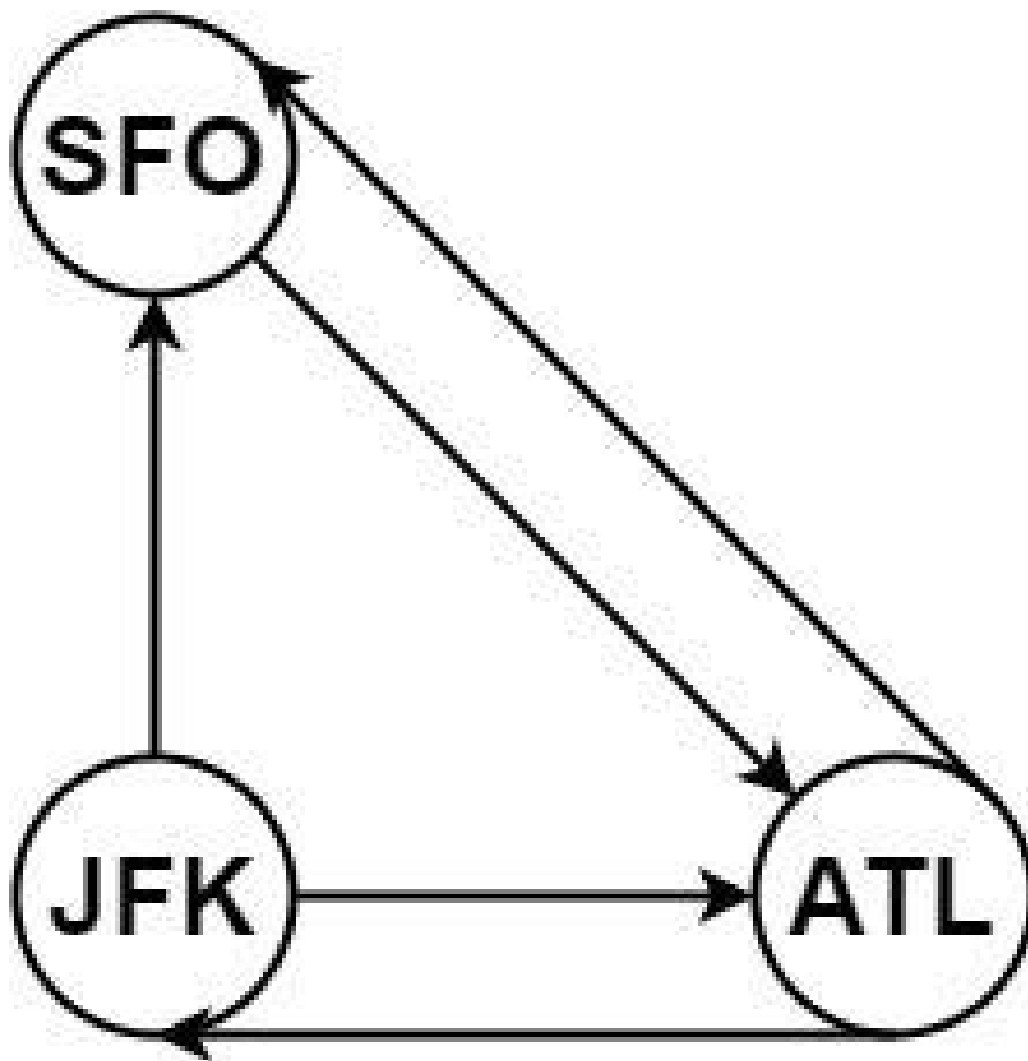
You may assume all tickets form at least one valid itinerary. You must use all the tickets once and only once.

Example 1:



Input: tickets = [["MUC", "LHR"], ["JFK", "MUC"], ["SFO", "SJC"], ["LHR", "SFO"]]
Output: ["JFK", "MUC", "LHR", "SFO", "SJC"]

Example 2:



Input: tickets =
[["JFK", "SFO"], ["JFK", "ATL"], ["SFO", "ATL"], ["ATL", "JFK"], ["ATL", "SFO"]]
Output: ["JFK", "ATL", "JFK", "SFO", "ATL", "SFO"]
Explanation: Another possible reconstruction is
["JFK", "SFO", "ATL", "JFK", "ATL", "SFO"] but it is larger in lexical order.

Constraints:

- $1 \leq \text{tickets.length} \leq 300$
- $\text{tickets}[i].\text{length} == 2$
- $\text{fromi}.\text{length} == 3$
- $\text{toi}.\text{length} == 3$

- fromi and toi consist of uppercase English letters.
- fromi != toi

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findItinerary(self, tickets: list[list[str]]) -> list[str]:
        ans = []
        graph = collections.defaultdict(list)

        for a, b in reversed(sorted(tickets)):
            graph[a].append(b)

        def dfs(u: str) -> None:
            while u in graph and graph[u]:

                dfs(graph[u].pop())
            ans.append(u)

        dfs('JFK')
        return ans[::-1]
```

• LeetDoocs

```
class Solution:
    def findItinerary(self, tickets: List[List[str]]) -> List[str]:
        def dfs(f: str):
            while g[f]:
                dfs(g[f].pop())
            ans.append(f)

        g = defaultdict(list)
        for f, t in sorted(tickets, reverse=True):
            g[f].append(t)
```

```
ans = []
dfs("JFK")
return ans[::-1]
```

• SimplyLeet

```
def findItinerary(tickets):
    # Build the graph as an adjacency list.
    graph = {}
    for src, dst in tickets:
        if src not in graph:
            graph[src] = []
        graph[src].append(dst)
    # Sort the destination lists in reverse lexicographical order.
    for src in graph:
        graph[src].sort(reverse=True)
```

```
itinerary = []
```

```
def dfs(airport):
    # Explore all flights out of the current airport.
    while airport in graph and graph[airport]:
        next_dest = graph[airport].pop()
        dfs(next_dest)
    itinerary.append(airport)
```

```
# Start DFS from 'JFK'
dfs("JFK")
# The itinerary is constructed in reverse.
return itinerary[::-1]
```

```
# Example usage:
tickets = [["JFK","SFO"], ["JFK","ATL"], ["SFO","ATL"], ["ATL","JFK"], ["ATL",
"SFO"]]
print(findItinerary(tickets))
```

◆ Quick Review

Key Insights

- Model the tickets as a directed graph where airports are nodes.
- The problem is akin to finding an Eulerian path through the graph.
- Use Depth-First Search (DFS) with Hierholzer's algorithm to construct the itinerary.
- To ensure lexical order, sort the destination lists in reverse order so that the smallest lexical destination can be popped during DFS.
- Build the itinerary in reverse during backtracking and then reverse it to get the final order.

Space & Time

Time Complexity: $O(E \log E)$, where E is the number of tickets, due to the sorting of edges.

Space Complexity: $O(E)$, to store the graph and recursion stack.

Solution

Construct an adjacency list (graph) mapping each departure airport to a list of destination airports sorted in reverse lexicographical order. Perform DFS starting from "JFK". During DFS, traverse while there are destinations

available from the current airport. Append airports to the itinerary as you backtrack. Reverse the built itinerary at the end to obtain the correct order.

Advanced Graphs

□ #778 Swim in Rising Water

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Depth-First Search ·
Breadth-First Search · Union-Find · Heap
(Priority Queue) · Matrix

Lists: NeetCode 150

Problem

You are given an $n \times n$ integer matrix grid where each value $\text{grid}[i][j]$ represents the elevation at that point (i, j) .

It starts raining, and water gradually rises over time. At time t , the water level is t , meaning any cell with elevation less than equal to t is submerged or reachable.

You can swim from a square to another 4-directionally adjacent square if and only if the elevation of both squares individually are at most t . You can swim infinite distances in zero time. Of course, you must stay within the boundaries of the grid during your swim.

Return the minimum time until you can reach the bottom right square ($n - 1, n - 1$) if you start at the top left square $(0, 0)$.

Example 1:

0	2
1	3

Input: `grid = [[0,2],[1,3]]`

Output: 3

Explanation:

At time 0, you are in grid location $(0, 0)$.

You cannot go anywhere else because 4-directionally adjacent neighbors have a higher elevation than $t = 0$.

You cannot reach point $(1, 1)$ until time 3.

When the depth of water is 3, we can swim anywhere inside the grid.

Example 2:

0	1	2	3	4
24	23	22	21	5
12	13	14	15	16
11	17	18	19	20
10	9	8	7	6

Input: grid =

[[0,1,2,3,4],[24,23,22,21,5],[12,13,14,15,16],[11,17,18,19,20],[10,9,8,7,6]]

Output: 16

Explanation: The final route is shown.

We need to wait until time 16 so that (0, 0) and (4, 4) are connected.

Constraints:

- $n == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq n \leq 50$
- $0 \leq \text{grid}[i][j] < n^2$

- Each value `grid[i][j]` is unique.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def swimInWater(self, grid: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(grid)
        m = n * n
        p = list(range(m))

        hi = [0] * m
        for i, row in enumerate(grid):
            for j, h in enumerate(row):
                hi[h] = i * n + j
        dirs = (-1, 0, 1, 0, -1)
        for t in range(m):
            x, y = divmod(hi[t], n)
            for dx, dy in pairwise(dirs):
                nx, ny = x + dx, y + dy
                if 0 <= nx < n and 0 <= ny < n and grid[nx][ny] <= t:
                    p[find(x * n + y)] = find(nx * n + ny)
            if find(0) == find(m - 1):
                return t
        return 0
```

• SimplyLeet

```

import heapq

def swimInWater(grid):
    n = len(grid)
    # Initialize a visited set to track cells we have processed.
    visited = [[False] * n for _ in range(n)]
    # Min-heap: (current max elevation along the path, row, col)
    min_heap = [(grid[0][0], 0, 0)]
    visited[0][0] = True
    # Directions for moving up, down, left, and right.

    directions = [(1,0), (-1,0), (0,1), (0,-1)]

    while min_heap:
        time, r, c = heapq.heappop(min_heap)
        # If we reach the bottom-right, return the maximum elevation
        encountered along the path.
        if r == n - 1 and c == n - 1:
            return time
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            # Check boundaries and if the cell is not already visited.

            if 0 <= nr < n and 0 <= nc < n and not visited[nr][nc]:
                visited[nr][nc] = True
                # The required time is the maximum of the current time and the
                elevation at the new cell.
                new_time = max(time, grid[nr][nc])
                heapq.heappush(min_heap, (new_time, nr, nc))

    return -1 # Should not reach here if input is as per constraints.

# Example Usage:
grid = [[0,2],[1,3]]

print(swimInWater(grid)) # Output: 3

```

◆ Quick Review

Key Insights

- The problem can be viewed as finding a path from the top-left to bottom-right where the cost is the maximum elevation encountered on that path.
- A greedy approach using Dijkstra's algorithm (or a min-heap based BFS) ensures that we always consider the next move with the smallest current elevation requirement.
- Once the destination is reached, the maximum elevation along that path is the minimum time required.
- Alternative approaches include binary search on time combined with a DFS/BFS check to see if a valid path exists.

Space & Time

Time Complexity: $O(n^2 \log(n^2))$ which simplifies to $O(n^2 \log n)$ since each cell is processed once and we use a priority queue.

Space Complexity: $O(n^2)$ due to the visited matrix and the priority queue.

Solution

We can solve the problem using a min-heap based BFS (Dijkstra's algorithm). Start at cell (0, 0) with its elevation as the initial cost, and

use a min-heap to always expand the cell with the lowest required water level. For each popped cell, update the maximum elevation encountered on the current path. When reaching the target cell ($n - 1, n - 1$), this value represents the least time needed. We maintain a visited matrix to avoid reprocessing cells.

Round 4 · Mid · Easy

Round 4

Mid Priority · Easy

6 questions · 3 patterns

- ☐ ● Linked List #21 #141 #206 ↗
- ☐ ● Stack #20 ↗
- ☐ ● Heap / Priority Queue #703 #1046 ↗

Round 3 · High · Hard

p.521

Linked List

Round 4 · Mid · Easy · 3 questions

Linked List

□ #21 Merge Two Sorted Lists

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Recursion

Lists: Grind 169

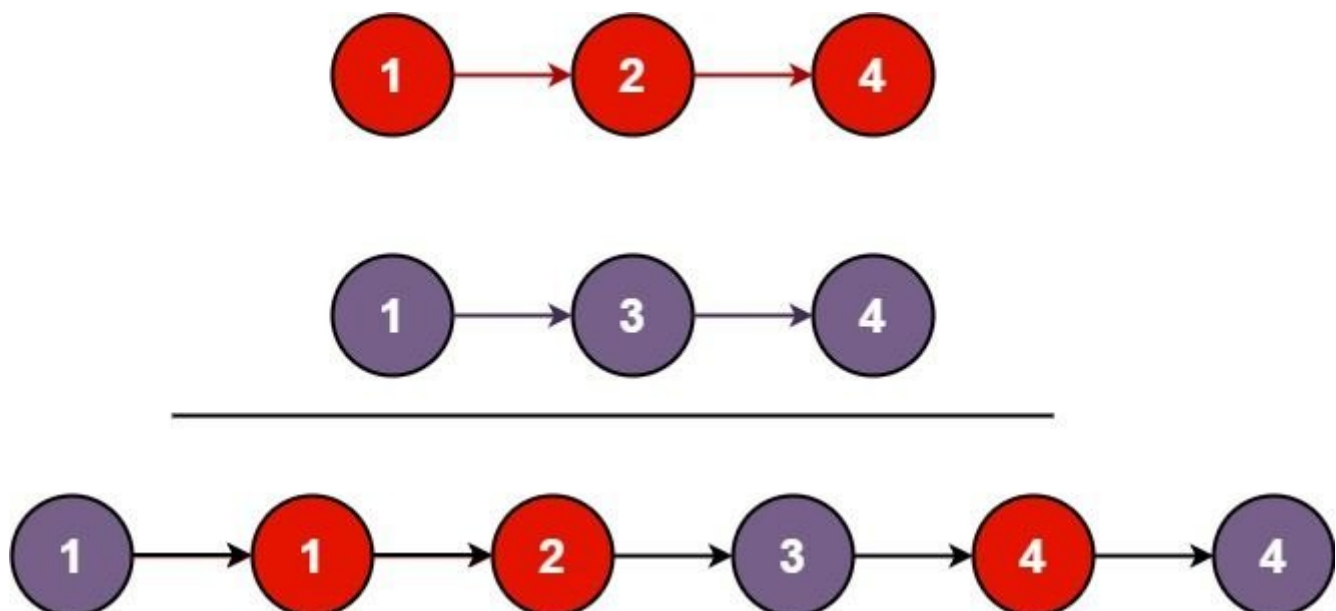
Problem

You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list.

Example 1:



Input: list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]

Example 2:

Input: list1 = [], list2 = []
Output: []

Example 3:

Input: list1 = [], list2 = [0]
Output: [0]

Constraints:

- The number of nodes in both lists is in the range [0, 50].
- $-100 \leq \text{Node.val} \leq 100$
- Both list1 and list2 are sorted in non-decreasing order.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mergeTwoLists(
        self,
        list1: ListNode | None,
        list2: ListNode | None,
    ) -> ListNode | None:
        if not list1 or not list2:
            return list1 if list1 else list2
        if list1.val > list2.val:
            list1, list2 = list2, list1

        list1.next = self.mergeTwoLists(list1.next, list2)
        return list1
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeTwoLists(
        self, list1: Optional[ListNode], list2: Optional[ListNode]
    ) -> Optional[ListNode]:
        if list1 is None or list2 is None:

            return list1 or list2
        if list1.val <= list2.val:
            list1.next = self.mergeTwoLists(list1.next, list2)
            return list1
        else:
            list2.next = self.mergeTwoLists(list1, list2.next)
            return list2
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def mergeTwoLists(list1: ListNode, list2: ListNode) -> ListNode:
    # Create a dummy node to serve as the start of the merged list
    dummy = ListNode(-1)
    current = dummy
```

```

# Iterate while both lists have nodes
while list1 and list2:
    # Compare the values of the current nodes of both lists
    if list1.val <= list2.val:
        current.next = list1 # Link the smaller node to the merged list
        list1 = list1.next   # Move the pointer in list1
    else:
        current.next = list2
        list2 = list2.next

    current = current.next # Move the pointer in the merged list

# Once one list is exhausted, attach the remaining nodes from the other
list
if list1:
    current.next = list1
else:
    current.next = list2

# The merged list head is at dummy.next
return dummy.next

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
# without ops after while loop
class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode()
        current = dummy

```

```

while list1 or list2:
    v1 = list1.val if list1 else float('inf')
    v2 = list2.val if list2 else float('inf')

    if v1 < v2:
        current.next = list1
        list1 = list1.next
    else:
        current.next = list2
        list2 = list2.next

```

```

current = current.next

```

```

return dummy.next

```

```

#####

```

```

# Definition for singly-linked list.

```

```

# class ListNode:

```

```

# def __init__(self, val=0, next=None):

```

```

# self.val = val

```

```

# self.next = next

```

```

class Solution:

```

```

    def mergeTwoLists(

```

```

        self, list1: Optional[ListNode], list2: Optional[ListNode]

```

```

    ) -> Optional[ListNode]:

```

```

        dummy = ListNode()

```

```

        curr = dummy

```

```

        while list1 and list2:

```

```

            if list1.val <= list2.val:

```

```

                curr.next = list1

```

```

                list1 = list1.next

```

```

            else:

```

```

                curr.next = list2

```

```

                list2 = list2.next

```

```

            curr = curr.next

```

```

        curr.next = list1 or list2

```

```

        return dummy.next

```

```

.....

```

```

curr.next = list1 or list2

```

better than:

```

    if list1:
        current.next = list1
    if list2:
        current.next = list2

```

```

    ....

```

```

#####

```

```

# recursion

```

```

class ListNode:

```

```

    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

```

```

class Solution:

```

```

    def mergeTwoLists(self, l1: ListNode, l2: ListNode) -> ListNode:
        if not l1: # If l1 is empty, return l2
            return l2
        if not l2: # If l2 is empty, return l1
            return l1

```

```

        if l1.val < l2.val:
            l1.next = self.mergeTwoLists(l1.next, l2)
            return l1
        else:

```

```

            l2.next = self.mergeTwoLists(l1, l2.next)
            return l2

```

◆ Quick Review

Key Insights

- Leverage the fact that both lists are sorted.

- Use two pointers (one for each list) to compare nodes sequentially.
- Create a dummy node to simplify edge cases and build the resulting list.
- Iterate until one list is exhausted, then append the remaining nodes from the other list.
- There is also a recursive approach that can simplify the logic but may use more stack space.

Space & Time

Time Complexity: $O(n + m)$, where n and m are the lengths of the two lists.

Space Complexity: $O(1)$ for the iterative solution ($O(n + m)$ stack space for the recursive solution).

Solution

We use an iterative approach with a dummy node to merge two sorted linked lists. Two pointers traverse list1 and list2; at each step, the node with the smaller value is appended to our merged list. This process repeats until one of the lists is exhausted, after which we append the remaining nodes from the other

list. This approach works in one pass over both lists, ensuring linear time complexity and constant extra space usage.

Linked List

□ #141 Linked List Cycle

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Two Pointers
Lists: Grind 169**

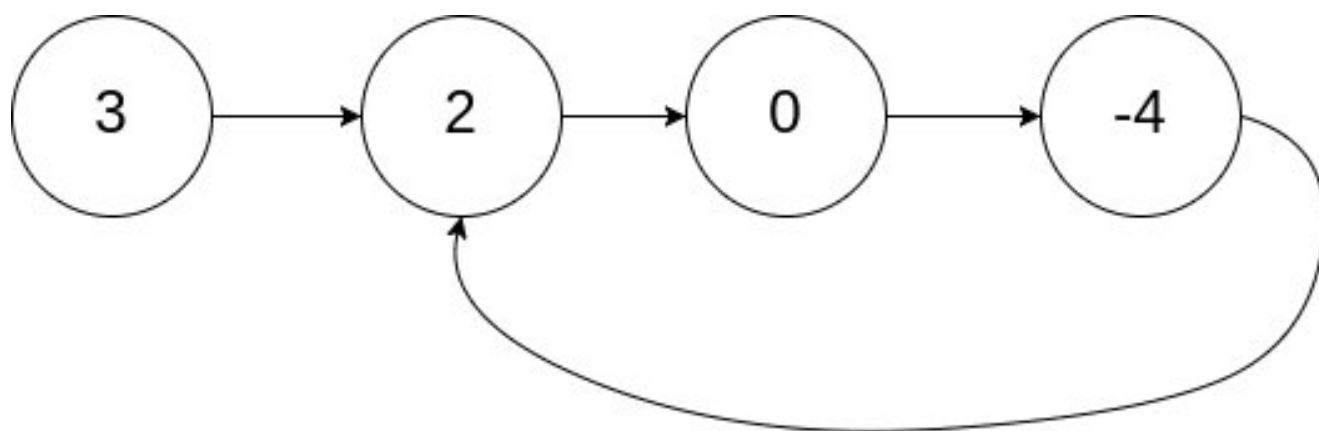
Problem

Given head, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer.

Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter. Return true if there is a cycle in the linked list. Otherwise, return false.

Example 1:

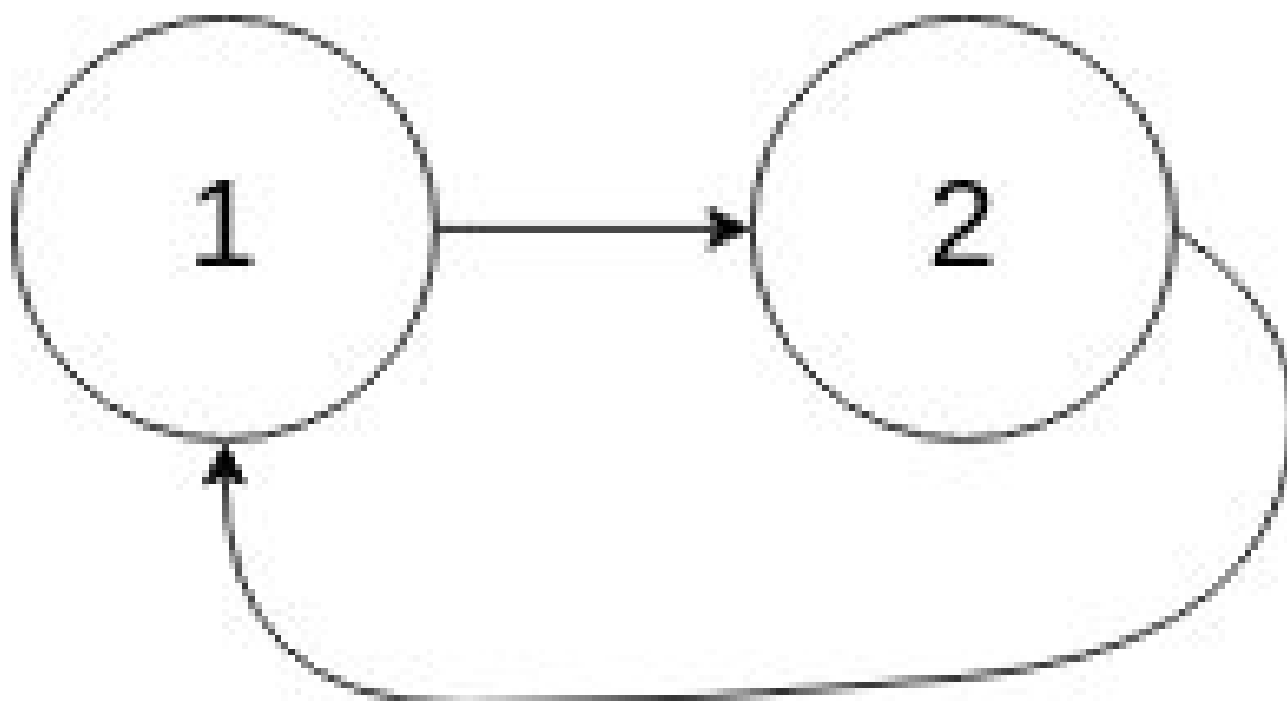


Input: head = [3,2,0,-4], pos = 1

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

Example 2:

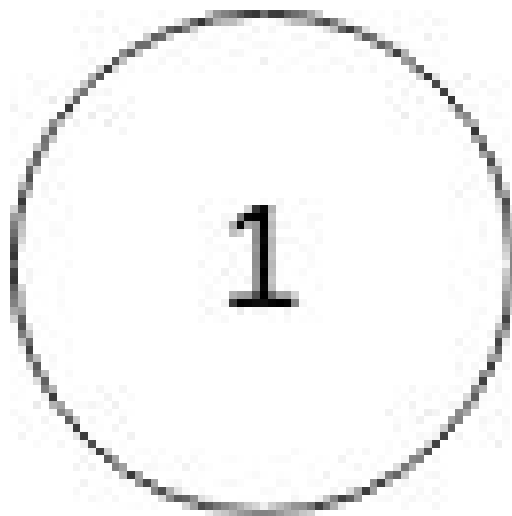


Input: head = [1,2], pos = 0

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.

Example 3:



Input: head = [1], pos = -1

Output: false

Explanation: There is no cycle in the linked list.

Constraints:

- The number of the nodes in the list is in the range $[0, 10^4]$.
- $-105 \leq \text{Node.val} \leq 105$
- pos is -1 or a valid index in the linked-list.

Follow up: Can you solve it using $O(1)$ (i.e. constant) memory?

Community Solutions (Python)

- WalkCC

```
class Solution:
    def hasCycle(self, head: ListNode) -> bool:
        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                return True

        return False
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        s = set()

        while head:
            if head in s:
                return True
            s.add(head)
            head = head.next
        return False
```

• SimplyLeet

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        # Initialize two pointers, slow and fast, starting from the head.
        slow = head

        fast = head

        # Traverse the list until fast pointer reaches the end.
        while fast and fast.next:
            slow = slow.next           # Move slow pointer one step.
            fast = fast.next.next      # Move fast pointer two steps.

            # If both pointers meet, there is a cycle.
            if slow == fast:
                return True

        # If loop terminates, there is no cycle.
        return False

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def hasCycle(self, head: ListNode) -> bool:
        slow = fast = head
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next
            if slow == fast:
                return True
        return False

```

◆ Quick Review

Key Insights

- Use two pointers (slow and fast) to traverse the list.
- The fast pointer moves two steps at a time and the slow pointer moves one step.
- If the fast pointer ever meets the slow pointer, a cycle exists.
- If the fast pointer reaches the end of the list (null), the list has no cycle.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes in the linked list.

Space Complexity: $O(1)$ as only two pointers are used.

Solution

We use the Floyd's Cycle Detection algorithm (Tortoise and Hare approach). Two pointers, slow and fast, are initialized at the head of the list. The slow pointer moves one step at a time while the fast pointer moves two steps. If the linked list has a cycle, the fast pointer will

eventually meet the slow pointer. If the fast pointer reaches the end (i.e., a null pointer), the linked list does not have a cycle.

Linked List

□ #206 Reverse Linked List

EAS

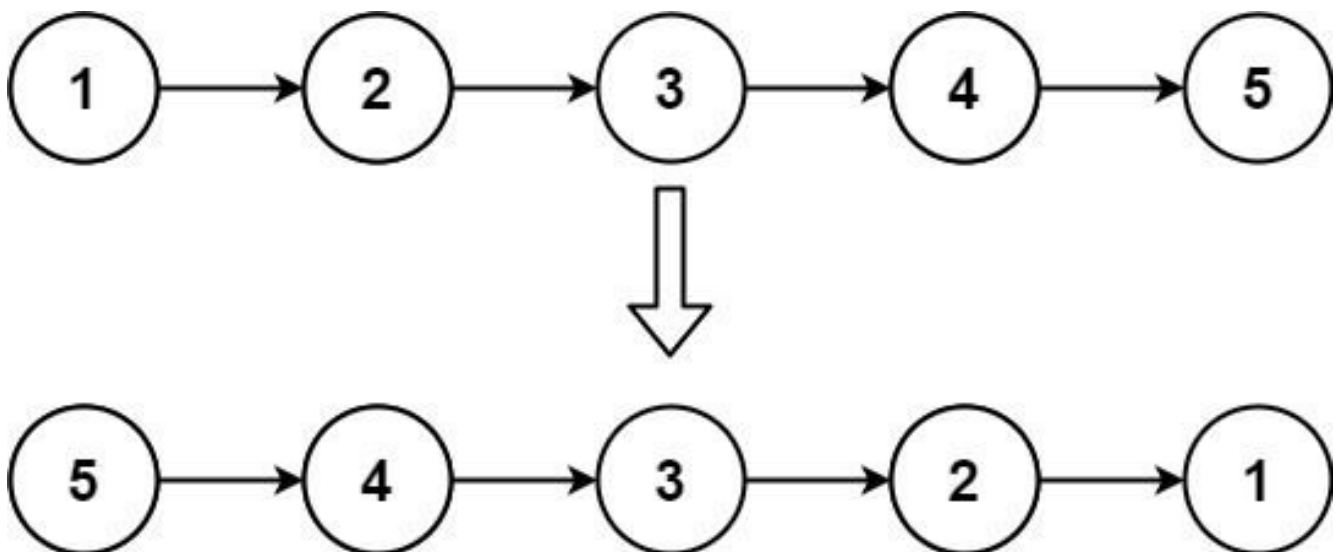
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Recursion

Lists: Grind 169

Problem

Given the head of a singly linked list, reverse the list, and return the reversed list.

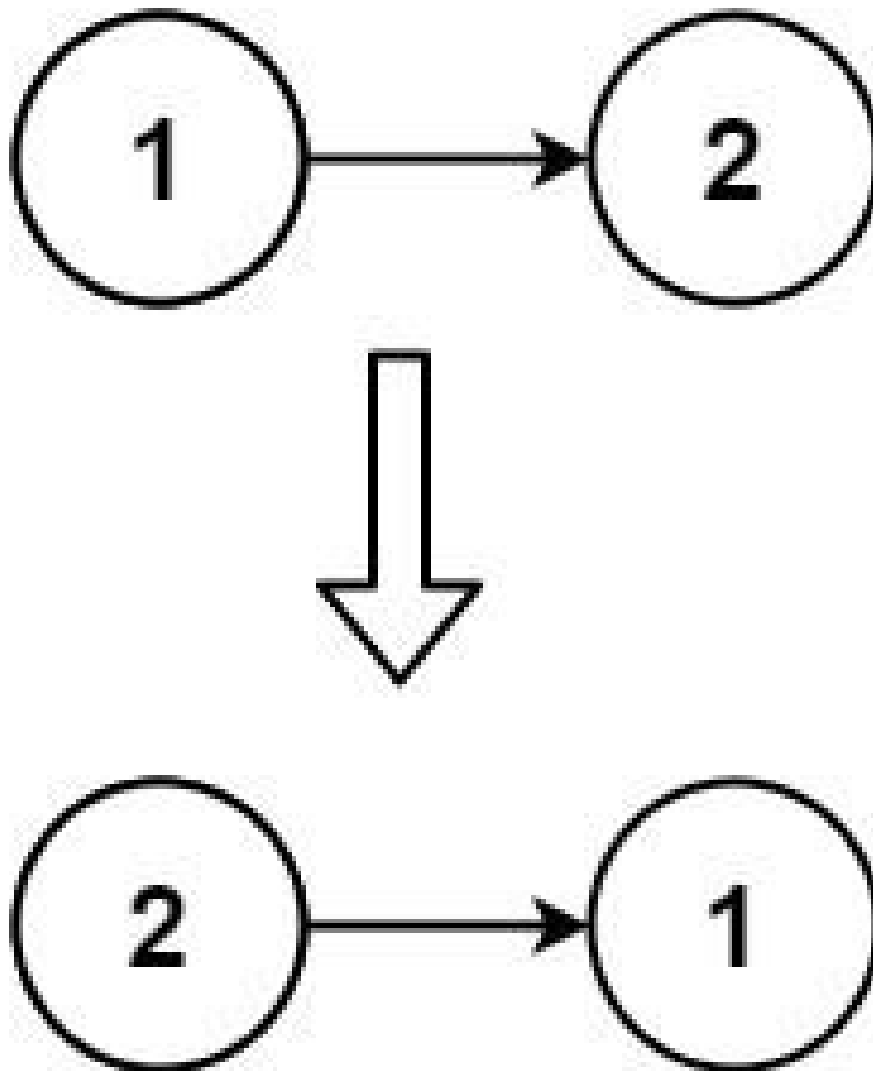
Example 1:



Input: head = [1,2,3,4,5]

Output: [5,4,3,2,1]

Example 2:



Input: head = [1,2]
Output: [2,1]

Example 3:

Input: head = []
Output: []

Constraints:

- The number of nodes in the list is the range [0, 5000].
- $-5000 \leq \text{Node.val} \leq 5000$

Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverseList(self, head: ListNode | None) -> ListNode | None:
        if not head or not head.next:
            return head

        newHead = self.reverseList(head.next)
        head.next.next = head
        head.next = None
        return newHead
```

```
class Solution:
    def reverseList(self, head: ListNode | None) -> ListNode | None:
        prev = None
        curr = head

        while curr:
            next = curr.next
            curr.next = prev
            prev = curr
            curr = next

        return prev
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: ListNode) -> ListNode:
        dummy = ListNode()
        curr = head
        while curr:

            next = curr.next
            curr.next = dummy.next
            dummy.next = curr
            curr = next
        return dummy.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val # Store the node value
        self.next = next # Pointer to the next node

# Iterative approach to reverse the linked list.
def reverseList(head):
    prev = None # Initially, there is no previous node
    current = head # Start with the head of the list

    while current:
        next_node = current.next # Temporarily store the next node
        current.next = prev # Reverse the current node's pointer
        prev = current # Move previous pointer one step forward
        current = next_node # Move to the next node in the list
    return prev # The new head of the reversed list

# Recursive approach to reverse the linked list.
def reverseListRecursive(head):
    # Base case: empty list or single node
```

```

if not head or not head.next:
    return head
new_head = reverseListRecursive(head.next) # Reverse the rest of the list
head.next.next = head # Reverse the pointer for the current node
head.next = None # Set the next pointer of the current node to None
return new_head # Return the new head of the reversed list

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        pre, p = None, head
        while p:
            q = p.next

```

```

            p.next = pre
            pre = p
            p = q
        return pre

```

```

class Solution(object): # recursive
    def reverseList(self, root):
        if not root or not root.next:
            return root

```

```

        ret = self.reverseList(root.next)
        root.next.next = root # root.next is end of the newly reversed list ret
        root.next = None
        return ret

```

```

class Solution:
    def reverseList(self, head: ListNode) -> ListNode:
        dummy = ListNode()

```

```
curr = head
while curr:
    next = curr.next
    curr.next = dummy.next
    dummy.next = curr
    curr = next
return dummy.next
```

◆ Quick Review

Key Insights

- The problem can be solved using either an iterative or recursive approach.
- Iterative method uses constant space by updating pointers in-place.
- Recursive method leverages the call stack to reverse the list, which could use $O(n)$ space in the worst case.
- Both approaches traverse each node exactly once, yielding linear time complexity.

Space & Time

Time Complexity: $O(n)$ for both iterative and recursive approaches, where n is the number of nodes.

Space Complexity: $O(1)$ for the iterative approach and $O(n)$ for the recursive approach because of the recursion call stack.

Solution

We can solve the problem by iterating through the list while reversing the next pointers of each node. We maintain two pointers (previous and current) and update them as we traverse the list, so that by the end, the previous pointer is at the new head of the reversed list. Alternatively, a recursive approach reverses the rest of the list first and then adjusts the pointers so that the original head becomes the tail.

Stack

Round 4 · Mid · Easy · 1 question

Stack

□ #20 Valid Parentheses

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack

Lists: Grind 169

Problem

Given a string *s* containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: *s* = "()"

Output: true

Example 2:

Input: *s* = "()[]{}"

Output: true

Example 3:

Input: s = "())"

Output: false

Example 4:

Input: s = "([])"

Output: true

Example 5:

Input: s = "([)]"

Output: false

Constraints:

- $1 \leq s.length \leq 104$
- s consists of parentheses only '()[]{}'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isValid(self, s: str) -> bool:
        stack = []

        for c in s:
            if c == '(':
                stack.append(')')
            elif c == '{':
                stack.append('}')
            elif c == '[':
```

```

        stack.append(']')
    elif not stack or stack.pop() != c:
        return False

    return not stack

```

• LeetCode

```

class Solution:
    def isValid(self, s: str) -> bool:
        stk = []
        d = {'(': ')', '[': ']', '{': '}' }
        for c in s:
            if c in '({[':
                stk.append(c)
            elif not stk or stk.pop() + c not in d:
                return False
        return not stk

```

• SimplyLeet

```

def isValid(s: str) -> bool:
    # Mapping from closing brackets to their corresponding opening brackets.
    paren_map = {')': '(', '}': '{', ']': '['}
    # Initialize an empty list to use as a stack.
    stack = []
    # Process each character in the string.
    for char in s:
        # If the character is a closing bracket.
        if char in paren_map:
            # Pop the top element from the stack if available, else use a
            placeholder.

```

```

        top_element = stack.pop() if stack else '#'
        # Check if the mapping does not match the top element.
        if paren_map[char] != top_element:
            return False
    else:
        # For opening brackets, push onto the stack.
        stack.append(char)
    # Return True if all brackets are matched (i.e., the stack is empty).
    return not stack

```

```

# Example usage
print(isValid("()[]{}")) # Expected output: True

```

• LC.ca

```

class Solution:
    def isValid(self, s: str) -> bool:
        stk = []
        d = {'()', '[]', '{}'}
        for c in s:
            if c in '([{':
                stk.append(c)
            elif not stk or stk.pop() + c not in d:
                return False
        return not stk

```

◆ Quick Review

Key Insights

- The problem can be solved using a stack data structure.
- When encountering an opening bracket, push it onto the stack.
- When encountering a closing bracket, check if the top of the stack is its matching opening

bracket.

- If the stack is empty before finding a match or if there is a mismatch, the string is invalid.
- At the end, the string is valid only if the stack is empty, meaning all brackets were properly matched.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, since we process each character once.

Space Complexity: $O(n)$ in the worst case for the stack when all characters are opening brackets.

Solution

We use a stack to check for valid parentheses. The algorithm iterates through the string, pushing opening brackets onto the stack and popping them when a corresponding closing bracket is encountered. A mapping of closing to opening brackets helps in validating the matching pairs. By ensuring that the stack is empty at the end, we confirm that every opening bracket was properly closed.

Heap / Priority Queue

Round 4 · Mid · Easy · 2 questions

Heap / Priority Queue

□ #703 Kth Largest Element in a Stream

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Design · Binary Search Tree · Heap
(Priority Queue) · Binary Tree · Data Stream
Lists: NeetCode 150

Problem

You are part of a university admissions office and need to keep track of the k th highest test score from applicants in real-time. This helps to determine cut-off marks for interviews and admissions dynamically as new applicants submit their scores.

You are tasked to implement a class which, for a given integer k , maintains a stream of test scores and continuously returns the k th highest test score after a new score has been submitted. More specifically, we are looking for the k th highest score in the sorted list of all scores.

Implement the `KthLargest` class:

- `KthLargest(int k, int[] nums)` Initializes the object with the integer k and the stream of

test scores nums.

- **int add(int val)** Adds a new test score val to the stream and returns the element representing the kth largest element in the pool of test scores so far.

Example 1:

Input: ["KthLargest", "add", "add", "add", "add", "add"] [[3, [4, 5, 8, 2]], [3], [5], [10], [9], [4]]

Output: [null, 4, 5, 5, 8, 8]

Explanation:

```
KthLargest kthLargest = new KthLargest(3, [4, 5, 8, 2]);  
kthLargest.add(3); // return 4  
kthLargest.add(5); // return 5  
kthLargest.add(10); // return 5  
kthLargest.add(9); // return 8  
kthLargest.add(4); // return 8
```

Example 2:

Input: ["KthLargest", "add", "add", "add", "add"] [[4, [7, 7, 7, 7, 8, 3]], [2], [10], [9], [9]]

Output: [null, 7, 7, 7, 8]

Explanation:

Constraints:

- $0 \leq \text{nums.length} \leq 104$
- $1 \leq k \leq \text{nums.length} + 1$

- $-104 \leq \text{nums}[i] \leq 104$
- $-104 \leq \text{val} \leq 104$
- At most 104 calls will be made to add.

Community Solutions (Python)

• LeetCode

```
class KthLargest:

    def __init__(self, k: int, nums: List[int]):
        self.k = k
        self.min_q = []
        for x in nums:
            self.add(x)

    def add(self, val: int) -> int:
        heappush(self.min_q, val)

        if len(self.min_q) > self.k:
            heappop(self.min_q)
        return self.min_q[0]

# Your KthLargest object will be instantiated and called as such:
# obj = KthLargest(k, nums)
# param_1 = obj.add(val)
```

• SimplyLeet

```
import heapq

class KthLargest:
    def __init__(self, k, nums):
        # Save kth value and initialize the min-heap.
        self.k = k
        self.min_heap = nums
        heapq.heapify(self.min_heap)
        # Reduce the heap size to exactly k elements.
        while len(self.min_heap) > k:
```

```
        heapq.heappop(self.min_heap)

    def add(self, val):
        if len(self.min_heap) < self.k:
            heapq.heappush(self.min_heap, val)
        elif val > self.min_heap[0]:
            heapq.heapreplace(self.min_heap, val)
        # The kth largest element is at the heap root.
        return self.min_heap[0]
```

```
# Example usage:
# kthLargest = KthLargest(3, [4, 5, 8, 2])
# print(kthLargest.add(3)) # returns 4
# print(kthLargest.add(5)) # returns 5
# print(kthLargest.add(10)) # returns 5
# print(kthLargest.add(9)) # returns 8
# print(kthLargest.add(4)) # returns 8
```

◆ Quick Review

Key Insights

- Use a min-heap (priority queue) of fixed size k .
- The heap always contains the k largest elements; the smallest element in the heap is the k th largest overall.
- When a new test score is added:
- If the heap has less than k elements, simply add the score.
- If the heap is full and the new score is greater than the smallest (heap root), replace

the root.

- Each add operation runs in $O(\log k)$ time.

Space & Time

Time Complexity: $O(\log k)$ per add operation, $O(n \log k)$ to build the initial heap.

Space Complexity: $O(k)$ for maintaining the heap.

Solution

We use a min-heap to track the k largest numbers. When initializing, we add numbers from the given list to the heap, ensuring not to exceed a size of k . For each add operation, if the new number is greater than the heap minimum, we replace the minimum. The k th largest number is always accessible as the minimum element in the heap.

Heap / Priority Queue

□ #1046 Last Stone Weight

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Heap (Priority Queue)

Lists: NeetCode 150

Problem

You are given an array of integers `stones` where `stones[i]` is the weight of the *i*th stone.

We are playing a game with the stones. On each turn, we choose the heaviest two stones and smash them together. Suppose the heaviest two stones have weights x and y with $x \leq y$. The result of this smash is:

- If $x == y$, both stones are destroyed, and
- If $x \neq y$, the stone of weight x is destroyed, and the stone of weight y has new weight $y - x$.

At the end of the game, there is at most one stone left.

Return the weight of the last remaining stone. If there are no stones left, return 0.

Example 1:

Input: stones = [2,7,4,1,8,1]

Output: 1

Explanation:

We combine 7 and 8 to get 1 so the array converts to [2,4,1,1,1] then, we combine 2 and 4 to get 2 so the array converts to [2,1,1,1] then, we combine 2 and 1 to get 1 so the array converts to [1,1,1] then, we combine 1 and 1 to get 0 so the array converts to [1] then that's the value of the last stone.

Example 2:

Input: stones = [1]

Output: 1

Constraints:

- $1 \leq \text{stones.length} \leq 30$
- $1 \leq \text{stones}[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lastStoneWeight(self, stones: list[int]) -> int:
        pq = [-stone for stone in stones]
        heapq.heapify(pq)

        while len(pq) >= 2:
            n1 = -heapq.heappop(pq)
            n2 = -heapq.heappop(pq)
            if n1 != n2:
                heapq.heappush(pq, -(n1 - n2))

        return 0 if not pq else -pq[0]
```

• LeetDoocs

```
class Solution:
    def lastStoneWeight(self, stones: List[int]) -> int:
        h = [-x for x in stones]
        heapify(h)
        while len(h) > 1:
            y, x = -heappop(h), -heappop(h)
            if x != y:
                heappush(h, x - y)
        return 0 if not h else -h[0]
```

• SimplyLeet

```
import heapq

def lastStoneWeight(stones):
    # Create a max heap by inserting negative values of stones
    max_heap = [-stone for stone in stones]
    heapq.heapify(max_heap)

    # While more than one stone remains
    while len(max_heap) > 1:
        # Get the heaviest stone (largest value, but stored as negative)

        heaviest = -heapq.heappop(max_heap)
        # Get the next heaviest stone
        next_heaviest = -heapq.heappop(max_heap)

        # If the stones are not equal, push the difference back into the heap
        if heaviest != next_heaviest:
            diff = heaviest - next_heaviest
            heapq.heappush(max_heap, -diff)

        # If there is a stone left, return its weight. Otherwise, return 0.

    return -max_heap[0] if max_heap else 0

# Example usage:
# print(lastStoneWeight([2,7,4,1,8,1]))
```

◆ Quick Review

Key Insights

- Use a max-heap (or simulate one) to always access the two largest stones quickly.
- If the two stones are equal, both are removed; otherwise, the difference is pushed back into the heap.
- Continue the process until the heap has one or no stone left.
- For languages without a max-heap in their standard library (e.g., Python), simulate one by pushing negative values.

Space & Time

Time Complexity: $O(n \log n)$ – Each stone removal and insertion takes $O(\log n)$ and in the worst-case scenario, we perform $O(n)$ operations.

Space Complexity: $O(n)$ – The heap stores all the stones initially.

Solution

We solve the problem by using a priority queue (max-heap). In Python, we simulate a max-heap by inserting the negative of each stone's weight into a min-heap. In JavaScript, due to the lack of a built-in heap structure, we

sort the array in descending order at each iteration (acceptable given the small constraints). In C++ and Java, we use the standard `priority_queue`. The algorithm repeatedly extracts the two largest stones, computes their difference, and if the difference is non-zero, it is inserted back into the heap. This process repeats until there is at most one stone left.

Round 5 · Mid · Medium

Round 5

Mid Priority · Medium

33 questions · 6 patterns

Linked List #2 #19 #138 #143 #146 #287

Stack #150 #155 #739 #853

Heap / Priority Queue #215 #355 #621 #973

Tries #208 #211

**Backtracking #17 #22 #39 #40 #46 #78 #79 #90
#131**

**Greedy #45 #53 #55 #134 #678 #763 #846
#1899**

Linked List

Round 5 · Mid · Medium · 6 questions

Linked List

□ #2 Add Two Numbers

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Linked List · Math · Recursion

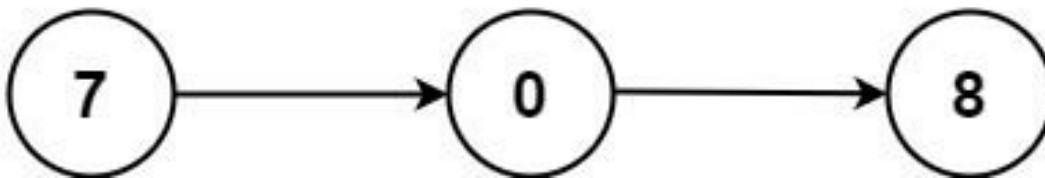
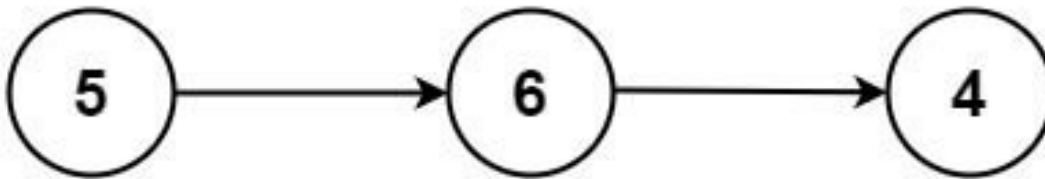
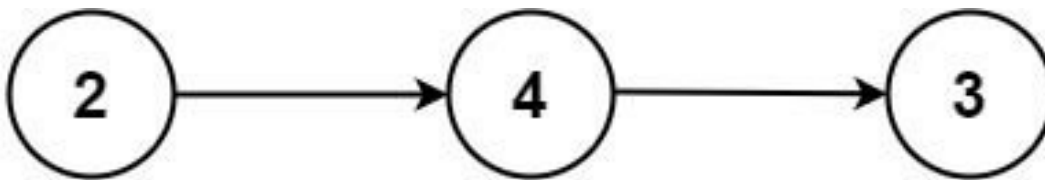
Lists: Grind 169

Problem

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:



Input: l1 = [2,4,3], l2 = [5,6,4]

Output: [7,0,8]

Explanation: 342 + 465 = 807.

Example 2:

Input: l1 = [0], l2 = [0]

Output: [0]

Example 3:

Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]

Output: [8,9,9,9,0,0,0,1]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].

- $0 \leq \text{Node.val} \leq 9$
- It is guaranteed that the list represents a number that does not have leading zeros.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def addTwoNumbers(self, l1: ListNode, l2: ListNode) -> ListNode:
        dummy = ListNode(0)
        curr = dummy
        carry = 0

        while carry or l1 or l2:
            if l1:
                carry += l1.val
                l1 = l1.next

            if l2:
                carry += l2.val
                l2 = l2.next
            curr.next = ListNode(carry % 10)
            carry //= 10
            curr = curr.next

        return dummy.next
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def addTwoNumbers(
        self, l1: Optional[ListNode], l2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode()
```

```

carry, curr = 0, dummy
while l1 or l2 or carry:
    s = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry
    carry, val = divmod(s, 10)
    curr.next = ListNode(val)
    curr = curr.next
    l1 = l1.next if l1 else None
    l2 = l2.next if l2 else None
return dummy.next

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def addTwoNumbers(l1, l2):
    # Initialize a dummy head node to simplify the result list construction.
    dummy = ListNode()
    current = dummy

    carry = 0

    # Traverse through both linked lists until all nodes and any carry are
    # processed.
    while l1 or l2 or carry:
        # Get the current values; if a node is missing, use 0.
        val1 = l1.val if l1 else 0
        val2 = l2.val if l2 else 0

        # Calculate the sum of current digits and carry.
        total = val1 + val2 + carry

```

```

        carry = total // 10    # Update carry for next iteration.
        digit = total % 10     # Compute current digit.

        # Append the digit as a new node in the result list.
        current.next = ListNode(digit)
        current = current.next

        # Move to the next nodes if available.
        if l1:
            l1 = l1.next

        if l2:
            l2 = l2.next

    # Return the head of the newly formed linked list.
    return dummy.next

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def addTwoNumbers(
        self, l1: Optional[ListNode], l2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode()

        carry, curr = 0, dummy
        while l1 or l2 or carry:
            s = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry
            carry, val = divmod(s, 10)
            curr.next = ListNode(val)
            curr = curr.next
            l1 = l1.next if l1 else None
            l2 = l2.next if l2 else None
        return dummy.next

```

◆ Quick Review

Key Insights

- The digits are stored in reverse order, meaning the first node is the least significant digit.
- A carry may need to propagate through subsequent nodes.
- The two linked lists might be of different lengths.
- Continue processing until both lists are fully traversed and any remaining carry is handled.

Space & Time

Time Complexity: $O(\max(n, m))$, where n and m are the lengths of the two linked lists.

Space Complexity: $O(\max(n, m) + 1)$ for the result list in the worst case.

Solution

The solution iterates through both linked lists node by node. For each pair of corresponding nodes, their values and any carry from the previous operation are summed. The result is decomposed into a current digit (modulo 10) and an updated carry (total divided by 10). A new node is created for the current digit and appended to the resulting linked list. This

process continues until all nodes are processed and any leftover carry is added as a final node.

Linked List

□ #19 Remove Nth Node From End of List

MED

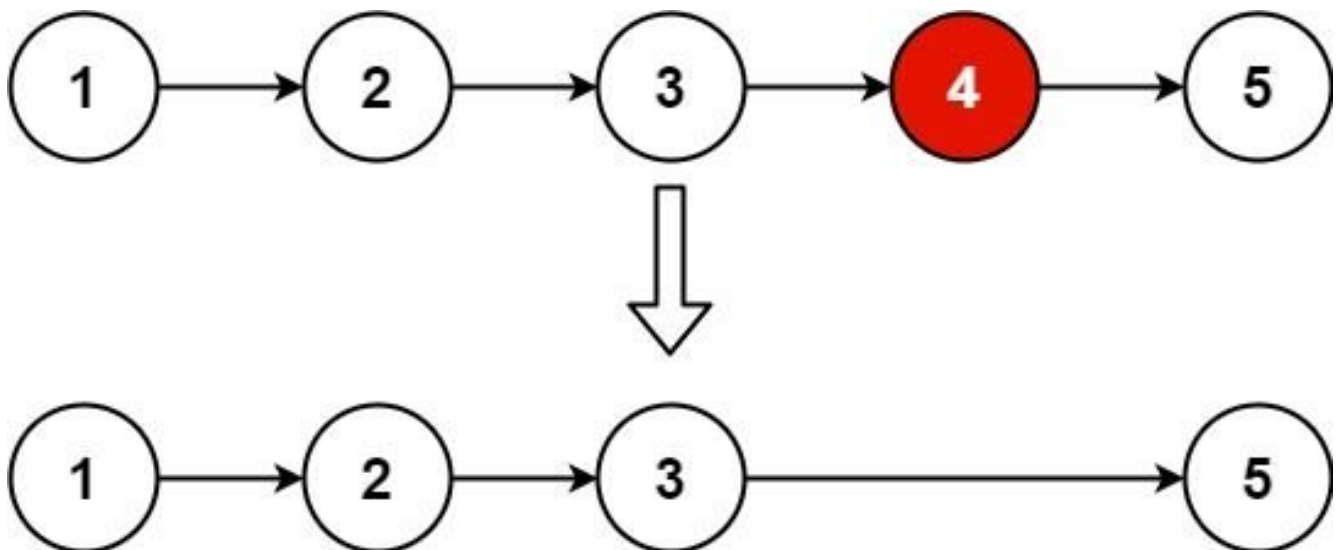
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Two Pointers

Lists: Grind 169

Problem

Given the head of a linked list, remove the n th node from the end of the list and return its head.

Example 1:



Input: head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1
Output: []

Example 3:

Input: head = [1,2], n = 1
Output: [1]

Constraints:

- The number of nodes in the list is sz.
- $1 \leq sz \leq 30$
- $0 \leq \text{Node.val} \leq 100$
- $1 \leq n \leq sz$

Follow up: Could you do this in one pass?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) -> ListNode:
        slow = head
        fast = head

        for _ in range(n):
            fast = fast.next
        if not fast:
            return head.next

        while fast.next:
            slow = slow.next
            fast = fast.next
        slow.next = slow.next.next

        return head
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        fast = slow = dummy
        for _ in range(n):
            fast = fast.next
        while fast.next:
            slow, fast = slow.next, fast.next
        slow.next = slow.next.next
        return dummy.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val      # Holds the value of the node
        self.next = next    # Points to the next node in the list

class Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) -> ListNode:
        # Create a dummy node to simplify edge cases (like removing the head)
        dummy = ListNode(0)

        dummy.next = head

        # Initialize two pointers starting at the dummy node
        first = dummy
        second = dummy

        # Advance first pointer n+1 steps ahead to maintain a gap of n nodes
        for _ in range(n + 1):
            first = first.next
```

```

# Move both pointers until first reaches the end of the list
while first:
    first = first.next
    second = second.next

# Skip the node to remove it from the list
second.next = second.next.next

# Return the adjusted list starting from dummy.next
return dummy.next

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        fast = slow = dummy
        for _ in range(n):
            fast = fast.next
        while fast.next:
            slow, fast = slow.next, fast.next
        slow.next = slow.next.next
        return dummy.next

```

◆ Quick Review

Key Insights

- Use a dummy node at the start to simplify edge cases, especially when the head is removed.

- Utilize two pointers (fast and slow) and maintain a gap of $n+1$ nodes between them.
- By moving both pointers simultaneously, the slow pointer will point to the node immediately before the target node when the fast pointer reaches the end.

Space & Time

Time Complexity: $O(L)$, where L is the length of the list

Space Complexity: $O(1)$

Solution

We use a two–pointer approach to achieve a one–pass solution. A dummy node is created and linked to the head to handle edge cases gracefully (such as needing to remove the head). The fast pointer is advanced $n+1$ steps ahead of the slow pointer to maintain the required gap. Then, both pointers are moved simultaneously until the fast pointer reaches the end of the list. At that point, the slow pointer is immediately before the node to be deleted. We then adjust the pointers to remove the target node from the list.

Linked List

□ #138 Copy List with Random Pointer

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List
Lists: NeetCode 150

Problem

A linked list of length n is given such that each node contains an additional random pointer, which could point to any node in the list, or null. Construct a deep copy of the list. The deep copy should consist of exactly n brand new nodes, where each new node has its value set to the value of its corresponding original node. Both the next and random pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. None of the pointers in the new list should point to nodes in the original list.

For example, if there are two nodes X and Y in the original list, where $X.random \rightarrow Y$, then for the corresponding two nodes x and y in the

copied list, $x.random \rightarrow y$.

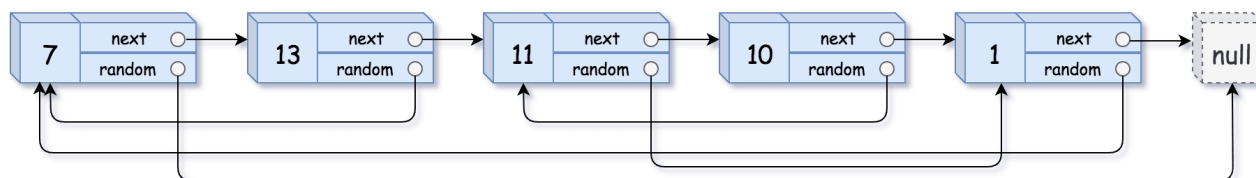
Return the head of the copied linked list.

The linked list is represented in the input/output as a list of n nodes. Each node is represented as a pair of $[val, random_index]$ where:

- **val**: an integer representing `Node.val`
- **random_index**: the index of the node (range from 0 to $n-1$) that the random pointer points to, or null if it does not point to any node.

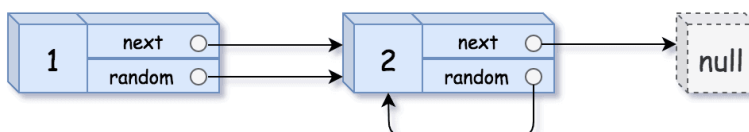
Your code will only be given the head of the original linked list.

Example 1:



Input: head = `[[7,null],[13,0],[11,4],[10,2],[1,0]]`
 Output: `[[7,null],[13,0],[11,4],[10,2],[1,0]]`

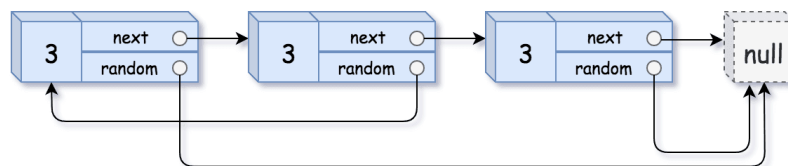
Example 2:



Input: head = [[1,1],[2,1]]

Output: [[1,1],[2,1]]

Example 3:



Input: head = [[3,null],[3,0],[3,null]]

Output: [[3,null],[3,0],[3,null]]

Constraints:

- $0 \leq n \leq 1000$
- $-104 \leq \text{Node.val} \leq 104$
- `Node.random` is null or is pointing to some node in the linked list.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def copyRandomList(self, head: 'Node') -> 'Node':
        if not head:
            return None
        if head in self.map:
            return self.map[head]

        newNode = Node(head.val)
        self.map[head] = newNode
        newNode.next = self.copyRandomList(head.next)
  
```

```

newNode.random = self.copyRandomList(head.random)
return newNode

```

```
map = {}
```

• LeetCode

```

"""
# Definition for a Node.
class Node:
    def __init__(self, x: int, next: 'Node' = None, random: 'Node' = None):
        self.val = int(x)
        self.next = next
        self.random = random
"""

```

```

class Solution:
    def copyRandomList(self, head: "Optional[Node]") -> "Optional[Node]":
        d = {}
        dummy = tail = Node(0)
        cur = head
        while cur:
            node = Node(cur.val)
            tail.next = node
            tail = tail.next
            d[cur] = node

            cur = cur.next
        cur = head
        while cur:
            d[cur].random = d[cur.random] if cur.random else None
            cur = cur.next
        return dummy.next

```

• SimplyLeet

```

# Definition for a Node.
class Node:
    def __init__(self, val, next=None, random=None):
        self.val = val        # value of the node.
        self.next = next      # pointer to the next node.
        self.random = random  # pointer to a random node.

class Solution:
    def copyRandomList(self, head: 'Node') -> 'Node':
        # Return None if the head is None.

        if not head:
            return None

        # Dictionary to hold old node as key and new node as value.
        old_to_new = {}

        # First pass: create a copy of each node.
        current = head
        while current:
            old_to_new[current] = Node(current.val)

            current = current.next

        # Second pass: assign next and random pointers for the copied nodes.
        current = head
        while current:
            # Using dict.get ensures proper assignment or returns None.
            old_to_new[current].next = old_to_new.get(current.next)
            old_to_new[current].random = old_to_new.get(current.random)
            current = current.next

        # Return the head of the new copied linked list.
        return old_to_new[head]

```

◆ Quick Review

Key Insights

- Use a hash map (or dictionary) to map each original node to its corresponding copy.
- Perform two passes: the first to create a copy of each node, and the second to assign the next and random pointers using the hash map.
- An alternative interleaving approach exists but using a hash table is typically more straightforward.

Space & Time

Time Complexity: $O(n)$ – Each node is processed a constant number of times.

Space Complexity: $O(n)$ – A hash map is used to store the mapping from original nodes to their copies.

Solution

The solution uses a two-pass algorithm. In the first pass, we iterate through the original list and for each node, create a new node (deep copy) while saving the mapping between the original and copy in a hash table. In the second pass, we iterate over the original list again to assign the next and random pointers of the new nodes using the previously built map. This guarantees that every pointer in the

copied list corresponds to a new node and not the original.

Linked List

□ #143 Reorder List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Two Pointers · Stack · Recursion
Lists: Grind 169

Problem

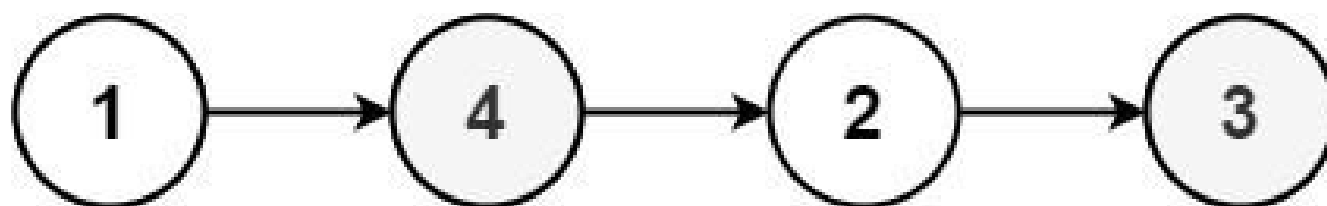
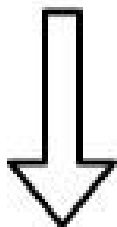
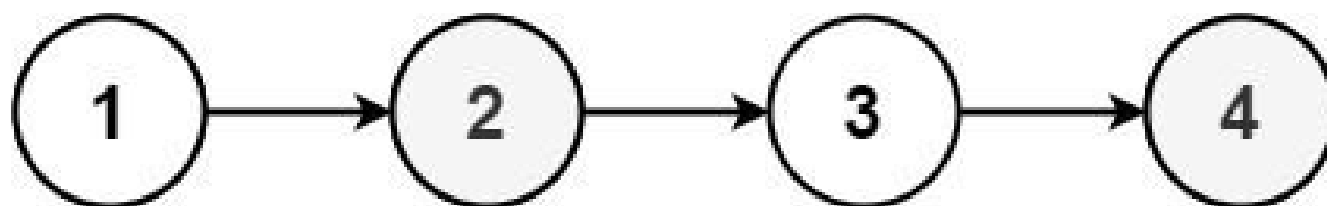
You are given the head of a singly linked-list.
The list can be represented as:

$$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$$

Reorder the list to be on the following form:

$$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$$

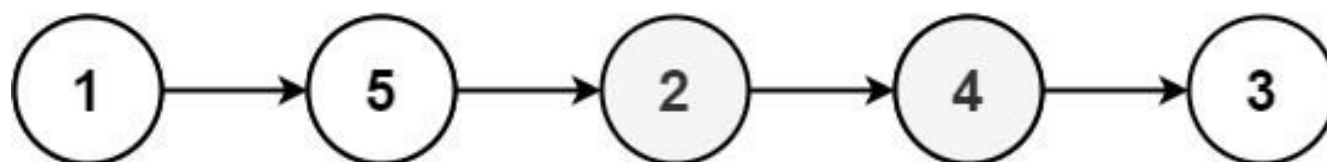
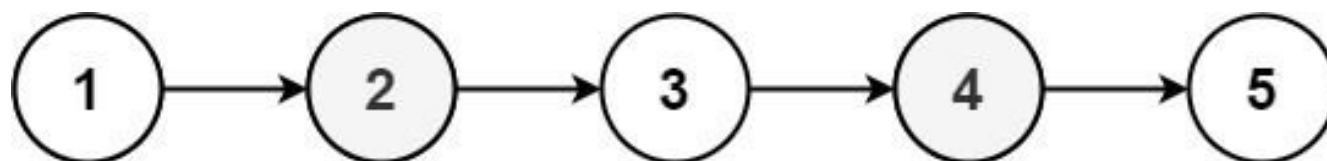
You may not modify the values in the list's nodes. Only nodes themselves may be changed.
Example 1:



Input: head = [1,2,3,4]

Output: [1,4,2,3]

Example 2:



Input: head = [1,2,3,4,5]

Output: [1,5,2,4,3]

Constraints:

- The number of nodes in the list is in the range $[1, 5 * 10^4]$.
- $1 \leq \text{Node.val} \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reorderList(self, head: ListNode) -> None:
        def findMid(head: ListNode):
            prev = None
            slow = head
            fast = head

            while fast and fast.next:
                prev = slow
                slow = slow.next
                fast = fast.next.next

            prev.next = None

            return slow

        def reverse(head: ListNode) -> ListNode:
            prev = None
            curr = head

            while curr:
                next = curr.next
                curr.next = prev
                prev = curr
                curr = next

            return prev

        def merge(l1: ListNode, l2: ListNode) -> None:
            while l2:
                next = l1.next
```

```

        l1.next = l2
        l1 = l2
        l2 = next

    if not head or not head.next:
        return

    mid = findMid(head)
    reversed = reverse(mid)
    merge(head, reversed)

```

• LeetDoocs

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reorderList(self, head: Optional[ListNode]) -> None:
        fast = slow = head
        while fast.next and fast.next.next:
            slow = slow.next

            fast = fast.next.next

        cur = slow.next
        slow.next = None

        pre = None
        while cur:
            t = cur.next
            cur.next = pre
            pre, cur = cur, t

        cur = head

        while pre:
            t = pre.next
            pre.next = cur.next
            cur.next = pre
            cur, pre = pre.next, t

```

• SimplyLeet

```
# Definition for singly-linked list.
```

```
class ListNode:
```

```
    def __init__(self, val=0, next=None):
```

```
        self.val = val
```

```
        self.next = next
```

```
class Solution:
```

```
    def reorderList(self, head: ListNode) -> None:
```

```
        if not head or not head.next:
```

```
            return
```

```
        # Step 1: Find the middle of the linked list using slow and fast pointers.
```

```
        slow, fast = head, head
```

```
        while fast and fast.next:
```

```
            slow = slow.next
```

```
            fast = fast.next.next
```

```
        # Step 2: Reverse the second half of the list.
```

```
        prev, curr = None, slow.next
```

```
        slow.next = None # Split the list into two halves.
```

```
        while curr:
```

```
            next_temp = curr.next # Save next node.
```

```
            curr.next = prev # Reverse current pointer.
```

```
            prev = curr # Move prev to current.
```

```
            curr = next_temp # Move to the next node.
```

```
        # Step 3: Merge the two halves, alternating nodes.
```

```
        first, second = head, prev
```

```
        while second:
```

```
            temp1 = first.next # Save next node from first half.
```

```
            temp2 = second.next # Save next node from reversed second half.
```

```
            first.next = second # Link first node to second list node.
```

```
            second.next = temp1 # Link second node to the next node in first
```

```
half.
```

```
            first = temp1 # Move pointer in first half.
```

```
            second = temp2 # Move pointer in second half.
```

• LC.ca

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reorderList(self, head: Optional[ListNode]) -> None:
        fast = slow = head
        while fast.next and fast.next.next:
            slow = slow.next

            fast = fast.next.next

        cur = slow.next
        slow.next = None

        pre = None
        while cur:
            t = cur.next
            cur.next = pre
            pre, cur = cur, t

        cur = head

        while pre:
            t = pre.next
            pre.next = cur.next
            cur.next = pre
            cur, pre = pre.next, t
```

◆ Quick Review

Key Insights

- Find the middle of the linked list using the fast and slow pointer technique.
- Reverse the second half of the list.

- Merge the two halves, alternating nodes from the first half and the reversed second half.
- The challenge is done in-place, ensuring a linear time solution with constant extra space.

Space & Time

Time Complexity: $O(n)$ – Each of the three main steps (finding the middle, reversing, merging) takes $O(n)$ time.

Space Complexity: $O(1)$ – The algorithm is done in-place with only a few pointers.

Solution

The solution involves three key steps: Use two pointers (slow and fast) to identify the middle of the list. Reverse the second half of the list. This can be done iteratively by adjusting the next pointers. Merge the two lists: One starting from the head and the other from the head of the reversed second half. Interleave the nodes by alternating between the two lists until complete. This approach uses in-place pointer manipulation, thus ensuring no extra space is used aside from a few temporary variables.

Linked List

□ #146 LRU Cache

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Design
Lists: Grind 169

Problem

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.

Implement the LRUCache class:

- **LRUCache(int capacity)** Initialize the LRU cache with positive size capacity.
- **int get(int key)** Return the value of the key if the key exists, otherwise return -1.
- **void put(int key, int value)** Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, evict the least recently used key.

The functions **get** and **put** must each run in $O(1)$ average time complexity.

Example 1:

Input

```
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]  
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
```

Output

```
[null, null, null, 1, null, -1, null, -1, 3, 4]
```

Explanation

```
LRUCache lRUCache = new LRUCache(2);
```

Constraints:

- $1 \leq \text{capacity} \leq 3000$
- $0 \leq \text{key} \leq 104$
- $0 \leq \text{value} \leq 105$
- At most $2 * 105$ calls will be made to get and put.

Community Solutions (Python)

• WalkCC

```
class Node:  
    def __init__(self, key: int, value: int):  
        self.key = key  
        self.value = value  
        self.prev = None  
        self.next = None
```

```
class LRUCache:  
    def __init__(self, capacity: int):
```

```
self.capacity = capacity
self.keyToNode = {}
self.head = Node(-1, -1)
self.tail = Node(-1, -1)
self.join(self.head, self.tail)

def get(self, key: int) -> int:
    if key not in self.keyToNode:
        return -1

    node = self.keyToNode[key]
    self.remove(node)
    self.moveToHead(node)
    return node.value

def put(self, key: int, value: int) -> None:
    if key in self.keyToNode:
        node = self.keyToNode[key]
        node.value = value
        self.remove(node)

    self.moveToHead(node)
    return

if len(self.keyToNode) == self.capacity:
    lastNode = self.tail.prev
    del self.keyToNode[lastNode.key]
    self.remove(lastNode)

self.moveToHead(Node(key, value))
self.keyToNode[key] = self.head.next

def join(self, node1: Node, node2: Node):
    node1.next = node2
    node2.prev = node1

def moveToHead(self, node: Node):
    self.join(node, self.head.next)
    self.join(self.head, node)

def remove(self, node: Node):
```



```
self.join(node.prev, node.next)
```

• SimplyLeet

```
# Define a node for the doubly linked list.
```

```
class Node:
```

```
    def __init__(self, key, value):
```

```
        self.key = key           # key of the node
```

```
        self.value = value       # value of the node
```

```
        self.prev = None         # pointer to previous node
```

```
        self.next = None         # pointer to next node
```

```
class LRUCache:
```

```
    def __init__(self, capacity):
```

```
        self.capacity = capacity
```

```
        # maximum capacity of the cache
```

```
        self.cache = {}
```

```
        # dictionary mapping key ->
```

```
node
```

```
    # Create dummy head and tail nodes.
```

```
    self.head = Node(0, 0)
```

```
    self.tail = Node(0, 0)
```

```
    self.head.next = self.tail
```

```
    self.tail.prev = self.head
```

```
# Helper function to remove a node from the linked list.
```

```
def _remove(self, node):
```

```
    prev_node = node.prev
```

```
    next_node = node.next
```

```
    prev_node.next = next_node
```

```
    next_node.prev = prev_node
```

```
# Helper function to add a node right after the head.
```

```
def _add(self, node):
```

```
    node.prev = self.head
```

```
    node.next = self.head.next
```

```
    self.head.next.prev = node
```

```

        self.head.next = node

    # Returns the value of the key if it exists; otherwise, returns -1.
    def get(self, key):
        if key in self.cache:
            node = self.cache[key]
            # Move the accessed node to the head (marking it as recently used)
            self._remove(node)
            self._add(node)
            return node.value

        return -1

    # Updates the value of the key if it exists; otherwise, adds the key-value
    pair.
    # Removes the least recently used key if the cache exceeds its capacity.
    def put(self, key, value):
        if key in self.cache:
            # Remove the old node.
            self._remove(self.cache[key])
        node = Node(key, value)
        self._add(node)

        self.cache[key] = node
        if len(self.cache) > self.capacity:
            # Remove the least recently used element.
            lru = self.tail.prev
            self._remove(lru)
            del self.cache[lru.key]

```

• LC.ca

```

class Node:
    def __init__(self, key=0, val=0):
        self.key = key
        self.val = val
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):

```

```
self.cache = {} # key ==> Node(val)
self.head = Node() # dummy node
self.tail = Node() # dummy node
self.capacity = capacity
self.size = 0
self.head.next = self.tail # note: key setup
self.tail.prev = self.head

def get(self, key: int) -> int:
    if key not in self.cache:

        return -1
    node = self.cache[key]
    self.move_to_head(node)
    return node.val

def put(self, key: int, value: int) -> None:
    if key in self.cache:
        node = self.cache[key]
        node.val = value
        self.move_to_head(node)

    else:
        node = Node(key, value)
        self.cache[key] = node
        self.add_to_head(node)
        self.size += 1
        if self.size > self.capacity:
            tail = self.remove_tail()
            self.cache.pop(tail.key)
            self.size -= 1

def move_to_head(self, node):
    self.remove_node(node)
    self.add_to_head(node)

def remove_node(self, node):
    node.prev.next = node.next
    node.next.prev = node.prev

def add_to_head(self, node):
    node.next = self.head.next
```

```

        node.prev = self.head
        self.head.next = node
        node.next.prev = node

    def remove_tail(self):
        node = self.tail.prev
        self.remove_node(node)
        return node

```

```

# Your LRUCache object will be instantiated and called as such:
# obj = LRUCache(capacity)
# param_1 = obj.get(key)
# obj.put(key,value)

#####

'''
example:

```

```

>>> od = collections.OrderedDict()
>>>
>>> od[1]=1
>>> od[2]=2
>>> od[3]=3
>>>
>>> od
OrderedDict([(1, 1), (2, 2), (3, 3)])
>>> od.move_to_end(1)
>>> od

```

```

OrderedDict([(2, 2), (3, 3), (1, 1)])
>>>
>>> od.get(1)
1
>>> od.popitem()
(1, 1)
>>> od
OrderedDict([(2, 2), (3, 3)])
>>>
>>> od[1]=1

```

```
>>> od
OrderedDict([(2, 2), (3, 3), (1, 1)])
>>>
>>> od.popitem(last=False)
(2, 2)
>>> od
OrderedDict([(3, 3), (1, 1)])

...

```

```
import collections

class LRUCache:
    def __init__(self, capacity: 'int'):
        self.cache = collections.OrderedDict()
        self.remain = capacity

    def get(self, key: 'int') -> 'int':
        if key not in self.cache:
            return -1

        self.cache.move_to_end(key) # meaning end is the most recently used
        return self.cache.get(key)

    def put(self, key: 'int', value: 'int') -> 'None':
        if key not in self.cache:
            if self.remain > 0:
                self.remain -= 1
            else:
                self.cache.popitem(last=False) # pop start position
        else:

```

```
        self.cache.pop(key)
        self.cache[key] = value # add to end of dict, meaning most recently
used
```

```
#####
```

```
### below solution with no ordered-dict
```

```
class DLinkedList:
```

```
    def __init__(self, key=0, value=0):
```

```
        self.key = key
        self.val = value
        self.next = None
        self.prev = None
```

```
class DLinkedList:
```

```
    def __init__(self):
```

```
        self.head = DLinkedList() # dummy head, its next is real head
```

```
        self.tail = DLinkedList() # dummy tail, its prev is real tail
        self.head.next, self.tail.prev = self.tail, self.head
```

```
    def add_node(self, node): # add to head
        node.prev, node.next = self.head, self.head.next
        node.next.prev, node.prev.next = node, node
```

```
    def remove_node(self, node):
        node.prev.next, node.next.prev = node.next, node.prev
        return node.key
```

```
def remove_tail(self):
    return self.remove_node(self.tail.prev) # dummy tail's prev is real
tail

def move_to_head(self, node):
    self.remove_node(node)
    self.add_node(node)

# def __repr__(self):
# ans = []

# h = self.head
# while h:
# ans.append(str(h.val))
# h = h.next
# return '<DLinkedList: {}>'.format('->'.join(ans))

class LRUCache:

    def __init__(self, capacity: int):
        self.capacity = capacity

        self.nodes_map = {}
        self.cache_list = DLinkedList()

    def get(self, key: int) -> int:
        node = self.nodes_map.get(key, None)
        if node:
            self.cache_list.move_to_head(node)
            return node.val
        else:
            return -1
```

```
def put(self, key: int, value: int) -> None:
    node = self.nodes_map.get(key, None)
    if not node:
        if self.capacity > 0:
            self.capacity -= 1
        else:
            rm_key = self.cache_list.remove_tail()
            self.nodes_map.pop(rm_key) # note api
            new_node = DLinkedNode(key, value)

            self.nodes_map[key] = new_node
            self.cache_list.add_node(new_node)
    else:
        self.cache_list.move_to_head(node)
        node.val = value
```

◆ Quick Review

Key Insights

- Use a Hash Table (dictionary) to allow $O(1)$ access to nodes by key.
- Use a Doubly Linked List to record the usage order. The head represents the most recently used element while the tail represents the least recently used.
- When a key is accessed or updated, move the corresponding node to the front (head) of the list.
- When the cache exceeds capacity, remove the node at the tail end which is the least recently used entry.

Space & Time

Time Complexity: $O(1)$ per get and put operation.

Space Complexity: $O(\text{capacity})$, storing key-node pairs and linked list nodes.

Solution

The solution combines a hash table (or dictionary) and a doubly linked list: The hash table stores keys and corresponding node addresses. The doubly linked list maintains the order of usage with the most recently used items near the head. For `get(key)`, check the dictionary. If found, move the node to the head and return its value; if not, return `-1`. For `put(key, value)`, if the key exists, update its value and move it to the head. If not, create a new node and add it right after the head. If the capacity is reached, remove the tail node (least recently used) and update the dictionary accordingly. Dummy head and tail nodes are used to simplify node addition and removal procedures without worrying about edge cases.

Linked List

□ #287 Find the Duplicate Number

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search · Bit
Manipulation

Lists: Grind 169

Problem

Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive.

There is only one repeated number in `nums`, return this repeated number.

You must solve the problem without modifying the array `nums` and using only constant extra space.

Example 1:

Input: `nums = [1,3,4,2,2]`
Output: 2

Example 2:

Input: `nums = [3,1,3,4,2]`
Output: 3

Example 3:

Input: nums = [3,3,3,3,3]

Output: 3

Constraints:

- $1 \leq n \leq 105$
- `nums.length == n + 1`
- $1 \leq \text{nums}[i] \leq n$
- All the integers in nums appear only once except for precisely one integer which appears two or more times.

Follow up:

- How can we prove that at least one duplicate number must exist in nums?
- Can you solve the problem in linear runtime complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findDuplicate(self, nums: list[int]) -> int:
        slow = nums[nums[0]]
        fast = nums[nums[nums[0]]]

        while slow != fast:
            slow = nums[slow]
            fast = nums[nums[fast]]

        slow = nums[0]
```

```

while slow != fast:
    slow = nums[slow]
    fast = nums[fast]

return slow

```

• LeetCode

```

class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        def f(x: int) -> bool:
            return sum(v <= x for v in nums) > x

        return bisect_left(range(len(nums)), True, key=f)

```

• SimplyLeet

Function to find the duplicate number using Floyd's Tortoise and Hare algorithm.

```

def findDuplicate(nums):
    # Phase 1: Find the intersection point inside the cycle.
    tortoise = nums[0] # Initialize tortoise pointer.
    hare = nums[0]     # Initialize hare pointer.
    while True:
        tortoise = nums[tortoise] # Move tortoise one step.
        hare = nums[nums[hare]]   # Move hare two steps.
        if tortoise == hare:      # If pointers meet, cycle is
detected.
            break

```

```

# Phase 2: Find the entrance to the cycle.
pointer = nums[0] # Initialize a new pointer from the start.
while pointer != tortoise:
    pointer = nums[pointer] # Move pointer one step.
    tortoise = nums[tortoise] # Move tortoise one step.
return pointer # The duplicate number.

```

Example usage:

```

if __name__ == "__main__":

```

```
sample = [1, 3, 4, 2, 2]
print(findDuplicate(sample)) # Expected output: 2
```

• LC.ca

```
class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        def f(x: int) -> bool:
            return sum(v <= x for v in nums) > x

        return bisect_left(range(len(nums)), True, key=f)
```

◆ Quick Review

Key Insights

- Since there are $n+1$ elements with values only from 1 to n , by the pigeonhole principle at least one number must be repeated.
- The problem can be approached by mapping the array values to indices, forming a cycle structure.
- Floyd's Tortoise and Hare algorithm (cycle detection) can be applied to find the duplicate number in linear time and constant space.
- There is no need for extra data structures like hash sets or alterations to the original array, satisfying the space and array-modification constraints.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution leverages Floyd's Tortoise and Hare cycle detection algorithm. Think of each array element as a pointer to another index, forming a linked list with a cycle (the duplicate number creates the cycle). The algorithm works in two phases: Detect a meeting point within the cycle by having two pointers (tortoise and hare) move at different speeds. Find the cycle entrance by resetting one pointer to the start and moving both pointers at the same speed. The meeting point reveals the duplicate number. This approach provides an elegant solution that satisfies both linear runtime and constant space requirements.

Stack

Round 5 · Mid · Medium · 4 questions

Stack

□ #150 Evaluate Reverse Polish Notation

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Stack
Lists: Grind 169

Problem

You are given an array of strings tokens that represents an arithmetic expression in a Reverse Polish Notation.

Evaluate the expression. Return an integer that represents the value of the expression.

Note that:

- The valid operators are '+', '-', '*', and '/'.
- Each operand may be an integer or another expression.
- The division between two integers always truncates toward zero.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.

- The answer and all the intermediate calculations can be represented in a 32-bit integer.

Example 1:

Input: tokens = ["2","1","+","3","*"]
Output: 9
Explanation: $((2 + 1) * 3) = 9$

Example 2:

Input: tokens = ["4","13","5","/","+"]
Output: 6
Explanation: $(4 + (13 / 5)) = 6$

Example 3:

Input: tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"]
Output: 22
Explanation: $((10 * (6 / ((9 + 3) * -11))) + 17) + 5$
 $= ((10 * (6 / (12 * -11))) + 17) + 5$
 $= ((10 * (6 / -132)) + 17) + 5$
 $= ((10 * 0) + 17) + 5$
 $= (0 + 17) + 5$
 $= 17 + 5$

Constraints:

- $1 \leq \text{tokens.length} \leq 104$
- $\text{tokens}[i]$ is either an operator: "+", "-", "*", or "/", or an integer in the range $[-200, 200]$.

Community Solutions (Python)

- WalkCC

```

class Solution:
    def evalRPN(self, tokens: list[str]) -> int:
        stack = []
        op = {
            '+': lambda a, b: a + b,
            '-': lambda a, b: a - b,
            '*': lambda a, b: a * b,
            '/': lambda a, b: int(a / b),
        }

        for token in tokens:
            if token in op:
                b = stack.pop()
                a = stack.pop()
                stack.append(op[token](a, b))
            else:
                stack.append(int(token))

        return stack.pop()

```

• LeetDoocs

```

import operator

class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        opt = {
            "+": operator.add,
            "-": operator.sub,
            "*": operator.mul,
            "/": operator.truediv,
        }

        s = []
        for token in tokens:
            if token in opt:
                s.append(int(opt[token](s.pop(-2), s.pop(-1))))
            else:
                s.append(int(token))
        return s[0]

```

```

class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        nums = []
        for t in tokens:
            if len(t) > 1 or t.isdigit():
                nums.append(int(t))
            else:
                if t == "+":
                    nums[-2] += nums[-1]
                elif t == "-":
                    nums[-2] -= nums[-1]
                elif t == "*":
                    nums[-2] *= nums[-1]
                else:
                    nums[-2] = int(nums[-2] / nums[-1])
                nums.pop()
        return nums[0]

```

• SimplyLeet

```

def evalRPN(tokens):
    # Initialize a stack to hold numbers
    stack = []
    # Loop through each token in the input list
    for token in tokens:
        if token in {"+", "-", "*", "/"}:
            # Pop the two topmost values; note the order is important
            b = stack.pop()
            a = stack.pop()
            # Apply the operation based on the token

            if token == "+":
                result = a + b
            elif token == "-":
                result = a - b
            elif token == "*":
                result = a * b
            else:
                # Use int division to truncate towards zero
                result = int(a / b)
            # Push the result back onto the stack

```

```

        stack.append(result)
    else:
        # Token is a number; push its integer value onto the stack
        stack.append(int(token))
    # The final result is the only element left in the stack
    return stack.pop()

# Example usage:
# tokens = ["2", "1", "+", "3", "*"]
# print(evalRPN(tokens)) # Output: 9

```

• LC.ca

```

class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        nums = []
        for t in tokens:
            if len(t) > 1 or t.isdigit():
                nums.append(int(t))
            else:
                if t == "+":
                    nums[-2] += nums[-1]
                elif t == "-":
                    nums[-2] -= nums[-1]
                elif t == "*":
                    nums[-2] *= nums[-1]
                else:
                    nums[-2] = int(nums[-2] / nums[-1])
                nums.pop()
        return nums[0]

```

◆ Quick Review

Key Insights

- Use a stack to process the tokens.
- Push numbers onto the stack.

- When an operator is encountered, pop the top two numbers, perform the operation, and push the result back.
- Handle division carefully to ensure it truncates toward zero.
- The final result remains as the only value in the stack.

Space & Time

Time Complexity: $O(n)$, where n is the number of tokens (each token is processed once).

Space Complexity: $O(n)$, in the worst-case scenario where all tokens are numbers and are stored in the stack.

Solution

We use a stack data structure to evaluate the RPN expression. Iterate through each token: If the token is a number, convert and push it onto the stack. If the token is an operator, pop the top two numbers (be careful about order), perform the corresponding arithmetic operation, and push the result back onto the stack. At the end of processing, the stack contains the result of the expression. Special attention is needed for division to ensure it

truncates toward zero (using language-specific functions when necessary).

Stack

□ #155 Min Stack

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Design
Lists: Grind 169

Problem

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

- **MinStack()** initializes the stack object.
- **void push(int val)** pushes the element val onto the stack.
- **void pop()** removes the element on the top of the stack.
- **int top()** gets the top element of the stack.
- **int getMin()** retrieves the minimum element in the stack.

You must implement a solution with $O(1)$ time complexity for each function.

Example 1:

Input

```
["MinStack", "push", "push", "push", "getMin", "pop", "top", "getMin"]
[[], [-2], [0], [-3], [], [], [], []]
```

Output

```
[null, null, null, null, -3, null, 0, -2]
```

Explanation

Constraints:

- $-231 \leq \text{val} \leq 231 - 1$
- Methods `pop`, `top` and `getMin` operations will always be called on non-empty stacks.
- At most $3 * 10^4$ calls will be made to `push`, `pop`, `top`, and `getMin`.

Community Solutions (Python)

• WalkCC

```
class MinStack:
    def __init__(self):
        self.stack = []

    def push(self, x: int) -> None:
        mn = x if not self.stack else min(self.stack[-1][1], x)
        self.stack.append([x, mn])

    def pop(self) -> None:
        self.stack.pop()

    def top(self) -> int:
        return self.stack[-1][0]

    def getMin(self) -> int:
        return self.stack[-1][1]
```


• LeetCode

```
class MinStack:
    def __init__(self):
        self.stk1 = []
        self.stk2 = [inf]

    def push(self, val: int) -> None:
        self.stk1.append(val)
        self.stk2.append(min(val, self.stk2[-1]))

    def pop(self) -> None:
        self.stk1.pop()
        self.stk2.pop()

    def top(self) -> int:
        return self.stk1[-1]

    def getMin(self) -> int:
        return self.stk2[-1]
```

```
# Your MinStack object will be instantiated and called as such:
# obj = MinStack()
# obj.push(val)
# obj.pop()
# param_3 = obj.top()
# param_4 = obj.getMin()
```

• SimplyLeet

```
# Python implementation of MinStack with detailed comments
```

```
class MinStack:
    def __init__(self):
        # Main stack to store all values
        self.stack = []
        # Auxiliary stack to store minimum values
        self.min_stack = []

    def push(self, val: int) -> None:
```

```

        # Push value onto main stack
        self.stack.append(val)
        # If min_stack is empty or the new value is less than or equal to
        current minimum, push it onto min_stack
        if not self.min_stack or val <= self.min_stack[-1]:
            self.min_stack.append(val)

    def pop(self) -> None:
        # Pop the top value from main stack
        popped_value = self.stack.pop()
        # If the popped value is the current minimum, pop it from min_stack as
        well

        if popped_value == self.min_stack[-1]:
            self.min_stack.pop()

    def top(self) -> int:
        # Return the top value from main stack without removing it
        return self.stack[-1]

    def getMin(self) -> int:
        # Return the minimum value from min_stack
        return self.min_stack[-1]

# Example usage:
# minStack = MinStack()
# minStack.push(-2)
# minStack.push(0)
# minStack.push(-3)
# print(minStack.getMin()) # -> -3
# minStack.pop()
# print(minStack.top()) # -> 0
# print(minStack.getMin()) # -> -2

```

• LC.ca

```

class MinStack:
    def __init__(self):
        self.sk = []
        self.mins = [inf] # trick, avoid later empty-check for min-stack

    def push(self, x: int) -> None:
        self.sk.append(x)
        self.mins.append(min(x, self.mins[-1]))

    def pop(self) -> None:

if not self.sk:
    return
    self.sk.pop()
    self.mins.pop()

    def top(self) -> int:
        return self.sk[-1]

    def getMin(self) -> int:
        return self.mins[-1]

```

```

# Your MinStack object will be instantiated and called as such:
# obj = MinStack()
# obj.push(x)
# obj.pop()
# param_3 = obj.top()
# param_4 = obj.getMin()

#####

```

```

class MinStack:

    def __init__(self):
        self.sk = []
        self.mins = []

    def push(self, val: int) -> None:
        self.sk.append(val)

```

```
        # empty check
self.minsk.append(val if not self.minsk else min(val, self.minsk[-1]))

def pop(self) -> None:
    if not self.sk:
        return

    self.sk.pop()
    self.minsk.pop()

def top(self) -> int:
    return self.sk[-1]

def getMin(self) -> int:
    return self.minsk[-1]

# optimize above, using only 1 stack, with stack element as tuple: (val, its
# associated min)
class MinStack:

    def __init__(self):

        self._stack = []

    def push(self, x: int) -> None:
        cur_min = self.getMin()
        if x < cur_min:
            cur_min = x
        self._stack.append((x, cur_min))

    def pop(self) -> None:
        self._stack.pop()
```

```
def top(self) -> int:
    if not self._stack:
        return None
    else:
        return self._stack[-1][0]

def getMin(self) -> int:
    if not self._stack:
        return float('inf')
    else:
        return self._stack[-1][1]
```

◆ Quick Review

Key Insights

- Use an auxiliary data structure to track the current minimum element.
- A common approach is to use two stacks: one regular stack for all elements and a second stack to keep track of the minimum values.
- When pushing a new element, push it onto the main stack and also update the min stack if the element is less than or equal to the current minimum.
- When popping, if the popped element is equal to the top of the min stack, pop from the min stack as well.

- This design guarantees that all operations (push, pop, top, getMin) run in constant time, $O(1)$.

Space & Time

Time Complexity: $O(1)$ per operation (push, pop, top, getMin)

Space Complexity: $O(n)$, where n is the number of elements in the stack, since an additional stack is used to keep track of minima.

Solution

The solution involves using two stacks: Main Stack: Stores all the elements. Min Stack: Stores the minimum elements encountered so far. When an element is pushed, we compare it with the top of the min stack. If it is smaller than or equal to the current minimum, we also push it onto the min stack. For the pop operation, if the element being popped is the same as the top of the min stack, we pop from the min stack as well. This approach ensures that the getMin operation always returns the top element of the min stack, which is the current minimum in constant time.

Stack

□ #739 Daily Temperatures

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: Grind 169

Problem

Given an array of integers `temperatures` represents the daily temperatures, return an array `answer` such that `answer[i]` is the number of days you have to wait after the `i`th day to get a warmer temperature. If there is no future day for which this is possible, keep `answer[i] == 0` instead.

Example 1:

Input: `temperatures = [73,74,75,71,69,72,76,73]`
Output: `[1,1,4,2,1,1,0,0]`

Example 2:

Input: `temperatures = [30,40,50,60]`
Output: `[1,1,1,0]`

Example 3:

Input: `temperatures = [30,60,90]`
Output: `[1,1,0]`

Constraints:

- $1 \leq \text{temperatures.length} \leq 105$
- $30 \leq \text{temperatures}[i] \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def dailyTemperatures(self, temperatures: list[int]) -> list[int]:
        ans = [0] * len(temperatures)
        stack = [] # a decreasing stack

        for i, temperature in enumerate(temperatures):
            while stack and temperature > temperatures[stack[-1]]:
                index = stack.pop()
                ans[index] = i - index
            stack.append(i)

        return ans
```

• LeetDoocs

```
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        stk = []
        n = len(temperatures)
        ans = [0] * n
        for i in range(n - 1, -1, -1):
            while stk and temperatures[stk[-1]] <= temperatures[i]:
                stk.pop()
            if stk:
                ans[i] = stk[-1] - i

            stk.append(i)
        return ans
```

• SimplyLeet


```
def dailyTemperatures(temperatures):
    n = len(temperatures)
    # Initialize answer list with zeros
    answer = [0] * n
    # Stack to keep indices of temperatures with unresolved next warmer days
    stack = []
    # Iterate through the temperature list
    for i, temp in enumerate(temperatures):
        # Check if the current temperature resolves any previous indices
        while stack and temperatures[stack[-1]] < temp:
            prev_index = stack.pop() # Get the index of the previous colder
            day
            answer[prev_index] = i - prev_index # Update the number of days
            waited
            # Push the current index onto the stack
            stack.append(i)
    return answer

# Example usage:
if __name__ == "__main__":
    temps = [73,74,75,71,69,72,76,73]
    print(dailyTemperatures(temps))
```

• LC.ca

```
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        ans = [0] * len(temperatures)
        stk = []
        for i, t in enumerate(temperatures):
            while stk and temperatures[stk[-1]] < t:
                j = stk.pop()
                ans[j] = i - j
            stk.append(i)
        return ans
```

◆ Quick Review

Key Insights

- Use a monotonic stack to keep track of indices of days with unresolved warmer temperatures.
- As you iterate through the temperatures, resolve previous days that have found a warmer temperature.
- The index differences yield the number of days waited until a warmer temperature.

Space & Time

Time Complexity: $O(n)$, where n is the number of temperatures, because each index is pushed and popped from the stack at most once.

Space Complexity: $O(n)$ for storing the stack and the answer array.

Solution

The solution uses a monotonic stack to track the indices whose next warmer temperature has not been found yet. As you iterate through the temperature list, compare the current temperature with the temperature at the index stored at the top of the stack. If the current temperature is higher, it means you've found a warmer day for the day represented by the index from the stack. Pop that index, calculate

the difference between the current index and the popped index, and store the result. Push the current index onto the stack for future comparisons. This approach efficiently finds the answer without nested iterations.

Stack

□ #853 Car Fleet

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Sorting · Monotonic Stack
Lists: NeetCode 150

Problem

There are n cars at given miles away from the starting mile 0, traveling to reach the mile target.

You are given two integer arrays `position` and `speed`, both of length n , where `position[i]` is the starting mile of the i th car and `speed[i]` is the speed of the i th car in miles per hour.

A car cannot pass another car, but it can catch up and then travel next to it at the speed of the slower car.

A car fleet is a single car or a group of cars driving next to each other. The speed of the car fleet is the minimum speed of any car in the fleet.

If a car catches up to a car fleet at the mile target, it will still be considered as part of the car fleet.

Return the number of car fleets that will arrive at the destination.

Example 1:

Input: target = 12, position = [10,8,0,5,3], speed = [2,4,1,1,3]

Output: 3

Explanation:

- The cars starting at 10 (speed 2) and 8 (speed 4) become a fleet, meeting each other at 12. The fleet forms at target.
- The car starting at 0 (speed 1) does not catch up to any other car, so it is a fleet by itself.
- The cars starting at 5 (speed 1) and 3 (speed 3) become a fleet, meeting each other at 6. The fleet moves at speed 1 until it reaches target.

Example 2:

Input: target = 10, position = [3], speed = [3]

Output: 1

Explanation:

Example 3:

Input: target = 100, position = [0,2,4], speed = [4,2,1]

Output: 1

Explanation:

- The cars starting at 0 (speed 4) and 2 (speed 2) become a fleet, meeting each other at 4. The car starting at 4 (speed 1) travels to 5.
- Then, the fleet at 4 (speed 2) and the car at position 5 (speed 1) become one fleet, meeting each other at 6. The fleet moves at speed 1 until it reaches target.

Constraints:

- $n == \text{position.length} == \text{speed.length}$
- $1 \leq n \leq 105$
- $0 < \text{target} \leq 10^6$
- $0 \leq \text{position}[i] < \text{target}$
- All the values of position are unique.
- $0 < \text{speed}[i] \leq 10^6$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def carFleet(self, target: int, position: list[int], speed: list[int]) -> int:
        ans = 0
        times = [
            float(target - p) / s for p, s in sorted(zip(position, speed),
                                                    reverse=True)]

        maxTime = 0 # the time of the slowest car to reach the target

        for time in times:
            # A car needs more time to reach the target, so it becomes the slowest.

            if time > maxTime:
                maxTime = time
                ans += 1

        return ans

```

• LeetCode

```

class Solution:
    def carFleet(self, target: int, position: List[int], speed: List[int]) -> int:
        idx = sorted(range(len(position)), key=lambda i: position[i])
        ans = pre = 0
        for i in idx[::-1]:
            t = (target - position[i]) / speed[i]
            if t > pre:
                ans += 1
                pre = t
        return ans

```

• SimplyLeet

```
# Python implementation of Car Fleet
def carFleet(target, position, speed):
    # Pair up position and speed and sort by starting position in descending
    # order.
    cars = sorted(zip(position, speed), key=lambda car: car[0], reverse=True)

    fleets = 0          # Count of car fleets
    last_time = 0.0     # Time taken by the last fleet to reach the target

    # Process each car from closest to farthest from the target.
    for pos, spd in cars:

        # Calculate time for this car to reach the target
        time = (target - pos) / spd

        # If this car takes longer than the last fleet time, it starts a new
        # fleet.
        if time > last_time:
            fleets += 1
            last_time = time # Update the last fleet time with this car's time

    return fleets

# Example usage:
# target = 12, position = [10,8,0,5,3], speed = [2,4,1,1,3]
# print(carFleet(12, [10,8,0,5,3], [2,4,1,1,3])) # Output: 3
```

◆ Quick Review

Key Insights

- Since cars cannot pass each other, the relative order (by starting position) is maintained.
- Sorting the cars based on their starting positions (descending order) helps to simulate

the catching-up process.

- Computing the time each car takes to reach the target is critical.
- Compare the current car's time to the time of the fleet ahead: if it takes longer, it forms a new fleet; otherwise, it gets merged.
- A stack-like approach (tracking the slowest arrival time seen so far) reduces the problem to a single pass after sorting.

Space & Time

Time Complexity: $O(n \log n)$ due to the sorting step.

Space Complexity: $O(n)$ for storing the computed times and sorted car information.

Solution

The solution involves sorting the cars by their starting positions in descending order so that we process the cars from nearest to the target to farthest. For each car, calculate the time to reach the target using the formula $(\text{target} - \text{position}) / \text{speed}$. As we iterate:

- If a car's time is greater than the slowest time encountered (tracked by a variable or stack), it cannot catch up to the fleet ahead

and forms a new fleet. – Otherwise, it merges into the current fleet. This approach ensures that we count each distinct fleet accurately with a single pass through the sorted list.

Heap / Priority Queue

Round 5 · Mid · Medium · 4 questions

Heap / Priority Queue

□ #215 Kth Largest Element in an Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Divide and Conquer · Sorting · Heap ·
Quickselect

Lists: Grind 169

Problem

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array.

Note that it is the `k`th largest element in the sorted order, not the `k`th distinct element.

Can you solve it without sorting?

Example 1:

Input: `nums = [3,2,1,5,6,4]`, `k = 2`
Output: 5

Example 2:

Input: `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`
Output: 4

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findKthLargest(self, nums: list[int], k: int) -> int:
        minHeap = []

        for num in nums:
            heapq.heappush(minHeap, num)
            if len(minHeap) > k:
                heapq.heappop(minHeap)

        return minHeap[0]

class Solution:
    def findKthLargest(self, nums: list[int], k: int) -> int:
        def quickSelect(l: int, r: int, k: int) -> int:
            pivot = nums[r]

            nextSwapped = l
            for i in range(l, r):
                if nums[i] >= pivot:
                    nums[nextSwapped], nums[i] = nums[i], nums[nextSwapped]
                    nextSwapped += 1

            nums[nextSwapped], nums[r] = nums[r], nums[nextSwapped]

            count = nextSwapped - l + 1 # Number of nums >= pivot
            if count == k:
                return nums[nextSwapped]
            if count > k:
                return quickSelect(l, nextSwapped - 1, k)
            return quickSelect(nextSwapped + 1, r, k - count)

        return quickSelect(0, len(nums) - 1, k)
```

• LeetDoocs

```

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quick_sort(l: int, r: int) -> int:
            if l == r:
                return nums[l]
            i, j = l - 1, r + 1
            x = nums[(l + r) >> 1]
            while i < j:
                while 1:
                    i += 1
                    if nums[i] >= x:
                        break
                while 1:
                    j -= 1
                    if nums[j] <= x:
                        break
                if i < j:
                    nums[i], nums[j] = nums[j], nums[i]
            if j < k:
                return quick_sort(j + 1, r)

        return quick_sort(l, j)

    n = len(nums)
    k = n - k
    return quick_sort(0, n - 1)

```

```

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        return nlargest(k, nums)[-1]

```

```

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        cnt = Counter(nums)
        for i in count(max(cnt), -1):
            k -= cnt[i]
            if k <= 0:
                return i

```

• SimplyLeet

```
import heapq

def findKthLargest(nums, k):
    # Create a min-heap with the first k elements
    min_heap = nums[:k]
    heapq.heapify(min_heap)  # O(k) time to build the initial heap

    # Process the remaining elements
    for num in nums[k:]:
        # If the current element is larger than the smallest in the heap,
        # replace it

        if num > min_heap[0]:
            heapq.heapreplace(min_heap, num)  # O(log k)
    # The smallest element in the heap is the kth largest overall
    return min_heap[0]

# Example usage:
nums = [3, 2, 1, 5, 6, 4]
k = 2
print(findKthLargest(nums, k))  # Expected output: 5
```

• LC.ca

```
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        lo, hi = 0, len(nums) - 1

        while lo <= hi:
            pos = self.partition(nums, lo, hi)
            if pos == k - 1: # pos starting from 0, so -1
                return nums[pos]  # partially sorted, desc
            elif pos > k - 1:
                hi = pos - 1
```

```

        else: # pos < k - 1
            lo = pos + 1

    return -1 # or raise exception

def partition(self, nums: List[int], lo: int, hi: int) -> int:
    pivot, l, r = nums[lo], lo + 1, hi
    while l <= r:
        if nums[l] < pivot and nums[r] > pivot:
            # larger num at left of pivot, easier to count for k-th largest

            nums[l], nums[r] = nums[r], nums[l]
            l += 1
            r -= 1
        if nums[l] >= pivot:
            l += 1
        if nums[r] <= pivot:
            r -= 1
    # use nums[l] will lead to infinite looping
    nums[lo], nums[r] = nums[r], nums[lo]
    # possible there is duplicated num, but will be covered here

    return r

#####

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quick_sort(left, right, k):
            if left == right:
                return nums[left]
            i, j = left - 1, right + 1

            x = nums[(left + right) >> 1]
            while i < j:
                while 1:
                    i += 1
                    if nums[i] >= x:
                        break
                while 1:
                    j -= 1
                    if nums[j] <= x:
                        break

```



```

        if i < j:
            nums[i], nums[j] = nums[j], nums[i]
        if j < k:
            return quick_sort(j + 1, right, k)
        return quick_sort(left, j, k)

n = len(nums)
return quick_sort(0, n - 1, n - k)

```

```
#####
```

```

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quick_sort(left, right, k):
            if left == right:
                return nums[left]
            i, j = left - 1, right + 1
            x = nums[(left + right) >> 1]
            while i < j:

```

```

                while 1:
                    i += 1
                    if nums[i] >= x:
                        break
                while 1:
                    j -= 1
                    if nums[j] <= x:
                        break
                if i < j:
                    nums[i], nums[j] = nums[j], nums[i]

```

```

            if j < k:
                return quick_sort(j + 1, right, k)
            return quick_sort(left, j, k)

n = len(nums)
return quick_sort(0, n - 1, n - k)

```

◆ Quick Review

Key Insights

- k th largest element can be converted to finding the element at index $n-k$ if the array were sorted.
- A min-heap of size k is a practical method to maintain the k largest elements seen so far.
- The Quickselect algorithm can find the k th largest element in average $O(n)$ time with in-place partitioning.
- Choose an appropriate method based on performance needs and worst-case scenarios.

Space & Time

Time Complexity:

Min-Heap: $O(n \log k)$ by maintaining a heap of size k .

Quickselect: Average $O(n)$ but worst-case $O(n^2)$ with poor pivot selection.

Space Complexity:

Min-Heap: $O(k)$

Quickselect: $O(1)$ additional space with in-place partitioning.

Solution

We can solve the problem using either a min-heap or the Quickselect algorithm.

Min-Heap Approach: Initialize a min-heap with the first k elements. Iterate through the remaining elements; if an element is greater than the heap's root (the smallest in the heap), replace the root. After processing all elements, the root of the min-heap is the k th largest element.

Quickselect Approach: Choose a pivot and partition the array so that elements larger than the pivot come before it. Determine the position of the pivot; if it matches the index corresponding to the k th largest element, return it. Recurse into the appropriate part of the array. Key points to consider include maintaining heap size for the first approach and proper pivot selection for the Quickselect approach.

Heap / Priority Queue

□ #355 Design Twitter

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Design · Heap
(Priority Queue)

Lists: NeetCode 150

Problem

Design a simplified version of Twitter where users can post tweets, follow/unfollow another user, and is able to see the 10 most recent tweets in the user's news feed.

Implement the Twitter class:

- **Twitter()** Initializes your twitter object.
- **void postTweet(int userId, int tweetId)**

Composes a new tweet with ID tweetId by the user userId. Each call to this function will be made with a unique tweetId.

- **List<Integer> getNewsFeed(int userId)**

Retrieves the 10 most recent tweet IDs in the user's news feed. Each item in the news feed must be posted by users who the user

followed or by the user themselves. Tweets must be ordered from most recent to least recent.

- `void follow(int followerId, int followeId)`

The user with ID `followerId` started following the user with ID `followeId`.

- `void unfollow(int followerId, int followeId)`

The user with ID `followerId` started unfollowing the user with ID `followeId`.

Example 1:

Input

```
["Twitter", "postTweet", "getNewsFeed", "follow", "postTweet", "getNewsFeed",  
"unfollow", "getNewsFeed"]
```

```
[[], [1, 5], [1], [1, 2], [2, 6], [1], [1, 2], [1]]
```

Output

```
[null, null, [5], null, null, [6, 5], null, [5]]
```

Explanation

```
Twitter twitter = new Twitter();
```

Constraints:

- $1 \leq \text{userId}, \text{followerId}, \text{followeId} \leq 500$
- $0 \leq \text{tweetId} \leq 10^4$
- All the tweets have unique IDs.
- At most $3 * 10^4$ calls will be made to `postTweet`, `getNewsFeed`, `follow`, and `unfollow`.
- A user cannot follow himself.

Community Solutions (Python)

• WalkCC

```
class Twitter:
    def __init__(self):
        self.timer = itertools.count(step=-1)
        self.tweets = collections.defaultdict(deque)
        self.followees = collections.defaultdict(set)

    def postTweet(self, userId: int, tweetId: int) -> None:
        self.tweets[userId].appendleft((next(self.timer), tweetId))
        if len(self.tweets[userId]) > 10:
            self.tweets[userId].pop()

    def getNewsFeed(self, userId: int) -> list[int]:
        tweets = list(
            heapq.merge(
                *
                (self.tweets[followee]
                 for followee in self.followees[userId] | {userId}))
        )
        return [tweetId for _, tweetId in tweets[:10]]

    def follow(self, followerId: int, followeeId: int) -> None:

        self.followees[followerId].add(followeeId)

    def unfollow(self, followerId: int, followeeId: int) -> None:
        self.followees[followerId].discard(followeeId)
```

• LeetDoocs

```
class Twitter:
    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.user_tweets = defaultdict(list)
        self.user_following = defaultdict(set)
        self.tweets = defaultdict()
        self.time = 0
```

```
def postTweet(self, userId: int, tweetId: int) -> None:
    """
    Compose a new tweet.
    """
    self.time += 1
    self.user_tweets[userId].append(tweetId)
    self.tweets[tweetId] = self.time

def getNewsFeed(self, userId: int) -> List[int]:
    """
```

Retrieve the 10 most recent tweet ids in the user's news feed. Each item in the news feed must be posted by users who the user followed or by the user herself. Tweets must be ordered from most recent to least recent.

```
    """
    following = self.user_following[userId]
    users = set(following)
    users.add(userId)
    tweets = [self.user_tweets[u][::-1][:10] for u in users]
    tweets = sum(tweets, [])
    return nlargest(10, tweets, key=lambda tweet: self.tweets[tweet])

def follow(self, followerId: int, followeeId: int) -> None:
```

```
    """
    Follower follows a followee. If the operation is invalid, it should be
    a no-op.
    """
    self.user_following[followerId].add(followeeId)

def unfollow(self, followerId: int, followeeId: int) -> None:
    """
    Follower unfollows a followee. If the operation is invalid, it should
    be a no-op.
    """
    following = self.user_following[followerId]
```

```

    if followeeId in following:
        following.remove(followeeId)

```

```

# Your Twitter object will be instantiated and called as such:
# obj = Twitter()
# obj.postTweet(userId,tweetId)
# param_2 = obj.getNewsFeed(userId)
# obj.follow(followerId,followeeId)
# obj.unfollow(followerId,followeeId)

```

• SimplyLeet

```

# Python implementation of the Twitter design problem

```

```

import heapq

```

```

class Twitter:

```

```

    def __init__(self):

```

```

        # Mapping from userId to a set of followeeIds

```

```

        self.follows = {}

```

```

        # Mapping from userId to list of (timestamp, tweetId)

```

```

        self.tweets = {}

```

```

        # Global timestamp to order tweets

```

```

        self.timestamp = 0

```

```

    def postTweet(self, userId: int, tweetId: int) -> None:

```

```

        # Ensure the user follows itself for easier news feed retrieval

```

```

        if userId not in self.follows:

```

```

            self.follows[userId] = set()

```

```

        self.follows[userId].add(userId)

```

```

        # Append the tweet with the current timestamp, then increment timestamp

```

```

        if userId not in self.tweets:

```



```

        self.tweets[userId] = []
        self.tweets[userId].append((self.timestamp, tweetId))
        self.timestamp += 1

def getNewsFeed(self, userId: int) -> list:
    # If the user does not exist in follows mapping, return empty feed
    if userId not in self.follows:
        return []

    # Use a heap (as a max-heap using negative timestamp)

    heap = []
    # Iterate over all followees of the user
    for followeeId in self.follows[userId]:
        if followeeId in self.tweets and self.tweets[followeeId]:
            # Start from the most recent tweet of the followee
            index = len(self.tweets[followeeId]) - 1
            ts, tid = self.tweets[followeeId][index]
            # Push the tweet along with followeeId and the index of the
            tweet in the list
            heapq.heappush(heap, (-ts, tid, followeeId, index))

    result = []
    # Retrieve up to 10 tweets
    while heap and len(result) < 10:
        neg_ts, tweetId, followeeId, idx = heapq.heappop(heap)
        result.append(tweetId)
        # If there is an older tweet from this followee, push it onto the
        heap

        if idx - 1 >= 0:
            ts, tid = self.tweets[followeeId][idx - 1]
            heapq.heappush(heap, (-ts, tid, followeeId, idx - 1))
    return result

```

```
def follow(self, followerId: int, followeeId: int) -> None:
    # A user should always follow itself
    if followerId not in self.follows:
        self.follows[followerId] = set()
    self.follows[followerId].add(followerId)
    # Add followeeId to follower's follow set if not self-follow
    if followerId != followeeId:
        self.follows[followerId].add(followeeId)

def unfollow(self, followerId: int, followeeId: int) -> None:
    # A user cannot unfollow itself
    if followerId in self.follows and followeeId != followerId:
        self.follows[followerId].discard(followeeId)
```

◆ Quick Review

Key Insights

- Use a global timestamp to order tweets as they are posted.
- Maintain a mapping of each user to the list of tweets they posted.
- Maintain a mapping of each user to the set of users they follow.
- For getNewsFeed, merge tweets from the user and their followees using a heap to efficiently retrieve the 10 most recent tweets.
- Ensure that a user always follows themselves to simplify retrieving their own tweets.

Space & Time

Time Complexity:

- postTweet: $O(1)$
- follow / unfollow: $O(1)$
- getNewsFeed: $O(N \log K)$ where N is the number of followees and K is the average number of tweets considered (capped at 10 tweets per merge iteration)

Space Complexity:

- $O(U + T)$ where U is the number of users (for follow mappings) and T is the total number of tweets stored.

Solution

We use two main data structures: a hash table to store tweets for each user and another hash table for follow relationships. Each tweet is stored as a tuple of (timestamp, tweetId). A global timestamp counter provides a unique ordering. A min-heap (or max-heap using negative timestamp values) is used to merge the tweets from different users efficiently.

When retrieving the news feed, we look up the tweet lists for the given user's followees (including self) and use the heap to extract the 10 most recent tweets.

Heap / Priority Queue

□ #621 Task Scheduler

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Greedy · Heap · Counting
Lists: Grind 169

Problem

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n . Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of at least n intervals between two tasks with the same label.

Return the minimum number of CPU intervals required to complete all tasks.

Example 1:

Input: tasks = ["A","A","A","B","B","B"], $n = 2$

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.

Example 2:

Input: tasks = ["A","C","A","B","D","B"], n = 1

Output: 6

Explanation: A possible sequence is: A -> B -> C -> D -> A -> B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: tasks = ["A","A","A", "B","B","B"], n = 3

Output: 10

Explanation: A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Constraints:

- $1 \leq \text{tasks.length} \leq 104$
- tasks[i] is an uppercase English letter.

- $0 \leq n \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        count = collections.Counter(tasks)
        maxFreq = max(count.values())
        # Put the most frequent task in the slot first.
        maxFreqTaskOccupy = (maxFreq - 1) * (n + 1)
        # Get the number of tasks with same frequency as maxFreq, we'll append them
        # after the
        # `maxFreqTaskOccupy`.
        nMaxFreq = sum(value == maxFreq for value in count.values())
        # max(

        # the most frequent task is frequent enough to force some idle slots,
        # the most frequent task is not frequent enough to force idle slots
        # )
        return max(maxFreqTaskOccupy + nMaxFreq, len(tasks))
```

• LeetDoocs

```
class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        cnt = Counter(tasks)
        x = max(cnt.values())
        s = sum(v == x for v in cnt.values())
        return max(len(tasks), (x - 1) * (n + 1) + s)
```

• SimplyLeet

```

# Python solution for Task Scheduler
from collections import Counter

def leastInterval(tasks, n):
    # Count the frequency of each task
    task_counts = Counter(tasks)
    # Find the maximum frequency among tasks
    max_freq = max(task_counts.values())
    # Count how many tasks have the maximum frequency
    max_count = sum(1 for count in task_counts.values() if count == max_freq)

    # Calculate the minimum intervals required based on the most frequent tasks
    part_count = max_freq - 1                # Number of parts between the
most frequent task
    part_length = n - (max_count - 1)         # Effective empty slots per
part after placing max frequency tasks
    empty_slots = part_count * part_length    # Total empty slots to fill
with idle or other tasks
    available_tasks = len(tasks) - max_freq * max_count # Remaining tasks to
fill the empty slots
    idles = max(0, empty_slots - available_tasks) # Remaining idle slots if
available tasks are not enough

    # The total intervals is the sum of all tasks and idle slots inserted
    return len(tasks) + idles

# Example usage:
if __name__ == "__main__":
    tasks1 = ["A","A","A","B","B","B"]
    n1 = 2
    print(leastInterval(tasks1, n1)) # Output: 8

```

• LC.ca

```

class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        cnt = Counter(tasks)
        x = max(cnt.values())
        s = sum(v == x for v in cnt.values())
        return max(len(tasks), (x - 1) * (n + 1) + s)

```

◆ Quick Review

Key Insights

- Count the frequency of each task.
- The task with the highest frequency determines the base structure.
- Identify how many tasks share the maximum frequency.
- Calculate the total time based on the formula: $\max(\text{total tasks}, (\text{max frequency} - 1) * (n + 1) + \text{count of tasks with maximum frequency})$.
- This approach avoids simulating the entire scheduling by using a greedy strategy.

Space & Time

Time Complexity: $O(N)$ where N is the number of tasks (counting frequency, then iterating over constant 26 letters).

Space Complexity: $O(1)$ since the frequency count uses an array or hash map with fixed size (26 letters).

Solution

We first count the frequency of each task. The task(s) with the maximum frequency establish

the minimum structure required. Compute the ideal idle slots based on the count of these maximum tasks and the cooling period n . Finally, compare this computed length with the total number of tasks to get the final answer. This greedy strategy avoids explicit simulation and leverages mathematical formulation.

Heap / Priority Queue

□ #973 K Closest Points to Origin

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Divide and Conquer · Sorting ·
Heap

Lists: Grind 169

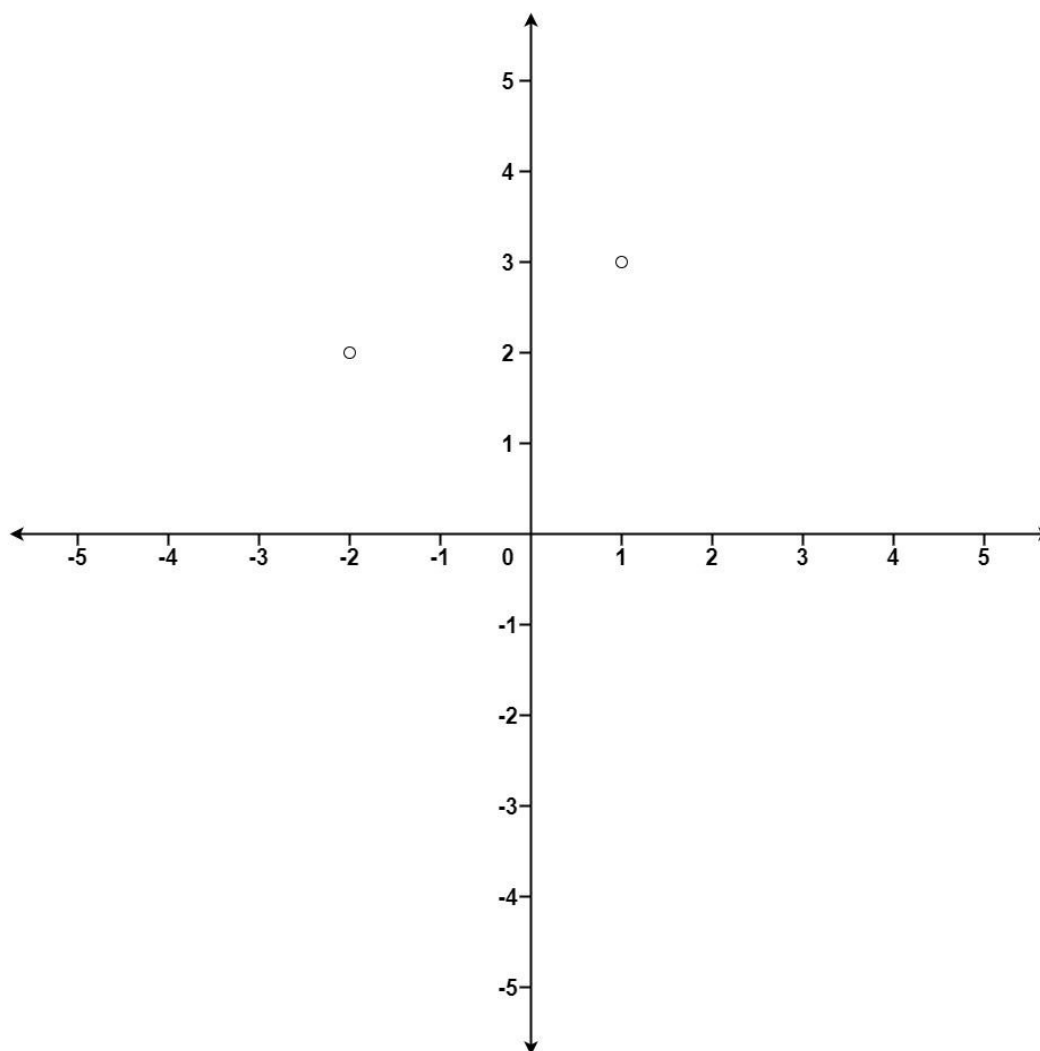
Problem

Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$.

The distance between two points on the X-Y plane is the Euclidean distance (i.e., $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$).

You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in).

Example 1:



Input: points = [[1,3],[-2,2]], k = 1

Output: [[-2,2]]

Explanation:

The distance between (1, 3) and the origin is $\sqrt{10}$.

The distance between (-2, 2) and the origin is $\sqrt{8}$.

Since $\sqrt{8} < \sqrt{10}$, (-2, 2) is closer to the origin.

We only want the closest k = 1 points from the origin, so the answer is just [[-2,2]].

Example 2:

Input: points = [[3,3],[5,-1],[-2,4]], k = 2

Output: [[3,3],[-2,4]]

Explanation: The answer [[-2,4],[3,3]] would also be accepted.

Constraints:

- $1 \leq k \leq \text{points.length} \leq 104$

- $-104 \leq x_i, y_i \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        maxHeap = []

        for x, y in points:
            heapq.heappush(maxHeap, (- x * x - y * y, [x, y]))
            if len(maxHeap) > k:
                heapq.heappop(maxHeap)

        return [pair[1] for pair in maxHeap]
```

```
class Solution:
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        def squareDist(p: list[int]) -> int:
            return p[0] * p[0] + p[1] * p[1]

        def quickSelect(l: int, r: int, k: int) -> None:
            pivot = points[r]

            nextSwapped = l
            for i in range(l, r):
                if squareDist(points[i]) <= squareDist(pivot):
                    points[nextSwapped], points[i] = points[i], points[nextSwapped]
                    nextSwapped += 1
            points[nextSwapped], points[r] = points[r], points[nextSwapped]

            count = nextSwapped - l + 1  # the number of points <= pivot
            if count == k:
                return
            if count > k:
                quickSelect(l, nextSwapped - 1, k)
```

```

        else:
            quickSelect(nextSwapped + 1, r, k - count)

    quickSelect(0, len(points) - 1, k)
    return points[0:k]

```

• LeetCode

```

class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        points.sort(key=lambda p: hypot(p[0], p[1]))
        return points[:k]

```

• SimplyLeet

```

# Python Solution
def kClosest(points, k):
    # Sort the points based on their squared Euclidean distance from the
    # origin.
    points.sort(key=lambda point: point[0]**2 + point[1]**2)
    # Return the first k points from the sorted list.
    return points[:k]

# Example usage:
# points = [[1,3], [-2,2]]
# k = 1

# print(kClosest(points, k))

```

• LC.ca

```

class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        points.sort(key=lambda p: p[0] * p[0] + p[1] * p[1])
        return points[:k]

```

◆ Quick Review

Key Insights

- Calculate the squared Euclidean distance ($x^2 + y^2$) to avoid the overhead of square root computation.
- Sorting the points based on their distance gives an $O(n \log n)$ solution.
- Alternatively, using a max-heap to keep track of the k closest points results in $O(n \log k)$ time, which is effective if k is much smaller than n .
- Using Quickselect can achieve an average time complexity of $O(n)$, though the worst-case is $O(n^2)$.
- The problem does not require the output to be sorted in any specific order.

Space & Time

Time Complexity: $O(n \log n)$ with a sorting approach, $O(n \log k)$ with a heap approach, or average $O(n)$ with a Quickselect approach.

Space Complexity: $O(n)$ if extra space is used for sorting, or $O(k)$ for the heap approach.

Solution

We can solve this problem by first computing the squared Euclidean distance for each point to avoid unnecessary square root operations.

Then, we can sort the points based on their computed distance and select the first k points. Alternatively, a max-heap or Quickselect algorithm can be used: Max-Heap: Maintain a heap of size k while iterating through the points, ensuring that only the k points with the smallest distances are kept. Quickselect: Partition the array like in QuickSort to position the k th closest point in its correct position. Once partitioned, all points to the left of that position will be the k closest. The sorting solution is straightforward and efficient enough given the constraints.

Tries

Round 5 · Mid · Medium · 2 questions

Tries

□ #208 Implement Trie (Prefix Tree)

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Design · Trie
Lists: Grind 169

Problem

A trie (pronounced as "try") or prefix tree is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- **Trie()** Initializes the trie object.
- **void insert(String word)** Inserts the string word into the trie.
- **boolean search(String word)** Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.
- **boolean startsWith(String prefix)** Returns true if there is a previously inserted string word that has the prefix prefix, and false

otherwise.

Example 1:

Input

```
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]  
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
```

Output

```
[null, null, true, false, true, null, true]
```

Explanation

```
Trie trie = new Trie();
```

Constraints:

- $1 \leq \text{word.length}, \text{prefix.length} \leq 2000$
- word and prefix consist only of lowercase English letters.
- At most $3 * 10^4$ calls in total will be made to insert, search, and startsWith.

Community Solutions (Python)

• WalkCC

```
class TrieNode:  
    def __init__(self):  
        self.children: dict[str, TrieNode] = {}  
        self.isWord = False  
  
class Trie:  
    def __init__(self):  
        self.root = TrieNode()
```

```
def insert(self, word: str) -> None:
    node: TrieNode = self.root
    for c in word:
        node = node.children.setdefault(c, TrieNode())
    node.isWord = True

def search(self, word: str) -> bool:
    node = self._find(word)
    return node is not None and node.isWord
```

```
def startsWith(self, prefix: str) -> bool:
    return self._find(prefix) is not None

def _find(self, prefix: str) -> TrieNode | None:
    node: TrieNode = self.root
    for c in prefix:
        if c not in node.children:
            return None
        node = node.children[c]
    return node
```

• LeetDoocs

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word: str) -> None:
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
```

```

        node.children[idx] = Trie()
        node = node.children[idx]
        node.is_end = True

    def search(self, word: str) -> bool:
        node = self._search_prefix(word)
        return node is not None and node.is_end

    def startsWith(self, prefix: str) -> bool:
        node = self._search_prefix(prefix)

```

```

        return node is not None

    def _search_prefix(self, prefix: str):
        node = self
        for c in prefix:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return None
            node = node.children[idx]
        return node

```

```

# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)

```

• SimplyLeet

```

class TrieNode:
    def __init__(self):
        # Dictionary to store child nodes and a flag for end of word.
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        # Initialize the Trie with the root node.
        self.root = TrieNode()

```

```
def insert(self, word: str) -> None:
    # Start from the root.
    node = self.root
    for char in word:
        # If character doesn't exist, add a new TrieNode.
        if char not in node.children:
            node.children[char] = TrieNode()
        node = node.children[char]
    # Mark the end of word.
```

```
node.is_end_of_word = True
```

```
def search(self, word: str) -> bool:
    # Traverse the Trie for each character.
    node = self.root
    for char in word:
        if char not in node.children:
            return False
        node = node.children[char]
    # Returns True if word ends exactly here.
```

```
return node.is_end_of_word
```

```
def startsWith(self, prefix: str) -> bool:
    # Traverse checking for the prefix.
    node = self.root
    for char in prefix:
        if char not in node.children:
            return False
        node = node.children[char]
    return True
```

• LC.ca

```
import collections
```

```
class TrieNode:
```

```
def __init__(self):
```

```
    """
```

Usually, a Python dictionary throws a `KeyError` if you try to get an item with a key that is not currently in the dictionary.

The `defaultdict` in contrast will simply create any items that you try to access

<https://stackoverflow.com/questions/5900578/how-does-collections-defaultdict-work>

```
    """
```

```
self.child = collections.defaultdict(TrieNode)
```

```
self.is_word = False
```

```
class Trie:
```

```
def __init__(self):
```

```
self.root = TrieNode()
```

```
def insert(self, word: str) -> None:
```

```
cur = self.root
```

```
for letter in word:
```

```
cur = cur.child[letter]
```

```
cur.is_word = True
```

```
def search(self, word: str) -> bool:
```

```
cur = self.root
```

```
for letter in word:
```

`cur = cur.child[letter]` ==> will not work, it will create a default node for this letter

```
cur = cur.child.get(letter)
```

```
if not cur:
    return False
return cur.is_word

def startsWith(self, prefix: str) -> bool:
    cur = self.root
    for letter in prefix:
        # cur = cur.child[letter] ==> will not work, it will creat a default node for
        # this letter
        cur = cur.child.get(letter)
        if not cur:
            return False
    return True

# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)

#####

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word: str) -> None:
        node = self
        for c in word:
```

```

        idx = ord(c) - ord('a')
        if node.children[idx] is None:
            node.children[idx] = Trie()
        node = node.children[idx]
    node.is_end = True

    def search(self, word: str) -> bool:
        node = self._search_prefix(word)
        return node is not None and node.is_end

```

```

    def startsWith(self, prefix: str) -> bool:
        node = self._search_prefix(prefix)
        return node is not None

```

```

    def _search_prefix(self, prefix: str):
        node = self
        for c in prefix:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return None

```

```

            node = node.children[idx]
        return node

```

```

# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)

```

```

# below is using reduce()
class Trie(object):

    def __init__(self):
        T = lambda: collections.defaultdict(T)
        self.root = T()

    def insert(self, word):

```



```
reduce(dict.__getitem__, word, self.root)['#'] = True

def search(self, word):
    return '#' in reduce(lambda cur, c: cur.get(c, {}), word, self.root)

def startsWith(self, prefix):
    return bool(reduce(lambda cur, c: cur.get(c, {}), prefix, self.root))
```

◆ Quick Review

Key Insights

- Use a tree-like structure where each node represents a character.
- Each node stores its children in a dictionary (or hash map) for $O(1)$ average time access.
- Mark the end of a word with a boolean flag.
- Insertion is done character by character, creating new nodes as needed.
- Searching and checking prefixes follow traversals down the tree.

Space & Time

Time Complexity:

Insert: $O(m)$ where m is the length of the word.

Search: $O(m)$

startsWith: $O(m)$

Space Complexity:

$O(N * M)$ in the worst-case scenario, where N is the number of words and M is the average length of the words, due to storing each character in the Trie.

Solution

We implement the Trie using a node-based approach. Each `TrieNode` contains a hash map (or dictionary) of its children and a boolean indicating if the node marks the end of a word. The Trie class uses these nodes to build out the words character by character. The `insert` method traverses and builds the necessary nodes; the `search` method verifies that all characters exist in order and that the final node marks the end of a word; the `startsWith` method checks if the prefix exists in the Trie. No tricky edge cases exist beyond handling null or empty inputs, as constraints guarantee valid lowercase strings.

Tries

□ #211 Design Add and Search Words Data Structure **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · DFS · Design · Trie
Lists: Grind 169

Problem

Design a data structure that supports adding new words and finding if a string matches any previously added string.

Implement the WordDictionary class:

- WordDictionary() Initializes the object.
- void addWord(word) Adds word to the data structure, it can be matched later.
- bool search(word) Returns true if there is any string in the data structure that matches word or false otherwise. word may contain dots '.' where dots can be matched with any letter.

Example:

Input

```
["WordDictionary", "addWord", "addWord", "addWord", "search", "search", "search", "search"]
```

```
[[], ["bad"], ["dad"], ["mad"], ["pad"], ["bad"], [".ad"], ["b.."]]
```

Output

```
[null, null, null, null, false, true, true, true]
```

Explanation

```
WordDictionary wordDictionary = new WordDictionary();
```

Constraints:

- $1 \leq \text{word.length} \leq 25$
- word in addWord consists of lowercase English letters.
- word in search consist of '.' or lowercase English letters.
- There will be at most 2 dots in word for search queries.
- At most 104 calls will be made to addWord and search.

Community Solutions (Python)

• WalkCC

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()
```

```

def addWord(self, word: str) -> None:
    node: TrieNode = self.root
    for c in word:
        node = node.children.setdefault(c, TrieNode())
    node.isWord = True

def search(self, word: str) -> bool:
    return self._dfs(word, 0, self.root)

def _dfs(self, word: str, s: int, node: TrieNode) -> bool:

    if s == len(word):
        return node.isWord
    if word[s] != '.':
        child: TrieNode = node.children.get(word[s], None)
        return self._dfs(word, s + 1, child) if child else False
    return any(self._dfs(word, s + 1, child) for child in node.children.values
())

```

• LeetDoocs

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

class WordDictionary:
    def __init__(self):
        self.trie = Trie()

    def addWord(self, word: str) -> None:
        node = self.trie
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, word: str) -> bool:

```

```
def search(word, node):
    for i in range(len(word)):
        c = word[i]
        idx = ord(c) - ord('a')
        if c != '.' and node.children[idx] is None:
            return False
        if c == '.':
            for child in node.children:
                if child is not None and search(word[i + 1 :], child):
                    return True
```

```
        return False
        node = node.children[idx]
    return node.is_end
```

```
return search(word, self.trie)
```

```
# Your WordDictionary object will be instantiated and called as such:
# obj = WordDictionary()
# obj.addWord(word)
```

```
# param_2 = obj.search(word)
```

• SimplyLeet

```
class TrieNode:
    def __init__(self):
        # Dictionary to store child nodes
        self.children = {}
        # Flag to indicate if a word ends here
        self.is_end = False

class WordDictionary:
    def __init__(self):
        # Initialize the Trie with an empty root node
```

```
self.root = TrieNode()

def addWord(self, word: str) -> None:
    # Start at the root of the Trie
    node = self.root
    for char in word:
        # Create a new node if the character is not present
        if char not in node.children:
            node.children[char] = TrieNode()
        node = node.children[char]

    # Mark the end of the word
    node.is_end = True

def search(self, word: str) -> bool:
    # DFS helper function
    def dfs(index, node):
        if index == len(word):
            return node.is_end
        char = word[index]
        if char == '.':
            # If wildcard, search all children nodes
            for child in node.children.values():
                if dfs(index + 1, child):
                    return True
            return False
        else:
            # If character not found, word does not exist
            if char not in node.children:
                return False
            return dfs(index + 1, node.children[char])

    return dfs(0, self.root)

# Example usage:
wd = WordDictionary()
wd.addWord("bad")
wd.addWord("dad")
wd.addWord("mad")
print(wd.search("pad")) # Output: False
print(wd.search("bad")) # Output: True
print(wd.search(".ad")) # Output: True
```

```
print(wd.search("b..")) # Output: True
```

• LC.ca

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

class WordDictionary:
    def __init__(self):
        self.trie = Trie()

    def addWord(self, word: str) -> None:
        node = self.trie
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, word: str) -> bool:

        def search(word, node):
            for i in range(len(word)):
                c = word[i]
                idx = ord(c) - ord('a')
                if c != '.' and node.children[idx] is None:
                    return False
                if c == '.':
                    for child in node.children:
                        if child is not None and search(word[i + 1 :], child):
                            return True
```



```
        return False
        node = node.children[idx]
        return node.is_end

    return search(word, self.trie)

# Your WordDictionary object will be instantiated and called as such:
# obj = WordDictionary()
# obj.addWord(word)

# param_2 = obj.search(word)
```

◆ Quick Review

Key Insights

- Utilize a Trie (prefix tree) to efficiently store words.
- Each node in the Trie contains a dictionary for its children and a flag indicating the end of a word.
- The search operation is implemented with a depth-first search (DFS) approach.
- When the search query contains a dot, explore all possible child nodes.

Space & Time

Time Complexity:

addWord: $O(w)$, where w is the length of the word.

search: Worst-case $O(26^w)$ for words with multiple wildcards, but practical performance is better given constraints.

Space Complexity: $O(n * w)$, where n is the number of words and w is the average word length.

Solution

The WordDictionary is implemented using a Trie. For adding, we insert each character of the word into the Trie. For searching, we use DFS. When encountering a wildcard '.', we recursively search all children nodes. This structure combined with DFS efficiently addresses the wildcard matching and ensures good performance for multiple operations.

Backtracking

Round 5 · Mid · Medium · 9 questions

Backtracking

□ #17 Letter Combinations of a Phone Number

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Backtracking
Lists: Grind 169

Problem

Given a string containing digits from 2–9 inclusive, return all possible letter combinations that the number could represent. Return the answer in any order.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.



Example 1:

Input: digits = "23"

Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Example 2:

Input: digits = "2"

Output: ["a","b","c"]

Constraints:

- $1 \leq \text{digits.length} \leq 4$
- $\text{digits}[i]$ is a digit in the range ['2', '9'].

Community Solutions (Python)

• WalkCC

```
class Solution:
    def letterCombinations(self, digits: str) -> list[str]:
        if not digits:
            return []

        digitToLetters = ['', '', 'abc', 'def', 'ghi',
                          'jkl', 'mno', 'pqrs', 'tuv', 'wxyz']

        ans = []

        def dfs(i: int, path: list[str]) -> None:

            if i == len(digits):
                ans.append(''.join(path))
                return

            for letter in digitToLetters[int(digits[i])]:
                path.append(letter)
                dfs(i + 1, path)
                path.pop()

        dfs(0, [])

        return ans
```

• LeetDoocs

```
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []
        d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"]
        ans = [""]
        for i in digits:
            s = d[int(i) - 2]
            ans = [a + b for a in ans for b in s]
        return ans
```

```

class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        def dfs(i: int):
            if i >= len(digits):
                ans.append("".join(t))
                return
            for c in d[int(digits[i]) - 2]:
                t.append(c)
                dfs(i + 1)
                t.pop()

        if not digits:
            return []
        d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"]
        ans = []
        t = []
        dfs(0)
        return ans

```

• SimplyLeet

Python solution for Letter Combinations of a Phone Number

```

def letterCombinations(digits):
    # Mapping of digits to corresponding letters
    digit_to_letters = {
        "2": "abc",
        "3": "def",
        "4": "ghi",
        "5": "jkl",
        "6": "mno",

        "7": "pqrs",
        "8": "tuv",
        "9": "wxyz"
    }

    # If input is empty, return an empty list
    if not digits:
        return []

    result = []

```

```

# Backtracking function to generate combinations
def backtrack(index, current_combination):
    # If the current combination length equals the digits length,
    # add the combination to the result list.
    if index == len(digits):
        result.append(current_combination)
        return

    # Get the corresponding letters for the current digit

    letters = digit_to_letters[digits[index]]
    # Explore all possible letter routes for the current digit.
    for letter in letters:
        backtrack(index + 1, current_combination + letter)

# Initiate backtracking starting from index 0 and empty combination.
backtrack(0, "")
return result

# Example usage:

#print(letterCombinations("23"))

```

• LC.ca

```

class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []
        d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"]
        ans = [""]
        for i in digits:
            s = d[int(i) - 2]
            ans = [a + b for a in ans for b in s]
        return ans

```

◆ Quick Review

Key Insights

- Use a mapping from digits to the respective letters as defined by a telephone keypad.
- Backtracking is an ideal approach to generate all possible combinations.
- Each digit adds a level in the recursive tree where you append one letter at a time.
- Handling edge cases such as an empty input string is crucial.

Space & Time

Time Complexity: $O(4^n * n)$ where n is the length of the input string (each digit may map to up to 4 letters and adding a combination takes $O(n)$ time).

Space Complexity: $O(n)$ for the recursion call stack, not counting the space required for storing the output.

Solution

We can solve the problem using a backtracking algorithm. First, we define a dictionary mapping each digit to its corresponding letters. Then we initiate a recursive function called `backtrack` that takes the current combination and the index of the next digit to process. At each call, if the combination is

complete (length equal to the input string), we add it to the result list. Otherwise, we iterate through the letters corresponding to the current digit, append a letter to the combination, and recursively call backtrack with the next index. This approach ensures that all possible letter combinations are generated.

Backtracking

□ #22 Generate Parentheses

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Backtracking
Lists: Grind 169

Problem

Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

Example 1:

Input: $n = 3$
Output: ["((()))", "(()())", "(())()", "()(())", "()()()"]

Example 2:

Input: $n = 1$
Output: ["()"]

Constraints:

- $1 \leq n \leq 8$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def generateParenthesis(self, n):
        ans = []

        def dfs(l: int, r: int, s: list[str]) -> None:
            if l == 0 and r == 0:
                ans.append(''.join(s))
            if l > 0:
                s.append('(')
                dfs(l - 1, r, s)

                s.pop()
            if l < r:
                s.append(')')
                dfs(l, r - 1, s)
                s.pop()

        dfs(n, n, [])
        return ans

```

• LeetCode

```

class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
        def dfs(l: int, r: int, t: str):
            if l > n or r > n or l < r:
                return
            if l == n and r == n:
                ans.append(t)
                return
            dfs(l + 1, r, t + "(")
            dfs(l, r + 1, t + ")")

        ans = []
        dfs(0, 0, "")
        return ans

```

• SimplyLeet

```
def generateParenthesis(n):
    # List to store valid combinations
    result = []

    # Helper function for backtracking
    def backtrack(current, open_count, close_count):
        # If current string reaches the maximum length, it's a valid
        # combination
        if len(current) == 2 * n:
            result.append(current)
            return

        # If we can add an opening parenthesis, do it
        if open_count < n:
            backtrack(current + '(', open_count + 1, close_count)
        # If adding a closing parenthesis maintains the validity, do it
        if close_count < open_count:
            backtrack(current + ')', open_count, close_count + 1)

    # Start the recursion with an empty string and zero counts for both
    backtrack("", 0, 0)
    return result

# Example usage:
print(generateParenthesis(3))
```

• LC.ca

```
class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
        def dfs(l, r, t):
            if l > n or r > n or l < r:
                return
            if l == n and r == n:
                ans.append(t)
                return
            dfs(l + 1, r, t + '(')
            dfs(l, r + 1, t + ')')
```

```

        ans = []
        dfs(0, 0, '')
        return ans

#####

class Solution(object):
    def generateParenthesis(self, n):
        #####

        :type n: int
        :rtype: List[str]
        #####

        def dfs(left, path, res, n):
            if len(path) == 2 * n:
                if left == 0:
                    res.append("".join(path))
                    return

            if left < n:
                path.append("(")
                dfs(left + 1, path, res, n)
                path.pop()
            if left > 0:
                path.append(")")
                dfs(left - 1, path, res, n)
                path.pop()

        res = []

        dfs(0, [], res, n)
        return res

```

◆ Quick Review

Key Insights

- Use a backtracking approach to build the valid combinations step by step.
- At any recursion step, you can add an opening parenthesis if you still have one available.
- You can add a closing parenthesis only if it would not result in an invalid sequence (i.e., the number of closing parentheses is less than the number of opening ones).
- The solution leverages recursion to explore all potential valid sequences while pruning invalid paths early.

Space & Time

Time Complexity: $O(4^n / (n^{3/2}))$

Space Complexity: $O(n)$ for the recursion stack, plus space for storing all valid combinations.

Solution

The solution uses a recursive backtracking technique to generate the combinations. The algorithm maintains counters for the number of '(' and ')' added to the current string. It recursively adds a '(' if the count is less than n and adds a ')' if doing so does not make the string invalid (i.e., if the count of ')' is less than

the count of '('). When a valid combination reaches a length of $2*n$, it is added to the result list.

Backtracking

□ #39 Combination Sum

MED

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking**

Lists: Grind 169

Problem

Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order.

The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1:

Input: candidates = [2,3,6,7], target = 7

Output: [[2,2,3],[7]]

Explanation:

2 and 3 are candidates, and $2 + 2 + 3 = 7$. Note that 2 can be used multiple times.

7 is a candidate, and $7 = 7$.

These are the only two combinations.

Example 2:

Input: candidates = [2,3,5], target = 8

Output: [[2,2,2,2],[2,3,3],[3,5]]

Example 3:

Input: candidates = [2], target = 1

Output: []

Constraints:

- $1 \leq \text{candidates.length} \leq 30$
- $2 \leq \text{candidates}[i] \leq 40$
- All elements of candidates are distinct.
- $1 \leq \text{target} \leq 40$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def combinationSum(self, candidates: list[int],
                      target: int) -> list[list[int]]:
        ans = []

        def dfs(s: int, target: int, path: list[int]) -> None:
            if target < 0:
                return
            if target == 0:
                ans.append(path.clone())
```

```

        return

    for i in range(s, len(candidates)):
        path.append(candidates[i])
        dfs(i, target - candidates[i], path)
        path.pop()

    candidates.sort()
    dfs(0, target, [])
    return ans

```

• LeetCode

```

class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                ans.append(t[:])
                return
            if s < candidates[i]:
                return
            for j in range(i, len(candidates)):
                t.append(candidates[j])

                dfs(j, s - candidates[j])
                t.pop()

        candidates.sort()
        t = []
        ans = []
        dfs(0, target)
        return ans

```

• SimplyLeet

```

# Function to find all unique combinations that sum to the target
def combinationSum(candidates, target):
    # List to store all valid combinations
    result = []
    # Sort candidates to facilitate pruning of the search
    candidates.sort()

    # Recursive helper function for backtracking
    def backtrack(remaining, start, path):
        # If remaining is 0, path is a valid combination so add a copy of it to
        result

        if remaining == 0:
            result.append(list(path))
            return

        # Loop through candidates starting from the current index
        for i in range(start, len(candidates)):
            # If candidate is greater than remaining, break out for efficiency
            if candidates[i] > remaining:
                break

            # Include candidate in the current combination path
            path.append(candidates[i])

            # Recursively call backtrack with updated remaining and same index
            # (allowing reuse)
            backtrack(remaining - candidates[i], i, path)

            # Backtrack by removing the candidate added in this iteration
            path.pop()

        # Start backtracking from index 0 with an empty path
        backtrack(target, 0, [])
    return result

# Example usage

print(combinationSum([2,3,6,7], 7))

```

• LC.ca

```

class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[
int]]:
        # i: start index for this recursion
        # s: sum
        def dfs(i, s):
            if s == target:
                ans.append(t.copy())
                return
            if s > target:
                return

            for j in range(i, len(candidates)):
                c = candidates[j]
                t.append(c)
                dfs(j, s + c)
                t.pop()

        ans = []
        t = []
        # candidates.sort() # diff from combinationSum-II, no need sorting it
        dfs(0, 0)

        return ans

#####

# dp version
class Solution_dp:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[
int]]:
        # for each-target (from 1 to target), its dp[i][j]
        # => so 3-D array dp[][][]
        dp = []

```

```

candidates.sort()

for i in range(1, target+1):
    cur = []
    for j in range(len(candidates)):
        if candidates[j] > i:
            break
        if candidates[j] == i:
            one = [candidates[j]]
            cur.append(one)

            break
        for a in dp[i - candidates[j] - 1]:
            if candidates[j] > a[0]:
                continue
            deepCopied = a.copy()
            deepCopied.insert(0, candidates[j])
            cur.append(deepCopied)
        dp.append(cur)

return dp[-1]

```

```
#####
```

```

class Solution(object):
    def combinationSum(self, candidates, target):
        """
        :type candidates: List[int]
        :type target: int
        :rtype: List[List[int]]
        """

    def dfs(candidates, start, target, path, res):
        if target == 0:
            return res.append(path + [])

        for i in range(start, len(candidates)):
            if target - candidates[i] >= 0:
                path.append(candidates[i])
                dfs(candidates, i, target - candidates[i], path, res)

```

```

        path.pop()

    res = []
    dfs(candidates, 0, target, [], res)
    return res

#####

class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[
int]]:

        def dfs(i: int, s: int):
            if s == 0:
                ans.append(t[:])
                return
            if i >= len(candidates) or s < candidates[i]:
                return
            dfs(i + 1, s)
            t.append(candidates[i])
            dfs(i, s - candidates[i])
            t.pop()

        candidates.sort()
        t = []
        ans = []
        dfs(0, target)
        return ans

```

◆ Quick Review

Key Insights

- Backtracking is a natural fit to explore all potential combinations.
- Sorting candidates simplifies the process by allowing early termination if the current

candidate exceeds the remaining target.

- Recursion is used to build combinations and backtrack once the current path exceeds or meets the target.
- The same candidate can be reused in the combination, so the recursive call does not advance the index.

Space & Time

Time Complexity:

$O(2^{(\text{target}/\min(\text{candidates}))})$ in the worst case where many combinations are possible.

Space Complexity: $O(\text{target}/\min(\text{candidates}))$ for the recursion stack and temporary space used to store combinations.

Solution

The solution uses a backtracking approach. First, sort the candidates to enable pruning of recursive calls once the candidate exceeds the remaining target. A recursive helper function builds up a combination, subtracting the candidate value from the target until it reaches zero (a valid combination) or becomes negative (an invalid path). The recursion allows the same candidate to be chosen again.

Each valid combination is then added to the result list.

Backtracking

□ #40 Combination Sum II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking
Lists: NeetCode 150

Problem

Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target.

Each number in candidates may only be used once in the combination.

Note: The solution set must not contain duplicate combinations.

Example 1:

```
Input: candidates = [10,1,2,7,6,1,5], target = 8
Output:
[
  [1,1,6],
  [1,2,5],
  [1,7],
  [2,6]
]
```

Example 2:

Input: candidates = [2,5,2,1,2], target = 5

Output:

```
[
  [1,2,2],
  [5]
]
```

Constraints:

- $1 \leq \text{candidates.length} \leq 100$
- $1 \leq \text{candidates}[i] \leq 50$
- $1 \leq \text{target} \leq 30$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def combinationSum2(self, candidates: list[int],
                       target: int) -> list[list[int]]:

        ans = []

        def dfs(s: int, target: int, path: list[int]) -> None:
            if target < 0:
                return
            if target == 0:
                ans.append(path.copy())

            return

        for i in range(s, len(candidates)):
            if i > s and candidates[i] == candidates[i - 1]:
                continue
            path.append(candidates[i])
            dfs(i + 1, target - candidates[i], path)
            path.pop()

        candidates.sort()

        dfs(0, target, [])
        return ans
```

• LeetCode

```
class Solution:
    def combinationSum2(self, candidates: List[int], target: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                ans.append(t[:])
                return
            if i >= len(candidates) or s < candidates[i]:
                return
            for j in range(i, len(candidates)):
                if j > i and candidates[j] == candidates[j - 1]:
                    continue
                t.append(candidates[j])
                dfs(j + 1, s - candidates[j])
                t.pop()

        candidates.sort()
        ans = []
        t = []
        dfs(0, target)
        return ans
```

• SimplyLeet

```
# Python Solution for Combination Sum II

def combinationSum2(candidates, target):
    # Sort candidates to handle duplicates easily.
    candidates.sort()
    results = []

    def backtrack(start, current_combination, current_sum):
        # If the current sum equals target, add a copy of the current
        # combination to results.
        if current_sum == target:
```

```
        results.append(list(current_combination))
        return
    # Iterate over possible candidates starting from index 'start'.
    for i in range(start, len(candidates)):
        # Skip duplicate elements.
        if i > start and candidates[i] == candidates[i - 1]:
            continue
        new_sum = current_sum + candidates[i]
        # If the new sum exceeds target, break the loop.
        if new_sum > target:
            break
        # Include the candidate and recurse.
        current_combination.append(candidates[i])
        backtrack(i + 1, current_combination, new_sum)
        # Backtrack: remove the last element added.
        current_combination.pop()

    backtrack(0, [], 0)
    return results

# Example usage:
print(combinationSum2([10,1,2,7,6,1,5], 8))
```

◆ Quick Review

Key Insights

- Sort the candidates first to easily skip duplicates.
- Use backtracking to explore all possible combinations.
- Avoid selecting the same number (at the same recursive level) to avoid duplicate combinations.

- Terminate recursion when the running sum exceeds the target.
- Use indices rather than reusing elements to ensure each number is used at most once.

Space & Time

Time Complexity: $O(2^n * k)$ in the worst case, where n is the number of candidates and k is the average length of a combination.

Space Complexity: $O(k)$ for the recursion stack and current combination (not including the space occupied by the output).

Solution

The solution is based on a backtracking algorithm. First, the candidates are sorted to handle duplicates easily. Then, a recursive function is used to build combinations. At each recursion level, iterate through the candidates starting from the current index. For each candidate:

- Skip the candidate if it is the same as the previous one (to avoid duplicates).
- If the candidate can be added without exceeding the target, add it to the current combination.
- Recurse with the next index (ensuring each

candidate is used only once). – Backtrack by removing the candidate from the current combination once completed.

Data structures used include:

- An array or list to hold the current combination.
- A results list to store all valid combinations.
- Sorting is leveraged to simplify duplicate elimination.

Backtracking

□ #46 Permutations

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking
Lists: Grind 169

Problem

Given an array `nums` of distinct integers, return all the possible permutations. You can return the answer in any order.

Example 1:

```
Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Example 2:

```
Input: nums = [0,1]
Output: [[0,1],[1,0]]
```

Example 3:

```
Input: nums = [1]
Output: [[1]]
```

Constraints:

- $1 \leq \text{nums.length} \leq 6$
- $-10 \leq \text{nums}[i] \leq 10$
- All the integers of `nums` are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def permute(self, nums: list[int]) -> list[list[int]]:
        ans = []
        used = [False] * len(nums)

        def dfs(path: list[int]) -> None:
            if len(path) == len(nums):
                ans.append(path.copy())
                return

            for i, num in enumerate(nums):
                if used[i]:
                    continue
                used[i] = True
                path.append(num)
                dfs(path)
                path.pop()
                used[i] = False

        dfs([])

        return ans
```

• LeetDoocs

```
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i >= n:
                ans.append(t[:])
                return
            for j, x in enumerate(nums):
                if not vis[j]:
                    vis[j] = True
                    t[i] = x
```

```

        dfs(i + 1)
        vis[j] = False

    n = len(nums)
    vis = [False] * n
    t = [0] * n
    ans = []
    dfs(0)
    return ans

```

• SimplyLeet

```

def permute(nums):
    result = []

    def backtrack(current, remaining):
        # If no remaining elements, current permutation is complete.
        if not remaining:
            result.append(current.copy())
            return
        # Explore each candidate number.
        for i in range(len(remaining)):

            # Add the chosen number to the current permutation.
            current.append(remaining[i])
            # Recursively generate permutations with the remaining numbers.
            backtrack(current, remaining[:i] + remaining[i+1:])
            # Backtrack by removing the chosen number.
            current.pop()

    backtrack([], nums)
    return result

# Example usage:
print(permute([1,2,3]))

```

• LC.ca

```
'''
remove an element by its value from a set
```

```
>>> my_set = {1, 2, 3, 4, 5}
>>> my_set.remove(3)
>>> print(my_set)
{1, 2, 4, 5}
```

```
-----
```

cannot remove an element by index from a set in Python3
need to convert the set to a list

```
>>> my_set = {1, 2, 3, 4, 5}
>>> my_list = list(my_set)
>>> del my_list[2] # remove the element at index 2
>>> my_set = set(my_list)
>>> print(my_set)
{1, 2, 4, 5}
'''
```

```
class Solution: # iterative
    def permute(self, nums: List[int]) -> List[List[int]]:
        res = [[]]

        if nums is None or len(nums) == 0:
            return ans

        for num in nums:
            new_res = []

            for perm in res:
                for i in range(len(perm) + 1):
                    new_perm = perm[:i] + [num] + perm[i:]
                    new_res.append(new_perm)

            res = new_res

        return res
```

```
#####
```

```

class Solution: # iterative, single_perm.insert(index, num)
    def permute(self, nums: List[int]) -> List[List[int]]:
        ans = []

        if nums is None or len(nums) == 0:
            return ans

        for num in nums:
            tmp_list = []

            for single_perm in ans:
                for index in range(len(single_perm) + 1):
                    single_perm.insert(index, num)
                    tmp_list.append(single_perm.copy())
                    single_perm.pop(index)

            ans = tmp_list

        return ans

```

```
#####
```

```
'''
```

In Python 3, both the `discard()` and `remove()` methods of a set object are used to remove an element from the set, but there is one key difference:

* `discard()` removes the specified element from the set if it is present, but does nothing if the element is not present.

* `remove()` removes the specified element from the set if it is present, but raises a `KeyError` exception if the element is not present.

```
>>> a
{33, 66, 11, 44, 22, 55}
>>> a.discard(22)
>>> a
{33, 66, 11, 44, 55}

>>> a.discard(200)
>>>
>>> a.remove(200)
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
KeyError: 200
'''
```

```
'''

>>> v1 = set([])
>>> v2 = set()
>>>
>>> v1
set()
```

```
>>> v2
set()
'''

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        res = []
        visited = set([])

        def dfs(nums, path, res, visited):
            if len(path) == len(nums):
```

```

        res.append(path + [])
        return

    for i in range(0, len(nums)):
        if i not in visited:
            visited.add(i)
            path.append(nums[i])
            dfs(nums, path, res, visited)
            path.pop()
            visited.discard(i) # remove(i) will throw exception if i
not existing

```

```

    dfs(nums, [], res, visited)
    return res

#####

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        def dfs(i):
            if i == n:

```

```

                ans.append(t[:])
                return
            for j in range(n):
                if not vis[j]:
                    vis[j] = True
                    t[i] = nums[j]
                    dfs(i + 1)
                    vis[j] = False

```

```

    n = len(nums)

```

```

    vis = [False] * n
    t = [0] * n
    ans = []
    dfs(0)
    return ans

```

◆ Quick Review

Key Insights

- Use backtracking to generate all possible permutations.
- Build each permutation by selecting numbers one at a time.
- When the current permutation reaches the length of nums, add it to the result.
- Backtracking allows you to undo choices and explore different orderings.

Space & Time

Time Complexity: $O(n * n!)$ – There are $n!$ permutations, and constructing each permutation takes $O(n)$ time.

Space Complexity: $O(n)$ – Recursion depth is limited to n , plus additional space for the current permutation.

Solution

The solution employs a backtracking algorithm. A helper function is used to build permutations recursively. At each step, the algorithm iterates through the available numbers, adds one to the current permutation if it has not been used yet, and recurses to handle the remaining numbers. Once the

current permutation matches the length of the input array, it is added to the result list. Then, by "backtracking" (removing the last number added), the algorithm explores alternative possibilities. This systematic exploration ensures that all possible permutations are generated.

Backtracking

□ #78 Subsets

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking · Bit Manipulation
Lists: Grind 169

Problem

Given an integer array `nums` of unique elements, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

Input: `nums = [1,2,3]`

Output: `[[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]`

Example 2:

Input: `nums = [0]`

Output: `[[], [0]]`

Constraints:

- $1 \leq \text{nums.length} \leq 10$
- $-10 \leq \text{nums}[i] \leq 10$
- All the numbers of `nums` are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def subsets(self, nums: list[int]) -> list[list[int]]:
        ans = []

        def dfs(s: int, path: list[int]) -> None:
            ans.append(path)

            for i in range(s, len(nums)):
                dfs(i + 1, path + [nums[i]])

        dfs(0, [])
        return ans
```

• LeetCode

```
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == len(nums):
                ans.append(t[:])
                return
            dfs(i + 1)
            t.append(nums[i])
            dfs(i + 1)
            t.pop()

        ans = []
        t = []
        dfs(0)
        return ans
```

• SimplyLeet

```
# Define the function to generate all subsets using backtracking.
def subsets(nums):
    # This list will store all the subsets
    result = []

    # Recursive helper function to generate subsets.
    def backtrack(start, current):
        # Append a copy of the current subset to the final result.
        result.append(current.copy())
        # Iterate over the possible next elements starting from index 'start'

        for i in range(start, len(nums)):
            # Include nums[i] in the current subset.
            current.append(nums[i])
            # Recurse with the next starting index.
            backtrack(i + 1, current)
            # Backtrack: remove the last element to explore other
            # possibilities.
            current.pop()

    # Start the recursion with an empty path.
    backtrack(0, [])

    return result

# Example usage:
print(subsets([1, 2, 3]))
```

• LC.ca

```
from typing import List

class Solution: # bfs
    def subsets(self, nums: List[int]) -> List[List[int]]:
        if not nums:
            return [[]]

        nums.sort() # sort() not necessary if no duplicates
        result = [[]]
```

```
for num in nums:
    result += [subset + [num] for subset in result]

return result

class Solution: # dfs
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(u, t):
            ans.append(t[:]) # or, t.copy()

            for i in range(u, len(nums)):
                t.append(nums[i])
                dfs(i + 1, t)
                t.pop()

        ans = []
        nums.sort() # sort() not necessary if no duplicates
        dfs(0, [])
        return ans

class Solution: # dfs, just pass down the final path
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(nums, index, path, ans):
            ans.append(path)
            [dfs(nums, i + 1, path + [nums[i]], ans) for i in range(index, len
(nums))]

        ans = []
        dfs(nums, 0, [], ans)
        return ans
```

```
class Solution: # dfs, but running slower, since need to reference from parent
method for 'nums' and 'ans'
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(index, path):
            ans.append(path)
            [dfs(i + 1, path + [nums[i]]) for i in range(index, len(nums))]

        ans = []
        dfs(0, [])

        return ans
```

◆ Quick Review

Key Insights

- The problem is equivalent to generating the power set of the given set.
- A recursive backtracking approach can be used to explore both including and excluding each element.
- Iterative (bit manipulation) solutions are also possible since there are 2^n subsets in total.
- The maximum array length is small (up to 10), making exponential time solutions feasible.

Space & Time

Time Complexity: $O(2^n * n)$ – There are 2^n subsets and creating each subset may take up

to $O(n)$ time.

Space Complexity: $O(2^n * n)$ – Space for storing all subsets. The recursion stack has a maximum depth of $O(n)$.

Solution

We solve the problem using backtracking, where we decide for each element whether to include it in the current subset or not. We recursively build all subsets starting at an empty list and at each step appending the current subset to our result list. We can also use iterative bit manipulation but here we focus on the backtracking approach. This algorithm leverages recursion and a list to maintain the current state, leading to a clear and concise implementation.

Backtracking

□ #79 Word Search

MED

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Backtracking · Matrix
Lists: Grind 169**

Problem

Given an $m \times n$ grid of characters board and a string word, return true if word exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"]], word = "ABCCED"
Output: true

Example 2:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"]], word = "SEE"
Output: true

Example 3:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCB"

Output: false

Constraints:

- $m == \text{board.length}$
- $n = \text{board}[i].\text{length}$
- $1 \leq m, n \leq 6$
- $1 \leq \text{word.length} \leq 15$
- board and word consists of only lowercase and uppercase English letters.

Follow up: Could you use search pruning to make your solution faster with a larger board?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def exist(self, board: list[list[str]], word: str) -> bool:
        m = len(board)
        n = len(board[0])

        def dfs(i: int, j: int, s: int) -> bool:
            if i < 0 or i == m or j < 0 or j == n:
                return False
            if board[i][j] != word[s] or board[i][j] == '*':
                return False

            if s == len(word) - 1:
                return True

            cache = board[i][j]
            board[i][j] = '*'
            isExist = (dfs(i + 1, j, s + 1) or
                      dfs(i - 1, j, s + 1) or
                      dfs(i, j + 1, s + 1) or
                      dfs(i, j - 1, s + 1))
            board[i][j] = cache

            return isExist

        return any(dfs(i, j, 0)
                   for i in range(m)
                   for j in range(n))
```

• LeetDoocs

```

class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        def dfs(i: int, j: int, k: int) -> bool:
            if k == len(word) - 1:
                return board[i][j] == word[k]
            if board[i][j] != word[k]:
                return False
            c = board[i][j]
            board[i][j] = "0"
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                ok = 0 <= x < m and 0 <= y < n and board[x][y] != "0"
                if ok and dfs(x, y, k + 1):
                    return True
            board[i][j] = c
            return False

        m, n = len(board), len(board[0])
        return any(dfs(i, j, 0) for i in range(m) for j in range(n))

```

• SimplyLeet

```

# Define the function to solve the Word Search problem.
def exist(board, word):
    # Get dimensions of the board.
    rows, cols = len(board), len(board[0])

    # Helper function for DFS search.
    def dfs(r, c, index):
        # If the entire word is found, return True.
        if index == len(word):
            return True

```

```

        # Check boundaries and if current cell already used or not matching.
        if r < 0 or c < 0 or r >= rows or c >= cols or board[r][c] != word[ind
ex]:
            return False

        # Temporarily mark the cell as visited.
        temp = board[r][c]
        board[r][c] = "#"

        # Explore all four directions.

        found = (dfs(r + 1, c, index + 1) or
                  dfs(r - 1, c, index + 1) or
                  dfs(r, c + 1, index + 1) or
                  dfs(r, c - 1, index + 1))

        # Backtrack: restore the original cell value.
        board[r][c] = temp
        return found

# Initiate DFS from each cell in the grid.

for i in range(rows):
    for j in range(cols):
        if dfs(i, j, 0):
            return True
    return False

# Example test case
if __name__ == "__main__":
    board = [
        ["A", "B", "C", "E"],
        ["S", "F", "C", "S"],
        ["A", "D", "E", "E"]
    ]
    word = "ABCCED"
    print(exist(board, word)) # Expected output: True

```

 • LC.ca

```

class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        def dfs(i, j, cur): # cur: current char count
            if cur == len(word):
                return True
            if (
                i < 0
                or i >= m
                or j < 0
                or j >= n

                or board[i][j] == '0'
                or word[cur] != board[i][j]
            ):
                return False
            t = board[i][j]
            board[i][j] = '0' # mark as visited
            for a, b in [[0, 1], [0, -1], [-1, 0], [1, 0]]:
                x, y = i + a, j + b
                if dfs(x, y, cur + 1):
                    return True

            board[i][j] = t
            return False

        m, n = len(board), len(board[0])
        return any(dfs(i, j, 0) for i in range(m) for j in range(n))

```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to explore each cell in the board as a potential starting point.
- Employ backtracking to mark cells as visited when exploring a path and revert them back when backtracking.

- Check boundary conditions and ensure the current cell's character matches the corresponding character in the word.
- Given the small grid and word sizes, a recursive solution is both feasible and straightforward.
- Search pruning can be applied by stopping a DFS branch as soon as the current path does not match the prefix of the target word.

Space & Time

Time Complexity: $O(m * n * 3^L)$, where L is the length of the word (each DFS may branch to at most 3 further cells, excluding the coming cell).

Space Complexity: $O(L)$ due to recursion call stack (where L is the word length).

Solution

The solution uses DFS with backtracking to traverse the grid. For each cell, we: Check if the cell matches the first character of the word. Recursively explore the adjacent cells (up, down, left, right) while marking the current cell as visited. If the path does not lead to a complete match, backtrack by resetting

the visited cell. The DFS stops if all characters in the word have been found in sequence. The algorithm leverages in-place modifications (or an auxiliary visited array) to avoid revisiting cells while exploring a candidate path.

Backtracking

□ #90 Subsets II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking · Bit Manipulation
Lists: Denny Zhang

Problem

Given an integer array `nums` that may contain duplicates, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

```
Input: nums = [1,2,2]
Output: [[],[1],[1,2],[1,2,2],[2],[2,2]]
```

Example 2:

```
Input: nums = [0]
Output: [[],[0]]
```

Constraints:

- $1 \leq \text{nums.length} \leq 10$
- $-10 \leq \text{nums}[i] \leq 10$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def subsetsWithDup(self, nums: list[int]) -> list[list[int]]:
        ans = []

        def dfs(s: int, path: list[int]) -> None:
            ans.append(path)
            if s == len(nums):
                return

            for i in range(s, len(nums)):

                if i > s and nums[i] == nums[i - 1]:
                    continue
                dfs(i + 1, path + [nums[i]])

        nums.sort()
        dfs(0, [])
        return ans
```

• LeetCode

```
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == len(nums):
                ans.append(t[:])
                return
            t.append(nums[i])
            dfs(i + 1)
            x = t.pop()
            while i + 1 < len(nums) and nums[i + 1] == x:
                i += 1
            dfs(i + 1)

        nums.sort()
        ans = []
        t = []
        dfs(0)
        return ans
```

• SimplyLeet

```
def subsetsWithDup(nums):
    # Sort the numbers to handle duplicates
    nums.sort()
    result = [] # To store all subsets
    subset = [] # Current subset being built

    def backtrack(start):
        # Append a copy of the current subset to the result
        result.append(list(subset))
        # Iterate over possible elements to include

        for i in range(start, len(nums)):
            # If the current element is a duplicate and not at the start of
            # this loop, skip it
            if i > start and nums[i] == nums[i - 1]:
                continue
            # Include current element into the subset
            subset.append(nums[i])
            # Recurse with next start index
            backtrack(i + 1)
            # Backtrack and remove the last element added
            subset.pop()

    backtrack(0)
    return result

# Example usage:
print(subsetsWithDup([1, 2, 2]))
```

• LC.ca

```

class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        def dfs(u, t):
            ans.append(t[:]) # or, t.copy()
            for i in range(u, len(nums)):
                # also good for [1,5,5], second '5' will be skipped here if
                # but second '5' is always covered, because i==u when
                ans=[1,5] and i=2
                if i != u and nums[i] == nums[i - 1]:
                    continue
                t.append(nums[i])

                dfs(i + 1, t)
                t.pop()

        ans = []
        nums.sort()
        dfs(0, [])
        return ans

# iteration

```

```

class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        if not nums:
            return [[]]

        nums.sort() # Sorting is necessary to handle duplicates.
        result = [[]]
        start_idx = 0 # Start index for the new subsets
        new_subsets_size = 0

```

```

    for i in range(len(nums)):
        size = len(result)
        # If the current number is the same as the previous one,
        # we only want to extend the subsets added in the previous
iteration.
        if i > 0 and nums[i] == nums[i - 1]:
            start_idx = size - new_subsets_size
        else:
            start_idx = 0

        new_subsets_size = 0 # Reset the size of newly created subsets

        for j in range(start_idx, size):
            current_subset = result[j] + [nums[i]]
            result.append(current_subset)
            new_subsets_size += 1

    return result

#####

```

```

class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        n = len(nums)
        ans = []
        for mask in range(1 << n):
            ok = True
            t = []
            for i in range(n):
                if mask >> i & 1:

                    if i and (mask >> (i - 1) & 1) == 0 and nums[i] == nums[i
- 1]:
                        ok = False
                        break
                    t.append(nums[i])
            if ok:
                ans.append(t)
        return ans

```

◆ Quick Review

Key Insights

- Sort the array first so that duplicate elements are adjacent.
- Use backtracking to generate all possible subsets.
- Skip duplicate elements in the recursion to avoid forming duplicate subsets.
- The use of sorted order and conditionally skipping elements is the trick to handle duplicates.

Space & Time

Time Complexity: $O(2^n * n)$ since each subset creation may take $O(n)$ in the worst case.

Space Complexity: $O(2^n * n)$ for storing the subsets, plus $O(n)$ for recursion stack.

Solution

The solution uses a backtracking technique that builds subsets incrementally. The input array is first sorted to bring duplicates together. During recursion, if the current element is the same as the previous one and it has not been used in the current recursion

path, it is skipped to prevent duplicate subsets. This ensures that each subset is unique. The algorithm stores each valid subset by copying the current subset state into the result list.

Backtracking

□ #131 Palindrome Partitioning

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Backtracking
Lists: NeetCode 150

Problem

Given a string s , partition s such that every substring of the partition is a palindrome.
Return all possible palindrome partitioning of s .

Example 1:

```
Input: s = "aab"  
Output: [["a","a","b"],["aa","b"]]
```

Example 2:

```
Input: s = "a"  
Output: [["a"]]
```

Constraints:

- $1 \leq s.length \leq 16$
- s contains only lowercase English letters.

Community Solutions (Python)

- • WalkCC


```

class Solution:
    def partition(self, s: str) -> list[list[str]]:
        ans = []

        def isPalindrome(s: str) -> bool:
            return s == s[::-1]

        def dfs(s: str, j: int, path: list[str], ans: list[list[str]]) -> None:
            if j == len(s):
                ans.append(path)

            return

            for i in range(j, len(s)):
                if isPalindrome(s[j: i + 1]):
                    dfs(s, i + 1, path + [s[j: i + 1]], ans)

        dfs(s, 0, [], ans)
        return ans

```

• LeetCode

```

class Solution:
    def partition(self, s: str) -> List[List[str]]:
        def dfs(i: int):
            if i == n:
                ans.append(t[:])
                return
            for j in range(i, n):
                if f[i][j]:
                    t.append(s[i : j + 1])
                    dfs(j + 1)

            t.pop()

        n = len(s)
        f = [[True] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 1, n):
                f[i][j] = s[i] == s[j] and f[i + 1][j - 1]
        ans = []
        t = []
        dfs(0)

```

```
return ans
```

• SimplyLeet

```
def partition(s):
    # Helper function to check if a given substring is a palindrome
    def is_palindrome(sub):
        return sub == sub[::-1]

    result = [] # final list to store all possible partitions

    # backtracking helper function: current_index points to the starting index
    # of the current search,
    # current_partition stores the partition done so far.
    def backtrack(current_index, current_partition):

        # if we reached the end of the string, add the current_partition to the
        # result list
        if current_index == len(s):
            result.append(list(current_partition))
            return

        # try all possible substrings starting from current_index
        for i in range(current_index, len(s)):
            substring = s[current_index:i+1] # current substring
            # if the substring is a palindrome, add it to current_partition
            if is_palindrome(substring):
                current_partition.append(substring)

                backtrack(i+1, current_partition)
                # backtrack: remove the last added substring
                current_partition.pop()

        backtrack(0, [])
    return result

# Example usage:
print(partition("aab"))
```

◆ Quick Review

Key Insights

- Use backtracking to explore all possible ways to partition the string.
- At each step, check whether the current substring is a palindrome.
- If it is, include that substring in the current partition and recursively partition the rest of the string.
- The limit on the string length (at most 16 characters) allows an exponential backtracking approach.
- Optimizations like caching palindrome checks can be used, though the simple approach is typically sufficient.

Space & Time

Time Complexity: $O(n * 2^n)$ in the worst case, due to exploring all possible partitions and checking palindromes.

Space Complexity: $O(n)$ for recursion stack and temporary storage for current partition.

Solution

The solution uses the backtracking algorithm:

- Define a helper function that will recursively try to partition the string.
- For each possible

split, check if the substring is a palindrome. – If it is, add the substring to the current partition list and recursively call the helper on the remaining substring. – When the end of the string is reached, add the current partition list to the final result. – Backtrack by removing the last added substring and try the next possible partition. This method systematically explores every possible palindrome partitioning of the string.

Greedy

Round 5 · Mid · Medium · 8 questions

Greedy

□ #45 Jump Game II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy
Lists: NeetCode 150

Problem

You are given a 0-indexed array of integers `nums` of length `n`. You are initially positioned at index 0.

Each element `nums[i]` represents the maximum length of a forward jump from index `i`. In other words, if you are at index `i`, you can jump to any index `(i + j)` where:

- $0 \leq j \leq \text{nums}[i]$ and
- $i + j < n$

Return the minimum number of jumps to reach index `n - 1`. The test cases are generated such that you can reach index `n - 1`.

Example 1:

Input: `nums = [2,3,1,1,4]`

Output: 2

Explanation: The minimum number of jumps to reach the last index is 2. Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: nums = [2,3,0,1,4]
Output: 2

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $0 \leq \text{nums}[i] \leq 1000$
- It's guaranteed that you can reach $\text{nums}[n - 1]$.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def jump(self, nums: list[int]) -> int:
        ans = 0
        end = 0
        farthest = 0

        # Start an implicit BFS.
        for i in range(len(nums) - 1):
            farthest = max(farthest, i + nums[i])
            if farthest >= len(nums) - 1:

                ans += 1
                break
            if i == end:      # Visited all the items on the current level.
                ans += 1     # Increment the level.
                end = farthest # Make the queue size for the next level.

        return ans
```

• LeetDoocs

```
class Solution:
    def jump(self, nums: List[int]) -> int:
        ans = mx = last = 0
        for i, x in enumerate(nums[:-1]):
            mx = max(mx, i + x)
            if last == i:
                ans += 1
                last = mx
        return ans
```

• SimplyLeet

```
# Function to calculate minimum number of jumps to reach the last index
def jump(nums):
    # If the list has only one element, we're already at the destination.
    if len(nums) == 1:
        return 0

    n = len(nums) # Length of the input array
    jumps = 0      # To count the number of jumps required
    currentEnd = 0 # The end of the current range that can be reached
    furthest = 0  # The furthest index that can be reached from the current
    range

    # Iterate till the second last element because we don't need to jump from
    the last element
    for i in range(n - 1):
        # Update the furthest index reachable from index i
        furthest = max(furthest, i + nums[i])
        # If we have reached the end of the current jump range
        if i == currentEnd:
            jumps += 1 # Increase the jump count as we make a jump
            currentEnd = furthest # Update the currentEnd to the furthest
    reachable index
    return jumps

# Example usage:
print(jump([2,3,1,1,4])) # Output: 2
```


◆ Quick Review

Key Insights

- Utilize a greedy algorithm to make optimal jumps.
- Track the furthest index that can be reached within the current jump range.
- When the current index reaches the end of the current range, a jump is made and the range updates to the furthest reachable index.
- Edge case: if the array has only one element, no jumps are needed.

Space & Time

Time Complexity: $O(n)$ – Single pass through the array.

Space Complexity: $O(1)$ – Only a few variables are used for tracking jumps and ranges.

Solution

We use a greedy approach by iterating through the array and keeping track of:

- The furthest index reachable from the current range (furthest).
- The end of the current jump's range (currentEnd).
- The number of jumps made (jumps).

At each iteration, we update the furthest reachable index. When the current index reaches the currentEnd, it indicates that one jump has been used and we update currentEnd to furthest. This ensures we are always making the optimal jump to cover the longest distance possible until the end of the array is reached.

Greedy

□ #53 Maximum Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Divide and Conquer

Lists: Grind 169

Problem

Given an integer array `nums`, find the subarray with the largest sum, and return its sum.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`
Output: 23
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

Follow up: If you have figured out the $O(n)$ solution, try coding another solution using the divide and conquer approach, which is more subtle.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        # dp[i] := the maximum sum subarray ending in i
        dp = [0] * len(nums)

        dp[0] = nums[0]
        for i in range(1, len(nums)):
            dp[i] = max(nums[i], dp[i - 1] + nums[i])

        return max(dp)
```

```
class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        ans = -math.inf
        summ = 0

        for num in nums:
            summ = max(num, summ + num)
            ans = max(ans, summ)

        return ans
```

```

from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    summ: int
    # the sum of the subarray starting from the first number
    maxSubarraySumLeft: int
    # the sum of the subarray ending in the last number
    maxSubarraySumRight: int

maxSubarraySum: int

class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        def divideAndConquer(l: int, r: int) -> T:
            if l == r:
                return T(nums[l], nums[l], nums[l], nums[l])
            m = (l + r) // 2
            left = divideAndConquer(l, m)

            right = divideAndConquer(m + 1, r)
            maxSubarraySumLeft = max(left.maxSubarraySumLeft,
                                     left.summ + right.maxSubarraySumLeft)
            maxSubarraySumRight = max(
                left.maxSubarraySumRight + right.summ, right.maxSubarraySumRight)
            maxSubarraySum = max(left.maxSubarraySumRight + right.maxSubarraySumLeft
                                ,
                                left.maxSubarraySum, right.maxSubarraySum)
            summ = left.summ + right.summ
            return T(summ, maxSubarraySumLeft, maxSubarraySumRight, maxSubarraySum)

        return divideAndConquer(0, len(nums) - 1).maxSubarraySum

```

• LeetDoocs

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        ans = f = nums[0]
        for x in nums[1:]:
            f = max(f, 0) + x
            ans = max(ans, f)
        return ans
```

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        def crossMaxSub(nums, left, mid, right):
            lsum = rsum = 0
            lmx = rmx = -inf
            for i in range(mid, left - 1, -1):
                lsum += nums[i]
                lmx = max(lmx, lsum)
            for i in range(mid + 1, right + 1):
                rsum += nums[i]
                rmx = max(rmx, rsum)
            return lmx + rmx

        def maxSub(nums, left, right):
            if left == right:
                return nums[left]
            mid = (left + right) >> 1
            lsum = maxSub(nums, left, mid)
            rsum = maxSub(nums, mid + 1, right)
            csum = crossMaxSub(nums, left, mid, right)

            return max(lsum, rsum, csum)

        left, right = 0, len(nums) - 1
        return maxSub(nums, left, right)
```

• SimplyLeet

```
# Python solution using Kadane's Algorithm
def maxSubArray(nums):
    # Initialize currentSum and maxSum to the first element
    current_sum = max_sum = nums[0]
    # Traverse through the array starting from the second element
    for num in nums[1:]:
        # If current_sum + num is less than num, restart at num
        current_sum = max(num, current_sum + num)
        # Update max_sum if current_sum is greater
        max_sum = max(max_sum, current_sum)

    return max_sum

# Example usage
if __name__ == "__main__":
    sample = [-2,1,-3,4,-1,2,1,-5,4]
    print(maxSubArray(sample)) # Output: 6
```

• LC.ca

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        res = cur_sum = nums[0]
        for num in nums[1:]:
            cur_sum = num + max(cur_sum, 0)
            res = max(res, cur_sum)
        return res
```

◆ Quick Review

Key Insights

- Use Kadane's Algorithm to solve the problem in $O(n)$ time.
- Keep track of a running sum (currentSum) and update it with the maximum between the current element and the sum including the

current element.

- The maximum sum encountered during the iteration (maxSum) is the answer.
- A divide and conquer approach also exists, splitting the array and merging solutions, but Kadane's method is optimal for this problem.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution uses Kadane's Algorithm. Iterate over the array while maintaining two variables: currentSum, which tracks the maximum subarray sum ending at the current position, and maxSum, which stores the best sum seen so far. For each element, update currentSum to be either the current element or the sum of currentSum and the current element, whichever is higher. Update maxSum accordingly. This approach efficiently computes the maximum contiguous subarray sum in a single pass.

Greedy

□ #55 Jump Game

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy
Lists: Grind 169

Problem

You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position.

Return `true` if you can reach the last index, or `false` otherwise.

Example 1:

```
Input: nums = [2,3,1,1,4]
Output: true
Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.
```

Example 2:

```
Input: nums = [3,2,1,0,4]
Output: false
Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.
```

Constraints:

- $1 \leq \text{nums.length} \leq 104$

- $0 \leq \text{nums}[i] \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canJump(self, nums: list[int]) -> bool:
        i = 0
        reach = 0

        while i < len(nums) and i <= reach:
            reach = max(reach, i + nums[i])
            i += 1

        return i == len(nums)
```

• LeetDoocs

```
class Solution:
    def canJump(self, nums: List[int]) -> bool:
        mx = 0
        for i, x in enumerate(nums):
            if mx < i:
                return False
            mx = max(mx, i + x)
        return True
```

• SimplyLeet

```
# Function to determine if you can reach the last index of the array
def canJump(nums):
    # Initialize maxReach which is the max index reachable at start (index 0)
    maxReach = 0
    # Iterate through each element and its index
    for i, jump in enumerate(nums):
        # If current index is greater than maximum reachable index, return
        False
        if i > maxReach:
            return False
        # Update maxReach with the maximum of current maxReach and i + jump
        length at index i
```

```

    maxReach = max(maxReach, i + jump)
    # If current maxReach is already at or beyond the last index, return
    True
    if maxReach >= len(nums) - 1:
        return True
    # Return True if the end is reachable after processing all indices
    return True

# Example usage:
# print(canJump([2,3,1,1,4])) # Expected output: True
# print(canJump([3,2,1,0,4])) # Expected output: False

```

• LC.ca

```

class Solution:
    def canJump(self, nums: List[int]) -> bool:
        mx = 0
        for i, x in enumerate(nums):
            if mx < i:
                return False
            mx = max(mx, i + x)
        return True

```

◆ Quick Review

Key Insights

- The greedy approach is optimal for this problem.
- Track the furthest index reachable as you iterate through the array.
- If the current index exceeds the maximum reachable index, it is impossible to proceed further.

- The solution only requires constant extra space.

Space & Time

Time Complexity: $O(n)$ where n is the length of the array.

Space Complexity: $O(1)$

Solution

The solution uses a greedy algorithm. As you iterate through each array element, you maintain a variable (`maxReach`) that represents the furthest index you can reach so far. For each index i , if i is greater than `maxReach` then it is not possible to reach index i and hence return false immediately. Otherwise, update `maxReach` to the maximum of its current value and $i + \text{nums}[i]$. If you can iterate through the array without i exceeding `maxReach`, then you can reach the end and return true.

Greedy

□ #134 Gas Station

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy
Lists: Grind 169

Problem

There are n gas stations along a circular route, where the amount of gas at the i th station is $gas[i]$.

You have a car with an unlimited gas tank and it costs $cost[i]$ of gas to travel from the i th station to its next $(i + 1)$ th station. You begin the journey with an empty tank at one of the gas stations.

Given two integer arrays gas and $cost$, return the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return -1 . If there exists a solution, it is guaranteed to be unique.

Example 1:

Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]

Output: 3

Explanation:

Start at station 3 (index 3) and fill up with 4 unit of gas. Your tank = $0 + 4 = 4$

Travel to station 4. Your tank = $4 - 1 + 5 = 8$

Travel to station 0. Your tank = $8 - 2 + 1 = 7$

Travel to station 1. Your tank = $7 - 3 + 2 = 6$

Travel to station 2. Your tank = $6 - 4 + 3 = 5$

Example 2:

Input: gas = [2,3,4], cost = [3,4,3]

Output: -1

Explanation:

You can't start at station 0 or 1, as there is not enough gas to travel to the next station.

Let's start at station 2 and fill up with 4 unit of gas. Your tank = $0 + 4 = 4$

Travel to station 0. Your tank = $4 - 3 + 2 = 3$

Travel to station 1. Your tank = $3 - 3 + 3 = 3$

You cannot travel back to station 2, as it requires 4 unit of gas but you only have 3.

Constraints:

- $n == \text{gas.length} == \text{cost.length}$
- $1 \leq n \leq 105$
- $0 \leq \text{gas}[i], \text{cost}[i] \leq 104$
- The input is generated such that the answer is unique.

Community Solutions (Python)

- WalkCC

```

class Solution:
    def canCompleteCircuit(self, gas: list[int], cost: list[int]) -> int:
        ans = 0
        net = 0
        summ = 0

        # Try to start from each index.
        for i in range(len(gas)):
            net += gas[i] - cost[i]
            summ += gas[i] - cost[i]

        if summ < 0:
            summ = 0
            ans = i + 1 # Start from the next index.

        return -1 if net < 0 else ans

```

• LeetCode

```

class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        n = len(gas)
        i = j = n - 1
        cnt = s = 0
        while cnt < n:
            s += gas[j] - cost[j]
            cnt += 1
            j = (j + 1) % n
            while s < 0 and cnt < n:

                i -= 1
                s += gas[i] - cost[i]
                cnt += 1
        return -1 if s < 0 else i

```

• SimplyLeet

```

# Define a function to find starting gas station index
def canCompleteCircuit(gas, cost):
    total_surplus = 0 # Total gas surplus after completing the route
    current_surplus = 0 # Gas surplus while iterating from a candidate start
    station
    start_index = 0 # Candidate starting station index

    # Loop through each station
    for i in range(len(gas)):
        # Calculate net gas after leaving station i
        surplus = gas[i] - cost[i]

        total_surplus += surplus
        current_surplus += surplus

        # If current surplus becomes negative, station i is not a valid start.
        if current_surplus < 0:
            # Reset starting station to the next station
            start_index = i + 1
            current_surplus = 0 # Reset current surplus
    # Check if overall surplus is non-negative
    return start_index if total_surplus >= 0 else -1

# Example usage:
gas = [1,2,3,4,5]
cost = [3,4,5,1,2]
print(canCompleteCircuit(gas, cost)) # Output: 3

```

• LC.ca

```

class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        n = len(gas)
        i = j = n - 1
        cnt = s = 0
        while cnt < n:
            s += gas[j] - cost[j]
            cnt += 1
            j = (j + 1) % n
            while s < 0 and cnt < n:

```



```
i -= 1
s += gas[i] - cost[i]
cnt += 1
return -1 if s < 0 else i
```

◆ Quick Review

Key Insights

- If the total amount of gas available is less than the total travel cost, then completing the circuit is impossible.
- A greedy approach can be used by keeping track of the current surplus of gas. If the surplus becomes negative at any station, the journey cannot start from any station before that point.
- Restart the journey from the next station whenever the surplus goes negative.
- The problem guarantees that if a solution exists, it is unique.

Space & Time

Time Complexity: $O(n)$, where n is the number of gas stations, as we traverse the list only once.

Space Complexity: $O(1)$, as no extra space is used other than a few variables.

Solution

The solution involves iterating over the list of stations while maintaining two variables: one for the current gas surplus and one for the total gas surplus. If at any station the current surplus drops below zero, the current starting station is not viable and we set the starting station to the next index and reset the current surplus. After one pass, if the total gas surplus is non-negative, then the identified station is the valid starting point.

Greedy

□ #678 Valid Parenthesis String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Stack · Greedy
Lists: NeetCode 150

Problem

Given a string *s* containing only three types of characters: '(', ')' and '*', return true if *s* is valid.

The following rules define a valid string:

- Any left parenthesis '(' must have a corresponding right parenthesis ')'.
• Any right parenthesis ')' must have a corresponding left parenthesis '('.
- Left parenthesis '(' must go before the corresponding right parenthesis ')'.
• '*' could be treated as a single right parenthesis ')' or a single left parenthesis '(' or an empty string "".

Example 1:

```
Input: s = "()"
Output: true
```

Example 2:

Input: s = "(*)"
Output: true

Example 3:

Input: s = "(*))"
Output: true

Constraints:

- $1 \leq s.length \leq 100$
- s[i] is '(', ')' or '*'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def checkValidString(self, s: str) -> bool:
        low = 0
        high = 0

        for c in s:
            if c == '(':
                low += 1
                high += 1
            elif c == ')':
                if low > 0:
                    low -= 1
                high -= 1
            else:
                if low > 0:
                    low -= 1
                high += 1
        if high < 0:
            return False
```

```
return low == 0
```

• LeetDoocs

```
class Solution:
    def checkValidString(self, s: str) -> bool:
        n = len(s)
        dp = [[False] * n for _ in range(n)]
        for i, c in enumerate(s):
            dp[i][i] = c == '*'
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                dp[i][j] = (
                    s[i] in '(*' and s[j] in '*)' and (i + 1 == j or dp[i + 1]
[j - 1])
                )
                dp[i][j] = dp[i][j] or any(
                    dp[i][k] and dp[k + 1][j] for k in range(i, j)
                )
        return dp[0][-1]
```

```
class Solution:
    def checkValidString(self, s: str) -> bool:
        x = 0
        for c in s:
            if c in '(':
                x += 1
            elif x:
                x -= 1
            else:
                return False
```

```
        x = 0
        for c in s[::-1]:
            if c in ')':
                x += 1
            elif x:
                x -= 1
            else:
                return False
        return True
```

• SimplyLeet

```
# Python implementation with detailed inline comments
```

```
def checkValidString(s):  
    # Initialize the minimum and maximum possible count of open parentheses  
    minOpen = 0  
    maxOpen = 0
```

```
    # Iterate over each character in the string
```

```
    for char in s:  
        if char == '(':
```

```
            # For an open parenthesis, both min and max increase by 1
```

```
            minOpen += 1
```

```
            maxOpen += 1
```

```
        elif char == ')':
```

```
            # For a close parenthesis, both min and max decrease by 1
```

```
            minOpen -= 1
```

```
            maxOpen -= 1
```

```
        else: # char == '*'
```

```
            # '*' can act as a ')' (decrease min) or '(' (increase max)
```

```
            minOpen -= 1
```

```
            maxOpen += 1
```

```
    # Ensure minOpen does not drop below 0
```

```
    if minOpen < 0:
```

```
        minOpen = 0
```

```
    # If maxOpen is negative, there are too many ')' to balance
```

```
    if maxOpen < 0:
```

```
        return False
```

```
    # The string is valid if there is a possible balance where minOpen equals 0
```

```
    return minOpen == 0
```

```
# Example usage:
```

```
print(checkValidString("(*)")) # Expected output: True
```

◆ Quick Review

Key Insights

- The '*' character provides flexibility, allowing multiple interpretations.
- A greedy approach can track the range of possible open parentheses count.
- Use two counters to represent the minimum and maximum open count possible at any moment.
- If the maximum count goes negative at any point, the string cannot be valid.
- At the end, the minimum count must be zero for a valid string.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string

Space Complexity: $O(1)$

Solution

We use a greedy algorithm by maintaining two counters:

- minOpen – the minimum number of open parentheses that could be present.
- maxOpen – the maximum number of open parentheses that could be present.

For every character in the string:

– For '(', increment both minOpen and maxOpen. – For ')', decrement both minOpen and maxOpen. – For '*', decrement minOpen (treat as ')') and increment maxOpen (treat as '(').

Clamp minOpen to not go below 0, since negative count of open parentheses is not possible in any valid interpretation. If maxOpen drops below 0 at any point, it means there are more closing brackets than can be balanced by available '*' and '(', so the string is invalid. Finally, if minOpen is 0, the string can be balanced, otherwise it is invalid.

Greedy

□ #763 Partition Labels

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Two Pointers · String · Greedy
Lists: NeetCode 150

Problem

You are given a string *s*. We want to partition the string into as many parts as possible so that each letter appears in at most one part. For example, the string "ababcc" can be partitioned into ["abab", "cc"], but partitions such as ["aba", "bcc"] or ["ab", "ab", "cc"] are invalid.

Note that the partition is done so that after concatenating all the parts in order, the resultant string should be *s*.

Return a list of integers representing the size of these parts.

Example 1:

Input: s = "ababcbacacdefegdehijklij"

Output: [9,7,8]

Explanation:

The partition is "ababcbaca", "defegde", "hijklij".

This is a partition so that each letter appears in at most one part.

A partition like "ababcbacacdefegde", "hijklij" is incorrect, because it splits s into less parts.

Example 2:

Input: s = "eccbbbbdec"

Output: [10]

Constraints:

- $1 \leq s.length \leq 500$
- s consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def partitionLabels(self, s: str) -> list[int]:
        ans = []
        letterToRightmostIndex = {c: i for i, c in enumerate(s)}

        l = 0 # the leftmost index of the current running string
        r = 0 # the rightmost index of the current running string

        for i, c in enumerate(s):
            r = max(r, letterToRightmostIndex[c])

            if i == r:
                ans.append(r - l + 1)
                l = r + 1

        return ans
```

• LeetDoocs

```

class Solution:
    def partitionLabels(self, s: str) -> List[int]:
        last = {c: i for i, c in enumerate(s)}
        mx = j = 0
        ans = []
        for i, c in enumerate(s):
            mx = max(mx, last[c])
            if mx == i:
                ans.append(i - j + 1)
                j = i + 1

        return ans

```

• SimplyLeet

```

# Python Solution
class Solution:
    def partitionLabels(self, s: str) -> list:
        # Step 1: Record the last occurrence of each character.
        last_occurrence = {char: i for i, char in enumerate(s)}
        # List to store the sizes of partitions.
        partitions = []
        start, end = 0, 0

        # Step 2: Iterate over the string to form partitions.

        for i, char in enumerate(s):
            # Update the end index for the current partition.
            end = max(end, last_occurrence[char])
            # When we reach the end of the partition.
            if i == end:
                partitions.append(end - start + 1)
                start = i + 1 # Move start to the next index.

        return partitions

```

◆ Quick Review

Key Insights

- Determine the last occurrence of every character in the string.
- Use a greedy approach to extend the current partition until all characters within it do not appear later in the string.
- When the current index reaches the furthest last occurrence for characters seen so far, a partition can be closed.
- Repeat the process for the remaining section of the string.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string because we traverse the string twice.

Space Complexity: $O(1)$ (or $O(26)$ to be precise) for storing the last occurrences of each character.

Solution

We first iterate through the string to store the last index at which each character appears. Then, we traverse the string while maintaining two pointers: one for the start of the current partition and one to track the furthest last occurrence (end) of any character seen so far.

For each character, we update the end pointer to be the maximum of its current value and the last index of that character. When the current index equals the end pointer, it means that all characters in the current partition do not appear later. We then determine the size of this partition, add it to the result list, and move to the next partition. We repeat this until the entire string is processed.

Greedy

□ #846 Hand of Straights

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Greedy · Sorting
Lists: NeetCode 150

Problem

Alice has some number of cards and she wants to rearrange the cards into groups so that each group is of size `groupSize`, and consists of `groupSize` consecutive cards.

Given an integer array `hand` where `hand[i]` is the value written on the *i*th card and an integer `groupSize`, return `true` if she can rearrange the cards, or `false` otherwise.

Example 1:

Input: `hand = [1,2,3,6,2,3,4,7,8]`, `groupSize = 3`
Output: `true`
Explanation: Alice's hand can be rearranged as `[1,2,3]`, `[2,3,4]`, `[6,7,8]`

Example 2:

Input: `hand = [1,2,3,4,5]`, `groupSize = 4`
Output: `false`
Explanation: Alice's hand can not be rearranged into groups of 4.

Constraints:

- $1 \leq \text{hand.length} \leq 104$
- $0 \leq \text{hand}[i] \leq 109$
- $1 \leq \text{groupSize} \leq \text{hand.length}$

Note: This question is the same as 1296: <https://leetcode.com/problems/divide-array-in-sets-of-k-consecutive-numbers/>

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isNStraightHand(self, hand: list[int], groupSize: int) -> bool:
        count = collections.Counter(hand)

        for start in sorted(count):
            value = count[start]
            if value > 0:
                for i in range(start, start + groupSize):
                    count[i] -= value
                    if count[i] < 0:
                        return False

        return True
```

• LeetDoocs

```

class Solution:
    def isNStraightHand(self, hand: List[int], groupSize: int) -> bool:
        if len(hand) % groupSize:
            return False
        cnt = Counter(hand)
        for x in sorted(hand):
            if cnt[x]:
                for y in range(x, x + groupSize):
                    if cnt[y] == 0:
                        return False
                    cnt[y] -= 1
        return True

```

• SimplyLeet

```

# Python solution using collections.Counter for frequency mapping
from collections import Counter

def isNStraightHand(hand, groupSize):
    # Check whether total cards can be divided into groups of groupSize
    if len(hand) % groupSize != 0:
        return False

    # Create frequency map of cards
    card_count = Counter(hand)

    # Process sorted keys to ensure we always start with the smallest available
    # card
    for card in sorted(card_count.keys()):
        # If current count is greater than 0, try to form a consecutive group
        # starting from card
        count = card_count[card]
        if count > 0:
            for i in range(groupSize):
                # Check if consecutive card exists in sufficient quantity
                if card_count[card + i] < count:
                    return False

```



```
# Deduct the frequency for the current group
card_count[card + i] -= count

return True

# Example usage:
# print(isNStraightHand([1,2,3,6,2,3,4,7,8], 3)) # Expected output: True
```

◆ Quick Review

Key Insights

- The total number of cards must be divisible by `groupSize`.
- Use a frequency count (hash table or map) to record the occurrence of each card.
- Always start forming groups from the smallest available card to maintain the consecutive order.
- For each card, if its count is non-zero, check that the subsequent `groupSize-1` cards exist with at least as many counts.
- Decrease the counts accordingly for each valid consecutive group.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the unique keys from the hand.

Space Complexity: $O(n)$ for storing the frequency count of the cards.

Solution

The solution uses a greedy approach along with a frequency hash map. The main steps are:

– Verify that the total number of cards is divisible by groupSize. – Count the frequency of each card. – Sort the keys of the frequency map. – Iterate over the sorted keys and for each card with a positive count, try to form a group of groupSize consecutive cards. – For each card in the group, decrease the frequency by the number of groups formed. If at any point the required card is missing or its count is insufficient, return false. – If all groups can be formed successfully, return true.

Greedy

□ #1899 Merge Triplets to Form Target Triplet

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy
Lists: NeetCode 150

Problem

A triplet is an array of three integers. You are given a 2D integer array triplets, where $\text{triplets}[i] = [a_i, b_i, c_i]$ describes the i th triplet. You are also given an integer array target = $[x, y, z]$ that describes the triplet you want to obtain.

To obtain target, you may apply the following operation on triplets any number of times (possibly zero):

- Choose two indices (0-indexed) i and j ($i \neq j$) and update $\text{triplets}[j]$ to become $[\max(a_i, a_j), \max(b_i, b_j), \max(c_i, c_j)]$. For example, if $\text{triplets}[i] = [2, 5, 3]$ and $\text{triplets}[j] = [1, 7, 5]$, $\text{triplets}[j]$ will be updated to $[\max(2, 1), \max(5, 7), \max(3, 5)] = [2, 7, 5]$.

Return true if it is possible to obtain the target triplet $[x, y, z]$ as an element of triplets, or false otherwise.

Example 1:

Input: triplets = $[[2,5,3],[1,8,4],[1,7,5]]$, target = $[2,7,5]$

Output: true

Explanation: Perform the following operations:

– Choose the first and last triplets $[[2,5,3],[1,8,4],[1,7,5]]$. Update the last triplet to be $[\max(2,1), \max(5,7), \max(3,5)] = [2,7,5]$. triplets = $[[2,5,3],[1,8,4],[2,7,5]]$

The target triplet $[2,7,5]$ is now an element of triplets.

Example 2:

Input: triplets = $[[3,4,5],[4,5,6]]$, target = $[3,2,5]$

Output: false

Explanation: It is impossible to have $[3,2,5]$ as an element because there is no 2 in any of the triplets.

Example 3:

Input: triplets = $[[2,5,3],[2,3,4],[1,2,5],[5,2,3]]$, target = $[5,5,5]$

Output: true

Explanation: Perform the following operations:

– Choose the first and third triplets $[[2,5,3],[2,3,4],[1,2,5],[5,2,3]]$. Update the third triplet to be $[\max(2,1), \max(5,2), \max(3,5)] = [2,5,5]$. triplets = $[[2,5,3],[2,3,4],[2,5,5],[5,2,3]]$.

– Choose the third and fourth triplets $[[2,5,3],[2,3,4],[2,5,5],[5,2,3]]$. Update the fourth triplet to be $[\max(2,5), \max(5,2), \max(5,3)] = [5,5,5]$. triplets = $[[2,5,3],[2,3,4],[2,5,5],[5,5,5]]$.

The target triplet $[5,5,5]$ is now an element of triplets.

Constraints:

- $1 \leq \text{triplets.length} \leq 105$
- $\text{triplets}[i].\text{length} == \text{target.length} == 3$
- $1 \leq a_i, b_i, c_i, x, y, z \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mergeTriplets(self, triplets: list[list[int]], target: list[int]) -> bool:
        merged = [0] * len(target)

        for triplet in triplets:
            if all(a <= b for a, b in zip(triplet, target)):
                for i in range(3):
                    merged[i] = max(merged[i], triplet[i])

        return merged == target
```

• LeetCode

```
class Solution:
    def mergeTriplets(self, triplets: List[List[int]], target: List[int]) -> bool:
        x, y, z = target
        d = e = f = 0
        for a, b, c in triplets:
            if a <= x and b <= y and c <= z:
                d = max(d, a)
                e = max(e, b)
                f = max(f, c)
        return [d, e, f] == target
```

• SimplyLeet

```
def mergeTriplets(triplets, target):
    # Initialize candidate triplet with zeros
    candidate = [0, 0, 0]
    # Iterate over every triplet in the input list
    for trip in triplets:
        # Only consider triplets where each element is <= corresponding target element
        if trip[0] <= target[0] and trip[1] <= target[1] and trip[2] <= target[2]:
            # Update candidate by taking the element-wise maximum
            candidate[0] = max(candidate[0], trip[0])
            candidate[1] = max(candidate[1], trip[1])
```

```
        candidate[2] = max(candidate[2], trip[2])
    # If the candidate matches target, return True; otherwise, return False
    return candidate == target

# Example usage:
triplets = [[2,5,3],[1,8,4],[1,7,5]]
target = [2,7,5]
print(mergeTriplets(triplets, target)) # Output: True
```

◆ Quick Review

Key Insights

- Only consider triplets that are "valid" in the sense that none of its values exceed the corresponding target value.
- For each component of the target, there must be at least one valid triplet that provides that maximum value.
- By taking the element-wise maximum across all valid triplets, if we can recreate the target, then the target can be formed using a series of allowed operations.

Space & Time

Time Complexity: $O(n)$, where n is the number of triplets.

Space Complexity: $O(1)$ extra space.

Solution

The solution approach involves iterating over the list of triplets and only considering those triplets that do not exceed the target values in any position. For each valid triplet, update a candidate triplet by taking the element-wise maximum with the current candidate. Finally, check if the candidate exactly matches the target triplet. The algorithm uses simple array operations and avoids any extra space beyond a few variables.

Round 6 · Mid · Hard

Round 6

Mid Priority · Hard

6 questions · 5 patterns

- ☐ ● **Linked List #23 #25** ↗
- ☐ ● **Stack #84** ↗
- ☐ ● **Heap / Priority Queue #295** ↗
- ☐ ● **Tries #212** ↗
- ☐ ● **Backtracking #51** ↗

Linked List

Round 6 · Mid · Hard · 2 questions

Linked List

□ #23 Merge k Sorted Lists

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Divide and Conquer · Heap ·
Merge Sort
Lists: Grind 169

Problem

You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked-lists are:
[
  1->4->5,
  1->3->4,
  2->6
]
```

Example 2:

```
Input: lists = []
Output: []
```

Example 3:

Input: lists = [[]]
Output: []

Constraints:

- $k == \text{lists.length}$
- $0 \leq k \leq 104$
- $0 \leq \text{lists}[i].\text{length} \leq 500$
- $-104 \leq \text{lists}[i][j] \leq 104$
- $\text{lists}[i]$ is sorted in ascending order.
- The sum of $\text{lists}[i].\text{length}$ will not exceed 104.

Community Solutions (Python)

• WalkCC

```
from queue import PriorityQueue

class Solution:
    def mergeKLists(self, lists: list[ListNode]) -> ListNode:
        dummy = ListNode(0)
        curr = dummy
        pq = PriorityQueue()

        for i, lst in enumerate(lists):
```

```

        if lst:
            pq.put((lst.val, i, lst))

    while not pq.empty():
        _, i, minNode = pq.get()
        if minNode.next:
            pq.put((minNode.next.val, i, minNode.next))
        curr.next = minNode
        curr = curr.next

    return dummy.next

```

• LeetDoocs

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        setattr(ListNode, "__lt__", lambda a, b: a.val < b.val)
        pq = [head for head in lists if head]
        heapify(pq)

        dummy = cur = ListNode()
        while pq:
            node = heappop(pq)
            if node.next:
                heappush(pq, node.next)
            cur.next = node
            cur = cur.next
        return dummy.next

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

import heapq

class Solution:
    def mergeKLists(self, lists):

        # Create a min-heap
        min_heap = []
        # Heap entry: (node value, unique id, node) to handle duplicate values
        for i, node in enumerate(lists):
            if node:
                heapq.heappush(min_heap, (node.val, i, node))

        # Dummy node to form the start of merged list
        dummy = ListNode(0)
        current = dummy

        # While there are elements in the heap
        while min_heap:
            # Pop the smallest node from the heap
            val, idx, node = heapq.heappop(min_heap)
            # Add the smallest node to the merged list
            current.next = node
            current = current.next
            # If there is a next node in the list, push it into the heap
            if node.next:
                heapq.heappush(min_heap, (node.next.val, idx, node.next))

        return dummy.next

```

• LC.ca

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        setattr(ListNode, "__lt__", lambda a, b: a.val < b.val)
        pq = [head for head in lists if head]
        heapify(pq)
```

```
dummy = cur = ListNode()
while pq:
    node = heappop(pq)
    if node.next:
        heappush(pq, node.next)
    cur.next = node
    cur = cur.next
return dummy.next
```

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
```

```
#####
```

```
import heapq
```

```

'''
__lt__(self, other) for <
__le__(self, other) for <=
__gt__(self, other) for >
__ge__(self, other) for >=
'''

'''

```

```

# or create a wrapper class, without modifying existing node class
class NodeWrapper:
    def __init__(self, node):
        self.node = node

    def __lt__(self, other):
        return self.node.val < other.node.val

'''

```

```

# overwrite the comparison function, so the node can be comparable
# or else, error, TypeError: '<' not supported between instances of 'ListNode'
and 'ListNode'

# define 'gt' will also work, so either lt or gt will do the compare job for
ListNode
# ListNode.__gt__ = lambda x, y: (x.val > y.val)

ListNode.__lt__ = lambda x, y: (x.val < y.val)

class Solution:

```

```

def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
    dummy = current = ListNode()
    heap = []
    for i, node in enumerate(lists):
        if node:
            # need to override
            # or else error: TypeError: '<' not supported between instances
            # of 'ListNode' and 'ListNode'
            heapq.heappush(heap, node)

    while heap:
        anode = heapq.heappop(heap)

        current.next = anode
        current = current.next

        if anode.next:
            heapq.heappush(heap, anode.next)

    return dummy.next

```

```

'''
tried to use node.val to order heap, but still got error TypeError: '<' not
supported between instances of 'ListNode' and 'ListNode':

heapq.heappush(heap, (node.val, node))

'''

#####

# based on merge 2 lists

```



```

# note for empty input
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        if lists is None or len(lists) == 0:
            return None
        return self.mergeKListsHelper(lists, 0, len(lists) - 1)

    def mergeKListsHelper(self, lists: List[Optional[ListNode]], start: int, end: int) -> Optional[ListNode]:
        if start == end:
            return lists[start]

        mid = int((start + end) / 2)

        left = self.mergeKListsHelper(lists, start, mid)
        # print(left)

        right = self.mergeKListsHelper(lists, mid + 1, end)
        # print(right)

        return self.mergeTwoLists(left, right)

    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode()
        current = dummy
        # print("merging: ", list1)
        # print("merging: ", list2)
        while list1 or list2:
            v1 = list1.val if list1 else float('inf')
            v2 = list2.val if list2 else float('inf')

```

```

        if v1 < v2:
            current.next = list1
            list1 = list1.next
        else:
            current.next = list2
            list2 = list2.next
        current = current.next

    # print("merging result: ", dummy.next)

```

```

    return dummy.next

```

```

#####

```

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

```

```

class Solution:
    def mergeKLists(self, lists: List[ListNode]) -> ListNode:
        n = len(lists)
        if n == 0:
            return None
        for i in range(n - 1): # this is o(N) times of merge, while if use
            # divide-mid-merge then it's o(logN) times of merge
            lists[i + 1] = self.mergeTwoLists(lists[i], lists[i + 1])
        return lists[-1]

    def mergeTwoLists(self, l1: ListNode, l2: ListNode) -> ListNode:

```

```
dummy = ListNode()
cur = dummy
while l1 and l2:
    if l1.val <= l2.val:
        cur.next = l1
        l1 = l1.next
    else:
        cur.next = l2
        l2 = l2.next
    cur = cur.next

cur.next = l1 or l2
return dummy.next

if __name__ == '__main__':

    l1 = ListNode(1)
    l1.next = ListNode(4)
    l1.next.next = ListNode(5)

    l2 = ListNode(1)
    l2.next = ListNode(3)
    l2.next.next = ListNode(4)

    l3 = ListNode(2)
    l3.next = ListNode(6)

    print(Solution().mergeKLists([l1, l2, l3]))

    print(Solution().mergeKLists([]))
    print(Solution().mergeKLists([[]]))
```

◆ Quick Review

Key Insights

- Use a min-heap (priority queue) to efficiently extract the smallest value among the current heads of the lists.
- By pushing the head of each non-empty list into the heap, we can pull the smallest node, then push that node's next element into the heap.
- Alternatively, a divide and conquer approach can be used by recursively merging pairs of lists.
- Careful management of pointers (or references) is key to constructing the new linked list.

Space & Time

Time Complexity: $O(N \log k)$ where N is the total number of nodes across all lists and k is the number of lists.

Space Complexity: $O(k)$ due to the heap containing at most one node from each list.

Solution

We use a min-heap to always retrieve the smallest node among the current nodes in the linked lists. First, initialize the heap with the head of each list. Then, continuously extract

the smallest node from the heap, attach it to the merged list, and if the extracted node has a next node, insert that into the heap. The process repeats until the heap is empty.

Linked List

□ #25 Reverse Nodes in k-Group

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Recursion

Lists: Grind 169

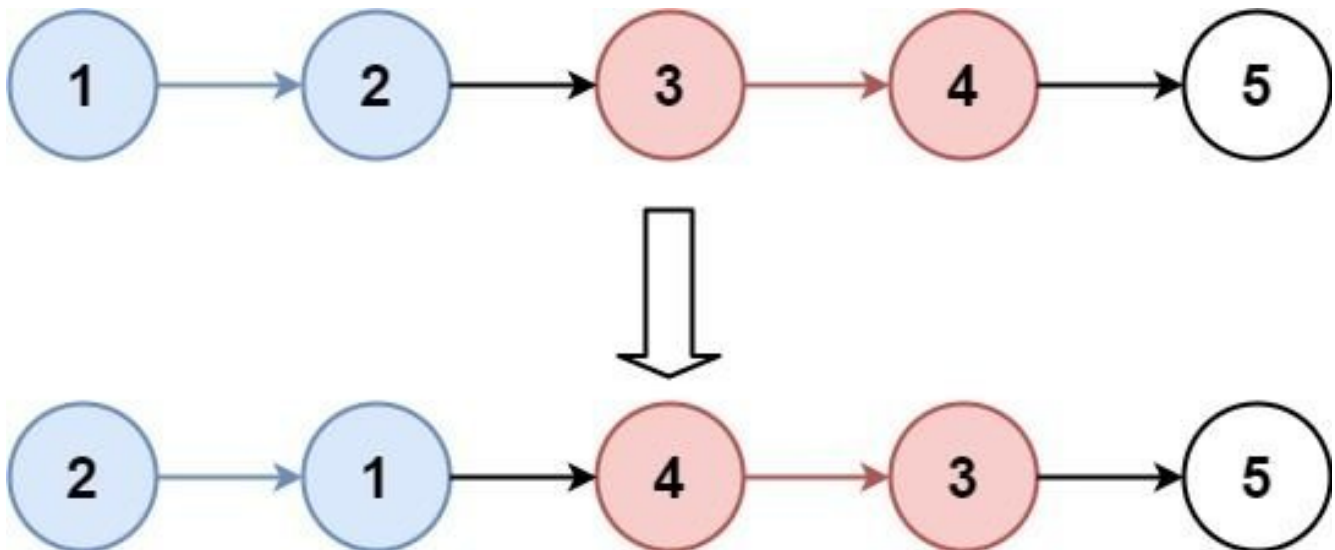
Problem

Given the head of a linked list, reverse the nodes of the list k at a time, and return the modified list.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.

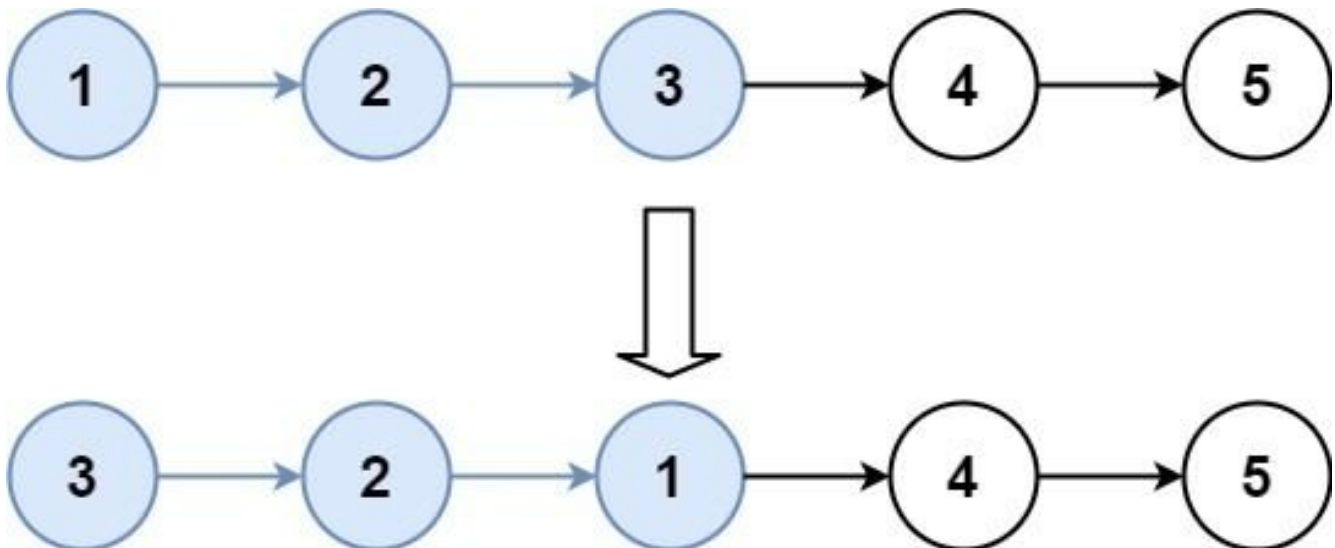
You may not alter the values in the list's nodes, only nodes themselves may be changed.

Example 1:



Input: head = [1,2,3,4,5], k = 2
Output: [2,1,4,3,5]

Example 2:



Input: head = [1,2,3,4,5], k = 3
Output: [3,2,1,4,5]

Constraints:

- The number of nodes in the list is n .
- $1 \leq k \leq n \leq 5000$
- $0 \leq \text{Node.val} \leq 1000$

Follow-up: Can you solve the problem in $O(1)$ extra memory space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverseKGroup(self, head: ListNode | None, k: int) -> ListNode | None:
        if not head:
            return None

        tail = head

        for _ in range(k):
            # There are less than k nodes in the list, do nothing.
            if not tail:
                return head
            tail = tail.next

        newHead = self._reverse(head, tail)
        head.next = self.reverseKGroup(tail, k)
        return newHead

    def _reverse(
        self,
        head: ListNode | None,
        tail: ListNode | None,
    ) -> ListNode | None:
        """Reverses [head, tail)."""
        prev = None
        curr = head
        while curr != tail:
            next = curr.next
            curr.next = prev
            prev = curr
            curr = next

        return prev
```



```

class Solution:
    def reverseKGroup(self, head: ListNode, k: int) -> ListNode:
        if not head or k == 1:
            return head

        def getLength(head: ListNode) -> int:
            length = 0
            while head:
                length += 1
                head = head.next

            return length

        length = getLength(head)
        dummy = ListNode(0, head)
        prev = dummy
        curr = head

        for _ in range(length // k):
            for _ in range(k - 1):
                next = curr.next

                curr.next = next.next
                next.next = prev.next
                prev.next = next
                prev = curr
                curr = curr.next

        return dummy.next

```

• LeetDoocs

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseKGroup(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        def reverse(head: Optional[ListNode]) -> Optional[ListNode]:
            dummy = ListNode()
            cur = head

```

```

        while cur:
            nxt = cur.next
            cur.next = dummy.next
            dummy.next = cur
            cur = nxt
        return dummy.next

    dummy = pre = ListNode(next=head)
    while pre:
        cur = pre

```

```

        for _ in range(k):
            cur = cur.next
            if cur is None:
                return dummy.next
        node = pre.next
        nxt = cur.next
        cur.next = None
        pre.next = reverse(node)
        node.next = nxt
        pre = node

```

```

    return dummy.next

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reverseKGroup(self, head: ListNode, k: int) -> ListNode:
        # Create a dummy node to simplify edge cases
        dummy = ListNode(0)

```

```

dummy.next = head
group_prev = dummy

while True:
    # Find the kth node from group_prev
    kth = self.getKthNode(group_prev, k)
    if not kth:
        break
    group_next = kth.next

    # Reverse the group starting at group_prev.next up to kth
    prev = kth.next
    curr = group_prev.next
    while curr != group_next:
        temp = curr.next    # Store next node
        curr.next = prev    # Reverse pointer
        prev = curr         # Move prev forward
        curr = temp         # Move curr forward

    # Reconnect the reversed group with the previous part

    temp = group_prev.next    # temp is the start of the original group,
    becomes the new tail
    group_prev.next = kth     # Connect previous part to the kth node
    (new head of reversed group)
    group_prev = temp         # Move group_prev to the end of the
    reversed group

    return dummy.next

# Helper function to get the kth node from the current node
def getKthNode(self, curr, k):
    while curr and k > 0:
        curr = curr.next

    k -= 1
    return curr

```

 • LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseKGroup(self, head: ListNode, k: int) -> ListNode:
        """
        for reverse 1->2->3->4->5, process is like
        1->None, 2->3->4->5

        2->1->None, 3->4->5
        3->2->1->None, 4->5
        4->3->2->1->None, 5
        5->4->3->2->1->None, None
        """
        def reverseList(head):
            pre, p = None, head
            while p:
                pnext = p.next
                p.next = pre
                pre = p
                p = pnext
            return pre

        dummy = ListNode(next=head)
        pre = cur = dummy
        while cur.next:
            for _ in range(k):
                cur = cur.next
            if cur is None:

```

```

        return dummy.next
    t = cur.next
    cur.next = None # cut from next k-group, so to reverseList() for
current k-group
    start = pre.next
    pre.next = reverseList(start)
    start.next = t # so now 'start' is the last node of k-group
    pre = cur = start # same as the reset dummy before while loop
    return dummy.next

```

```
#####
```

```

# Definition for singly-linked list.
# class ListNode(object):
# def __init__(self, x):
# self.val = x
# self.next = None

```

```

class Solution(object):
    def reverseKGroup(self, head, k):
        """

```

```

        :type head: ListNode
        :type k: int
        :rtype: ListNode
        """

```

```

    def reverseList(head, k):
        pre = None
        cur = head
        while cur and k > 0:
            tmp = cur.next

```

```
        cur.next = pre
        pre = cur
        cur = tmp
        k -= 1
    head.next = cur
    return cur, pre

length = 0
p = head
while p:

    length += 1
    p = p.next
    if length < k:
        return head
    step = length / k
    ret = None
    pre = None
    p = head
    while p and step:
        next, newHead = reverseList(p, k)

        if ret is None:
            ret = newHead
        if pre:
            pre.next = newHead
        pre = p
        p = next
        step -= 1
    return ret
```

◆ Quick Review

Key Insights

- Use a dummy node to simplify edge cases.
- Identify groups of k nodes using a helper function.

- Reverse the nodes in each group using pointer manipulation.
- If a remaining group has fewer than k nodes, leave it unchanged.
- The reversal is done in-place with $O(1)$ extra space.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes (each node is processed a constant number of times)

Space Complexity: $O(1)$ (only constant extra space is used)

Solution

The solution involves iterating through the linked list by identifying groups of k consecutive nodes. For each group: Find the k th node from the current starting point. If it does not exist, the remainder of the list is not reversed. Reverse the nodes in the identified group in-place by adjusting the next pointers. Reconnect the reversed group with the previous part of the list and update pointers to move to the next group. The approach leverages a dummy node for easier handling

of head modifications and uses a helper function to fetch the kth node. The in-place reversal ensures that the extra memory usage is constant.

Stack

Round 6 · Mid · Hard · 1 question

Stack

□ #84 Largest Rectangle in Histogram

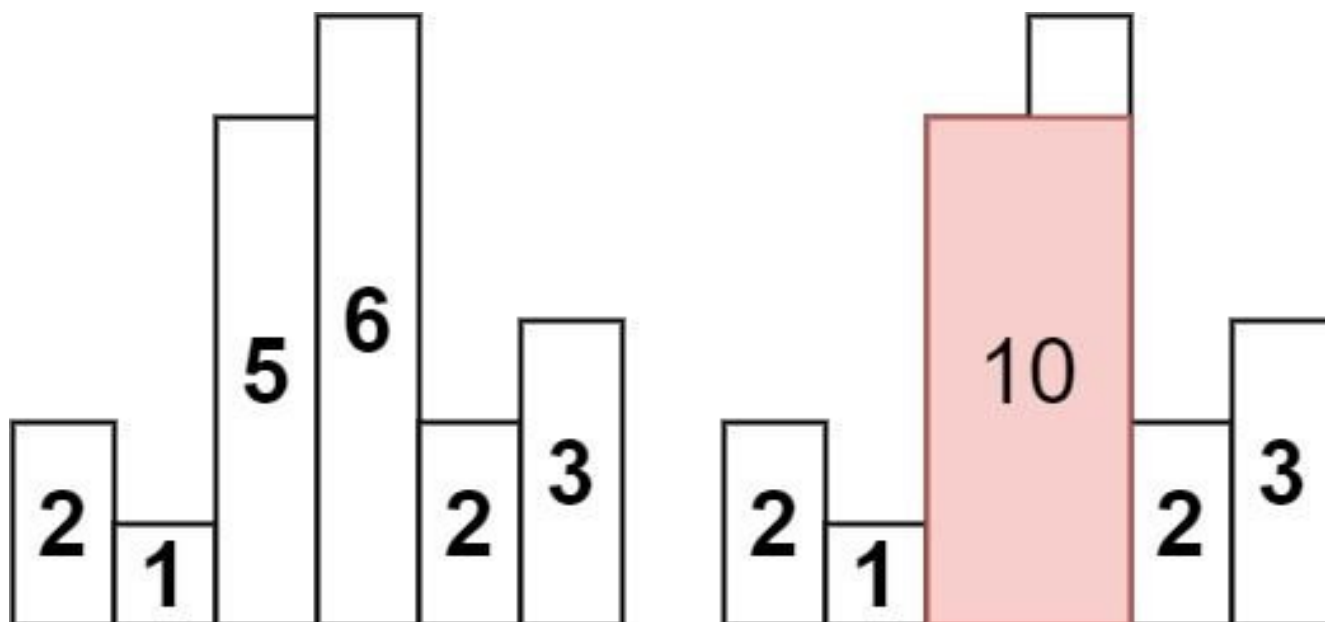
HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: Grind 169

Problem

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Example 1:



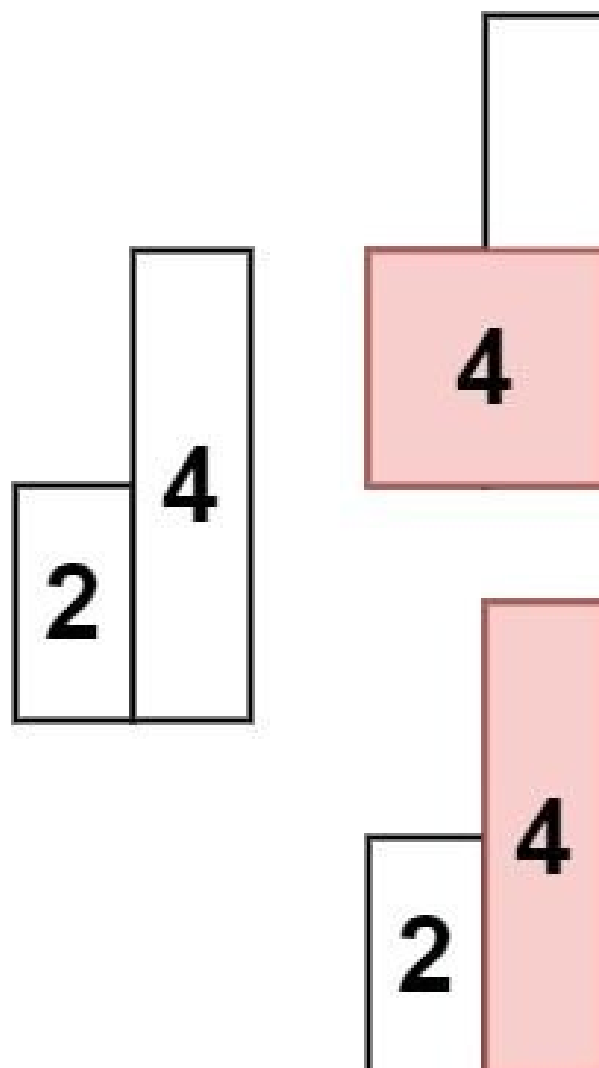
Input: heights = [2,1,5,6,2,3]

Output: 10

Explanation: The above is a histogram where width of each bar is 1.

The largest rectangle is shown in the red area, which has an area = 10 units.

Example 2:



Input: heights = [2,4]

Output: 4

Constraints:

- $1 \leq \text{heights.length} \leq 105$
- $0 \leq \text{heights}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def largestRectangleArea(self, heights: list[int]) -> int:
        ans = 0
        stack = []

        for i in range(len(heights) + 1):
            while stack and (i == len(heights) or heights[stack[-1]] > heights[i]):
                h = heights[stack.pop()]
                w = i - stack[-1] - 1 if stack else i
                ans = max(ans, h * w)

            stack.append(i)

        return ans
```

• LeetDoocs

```
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        n = len(heights)
        stk = []
        left = [-1] * n
        right = [n] * n
        for i, h in enumerate(heights):
            while stk and heights[stk[-1]] >= h:
                right[stk[-1]] = i
                stk.pop()

            if stk:
                left[i] = stk[-1]
            stk.append(i)
        return max(h * (right[i] - left[i] - 1) for i, h in enumerate(heights))
```

• SimplyLeet


```

    right = [n] * n
    for i, h in enumerate(heights):
        while stk and heights[stk[-1]] >= h:
            right[stk[-1]] = i
            stk.pop()
        if stk:
            left[i] = stk[-1] # same as below: stk[-1] in 'i - stack[-1] -
1'
        stk.append(i)

    # valid for one element input [3]

    return max(h * (right[i] - left[i] - 1) for i, h in enumerate(heights)
)

#####

class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        if not heights:
            return 0

        heights.append(-1) # make for loop running
        stack = []
        ans = 0
        for i in range(0, len(heights)):
            while stack and heights[i] < heights[stack[-1]]:
                currentLowestBarIndex = stack.pop()
                h = heights[currentLowestBarIndex]
                # stack[-1] is after pop(). why not i-currentLowestBarIndex?
                # because not working for tie case in the stack,
                # eg. for input=[4,2,0,3,2,5], expected=6 but output=5

```

```

        # 计算当前高度 h 的左右边界 left_i, right_i
        # 计算当前高度 h 的宽度 right-left+1
        # 计算当前高度 h 的左右边界 right-left+1 和当前高度 h 的左右边界 index+1 -2 和 -1
        w = i - stack[-1] - 1 if stack else i
        ans = max(ans, h * w)
        stack.append(i)

    heights.pop() # restore, pop -1
    return ans

```

◆ Quick Review

Key Insights

- Each bar can potentially extend left and right as long as the neighboring bars are not shorter.
- A monotonic (increasing) stack can be used to efficiently track indices of bars.
- When encountering a bar lower than the one on the top of the stack, pop the stack and calculate the area of the rectangle formed with the popped bar height.
- A dummy bar (of height 0) is appended at the end to ensure all bars are processed.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution employs a monotonic stack to traverse the histogram once. For each bar, while the current bar's height is less than the height at the top index of the stack, it pops from the stack and calculates the area using the popped height as the smallest bar. The width for the rectangle is determined by the difference between the current index and the index from the stack (or the current index if the stack is empty). This effectively finds the maximum rectangular area possible. A sentinel value (0) is added at the end of the histogram to flush any remaining bars from the stack.

Heap / Priority Queue

Round 6 · Mid · Hard · 1 question

Heap / Priority Queue

□ #295 Find Median from Data Stream

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · Design · Sorting · Heap
Lists: Grind 169

Problem

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for $\text{arr} = [2,3,4]$, the median is 3.
- For example, for $\text{arr} = [2,3]$, the median is $(2 + 3) / 2 = 2.5$.

Implement the MedianFinder class:

- `MedianFinder()` initializes the MedianFinder object.
- `void addNum(int num)` adds the integer num from the data stream to the data structure.
- `double findMedian()` returns the median of all elements so far. Answers within 10^{-5} of

the actual answer will be accepted.

Example 1:

Input

```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
```

```
[[], [1], [2], [], [3], []]
```

Output

```
[null, null, null, 1.5, null, 2.0]
```

Explanation

```
MedianFinder medianFinder = new MedianFinder();
```

Constraints:

- $-105 \leq \text{num} \leq 105$
- There will be at least one element in the data structure before calling findMedian.
- At most $5 * 10^4$ calls will be made to addNum and findMedian.

Follow up:

- If all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?
- If 99% of all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?

Community Solutions (Python)

- WalkCC

```

class MedianFinder:
    def __init__(self):
        self.maxHeap = []
        self.minHeap = []

    def addNum(self, num: int) -> None:
        if not self.maxHeap or num <= -self.maxHeap[0]:
            heapq.heappush(self.maxHeap, -num)
        else:
            heapq.heappush(self.minHeap, num)

        # Balance the two heaps s.t.
        # |maxHeap| >= |minHeap| and |maxHeap| - |minHeap| <= 1.
        if len(self.maxHeap) < len(self.minHeap):
            heapq.heappush(self.maxHeap, -heapq.heappop(self.minHeap))
        elif len(self.maxHeap) - len(self.minHeap) > 1:
            heapq.heappush(self.minHeap, -heapq.heappop(self.maxHeap))

    def findMedian(self) -> float:
        if len(self.maxHeap) == len(self.minHeap):
            return (-self.maxHeap[0] + self.minHeap[0]) / 2.0
        return -self.maxHeap[0]

```

• LeetDoocs

```

class MedianFinder:

    def __init__(self):
        self.minq = []
        self.maxq = []

    def addNum(self, num: int) -> None:
        heappush(self.minq, -heappushpop(self.maxq, -num))
        if len(self.minq) - len(self.maxq) > 1:
            heappush(self.maxq, -heappop(self.minq))

```

```
def findMedian(self) -> float:
    if len(self.minq) == len(self.maxq):
        return (self.minq[0] - self.maxq[0]) / 2
    return self.minq[0]
```

```
# Your MedianFinder object will be instantiated and called as such:
# obj = MedianFinder()
# obj.addNum(num)
```

```
# param_2 = obj.findMedian()
```

• SimplyLeet

```
import heapq
```

```
class MedianFinder:
```

```
    def __init__(self):
```

```
        # max-heap for the lower half (store negatives to simulate max-heap)
```

```
        self.lowerHalf = []
```

```
        # min-heap for the upper half
```

```
        self.upperHalf = []
```

```
    def addNum(self, num: int) -> None:
```

```
        # Push to max-heap (simulate by pushing negative)
```

```
        heapq.heappush(self.lowerHalf, -num)
```

```
        # Ensure every element in lowerHalf is <= every element in upperHalf
```

```
        # Move maximum of lowerHalf to upperHalf
```

```
        heapq.heappush(self.upperHalf, -heapq.heappop(self.lowerHalf))
```

```
        # Balance the heaps: if upperHalf exceeds lowerHalf in size
```

```
        if len(self.upperHalf) > len(self.lowerHalf):
```

```
            heapq.heappush(self.lowerHalf, -heapq.heappop(self.upperHalf))
```

```

def findMedian(self) -> float:
    # If both heaps are balanced, median is average of both heaps' top
    elements
    if len(self.lowerHalf) == len(self.upperHalf):
        return (-self.lowerHalf[0] + self.upperHalf[0]) / 2.0
    # Otherwise, median is the top element of the larger heap
    return -self.lowerHalf[0]

# Example usage:
# medianFinder = MedianFinder()
# medianFinder.addNum(1)

# medianFinder.addNum(2)
# print(medianFinder.findMedian()) # Outputs 1.5
# medianFinder.addNum(3)
# print(medianFinder.findMedian()) # Outputs 2.0

```

• LC.ca

```

from heapq import heappush, heappop

class MedianFinder:

    def __init__(self):
        # the smaller half of the list, max heap (invert min-heap)
        self.smallerHalf = []
        # the larger half of the list, min heap
        self.largerHalf = []

    def addNum(self, num: int) -> None:

        # trick for smaller half, use -1*val
        # heapq in python does NOT have comparator like in Java
        heappush(self.smallerHalf, -num)
        heappush(self.largerHalf, -heappop(self.smallerHalf)) # note: not
        self.smallerHalf.pop()

        if len(self.smallerHalf) < len(self.largerHalf):
            heappush(self.smallerHalf, -heappop(self.largerHalf))

```

```
def findMedian(self) -> float:
    if len(self.smallerHalf) == len(self.largerHalf):
        return (-self.smallerHalf[0] + self.largerHalf[0]) / 2.0
    else:
        return float(-self.smallerHalf[0])

# Your MedianFinder object will be instantiated and called as such:
# obj = MedianFinder()
# obj.addNum(num)
```

```
# param_2 = obj.findMedian()
```

```
# follow up
```

```
class MedianFinder:
    def __init__(self):
        self.counts = [0] * 101
        self.total = 0

    def addNum(self, num: int) -> None:
```

```
        self.counts[num] += 1
        self.total += 1
```

```
    def findMedian(self) -> float:
        if self.total % 2 == 0:
            # even number of elements
            middle1 = self.total // 2
            middle2 = middle1 + 1
            count = 0
            i1 = i2 = 0
```

```
            for i in range(101):
                count += self.counts[i]
                if count >= middle1 and i1 == 0:
                    i1 = i
                if count >= middle2:
                    i2 = i
                    break
            return (i1 + i2) / 2
        else:
            # odd number of elements
```

```

        middle = self.total // 2 + 1
        count = 0
        for i in range(101):
            count += self.counts[i]
            if count >= middle:
                return i

#####

class MedianFinder:
    def __init__(self):
        """
        initialize your data structure here.
        """
        self.h1 = []
        self.h2 = []

    def addNum(self, num: int) -> None:
        heappush(self.h1, num)

        heappush(self.h2, -heappop(self.h1))
        if len(self.h2) - len(self.h1) > 1:
            heappush(self.h1, -heappop(self.h2))

    def findMedian(self) -> float:
        if len(self.h2) > len(self.h1):
            return -self.h2[0]
        return (self.h1[0] - self.h2[0]) / 2

# Your MedianFinder object will be instantiated and called as such:

# obj = MedianFinder()
# obj.addNum(num)
# param_2 = obj.findMedian()

```

◆ Quick Review

Key Insights

- Use two heaps: a max-heap for the lower half and a min-heap for the upper half of numbers.
- Balancing the heaps ensures the median can be computed in $O(1)$ time once numbers are added.
- Rebalance the heaps after each insertion to ensure size properties hold (difference in sizes is at most 1).

Space & Time

Time Complexity:

addNum: $O(\log n)$ per insertion (heap operations).

findMedian: $O(1)$ (accessing the top elements).

Space Complexity:

$O(n)$, where n is the number of elements added in the data stream, as they are stored in the two heaps.

Solution

We maintain two heaps: A max-heap (lowerHalf) for the smaller half of the numbers. A min-heap (upperHalf) for the larger half of the numbers. On inserting a new

number, decide the heap membership based on the current median. After each insertion, ensure that the heaps have balanced sizes (the size difference is at most one). When computing the median: If both heaps have the same number of elements, the median is the average of the max of lowerHalf and the min of upperHalf. If one heap has more elements, the median is the top element of that heap. This method makes it efficient to dynamically update and retrieve the median as new numbers are inserted.

Tries

Round 6 · Mid · Hard · 1 question

Tries

□ #212 Word Search II

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Backtracking · Trie · Matrix
Lists: Grind 169

Problem

Given an $m \times n$ board of characters and a list of strings words, return all words on the board.

Each word must be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

Example 1:

o	a	a	n
e	t	a	e
i	h	k	r
i	f	l	v

Input: board =
[["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]],
words = ["oath","pea","eat","rain"]
Output: ["eat","oath"]

Example 2:

a	b
c	d

Input: board = [["a","b"],["c","d"]], words = ["abcb"]
Output: []

Constraints:

- $m == \text{board.length}$
- $n == \text{board}[i].\text{length}$
- $1 \leq m, n \leq 12$
- $\text{board}[i][j]$ is a lowercase English letter.
- $1 \leq \text{words.length} \leq 3 * 10^4$
- $1 \leq \text{words}[i].\text{length} \leq 10$

- words[i] consists of lowercase English letters.
- All the strings of words are unique.

Community Solutions (Python)

• WalkCC

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.word: str | None = None

class Solution:
    def findWords(self, board: list[list[str]], words: list[str]) -> list[str]:
        m = len(board)
        n = len(board[0])

        ans = []
        root = TrieNode()

        def insert(word: str) -> None:
            node = root
            for c in word:
                node = node.children.setdefault(c, TrieNode())
            node.word = word

        for word in words:
            insert(word)

        def dfs(i: int, j: int, node: TrieNode) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return
            if board[i][j] == '*':
                return

            c = board[i][j]
            if c not in node.children:
```

```

        return

    child = node.children[c]
    if child.word:
        ans.append(child.word)
        child.word = None

    board[i][j] = '*'
    dfs(i + 1, j, child)
    dfs(i - 1, j, child)

    dfs(i, j + 1, child)
    dfs(i, j - 1, child)
    board[i][j] = c

    for i in range(m):
        for j in range(n):
            dfs(i, j, root)

    return ans

```

• SimplyLeet

```

# Define a TrieNode for building the Trie.
class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None # Stores complete word at the end node

class Solution:
    def findWords(self, board, words):
        # Build the Trie from the list of words.
        root = TrieNode()

```



```

for word in words:
    node = root
    for char in word:
        if char not in node.children:
            node.children[char] = TrieNode()
        node = node.children[char]
    node.word = word # Mark the end of a word

result = []
m, n = len(board), len(board[0])

# Define the DFS helper function.
def dfs(i, j, node):
    char = board[i][j]
    if char not in node.children:
        return
    next_node = node.children[char]
    if next_node.word:
        result.append(next_node.word)
        next_node.word = None # Avoid duplicate entries

    board[i][j] = '#' # Mark the current cell as visited
    # Explore all four adjacent directions.
    for dx, dy in ((0, 1), (1, 0), (0, -1), (-1, 0)):
        ni, nj = i + dx, j + dy
        if 0 <= ni < m and 0 <= nj < n and board[ni][nj] != '#':
            dfs(ni, nj, next_node)
    board[i][j] = char # Restore the original character

# Start DFS from each cell.

for i in range(m):
    for j in range(n):
        dfs(i, j, root)
return result

```

• LC.ca

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.w = ''
        # minor ajust, not a boolean is_end, but the whole word
        # so it's easier for dfs to save the word

    def insert(self, w):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.w = w

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
        :
        def dfs(node, i, j):
            idx = ord(board[i][j]) - ord('a')
            if node.children[idx] is None:
                return
            node = node.children[idx]
            if node.w:
                ans.add(node.w)
            c = board[i][j]
            board[i][j] = '0' # 0 for visited already
            for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
                x, y = i + a, j + b
```

```

        if 0 <= x < m and 0 <= y < n and board[x][y] != '0':
            dfs(node, x, y)
        board[i][y] = c

```

```

trie = Trie()
for w in words:
    trie.insert(w)
ans = set()
m, n = len(board), len(board[0])
for i in range(m):

```

```

    for j in range(n):
        dfs(trie, i, j)
return list(ans)

```

```
#####
```

```

class Trie:
    def __init__(self):

```

```

        self.children: List[Trie | None] = [None] * 26
        self.ref: int = -1

```

```

    def insert(self, w: str, ref: int):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]

```

```

node.ref = ref

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]
    :
        def dfs(node: Trie, i: int, j: int):
            idx = ord(board[i][j]) - ord('a')
            if node.children[idx] is None:
                return
            node = node.children[idx]

            if node.ref >= 0:
                ans.append(words[node.ref])
                node.ref = -1
            c = board[i][j]
            board[i][j] = '#'
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and board[x][y] != '#':
                    dfs(node, x, y)
            board[i][j] = c

        tree = Trie()
        for i, w in enumerate(words):
            tree.insert(w, i)
        m, n = len(board), len(board[0])
        ans = []
        for i in range(m):
            for j in range(n):
                dfs(tree, i, j)
        return ans

```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to explore possible words starting from each board cell.

- Build a Trie (prefix tree) to store the words for $O(1)$ prefix checks and efficient pruning.
- Mark visited cells and restore them after DFS to avoid revisiting in the current search path.
- Prune search paths early if the current prefix is not part of any word in the Trie.

Space & Time

Time Complexity: $O(m * n * 4^L)$ in the worst case, where L is the maximum word length; however, using a Trie prunes unnecessary paths.

Space Complexity: $O(K)$ for the Trie storage, where K is the total number of letters in all words, plus $O(m * n)$ auxiliary space for recursion and board marking.

Solution

We first insert all words into a Trie data structure to allow quick lookups of prefixes. Then, for each cell in the board, we perform DFS. During DFS, if the current path matches a Trie node that represents a complete word, we add that word to the result list and mark it as found to avoid duplicates. We continue exploring adjacent cells (up, down, left, right)

while marking the cell as visited. After exploring, we restore the original letter for further DFS calls. This combination of Trie and DFS with backtracking efficiently finds valid words on the board.

Backtracking

Round 6 · Mid · Hard · 1 question

Backtracking

□ #51 N-Queens

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking
Lists: Grind 169

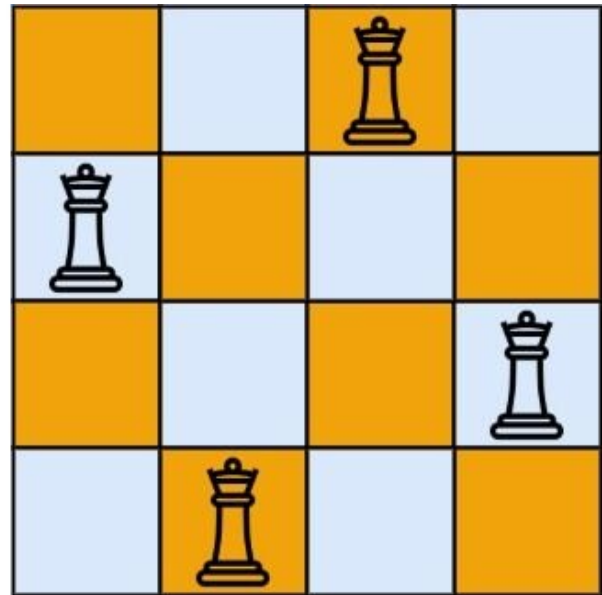
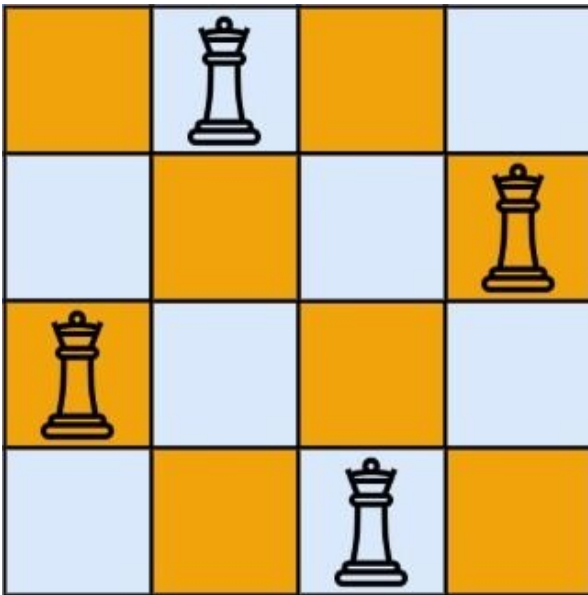
Problem

The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other.

Given an integer n, return all distinct solutions to the n-queens puzzle. You may return the answer in any order.

Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Example 1:



Input: $n = 4$

Output: `[[".Q...", "...Q", "Q...", "..Q."], ["..Q.", "Q...", "...Q", ".Q..."]]`

Explanation: There exist two distinct solutions to the 4-queens puzzle as shown above

Example 2:

Input: $n = 1$

Output: `[["Q"]]`

Constraints:

- $1 \leq n \leq 9$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def solveNQueens(self, n: int) -> list[list[str]]:
        ans = []
        cols = [False] * n
        diag1 = [False] * (2 * n - 1)
        diag2 = [False] * (2 * n - 1)

        def dfs(i: int, board: list[int]) -> None:
            if i == n:
                ans.append(board)

            return

        for j in range(n):
            if cols[j] or diag1[i + j] or diag2[j - i + n - 1]:
                continue
            cols[j] = diag1[i + j] = diag2[j - i + n - 1] = True
            dfs(i + 1, board + ['. ' * j + 'Q' + ' ' * (n - j - 1)])
            cols[j] = diag1[i + j] = diag2[j - i + n - 1] = False

        dfs(0, [])

    return ans

```

• LeetDoocs

```

class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        def dfs(i: int):
            if i == n:
                ans.append(''.join(row) for row in g)
                return
            for j in range(n):
                if col[j] + dg[i + j] + udg[n - i + j] == 0:
                    g[i][j] = "Q"
                    col[j] = dg[i + j] = udg[n - i + j] = 1

```

```

        dfs(i + 1)
        col[j] = dg[i + j] = udg[n - i + j] = 0
        g[i][j] = "."

    ans = []
    g = [["."] * n for _ in range(n)]
    col = [0] * n
    dg = [0] * (n << 1)
    udg = [0] * (n << 1)
    dfs(0)

    return ans

```

• SimplyLeet

```

def solveNQueens(n):
    result = []

    # Helper function to backtrack
    def backtrack(row, columns, diag1, diag2, board):
        # Base case: if all rows are filled, add a copy of board to result
        if row == n:
            result.append(list(board))
            return

        # Try to place a queen in each column in the current row
        for col in range(n):
            # Check if the column or diagonals are not under attack
            if col in columns or (row - col) in diag1 or (row + col) in diag2:
                continue

            # Place the queen
            board[row] = '.' * col + 'Q' + '.' * (n - col - 1)
            # Mark the column and diagonals as occupied
            columns.add(col)

```

```

diag1.add(row - col)
diag2.add(row + col)

# Move to the next row
backtrack(row + 1, columns, diag1, diag2, board)

# Backtrack: remove the queen and unmark the sets
columns.remove(col)
diag1.remove(row - col)
diag2.remove(row + col)

```

```

# Initialize board with empty rows
board = ["." * n for _ in range(n)]
backtrack(0, set(), set(), set(), board)
return result

```

```

# Example usage:
print(solveNQueens(4))

```

• LC.ca

```

class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        def dfs(i: int):
            if i == n:
                ans.append(["".join(row) for row in g])
                return
            for j in range(n):
                if col[j] + dg[i + j] + udg[n - i + j] == 0:
                    g[i][j] = "Q"
                    col[j] = dg[i + j] = udg[n - i + j] = 1

                    dfs(i + 1)
                    col[j] = dg[i + j] = udg[n - i + j] = 0
                    g[i][j] = "."

        ans = []
        g = [". " * n for _ in range(n)]
        col = [0] * n
        dg = [0] * (n << 1)
        udg = [0] * (n << 1)
        dfs(0)

```

```
return ans
```

◆ Quick Review

Key Insights

- Use backtracking to explore potential positions row by row.
- Use sets to track which columns and diagonals are under attack.
- Diagonals can be tracked by (row – col) and (row + col), ensuring that queens do not conflict along any diagonal.
- Recursively build each valid board configuration and backtrack when a conflict arises.

Space & Time

Time Complexity: $O(n!)$ in the worst-case scenario, as the solution explores permutations for queen placements.

Space Complexity: $O(n^2)$ for storing board configurations and $O(n)$ for recursion stack and conflict-tracking sets.

Solution

We use a backtracking algorithm that places queens row by row. For each row, we try placing a queen in each column. Before placing, we check if the column, main diagonal ($\text{row} - \text{col}$), and anti-diagonal ($\text{row} + \text{col}$) are free using dedicated sets. If a queen can be safely placed, we mark the column and diagonals as occupied and recurse to the next row. Once all rows are processed, the board configuration is a valid solution and is added to the result. After that, we backtrack by unmarking the sets and removing the queen from the board. This systematic approach ensures all possible configurations are considered.

Round 7 · Low · Easy

Round 7

Low Priority · Easy

9 questions · 3 patterns

1-D Dynamic Programming #70 #746

Bit Manipulation #136 #190 #191 #268 #338

Math & Geometry #66 #202

1-D Dynamic Programming

Round 7 · Low · Easy · 2 questions

1-D Dynamic Programming

□ #70 Climbing Stairs

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Memoization
Lists: Grind 169

Problem

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

```
Input: n = 2
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps
```

Example 2:

```
Input: n = 3
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step
```

Constraints:

- $1 \leq n \leq 45$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def climbStairs(self, n: int) -> int:
        # dp[i] := the number of ways to climb to the i-th stair
        dp = [1, 1] + [0] * (n - 1)

        for i in range(2, n + 1):
            dp[i] = dp[i - 1] + dp[i - 2]

        return dp[n]
```

```
class Solution:
    def climbStairs(self, n: int) -> int:
        prev1 = 1 # dp[i - 1]
        prev2 = 1 # dp[i - 2]

        for _ in range(2, n + 1):
            dp = prev1 + prev2
            prev2 = prev1
            prev1 = dp

        return prev1
```

• LeetDoocs

```
class Solution:
    def climbStairs(self, n: int) -> int:
        a, b = 0, 1
        for _ in range(n):
            a, b = b, a + b
        return b
```

```
import numpy as np

class Solution:
    def climbStairs(self, n: int) -> int:
        res = np.asmatrix([(1, 1)], np.dtype("0"))
        factor = np.asmatrix([(1, 1), (1, 0)], np.dtype("0"))
        n -= 1
        while n:
            if n & 1:

                res *= factor
                factor *= factor
                n >>= 1
        return res[0, 0]
```

• SimplyLeet

```
# Define a function to compute the number of ways to climb stairs.
def climbStairs(n):
    # Base case: if there is only one step, return 1.
    if n == 1:
        return 1
    # Initialize the dp array with n+1 elements.
    dp = [0] * (n + 1)
    # There is 1 way to climb 1 step.
    dp[1] = 1
    # There are 2 ways to climb 2 steps.

    dp[2] = 2
    # Compute the number of ways for each step from 3 to n.
    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2] # Sum of ways from the previous two
steps.
    return dp[n]

# Example usage:
print(climbStairs(3)) # Output: 3
```

• LC.ca

```
class Solution:
    def climbStairs(self, n: int) -> int:
        a, b = 0, 1
        for _ in range(n):
            a, b = b, a + b
        return b
```

◆ Quick Review

Key Insights

- The problem can be solved using dynamic programming because the solution for a given step depends on the solutions of the previous two steps.
- The recurrence relation is: $\text{ways}[i] = \text{ways}[i-1] + \text{ways}[i-2]$.
- Base cases: when there is 1 step, there is 1 way; when there are 2 steps, there are 2 ways.
- This is analogous to the Fibonacci sequence, shifting indices accordingly.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ (can be optimized to $O(1)$ using constant space)

Solution

We use a dynamic programming approach to build a solution gradually. The idea is that to reach step i , you could have come from step $i-1$ (taking one step) or from step $i-2$ (taking two steps). Therefore, the total number of ways to reach step i is the sum of ways to reach step $i-1$ and $i-2$. We initialize $dp[1] = 1$ and $dp[2] = 2$, and iterate from 3 to n to fill $dp[]$. Each code solution below uses this approach with comments to explain the logic.

1-D Dynamic Programming

□ #746 Min Cost Climbing Stairs

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: NeetCode 150

Problem

You are given an integer array `cost` where `cost[i]` is the cost of *i*th step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

Return the minimum cost to reach the top of the floor.

Example 1:

```
Input: cost = [10,15,20]
Output: 15
Explanation: You will start at index 1.
- Pay 15 and climb two steps to reach the top.
The total cost is 15.
```

Example 2:

Input: cost = [1,100,1,1,1,100,1,1,100,1]

Output: 6

Explanation: You will start at index 0.

- Pay 1 and climb two steps to reach index 2.
- Pay 1 and climb two steps to reach index 4.
- Pay 1 and climb two steps to reach index 6.
- Pay 1 and climb one step to reach index 7.
- Pay 1 and climb two steps to reach index 9.

Constraints:

- $2 \leq \text{cost.length} \leq 1000$
- $0 \leq \text{cost}[i] \leq 999$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minCostClimbingStairs(self, cost: list[int]) -> int:
        cost.append(0)

        for i in range(2, len(cost)):
            cost[i] += min(cost[i - 1], cost[i - 2])

        return cost[-1]
```

• LeetDoocs

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        @cache
        def dfs(i: int) -> int:
            if i >= len(cost):
                return 0
            return cost[i] + min(dfs(i + 1), dfs(i + 2))

        return min(dfs(0), dfs(1))
```

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        n = len(cost)
        f = [0] * (n + 1)
        for i in range(2, n + 1):
            f[i] = min(f[i - 2] + cost[i - 2], f[i - 1] + cost[i - 1])
        return f[n]
```

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        f = g = 0
        for i in range(2, len(cost) + 1):
            f, g = g, min(f + cost[i - 2], g + cost[i - 1])
        return g
```

• SimplyLeet

Python implementation with detailed comments

```
def minCostClimbingStairs(cost):
    n = len(cost) # number of steps
    # dp array to store minimum cost to reach each step
    dp = [0] * n
    dp[0] = cost[0] # cost to start at step 0
    dp[1] = cost[1] # cost to start at step 1

    # Compute minimum cost for each subsequent step

    for i in range(2, n):
        # cost to reach current step is current cost + min cost of the two
        # previous steps
        dp[i] = cost[i] + min(dp[i-1], dp[i-2])

    # The top can be reached from the last or second last step
    return min(dp[n-1], dp[n-2])

# Example usage:
cost = [10, 15, 20]
print(minCostClimbingStairs(cost))
```

◆ Quick Review

Key Insights

- Use Dynamic Programming to solve overlapping subproblems.
- The minimum cost to reach a step is the cost of that step plus the minimum cost of reaching one of the two previous steps.
- The final result is the minimum cost to reach the last or second last step since you can jump from either directly to the top.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ (can be optimized to $O(1)$ by using only two variables)

Solution

We use a dynamic programming approach. Define a dp array where $dp[i]$ is the minimum cost required to reach step i . Initialize $dp[0]$ with $cost[0]$ and $dp[1]$ with $cost[1]$ because you can start at either step. For each subsequent step, the recurrence is $dp[i] = cost[i] + \min(dp[i-1], dp[i-2])$. Finally, the answer is the minimum of $dp[n-1]$ and $dp[n-2]$ since you can reach the top from either of these steps. This method efficiently

computes the answer in one pass through the array.

Bit Manipulation

Round 7 · Low · Easy · 5 questions

Bit Manipulation

□ #136 Single Number

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Bit Manipulation
Lists: Grind 169

Problem

Given a non-empty array of integers `nums`, every element appears twice except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: `nums = [2,2,1]`

Output: 1

Example 2:

Input: `nums = [4,1,2,1,2]`

Output: 4

Example 3:

Input: `nums = [1]`

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-3 * 10^4 \leq \text{nums}[i] \leq 3 * 10^4$
- Each element in the array appears twice except for one element which appears only once.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        return functools.reduce(operator.xor, nums, 0)
```

• LeetDoocs

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        return reduce(xor, nums)
```

• SimplyLeet

```
# Function to find the single number in the list
def singleNumber(nums):
    # Initialize result to 0; any number XOR 0 is the number itself
    result = 0
    # Iterate through every number in the list
    for num in nums:
        # XOR the current result with the current number
        result ^= num # Using XOR to cancel duplicates
    # Return the unique number that did not cancel out
    return result
```

```
# Example usage
nums = [4, 1, 2, 1, 2]
print(singleNumber(nums)) # Output should be 4
```

• LC.ca

```
'''
>>> from functools import reduce
>>> reduce(lambda x, y: x ^ y, [3, 5, 3])
5
'''

from functools import reduce

class Solution:
    def singleNumber(self, nums: List[int]) -> int:

        return reduce(lambda x, y: x ^ y, nums)

#####

class Solution(object):
    def singleNumber(self, nums):
        '''
        :type nums: List[int]
        :rtype: int
        '''

        for i in range(1, len(nums)):
            nums[0] ^= nums[i]
        return nums[0]
```

◆ Quick Review

Key Insights

- Using bitwise XOR: XOR-ing a number with itself cancels out to 0.

- XOR is both commutative and associative, so the order of operations does not matter.
- A single traversal (linear time) and constant extra space is sufficient for solving the problem.

Space & Time

Time Complexity: $O(n)$ – We only traverse the array once.

Space Complexity: $O(1)$ – Only a single extra variable is used regardless of input size.

Solution

We can solve this problem by initializing a variable to 0 and then traversing the array while XOR-ing each element with the variable. Due to the properties of XOR, pairs of identical numbers will cancel each other out ($x \text{ XOR } x = 0$) and the unique number will be left as the final result. This approach uses constant extra space and performs in linear time.

Bit Manipulation

□ #190 Reverse Bits

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Divide and Conquer · Bit Manipulation
Lists: Grind 169

Problem

Reverse bits of a given 32 bits signed integer.

Example 1:

Input: $n = 43261596$

Output: 964176192

Explanation:

Example 2:

Input: $n = 2147483644$

Output: 1073741822

Explanation:

Constraints:

- $0 \leq n \leq 2^{31} - 2$
- n is even.

Follow up: If this function is called many times, how would you optimize it?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverseBits(self, n: int) -> int:
        ans = 0

        for i in range(32):
            if n >> i & 1:
                ans |= 1 << 31 - i

        return ans
```

• LeetDoocs

```
class Solution:
    def reverseBits(self, n: int) -> int:
        ans = 0
        for i in range(32):
            ans |= (n & 1) << (31 - i)
            n >>= 1
        return ans
```

• SimplyLeet

```
# Function to reverse bits of a 32-bit binary string.
def reverse_bits(n: str) -> int:
    # Reverse the binary string using slicing.
    reversed_binary = n[::-1]
    # Convert the reversed binary string to an integer.
    result = int(reversed_binary, 2)
    return result

# Example usage:
binary_input = "00000010100101000001111010011100"

print(reverse_bits(binary_input)) # Expected output: 964176192
```

• LC.ca

```
class Solution:
    def reverseBits(self, n: int) -> int:
        res = 0
        for i in range(32):
            res |= (n & 1) << (31 - i)
            n >>= 1
        return res
```

◆ Quick Review

Key Insights

- The input is a binary string of fixed length (32 bits).
- Reversing the bits is equivalent to reversing the string.
- The reversed binary string can be converted back into its integer representation.
- Since the number of bits is constant (32), the solution can be achieved in constant time and constant space.
- For performance optimization when this function is called many times, caching may be applied for common intermediate results.

Space & Time

Time Complexity: $O(1)$ – We always process 32 bits regardless of the input.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

We solve the problem by reversing the 32-character binary string representing the unsigned integer. Once reversed, the new binary string is converted into its integer value. An alternative solution involves using bitwise operations where you iterate over each bit, shift the result to the left, and add the current bit. This approach does the reversal one bit at a time. The chosen approach leverages string reversal which is straightforward to implement and understand.

Bit Manipulation

□ #191 Number of 1 Bits

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Divide and Conquer · Bit Manipulation
Lists: Grind 169**

Problem

Given a positive integer n , write a function that returns the number of set bits in its binary representation (also known as the Hamming weight).

Example 1:

Input: $n = 11$

Output: 3

Explanation:

The input binary string 1011 has a total of three set bits.

Example 2:

Input: $n = 128$

Output: 1

Explanation:

The input binary string 10000000 has a total of one set bit.

Example 3:

Input: n = 2147483645

Output: 30

Explanation:

The input binary string

11111111111111111111111111111111101 has a total of thirty set bits.

Constraints:

- $1 \leq n \leq 2^{31} - 1$

Follow up: If this function is called many times, how would you optimize it?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        ans = 0

        for i in range(32):
            if (n >> i) & 1:
                ans += 1

        return ans
```

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        return n.bit_count()
```

• LeetCode

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        ans = 0
        while n:
            n &= n - 1
            ans += 1
        return ans
```

• SimplyLeet

```
# Function to count the number of set bits in a given integer n
def hammingWeight(n: int) -> int:
    count = 0 # Initialize count of set bits
    while n:
        # Remove the lowest set bit from n
        n &= n - 1
        count += 1 # Increment count for each set bit removed
    return count
```

```
# Example usage:
```

```
print(hammingWeight(11)) # Output: 3
```

• LC.ca

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        ans = 0
        while n:
            n &= n - 1
            ans += 1
        return ans
```

◆ Quick Review

Key Insights

- The problem is essentially counting the number of 1s in the binary representation of a number.
- A common efficient approach uses bit manipulation: repeatedly clear the lowest set bit until n becomes zero.
- The expression $n \& (n - 1)$ clears the lowest set bit of n .
- This method only iterates as many times as there are set bits, making it efficient.

Space & Time

Time Complexity: $O(k)$, where k is the number of set bits, worst-case $O(\log n)$ (or $O(1)$ considering 32-bit integers).

Space Complexity: $O(1)$.

Solution

To solve the problem, we use the bit manipulation trick $n \& (n - 1)$ which removes the lowest set bit from n . The algorithm is as follows: Initialize a counter to 0. While n is not zero, update n by $n = n \& (n - 1)$, and increment the counter. The counter will hold the number of set bits by the end of the loop. This approach leverages efficient bit-level

operations and works well even when the function is called frequently.

Bit Manipulation

□ #268 Missing Number

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Binary Search · Bit
Manipulation**

Lists: Grind 169

Problem

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return the only number in the range that is missing from the array.

Example 1:

Input: `nums = [3,0,1]`

Output: 2

Explanation:

`n = 3` since there are 3 numbers, so all numbers are in the range `[0,3]`. 2 is the missing number in the range since it does not appear in `nums`.

Example 2:

Input: `nums = [0,1]`

Output: 2

Explanation:

$n = 2$ since there are 2 numbers, so all numbers are in the range $[0,2]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 3:

Input: `nums = [9,6,4,2,3,5,7,0,1]`

Output: 8

Explanation:

$n = 9$ since there are 9 numbers, so all numbers are in the range $[0,9]$. 8 is the missing number in the range since it does not appear in `nums`.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 10^4$
- $0 \leq \text{nums}[i] \leq n$
- All the numbers of `nums` are unique.

Follow up: Could you implement a solution using only $O(1)$ extra space complexity and $O(n)$ runtime complexity?

Community Solutions (Python)

- WalkCC

```
class Solution:
    def missingNumber(self, nums: list[int]) -> int:
        ans = len(nums)

        for i, num in enumerate(nums):
            ans ^= i ^ num

        return ans
```

• LeetCode

```
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        return reduce(xor, (i ^ v for i, v in enumerate(nums, 1)))
```

```
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        n = len(nums)
        return (1 + n) * n // 2 - sum(nums)
```

• SimplyLeet

```
# Function to find the missing number in the array
def missingNumber(nums):
    # Calculate n, where n is the length of the nums array
    n = len(nums)
    # Compute the expected total sum using the arithmetic sum formula for
    # numbers 0 to n
    expected_sum = n * (n + 1) // 2
    # Compute the actual sum of the numbers in the array
    actual_sum = sum(nums)
    # The missing number is the difference between expected and actual sums
    return expected_sum - actual_sum
```

```
# Example Usage
print(missingNumber([3, 0, 1])) # Output: 2
```

• LC.ca

```
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        return reduce(xor, (i ^ v for i, v in enumerate(nums, 1)))
```

◆ Quick Review

Key Insights

- The range of numbers is continuous from 0 to n, but one number is missing.
- The sum of numbers from 0 to n can be computed using the arithmetic sum formula.
- Subtracting the sum of the given array from the expected sum gives the missing number.
- There is a follow-up challenge to solve the problem in $O(n)$ time and $O(1)$ extra space.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We compute the expected sum of all numbers from 0 to n using the formula $\text{sum} = n*(n+1)/2$. Then, we compute the actual sum of the numbers in the array. The missing number is obtained by subtracting the actual sum from the expected sum. This approach

**uses arithmetic operations ($O(1)$ extra space)
and loops over the array once ($O(n)$ time).**

Bit Manipulation

□ #338 Counting Bits

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Bit Manipulation
Lists: Grind 169

Problem

Given an integer n , return an array `ans` of length $n + 1$ such that for each i ($0 \leq i \leq n$), `ans[i]` is the number of 1's in the binary representation of i .

Example 1:

```
Input: n = 2
Output: [0,1,1]
Explanation:
0 --> 0
1 --> 1
2 --> 10
```

Example 2:

```
Input: n = 5
Output: [0,1,1,2,1,2]
Explanation:
0 --> 0
1 --> 1
2 --> 10
3 --> 11
4 --> 100
```

Constraints:

- $0 \leq n \leq 105$

Follow up:

- It is very easy to come up with a solution with a runtime of $O(n \log n)$. Can you do it in linear time $O(n)$ and possibly in a single pass?
- Can you do it without using any built-in function (i.e., like `__builtin_popcount` in C++)?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countBits(self, n: int) -> List[int]:
        # f(i) := i's number of 1s in bitmask
        # f(i) = f(i / 2) + i % 2
        ans = [0] * (n + 1)

        for i in range(1, n + 1):
            ans[i] = ans[i // 2] + (i & 1)

        return ans
```

• LeetDoocs

```
class Solution:
    def countBits(self, n: int) -> List[int]:
        return [i.bit_count() for i in range(n + 1)]
```

```
class Solution:
    def countBits(self, n: int) -> List[int]:
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i & (i - 1)] + 1
        return ans
```

• SimplyLeet

```
def countBits(n):
    # Create a list to store the number of 1's for each number from 0 to n
    dp = [0] * (n + 1)
    # Loop over each number starting from 1, since dp[0] is already 0
    for i in range(1, n + 1):
        # dp[i] is computed as the count for i//2 plus 1 if i is odd (i & 1)
        dp[i] = dp[i >> 1] + (i & 1)
    return dp

# Example usage:

# print(countBits(5)) -> [0, 1, 1, 2, 1, 2]
```

• LC.ca

```
class Solution:
    def countBits(self, n: int) -> List[int]:
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i & (i - 1)] + 1
        return ans
```

◆ Quick Review

Key Insights

- We can use dynamic programming to build the solution by utilizing the count of bits for smaller numbers.
- The relationship $dp[i] = dp[i \gg 1] + (i \& 1)$ holds, where $i \gg 1$ gives the count for i divided by 2 (dropping the least significant bit) and $(i \& 1)$ checks if the least significant bit is 1.

- This approach computes the result for each number in constant time, resulting in a linear time solution.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

We use a dynamic programming approach where an array (or list) `dp` is initialized with size $n+1$. The key idea is to iterate from 1 to n and for each number, use the formula $dp[i] = dp[i \gg 1] + (i \& 1)$. The expression $(i \gg 1)$ effectively divides i by 2, giving the count of 1's for that half number. The $(i \& 1)$ operation checks if i is odd, adding 1 if true. This method ensures each $dp[i]$ is computed in constant time, leading to an overall $O(n)$ time complexity.

Math & Geometry

Round 7 · Low · Easy · 2 questions

Math & Geometry

□ #66 Plus One

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math**

Lists: NeetCode 150

Problem

You are given a large integer represented as an integer array `digits`, where each `digits[i]` is the *i*th digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's.

Increment the large integer by one and return the resulting array of digits.

Example 1:

Input: `digits = [1,2,3]`

Output: `[1,2,4]`

Explanation: The array represents the integer 123.

Incrementing by one gives $123 + 1 = 124$.

Thus, the result should be `[1,2,4]`.

Example 2:

Input: digits = [4,3,2,1]

Output: [4,3,2,2]

Explanation: The array represents the integer 4321.

Incrementing by one gives $4321 + 1 = 4322$.

Thus, the result should be [4,3,2,2].

Example 3:

Input: digits = [9]

Output: [1,0]

Explanation: The array represents the integer 9.

Incrementing by one gives $9 + 1 = 10$.

Thus, the result should be [1,0].

Constraints:

- $1 \leq \text{digits.length} \leq 100$
- $0 \leq \text{digits}[i] \leq 9$
- digits does not contain any leading 0's.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def plusOne(self, digits: list[int]) -> list[int]:
        for i, d in reversed(list(enumerate(digits))):
            if d < 9:
                digits[i] += 1
                return digits
            digits[i] = 0

        return [1] + digits
```

• LeetDoocs

```
class Solution:
    def plusOne(self, digits: List[int]) -> List[int]:
        n = len(digits)
        for i in range(n - 1, -1, -1):
            digits[i] += 1
            digits[i] %= 10
            if digits[i] != 0:
                return digits
        return [1] + digits
```

• SimplyLeet

```
# Function to add one to the array of digits
def plusOne(digits):
    # Process digits starting from the least significant digit
    for i in range(len(digits) - 1, -1, -1):
        # Add one to the current digit
        digits[i] += 1
        # Check if digit is less than 10, no carry needed
        if digits[i] < 10:
            return digits # No further carry, return the result
        # Set current digit to 0 because of carry over

        digits[i] = 0
    # If carry remains after processing all digits, add a new digit at start
    return [1] + digits

# Example usage
# print(plusOne([1, 2, 3])) # Output: [1, 2, 4]
```

◆ Quick Review

Key Insights

- The digits are stored from most significant to least significant.
- Start processing from the least significant digit (end of array) to handle potential carry.

- Carefully handle the case where all digits are 9, in which case an additional digit is needed.
- Use an in-place update approach to reduce extra space usage when possible.

Space & Time

Time Complexity: $O(n)$ in the worst-case, where n is the number of digits.

Space Complexity: $O(n)$ in the worst-case for the output array (when an extra digit is added).

Solution

We iterate from the end of the digits array towards the beginning, adding one to the current digit. If the digit becomes 10, we set it to 0 and propagate the carry to the next digit. If no carry is needed, we can return the result immediately. If we finish processing the array and still have a carry (e.g., the number was all 9s), we create a new array with one extra digit at the beginning set to 1.

Math & Geometry

□ #202 Happy Number

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Math · Two Pointers
Lists: NeetCode 150

Problem

Write an algorithm to determine if a number n is happy.

A happy number is a number defined by the following process:

- Starting with any positive integer, replace the number by the sum of the squares of its digits.
- Repeat the process until the number equals 1 (where it will stay), or it loops endlessly in a cycle which does not include 1.
- Those numbers for which this process ends in 1 are happy.

Return true if n is a happy number, and false if not.

Example 1:

Input: $n = 19$
 Output: true
 Explanation:
 $12 + 92 = 82$
 $82 + 22 = 68$
 $62 + 82 = 100$
 $12 + 02 + 02 = 1$

Example 2:

Input: $n = 2$
 Output: false

Constraints:

- $1 \leq n \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isHappy(self, n: int) -> bool:
        def squaredSum(n: int) -> bool:
            summ = 0
            while n > 0:
                summ += pow(n % 10, 2)
                n //= 10
            return summ

        slow = squaredSum(n)

        fast = squaredSum(squaredSum(n))

        while slow != fast:
            slow = squaredSum(slow)
            fast = squaredSum(squaredSum(fast))

        return slow == 1
```

• LeetDoocs


```
class Solution:
    def isHappy(self, n: int) -> bool:
        vis = set()
        while n != 1 and n not in vis:
            vis.add(n)
            x = 0
            while n:
                n, v = divmod(n, 10)
                x += v * v
            n = x
```

```
        return n == 1
```

```
class Solution:
    def isHappy(self, n: int) -> bool:
        def next(x):
            y = 0
            while x:
                x, v = divmod(x, 10)
                y += v * v
            return y
```

```
        slow, fast = n, next(n)
```

```
        while slow != fast:
            slow, fast = next(slow), next(next(fast))
        return slow == 1
```

• SimplyLeet

```
# Function to compute the sum of squares of digits
def sum_of_squares(num):
    total = 0
    while num:
        digit = num % 10      # Extract the last digit
        total += digit * digit # Add its square to the total
        num //= 10           # Remove the last digit
    return total

def isHappy(n):
```

```
seen = set()                # Initialize a set to keep track of seen numbers
# Iterate until n becomes 1 or we detect a cycle
while n != 1 and n not in seen:
    seen.add(n)              # Add current n to the set
    n = sum_of_squares(n)    # Compute new n as the sum of squares of digits
return n == 1               # Return True if n is 1, else False

# Example usage:
print(isHappy(19))          # Expected output: True
print(isHappy(2))           # Expected output: False
```

◆ Quick Review

Key Insights

- The process involves repeatedly computing the sum of squares of digits.
- If the process reaches 1, the number is happy.
- If the computed values start repeating (cycle detection), the number is not happy.
- The cycle can be detected using a hash set to record previously seen numbers or using a two pointers (Floyd's cycle detection) approach.

Space & Time

Time Complexity: $O(1)$ – Although the process appears iterative, the sum of squares quickly reduces any number to a bounded range.

Space Complexity: $O(1)$ – The number of unique sums computed is bounded, resulting in constant space usage.

Solution

The solution involves computing the sum of the squares of a number's digits in each iteration. We use a hash set to track numbers that have already been seen. If a number repeats, it means we are in a cycle and the number is not happy. If the number becomes 1, it is a happy number. Data structures used include a hash set for cycle detection, and the algorithm uses iterative computation.

Round 8 · Low · Medium

Round 8

Low Priority · Medium

25 questions · 4 patterns

**1-D Dynamic Programming #5 #91 #139 #152
#198 #213 #300 #322 #416 #647**

**2-D Dynamic Programming #62 #72 #97 #309
#494 #518 #1143**

Bit Manipulation #7 #371

Math & Geometry #43 #48 #50 #54 #73 #2013

1-D Dynamic Programming

Round 8 · Low · Medium · 10 questions

1-D Dynamic Programming

□ #5 Longest Palindromic Substring

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Grind 169

Problem

Given a string s , return the longest palindromic substring in s .

Example 1:

Input: $s = \text{"babad"}$
Output: "bab"
Explanation: "aba" is also a valid answer.

Example 2:

Input: $s = \text{"cbabd"}$
Output: "bb"

Constraints:

- $1 \leq s.length \leq 1000$
- s consist of only digits and English letters.

Community Solutions (Python)

- • WalkCC

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        if not s:
            return ''

        # (start, end) indices of the longest palindrome in s
        indices = [0, 0]

        def extend(s: str, i: int, j: int) -> tuple[int, int]:
            """

```

Returns the (start, end) indices of the longest palindrome extended from the substring s[i..j].

"""

```

        while i >= 0 and j < len(s):
            if s[i] != s[j]:
                break
            i -= 1
            j += 1
        return i + 1, j - 1

```

```

        for i in range(len(s)):
            l1, r1 = extend(s, i, i)
            if r1 - l1 > indices[1] - indices[0]:
                indices = l1, r1
            if i + 1 < len(s) and s[i] == s[i + 1]:
                l2, r2 = extend(s, i, i + 1)
                if r2 - l2 > indices[1] - indices[0]:
                    indices = l2, r2

        return s[indices[0]:indices[1] + 1]

```

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        t = '#'.join('@' + s + '$')
        p = self._manacher(t)
        maxPalindromeLength, bestCenter = max((extend, i)
                                              for i, extend in enumerate(p))

        l = (bestCenter - maxPalindromeLength) // 2
        r = (bestCenter + maxPalindromeLength) // 2
        return s[l:r]

```

```
def _manacher(self, t: str) -> list[int]:
    """
    Returns an array `p` s.t. `p[i]` is the length of the longest palindrome
    centered at `t[i]`, where `t` is a string with delimiters and sentinels.
    """
    p = [0] * len(t)
    center = 0
    for i in range(1, len(t) - 1):
        rightBoundary = center + p[center]
        mirrorIndex = center - (i - center)

        if rightBoundary > i:
            p[i] = min(rightBoundary - i, p[mirrorIndex])
        # Try to expand the palindrome centered at i.
        while t[i + 1 + p[i]] == t[i - 1 - p[i]]:
            p[i] += 1
        # If a palindrome centered at i expands past `rightBoundary`, adjust
        # the center based on the expanded palindrome.
        if i + p[i] > rightBoundary:
            center = i
    return p
```

• LeetDoocs

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        f = [[True] * n for _ in range(n)]
        k, mx = 0, 1
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                f[i][j] = False
                if s[i] == s[j]:
                    f[i][j] = f[i + 1][j - 1]

                    if f[i][j] and mx < j - i + 1:
                        k, mx = i, j - i + 1
        return s[k : k + mx]
```



```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        def f(l, r):
            while l >= 0 and r < n and s[l] == s[r]:
                l, r = l - 1, r + 1
            return r - l - 1

        n = len(s)
        start, mx = 0, 1
        for i in range(n):
            a = f(i, i)
            b = f(i, i + 1)
            t = max(a, b)
            if mx < t:
                mx = t
                start = i - ((t - 1) >> 1)
        return s[start : start + mx]

```

• SimplyLeet

```

# Function to expand from the center and return the palindrome substring
def expand_from_center(s, left, right):
    # Expand while the characters on both sides are the same
    while left >= 0 and right < len(s) and s[left] == s[right]:
        left -= 1
        right += 1
    # Return the palindromic substring found after expansion
    return s[left + 1:right]

def longest_palindromic_substring(s):
    if not s:
        return ""
    longest = ""
    # Iterate over each index in string as potential center
    for i in range(len(s)):
        # Check for odd-length palindrome
        odd_palindrome = expand_from_center(s, i, i)
        if len(odd_palindrome) > len(longest):
            longest = odd_palindrome
        # Check for even-length palindrome

```

```

        even_palindrome = expand_from_center(s, i, i + 1)
        if len(even_palindrome) > len(longest):
            longest = even_palindrome
    return longest

```

Example usage

```
print(longest_palindromic_substring("babad"))
```

• LC.ca

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        mlen = 0
        start = end = 0
        n = len(s)
        dp = [ [False] * n for i in range(n) ]

        for j in range(n):
            for i in range(j + 1):
                dp[i][j] = (i == j) or (s[i] == s[j] and j - i == 1) or (s[i]
                == s[j] and dp[i + 1][j - 1])

                if dp[i][j] is True and j - i + 1 > mlen:
                    mlen = j - i + 1
                    start = i
                    end = j

        return s[start: end + 1]

```

#####

```
class Solution:
```

```

    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        f = [[True] * n for _ in range(n)]
        k, mx = 0, 1
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                f[i][j] = False
                if s[i] == s[j]:
                    f[i][j] = f[i + 1][j - 1]
                    if f[i][j] and mx < j - i + 1:

```

```
        k, mx = i, j - i + 1  
    return s[k : k + mx]
```

◆ Quick Review

Key Insights

- Every palindrome is centered around a character (for odd-length) or a pair of characters (for even-length).
- Expand from each center until the substring is no longer a palindrome.
- Update the longest palindrome when a longer one is found during the expansion.
- The solution employs a two-pointer technique without extra storage for dynamic programming tables.

Space & Time

Time Complexity: $O(n^2)$ in the worst case as we expand around each center.

Space Complexity: $O(1)$, excluding the space required for the output substring.

Solution

The approach is based on expanding around potential centers of palindromes. For each

character in the string, consider it as the center for an odd-length palindrome and the gap between it and the next character for an even-length palindrome. Using two pointers, expand outward until the characters differ. Track the longest palindrome found during these expansions. This method is efficient given the input constraints, and no extra data structures are needed besides variables to store indices or substrings.

1-D Dynamic Programming

□ #91 Decode Ways

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Grind 169

Problem

You have intercepted a secret message encoded as a string of numbers. The message is decoded via the following mapping:

"1" -> 'A' "2" -> 'B' ... "25" -> 'Y' "26" -> 'Z'

However, while decoding the message, you realize that there are many different ways you can decode the message because some codes are contained in other codes ("2" and "5" vs "25").

For example, "11106" can be decoded into:

- "AAJF" with the grouping (1, 1, 10, 6)
- "KJF" with the grouping (11, 10, 6)
- The grouping (1, 11, 06) is invalid because "06" is not a valid code (only "6" is valid).

Note: there may be strings that are impossible to decode. Given a string *s* containing only digits, return the number of ways to decode it. If the entire string cannot be decoded in any valid way, return 0.

The test cases are generated so that the answer fits in a 32-bit integer.

Example 1:

Input: *s* = "12"

Output: 2

Explanation:

"12" could be decoded as "AB" (1 2) or "L" (12).

Example 2:

Input: *s* = "226"

Output: 3

Explanation:

"226" could be decoded as "BZ" (2 26), "VF" (22 6), or "BBF" (2 2 6).

Example 3:

Input: *s* = "06"

Output: 0

Explanation:

"06" cannot be mapped to "F" because of the leading zero ("6" is different from "06"). In this case, the string is not a valid encoding, so return 0.

Constraints:

- $1 \leq s.length \leq 100$
- s contains only digits and may contain leading zero(s).

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        # dp[i] := the number of ways to decode s[i..n)
        dp = [0] * n + [1]

        def isValid(a: str, b=None) -> bool:
            if b:
                return a == '1' or a == '2' and b < '7'
            return a != '0'

        if isValid(s[-1]):
            dp[n - 1] = 1

        for i in reversed(range(n - 1)):
            if isValid(s[i]):
                dp[i] += dp[i + 1]
            if isValid(s[i], s[i + 1]):
                dp[i] += dp[i + 2]

        return dp[0]
```

• LeetCode

```
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        f = [1] + [0] * n
        for i, c in enumerate(s, 1):
            if c != "0":
                f[i] = f[i - 1]
            if i > 1 and s[i - 2] != "0" and int(s[i - 2 : i]) <= 26:
                f[i] += f[i - 2]
        return f[n]
```

```
class Solution:
    def numDecodings(self, s: str) -> int:
        f, g = 0, 1
        for i, c in enumerate(s, 1):
            h = g if c != "0" else 0
            if i > 1 and s[i - 2] != "0" and int(s[i - 2 : i]) <= 26:
                h += f
            f, g = g, h
        return g
```

• SimplyLeet

```
# Python solution using dynamic programming
def num_decodings(s: str) -> int:
    # If the string starts with '0', it's not decodable
    if not s or s[0] == '0':
        return 0

    # dp array where dp[i] represents the number of ways to decode s[:i]
    n = len(s)
    dp = [0] * (n + 1)
```



```

# Base case: an empty string has one decoding
dp[0] = 1

for i in range(1, n + 1):
    # Check for single digit decoding (s[i-1] is valid if not '0')
    if s[i - 1] != '0':
        dp[i] += dp[i - 1]
    # Check for two-digit decoding (if i>=2, s[i-2:i] forms a valid number
    # between 10 and 26)
    if i >= 2 and s[i - 2] != '0':
        two_digit = int(s[i - 2:i])

        if 10 <= two_digit <= 26:
            dp[i] += dp[i - 2]
return dp[n]

# Example usage:
print(num_decodings("12")) # Expected output: 2

```

• LC.ca

```

'''
>>> s = "abcdef"
>>> [(i,c) for i, c in enumerate(s, 1)]
[(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e'), (6, 'f')]
'''

class Solution:
    def numDecodings(self, s: str) -> int:
        # f: i-2
        # g: i-1
        f, g = 0, 1

        for i, c in enumerate(s, 1):
            h = g if c != "0" else 0
            if i > 1 and s[i - 2] != "0" and int(s[i - 2 : i]) <= 26:
                h += f
            f, g = g, h
        return g

#####

class Solution:

```

```
def numDecodings(self, s: str) -> int:
    if len(s) == 0:
        return 0

    length = len(s)
    dp = [0] * (length + 1) # dp[i] => at index i, its decode ways

    dp[length] = 1 # initiator, just make forloop flow working
    dp[length - 1] = 0 if s[length - 1] == '0' else 1 # assumption is
    starting at i, so starting as 0 is not decodable
```

```
# start at 'len-2'
# or else below 'dp[i+2]' will out of index boundary
for i in range(length - 2, -1, -1):
    if s[i] == '0':
        continue
    tem = int(s[i:i + 2])

    if tem > 26:
        dp[i] = dp[i + 1]
    else:
```

```
        dp[i] = dp[i + 1] + dp[i + 2]
    return dp[0]
```

optimized from above solution, no dp array

```
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        # a: dp[i-2], b: dp[i-1], c: count for current index i
        a, b, c = 0, 1, 0
        for i in range(1, n + 1):
```

```

        c = 0
        if s[i - 1] != '0':
            c += b
            if i > 1 and s[i - 2] != '0' and (int(s[i - 2]) * 10 + int(s[i - 1]) <= 26):
                c += a
            a, b = b, c
        return c

#####

```

```

class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        dp = [0] * (n + 1)
        dp[0] = 1
        for i in range(1, n + 1):
            if s[i - 1] != '0':
                dp[i] += dp[i - 1]
            if i > 1 and s[i - 2] != '0' and (int(s[i - 2]) * 10 + int(s[i - 1]) <= 26):
                dp[i] += dp[i - 2]

        return dp[n]

```

◆ Quick Review

Key Insights

- A digit '0' cannot be mapped on its own; it must be part of "10" or "20".
- Use dynamic programming where $dp[i]$ represents the number of ways to decode the substring $s[0...i-1]$.

- For each index i , if the digit at position $i-1$ is non-zero, add $dp[i-1]$ to $dp[i]$.
- If the two-digit number formed by $s[i-2]$ and $s[i-1]$ is between 10 and 26, add $dp[i-2]$ to $dp[i]$.
- Initialize $dp[0]$ as 1 (an empty string has one way to be decoded).

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(n)$, due to the dp array (which can be optimized to $O(1)$).

Solution

We solve the problem using a dynamic programming approach. We initialize a dp array where $dp[i]$ indicates the number of ways to decode the substring $s[0...i-1]$. We set $dp[0] = 1$ since an empty string is trivially decodable. We loop through the string, and at each step: If $s[i-1]$ is not '0', we add $dp[i-1]$ to $dp[i]$ because $s[i-1]$ can be mapped to a valid letter. If the substring $s[i-2:i]$ (for $i \geq 2$) forms a valid two-digit number between 10 and 26, we add $dp[i-2]$ to $dp[i]$. This

cumulative approach yields the total ways to decode the string, and if the string starts with '0' or contains invalid sequences, $dp[n]$ will eventually be 0.

1-D Dynamic Programming

□ #139 Word Break

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Dynamic
Programming · Trie · Memoization
Lists: Grind 169

Problem

Given a string *s* and a dictionary of strings *wordDict*, return true if *s* can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

Input: *s* = "leetcode", *wordDict* = ["leet","code"]
Output: true
Explanation: Return true because "leetcode" can be segmented as "leet code".

Example 2:

Input: *s* = "applepenapple", *wordDict* = ["apple","pen"]
Output: true
Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".
Note that you are allowed to reuse a dictionary word.

Example 3:

Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
Output: false

Constraints:

- $1 \leq s.length \leq 300$
- $1 \leq wordDict.length \leq 1000$
- $1 \leq wordDict[i].length \leq 20$
- s and wordDict[i] consist of only lowercase English letters.
- All the strings of wordDict are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        wordSet = set(wordDict)

        @functools.lru_cache(None)
        def wordBreak(s: str) -> bool:
            """Returns True if s can be segmented."""
            if s in wordSet:
                return True
            return any(s[:i] in wordSet and wordBreak(s[i:]) for i in range(len(s)))

        return wordBreak(s)
```

```

class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        n = len(s)
        wordSet = set(wordDict)
        # dp[i] := True if s[0..i) can be segmented
        dp = [True] + [False] * n

        for i in range(1, n + 1):
            for j in range(i):
                # s[0..j) can be segmented and s[j..i) is in `wordSet`, so s[0..i) can
                # be segmented.
                if dp[j] and s[j:i] in wordSet:
                    dp[i] = True
                    break

        return dp[n]

```

```

class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        n = len(s)
        maxLength = len(max(wordDict, key=len))
        wordSet = set(wordDict)
        # dp[i] := True if s[0..i) can be segmented
        dp = [True] + [False] * n

        for i in range(1, n + 1):
            for j in range(i - 1, -1, -1):
                if i - j > maxLength:
                    break
                # s[0..j) can be segmented and s[j..i) is in the wordSet, so s[0..i)
                # can be segmented.
                if dp[j] and s[j:i] in wordSet:
                    dp[i] = True
                    break

        return dp[n]

```

• LeetDoocs


```
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        words = set(wordDict)
        n = len(s)
        f = [True] + [False] * n
        for i in range(1, n + 1):
            f[i] = any(f[j] and s[j:i] in words for j in range(i))
        return f[n]
```

```
class Trie:
    def __init__(self):
        self.children: List[Trie | None] = [None] * 26
        self.isEnd = False

    def insert(self, w: str):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if not node.children[idx]:
```

```
                node.children[idx] = Trie()
            node = node.children[idx]
        node.isEnd = True
```

```
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        trie = Trie()
        for w in wordDict:
            trie.insert(w)
```

```
        n = len(s)
        f = [False] * (n + 1)
        f[n] = True
        for i in range(n - 1, -1, -1):
            node = trie
            for j in range(i, n):
                idx = ord(s[j]) - ord('a')
                if not node.children[idx]:
                    break
                node = node.children[idx]
```

```

        if node.isEnd and f[j + 1]:
            f[i] = True
            break
    return f[0]

```

• SimplyLeet

Python solution for the Word Break problem

```

def wordBreak(s, wordDict):
    # Convert the list to a set for O(1) look-ups
    wordSet = set(wordDict)

    # dp[i] is True if s[0:i] can be segmented into words from the wordDict
    n = len(s)
    dp = [False] * (n + 1)

```

```

    # Base case: empty string is always segmentable
    dp[0] = True

```

```

    # Iterate through the string positions
    for i in range(1, n + 1):
        # Check every possible partition s[0:j] and s[j:i]
        for j in range(i):
            # If s[0:j] can be segmented and s[j:i] is in the dictionary
            if dp[j] and s[j:i] in wordSet:
                dp[i] = True # s[0:i] can be segmented

```

```

                break # No need to check further partitions
    return dp[n]

```

```

# Example usage:
s = "leetcode"
wordDict = ["leet", "code"]
print(wordBreak(s, wordDict)) # Output: True

```

• LC.ca

```

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        words = set(wordDict)
        n = len(s)

        # dp[j] meaning s from 0 to index=i-1~ is breakable
        dp = [True] + [False] * n # so actually size is n+1
        for i in range(1, n + 1):
            dp[i] = any( (dp[j] and s[j:i] in words) for j in range(i) )
        return dp[n]

```

```
#####
```

```

class Solution:
    def wordBreak(self, s: str, wordDict: Set[str]) -> bool:
        if not s or not wordDict:
            return False

        length = len(s)

```

```

        # construct dp, dp[i] meaning position i-1 can from dict or not
        dp[-1] meaning at last-index plus 1, checking up to last-index
        dp = [False] * (length + 1)
        dp[0] = True # initiate

```

```

        for i in range(length + 1):
            if not dp[i]:
                continue
            for word in wordDict:
                if i + len(word) > length:

```

```

                    continue
                if s[i:i + len(word)] == word:
                    dp[i + len(word)] = True

```

```

        return dp[length]

```

```
#####
```

```

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:

```

```

words = set(wordDict)
n = len(s)
dp = [False] * (n + 1)
dp[0] = True # i'th char in string, is good
for j in range(1, n + 1):
    for i in range(j): # starting at 0, including dp[0]
        if dp[i] and s[i:j] in words:
            dp[j] = True # j is exclusive, meaning True until index
i-1
            break
return dp[-1]

```

```
#####
```

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, w):

        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, w):
        node = self

```

```

        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return False
            node = node.children[idx]
        return node.is_end

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        # https://docs.python.org/3/library/functools.html#functools.cache

        # creating a thin wrapper around a dictionary lookup for the function
        arguments
        @cache
        def dfs(s):
            return not s or any(trie.search(s[:i]) and dfs(s[i:]) for i in range(1, len(s) + 1))

        trie = Trie()
        for w in wordDict:
            trie.insert(w)
        return dfs(s)

#####

class Solution(object):
    def wordBreak(self, s, wordDict):
        """
        :type s: str
        :type wordDict: Set[str]
        :rtype: bool
        """
        queue = [0]

```

```
ls = len(s)
lenList = [l for l in set(map(len, wordDict))]
visited = [0 for _ in range(0, ls + 1)]
while queue:
    start = queue.pop(0)
    for l in lenList:
        if s[start:start + l] in wordDict:
            if start + l == ls:
                return True
            if visited[start + l] == 0:

                queue.append(start + l)
                visited[start + l] = 1
return False
```

◆ Quick Review

Key Insights

- Use Dynamic Programming (DP) to break the problem into subproblems.
- Create a DP array where $dp[i]$ is true if the substring $s[0:i]$ can be segmented using words from the dictionary.
- Leverage a hash set for fast look-up of dictionary words.
- For every index i , check all possible previous break points j such that $dp[j]$ is true and $s[j:i]$ is in the wordDict.
- The problem can also be approached with recursion and memoization, but DP is more straightforward for iterative implementation.

Space & Time

Time Complexity: $O(n^2)$ in the worst-case scenario, where n is the length of s (each substring check is $O(1)$ if using a hash set, and there could be $O(n^2)$ checks overall).

Space Complexity: $O(n)$ for the dp array and $O(k)$ for the hash set where k is the number of words in the dictionary.

Solution

The solution uses a Dynamic Programming (DP) approach: Convert wordDict to a hash set for constant time lookup. Create a boolean dp array of size $n+1$ (n is the length of s), where $dp[i]$ indicates if $s[0:i]$ can be segmented. Set $dp[0]$ to true because an empty string is always segmentable. For each index i from 1 to n , iterate through all indices j from 0 to i : If $dp[j]$ is true and the substring $s[j:i]$ is in the dictionary, then set $dp[i]$ to true. Break early from the inner loop when a valid break is found. Return $dp[n]$ as the result. This method efficiently builds up the solution by utilizing previous results to determine if the entire string can be segmented.

1-D Dynamic Programming

□ #152 Maximum Product Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

Given an integer array `nums`, find a subarray that has the largest product, and return the product.

The test cases are generated so that the answer will fit in a 32-bit integer.

Note that the product of an array with a single element is the value of that element.

Example 1:

Input: `nums = [2,3,-2,4]`
Output: 6
Explanation: `[2,3]` has the largest product 6.

Example 2:

Input: `nums = [-2,0,-1]`
Output: 0
Explanation: The result cannot be 2, because `[-2,-1]` is not a subarray.

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$

- $-10 \leq \text{nums}[i] \leq 10$
- The product of any subarray of `nums` is guaranteed to fit in a 32-bit integer.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProduct(self, nums: list[int]) -> int:
        ans = nums[0]
        dpMin = nums[0] # the minimum so far
        dpMax = nums[0] # the maximum so far

        for i in range(1, len(nums)):
            num = nums[i]
            prevMin = dpMin # dpMin[i - 1]
            prevMax = dpMax # dpMax[i - 1]

            if num < 0:
                dpMin = min(prevMax * num, num)
                dpMax = max(prevMin * num, num)
            else:
                dpMin = min(prevMin * num, num)
                dpMax = max(prevMax * num, num)

            ans = max(ans, dpMax)

        return ans
```

• LeetDoocs

```
class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        ans = f = g = nums[0]
        for x in nums[1:]:
            ff, gg = f, g
            f = max(x, ff * x, gg * x)
            g = min(x, ff * x, gg * x)
            ans = max(ans, f)
        return ans
```

• SimplyLeet

```
def maxProduct(nums):
    # If the input array is empty, return 0 (though problem guarantees at least
    one element)
    if not nums:
        return 0
    # Initialize maximum, minimum product ending here and the result.
    max_ending_here = nums[0]
    min_ending_here = nums[0]
    global_max = nums[0]

    # Iterate over the array starting from the second element.

    for num in nums[1:]:
        # If the current number is negative, swap max and min.
        if num < 0:
            max_ending_here, min_ending_here = min_ending_here, max_ending_her
e
        # Update the max product at the current index.
        max_ending_here = max(num, max_ending_here * num)
        # Update the min product at the current index.
        min_ending_here = min(num, min_ending_here * num)
        # Update the global maximum product.
        global_max = max(global_max, max_ending_here)

    return global_max

# Example usage:
# print(maxProduct([2,3,-2,4])) # Output: 6
```

• LC.ca

```
class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        ans = f = g = nums[0]
        for x in nums[1:]:
            ff, gg = f, g
            f = max(x, ff * x, gg * x)
            g = min(x, ff * x, gg * x)
            ans = max(ans, f)
        return ans
```

◆ Quick Review

Key Insights

- Maintain both the maximum and minimum product at the current index because a negative number can swap the max and min.
- A subarray must be contiguous.
- Use dynamic programming to update both the maximum product and the minimum product ending at each index.
- Resetting the product in case of a zero is crucial since the product becomes 0.

Space & Time

Time Complexity: $O(n)$ – The solution iterates through the array once.

Space Complexity: $O(1)$ – Constant space is used for tracking products.

Solution

The solution uses an iterative dynamic programming approach. At each index, compute the maximum product (maxEndingHere) and minimum product (minEndingHere) ending at that index. If the current element is negative, swap these two values, because multiplying a negative with a minimum product (possibly negative) might yield a larger product. After processing each element, update the global maximum. This method handles edge cases like zeros and negatives efficiently with constant additional space.

1-D Dynamic Programming

□ #198 House Robber

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).

Total amount you can rob = 1 + 3 = 4.

Example 2:

Input: nums = [2,7,9,3,1]

Output: 12

Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).

Total amount you can rob = 2 + 9 + 1 = 12.

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 400$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rob(self, nums: list[int]) -> int:
        if not nums:
            return 0
        if len(nums) == 1:
            return nums[0]

        # dp[i] := max money of robbing nums[0..i]
        dp = [0] * len(nums)
        dp[0] = nums[0]

        dp[1] = max(nums[0], nums[1])

        for i in range(2, len(nums)):
            dp[i] = max(dp[i - 1], dp[i - 2] + nums[i])

        return dp[-1]
```

```

class Solution:
    def rob(self, nums: list[int]) -> int:
        prev1 = 0 # dp[i - 1]
        prev2 = 0 # dp[i - 2]

        for num in nums:
            dp = max(prev1, prev2 + num)
            prev2 = prev1
            prev1 = dp

        return prev1

```

• LeetDoocs

```

class Solution:
    def rob(self, nums: List[int]) -> int:
        @cache
        def dfs(i: int) -> int:
            if i >= len(nums):
                return 0
            return max(nums[i] + dfs(i + 2), dfs(i + 1))

        return dfs(0)

```

```

class Solution:
    def rob(self, nums: List[int]) -> int:
        n = len(nums)
        f = [0] * (n + 1)
        f[1] = nums[0]
        for i in range(2, n + 1):
            f[i] = max(f[i - 1], f[i - 2] + nums[i - 1])
        return f[n]

```

```

class Solution:
    def rob(self, nums: List[int]) -> int:
        f = g = 0
        for x in nums:
            f, g = max(f, g), f + x
        return max(f, g)

```

• SimplyLeet

```

# Definition of the Solution class
class Solution:
    def rob(self, nums: List[int]) -> int:
        # Edge case: if there are no houses, return 0
        if not nums:
            return 0

        # Initialize two variables to keep track of the max amounts
        prev1, prev2 = 0, 0

        # Iterate over each house's money
        for money in nums:
            # Calculate the maximum money that can be robbed if we consider the
            current house
            current = max(prev1, prev2 + money) # Either skip current house or
            rob it
            # Update variables: prev2 becomes previous best, and prev1 becomes
            current best
            prev2 = prev1
            prev1 = current

        # Return the maximum money that can be robbed
        return prev1

```

• LC.ca

```

# greedy
class Solution:
    def rob(self, nums: List[int]) -> int:
        not_rob, rob = 0, nums[0]
        for num in nums[1:]:
            # must max check
            # eg. first robbed 99999, then following ones are just 3,3,3,3,3
            not_rob, rob = rob, max(num + not_rob, rob)
        return rob

```



```
#####

class Solution(object):
    def rob(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        if len(nums) == 0:
            return 0

        if len(nums) <= 2:
            return max(nums)
        dp = [0 for i in range(0, 2)]
        dp[0] = nums[0]
        dp[1] = max(nums[1], nums[0])
        for i in range(2, len(nums)):
            dp[i % 2] = max(dp[(i - 1) % 2], dp[(i - 2) % 2] + nums[i])
        return dp[(len(nums) - 1) % 2]
```

◆ Quick Review

Key Insights

- The problem can be solved using Dynamic Programming.
- For each house, decide whether to rob it or not.
- Recurrence relation: $dp[i] = \max(dp[i-1], dp[i-2] + \text{currentHouseMoney})$
- Use iterative computation to keep track of the optimal solutions without needing extra space for the entire dp array.

Space & Time

Time Complexity: $O(n)$, where n is the number of houses.

Space Complexity: $O(1)$, using only two variables to track the previous decisions.

Solution

We use a dynamic programming approach by iterating through each house in the array. At each step, we consider two scenarios: either we rob the current house (and add its money to the total from two houses ago) or we skip it (keeping the optimal solution up to the previous house). By using two variables to store these intermediate results, we achieve an optimized space complexity of $O(1)$.

1-D Dynamic Programming

□ #213 House Robber II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: NeetCode 150

Problem

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have a security system connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

Input: nums = [2,3,2]

Output: 3

Explanation: You cannot rob house 1 (money = 2) and then rob house 3 (money = 2), because they are adjacent houses.

Example 2:

Input: nums = [1,2,3,1]

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).

Total amount you can rob = 1 + 3 = 4.

Example 3:

Input: nums = [1,2,3]

Output: 3

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rob(self, nums: list[int]) -> int:
        if not nums:
            return 0
        if len(nums) < 2:
            return nums[0]

        def rob(l: int, r: int) -> int:
            dp1 = 0
            dp2 = 0
```

```

    for i in range(l, r + 1):
        temp = dp1
        dp1 = max(dp1, dp2 + nums[i])
        dp2 = temp

    return dp1

return max(rob(0, len(nums) - 2),
           rob(1, len(nums) - 1))

```

• LeetCode

```

class Solution:
    def rob(self, nums: List[int]) -> int:
        def _rob(nums):
            f = g = 0
            for x in nums:
                f, g = max(f, g), f + x
            return max(f, g)

        if len(nums) == 1:
            return nums[0]

        return max(_rob(nums[1:]), _rob(nums[:-1]))

```

• SimplyLeet

```

# Python solution for House Robber II

def rob(nums):
    # Helper function to handle the linear House Robber problem.
    def rob_linear(houses):
        prev_max, curr_max = 0, 0
        for money in houses:
            temp = curr_max
            curr_max = max(curr_max, prev_max + money)
            prev_max = temp

```

```
        return curr_max

    n = len(nums)
    # When there's only one house, just return its value.
    if n == 1:
        return nums[0]

    # Rob excluding the first house and excluding the last house.
    return max(rob_linear(nums[1:]), rob_linear(nums[:-1]))

# Test example
print(rob([2, 3, 2])) # Expected output: 3
```

◆ Quick Review

Key Insights

- The houses are arranged in a circle, meaning the first and last houses are neighbors.
- Because of the circular arrangement, you cannot rob both the first and last houses.
- The problem can be reduced into two separate linear House Robber problems:
- One considering houses from index 1 to $n-1$ (exclude the first house).
- The other considering houses from index 0 to $n-2$ (exclude the last house).
- Use dynamic programming with constant space (using two variables) to compute the solution for a linear array of houses.

Space & Time

Time Complexity: $O(n)$, where n is the number of houses.

Space Complexity: $O(1)$, as only a constant amount of extra space is used.

Solution

To handle the circular dependency, we solve two linear versions of the problem:

- Exclude the first house and calculate the maximum rob amount for houses from index 1 to $n-1$.
- Exclude the last house and calculate the maximum rob amount for houses from index 0 to $n-2$.

Then, the final answer is the maximum of these two calculated values. We use dynamic programming by iterating through the list and at each step, calculating the maximum money that can be robbed up to that point with two variables.

1-D Dynamic Programming

□ #300 Longest Increasing Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Dynamic Programming
Lists: Grind 169

Problem

Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

Example 1:

Input: `nums = [10,9,2,5,3,7,101,18]`

Output: 4

Explanation: The longest increasing subsequence is `[2,3,7,101]`, therefore the length is 4.

Example 2:

Input: `nums = [0,1,0,3,2,3]`

Output: 4

Example 3:

Input: `nums = [7,7,7,7,7,7,7]`

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 2500$
- $-104 \leq \text{nums}[i] \leq 104$

Follow up: Can you come up with an algorithm that runs in $O(n \log(n))$ time complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lengthOfLIS(self, nums: list[int]) -> int:
        if not nums:
            return 0

        # dp[i] := the length of LIS ending in nums[i]
        dp = [1] * len(nums)

        for i in range(1, len(nums)):
            for j in range(i):

                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)

        return max(dp)

class Solution:
    def lengthOfLIS(self, nums: list[int]) -> int:
        # tails[i] := the minimum tails of all the increasing subsequences having
        # length i + 1
        tails = []

        for num in nums:
            if not tails or num > tails[-1]:
                tails.append(num)
            else:
                tails[bisect.bisect_left(tails, num)] = num

        return len(tails)
```

• LeetDoocs

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        f = [1] * n
        for i in range(1, n):
            for j in range(i):
                if nums[j] < nums[i]:
                    f[i] = max(f[i], f[j] + 1)
        return max(f)
```

```
class BinaryIndexedTree:
    def __init__(self, n: int):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, v: int):
        while x <= self.n:
            self.c[x] = max(self.c[x], v)
            x += x & -x
```

```
    def query(self, x: int) -> int:
        mx = 0
        while x:
            mx = max(mx, self.c[x])
            x -= x & -x
        return mx
```

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
```

```
        s = sorted(set(nums))
        m = len(s)
        tree = BinaryIndexedTree(m)
        for x in nums:
            x = bisect_left(s, x) + 1
            t = tree.query(x - 1) + 1
            tree.update(x, t)
        return tree.query(m)
```

• SimplyLeet

```
import bisect

def lengthOfLIS(nums):
    dp = [] # dp array stores smallest tail for each subsequence length
    for num in nums:
        # Find the leftmost insertion point using binary search
        idx = bisect.bisect_left(dp, num)
        if idx == len(dp):
            dp.append(num) # Extend dp if no larger tail exists
        else:
            dp[idx] = num # Replace to maintain the smallest tail
    return len(dp)

# Example usage:
# print(lengthOfLIS([10,9,2,5,3,7,101,18])) # Output: 4
```

• LC.ca

```
# find the largest end element in tails that is smaller than nums[i]
# and then replace it with nums[i] and discard the list in the same length
# which is implemented by `tail[idx] = num`

class Solution(object):
    def lengthOfLIS(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """

        tail = []
        for num in nums:
            # if using bisect_right(tail, num),
            # then input=[7,7,7,7,7,7,7] will output 7 but expected result is 1
            idx = bisect.bisect_left(tail, num)
            if idx == len(tail): # same as in java: if (i == len) len++;
                tail.append(num)
            else:
                tail[idx] = num
        return len(tail)
```

```

# implementation of bisect.bisect_left()
# similar to Leetcode-302, find left/right/top/bottom callable
def bisect_left(a, x, lo=0, hi=None):
    if hi is None:
        hi = len(a)

    while lo < hi:
        mid = (lo + hi) // 2

        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid

    return lo

#####

class BinaryIndexedTree:

    def __init__(self, n: int):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, v: int):
        while x <= self.n:
            self.c[x] = max(self.c[x], v)
            x += x & -x

    def query(self, x: int) -> int:

        mx = 0
        while x:
            mx = max(mx, self.c[x])
            x -= x & -x
        return mx

class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        s = sorted(set(nums))

```

```
m = len(s)
tree = BinaryIndexedTree(m)
for x in nums:
    x = bisect_left(s, x) + 1
    t = tree.query(x - 1) + 1
    tree.update(x, t)
return tree.query(m)
```

◆ Quick Review

Key Insights

- A dynamic programming solution with $O(n^2)$ time complexity exists, but we focus on an optimized approach.
- Using binary search and a dp array, we can achieve $O(n \log n)$ time.
- The dp array maintains the smallest possible tail for increasing subsequences of various lengths.
- For each number, perform binary search to decide if it can extend or replace a value in dp.

Space & Time

Time Complexity: $O(n \log n)$ where n is the length of nums

Space Complexity: $O(n)$ for storing the dp array

Solution

We use a dynamic programming approach combined with binary search. The idea is to maintain a dp array where $dp[i]$ holds the smallest tail value for any increasing subsequence with length $i+1$. For each number in the array, we perform a binary search on dp to find its correct insertion position: If the number is larger than all elements, we append it, effectively extending the current longest subsequence. Otherwise, we replace the first element in dp that is greater than or equal to the current number to keep the tail as small as possible. This process ensures that dp remains sorted and its length gives the length of the longest increasing subsequence.

1-D Dynamic Programming

□ #322 Coin Change

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · BFS
Lists: Grind 169

Problem

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return `-1`.

You may assume that you have an infinite number of each kind of coin.

Example 1:

```
Input: coins = [1,2,5], amount = 11  
Output: 3  
Explanation: 11 = 5 + 5 + 1
```

Example 2:

Input: coins = [2], amount = 3
Output: -1

Example 3:

Input: coins = [1], amount = 0
Output: 0

Constraints:

- $1 \leq \text{coins.length} \leq 12$
- $1 \leq \text{coins}[i] \leq 231 - 1$
- $0 \leq \text{amount} \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def coinChange(self, coins: list[int], amount: int) -> int:
        # dp[i] := the minimum number of coins to make up i
        dp = [0] + [amount + 1] * amount

        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] = min(dp[i], dp[i - coin] + 1)

        return -1 if dp[amount] == amount + 1 else dp[amount]
```

• LeetDoocs


```
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        m, n = len(coins), amount
        f = [[inf] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 0
        for i, x in enumerate(coins, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] = min(f[i][j], f[i][j - x] + 1)

        return -1 if f[m][n] >= inf else f[m][n]
```

• SimplyLeet

```
def coinChange(coins, amount):
    # Create a dp array with amount+1 entries set to a value greater than any
    # possible minimum number of coins.
    dp = [amount + 1] * (amount + 1)
    # Base case: 0 coins are needed to form amount 0.
    dp[0] = 0

    # Build up the dp table.
    for current_amount in range(1, amount + 1):
        for coin in coins:
            if coin <= current_amount:
                dp[current_amount] = min(dp[current_amount], dp[current_amount
                - coin] + 1)

    # If dp[amount] remains greater than amount, no solution was found.
    return dp[amount] if dp[amount] != amount + 1 else -1
```

• LC.ca

```

'''
>>> float("inf")
inf
>>> float("inf") + 1
inf
'''

class Solution(object):
    def coinChange(self, coins, amount):
        '''

        :type coins: List[int]
        :type amount: int
        :rtype: int
        '''

        dp = [float("inf")] * (amount + 1)
        dp[0] = 0
        for i in range(1, amount + 1):
            for coin in coins:
                if i - coin >= 0:

                    dp[i] = min(dp[i], dp[i - coin] + 1)
        return dp[-1] if dp[-1] != float("inf") else -1

#####

class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        dp = [amount + 1] * (amount + 1)
        dp[0] = 0
        for coin in coins:

            for j in range(coin, amount + 1):
                dp[j] = min(dp[j], dp[j - coin] + 1)
        return -1 if dp[-1] > amount else dp[-1]

```

◆ Quick Review

Key Insights

- Use a dynamic programming approach to build up a solution from smaller subproblems.
- Define $dp[x]$ as the minimum number of coins required for amount x .
- The recurrence relation is: $dp[x] = \min(dp[x], dp[x - \text{coin}] + 1)$ for each coin.
- Initialize $dp[0] = 0$ and set other values to a number larger than the maximum possible coins ($\text{amount} + 1$ works as an infinity substitute).
- An alternative approach is through BFS, treating each amount as a node and each coin as an edge, but the DP method is more intuitive here.

Space & Time

Time Complexity: $O(\text{amount} * n)$ where n is the number of coins.

Space Complexity: $O(\text{amount})$ for the dp array.

Solution

We use a bottom-up dynamic programming approach. The idea is to use a one-dimensional dp array where each element at index x represents the minimum number of coins required to form the amount x . Starting

from `dp[0]` which is 0, we iterate through all amounts up to the target amount and update `dp[x]` based on each coin denomination. If after filling the `dp` array the value `dp[amount]` remains unchanged (i.e., greater than amount), it means the amount cannot be formed and we return `-1`.

1-D Dynamic Programming

□ #416 Partition Equal Subset Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

Given an integer array `nums`, return `true` if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or `false` otherwise.

Example 1:

Input: `nums = [1,5,11,5]`
Output: `true`
Explanation: The array can be partitioned as `[1, 5, 5]` and `[11]`.

Example 2:

Input: `nums = [1,2,3,5]`
Output: `false`
Explanation: The array cannot be partitioned into equal sum subsets.

Constraints:

- $1 \leq \text{nums.length} \leq 200$
- $1 \leq \text{nums}[i] \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canPartition(self, nums: list[int]) -> bool:
        summ = sum(nums)
        if summ % 2 == 1:
            return False
        return self.knapsack_(nums, summ // 2)

    def knapsack_(self, nums: list[int], subsetSum: int) -> bool:
        n = len(nums)
        # dp[i][j] := True if j can be formed by nums[0..i)

        dp = [[False] * (subsetSum + 1) for _ in range(n + 1)]
        dp[0][0] = True

        for i in range(1, n + 1):
            num = nums[i - 1]
            for j in range(subsetSum + 1):
                if j < num:
                    dp[i][j] = dp[i - 1][j]
                else:
                    dp[i][j] = dp[i - 1][j] or dp[i - 1][j - num]

        return dp[n][subsetSum]
```

• LeetDoocs

```
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        m, mod = divmod(sum(nums), 2)
        if mod:
            return False
        n = len(nums)
        f = [[False] * (m + 1) for _ in range(n + 1)]
        f[0][0] = True
        for i, x in enumerate(nums, 1):
            for j in range(m + 1):
                f[i][j] = f[i - 1][j] or (j >= x and f[i - 1][j - x])
        return f[n][m]
```

• SimplyLeet

```
# Python solution for Partition Equal Subset Sum

def canPartition(nums):
    # Calculate the total sum of the array
    total_sum = sum(nums)

    # If the total sum is odd, equal partition is not possible
    if total_sum % 2 != 0:
        return False

    # Set the target sum to half of the total sum
    target = total_sum // 2

    # Initialize dp array where dp[i] is True if a subset sum i is achievable
    dp = [False] * (target + 1)
    dp[0] = True # Base case: subset sum of 0 is always possible

    # Iterate through each number in the array
    for num in nums:
        # Iterate backwards to avoid overwriting dp values needed for current
        # iteration

        for i in range(target, num - 1, -1):
            # Update dp[i] if dp[i-num] is True
            if dp[i - num]:
                dp[i] = True

    # dp[target] is True if a subset adds up to target sum
    return dp[target]

# Example usage:
# print(canPartition([1, 5, 11, 5]))
```

• LC.ca

```

class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        s = sum(nums)
        if s % 2 != 0:
            return False
        n = s >> 1
        dp = [False] * (n + 1)
        dp[0] = True
        for v in nums:
            for target in range(n, v - 1, -1): # including v itself
                dp[target] = dp[target] or dp[target - v]
        return dp[-1]

```

```

class Solution: # recursive, but over time limit in OJ
    def canPartition(self, nums: List[int]) -> bool:
        total_sum = sum(nums)
        if total_sum % 2 != 0:
            return False
        target_sum = total_sum // 2

```

```

memo = {} # ==> or just use @cache for dfs()

def dfs(idx, curr_sum):
    if curr_sum == target_sum:
        return True
    if curr_sum > target_sum or idx >= len(nums):
        return False
    if (idx, curr_sum) in memo:
        return memo[(idx, curr_sum)]

```

```

    # Explore two options: include the current number or exclude it
    include = dfs(idx + 1, curr_sum + nums[idx])
    exclude = dfs(idx + 1, curr_sum)

    memo[(idx, curr_sum)] = include or exclude
    return memo[(idx, curr_sum)]

```

```

return dfs(0, 0)

```

```

#####

```



```
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        m, mod = divmod(sum(nums), 2)
        if mod:
            return False
        f = [True] + [False] * m
        for x in nums:
            for j in range(m, x - 1, -1):
                f[j] = f[j] or f[j - x]

        return f[m]
```

◆ Quick Review

Key Insights

- The total sum of the array must be even, otherwise, partitioning into two equal subsets is impossible.
- The problem can be transformed into a "subset sum" problem where we need to check if there exists a subset of numbers that sums to half of the total sum.
- A dynamic programming approach can efficiently decide if such a subset exists.

Space & Time

Time Complexity: $O(n * \text{target})$, where target is half of the total sum.

Space Complexity: $O(\text{target})$, using a one-dimensional DP array.

Solution

First, compute the total sum of the array. If the total is odd, return false since an odd number cannot be equally partitioned. Next, set the target sum to half of the total. Use a dynamic programming array (dp) where $\text{dp}[i]$ represents whether a subset with sum i is achievable. Initialize $\text{dp}[0]$ as true because a subset sum of 0 is always possible. Then, for each number in the array, iterate through the dp array backwards from target to the number's value and update $\text{dp}[i]$ to true if $\text{dp}[i - \text{num}]$ is true. Ultimately, if $\text{dp}[\text{target}]$ is true, the array can be partitioned into two subsets with equal sum.

1-D Dynamic Programming

□ #647 Palindromic Substrings

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String · Dynamic Programming
Lists: NeetCode 150

Problem

Given a string s , return the number of palindromic substrings in it.

A string is a palindrome when it reads the same backward as forward.

A substring is a contiguous sequence of characters within the string.

Example 1:

Input: $s = \text{"abc"}$

Output: 3

Explanation: Three palindromic strings: "a", "b", "c".

Example 2:

Input: $s = \text{"aaa"}$

Output: 6

Explanation: Six palindromic strings: "a", "a", "a", "aa", "aa", "aaa".

Constraints:

- $1 \leq s.length \leq 1000$
- s consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countSubstrings(self, s: str) -> int:
        def extendPalindromes(l: int, r: int) -> int:
            count = 0

            while l >= 0 and r < len(s) and s[l] == s[r]:
                count += 1
                l -= 1
                r += 1

            return count

        ans = 0

        for i in range(len(s)):
            ans += extendPalindromes(i, i)
            ans += extendPalindromes(i, i + 1)

        return ans
```

• LeetDoocs

```
class Solution:
    def countSubstrings(self, s: str) -> int:
        ans, n = 0, len(s)
        for k in range(n * 2 - 1):
            i, j = k // 2, (k + 1) // 2
            while ~i and j < n and s[i] == s[j]:
                ans += 1
                i, j = i - 1, j + 1
        return ans
```

```

class Solution:
    def countSubstrings(self, s: str) -> int:
        t = '^#' + '#'.join(s) + '#$'
        n = len(t)
        p = [0 for _ in range(n)]
        pos, maxRight = 0, 0
        ans = 0
        for i in range(1, n - 1):
            p[i] = min(maxRight - i, p[2 * pos - i]) if maxRight > i else 1
            while t[i - p[i]] == t[i + p[i]]:
                p[i] += 1
            if i + p[i] > maxRight:
                maxRight = i + p[i]
                pos = i
            ans += p[i] // 2
        return ans

```

• SimplyLeet

```

# Define a function to count palindromic substrings in a string
def countSubstrings(s: str) -> int:
    # Initialize the result variable to accumulate count of palindromic
    substrings
    result = 0

    # Helper function that expands around a center (left, right) and counts
    palindromes
    def expandAroundCenter(left: int, right: int) -> int:
        count = 0
        # Expand as long as indices are within bounds and characters at left
        and right are equal
        while left >= 0 and right < len(s) and s[left] == s[right]:

```

```
        # Found a palindrome, so increment count
        count += 1
        left -= 1 # Move left pointer outward
        right += 1 # Move right pointer outward
    return count

# Iterate over each character in the string
for i in range(len(s)):
    # Count odd-length palindromes (center is i)
    result += expandAroundCenter(i, i)

    # Count even-length palindromes (center is between i and i+1)
    result += expandAroundCenter(i, i + 1)

return result

# Example usage:
# result = countSubstrings("aaa")
# print(result)
```

◆ Quick Review

Key Insights

- Every individual character is a palindrome.
- Palindromes can be expanded from a center point; centers can be single characters (odd length) or between two characters (even length).
- The "expand around center" technique efficiently counts palindromes in $O(n^2)$ time.
- Dynamic programming could also be used but requires extra space and is more complex to implement.

Space & Time

Time Complexity: $O(n^2)$ where n is the length of the string.

Space Complexity: $O(1)$ if using the expand-around-center method (ignoring recursion/stack overhead).

Solution

The solution employs the expand-around-center approach:

- Iterate over each index of the string considering it as the center of a palindrome.
- For each center, expand in both directions (left and right) as long as the characters are the same and within bounds.
- Count every valid palindrome found during the expansion.
- Repeat for both odd-length centers (a single index) and even-length centers (between two indices).

This method efficiently checks all potential palindromic substrings without extra space for dynamic programming tables.

2-D Dynamic Programming

Round 8 · Low · Medium · 7 questions

2-D Dynamic Programming

□ #62 Unique Paths

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Combinatorics
Lists: Grind 169

Problem

There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., `grid[0][0]`). The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:



Input: $m = 3, n = 7$

Output: 28

Example 2:

Input: $m = 3, n = 2$

Output: 3

Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner:

1. Right -> Down -> Down
2. Down -> Down -> Right
3. Down -> Right -> Down

Constraints:

- $1 \leq m, n \leq 100$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # dp[i][j] := the number of unique paths from (0, 0) to (i, j)
        dp = [[1] * n for _ in range(m)]

        for i in range(1, m):
            for j in range(1, n):
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

        return dp[-1][-1]
```

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        dp = [1] * n

        for _ in range(1, m):
            for j in range(1, n):
                dp[j] += dp[j - 1]

        return dp[n - 1]
```

• LeetDoocs

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [[0] * n for _ in range(m)]
        f[0][0] = 1
        for i in range(m):
            for j in range(n):
                if i:
                    f[i][j] += f[i - 1][j]
                if j:
                    f[i][j] += f[i][j - 1]

        return f[-1][-1]
```

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [[1] * n for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                f[i][j] = f[i - 1][j] + f[i][j - 1]
        return f[-1][-1]
```

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [1] * n
        for _ in range(1, m):
            for j in range(1, n):
                f[j] += f[j - 1]
        return f[-1]
```

• SimplyLeet

Python solution using Dynamic Programming

```
def uniquePaths(m, n):
    # Initialize a 2D list with 1's for the first row and first column
    dp = [[1] * n for _ in range(m)]

    # Fill in the DP table
    for i in range(1, m):
        for j in range(1, n):
            # The number of ways to reach this cell is the sum of the ways from
            # the cell above and the cell on the left

            dp[i][j] = dp[i-1][j] + dp[i][j-1]
    # Return the value in the bottom-right cell
    return dp[m-1][n-1]

# Example usage:
print(uniquePaths(3, 7))
```

• LC.ca

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # avoid setting dp[][] to 1 for i==0 or j==0 as initialization
        dp = [[1] * n for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
        return dp[-1][-1]
```

◆ Quick Review

Key Insights

- The robot is constrained to only move right or down.
- Any valid path consists of exactly $(m-1)$ down moves and $(n-1)$ right moves.
- The problem can be solved using Dynamic Programming where each cell is the sum of the ways to reach the cell from the top and left.
- Alternatively, a combinatorial solution is possible by calculating the number of unique sequences of moves, equivalent to choosing $(m-1)$ moves from $(m+n-2)$ moves.

Space & Time

Time Complexity: $O(mn)$

Space Complexity: $O(mn)$ – can be optimized to $O(n)$ using a 1D DP array.

Solution

We can solve the problem using a dynamic programming approach. The idea is to create a 2D DP table where each cell (i, j) represents the number of ways to reach that cell. Since

the robot can only come from above or from the left, we update the DP table as follows: Initialize $dp[0][j] = 1$ for all j (only one way to move right in the first row). Initialize $dp[i][0] = 1$ for all i (only one way to move down in the first column). For each other cell, compute $dp[i][j] = dp[i-1][j] + dp[i][j-1]$. The final answer will be at $dp[m-1][n-1]$. The combinatorial alternative computes $C(m+n-2, m-1)$ or $C(m+n-2, n-1)$ using basic factorial arithmetic, which is efficient for the given constraints.

2-D Dynamic Programming

□ #72 Edit Distance

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given two strings word1 and word2, return the minimum number of operations required to convert word1 to word2.

You have the following three operations permitted on a word:

- Insert a character
- Delete a character
- Replace a character

Example 1:

```
Input: word1 = "horse", word2 = "ros"
Output: 3
Explanation:
horse -> rorse (replace 'h' with 'r')
rorse -> rose (remove 'r')
rose -> ros (remove 'e')
```

Example 2:

Input: word1 = "intention", word2 = "execution"

Output: 5

Explanation:

intention -> inention (remove 't')

inention -> enention (replace 'i' with 'e')

enention -> exention (replace 'n' with 'x')

exention -> exection (replace 'n' with 'c')

exection -> execution (insert 'u')

Constraints:

- $0 \leq \text{word1.length}, \text{word2.length} \leq 500$
- word1 and word2 consist of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m = len(word1)
        n = len(word2)
        # dp[i][j] := the minimum number of operations to convert word1[0..i) to
        # word2[0..j)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m + 1):
            dp[i][0] = i

        for j in range(1, n + 1):
            dp[0][j] = j

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if word1[i - 1] == word2[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1]
                else:
                    dp[i][j] = min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]) + 1
```



```
return dp[m][n]
```

• LeetCode

```
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for j in range(1, n + 1):
            f[0][j] = j
        for i, a in enumerate(word1, 1):
            f[i][0] = i
            for j, b in enumerate(word2, 1):
                if a == b:
                    f[i][j] = f[i - 1][j - 1]
                else:
                    f[i][j] = min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1]) + 1
        return f[m][n]
```

• SimplyLeet

```
# Python solution to compute the Edit Distance
def minDistance(word1, word2):
    m = len(word1)
    n = len(word2)

    # Initialize the DP table with size (m+1) x (n+1)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Base cases: transform word1[0:i] to empty word2 requires i deletions
    for i in range(m + 1):
```

```

        dp[i][0] = i
    # Base cases: transform empty word1 to word2[0:j] requires j insertions
    for j in range(n + 1):
        dp[0][j] = j

    # Fill up the dp table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            # If the characters are the same, no new operation is needed.
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                # consider insertion, deletion, and replacement
                dp[i][j] = 1 + min(dp[i][j - 1],    # Insert
                                   dp[i - 1][j],    # Delete
                                   dp[i - 1][j - 1]) # Replace

    return dp[m][n]

# Example usage:
print(minDistance("horse", "ros"))

```

• LC.ca

```

class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(m + 1):
            dp[i][0] = i
        for j in range(n + 1):
            dp[0][j] = j
        for i in range(1, m + 1):
            for j in range(1, n + 1):

```

```

        if word1[i - 1] == word2[j - 1]:
            dp[i][j] = dp[i - 1][j - 1]
        else:
            # dp[i - 1][j - 1] meaning replace.
            # e.g. "abc" and "bf", checking "ab" and "b" distance,
            # then replace either way for "c" or "f"
            dp[i][j] = 1 + min(dp[i][j - 1], dp[i - 1][j], dp[i - 1][j
- 1])
    return dp[-1][-1]

```

```
#####
```

```

class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for j in range(1, n + 1):
            f[0][j] = j
        for i, a in enumerate(word1, 1):
            f[i][0] = i # merged for loops from above solution, but not good
            for readability
                for j, b in enumerate(word2, 1):

                    if a == b:
                        f[i][j] = f[i - 1][j - 1]
                    else:
                        f[i][j] = min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1]) +
1
        return f[m][n]

```

◆ Quick Review

Key Insights

- This is a classic dynamic programming problem.
- Use a 2D DP table where $dp[i][j]$ represents the minimum edit distance between

word1[0...i-1] and word2[0...j-1].

- **Base cases:** converting an empty string to a string of length j requires j insertions, and vice versa.
- **Transition:** If characters match then $dp[i][j] = dp[i-1][j-1]$, otherwise take minimum among insertion, deletion, and replacement operations and add 1.

Space & Time

Time Complexity: $O(mn)$, where m and n are the lengths of word1 and word2 respectively.

Space Complexity: $O(mn)$ due to the DP table used for storing computed distances.

Solution

We use a dynamic programming approach by constructing a 2D table dp of size $(m+1) \times (n+1)$, where m and n are the lengths of word1 and word2. Each $dp[i][j]$ holds the minimum number of operations needed to convert the first i characters of word1 to the first j characters of word2. The algorithm includes the following steps: Initialize the base cases:

$dp[0][j] = j$ (converting an empty word1 to j characters of word2 requires j insertions)
 $dp[i][0] = i$ (converting i characters of word1 to an empty word2 requires i deletions) Iterate over each character in word1 and word2: If the current characters are equal, no additional operation is needed and $dp[i][j] = dp[i-1][j-1]$. Otherwise, choose the best operation among insertion ($dp[i][j-1]$), deletion ($dp[i-1][j]$), or substitution ($dp[i-1][j-1]$) and add 1. The answer is found in $dp[m][n]$. This method reliably computes the edit distance using a systematic DP approach, ensuring that all subproblems are solved and reused efficiently.

2-D Dynamic Programming

□ #97 Interleaving String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: NeetCode 150

Problem

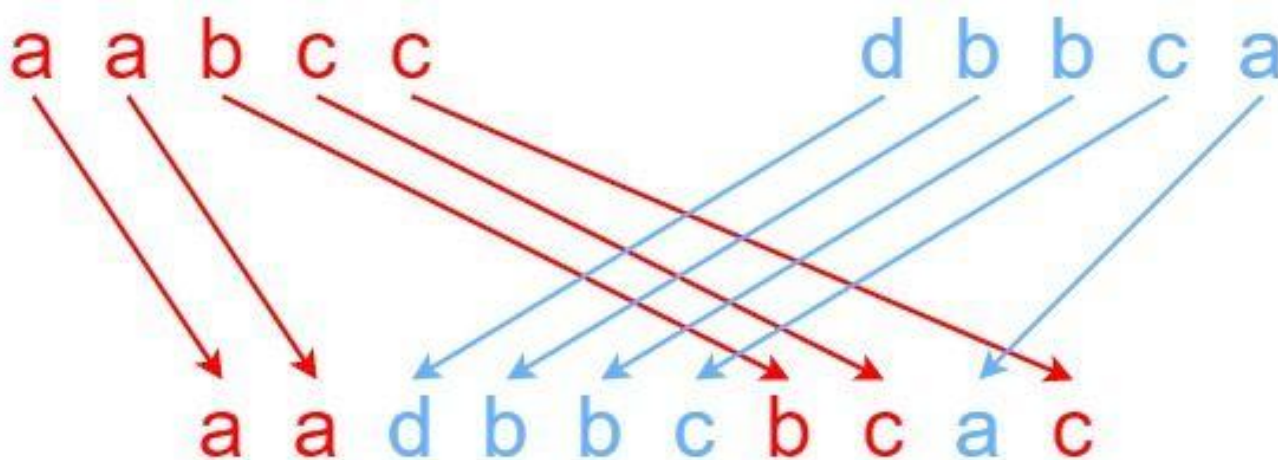
Given strings s_1 , s_2 , and s_3 , find whether s_3 is formed by an interleaving of s_1 and s_2 .

An interleaving of two strings s and t is a configuration where s and t are divided into n and m substrings respectively, such that:

- $s = s_1 + s_2 + \dots + s_n$
- $t = t_1 + t_2 + \dots + t_m$
- $|n - m| \leq 1$
- The interleaving is $s_1 + t_1 + s_2 + t_2 + s_3 + t_3 + \dots$ or $t_1 + s_1 + t_2 + s_2 + t_3 + s_3 + \dots$

Note: $a + b$ is the concatenation of strings a and b .

Example 1:



Input: s1 = "aabcc", s2 = "dbbca", s3 = "aadbcbcbac"

Output: true

Explanation: One way to obtain s3 is:

Split s1 into s1 = "aa" + "bc" + "c", and s2 into s2 = "dbbc" + "a".

Interleaving the two splits, we get "aa" + "dbbc" + "bc" + "a" + "c" = "aadbcbcbac".

Since s3 can be obtained by interleaving s1 and s2, we return true.

Example 2:

Input: s1 = "aabcc", s2 = "dbbca", s3 = "aadbcbcbacc"

Output: false

Explanation: Notice how it is impossible to interleave s2 with any other string to obtain s3.

Example 3:

Input: s1 = "", s2 = "", s3 = ""

Output: true

Constraints:

- $0 \leq s1.length, s2.length \leq 100$
- $0 \leq s3.length \leq 200$
- s1, s2, and s3 consist of lowercase English letters.

Follow up: Could you solve it using only $O(s2.length)$ additional memory space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m = len(s1)
        n = len(s2)
        if m + n != len(s3):
            return False

        # dp[i][j] := true if s3[0..i + j) is formed by the interleaving of
        # s1[0..i) and s2[0..j)
        dp = [[False] * (n + 1) for _ in range(m + 1)]

        dp[0][0] = True

        for i in range(1, m + 1):
            dp[i][0] = dp[i - 1][0] and s1[i - 1] == s3[i - 1]

        for j in range(1, n + 1):
            dp[0][j] = dp[0][j - 1] and s2[j - 1] == s3[j - 1]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                dp[i][j] = (dp[i - 1][j] and s1[i - 1] == s3[i + j - 1] or
                           dp[i][j - 1] and s2[j - 1] == s3[i + j - 1])

        return dp[m][n]
```

• LeetDoocs


```

class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        @cache
        def dfs(i: int, j: int) -> bool:
            if i >= m and j >= n:
                return True
            k = i + j
            if i < m and s1[i] == s3[k] and dfs(i + 1, j):
                return True
            if j < n and s2[j] == s3[k] and dfs(i, j + 1):
                return True
            return False

        m, n = len(s1), len(s2)
        if m + n != len(s3):
            return False
        return dfs(0, 0)

```

```

class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m, n = len(s1), len(s2)
        if m + n != len(s3):
            return False
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for i in range(m + 1):
            for j in range(n + 1):
                k = i + j - 1
                if i and s1[i - 1] == s3[k]:
                    f[i][j] = f[i - 1][j]
                if j and s2[j - 1] == s3[k]:
                    f[i][j] |= f[i][j - 1]
        return f[m][n]

```

• SimplyLeet

```
def isInterleave(s1, s2, s3):
    # Check if the combined lengths of s1 and s2 match s3
    if len(s1) + len(s2) != len(s3):
        return False
    # Initialize dp table with dimensions (len(s1)+1) x (len(s2)+1)
    dp = [[False] * (len(s2) + 1) for _ in range(len(s1) + 1)]
    dp[0][0] = True # Base case: empty strings form an empty string
    # Fill first row using characters from s2 only
    for j in range(1, len(s2) + 1):
        dp[0][j] = dp[0][j - 1] and s2[j - 1] == s3[j - 1]

    # Fill first column using characters from s1 only
    for i in range(1, len(s1) + 1):
        dp[i][0] = dp[i - 1][0] and s1[i - 1] == s3[i - 1]
    # Fill the rest of the dp table
    for i in range(1, len(s1) + 1):
        for j in range(1, len(s2) + 1):
            dp[i][j] = (dp[i - 1][j] and s1[i - 1] == s3[i + j - 1]) or \
                (dp[i][j - 1] and s2[j - 1] == s3[i + j - 1])
    return dp[len(s1)][len(s2)]

# Example usage:
print(isInterleave("aabcc", "dbbca", "aadbbcbcac")) # Expected output: True
print(isInterleave("aabcc", "dbbca", "aadbbbacc")) # Expected output: False
```

◆ Quick Review

Key Insights

- The problem asks if $s3$ can be constructed by preserving the order of characters in $s1$ and $s2$.
- A dynamic programming approach is effective by defining $dp[i][j]$ to represent whether $s3[0 \dots i+j-1]$ is an interleaving of $s1[0 \dots i-1]$ and $s2[0 \dots j-1]$.

- The solution must check that the total lengths match; otherwise, it is immediately false.
- Transitions are based on whether the current character in $s3$ matches the next char in $s1$ or $s2$.
- Space optimization to $O(s2.length)$ is possible by reusing a 1D DP array.

Space & Time

Time Complexity: $O(n * m)$ where n = length of $s1$ and m = length of $s2$.

Space Complexity: $O(n * m)$ with the basic DP solution. (Can be optimized to $O(m)$ with further space reduction.)

Solution

We use a 2D dynamic programming table where $dp[i][j]$ is true if $s3[0 \dots i+j-1]$ can be formed by interleaving $s1[0 \dots i-1]$ and $s2[0 \dots j-1]$. First, we check if the lengths of $s1$ and $s2$ add up to $s3$'s length; if not, return false. Then, we initialize $dp[0][0]$ to true and fill in the first row and column based on whether the characters in $s2$ or $s1$ correspond to $s3$. For the rest of the table, $dp[i][j]$ is true if either the

previous state $dp[i-1][j]$ is true and $s1[i-1]$ matches $s3[i+j-1]$, or $dp[i][j-1]$ is true and $s2[j-1]$ matches $s3[i+j-1]$. The answer is found in $dp[s1.length][s2.length]$.

2-D Dynamic Programming

□ #309 Best Time to Buy and Sell Stock with Cooldown

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Denny Zhang

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the i th day.

Find the maximum profit you can achieve. You may complete as many transactions as you like (i.e., buy one and sell one share of the stock multiple times) with the following restrictions:

- After you sell your stock, you cannot buy stock on the next day (i.e., cooldown one day).

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

Input: `prices = [1,2,3,0,2]`

Output: 3

Explanation: `transactions = [buy, sell, cooldown, buy, sell]`

Example 2:

Input: prices = [1]
Output: 0

Constraints:

- $1 \leq \text{prices.length} \leq 5000$
- $0 \leq \text{prices}[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        sell = 0
        hold = -math.inf
        prev = 0

        for price in prices:
            cache = sell
            sell = max(sell, hold + price)
            hold = max(hold, prev - price)

        prev = cache

        return sell
```

• LeetDoocs

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            if i >= len(prices):
                return 0
            ans = dfs(i + 1, j)
            if j:
                ans = max(ans, prices[i] + dfs(i + 2, 0))
            else:
```

```

        ans = max(ans, -prices[i] + dfs(i + 1, 1))
    return ans

    return dfs(0, 0)

```

```

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        n = len(prices)
        f = [[0] * 2 for _ in range(n)]
        f[0][1] = -prices[0]
        for i in range(1, n):
            f[i][0] = max(f[i - 1][0], f[i - 1][1] + prices[i])
            f[i][1] = max(f[i - 1][1], f[i - 2][0] - prices[i])
        return f[n - 1][0]

```

```

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        f, f0, f1 = 0, 0, -prices[0]
        for x in prices[1:]:
            f, f0, f1 = f0, max(f0, f1 + x), max(f1, f - x)
        return f0

```

• SimplyLeet

```

# Python Solution with line-by-line comments

def maxProfit(prices):
    # Handle edge case: if there are no prices, profit is 0
    if not prices:
        return 0

    # Initialize states:
    # s0: Maximum profit when ready to buy
    # s1: Maximum profit when holding a stock (bought)

```

```

# s2: Maximum profit during cooldown after a sale
s0, s1, s2 = 0, -prices[0], 0

# Iterate over prices starting from day 1
for price in prices[1:]:
    # Store previous state values
    prev_s0, prev_s1, prev_s2 = s0, s1, s2
    # s0 can come either from staying in s0 or coming from a cooldown (s2)
    s0 = max(prev_s0, prev_s2)
    # s1 is updated by either keeping the stock or buying today with profit
    # from s0

    s1 = max(prev_s1, prev_s0 - price)
    # s2 is achieved by selling the stock that we were holding (prev_s1 +
    price)
    s2 = prev_s1 + price
    # The final maximum profit is the maximum of not holding a stock (s0 or s2)
    return max(s0, s2)

# Example usage:
print(maxProfit([1,2,3,0,2]))

```

• LC.ca

```

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        f1, f2, f3 = -prices[0], 0, 0
        for price in prices[1:]:
            pf1, pf2, pf3 = f1, f2, f3
            f1 = max(pf1, pf3 - price)
            f2 = max(pf2, pf1 + price)
            f3 = max(pf3, pf2) # cooldown
        return f2

```


#####

```

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        f, f0, f1 = 0, 0, -prices[0]
        for x in prices[1:]:
            f, f0, f1 = f0, max(f0, f1 + x), max(f1, f - x)
        return f0

```

#####

```

class Solution(object):
    def maxProfit(self, prices):
        """
        :type prices: List[int]
        :rtype: int
        """
        if len(prices) < 2:
            return 0
        buy = [0] * len(prices)

```

```

        sell = [0] * len(prices)
        buy[0] = -prices[0]
        buy[1] = max(-prices[1], buy[0])
        sell[0] = 0
        sell[1] = max(prices[1] - prices[0], 0)
        for i in range(2, len(prices)):
            buy[i] = max(sell[i - 2] - prices[i], buy[i - 1])
            sell[i] = max(prices[i] + buy[i - 1], sell[i - 1])
        return max(sell)

```

◆ Quick Review

Key Insights

- Use Dynamic Programming to solve the problem.
- Maintain three states to represent:

- **s0**: the state where you are ready to buy (not holding any stock).
- **s1**: the state where you are holding a stock.
- **s2**: the cooldown state encountered right after selling a stock.
- **Transition between states:**
- Transition into **s0** can come from staying in **s0** or coming out from the cooldown state **s2**.
- To enter **s1**, you either continue holding your stock, or buy a stock using the profit available from **s0**.
- The only way to reach **s2** is by selling the stock from **s1**.
- The final answer is determined by the maximum profit when you are not holding a stock (i.e., $\max(s0, s2)$).

Space & Time

Time Complexity: $O(n)$, where n is the number of days.

Space Complexity: $O(1)$, since only a few variables are maintained for the states.

Solution

The solution involves updating three state variables for each day: s_0 (ready to buy) is updated as the maximum of the previous s_0 and the previous cooldown s_2 . s_1 (holding a stock) is updated as the maximum of the previous s_1 or buying today with the profit from s_0 minus the current price. s_2 (cooldown) is updated by selling the stock held in s_1 (i.e., previous s_1 plus the current price). At the end of the iteration, the maximum profit will be in state s_0 (ready to buy) or s_2 (cooldown), since remaining in s_1 means you are holding a stock, which does not constitute a realized profit.

2-D Dynamic Programming

□ #494 Target Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Backtracking
Lists: NeetCode 150

Problem

You are given an integer array `nums` and an integer `target`.

You want to build an expression out of `nums` by adding one of the symbols '+' and '-' before each integer in `nums` and then concatenate all the integers.

- For example, if `nums = [2, 1]`, you can add a '+' before 2 and a '-' before 1 and concatenate them to build the expression "+2-1".

Return the number of different expressions that you can build, which evaluates to `target`.

Example 1:

Input: nums = [1,1,1,1,1], target = 3

Output: 5

Explanation: There are 5 ways to assign symbols to make the sum of nums be target 3.

-1 + 1 + 1 + 1 + 1 = 3

+1 - 1 + 1 + 1 + 1 = 3

+1 + 1 - 1 + 1 + 1 = 3

+1 + 1 + 1 - 1 + 1 = 3

+1 + 1 + 1 + 1 - 1 = 3

Example 2:

Input: nums = [1], target = 1

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 20$
- $0 \leq \text{nums}[i] \leq 1000$
- $0 \leq \text{sum}(\text{nums}[i]) \leq 1000$
- $-1000 \leq \text{target} \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findTargetSumWays(self, nums: list[int], target: int) -> int:
        summ = sum(nums)
        if summ < abs(target) or (summ + target) % 2 == 1:
            return 0

        def knapsack(target: int) -> int:
            # dp[i] := the number of ways to sum to i by nums so far
            dp = [1] + [0] * summ
```

```

    for num in nums:
        for j in range(summ, num - 1, -1):
            dp[j] += dp[j - num]

    return dp[target]

return knapsack((summ + target) // 2)

```

• LeetCode

```

class Solution:
    def findTargetSumWays(self, nums: List[int], target: int) -> int:
        s = sum(nums)
        if s < target or (s - target) % 2:
            return 0
        m, n = len(nums), (s - target) // 2
        f = [[0] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 1
        for i, x in enumerate(nums, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] += f[i - 1][j - x]
        return f[m][n]

```

• SimplyLeet

```

# Define a function to compute the number of ways to reach the target sum.
def findTargetSumWays(nums, target):
    # Calculate the total sum of all numbers in the list.
    total_sum = sum(nums)
    # If the target is greater than total_sum, or (target + total_sum) is odd,
    no solution exists.
    if target > total_sum or (target + total_sum) % 2 == 1:
        return 0

    # Compute the required subset sum.
    subset_sum = (target + total_sum) // 2

```

```
# Initialize the dp array with zeros, with dp[0] set to 1.
dp = [0] * (subset_sum + 1)
dp[0] = 1

# Update the dp array for each number in nums.
for num in nums:
    # Iterate backwards to ensure each number is only counted once.
    for s in range(subset_sum, num - 1, -1):
        dp[s] += dp[s - num]
return dp[subset_sum]
```

```
# Example usage:
print(findTargetSumWays([1,1,1,1,1], 3)) # Expected output: 5
```

◆ Quick Review

Key Insights

- The problem can be converted into a subset sum problem by considering two groups: numbers with a '+' sign and numbers with a '-' sign.
- Let the sum of numbers with a '+' sign be $\text{sum}(P)$ and with a '-' sign be $\text{sum}(N)$. Then the problem reduces to solving:
 - $\text{sum}(P) - \text{sum}(N) = \text{target}$
 - and $\text{sum}(P) + \text{sum}(N) = \text{totalSum}$, where totalSum is the sum of all numbers in nums .
- From these equations, we derive $\text{sum}(P) = (\text{target} + \text{totalSum}) / 2$.

- We only proceed if $(\text{target} + \text{totalSum})$ is even and not negative; otherwise, there are no valid expressions.
- Dynamic programming is used to count the number of subsets that sum to $\text{sum}(P)$.

Space & Time

Time Complexity: $O(n * \text{subsetSum})$, where n is the length of the `nums` array and $\text{subsetSum} = (\text{target} + \text{totalSum})/2$.

Space Complexity: $O(\text{subsetSum})$, as we use a one-dimensional DP array.

Solution

The solution converts the problem into a subset sum DP problem. First, compute the total sum of the array. If $(\text{target} + \text{totalSum})$ is not even or target is greater than totalSum , return 0 since no valid configuration exists. Otherwise, compute the required subset sum (subsetSum) as $(\text{target} + \text{totalSum}) / 2$ and use a dynamic programming array to count the number of ways to reach subsetSum . The DP approach iterates through each number in the array, updating the counts using a reverse iteration to ensure each number is only used

once.

2-D Dynamic Programming

□ #518 Coin Change II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: NeetCode 150

Problem

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return the number of combinations that make up that amount. If that amount of money cannot be made up by any combination of the coins, return 0.

You may assume that you have an infinite number of each kind of coin.

The answer is guaranteed to fit into a signed 32-bit integer.

Example 1:

Input: amount = 5, coins = [1,2,5]
Output: 4
Explanation: there are four ways to make up the amount:
5=5
5=2+2+1
5=2+1+1+1
5=1+1+1+1+1

Example 2:

Input: amount = 3, coins = [2]
Output: 0
Explanation: the amount of 3 cannot be made up just with coins of 2.

Example 3:

Input: amount = 10, coins = [10]
Output: 1

Constraints:

- $1 \leq \text{coins.length} \leq 300$
- $1 \leq \text{coins}[i] \leq 5000$
- All the values of coins are unique.
- $0 \leq \text{amount} \leq 5000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def change(self, amount: int, coins: list[int]) -> int:
        dp = [1] + [0] * amount

        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] += dp[i - coin]

        return dp[amount]
```

• LeetCode

```
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        m, n = len(coins), amount
        f = [[0] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 1
        for i, x in enumerate(coins, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] += f[i][j - x]

        return f[m][n]
```

```
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        n = amount
        f = [1] + [0] * n
        for x in coins:
            for j in range(x, n + 1):
                f[j] += f[j - x]
        return f[n]
```

• SimplyLeet

```
# Python solution for Coin Change II
def change(amount, coins):
    # Initialize dp array where dp[j] represents the number of ways to form
    # amount j
    dp = [0] * (amount + 1)
    dp[0] = 1 # One way to achieve amount 0 (use no coins)

    # Process each coin
    for coin in coins:
        # Update dp for all amounts from coin to target amount
        for j in range(coin, amount + 1):
            dp[j] += dp[j - coin] # Accumulate ways by using coin
    return dp[amount]

# Example usage:
print(change(5, [1, 2, 5])) # Output: 4
```

◆ Quick Review

Key Insights

- Use dynamic programming to build up the number of ways to form every amount up to the target.
- A one-dimensional dp array suffices, where $dp[j]$ represents the number of ways to form sum j .
- When processing each coin, iterate over amounts from the coin value to the target, updating dp.
- Order of iteration ensures that combinations, not permutations, are counted.

Space & Time

Time Complexity: $O(n * \text{amount})$, where n is the number of coins.

Space Complexity: $O(\text{amount})$, due to the dp array used for computations.

Solution

We solve the problem by using dynamic programming with a one-dimensional dp array. The dp array is sized as $\text{amount} + 1$

with $dp[0]$ initialized to 1, representing the single way to achieve the amount of 0 (by choosing no coins). For each coin in the coins array, we update every $dp[j]$ for j from the coin's value up to the target amount: $dp[j]$ is increased by $dp[j - \text{coin}]$. This ensures we consider each coin only once per combination, hence avoiding duplicate counts of the same combination in different orders.

2-D Dynamic Programming

□ #1143 Longest Common Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given two strings `text1` and `text2`, return the length of their longest common subsequence. If there is no common subsequence, return 0.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

A common subsequence of two strings is a subsequence that is common to both strings.

Example 1:

Input: `text1 = "abcde"`, `text2 = "ace"`

Output: 3

Explanation: The longest common subsequence is "ace" and its length is 3.

Example 2:

Input: text1 = "abc", text2 = "abc"

Output: 3

Explanation: The longest common subsequence is "abc" and its length is 3.

Example 3:

Input: text1 = "abc", text2 = "def"

Output: 0

Explanation: There is no such common subsequence, so the result is 0.

Constraints:

- $1 \leq \text{text1.length}, \text{text2.length} \leq 1000$
- text1 and text2 consist of only lowercase English characters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m = len(text1)
        n = len(text2)
        # dp[i][j] := the length of LCS(text1[0..i), text2[0..j))
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m):
            for j in range(n):
                dp[i + 1][j + 1] = (1 + dp[i][j] if text1[i] == text2[j]
                                   else max(dp[i][j + 1], dp[i + 1][j]))

        return dp[m][n]
```

• LeetDoocs


```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i - 1] == text2[j - 1]:
                    f[i][j] = f[i - 1][j - 1] + 1
                else:
                    f[i][j] = max(f[i - 1][j], f[i][j - 1])

        return f[m][n]
```

• SimplyLeet

```
# Function to find the longest common subsequence length
def longestCommonSubsequence(text1: str, text2: str) -> int:
    m, n = len(text1), len(text2)
    # Initialize a (m+1) x (n+1) DP table filled with zeros
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Build the DP table bottom-up
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            # If characters match, extend the subsequence length by 1

            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                # Otherwise, choose the maximum from the left or top cell
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    return dp[m][n]

# Example usage
if __name__ == "__main__":

    text1 = "abcde"
    text2 = "ace"
    print(longestCommonSubsequence(text1, text2)) # Output: 3
```

• LC.ca

```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i - 1] == text2[j - 1]:
                    f[i][j] = f[i - 1][j - 1] + 1
                else:
                    f[i][j] = max(f[i - 1][j], f[i][j - 1])

        return f[m][n]
```

◆ Quick Review

Key Insights

- The problem is well-suited for a dynamic programming approach.
- Use a 2D table where $dp[i][j]$ stores the length of the longest common subsequence for $text1[0...i-1]$ and $text2[0...j-1]$.
- If the characters match at the current position, extend the subsequence by 1 from $dp[i-1][j-1]$; otherwise, take the maximum from $dp[i-1][j]$ or $dp[i][j-1]$.
- Filling the table in a bottom-up manner ensures that all subproblems are solved optimally.

Space & Time

Time Complexity: $O(m * n)$, where m and n are the lengths of `text1` and `text2`.

Space Complexity: $O(m * n)$ for maintaining the 2D DP table. Space can be optimized to $O(\min(m, n))$ if needed.

Solution

This solution uses a dynamic programming approach with a 2D array (`dp`) to record the lengths of common subsequences at each substring pair. We initialize a table of dimensions $(m+1) \times (n+1)$ with zeros, where m and n are the lengths of `text1` and `text2`. We iterate over both strings, comparing characters; if they match, we set $dp[i][j] = dp[i-1][j-1] + 1$, otherwise we choose the maximum from $dp[i-1][j]$ and $dp[i][j-1]$. The final answer is found at $dp[m][n]$.

Bit Manipulation

Round 8 · Low · Medium · 2 questions

Bit Manipulation

□ #7 Reverse Integer

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math

Lists: Grind 169

Problem

Given a signed 32-bit integer x , return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

Input: $x = 123$
Output: 321

Example 2:

Input: $x = -123$
Output: -321

Example 3:

Input: $x = 120$
Output: 21

Constraints:

- $-231 \leq x \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverse(self, x: int) -> int:
        ans = 0
        sign = -1 if x < 0 else 1
        x *= sign

        while x:
            ans = ans * 10 + x % 10
            x //= 10

        return 0 if ans < -2**31 or ans > 2**31 - 1 else sign * ans
```

• LeetDoocs

```
class Solution:
    def reverse(self, x: int) -> int:
        ans = 0
        mi, mx = -(2**31), 2**31 - 1
        while x:
            if ans < mi // 10 + 1 or ans > mx // 10:
                return 0
            y = x % 10
            if x < 0 and y > 0:
                y -= 10

            ans = ans * 10 + y
            x = (x - y) // 10
        return ans
```

• SimplyLeet

```

# Function to reverse an integer
def reverse(x: int) -> int:
    # Define the maximum and minimum limits for a 32-bit signed integer
    INT_MAX = 2**31 - 1
    INT_MIN = -2**31

    reversed_num = 0 # This will store the reversed number

    # Process each digit until x becomes 0
    while x != 0:

        # Pop the last digit from x
        # Use modulo 10 to get the last digit
        # For negative numbers, modulo may yield negative remainder
        pop = x % 10 if x > 0 else x % -10
        # Remove the last digit from x by integer division
        x = x // 10

        # Check if appending the digit would cause overflow
        if reversed_num > INT_MAX // 10 or (reversed_num == INT_MAX // 10 and
pop > 7):
            return 0

        if reversed_num < INT_MIN // 10 or (reversed_num == INT_MIN // 10 and
pop < -8):
            return 0

        # Append the digit to the reversed number
        reversed_num = reversed_num * 10 + pop

    return reversed_num

# Example usage:
print(reverse(123)) # Output: 321

print(reverse(-123)) # Output: -321
print(reverse(120)) # Output: 21

```

• LC.ca

```
class Solution:
    def reverse(self, x: int) -> int:
        ans = 0
        mi, mx = -(2**31), 2**31 - 1
        while x:
            if ans < mi // 10 + 1 or ans > mx // 10:
                return 0
            y = x % 10
            if x < 0 and y > 0:
                y -= 10
            ans = ans * 10 + y
            x = (x - y) // 10
        return ans
```

◆ Quick Review

Key Insights

- The problem requires reversing the digits while preserving the sign.
- Watch out for integer overflow due to the reversed number exceeding the 32-bit signed integer limits.
- The approach involves repeatedly extracting the last digit from the number and appending it to a new reversed integer.
- Overflow checks are done during each iteration before actually appending the digit.

Space & Time

Time Complexity: $O(n)$, where n is the number of digits in the integer.

Space Complexity: $O(1)$, only constant extra space is used.

Solution

The solution uses a while loop to extract the last digit from the original number using modulo operation ($x \% 10$) and then removes this digit using integer division ($x /= 10$). A reversed number is constructed by multiplying the current reversed number by 10 and adding the extracted digit. Before appending each digit, the algorithm checks if the new reversed number will overflow a 32-bit signed integer by comparing it with the thresholds derived from `INT_MAX` and `INT_MIN`. The algorithm handles negative values by working with x directly since the modulo and division operations work properly with negatives in most languages.

Bit Manipulation

□ #371 Sum of Two Integers

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Bit Manipulation
Lists: NeetCode 150

Problem

Given two integers a and b , return the sum of the two integers without using the operators $+$ and $-$.

Example 1:

Input: $a = 1, b = 2$
Output: 3

Example 2:

Input: $a = 2, b = 3$
Output: 5

Constraints:

- $-1000 \leq a, b \leq 1000$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def getSum(self, a: int, b: int) -> int:
        mask = 0xFFFFFFFF
        MAX = 2000

        while b != 0:
            a, b = (a ^ b) & mask, ((a & b) << 1) & mask

        return a if a < MAX else ~(a ^ mask)
```

• LeetDoocs

```
class Solution:
    def getSum(self, a: int, b: int) -> int:
        a, b = a & 0xFFFFFFFF, b & 0xFFFFFFFF
        while b:
            carry = ((a & b) << 1) & 0xFFFFFFFF
            a, b = a ^ b, carry
        return a if a < 0x80000000 else ~(a ^ 0xFFFFFFFF)
```

• SimplyLeet

```
# Python implementation to add two integers without using + or -
def getSum(a, b):
    # Continue till there is no carry
    while b != 0:
        # XOR gets the sum excluding carries
        sum_without_carry = a ^ b
        # AND finds carries, left shift to apply them to the next bit
        carry = (a & b) << 1
        # Update a with sum without carry and b with carry
        a = sum_without_carry
        b = carry
    return a

# Example usage:
result = getSum(1, 2)
print("Sum:", result) # Output: Sum: 3
```

◆ Quick Review

Key Insights

- Use bitwise XOR (^) to perform addition without carrying.
- Use bitwise AND (&) followed by a left shift (<<) to calculate the carry.
- Repeat the process until there is no carry.
- This method simulates the addition process using binary arithmetic.

Space & Time

Time Complexity: $O(1)$ – The loop runs a fixed number of times based on the number of bits.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

The solution uses bit manipulation to add two numbers without using the + or – operators.

- Use XOR (^) on a and b to add the bits where there is no carry.
- Use AND (&) to find the positions where both a and b have a 1 (i.e., potential carry). Then shift the result left by one to add in the next higher bit.
- Set a to the XOR result and b to the shifted carry.
- Repeat steps 1–3 until there is no carry (b equals 0).
-

Return a which holds the final sum. This approach leverages the bitwise operations to effectively simulate binary addition.

Math & Geometry

Round 8 · Low · Medium · 6 questions

Math & Geometry

□ #43 Multiply Strings

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Simulation

Lists: NeetCode 150

Problem

Given two non-negative integers `num1` and `num2` represented as strings, return the product of `num1` and `num2`, also represented as a string.

Note: You must not use any built-in BigInteger library or convert the inputs to integer directly.

Example 1:

```
Input: num1 = "2", num2 = "3"
Output: "6"
```

Example 2:

```
Input: num1 = "123", num2 = "456"
Output: "56088"
```

Constraints:

- $1 \leq \text{num1.length}, \text{num2.length} \leq 200$
- `num1` and `num2` consist of digits only.
- Both `num1` and `num2` do not contain any leading zero, except the number 0 itself.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        s = [0] * (len(num1) + len(num2))

        for i in reversed(range(len(num1))):
            for j in reversed(range(len(num2))):
                mult = int(num1[i]) * int(num2[j])
                summ = mult + s[i + j + 1]
                s[i + j] += summ // 10
                s[i + j + 1] = summ % 10

        for i, c in enumerate(s):
            if c != 0:
                break

        return ''.join(map(str, s[i:]))
```

• LeetDoocs

```
class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        if num1 == "0" or num2 == "0":
            return "0"
        m, n = len(num1), len(num2)
        arr = [0] * (m + n)
        for i in range(m - 1, -1, -1):
            a = int(num1[i])
            for j in range(n - 1, -1, -1):
                b = int(num2[j])

                arr[i + j + 1] += a * b
            for i in range(m + n - 1, 0, -1):
                arr[i - 1] += arr[i] // 10
                arr[i] %= 10
            i = 0 if arr[0] else 1
        return "".join(str(x) for x in arr[i:])
```


• SimplyLeet

```
# Python implementation with line-by-line comments

def multiply(num1, num2):
    # If either string is "0", the product is "0".
    if num1 == "0" or num2 == "0":
        return "0"

    n, m = len(num1), len(num2)
    # Initialize an array to store the product results.
    result = [0] * (n + m)

    # Multiply each digit of num1 with each digit of num2.
    for i in range(n - 1, -1, -1):
        for j in range(m - 1, -1, -1):
            # Multiply current digits.
            product = int(num1[i]) * int(num2[j])
            # Position in the result array where the product should be added.
            pos = i + j + 1
            total = product + result[pos]
            result[pos] = total % 10 # Place the digit at the current index.

            result[pos - 1] += total // 10 # Carry over the remaining value.

    # Convert the result array into a string, skipping any leading zero.
    result_str = ''.join(map(str, result)).lstrip('0')
    return result_str if result_str else "0"

# Example usage:
print(multiply("123", "456")) # Expected output: "56088"
```

◆ Quick Review

Key Insights

- Simulate the multiplication process similar to hand multiplication.

- Use an array to store intermediate results where the index correlation is based on the position of digits.
- Each multiplication may contribute a value that needs to be managed with a carry over in the result array.
- Handle the special case when either input is "0".

Space & Time

Time Complexity: $O(n * m)$, where n and m are the lengths of `num1` and `num2` respectively.

Space Complexity: $O(n + m)$, used for storing the intermediate multiplication results.

Solution

The solution multiplies each digit of `num1` with each digit of `num2` by simulating the grade-school multiplication algorithm. An array is used to store the intermediate sums at positions that correspond to the sum of the indices from `num1` and `num2`. For two digits multiplied together, the product is added to the corresponding index in the result array, and any carry is added to the adjacent left cell. Finally, the array is converted back to a string

while taking care to remove any leading zeros.

Data structures used:

- Array for intermediate multiplication results.**
- Two nested loops to multiply each digit pair.**

Algorithmic steps:

- Initialize an array of size (length of num1 + length of num2) with all zeros.**
- Loop through digits of num1 and num2 in reverse order.**
- Multiply the two digits, add the product to the corresponding position in the result array (taking into account any previous values).**
- Manage the carry for each index.**
- Convert the result array to a string, skipping any leading zeros.**
- Return the final product string.**

Math & Geometry

□ #48 Rotate Image

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Matrix

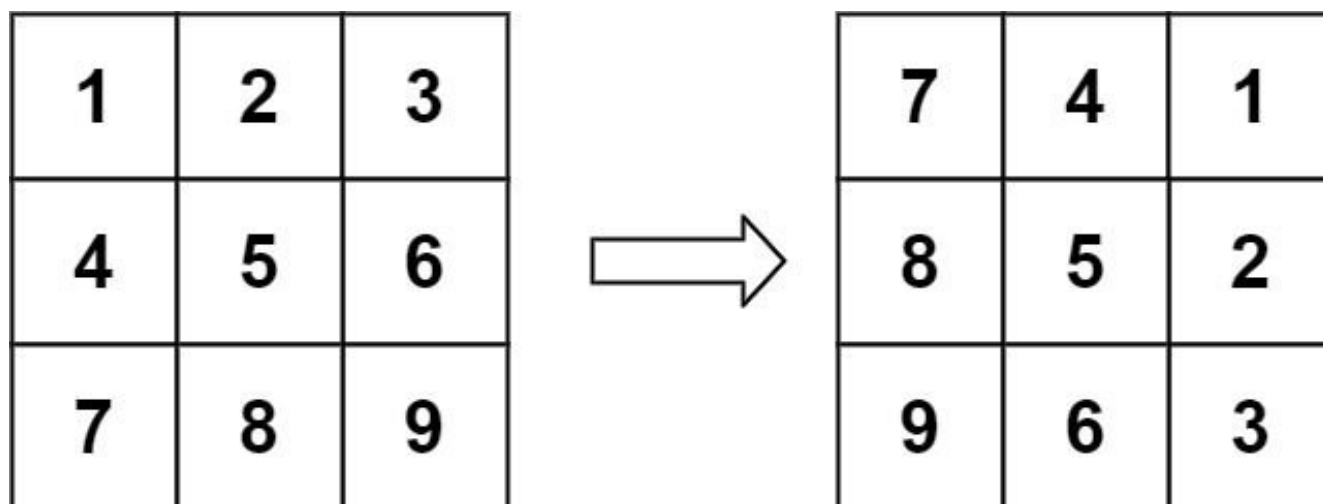
Lists: Grind 169 | CodeSignal

Problem

You are given an $n \times n$ 2D matrix representing an image, rotate the image by 90 degrees (clockwise).

You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.

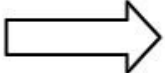
Example 1:



Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
 Output: [[7,4,1],[8,5,2],[9,6,3]]

Example 2:

5	1	9	11
2	4	8	10
13	3	6	7
15	14	12	16



15	13	2	5
14	3	4	1
12	6	8	9
16	7	10	11

Input: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
 Output: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $1 \leq n \leq 20$
- $-1000 \leq \text{matrix}[i][j] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rotate(self, matrix: list[list[int]]) -> None:
        matrix.reverse()
        for i, j in itertools.combinations(range(len(matrix)), 2):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

```
class Solution:
    def rotate(self, matrix: list[list[int]]) -> None:
        for mn in range(len(matrix) // 2):
            mx = len(matrix) - mn - 1
            for i in range(mn, mx):
                offset = i - mn
                top = matrix[mn][i]
                matrix[mn][i] = matrix[mx - offset][mn]
                matrix[mx - offset][mn] = matrix[mx][mx - offset]
                matrix[mx][mx - offset] = matrix[i][mx]

            matrix[i][mx] = top
```

• LeetDoocs

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for i in range(n >> 1):
            for j in range(n):
                matrix[i][j], matrix[n - i - 1][j] = matrix[n - i - 1][j], matrix[i][j]
            for i in range(n):
                for j in range(i):
                    matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

• SimplyLeet

```
def rotate(matrix):
    n = len(matrix)
    # Transpose the matrix by swapping elements across the diagonal
    for i in range(n):
        for j in range(i, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # Reverse each row to complete the clockwise rotation
    for i in range(n):
        matrix[i].reverse()
```

```
# Example usage:
matrix = [[1,2,3],[4,5,6],[7,8,9]]
rotate(matrix)
print(matrix) # Outputs: [[7,4,1],[8,5,2],[9,6,3]]
```

• LC.ca

```
class Solution(object):
    def rotate(self, matrix):
        """
        :type matrix: List[List[int]]
        :rtype: void Do not return anything, modify matrix in-place instead.
        """
        if len(matrix) == 0:
            return
        h = len(matrix)
        w = len(matrix[0])

        for i in range(0, h): # mirror
            for j in range(0, w / 2):
                matrix[i][j], matrix[i][w - j - 1] = matrix[i][w - j - 1], matrix[i][j]

        for i in range(0, h): # transpos
            for j in range(0, w - 1 - i):
                matrix[i][j], matrix[w - 1 - j][h - 1 - i] = matrix[w - 1 - j][h - 1 - i], matrix[i][j]

        #####

class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for i in range(n >> 1):
            for j in range(n):
                matrix[i][j], matrix[n - i - 1][j] = matrix[n - i - 1][j], matrix[i][j]
            for i in range(n):
                for j in range(i):
                    matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

◆ Quick Review

Key Insights

- The rotation can be achieved in two steps: first transpose the matrix and then reverse each row.
- Transposing the matrix swaps $\text{matrix}[i][j]$ with $\text{matrix}[j][i]$, converting rows into columns.
- Reversing each row post-transposition yields the desired clockwise rotation.
- This approach works in-place with $O(1)$ extra space.

Space & Time

Time Complexity: $O(n^2)$ since every element is processed during the transpose and row reversal.

Space Complexity: $O(1)$ as the operations are performed in-place without extra data structures.

Solution

The solution leverages a two-step in-place transformation: Transpose the matrix: Iterate through the matrix and swap each element (i, j) with element (j, i) for $j \geq i$. Reverse each

row: After the transpose, each row is reversed to rotate the matrix 90 degrees clockwise. The key trick is to realize that a transpose followed by a reverse of each row achieves the rotation without needing extra space.

Math & Geometry

□ #50 Pow(x, n)

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Recursion

Lists: Grind 169

Problem

Implement $\text{pow}(x, n)$, which calculates x raised to the power n (i.e., x^n).

Example 1:

Input: $x = 2.00000$, $n = 10$
Output: 1024.00000

Example 2:

Input: $x = 2.10000$, $n = 3$
Output: 9.26100

Example 3:

Input: $x = 2.00000$, $n = -2$
Output: 0.25000
Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

Constraints:

- $-100.0 < x < 100.0$
- $-2^{31} \leq n \leq 2^{31} - 1$
- n is an integer.

- Either x is not zero or $n > 0$.
- $-104 \leq x^n \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def myPow(self, x: float, n: int) -> float:
        if n == 0:
            return 1
        if n < 0:
            return 1 / self.myPow(x, -n)
        if n % 2 == 1:
            return x * self.myPow(x, n - 1)
        return self.myPow(x * x, n // 2)
```

• LeetDoocs

```
class Solution:
    def myPow(self, x: float, n: int) -> float:
        def qpow(a: float, n: int) -> float:
            ans = 1
            while n:
                if n & 1:
                    ans *= a
                a *= a
                n >>= 1
            return ans

        return qpow(x, n) if n >= 0 else 1 / qpow(x, -n)
```

• SimplyLeet

```
# Python implementation of pow(x, n) using exponentiation by squaring
def myPow(x, n):
    # if exponent is negative, invert x and make n positive
    if n < 0:
        x = 1 / x
        n = -n

    result = 1.0 # initialize result
    while n:
        # if the current exponent is odd, multiply result by the base

        if n % 2:
            result *= x
        x *= x # square the base for the next iteration
        n //= 2 # divide the exponent by 2
    return result

# Example usage:
print(myPow(2.0, 10)) # Output: 1024.0
print(myPow(2.1, 3))  # Output: 9.261000000000001
print(myPow(2.0, -2)) # Output: 0.25
```

• LC.ca

```
class Solution:
    def myPow(self, x: float, n: int) -> float:
        def qpow(a: float, n: int) -> float:
            ans = 1
            while n:
                if n & 1:
                    ans *= a
                a *= a
                n >>= 1
            return ans

        return qpow(x, n) if n >= 0 else 1 / qpow(x, -n)
```

◆ Quick Review

Key Insights

- Use exponentiation by squaring to reduce the number of multiplications.
- For a negative exponent, compute the power for $|n|$ and then take the reciprocal.
- Handle the edge case when n is 0 (return 1).
- The algorithm works in $O(\log n)$ time complexity.

Space & Time

Time Complexity: $O(\log |n|)$

Space Complexity: $O(1)$ for the iterative solution, or $O(\log |n|)$ if implemented recursively

Solution

We use exponentiation by squaring, an algorithm that reduces the number of multiplications by repeatedly squaring the base and halving the exponent. Here's how it works: If n is negative, switch x to $1/x$ and n to $-n$. Initialize the result as 1. Loop while n is greater than 0: If the current n is odd, multiply the result by x . Square the base x . Divide n by 2 (using integer division). This iterative approach efficiently computes the power in logarithmic steps. The same approach can be

implemented recursively.

Math & Geometry

□ #54 Spiral Matrix

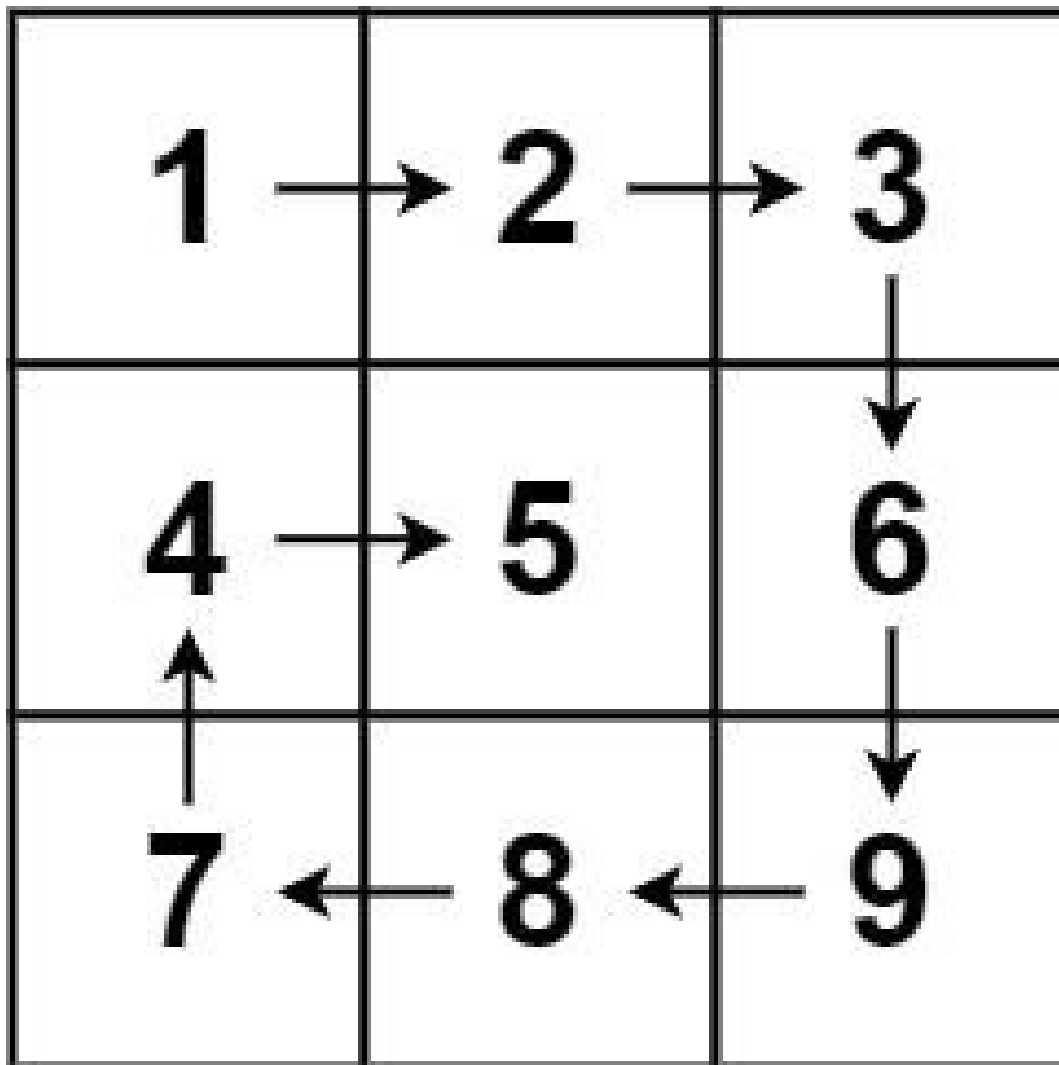
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation
Lists: Grind 169 | CodeSignal

Problem

Given an $m \times n$ matrix, return all elements of the matrix in spiral order.

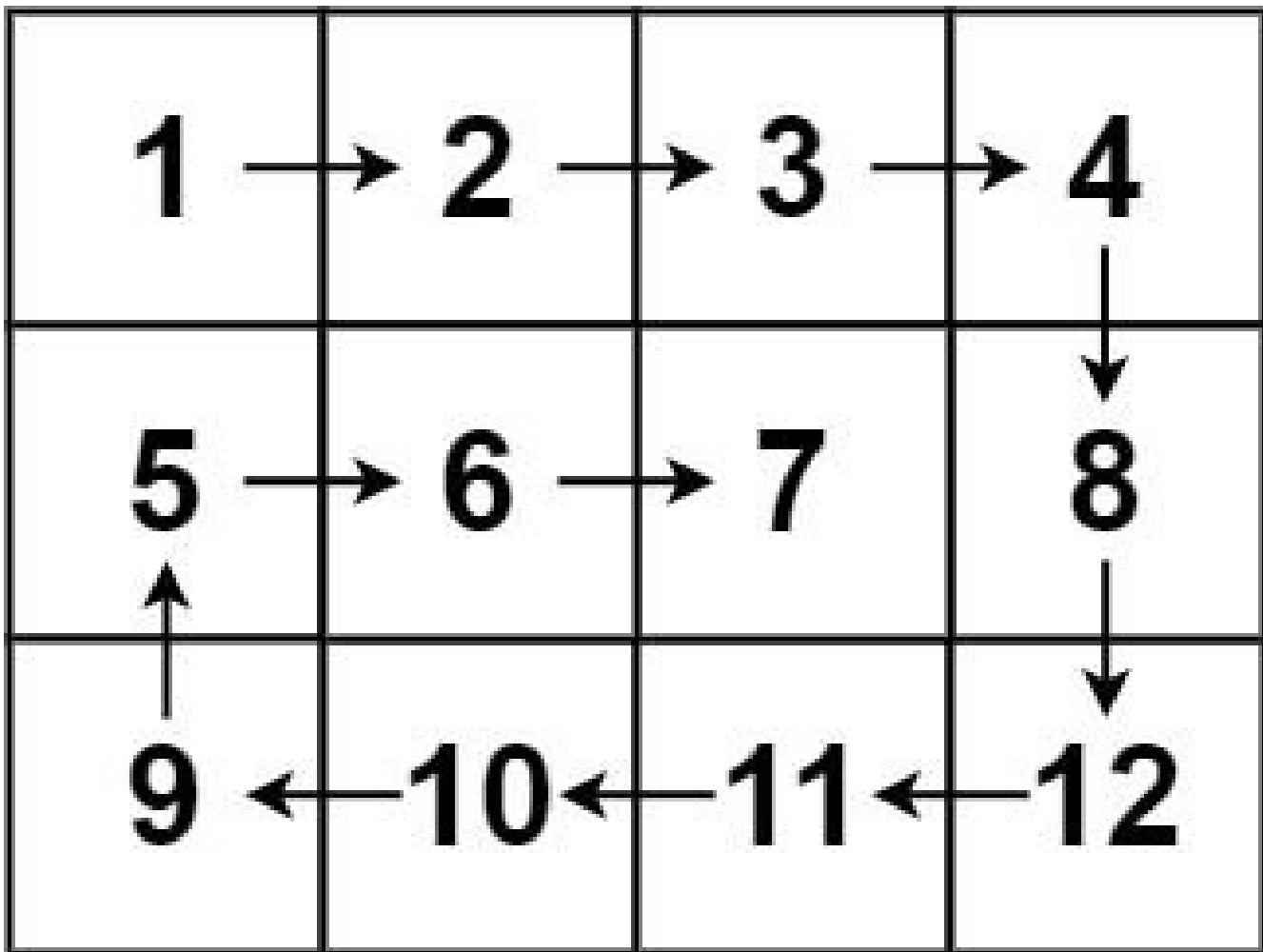
Example 1:



Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]

Output: [1,2,3,6,9,8,7,4,5]

Example 2:



Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]

Output: [1,2,3,4,8,12,11,10,9,5,6,7]

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 10$
- $-100 \leq \text{matrix}[i][j] \leq 100$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def spiralOrder(self, matrix: list[list[int]]) -> list[int]:
        if not matrix:
            return []

        m = len(matrix)
        n = len(matrix[0])
        ans = []
        r1 = 0
        c1 = 0

        r2 = m - 1
        c2 = n - 1

        # Repeatedly add matrix[r1..r2][c1..c2] to `ans`.
        while len(ans) < m * n:
            j = c1
            while j <= c2 and len(ans) < m * n:
                ans.append(matrix[r1][j])
                j += 1
            i = r1 + 1

            while i <= r2 - 1 and len(ans) < m * n:
                ans.append(matrix[i][c2])
                i += 1
            j = c2
            while j >= c1 and len(ans) < m * n:
                ans.append(matrix[r2][j])
                j -= 1
            i = r2 - 1
            while i >= r1 + 1 and len(ans) < m * n:
                ans.append(matrix[i][c1])

                i -= 1
            r1 += 1
            c1 += 1
            r2 -= 1
            c2 -= 1

        return ans

```

• LeetDoocs

```

class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        m, n = len(matrix), len(matrix[0])
        dirs = (0, 1, 0, -1, 0)
        vis = [[False] * n for _ in range(m)]
        i = j = k = 0
        ans = []
        for _ in range(m * n):
            ans.append(matrix[i][j])
            vis[i][j] = True

            x, y = i + dirs[k], j + dirs[k + 1]
            if x < 0 or x >= m or y < 0 or y >= n or vis[x][y]:
                k = (k + 1) % 4
            i += dirs[k]
            j += dirs[k + 1]
        return ans

```

• SimplyLeet

```

def spiralOrder(matrix):
    # Initialize the boundaries of the matrix
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    result = [] # This will store the spiral order

    # Loop until the pointers cross each other
    while top <= bottom and left <= right:
        # Traverse from left to right on the current top row
        for col in range(left, right + 1):
            result.append(matrix[top][col])
            top += 1 # Move the top boundary down

        # Traverse downwards on the current right column
        for row in range(top, bottom + 1):
            result.append(matrix[row][right])
            right -= 1 # Move the right boundary to the left

        # Traverse from right to left on the current bottom row, if valid
        if top <= bottom:

```

```

        for col in range(right, left - 1, -1):
            result.append(matrix[bottom][col])
        bottom -= 1 # Move the bottom boundary up

# Traverse upwards on the current left column, if valid
if left <= right:
    for row in range(bottom, top - 1, -1):
        result.append(matrix[row][left])
    left += 1 # Move the left boundary right

```

```

return result

```

```

# Example usage:
# print(spiralOrder([[1,2,3],[4,5,6],[7,8,9]]))

```

• LC.ca

```

class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        m, n = len(matrix), len(matrix[0])
        dirs = (0, 1, 0, -1, 0)
        i = j = k = 0
        ans = []
        vis = set()
        for _ in range(m * n):
            ans.append(matrix[i][j])
            vis.add((i, j))

            x, y = i + dirs[k], j + dirs[k + 1]
            if not 0 <= x < m or not 0 <= y < n or (x, y) in vis:
                k = (k + 1) % 4
            i = i + dirs[k]
            j = j + dirs[k + 1]
        return ans

```

◆ Quick Review

Key Insights

- Use four boundaries: top, bottom, left, and right to track the current layer.
- Traverse right across the top row, then down the right column, then left on the bottom row, and finally up the left column.
- After each direction, adjust the corresponding boundary to move inward.
- Continue the traversal until the boundaries cross.

Space & Time

Time Complexity: $O(m * n)$ since each element is visited exactly once.

Space Complexity: $O(1)$ extra space (excluding the space required for the output list).

Solution

The solution uses a simulation approach with boundary pointers (top, bottom, left, right). At each step, the boundaries indicate the current submatrix to traverse. You iterate as follows: Traverse from left to right along the top boundary. Traverse downward along the right boundary. If the top boundary has not passed the bottom, traverse from right to left along the bottom boundary. If the left boundary has

not passed the right, traverse upward along the left boundary. After each traversal, the boundaries are adjusted inward. This continues until all elements have been visited.

Math & Geometry

□ #73 Set Matrix Zeroes

MED

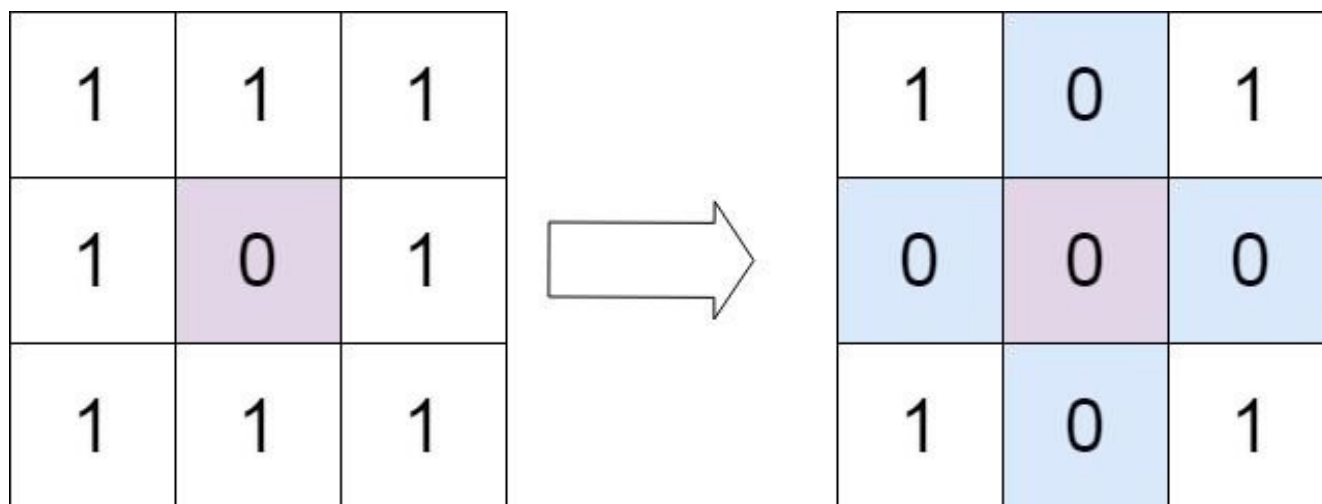
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Matrix
Lists: Grind 169

Problem

Given an $m \times n$ integer matrix `matrix`, if an element is 0, set its entire row and column to 0's.

You must do it in place.

Example 1:



Input: `matrix = [[1,1,1],[1,0,1],[1,1,1]]`

Output: `[[1,0,1],[0,0,0],[1,0,1]]`

Example 2:

0	1	2	0
3	4	5	2
1	3	1	5

→

0	0	0	0
0	4	5	0
0	3	1	0

Input: matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]

Output: [[0,0,0,0],[0,4,5,0],[0,3,1,0]]

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[0].\text{length}$
- $1 \leq m, n \leq 200$
- $-231 \leq \text{matrix}[i][j] \leq 231 - 1$

Follow up:

- A straightforward solution using $O(mn)$ space is probably a bad idea.
- A simple improvement uses $O(m + n)$ space, but still not the best solution.
- Could you devise a constant space solution?

Community Solutions (Python)

- WalkCC


```

class Solution:
    def setZeroes(self, matrix: list[list[int]]) -> None:
        m = len(matrix)
        n = len(matrix[0])
        shouldFillFirstRow = 0 in matrix[0]
        shouldFillFirstCol = 0 in list(zip(*matrix))[0]

        # Store the information in the first row and the first column.
        for i in range(1, m):
            for j in range(1, n):

                if matrix[i][j] == 0:
                    matrix[i][0] = 0
                    matrix[0][j] = 0

        # Fill 0s for the matrix except the first row and the first column.
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][0] == 0 or matrix[0][j] == 0:
                    matrix[i][j] = 0

        # Fill 0s for the first row if needed.
        if shouldFillFirstRow:
            matrix[0] = [0] * n

        # Fill 0s for the first column if needed.
        if shouldFillFirstCol:
            for row in matrix:
                row[0] = 0

```

• LeetDoocs

```

class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])
        row = [False] * m
        col = [False] * n
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == 0:
                    row[i] = col[j] = True
        for i in range(m):

```

```

    for j in range(n):
        if row[i] or col[j]:
            matrix[i][j] = 0

```

• SimplyLeet

Python solution for the Set Matrix Zeroes problem

```

def setZeroes(matrix):
    # Step 1: Determine the dimensions of matrix
    m = len(matrix)
    n = len(matrix[0])

    # Determine if first row or first column should be zeroed
    first_row_zero = any(matrix[0][j] == 0 for j in range(n))
    first_col_zero = any(matrix[i][0] == 0 for i in range(m))

    # Step 2: Use first row and column as markers
    for i in range(1, m):
        for j in range(1, n):
            if matrix[i][j] == 0:
                matrix[i][0] = 0 # mark the first element of the row
                matrix[0][j] = 0 # mark the first element of the column

    # Step 3: Zero out cells based on the markers in first row and column
    for i in range(1, m):
        for j in range(1, n):
            if matrix[i][0] == 0 or matrix[0][j] == 0:
                matrix[i][j] = 0

    # Step 4: Zero out the first row if needed
    if first_row_zero:
        for j in range(n):
            matrix[0][j] = 0

    # Step 5: Zero out the first column if needed

```

```

if first_col_zero:
    for i in range(m):
        matrix[i][0] = 0

# Example usage:
# matrix = [[1,1,1],[1,0,1],[1,1,1]]
# setZeroes(matrix)
# print(matrix)

```

• LC.ca

```

class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])
        i0 = any(v == 0 for v in matrix[0])
        j0 = any(matrix[i][0] == 0 for i in range(m))
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] == 0:
                    matrix[i][0] = matrix[0][j] = 0
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][0] == 0 or matrix[0][j] == 0:
                    matrix[i][j] = 0
        if i0:
            for j in range(n):
                matrix[0][j] = 0
        if j0:
            for i in range(m):
                matrix[i][0] = 0

```

◆ Quick Review

Key Insights

- The challenge is to mark rows and columns that need to be zeroed without using extra space.

- A common trick is to use the first row and first column as markers.
- Special care is needed for the first row and first column, as they are also used as markers.
- The algorithm involves two passes: one for marking and one for updating the matrix based on the markers.

Space & Time

Time Complexity: $O(m * n)$

Space Complexity: $O(1)$ (in-place, using only constant extra space)

Solution

We can solve this problem by using the first row and first column of the matrix as markers. First, determine if the first row or first column needs to be zeroed, then traverse the rest of the matrix and mark the corresponding first row and column positions if a zero is encountered. Finally, update the matrix backward using the markers, and finally update the first row and column if needed. The algorithm works in three main steps: Check if the first row or first column should be zeroed (store this in two variables). Iterate through

the rest of the matrix; for any element that is zero, mark its row and column by setting the corresponding element in the first row and first column to zero. Iterate through the matrix again (excluding the first row and column) and set elements to zero if their row or column marker is zero. Finally, zero out the first row and/or first column if needed. This approach avoids extra space usage apart from the two boolean flags while ensuring that the matrix is updated correctly.

Math & Geometry

□ #2013 Detect Squares

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Design · Counting · Data
Stream

Lists: NeetCode 150

Problem

You are given a stream of points on the X-Y plane. Design an algorithm that:

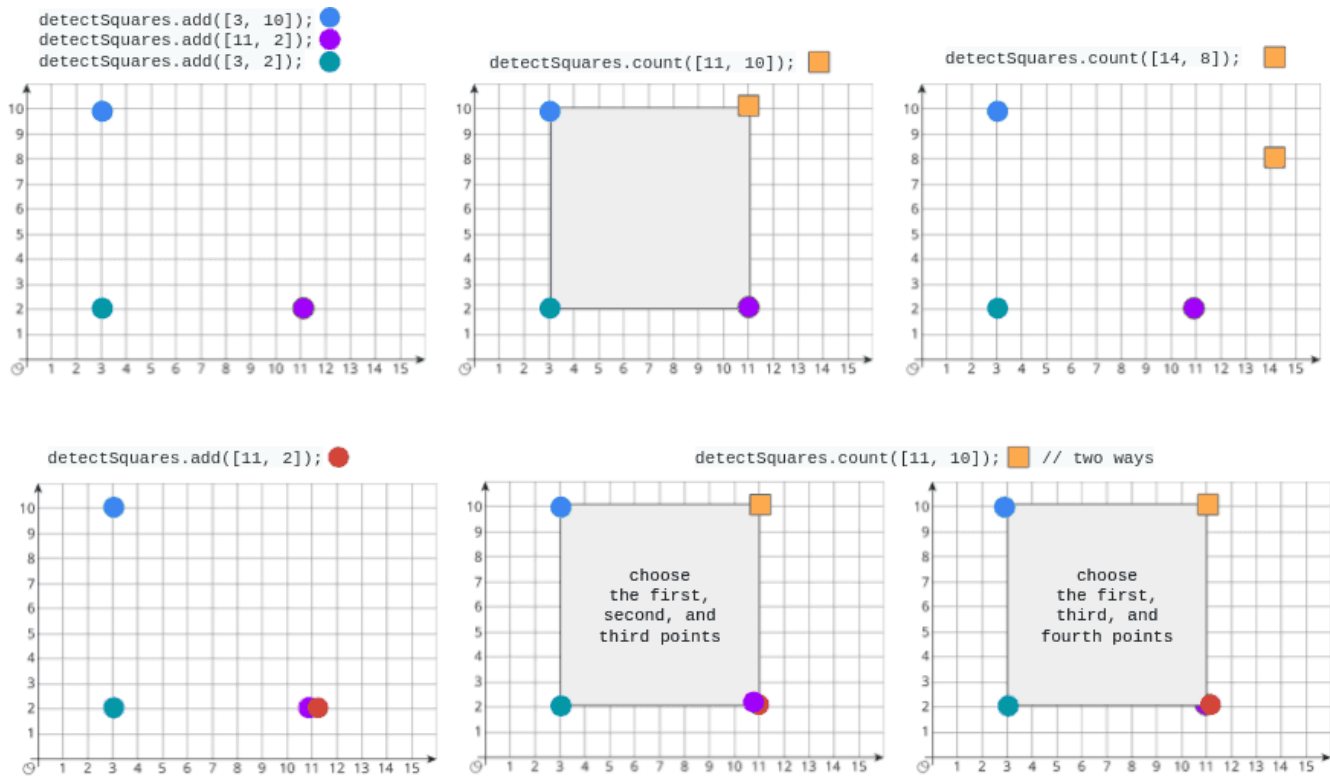
- Adds new points from the stream into a data structure. Duplicate points are allowed and should be treated as different points.
- Given a query point, counts the number of ways to choose three points from the data structure such that the three points and the query point form an axis-aligned square with positive area.

An axis-aligned square is a square whose edges are all the same length and are either parallel or perpendicular to the x-axis and y-axis.

Implement the DetectSquares class:

- **DetectSquares()** Initializes the object with an empty data structure.
- **void add(int[] point)** Adds a new point $\text{point} = [x, y]$ to the data structure.
- **int count(int[] point)** Counts the number of ways to form axis-aligned squares with point $\text{point} = [x, y]$ as described above.

Example 1:



Input

```
["DetectSquares", "add", "add", "add", "count", "count", "add", "count"]
[[], [[3, 10]], [[11, 2]], [[3, 2]], [[11, 10]], [[14, 8]], [[11, 2]], [[11, 10]]]
```

Output

```
[null, null, null, null, 1, 0, null, 2]
```

Explanation

```
DetectSquares detectSquares = new DetectSquares();
```

Constraints:

- `point.length == 2`
- `0 <= x, y <= 1000`
- At most 3000 calls in total will be made to add and count.

Community Solutions (Python)

• WalkCC

```
class DetectSquares:
    def __init__(self):
        self.pointCount = collections.Counter()

    def add(self, point: list[int]) -> None:
        self.pointCount[tuple(point)] += 1

    def count(self, point: list[int]) -> int:
        x1, y1 = point
        ans = 0

        for (x3, y3), c in self.pointCount.items():
            if x1 != x3 and abs(x1 - x3) == abs(y1 - y3):
                ans += c * self.pointCount[(x1, y3)] * self.pointCount[(x3, y1)]
        return ans
```

• LeetDoocs

```
class DetectSquares:
    def __init__(self):
        self.cnt = defaultdict(Counter)

    def add(self, point: List[int]) -> None:
        x, y = point
        self.cnt[x][y] += 1

    def count(self, point: List[int]) -> int:
        x1, y1 = point
```



```

        if x1 not in self.cnt:
            return 0
        ans = 0
        for x2 in self.cnt.keys():
            if x2 != x1:
                d = x2 - x1
                ans += self.cnt[x2][y1] * self.cnt[x1][y1 + d] * self.cnt[x2][
y1 + d]
                ans += self.cnt[x2][y1] * self.cnt[x1][y1 - d] * self.cnt[x2][
y1 - d]
        return ans

```

```

# Your DetectSquares object will be instantiated and called as such:
# obj = DetectSquares()
# obj.add(point)
# param_2 = obj.count(point)

```

• SimplyLeet

```

# Python implementation of DetectSquares
from collections import Counter

class DetectSquares:
    def __init__(self):
        # Use a counter to store the frequency of each point (x, y)
        self.point_counts = Counter()

    def add(self, point):
        x, y = point[0], point[1]

        # Increment the count for the point (x, y)
        self.point_counts[(x, y)] += 1

    def count(self, point):
        x, y = point[0], point[1]
        total_squares = 0
        # Iterate through all points in our data structure
        for (px, py), count in self.point_counts.items():
            # Only consider points that share the same y-coordinate with the
            query point
            if py == y and px != x:

```

```

        # Calculate the potential square's side length
        d = px - x
        # Check for a square above the query point
        total_squares += count * self.point_counts.get((x, y + d), 0)
* self.point_counts.get((px, y + d), 0)
        # Check for a square below the query point
        total_squares += count * self.point_counts.get((x, y - d), 0)
* self.point_counts.get((px, y - d), 0)
        return total_squares

# Example Usage:
# detectSquares = DetectSquares()

# detectSquares.add([3, 10])
# detectSquares.add([11, 2])
# detectSquares.add([3, 2])
# print(detectSquares.count([11, 10])) # Output: 1

```

◆ Quick Review

Key Insights

- Leverage the fact that an axis-aligned square has vertical and horizontal edges.
- For a query point (x, y) , iterate over all points that share the same y -axis. For each of these points (px, y) (with $px \neq x$), determine the potential square's side length, $d = |px - x|$.
- Check for the existence of the two other points $(x, y+d)$ and $(px, y+d)$ (or $(x, y-d)$ and $(px, y-d)$) that would complete the square.
- Use a hash map (or dictionary) to store each point along with its count since duplicate

points are allowed.

Space & Time

Time Complexity:

- The add operation is $O(1)$.
- The count operation is $O(n)$ in the worst-case scenario, where n is the number of distinct points with the same y -coordinate as the query point.

Space Complexity:

- $O(m)$, where m is the number of added points, since we store counts for each point.

Solution

We implement the `DetectSquares` class by maintaining a hash map (or dictionary) to store the frequency count for each point using its (x, y) coordinates as the key. In the `add(point)` method, we simply increment the count for the given point. For the `count(point)` method, we iterate through all points that share the same y -coordinate as the query point. For every such point (px, y) (with $px \neq x$), we calculate the potential side length of a square ($d = px - x$). Then, we check for two possible squares: one above the query point

and one below it. The count is increased by the number of times these needed points exist multiplied by the count of the current candidate point. This method efficiently counts all possible squares that can be formed with the query point.

Round 9

Low Priority · Hard

4 questions · 1 pattern

2-D Dynamic Programming #10 #115 #312

☐ **#329**

2-D Dynamic Programming

Round 9 · Low · Hard · 4 questions

2-D Dynamic Programming

#10 Regular Expression Matching

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Recursion
Lists: Denny Zhang

Problem

Given an input string *s* and a pattern *p*, implement regular expression matching with support for '.' and '*' where:

- '.' Matches any single character.
- '*' Matches zero or more of the preceding element.

Return a boolean indicating whether the matching covers the entire input string (not partial).

Example 1:

Input: *s* = "aa", *p* = "a"

Output: false

Explanation: "a" does not match the entire string "aa".

Example 2:

Input: s = "aa", p = "a*"

Output: true

Explanation: '*' means zero or more of the preceding element, 'a'. Therefore, by repeating 'a' once, it becomes "aa".

Example 3:

Input: s = "ab", p = ".*"

Output: true

Explanation: ".*" means "zero or more (*) of any character (.)".

Constraints:

- $1 \leq \text{s.length} \leq 20$
- $1 \leq \text{p.length} \leq 20$
- s contains only lowercase English letters.
- p contains only lowercase English letters, '.', and '*'.
- It is guaranteed for each appearance of the character '*', there will be a previous valid character to match.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m = len(s)
        n = len(p)
        # dp[i][j] := True if s[0..i) matches p[0..j)
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True

        def isMatch(i: int, j: int) -> bool:
            return j >= 0 and p[j] == '.' or s[i] == p[j]
```



```

for j, c in enumerate(p):
    if c == '*' and dp[0][j - 1]:
        dp[0][j + 1] = True

for i in range(m):
    for j in range(n):
        if p[j] == '*':
            # The minimum index of '*' is 1.
            noRepeat = dp[i + 1][j - 1]

            doRepeat = isMatch(i, j - 1) and dp[i][j + 1]
            dp[i + 1][j + 1] = noRepeat or doRepeat
        elif isMatch(i, j):
            dp[i + 1][j + 1] = dp[i][j]

return dp[m][n]

```

• LeetDoocs

```

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        @cache
        def dfs(i, j):
            if j >= n:
                return i == m
            if j + 1 < n and p[j + 1] == '*':
                return dfs(i, j + 2) or (
                    i < m and (s[i] == p[j] or p[j] == '.') and dfs(i + 1, j)
                )

            return i < m and (s[i] == p[j] or p[j] == '.') and dfs(i + 1, j + 1)

        m, n = len(s), len(p)
        return dfs(0, 0)

```

```

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for i in range(m + 1):
            for j in range(1, n + 1):
                if p[j - 1] == "*":
                    f[i][j] = f[i][j - 2]
                    if i > 0 and (p[j - 2] == "." or s[i - 1] == p[j - 2]):
                        f[i][j] |= f[i - 1][j]
                elif i > 0 and (p[j - 1] == "." or s[i - 1] == p[j - 1]):
                    f[i][j] = f[i - 1][j - 1]
        return f[m][n]

```

• SimplyLeet

```

# Python solution using recursion with memoization
def isMatch(s: str, p: str) -> bool:
    memo = {}

    def dp(i: int, j: int) -> bool:
        # If result is already computed, return it.
        if (i, j) in memo:
            return memo[(i, j)]

        # If we have reached the end of the pattern, s must also be at the end.

        if j == len(p):
            ans = i == len(s)
        else:
            # Check if current characters match
            first_match = i < len(s) and (p[j] == s[i] or p[j] == '.')

            # If there is a '*' coming after the current pattern character
            if j + 1 < len(p) and p[j+1] == '*':
                # Two cases:
                # 1. '*' stands for zero occurrence -> skip current pattern
                char and '*'

```

```

        # 2. first_match is true, so we try to consume one character in
        # s and remain at same pattern index
        ans = dp(i, j+2) or (first_match and dp(i+1, j))
    else:
        # No '*' means we directly check next characters in both s and
        # p.
        ans = first_match and dp(i+1, j+1)

    memo[(i, j)] = ans
    return ans

return dp(0, 0)

```

```

# Example test run
#print(isMatch("aa", "a*")) # Expected output: True

```

• LC.ca

```

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for i in range(m + 1):
            for j in range(1, n + 1):
                if p[j - 1] == "*":
                    f[i][j] = f[i][j - 2]
                    if i > 0 and (p[j - 2] == "." or s[i - 1] == p[j - 2]):
                        f[i][j] |= f[i - 1][j]
                elif i > 0 and (p[j - 1] == "." or s[i - 1] == p[j - 1]):
                    f[i][j] = f[i - 1][j - 1]
        return f[m][n]

```

◆ Quick Review

Key Insights

- Use recursion or dynamic programming to simulate matching as the '*' operator can affect the current and following characters.
- The '.' operator is a wildcard that matches any single character.
- When encountering a '*' in the pattern, consider both skipping the preceding element or consuming one character from the string if it matches.
- Using memoization helps to save intermediate results and efficiently process overlapping subproblems.

Space & Time

Time Complexity: $O(mn)$

where $m = \text{length}(s)$ and $n = \text{length}(p)$, due to exploring each subproblem combination.

Space Complexity: $O(mn)$

for storing the memoization table in the dynamic programming or recursive approach.

Solution

The solution uses a top-down recursive strategy with memoization to efficiently match the string against the pattern. For each

character position in *s* and *p*, we check if they match directly or if the pattern character is '.' to allow any match. When encountering '.' in the pattern, two possibilities are explored: either the '.' stands for zero occurrences of the preceding element, or if there is a valid match, it accounts for one occurrence and we proceed further in the string while maintaining the same pattern index. The memoization aspect prevents repeated computations of the same (i, j) state in the recursion, resulting in an improved overall runtime.

2-D Dynamic Programming

#115 Distinct Subsequences

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given two strings *s* and *t*, return the number of distinct subsequences of *s* which equals *t*.

The test cases are generated so that the answer fits on a 32-bit signed integer.

Example 1:

```
Input: s = "rabbbit", t = "rabbit"
Output: 3
Explanation:
As shown below, there are 3 ways you can generate "rabbit" from s.
rabbbit
rabbbit
rabbbit
```

Example 2:

```
Input: s = "babgbag", t = "bag"
Output: 5
Explanation:
As shown below, there are 5 ways you can generate "bag" from s.
babgbag
babgbag
babgbag
babgbag
babgbag
```

Constraints:

- $1 \leq s.length, t.length \leq 1000$
- s and t consist of English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        m = len(s)
        n = len(t)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m + 1):
            dp[i][0] = 1

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if s[i - 1] == t[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]
                else:
                    dp[i][j] = dp[i - 1][j]

        return dp[m][n]
```

• LeetDoocs

```
class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        m, n = len(s), len(t)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(m + 1):
            f[i][0] = 1
        for i, a in enumerate(s, 1):
            for j, b in enumerate(t, 1):
                f[i][j] = f[i - 1][j]
                if a == b:
```

```

        f[i][j] += f[i - 1][j - 1]
    return f[m][n]

```

```

class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        n = len(t)
        f = [1] + [0] * n
        for a in s:
            for j in range(n, 0, -1):
                if a == t[j - 1]:
                    f[j] += f[j - 1]
        return f[n]

```

• SimplyLeet

```

# Python solution for Distinct Subsequences
def numDistinct(s: str, t: str) -> int:
    # Get lengths of the strings s and t
    m, n = len(s), len(t)
    # Initialize a dp table with (m+1) rows and (n+1) columns filled with 0
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Base case: An empty string t (j = 0) is a subsequence of any prefix of s,
    # so set dp[i][0] = 1
    for i in range(m + 1):
        dp[i][0] = 1

    # Fill the dp table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            # If the current characters match, add the count of sequences by
            # either using or skipping s[i-1]
            if s[i - 1] == t[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]
            else:
                # If they do not match, skip s[i-1]
                dp[i][j] = dp[i - 1][j]

```



```
# The answer is the way to form all of t from all of s
return dp[m][n]

# Example usage:
print(numDistinct("rabbbit", "rabbit")) # Output: 3
```

• LC.ca

```
class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        n = len(t)
        f = [1] + [0] * n
        for a in s:
            for j in range(n, 0, -1):
                if a == t[j - 1]:
                    f[j] += f[j - 1]
        return f[n]
```

◆ Quick Review

Key Insights

- Use dynamic programming to build the solution by comparing characters of s and t.
- Define a 2D dp table where $dp[i][j]$ represents the number of ways to form the first j characters of t using the first i characters of s.
- The recurrence relation:
- If $s[i-1]$ equals $t[j-1]$, then include both the cases of matching this character as well as skipping it: $dp[i][j] = dp[i-1][j-1] + dp[i-1][j]$.

- If they do not match, then simply skip the current character of s : $dp[i][j] = dp[i-1][j]$.
- Initialize $dp[i][0] = 1$ for all i since an empty t can always be formed.
- The final answer is $dp[s.length][t.length]$.

Space & Time

Time Complexity: $O(n * m)$, where n is the length of s and m is the length of t .

Space Complexity: $O(n * m)$, which can be optimized to $O(m)$ with careful row reuse.

Solution

We use a dynamic programming approach with a 2D table. The table has dimensions $(len(s) + 1) \times (len(t) + 1)$. Each cell $dp[i][j]$ stores the number of distinct subsequences from the first i characters of s that match the first j characters of t . Steps: Initialize $dp[i][0] = 1$ for every i , because an empty t is a subsequence of any prefix of s . Iterate over each character of s (outer loop) and for each, iterate over t (inner loop). For every character in s , if it equals the current character in t , update $dp[i][j]$ as the sum of the ways by using or skipping this character. Otherwise, carry over

the previous count. Return `dp[len(s)][len(t)]` as the result.

2-D Dynamic Programming

#312 Burst Balloons

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Denny Zhang

Problem

You are given n balloons, indexed from 0 to $n - 1$. Each balloon is painted with a number on it represented by an array `nums`. You are asked to burst all the balloons.

If you burst the i th balloon, you will get $\text{nums}[i - 1] * \text{nums}[i] * \text{nums}[i + 1]$ coins. If $i - 1$ or $i + 1$ goes out of bounds of the array, then treat it as if there is a balloon with a 1 painted on it.

Return the maximum coins you can collect by bursting the balloons wisely.

Example 1:

```
Input: nums = [3,1,5,8]
Output: 167
Explanation:
nums = [3,1,5,8] --> [3,5,8] --> [3,8] --> [8] --> []
coins = 3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 167
```

Example 2:

Input: nums = [1,5]

Output: 10

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 300$
- $0 \leq \text{nums}[i] \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxCoins(self, nums: list[int]) -> int:
        n = len(nums)
        nums = [1] + nums + [1]

        @functools.lru_cache(None)
        def dp(i: int, j: int) -> int:
            """Returns maxCoins(nums[i..j])."""
            if i > j:
                return 0

            return max(dp(i, k - 1) +
                       dp(k + 1, j) +
                       nums[i - 1] * nums[k] * nums[j + 1]
                       for k in range(i, j + 1))

        return dp(1, n)
```

```

class Solution:
    def maxCoins(self, nums: list[int]) -> int:
        n = len(nums)
        # dp[i][j] := maxCoins(nums[i..j])
        dp = [[0] * (n + 2) for _ in range(n + 2)]

        nums = [1] + nums + [1]

        for d in range(n):
            for i in range(1, n - d + 1):
                j = i + d
                for k in range(i, j + 1):
                    dp[i][j] = max(
                        dp[i][j],
                        dp[i][k - 1] +
                        dp[k + 1][j] +
                        nums[i - 1] * nums[k] * nums[j + 1])

        return dp[1][n]

```

• LeetCode

```

class Solution:
    def maxCoins(self, nums: List[int]) -> int:
        n = len(nums)
        arr = [1] + nums + [1]
        f = [[0] * (n + 2) for _ in range(n + 2)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 2, n + 2):
                for k in range(i + 1, j):
                    f[i][j] = max(f[i][j], f[i][k] + f[k][j] + arr[i] * arr[k]
* arr[j])
        return f[0][-1]

```

• SimplyLeet

```

# Define the function to compute maximum coins
def maxCoins(nums):
    # Add virtual balloons with value 1 at both boundaries
    n = len(nums)
    extended = [1] + nums + [1]
    # Initialize dp table with 0's; dimensions: (n+2) x (n+2)
    dp = [[0] * (n + 2) for _ in range(n + 2)]

    # Calculate dp values for intervals with length 2 to n+1
    for length in range(2, n + 2): # interval length from 2 to n+1

        for left in range(0, n + 2 - length):
            right = left + length
            # Iterate through all possible last burst balloons in the interval
            (left, right)
            for k in range(left + 1, right):
                # Coins when bursting balloon k last in the interval:
                # value from bursting: extended[left] * extended[k] *
                extended[right]
                # plus coins from subintervals [left, k] and [k, right]
                coins = extended[left] * extended[k] * extended[right] + dp[left][k] + dp[k][right]
                dp[left][right] = max(dp[left][right], coins)

    # The final answer is stored in dp[0][n+1] which represents bursting all
    real balloons.
    return dp[0][n+1]

# Example test cases
print(maxCoins([3,1,5,8])) # Expected output: 167
print(maxCoins([1,5]))     # Expected output: 10

```

☐ • LC.ca

```

class Solution:
    def maxCoins(self, nums: List[int]) -> int:
        nums = [1] + nums + [1]
        n = len(nums)
        dp = [[0] * n for _ in range(n)]
        for l in range(2, n):
            for i in range(n - l):
                j = i + l
                for k in range(i + 1, j):
                    dp[i][j] = max(
                        dp[i][j], dp[i][k] + dp[k][j] + nums[i] * nums[k] * nu
ms[j]
                    )
        return dp[0][-1]

```

◆ Quick Review

Key Insights

- Transform the problem by adding virtual balloons with value 1 at both ends to simplify edge handling.
- Use interval dynamic programming where $dp[i][j]$ represents the maximum coins that can be obtained by bursting all balloons between indices i and j (exclusive).
- Realize that the order of bursting balloons matters; choose the last balloon to burst in each interval to break dependencies into independent subproblems.

- The solution involves iterating over all possible intervals and determining the optimal burst order.

Space & Time

Time Complexity: $O(n^3)$ – Three nested loops: one for the interval length, one for the starting index, and one for the final balloon choice.

Space Complexity: $O(n^2)$ – A 2D dynamic programming table is used to store the maximum coins for each interval.

Solution

The problem is solved using interval dynamic programming. First, extend the input array by adding 1 at both ends to handle edge cases seamlessly. Define a dp table where $dp[i][j]$ represents the maximum coins that can be obtained by bursting all the balloons between i and j (not including i and j , as these are the virtual boundaries). For every interval $[i, j]$, iterate through the possible choices for the last balloon k to burst in that interval. The value for bursting balloon k will be determined by the coins from bursting it last (i.e., $nums[i] * nums[k] * nums[j]$) plus the maximum coins

from the two resulting subintervals: (i, k) and (k, j) . By filling the dp table in increasing interval lengths, you build up to the solution for the overall interval that covers the entire (extended) array.

2-D Dynamic Programming

#329 Longest Increasing Path in a Matrix **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Graph · Topological Sort ·
Memoization · Matrix

Lists: Grind 169

Problem

Given an $m \times n$ integers matrix, return the length of the longest increasing path in matrix. From each cell, you can either move in four directions: left, right, up, or down. You may not move diagonally or move outside the boundary (i.e., wrap-around is not allowed).

Example 1:

9	9	4
6	6	8
2	1	1

Input: matrix = [[9,9,4],[6,6,8],[2,1,1]]

Output: 4

Explanation: The longest increasing path is [1, 2, 6, 9].

Example 2:

3 →	4 →	5 ↓
3	2	6
2	2	1

Input: matrix = [[3,4,5],[3,2,6],[2,2,1]]

Output: 4

Explanation: The longest increasing path is [3, 4, 5, 6]. Moving diagonally is not allowed.

Example 3:

Input: matrix = [[1]]

Output: 1

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 200$

- $0 \leq \text{matrix}[i][j] \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestIncreasingPath(self, matrix: list[list[int]]) -> int:
        m = len(matrix)
        n = len(matrix[0])

        @functools.lru_cache(None)
        def dfs(i: int, j: int, prev: int) -> int:
            if i < 0 or i == m or j < 0 or j == n:
                return 0
            if matrix[i][j] <= prev:
                return 0

            curr = matrix[i][j]
            return 1 + max(dfs(i + 1, j, curr),
                          dfs(i - 1, j, curr),
                          dfs(i, j + 1, curr),
                          dfs(i, j - 1, curr))

        return max(dfs(i, j, -math.inf) for i in range(m) for j in range(n))
```

• LeetDoocs

```
class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            ans = 0
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and matrix[x][y] > matrix[i][j]:
                    ans = max(ans, dfs(x, y))
            return ans + 1
```

```

m, n = len(matrix), len(matrix[0])
return max(dfs(i, j) for i in range(m) for j in range(n))

```

• SimplyLeet

Python implementation for Longest Increasing Path in a Matrix

```

class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        # Return 0 if matrix is empty
        if not matrix or not matrix[0]:
            return 0

        # Dimensions of the matrix
        m, n = len(matrix), len(matrix[0])

        # Create a memoization table (same dimensions as the matrix)
        # initialized to 0
        memo = [[0] * n for _ in range(m)]

        # Define the DFS function to explore the matrix
        def dfs(i, j):
            # If result is already computed, return it
            if memo[i][j]:
                return memo[i][j]

            # Start with a path length of 1 (the cell itself)
            max_len = 1
            # Define directions: up, down, left, right
            directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

            # Explore each direction
            for dx, dy in directions:
                ni, nj = i + dx, j + dy
                # If neighbor is within bounds and its value is greater
                if 0 <= ni < m and 0 <= nj < n and matrix[ni][nj] > matrix[i][j]:
                    length = 1 + dfs(ni, nj)

```

```

        max_len = max(max_len, length)

    # Store computed longest length in memoization table
    memo[i][j] = max_len
    return max_len

# Compute the longest increasing path starting from each cell
longest_path = 0
for i in range(m):
    for j in range(n):

        longest_path = max(longest_path, dfs(i, j))

return longest_path

```

• LC.ca

```

class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            ans = 0
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and matrix[x][y] > matrix[i][j]:
                    ans = max(ans, dfs(x, y))
            return ans + 1

        m, n = len(matrix), len(matrix[0])
        return max(dfs(i, j) for i in range(m) for j in range(n))

```

◆ Quick Review

Key Insights

- The problem can be modeled as a graph where each cell is a node connected to its neighbors if the neighbor's value is greater.

- A Depth First Search (DFS) with memoization is effective to avoid recalculations.
- Dynamic Programming (DP) is used to cache the longest path found from each cell.
- Since each cell can be a start point, compute DFS for every cell and update the global maximum.

Space & Time

Time Complexity: $O(m * n)$ – each cell is computed once due to memoization.

Space Complexity: $O(m * n)$ – space is used for the memoization cache and recursion stack.

Solution

We solve the problem using a DFS approach combined with memoization to keep track of already computed longest paths from each cell. For each cell in the matrix, a DFS is initiated to explore all four valid directions (up, down, left, right). If a neighboring cell has a larger value, the DFS recurses from that neighbor. The result for each cell is cached in a 2D memoization table, ensuring that each cell is processed only once. After processing all cells, the maximum value in the memo table

represents the longest increasing path in the entire matrix.